DEVOPS ENGINEERING MASTERY

CONFIGURATION MANAGEMENT & CLOUD PROVISIONING

Ansible Automation, Server Configuration, Infrastructure Management & Cloud Provisioning

VOLUME 09 • FIRST EDITION

```
nahid@technova:~$ ansible-playbook site.yml --check --diff
✓ ok=42  •  ✓ changed=3  •  ✓ failed=0
nahid@technova:~$ ansible all -m ping
✓ web-01 | SUCCESS  •  ✓ api-01 | SUCCESS  •  ✓ db-01 | SUCCESS
nahid@technova:~$ ansible-vault encrypt group_vars/prod/vault.yml
```

Ansible • Playbooks • Roles • Vault • Jinja2 • AWX

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120 MCQs

Configuration Management & Cloud Provisioning Engineering

DEVOPS ENGINEERING MASTERY — Volume 09

Ansible Automation, Server Configuration, Infrastructure Management ও Cloud Provisioning

First Edition — Published 9 November 2026.

Hands-on environment: Ubuntu Server 24.04 LTS, Ansible Core 2.17+, Python 3.12, with a MERN (MongoDB, Express, React, Node.js) example application called TechNova. All domains, IP addresses and credentials in examples are fictional (technova.example, 203.0.113.x, CHANGE-ME-...). Sample terminal outputs are illustrative. Cloud-provider coverage (AWS, Microsoft Azure, Google Cloud, IBM Cloud) is practical but introductory; provider details vary by region and plan, and official provider documentation is the authority — no pricing or service limits are stated in this book.

How to use this book. Read the parts in order — each part builds on the previous one. Keep two or three lab virtual machines beside you and do every hands-on exercise and every lab in the companion Lab Workbook before moving on. Do not copy a playbook blindly; read the explanation under each task first. Destructive operations are always flagged with a WARNING box and a safe alternative. Never practise on a production host: use disposable virtual machines or an isolated lab environment, and never paste a real secret into an example — use the fictional placeholders shown. Never commit a real secret or a plaintext vault password to a Git repository.

A note on honesty. Reading a book alone does not make anyone an expert. This book teaches the foundations and the practical judgment to progress toward professional engineering — but the judgment itself comes from doing the labs, breaking a test server, fixing it with a playbook, and repeating. There is no shortcut for that practice.

Conventions. Bangla is the explanatory language; technical terms, command names, APIs and configuration keys stay in English. `monospace` marks commands, file paths and code. **CONCEPT** explains an idea, **REAL WORLD** shows an industry scenario, **WARNING** marks risk, **PRO TIP** gives professional advice, and **LAB** marks hands-on work.

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Configuration changes should first be practised in an isolated lab, never directly on a production system. The author and publisher accept no liability for data loss or service disruption caused by running commands without a backup or an approval gate.

Who this book is for

A learner who knows JavaScript, React, Node.js, Express and MongoDB, understands basic Git and networking, has studied Linux server administration, CI/CD, Docker, Kubernetes and Terraform, and wants to automate real servers — from a first `ansible all -m ping` to a governed, multi-server production configuration platform with roles, Vault secrets, CI/CD and drift detection.

Learning outcomes

By the end of this book you will be able to: explain configuration management, declarative state and idempotency from first principles; explain Ansible's agentless SSH execution model; write inventories, playbooks, roles and collections; manage variables, facts and Jinja2 templates; handle errors and control execution; protect secrets with Ansible Vault; automate Linux servers, Nginx, Node.js and databases; harden servers; automate AWS, Azure, GCP and IBM Cloud; configure Docker and Kubernetes environments; test, troubleshoot and run Ansible in CI/CD and AWX; integrate Terraform provisioning with Ansible configuration; and operate an enterprise configuration management platform with governance and runbooks.

সূচিপত্র — Contents

PART I — Configuration Management Foundations

Chapter 01 — Configuration Management — Concepts, History, and Engineering Principles 7

Chapter 02 — Manual Server Administration vs Configuration Automation 16

Chapter 03 — Declarative Configuration, Idempotency, and Desired State 24

Chapter 04 — Ansible Architecture — Control Node, Managed Nodes, and Execution Model 33

Chapter 05 — Ansible Installation, SSH Connectivity, and First Automation 43

PART II — Ansible Core Engineering

Chapter 06 — Ansible Inventory — Static, Dynamic, and Host Groups 55

Chapter 07 — Ansible Ad-hoc Commands and Modules 64

Chapter 08 — Playbooks, Plays, Tasks, and Handlers 72

Chapter 09 — Variables, Facts, Registers, and Conditionals 82

Chapter 10 — Loops, Filters, Templates, and Jinja2 92

Chapter 11 — Error Handling, Tags, Blocks, and Execution Control 101

PART III — Advanced Ansible Engineering

Chapter 12 — Roles — Reusable Automation Architecture 112

Chapter 13 — Collections, Plugins, and Ansible Galaxy 126

Chapter 14 — Ansible Vault and Secrets Management 138

Chapter 15 — SSH, Privilege Escalation, and Secure Remote Execution 149

Chapter 16 — Dynamic Inventory, Cloud Integration, and API Automation 162

PART IV — Server and Cloud Configuration Automation

Chapter 17 — Linux Server Provisioning and Configuration 177

Chapter 18 — Nginx, Node.js, Database, and Application Deployment 188

Chapter 19 — Users, SSH, Firewall, Packages, and Security Hardening 200

Chapter 20 — AWS, Azure, GCP, and IBM Cloud Automation 212

Chapter 21 — Docker and Kubernetes Environment Configuration 223

PART V — Production Configuration Management

Chapter 22 — Ansible Testing, Validation, and Troubleshooting 238

Chapter 23 — Ansible CI/CD, AWX, Automation Controller, and Workflow 249

Chapter 24 — Enterprise Configuration Management, Governance, and Production Operations 261

Special Section

Terraform ও Ansible Integration — পরিপূরক দায়িত্ব 272

Companion Volumes

Lab Workbook — 24 Guided Labs • 15 Assignments • 3 Major Projects • Capstone

Assessment and Answer Guide — 120 MCQs • Short Answers • Scenarios • Case Studies • Final Exam

Quick Reference — Glossary (150+ Terms) • Ansible CLI / Inventory / Playbook / Module References • Checklists • Decision Trees

Guided labs (24), major projects (3) and the capstone are in the separate *Ansible Lab Workbook*. MCQs, assignments, examinations and the answer key are in the separate *Assessment and Answer Guide*. Glossary, Ansible CLI, inventory, playbook and module references, checklists and decision trees are in the separate *Quick Reference*.

PART I • DEVOPS ENGINEERING MASTERY

Configuration Management Foundations

কনফিগারেশন ম্যানেজমেন্টের ধারণা, ম্যানুয়াল প্রশাসন বনাম অটোমেশন, declarative কনফিগারেশন ও idempotency, Ansible আর্কিটেকচার, ইনস্টলেশন ও প্রথম অটোমেশন।

অধ্যায় 1 • PART I — Configuration Management Foundations

Chapter 1 — Configuration Management — Concepts, History, and Engineering Principles

১.১ ভূমিকা

একটি সার্ভার চালু করা সহজ। তাকে চালু রাখা, নিরাপদ রাখা, আর দশটি, একশোটি, এক হাজারটি সার্ভারের সঙ্গে একই রকম রাখা — সেটাই কঠিন। TechNova যখন শুরু করেছিল, তখন তাদের একটি মাত্র সার্ভার ছিল: `web-01`। একজন প্রকৌশলী SSH দিয়ে ঢুকে Nginx ইনস্টল করলেন, একটি কনফিগারেশন ফাইল হাতে সম্পাদনা করলেন, ফায়ারওয়ালের নিয়ম টাইপ করলেন, আর সবকিছু চলতে লাগল। ছয় মাস পরে TechNova-র সার্ভার সংখ্যা বারো। প্রতিটি সার্ভার একটু একটু করে আলাদা — একটিতে Nginx 1.24, আরেকটিতে 1.18; একটিতে একটি নিরাপত্তা-প্যাচ আছে, আরেকটিতে নেই; একজনের মনে আছে কোন সার্ভারে কী করা হয়েছিল, আরেকজনের মনে নেই। একদিন একটি সার্ভার নষ্ট হলো, আর কেউ ঠিক করে বলতে পারল না সেটিকে আবার তৈরি করতে কী কী করতে হবে। এই পরিস্থিতির নাম **Configuration Drift** — আর এর সমাধানের নাম **Configuration Management (CM)**।

Configuration Management হলো সার্ভার, নেটওয়ার্ক ডিভাইস ও অ্যাপ্লিকেশনের অবস্থা (State) — কী ইনস্টল থাকবে, কোন ফাইলে কী লেখা থাকবে, কোন সার্ভিস চলবে, কোন Port খোলা থাকবে — সেই সবকিছুকে একটি লিখিত, সংস্করণ-নিয়ন্ত্রিত (Version-Controlled) সংজ্ঞায় বেঁধে ফেলা এবং সেই সংজ্ঞা থেকে স্বয়ংক্রিয়ভাবে সার্ভার তৈরি ও রক্ষণাবেক্ষণ করার শৃঙ্খলা। এই অধ্যায়ে আমরা CM-এর ধারণা, তার ইতিহাস — হাতে করা কাজ থেকে Script, সেখান থেকে CFEngine, Puppet, Chef হয়ে Ansible পর্যন্ত — এবং তার মূলনীতিগুলো (Desired State, Drift, Idempotency) প্রথম নীতি থেকে শিখব। এই বইয়ের বাকি সব অধ্যায় এই ভিত্তির ওপর দাঁড়িয়ে আছে।

১.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Configuration Management-এর সংজ্ঞা দিতে ও ব্যাখ্যা করতে কেন এটি DevOps-এর ভিত্তি; (২) Manual Administration থেকে Script, সেখান থেকে CFEngine, Puppet, Chef ও Ansible পর্যন্ত CM-এর ইতিহাসের ধারাটি বলতে; (৩) Desired State, Actual State ও Drift — এই তিনটি ধারণা আলাদা করতে ও উদাহরণ দিতে; (৪) Idempotency কী তা প্রাথমিকভাবে ব্যাখ্যা করতে; (৫) Configuration Management, Provisioning ও Orchestration-এর মধ্যে পার্থক্য করতে; (৬) TechNova-র মতো একটি কোম্পানিতে CM না থাকলে কী কী ঝুঁকি তৈরি হয় তা তালিকা করতে।

১.৩ পূর্বশর্ত ধারণা

Linux টার্মিনালে কাজ করার অভিজ্ঞতা (`ssh`, `apt`, `systemctl`, ফাইল সম্পাদনা), একটি Web Server (যেমন Nginx) কী করে তার প্রাথমিক ধারণা, এবং Git-এ Version Control-এর মৌলিক ধারণা (Commit, Branch) থাকলেই চলবে। YAML, Ansible বা Python-এর পূর্ব অভিজ্ঞতা প্রয়োজন নেই — প্রতিটি ধারণা এই বইয়ে প্রথম ব্যবহারে ব্যাখ্যা করা হবে। আগের Volume-গুলো থেকে Docker ও Linux-এর ধারণা থাকলে ভালো, না থাকলেও এই অধ্যায় নিজে থেকেই বোঝা যাবে।

১.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি সার্ভার আসলে কী? হার্ডওয়্যার বা ভার্চুয়াল মেশিনের ওপরে একটি Operating System, তার ওপরে ইনস্টল করা প্যাকেজ, ডিস্কে লেখা কনফিগারেশন ফাইল, চলমান সার্ভিস, খোলা Port, ব্যবহারকারী-অ্যাকাউন্ট ও ফায়ারওয়াল-নিয়ম — এই সবকিছুর সমষ্টিই সার্ভারের **অবস্থা (State)**। সার্ভার পরিচালনা করা মানে আসলে এই অবস্থাকে নিয়ন্ত্রণ করা: অবস্থা যেমন হওয়া উচিত, তেমন রাখা। সমস্যা হলো, মানুষের হাতে এই অবস্থা বদলাতে বদলাতে একসময় কেউ আর জানে না অবস্থাটি আসলে কী। কেউ একজন একটি ফাইল সম্পাদনা করল, কেউ একটি প্যাকেজ আপডেট করল, কেউ একটি সার্ভিস বন্ধ করল — কোনোটারই লিখিত হিসাব নেই। এই অবস্থাকে বলে **Undocumented State** — আর এর থেকে জন্ম নেয় প্রায় সব প্রোডাকশন-দুর্ঘটনা।

Configuration Management এই সমস্যার মূলে আঘাত করে একটি সহজ নীতি দিয়ে: **সার্ভারের অবস্থা মানুষের মাথায় বা হাতের টাইপিং-এ নয়, লিখিত কোডে থাকবে।** আপনি লিখবেন "এই সার্ভারে Nginx ইনস্টল থাকবে, সংস্করণ 1.24, এই কনফিগারেশন ফাইল থাকবে, Port 80 ও 443 খোলা থাকবে" — আর একটি টুল সেই লেখা পড়ে সার্ভারকে সেই অবস্থায় নিয়ে যাবে। এই লেখাটি Git-এ সংরক্ষিত হয়, তাই কে কখন কী বদলেছে তার হিসাব থাকে, ভুল হলে আগের সংস্করণে ফেরা যায়, আর একই লেখা দিয়ে দশটি বা এক হাজারটি সার্ভার একই রকম করে তৈরি করা যায়। এই নীতির নাম **Infrastructure as Code (IaC)** — অবকাঠামোকে কোড হিসেবে লেখা — আর Configuration Management হলো IaC-এর সেই শাখা যা সার্ভারের ভেতরের অবস্থা নিয়ে কাজ করে।

> **CONCEPT:** Configuration Management-এর মূল প্রশ্নটি হলো: "সার্ভারটি এখন কেমন আছে?" — এর উত্তর যদি হয় "অমুক প্রকৌশলীর মনে আছে", তাহলে আপনার কোনো Configuration Management নেই। উত্তর যদি হয় "এই Git Repository-র এই ফাইলে লেখা আছে", তাহলে আছে। CM মানে সার্ভারের সত্য (Truth) একটি লিখিত, পর্যালোচনাযোগ্য (Reviewable), সংস্করণ-নিয়ন্ত্রিত জায়গায় রাখা — মানুষের স্মৃতিতে নয়।

এখানে তিনটি শব্দের পার্থক্য মনে রাখা জরুরি। **Provisioning** হলো সম্পদ তৈরি করা — VM তৈরি করা, Disk বরাদ্দ দেওয়া, নেটওয়ার্ক বানানো (Terraform এই কাজটি করে)। **Configuration Management** হলো তৈরি হওয়া সার্ভারের ভেতরের অবস্থা ঠিক করা — প্যাকেজ ইনস্টল, ফাইল লেখা, সার্ভিস চালানো (Ansible এই কাজটি করে)। **Orchestration** হলো অনেকগুলো কাজকে সঠিক ক্রমে, সঠিক সময়ে সমন্বয় করা — যেমন নতুন সংস্করণ Deploy করার সময় প্রথমে ডেটাবেস মাইগ্রেট করা, তারপর একে একে সার্ভার আপডেট করা, লোড ব্যালান্সার থেকে সার্ভার সরিয়ে আবার যোগ করা (Kubernetes বা Ansible Playbook-এর ভূমিকাই এখানে Orchestration)। একই টুল একাধিক ভূমিকা পালন করতে পারে, কিন্তু ধারণাগুলো আলাদা — এই পার্থক্য না বুঝলে ভুল টুল দিয়ে ভুল সমস্যা সমাধানের চেষ্টা হয়।

১.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Configuration Management-এর ইতিহাস — চারটি যুগ

প্রথম যুগ — হাতে করা কাজ (১৯৯০-এর দশক পর্যন্ত): সার্ভার কম ছিল, প্রতিটি সার্ভারে প্রকৌশলী সরাসরি ঢুকে কাজ করতেন। একটি সার্ভারের "ডকুমেন্টেশন" ছিল হয়তো একটি Wiki পাতা বা একজন জ্যেষ্ঠ প্রকৌশলীর স্মৃতি। সার্ভার নষ্ট হলে পুনর্নির্মাণ করতে দিন লেগে যেত। এই যুগের সমস্যাকে আজ আমরা বলি **Snowflake Server** — প্রতিটি সার্ভার তুষারকণার মতো অনন্য, কোনো দুটি এক রকম নয়।

দ্বিতীয় যুগ — Shell Script (১৯৯০-এর দশকের শেষ): প্রকৌশলীরা বারবার করা কাজগুলো Shell Script-এ লিখতে শুরু করলেন — একটি Script চালালেই Nginx ইনস্টল হয়, ফায়ারওয়াল বসে। এটি বড় অগ্রগতি, কিন্তু Script-এর একটি মৌলিক দুর্বলতা আছে: Script বলে কী করতে হবে (Imperative), কিন্তু বলে না শেষ অবস্থাটি কী হওয়া উচিত

(Declarative)। একই Script দুবার চালালে দ্বিতীয়বার ভিন্ন ফল হতে পারে — যেমন `echo "x" >> file` দুবার চালালে লাইনটি দুবার যোগ হয়। এই সমস্যার নাম **Non-Idempotent** আচরণ, যার বিস্তারিত ব্যাখ্যা অধ্যায় ৩-এ আসবে।

তৃতীয় যুগ — CFEngine, Puppet ও Chef (১৯৯৩–২০০৯): ১৯৯৩ সালে Mark Burgess CFEngine তৈরি করেন — প্রথম সত্যিকারের CM টুল, যা Desired State-এর ধারণা ও Convergence (সার্ভারকে বারবার কাঙ্ক্ষিত অবস্থার দিকে টেনে আনা) চালু করে। ২০০৫ সালে Puppet আসে — Declarative ভাষা, Client-Server (Agent) মডেল, যেখানে প্রতিটি সার্ভারে একটি Agent চলে ও নিয়মিত Central Server থেকে কনফিগারেশন টেনে নেয় (Pull Model)। ২০০৯ সালে Chef আসে — Ruby-ভিত্তিক, "Recipe" ও "Cookbook" ধারণা নিয়ে, যা প্রকৌশলীদের কাছে প্রোগ্রামিং-এর মতো মনে হতো। এই টুলগুলো CM-কে পেশাদার শৃঙ্খলায় পরিণত করে, কিন্তু তাদের দুটি খরচ ছিল: প্রতিটি সার্ভারে Agent ইনস্টল ও রক্ষণাবেক্ষণ করতে হতো, আর শিখতে হতো বিশেষায়িত ভাষা (Puppet-এর DSL বা Ruby)।

চতুর্থ যুগ — Ansible (২০১২–বর্তমান): ২০১২ সালে Michael DeHaan Ansible তৈরি করেন একটি ভিন্ন দর্শন নিয়ে: কোনো Agent নয়, শুধু SSH; কোনো বিশেষ ভাষা নয়, শুধু YAML; কোনো Central Server নয়, শুধু একটি Control Node থেকে Push। Ansible-এর সরলতাই তার শক্তি — একজন Linux প্রকৌশলী এক বিকেলেই প্রথম Automation লিখতে পারেন। ২০১৫ সালে Red Hat Ansible কিনে নেয়, আর আজ Ansible হলো বিশ্বের সর্বাধিক ব্যবহৃত CM টুলগুলোর একটি। এই বইয়ের বিষয় এই চতুর্থ যুগের শ্রেষ্ঠ প্রতিনিধি — Ansible — কিন্তু তার নীতিগুলো (Desired State, Idempotency, Convergence) সব যুগের সব টুলের ক্ষেত্রেই প্রযোজ্য।

টুল / যুগ	সময়	মডেল	ভাষা	মূল অবদান
হাতে করা কাজ	—১৯৯০	সরাসরি SSH	মানুষের স্মৃতি	— (সমস্যার উৎস)
Shell Script	১৯৯০-এর দশক	Push (হাতে চালানো)	Bash	পুনরাবৃত্তি সম্ভব হলো; কিন্তু Idempotency নেই
CFEngine	১৯৯৩	Pull (Agent)	নিজস্ব Policy ভাষা	Desired State ও Convergence-এর জন্ম
Puppet	২০০৫	Pull (Agent + Master)	Declarative DSL	CM-কে মূলধারায় আনা; Resource Abstraction
Chef	২০০৯	Pull (Agent + Server)	Ruby	Recipe/Cookbook; কোড হিসেবে অবকাঠামো
Ansible	২০১২	Push (Agentless, SSH)	YAML	Agent ছাড়া সরলতা; সবচেয়ে কম শেখার খরচ

Desired State, Actual State ও Drift

CM-এর পুরো তত্ত্বটি তিনটি শব্দে ধরা যায়। **Desired State** — সার্ভার কেমন হওয়া উচিত, যা কোডে লেখা থাকে। **Actual State** — সার্ভার এখন আসলে কেমন আছে। **Drift** — এই দুয়ের মধ্যে পার্থক্য। Drift তৈরি হয় অনেকভাবে: কেউ হাতে একটি ফাইল বদলেছে, একটি স্বয়ংক্রিয় আপডেট একটি প্যাকেজ বদলেছে, একটি সার্ভিস নিজে থেকে বন্ধ হয়েছে, বা কেউ একটি Port খুলেছে "সাময়িকভাবে" এবং ভুলে গেছে বন্ধ করতে। CM টুলের কাজ হলো Drift মাপা ও দূর করা — সার্ভারকে বারবার Desired State-এর দিকে ফিরিয়ে আনা। এই প্রক্রিয়ার নাম **Convergence**। Puppet ও Chef-এর Agent প্রতি ৩০ মিনিটে Convergence চালাত; Ansible-এ আপনি যখন Playbook চালান, তখন Convergence ঘটে।

CM বনাম Provisioning বনাম Orchestration — একটি উদাহরণ

TechNova একটি নতুন Web Server চায়। **Provisioning:** Terraform দিয়ে একটি Ubuntu 24.04 VM তৈরি করা হলো (IP 203.0.113.21), Disk ও নেটওয়ার্ক বরাদ্দ হলো। **Configuration Management:** Ansible দিয়ে সেই VM-এ Nginx ইনস্টল করা হলো, `technova.example`-এর কনফিগারেশন বসানো হলো, UFW-তে Port 80/443 খোলা হলো। **Orchestration:** Deploy-এর সময় Ansible Playbook প্রথমে Load Balancer থেকে সার্ভারটিকে সরাল, নতুন কোড Deploy করল, Health Check করল, তারপর আবার Load Balancer-এ যোগ করল — যাতে গ্রাহক কোনো বিঘ্ন না পান। তিনটি কাজই স্বয়ংক্রিয়, কিন্তু তিনটি আলাদা প্রশ্নের উত্তর দেয়: "সম্পদ আছে কি না", "সম্পদ কেমন আছে", "কাজগুলো কোন ক্রমে হবে"।

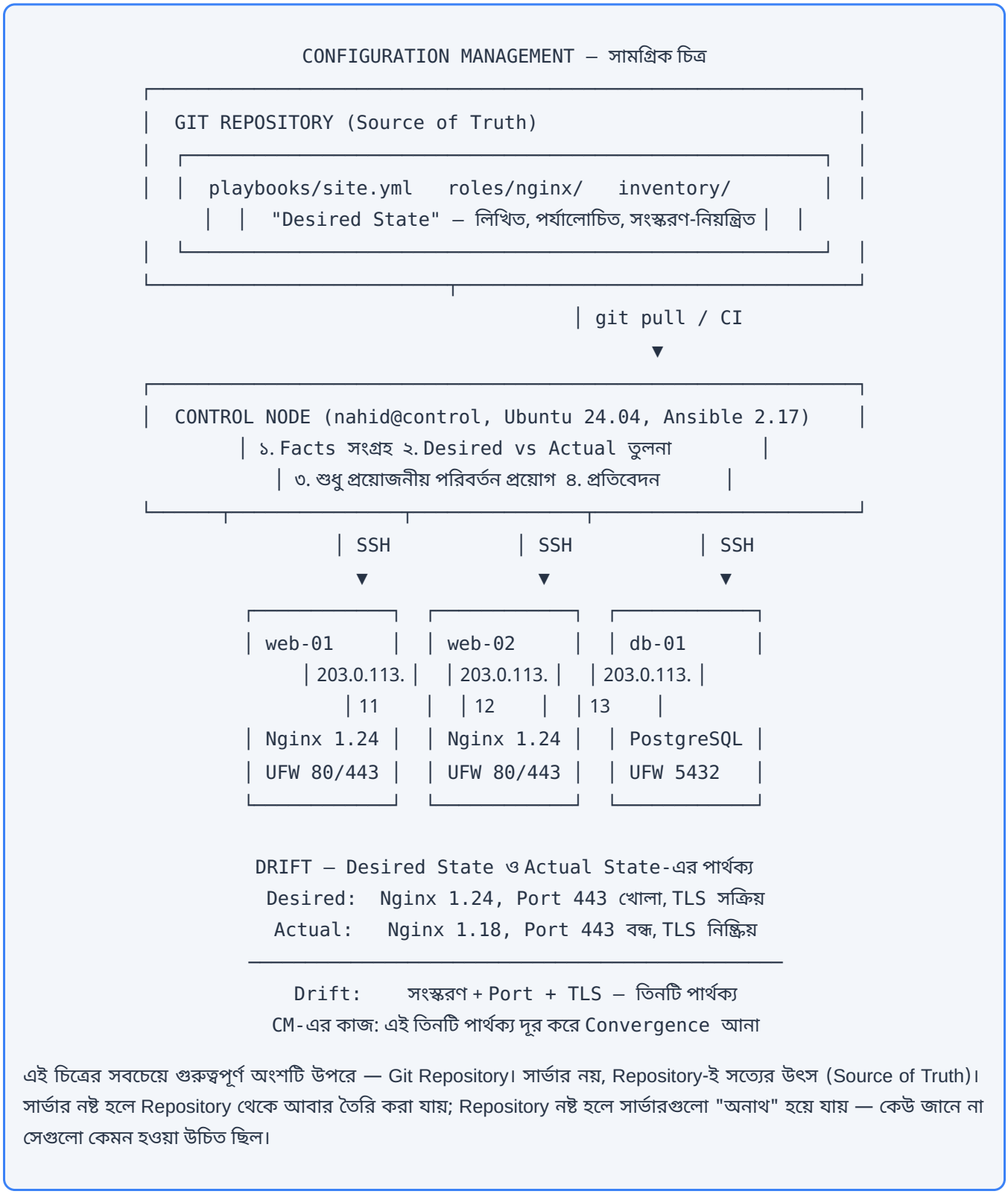
১.৬ অভ্যন্তরীণ কার্যপ্রণালী

একটি CM সিস্টেম ভেতরে ভেতরে কী করে? পাঁচটি ধাপে ভাবা যায়। **ধাপ ১ — ঘোষণা (Declare):** প্রকৌশলী Desired State লেখেন — যেমন "Nginx ইনস্টল থাকবে" — একটি Manifest, Recipe বা Playbook-এ। **ধাপ ২ — সংগ্রহ (Gather):** টুলটি লক্ষ্য সার্ভারের বর্তমান অবস্থা সংগ্রহ করে — কী ইনস্টল আছে, কোন ফাইলে কী আছে, কোন সার্ভিস চলছে (Ansible-এ একে বলে **Facts** সংগ্রহ)। **ধাপ ৩ — তুলনা (Compare):** Desired State ও Actual State মিলিয়ে দেখা হয় — কোথায় পার্থক্য, অর্থাৎ Drift কোথায়। **ধাপ ৪ — সংশোধন (Remediate):** শুধু সেই জায়গাগুলোতেই পরিবর্তন করা হয় যেখানে পার্থক্য আছে — যা ঠিক আছে তা ছোঁয়া হয় না। এই "শুধু প্রয়োজনীয় পরিবর্তন" নীতিই Idempotency-র ভিত্তি। **ধাপ ৫ — প্রতিবেদন (Report):** কী বদলেছে, কী বদলায়নি, কোথায় ব্যর্থ হয়েছে — তার হিসাব দেওয়া হয়, যাতে নিরীক্ষা (Audit) করা যায়।

এই পাঁচ ধাপের মধ্যে সবচেয়ে গুরুত্বপূর্ণ হলো ধাপ ৩ ও ৪-এর যোগসূত্র: টুলটি অন্ধভাবে কমান্ড চালায় না, প্রথমে দেখে তারপর সিদ্ধান্ত নেয়। একটি Shell Script বলে "Nginx ইনস্টল করো" — ইনস্টল থাকলেও আবার চালায়। একটি CM টুল বলে "Nginx ইনস্টল আছে কি না দেখো; না থাকলে ইনস্টল করো; থাকলে কিছু করো না"। এই পার্থক্যটিই Script ও CM টুলের মধ্যে মূল পার্থক্য, এবং অধ্যায় ৩-এ আমরা এটি হাতে-কলমে প্রমাণ করব।

> **TIP:** CM শেখার সময় একটি অভ্যাস গড়ে তুলুন: কোনো সার্ভারে হাতে কিছু বদলানোর আগে নিজেকে জিজ্ঞেস করুন — "এই পরিবর্তনটি কি আমার কোডে লেখা আছে?" যদি উত্তর "না" হয়, তাহলে প্রথমে কোডে লিখুন, তারপর টুল দিয়ে প্রয়োগ করুন। হাতে বদলে পরে কোডে লেখার অভ্যাসই Drift-এর জন্ম দেয়। এই নীতিকে বলে **"Code First, Hands Never"** — প্রোডাকশন সার্ভারে সরাসরি হাত দেওয়া নিষিদ্ধ, সব পরিবর্তন কোড দিয়ে আসবে।

১.৭ আর্কিটেকচার চিত্র



এই চিত্রের সবচেয়ে গুরুত্বপূর্ণ অংশটি উপরে — Git Repository। সার্ভার নয়, Repository-ই সত্যের উৎস (Source of Truth)। সার্ভার নষ্ট হলে Repository থেকে আবার তৈরি করা যায়; Repository নষ্ট হলে সার্ভারগুলো "অনাথ" হয়ে যায় — কেউ জানে না সেগুলো কেমন হওয়া উচিত ছিল।

১.৮ ব্যবহারিক উদাহরণ

উদাহরণ ১ — Desired State লেখা: TechNova-র Web Server-এর Desired State এক বাক্যে: "Ubuntu 24.04 সার্ভারে Nginx 1.24 ইনস্টল থাকবে, `/etc/nginx/sites-available/technova.example` ফাইলে আমাদের

কনফিগারেশন থাকবে, Nginx সার্ভিস চলবে ও Boot-এ চালু হবে, UFW-তে 80 ও 443 Port খোলা থাকবে।" লক্ষ্য করুন — এখানে কোনো কমান্ড নেই, শুধু শেষ অবস্থার বর্ণনা আছে। এই বর্ণনাই পরে Ansible Playbook হবে (অধ্যায় ৫)।

উদাহরণ ২ — Drift শনাক্ত করা: একদিন একজন প্রকৌশলী "সাময়িকভাবে" web-02-এ Port 8080 খুললেন পরীক্ষার জন্য, আর বন্ধ করতে ভুলে গেলেন। Desired State-এ Port 8080 নেই, Actual State-এ আছে — এটি Drift। CM টুল পরের বার চললে এই পার্থক্য ধরে ফায়ারওয়ালের নিয়মটি সরিয়ে দেবে (যদি Playbook সেভাবে লেখা হয়)। হাতে পরিচালিত পরিবেশে এই Port হয়তো মাসের পর মাস খোলা থাকত — একটি নিরাপত্তা-ছিদ্র হিসেবে।

উদাহরণ ৩ — CM বনাম Provisioning বনাম Orchestration, এক লাইনে: Provisioning বলে "একটি সার্ভার দাও", CM বলে "সার্ভারটি এই রকম হোক", Orchestration বলে "প্রথমে A করো, তারপর B, তারপর C — আর B ব্যর্থ হলে থেমে যাও"।

```
# উদাহরণ: একই কাজ — তিনটি ভিন্ন স্তরের টুল
# Provisioning (Terraform — ধারণাগত):
#   resource "aws_instance" "web" { ami = "ubuntu-24.04" }   # সার্ভার তৈরি
# Configuration Management (Ansible — ধারণাগত):
#   - name: Install Nginx
#     apt: name=nginx state=present                          # ভেতরের অবস্থা
# Orchestration (Ansible Playbook-এর ক্রম — ধারণাগত):
#   1. Load Balancer থেকে সার্ভার সরাও → 2. Deploy করো → 3. Health Check → 4. আবার যোগ করো
```

১.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

এই অধ্যায়ে আমরা এখনো Ansible ইনস্টল করিনি, তাই নিচের ফাইলটি একটি ধারণাগত Desired State ঘোষণা — কীভাবে CM "কোড" দেখতে হয়, তার প্রথম ঝলক। এখানে YAML ব্যবহার করা হয়েছে কারণ Ansible YAML-এই লেখা হয়; বিস্তারিত ব্যাখ্যা অধ্যায় ৪ ও ৫-এ আসবে।

```
# desired-state.yml — TechNova Web Server-এর Desired State (ধারণাগত উদাহরণ)
# এই ফাইলটি Git-এ থাকবে; সার্ভার নয়, এই ফাইলই সত্যের উৎস।
---
web_server:
  operating_system: "Ubuntu 24.04 LTS"
  packages:
    - name: nginx
      version: "1.24"
      state: present
    - name: ufw
      state: present
  files:
    - path: /etc/nginx/sites-available/technova.example
      owner: root
      group: root
      mode: "0644"
      content_from: templates/technova.example.j2
  services:
    - name: nginx
      state: started
      enabled: true        # Boot-এ চালু হবে
  firewall:
    - port: 80
      proto: tcp
      state: allowed
    - port: 443
```

```
      proto: tcp
      state: allowed
  users:
    - name: deploy
      groups: [sudo]
      shell: /bin/bash
# পড়ার নিয়ম: এই ফাইলে কোনো "কীভাবে করব" নেই — শুধু "শেষে কেমন হবে" আছে।
# টুলের কাজ হলো এই বর্ণনা পড়ে সার্ভারকে এই অবস্থায় নিয়ে যাওয়া।
```

১.১০ ধাপে ধাপে বাস্তবায়ন

অনুশীলন: আপনার প্রথম Desired State নথি লেখা (কোনো টুল ছাড়াই)

1. একটি খাতা বা টেক্সট ফাইলে আপনার নিজের ল্যাপটপ বা একটি পরীক্ষামূলক সার্ভারের বর্তমান অবস্থা লিখুন — কোন OS, কোন প্যাকেজ ইনস্টল আছে, কোন সার্ভিস চলছে, কোন Port খোলা আছে। এটি আপনার **Actual State**।
2. এবার লিখুন সার্ভারটি কেমন হওয়া উচিত — অর্থাৎ **Desired State**। যেমন: "Python 3.12 ইনস্টল থাকবে, Git ইনস্টল থাকবে, SSH সার্ভিস চলবে, Port 22 শুধু আমার IP থেকে খোলা থাকবে।"
3. দুটি তালিকা পাশাপাশি রেখে পার্থক্যগুলো চিহ্নিত করুন — এই পার্থক্যগুলোই আপনার **Drift**।
4. প্রতিটি পার্থক্যের পাশে লিখুন — এটি কীভাবে তৈরি হলো? হাতে করা কোনো পরিবর্তন, নাকি স্বয়ংক্রিয় আপডেট? এই প্রশ্নের উত্তরই বলে দেবে আপনার পরিবেশে CM কেন দরকার।
5. Desired State-টি উপরের ১.৯-এর মতো YAML আকারে লিখুন ও একটি Git Repository-তে Commit করুন। অভিনন্দন — আপনি আপনার প্রথম Infrastructure as Code লিখলেন, যদিও কোনো টুল এখনো তা প্রয়োগ করেনি।

১.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

এই অধ্যায়ের অনুশীলনের কোনো টার্মিনাল আউটপুট নেই — কারণ আমরা এখনো কোনো টুল চালাইনি। প্রত্যাশিত ফল হলো একটি লিখিত নথি: একটি YAML ফাইল যেখানে আপনার সার্ভারের Desired State বর্ণিত আছে, আর একটি Drift-তালিকা যেখানে Desired ও Actual-এর পার্থক্য লেখা আছে। এই নথিটি যদি অন্য একজন প্রকৌশলী পড়ে আপনার সার্ভারটি হুবহু আবার তৈরি করতে পারেন — তাহলে নথিটি সফল। যদি না পারেন — যেমন "Nginx ইনস্টল করো" লেখা আছে কিন্তু কোন সংস্করণ, কোন কনফিগারেশন তা লেখা নেই — তাহলে নথিটি অসম্পূর্ণ, আর এটাই CM-এর প্রথম শিক্ষা: **Desired State হতে হবে সম্পূর্ণ ও দ্ব্যর্থহীন (Unambiguous)** — এতটাই স্পষ্ট যে দুজন মানুষ একইভাবে বুঝবেন।

```
# একটি ভালো Drift-তালিকার উদাহরণ (চিত্রণমূলক):
# Desired: Nginx 1.24  | Actual: Nginx 1.18  | Drift: সংস্করণ পুরনো — আপডেট দরকার
# Desired: Port 443 খোলা | Actual: বন্ধ       | Drift: ফায়ারওয়াল-নিয়ম অনুপস্থিত
# Desired: deploy ব্যবহারকারী আছে | Actual: নেই | Drift: ব্যবহারকারী তৈরি হয়নি
# এই তালিকাই CM টুলের কাজের তালিকা: প্রতিটি লাইন একটি সংশোধন (Remediation)।
```

১.১২ সাধারণ ভুল

- **ভুল ধারণা:** "Configuration Management মানে শুধু Script লেখা।" — না; Script Imperative ও সাধারণত Non-Idempotent হয়। CM মানে Desired State ঘোষণা করা ও Convergence নিশ্চিত করা।
- **ভুল ধারণা:** "আমাদের সার্ভার মাত্র তিনটি, CM দরকার নেই।" — তিনটি সার্ভারই Drift-এর জন্য যথেষ্ট; আর তিনটি থেকে ত্রিশটি হওয়ার সময় CM না থাকলে বিপর্যয় আসে। CM ছোট থেকে শুরু করার জিনিস।

- **ভুল:** CM কোড লেখা কিন্তু সার্ভারে হাতেও পরিবর্তন করা — এতে কোড ও বাস্তবতা আলাদা হয়ে যায়, আর CM-এর পুরো মূল্য নষ্ট হয়। নিয়ম: সব পরিবর্তন কোড দিয়ে।
- **ভুল ধারণা:** "Provisioning আর CM একই জিনিস।" — না; Terraform সার্ভার তৈরি করে, Ansible তার ভেতরের অবস্থা ঠিক করে। দুটি আলাদা স্তর, দুটি আলাদা টুল — যদিও Ansible দিয়ে ছোট পরিসরে Provisioning-ও করা যায়।
- **ভুল:** Desired State অসম্পূর্ণ লেখা — যেমন "Nginx ইনস্টল থাকবে" লেখা কিন্তু সংস্করণ, কনফিগারেশন ও সার্ভিসের অবস্থা না লেখা। অসম্পূর্ণ Desired State মানে আংশিক সত্য, যা Drift লুকিয়ে রাখে।
- **ভুল ধারণা:** "Ansible শিখলেই CM শেখা হয়ে গেল।" — না; Ansible একটি টুল। Desired State, Drift, Idempotency, Convergence — এই নীতিগুলো না বুঝলে Ansible দিয়ে ভুল Automation লেখা হয়, যা অধ্যায় ৩-এ দেখব।

১.১৩ সমস্যা সমাধান (Troubleshooting)

1. **সমস্যা:** "আমি জানি না আমার সার্ভারে এখন কী আছে।" — **পরীক্ষা:** সার্ভারে `dpkg -l | grep nginx`, `systemctl status nginx`, `ss -tlnp` ও `ufw status` চালিয়ে বর্তমান অবস্থা সংগ্রহ করুন। **সমাধান:** এই তথ্যই আপনার Actual State নথির শুরু; CM-এর প্রথম ধাপ হলো বর্তমান অবস্থা জানা।
2. **সমস্যা:** "দুটি সার্ভার একই কাজ করে, কিন্তু আচরণ আলাদা।" — **পরীক্ষা:** দুটি সার্ভারে প্যাকেজের সংস্করণ ও কনফিগারেশন ফাইল তুলনা করুন (`diff` ব্যবহার করুন)। **সমাধান:** পার্থক্যগুলো Drift — একটি Desired State লিখে দুটি সার্ভারকেই সেখানে নিয়ে আসুন।
3. **সমস্যা:** "একজন প্রকৌশলী চলে গেছেন, আর কেউ জানে না সার্ভার কীভাবে তৈরি হয়েছিল।" — **পরীক্ষা:** Shell History (`~/.bash_history`) ও পুরনো নোট খুঁজুন। **সমাধান:** এটি CM না থাকার ক্লাসিক লক্ষণ; বর্তমান অবস্থা থেকে Desired State পুনর্গঠন করুন ও Git-এ রাখুন — ভবিষ্যতে আর কারো স্মৃতির ওপর নির্ভর করতে হবে না।
4. **সমস্যা:** "CM টুল চালালাম, কিন্তু সার্ভার বদলাল না।" — **পরীক্ষা:** টুলটি কি আসলে Drift খুঁজে পেয়েছে? হয়তো Desired State ভুল লেখা হয়েছে, বা টুলটি ভুল সার্ভারে চলেছে। **সমাধান:** প্রথমে Facts/Actual State দেখুন, তারপর Desired State-এর সঙ্গে মিলান — ধাপ ৩ (তুলনা) ছাড়া ধাপ ৪ (সংশোধন) অর্থহীন।

১.১৪ নিরাপত্তা বিবেচনা

> **WARNING:** Configuration Management ছাড়া পরিবেশে সবচেয়ে বড় নিরাপত্তা-ঝুঁকি হলো অদৃশ্য Drift: একটি সার্ভারে নিরাপত্তা-প্যাচ লাগানো হলো, আরেকটিতে ভুলে যাওয়া হলো; একটি সার্ভারে একটি Port "সাময়িকভাবে" খোলা হলো, আর বন্ধ করা হলো না। আক্রমণকারী এই একটি ভুলে যাওয়া সার্ভার দিয়েই পুরো নেটওয়ার্কে ঢোকে। CM এই ঝুঁকি কমায় — কারণ Desired State-এ লেখা থাকে কোন Port খোলা থাকবে, কোন প্যাকেজের কোন সংস্করণ থাকবে — আর নিয়মিত Convergence সেই অবস্থা ফিরিয়ে আনে। কিন্তু সতর্কতা: CM টুলের হাতে সব সার্ভারের Root ক্ষমতা থাকে, তাই CM-এর নিজের নিরাপত্তা (Git Repository-র Access, SSH Key-র সুরক্ষা, Secret-এর ব্যবস্থাপনা) পুরো অবকাঠামোর নিরাপত্তার সমান গুরুত্বপূর্ণ।

Secret ব্যবস্থাপনা CM-এর একটি বিশেষ নিরাপত্তা-বিষয়: ডেটাবেসের পাসওয়ার্ড, API Key, TLS Private Key — এগুলো কখনোই CM কোডে সরাসরি লেখা যাবে না। এই বইয়ে সব Secret-এর জায়গায় `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে; বাস্তব কাজে Ansible Vault বা HashiCorp Vault-এর মতো Secret ব্যবস্থাপনা ব্যবহার করতে হবে (Volume 09-এর পরের Part-এ বিস্তারিত)। আর একটি নীতি: CM Repository-তে কে কী বদলাতে পারে, তার Access Control

থাকতে হবে — Production-এর Desired State বদলানো মানে সব সার্ভার বদলে দেওয়া, তাই এই Repository-তে Pull Request ও পর্যালোচনা (Review) বাধ্যতামূলক হওয়া উচিত।

১.১৫ প্রোডাকশন ইঞ্জিনিয়ারিং অনুশীলন

সেরা অনুশীলন: (১) সব CM কোড Git-এ রাখুন — কোনো Playbook শুধু কারো ল্যাপটপে থাকবে না; (২) Production-এর পরিবর্তন Pull Request ও পর্যালোচনার মাধ্যমে আসবে — সরাসরি Main Branch-এ Push নয়; (৩) প্রথমে Staging পরিবেশে Playbook চালিয়ে পরীক্ষা করুন, তারপর Production-এ; (৪) প্রতিটি Playbook-এ `--check` (Dry Run) দিয়ে প্রথমে দেখুন কী বদলাবে, তারপর প্রয়োগ করুন; (৫) CM-এর প্রতিটি Run-এর লগ সংরক্ষণ করুন — নিরীক্ষার (Audit) সময় "কে, কখন, কী বদলেছে" প্রমাণ করতে লাগবে; (৬) Desired State-কে ছোট ছোট Role-এ ভাগ করুন (যেমন nginx Role, firewall Role) যাতে পুনঃব্যবহার করা যায়। **অ্যান্টি-প্যাটার্ন:** "একটি বিশাল Playbook যেখানে সবকিছু আছে" — রক্ষণাবেক্ষণ অসম্ভব হয়; "সরাসরি Production-এ পরীক্ষা" — ভুল হলে গ্রাহক ভোগে; "Secret কোডে লেখা" — Git History থেকে Secret মুছে ফেলা প্রায় অসম্ভব।

১.১৬ অধ্যায়ের সারাংশ

Configuration Management হলো সার্ভারের অবস্থাকে লিখিত, সংস্করণ-নিয়ন্ত্রিত Desired State হিসেবে সংজ্ঞায়িত করা ও স্বয়ংক্রিয়ভাবে সেই অবস্থা বজায় রাখার শৃঙ্খলা। এর ইতিহাস হাতে করা কাজ থেকে Script, সেখান থেকে CFEngine, Puppet, Chef হয়ে Ansible পর্যন্ত এসেছে — প্রতিটি ধাপে মূল অগ্রগতি ছিল Desired State ঘোষণা করা ও Convergence নিশ্চিত করা। Desired State, Actual State ও Drift — এই তিনটি ধারণা CM-এর কেন্দ্র; Provisioning সম্পদ তৈরি করে, CM তার ভেতরের অবস্থা ঠিক করে, আর Orchestration কাজের ক্রম সমন্বয় করে। CM-এর সত্যের উৎস হলো Git Repository, মানুষের স্মৃতি নয়।

১.১৭ জ্ঞান যাচাই — প্রশ্ন

1. Configuration Management-এর সংজ্ঞা দিন — Desired State ও Source of Truth শব্দ দুটি ব্যবহার করে।
2. Desired State, Actual State ও Drift — তিনটির পার্থক্য একটি বাস্তব উদাহরণ দিয়ে ব্যাখ্যা করুন।
3. CFEngine, Puppet, Chef ও Ansible — প্রতিটি CM-এর ইতিহাসে কী নতুন অবদান রেখেছে?
4. Provisioning, Configuration Management ও Orchestration-এর পার্থক্য একটি নতুন Web Server স্থাপনের উদাহরণ দিয়ে বোঝান।
5. একটি সার্ভারে হাতে Port খুলে ভুলে যাওয়া কেন নিরাপত্তা-ঝুঁকি, আর CM কীভাবে তা প্রতিরোধ করে?

১.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. **Desired State নথি:** আপনার নিজের ল্যাপটপ বা একটি পরীক্ষামূলক Ubuntu সার্ভারের জন্য একটি Desired State YAML লিখুন (১.৯-এর আদলে) — OS, অন্তত ৫টি প্যাকেজ, ২টি সার্ভিস ও ফায়ারওয়াল-নিয়মসহ — ও Git-এ Commit করুন।
2. **Drift নিরীক্ষা:** দুটি সার্ভার বা দুটি Container-এর প্যাকেজ তালিকা তুলনা করুন (`dpkg -l` ব্যবহার করে) ও Drift-তালিকা তৈরি করুন।
3. **ইতিহাস গবেষণা:** CFEngine-এর "Promise Theory" সম্পর্কে সংক্ষেপে পড়ুন ও এক প্যারায় লিখুন — এটি Desired State ধারণার সঙ্গে কীভাবে সম্পর্কিত।

অধ্যায় 2 • PART I — Configuration Management Foundations

Chapter 2 — Manual Server Administration vs Configuration Automation

২.১ ভূমিকা

ধরুন TechNova-র তিনটি সার্ভার আছে: `web-01` (203.0.113.11), `web-02` (203.0.113.12) ও `web-03` (203.0.113.13)। তিনটিই Ubuntu 24.04, তিনটিতেই Nginx চলার কথা, তিনটির পেছনেই একই Load Balancer। এক সোমবার সকালে গ্রাহকরা অভিযোগ করলেন — কিছু অনুরোধ (Request) ব্যর্থ হচ্ছে, কিছু পাতা পুরনো সংস্করণ দেখাচ্ছে। তদন্তে বেরিয়ে এলো: web-01-এ Nginx 1.24 ও নতুন কনফিগারেশন আছে; web-02-এ Nginx 1.18 ও পুরনো কনফিগারেশন; web-03-এ একটি নিরাপত্তা-প্যাচই লাগানো হয়নি, কারণ যিনি প্যাচ লাগিয়েছিলেন তিনি ছুটিতে গিয়েছিলেন ও কাউকে বলেননি। তিনটি সার্ভার একই কাজের জন্য তৈরি হয়েছিল, কিন্তু হাতে হাতে পরিচালনা করতে করতে তারা তিনটি আলাদা সার্ভার হয়ে গেছে। এই অধ্যায়টি সেই গল্পের ব্যবচ্ছেদ — হাতে সার্ভার চালানো কেন ভাঙে, তার খরচ কত, আর Automation কীভাবে সেই ভাঙন সারায়।

২.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Snowflake Server বলতে কী বোঝায় তা ব্যাখ্যা করতে; (২) Pets বনাম Cattle উপমাটি ব্যাখ্যা করতে ও TechNova-র ক্ষেত্রে প্রয়োগ করতে; (৩) হাতে করা প্রশাসনের খরচ — সময়, মানবিক ভুল, নিরীক্ষার অভাব, পুনরুৎপাদনের অসম্ভবতা — চারটি দিক থেকে বিশ্লেষণ করতে; (৪) একটি বাস্তব দৃশ্যে তিনটি সার্ভারের Drift কীভাবে তৈরি হয় তা ধাপে ধাপে বর্ণনা করতে; (৫) Automation কখন বিনিয়োগের যোগ্য, তার হিসাব দিতে; (৬) "কাজটি একবারই করতে হবে" — এই যুক্তিটি কেন ভুল, তা ব্যাখ্যা করতে।

২.৩ পূর্বশর্ত ধারণা

অধ্যায় ১-এর Desired State, Actual State ও Drift-এর ধারণা; SSH দিয়ে সার্ভারে ঢুকে `apt install`, ফাইল সম্পাদনা (`nano` / `vim`) ও `systemctl restart` করার অভিজ্ঞতা; এবং একটি Web Server-এর কনফিগারেশন ফাইল কী করে তার প্রাথমিক ধারণা। এই অধ্যায়ের TechNova দৃশ্যটি বুঝতে Ansible জানার দরকার নেই — দৃশ্যটি হাতে করা কাজেরই গল্প।

২.৪ প্রথম নীতি থেকে ব্যাখ্যা

মানুষের হাত দিয়ে সার্ভার পরিচালনার মূল সমস্যাটি একটি সহজ সত্যে নিহিত: **মানুষ ভুল করে, ভুলে যায়, আর একই কাজ দুবার একইভাবে করে না।** একটি কমান্ড টাইপ করতে গিয়ে একটি অক্ষর ভুল হলে সার্ভারের অবস্থা বদলে যায়, কিন্তু সেই ভুলের কোনো রেকর্ড থাকে না। একজন প্রকৌশলী একটি ফাইল সম্পাদনা করলেন — সেই সম্পাদনার কারণ, সময় ও সঠিক বিষয়বস্তু শুধু তার মাথায় থাকে। তিনি ছুটিতে গেলে বা চাকরি ছাড়লে সেই জ্ঞান চলে যায়। আর সবচেয়ে বিপজ্জনক দিকটি হলো: হাতে করা কাজ **পরীক্ষাযোগ্য নয়** — আপনি Staging-এ একটি পরিবর্তন হাতে করে পরীক্ষা করলেন, Production-এ আবার হাতে করলেন, কিন্তু দুটি হাতের কাজ কখনোই হুবহু এক হয় না। একটি টাইপো, একটি ভুলে যাওয়া ধাপ — আর Production Staging-এর মতো আচরণ করে না।

Automation এই সমস্যার সমাধান করে কাজটিকে মানুষের হাত থেকে একটি লিখিত, পুনরাবৃত্তিযোগ্য (Repeatable) প্রক্রিয়ায় সরিয়ে। একই নির্দেশ দশবার চালালে দশবার একই ফল হয় — কারণ নির্দেশটি বদলায় না, ক্লান্ত হয় না, ভুলে যায়

না। এই নীতির সবচেয়ে পরিচিত উপমা হলো **Pets বনাম Cattle**: পুরনো যুগে সার্ভার ছিল "পোষা প্রাণী" (Pet) — প্রতিটির নাম আছে (web-01), প্রতিটিকে হাতে যত্ন করা হয়, একটি অসুস্থ হলে তাকে সুস্থ করার চেষ্টা করা হয়। নতুন যুগে সার্ভার হলো "গবাদি পশু" (Cattle) — প্রতিটি একই রকম, কোনোটির বিশেষ নাম-পরিচয় নেই, একটি নষ্ট হলে তাকে সারিয়ে তোলার চেষ্টা না করে মুছে ফেলে একই রকম আরেকটি তৈরি করা হয়। Cattle মডেল সম্ভব হয় শুধু Automation থাকলে — কারণ একই রকম সার্ভার বারবার তৈরি করতে হলে তার সংজ্ঞা কোডে থাকতে হবে।

> **CONCEPT:** Snowflake Server আর Pet Server একই সমস্যার দুটি নাম। Snowflake মানে "কোনো দুটি সার্ভার এক রকম নয়" — আর Pet মানে "প্রতিটি সার্ভারকে আলাদাভাবে হাতে যত্ন করতে হয়"। দুটিরই মূল কারণ একটাই: সার্ভারের অবস্থা কোডে লেখা নেই, মানুষের হাতের কাজে তৈরি হয়েছে। Automation-এর লক্ষ্য সার্ভারকে Cattle বানানো নয় — লক্ষ্য হলো সার্ভারের অবস্থাকে এমনভাবে লেখা যাতে যেকোনো সার্ভার যেকোনো সময় হুবহু আবার তৈরি করা যায়।

২.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

হাতে করা প্রশাসনের চারটি খরচ

১. সময়ের খরচ (Time Cost): একটি সার্ভারে Nginx ইনস্টল ও কনফিগার করতে একজন দক্ষ প্রকৌশলীর ২০–৩০ মিনিট লাগে। তিনটি সার্ভারে ৯০ মিনিট। ত্রিশটি সার্ভারে এক কর্মদিবসের বেশি — আর এর মধ্যে একটি সার্ভারে ভুল হলে আবার সময় লাগে খুঁজে বের করতে। Automation-এ প্রথম Playbook লিখতে হয়তো দুই ঘণ্টা লাগে, কিন্তু তারপর ত্রিশটি সার্ভারে চালাতে লাগে পাঁচ মিনিট। গণিতটি সহজ: কাজটি যতবার পুনরাবৃত্তি হয়, Automation তত সস্তা হয়।

২. মানবিক ভুলের খরচ (Human Error Cost): গবেষণায় দেখা গেছে, প্রোডাকশন-বিভ্রাটের একটি বড় অংশের কারণ মানুষের ভুল — ভুল সার্ভারে কমান্ড চালানো, ভুল ফাইল মুছে ফেলা, ভুল Port খোলা। হাতে কাজ করার সময় কোনো সুরক্ষা-জাল (Safety Net) নেই: `rm -rf` ভুল ডিরেক্টরিতে চালালে তা ফিরিয়ে আনা যায় না। Automation-এ ভুলও হতে পারে — কিন্তু ভুলটি কোডে লেখা থাকে, পর্যালোচনা (Review) করা যায়, Staging-এ পরীক্ষা করা যায়, আর ভুল ধরা পড়লে কোড ঠিক করে সব সার্ভারে একসঙ্গে প্রয়োগ করা যায়।

৩. নিরীক্ষার অভাব (Lack of Auditability): "গত মঙ্গলবার web-02-এ কে কী বদলেছিল?" — হাতে পরিচালিত পরিবেশে এই প্রশ্নের উত্তর দেওয়া প্রায় অসম্ভব। Shell History আংশিক তথ্য দেয়, কিন্তু কে কোন উদ্দেশ্যে কী করেছিলেন তা বলে না। Automation-এ প্রতিটি পরিবর্তন Git Commit — কে, কখন, কেন, কী বদলেছে, তার সম্পূর্ণ হিসাব। নিয়ন্ত্রক সংস্থা (Regulator) বা গ্রাহকের নিরীক্ষার সময় এই হিসাবই প্রমাণ।

৪. পুনরুৎপাদনের অসম্ভবতা (Non-Reproducibility): হাতে তৈরি সার্ভার নষ্ট হলে তাকে হুবহু আবার তৈরি করা যায় না — কারণ তৈরির প্রক্রিয়াটি কোথাও লেখা নেই। ফলে Disaster Recovery মানে "মনে করে করে আবার তৈরি করা" — যা ধীর ও ভুলপ্রবণ। Automation-এ Recovery মানে একই Playbook আবার চালানো — যা পরীক্ষিত ও নির্ভরযোগ্য।

TechNova দৃশ্য — তিনটি সার্ভার কীভাবে আলাদা হয়ে গেল

TechNova-র তিনটি সার্ভার একই দিনে একইভাবে তৈরি হয়েছিল — একই Ubuntu Image থেকে। তারপর যা ঘটল: **সপ্তাহ ১:** রাহাত web-01-এ Nginx 1.24-এ আপডেট করলেন, কিন্তু web-02 ও web-03-এ করতে ভুলে গেলেন — কারণ কোনো তালিকা ছিল না। **সপ্তাহ ৩:** নাহিদ web-02-এ একটি নতুন কনফিগারেশন বসালেন পরীক্ষার জন্য, কিন্তু web-01 ও web-03-এ বসানো হয়নি — কারণ পরিবর্তনটি কোথাও লেখা ছিল না। **সপ্তাহ ৫:** একটি নিরাপত্তা-সতর্কতা এলো — OpenSSL-এর দুর্বলতা। সুমাইয়া web-01 ও web-02 প্যাচ করলেন, কিন্তু web-03-এর কথা মনে ছিল না — কারণ সার্ভারের তালিকাটি তার মাথায় ছিল, কোনো Inventory ফাইলে নয়। **সপ্তাহ ৮:** Load Balancer-এর পেছনে তিনটি সার্ভার

তিন রকম আচরণ করছে — গ্রাহকরা কখনো নতুন পাতা পান, কখনো পুরনো; একটি সার্ভার নিরাপত্তা-ঝুঁকিতে। এই গল্পের প্রতিটি ধাপে কেউ "খারাপ কাজ" করেননি — প্রত্যেকে সাধ্যমতো ভালো কাজ করেছেন। ভাঙনটি ব্যক্তিগত নয়, পদ্ধতিগত: হাতে করা কাজে Drift অনিবার্য।

বিষয়	হাতে করা প্রশাসন (Manual)	কনফিগারেশন অটোমেশন
গতি (৩০ সার্ভার)	দিনের পর দিন, সার্ভার প্রতি আলাদা সময়	একবার লিখে মিনিটে সব সার্ভারে প্রয়োগ
সামঞ্জস্য	প্রতিটি সার্ভার আলাদা (Snowflake)	সব সার্ভার Desired State-এর সমান
ভুলের ধরন	অদৃশ্য, এলোমেলো, পুনরাবৃত্তি হয়	কোডে ধরা পড়ে, একবার ঠিক করলে সব জায়গায় ঠিক
নিরীক্ষা	কারো মনে নেই কে কী করেছিল	Git History-তে সম্পূর্ণ হিসাব
পুনর্নির্মাণ	মনে করে করে, দিন লাগে	একই Playbook চালিয়ে, মিনিটে
জ্ঞান-নির্ভরতা	নির্দিষ্ট ব্যক্তির ওপর নির্ভরশীল	কোডে লেখা, যে কেউ চালাতে পারে
পরীক্ষাযোগ্যতা	Staging ও Production কখনো এক হয় না	একই Playbook দুই পরিবেশে চলে

"কাজটি একবারই করতে হবে" — ভুল যুক্তি

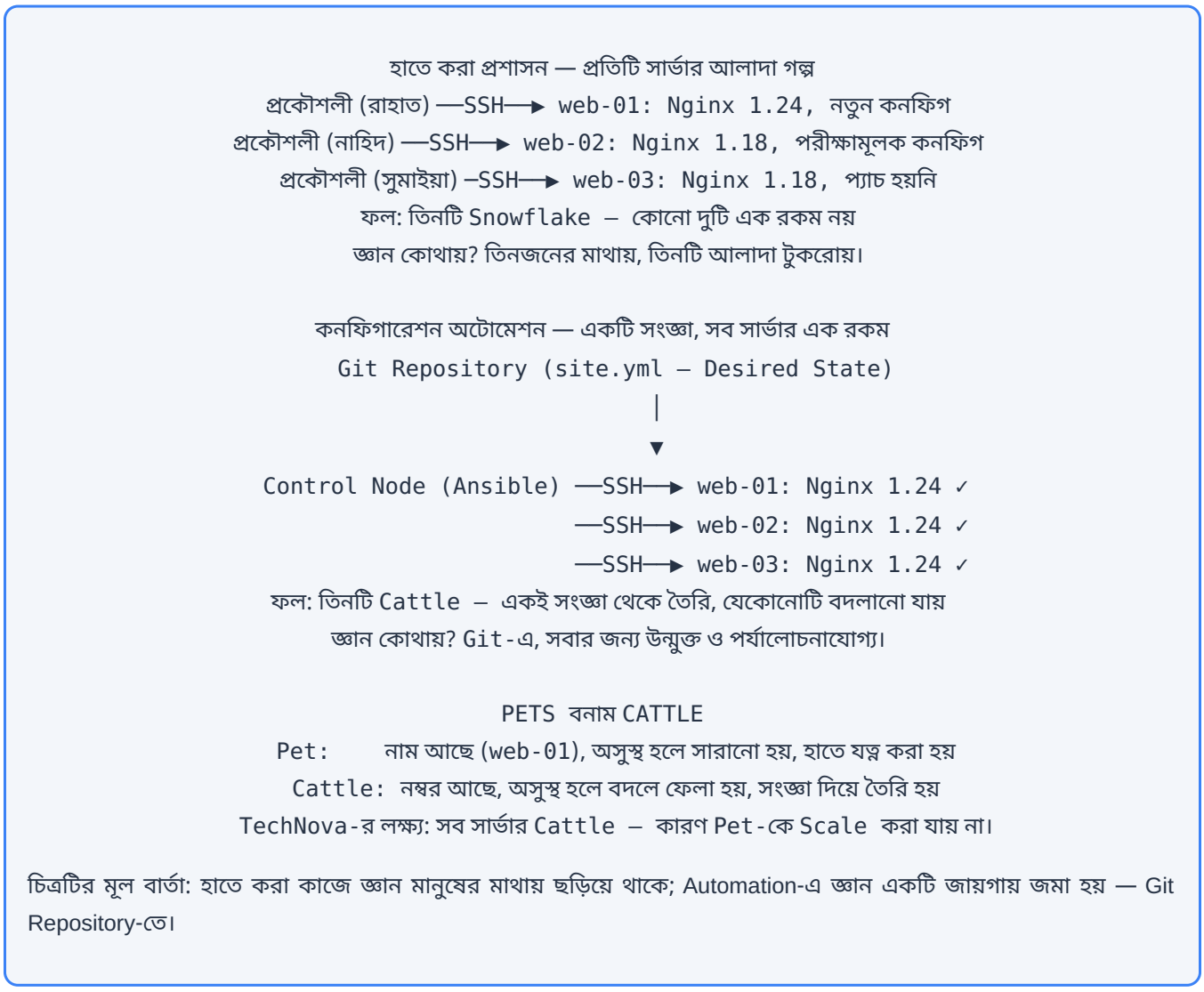
Automation-এর বিরুদ্ধে সবচেয়ে সাধারণ যুক্তি: "এই কাজটি তো একবারই করতে হবে, Script লেখার সময় নেই।" এই যুক্তিটি তিন কারণে ভুল। প্রথমত, প্রায় কোনো কাজই একবার করা হয় না — সার্ভার বাড়ে, সার্ভার বদলায়, সার্ভার নষ্ট হয়, আর প্রতিবার একই কাজ আবার করতে হয়। দ্বিতীয়ত, "একবার" করা কাজটিও পরীক্ষা করতে হয় — Staging-এ একবার, Production-এ একবার — আর দুবার হাতে করা মানেই দুটি ভিন্ন কাজ। তৃতীয়ত, কাজটি করার জ্ঞানটিই মূল্যবান — কোডে লেখা জ্ঞান থেকে যায়, হাতে করা জ্ঞান মানুষের সঙ্গে চলে যায়। অভিজ্ঞ DevOps প্রকৌশলীদের নিয়ম: **কোনো কাজ দ্বিতীয়বার করতে হলে তা Automation দিয়ে করতে হবে; আর প্রথমবার করার সময়ই ভেবে নিতে হবে দ্বিতীয়বার আসবে।**

২.৬ অভ্যন্তরীণ কার্যপ্রণালী

হাতে করা কাজ কীভাবে Drift তৈরি করে, তার যন্ত্রকৌশলটি বোঝা গুরুত্বপূর্ণ — কারণ Automation সেই যন্ত্রকৌশলটিই বদলে দেয়। হাতে করা কাজে পরিবর্তনের প্রবাহটি এরকম: **প্রকৌশলীর মাথা → টার্মিনালে টাইপ করা কমান্ড → সার্ভারের অবস্থা।** এই প্রবাহে কোনো মধ্যবর্তী সংরক্ষণ নেই — কমান্ডটি চালানোর সঙ্গে সঙ্গে তা হারিয়ে যায় (Shell History ছাড়া, যা অসম্পূর্ণ ও ব্যক্তিগত)। Automation-এ প্রবাহটি বদলে যায়: **কোড (Git) → পর্যালোচনা → টুল → সার্ভারের অবস্থা।** এখানে প্রতিটি ধাপে সংরক্ষণ ও পরীক্ষা আছে: কোডটি পড়া যায়, পর্যালোচনা করা যায়, Staging-এ চালিয়ে দেখা যায়, আর তারপরেই Production-এ যায়। আরও একটি পার্থক্য: হাতে করা কাজে পরিবর্তন একটি সার্ভারে ঘটে; Automation-এ পরিবর্তন একটি সংজ্ঞায় ঘটে, আর সেই সংজ্ঞা সব সার্ভারে প্রয়োগ হয়। এই কারণেই Automation-এ একটি ভুল সব সার্ভারে ছড়াতে পারে — এটাই তার ঝুঁকি, আর এ কারণেই পর্যালোচনা ও Staging-পরীক্ষা বাধ্যতামূলক (অধ্যায় ৫ ও ১৫-এ বিস্তারিত)।

> **WARNING:** Automation ভুলকে দ্রুত ছড়িয়ে দেয় — এটাই তার সবচেয়ে বড় ঝুঁকি। হাতে একটি সার্ভারে ভুল করলে ক্ষতি একটি সার্ভারে সীমাবদ্ধ থাকে; একটি ভুল Playbook ত্রিশটি সার্ভারে একসঙ্গে চালালে ক্ষতি ত্রিশ গুণ হয়। এই কারণে অভিজ্ঞ দলগুলো কখনো সরাসরি সব Production সার্ভারে নতুন Playbook চালায় না — প্রথমে একটি সার্ভারে (Canary), তারপর ধীরে ধীরে বাকিগুলোতে। Automation শক্তি দেয়, আর শক্তির সঙ্গে আসে পরীক্ষা ও সতর্কতার দায়িত্ব।

২.৭ আর্কিটেকচার চিত্র



চিত্রটির মূল বার্তা: হাতে করা কাজে জ্ঞান মানুষের মাথায় ছড়িয়ে থাকে; Automation-এ জ্ঞান একটি জায়গায় জমা হয় — Git Repository-তে।

২.৮ ব্যবহারিক উদাহরণ

উদাহরণ ১ — হাতে তিনটি সার্ভার কনফিগার করা (Drift-এর জন্ম): নিচের কমান্ডগুলো একজন প্রকৌশলী তিনটি সার্ভারে আলাদা আলাদা দিনে, আলাদা মনোযোগ দিয়ে চালিয়েছিলেন। প্রতিটি কমান্ড আলাদাভাবে সঠিক, কিন্তু সমষ্টিটি ভুল — কারণ কোনো একক সংজ্ঞা নেই।

```
# web-01-এ রাহাত চালিয়েছিলেন (সোমবার, সতর্কভাবে):
sudo apt update && sudo apt install -y nginx=1.24.0-2ubuntu7
sudo systemctl enable --now nginx
```

```
# web-02-এ নাহিদ চালিয়েছিলেন (বুধবার, তাড়াহুড়োয় — সংস্করণ নির্দিষ্ট করেননি):
sudo apt update && sudo apt install -y nginx
# ফল: তখনকার Latest সংস্করণ 1.18 ইনস্টল হলো — web-01-এর সঙ্গে মিলল না।

# web-03-এ সুমাইয়া চালিয়েছিলেন (শুক্রবার, প্যাচের দিন — কিন্তু nginx ইনস্টলই ভুলে গেলেন):
sudo apt update && sudo apt upgrade -y
# ফল: OS আপডেট হলো, কিন্তু Nginx ইনস্টল হলো না — সার্ভারটি Load Balancer-এ যোগ
# করার সময় ব্যর্থ হলো, আর কারণ খুঁজতে দুই ঘণ্টা লাগল।
```

উদাহরণ ২ — একই কাজ Automation-এর ভাষায় (ধারণাগত): একটি Playbook-এ একবার লেখা হয় "সব Web Server-এ Nginx 1.24 ইনস্টল থাকবে ও সার্ভিস চলবে" — তারপর একই Playbook তিনটি সার্ভারে চলে। কোনো সার্ভারে কী করা হয়েছিল, তা নিয়ে আর কারো মাথায় রাখতে হয় না।

উদাহরণ ৩ — খরচের হিসাব: TechNova-র হিসাবে, হাতে ৩০টি সার্ভার পরিচালনা করতে প্রতি মাসে প্রায় ৪০ ঘণ্টা লাগত — বেশিরভাগ সময় Drift খুঁজে বের করতে ও ঠিক করতে। Automation চালু করার পর একই কাজে লাগে ৪ ঘণ্টা — Playbook রক্ষণাবেক্ষণ ও পর্যালোচনায়। বাকি ৩৬ ঘণ্টা দলটি নতুন বৈশিষ্ট্য (Feature) তৈরিতে ব্যয় করে। এই হিসাবটি কাল্পনিক, কিন্তু অনুপাতটি বাস্তব: Drift খোঁজা ও ঠিক করাই হাতে পরিচালনার সবচেয়ে বড় লুকানো খরচ।

২.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচের ফাইলটি TechNova-র তিনটি Web Server-এর জন্য একটি Inventory — অর্থাৎ "কোন সার্ভারগুলো আছে" তার তালিকা। হাতে পরিচালনায় এই তালিকাটি কারো মাথায় বা একটি Spreadsheet-এ থাকত; Automation-এ এটি একটি ফাইল — যা Playbook পড়ে জানতে পারে কোথায় কাজ করতে হবে।

```
# inventory/hosts.yml — TechNova Web Server-এর তালিকা (YAML)
# এই ফাইলটি Automation-এর "মানচিত্র": কোন সার্ভার কোন দলে, তার ঠিকানা কী।
---
all:
  children:
    webservers:
      hosts:
        web-01:
          ansible_host: 203.0.113.11
          ansible_user: nahid
        web-02:
          ansible_host: 203.0.113.12
          ansible_user: nahid
        web-03:
          ansible_host: 203.0.113.13
          ansible_user: nahid
      vars:
        nginx_version: "1.24.0-2ubuntu7"
        http_port: 80
        https_port: 443
# এই একটি ফাইলই তিনটি সমস্যার সমাধান করে:
# ১. কোনো সার্ভার "ভুলে যাওয়া" যায় না — তালিকায় না থাকলে Playbook চলবেই না।
# ২. সব সার্ভারের সংস্করণ এক জায়গায় লেখা — আলাদা আলাদা সিদ্ধান্ত হয় না।
# ৩. নতুন সার্ভার যোগ করা মানে ফাইলে একটি লাইন যোগ করা — কোনো নতুন পদ্ধতি শেখা নয়।
```

২.১০ ধাপে ধাপে বাস্তবায়ন

অনুশীলন: আপনার পরিবেশের Drift মাপা (কোনো টুল ইনস্টল না করে)

1. দুটি সার্ভার বা দুটি Container বেছে নিন যেগুলো একই কাজ করার কথা (যেমন দুটি Web Server)।
2. দুটিতে `nginx -v` বা `dpkg -l | grep -E 'nginx|openssl'` চালিয়ে সংস্করণ লিখুন।
3. দুটিতে `systemctl is-enabled nginx` ও `systemctl is-active nginx` চালিয়ে সার্ভিসের অবস্থা লিখুন।
4. দুটিতে `sudo ufw status` বা `ss -tlnp` চালিয়ে খোলা Port-গুলো লিখুন।
5. তিনটি বিষয়ের (সংস্করণ, সার্ভিস, Port) যেকোনো পার্থক্য একটি Drift-তালিকায় লিখুন — প্রতিটি পার্থক্যের পাশে অনুমান করুন এটি কীভাবে তৈরি হয়েছিল।
6. এবার ভাবুন: এই পার্থক্যগুলো দূর করতে হাতে কত সময় লাগবে? আর একটি লিখিত সংজ্ঞা থাকলে কত সময় লাগত? পার্থক্যটিই Automation-এর মূল্য।

২.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

এই অনুশীলনের প্রত্যাশিত ফল একটি Drift-তালিকা — যেমন নিচের চিত্রণমূলক উদাহরণটি। অধিকাংশ শিক্ষার্থীই অবাক হন: "একইভাবে তৈরি" দুটি সার্ভারেও দুই-তিনটি পার্থক্য পাওয়া যায়। এই অবাক হওয়াটাই শিক্ষা — Drift দেখতে হলে মাপতে হয়, আর মাপলেই দেখা যায় হাতে পরিচালনায় Drift অনিবার্য।

```
# Drift-তালিকার উদাহরণ (চিত্রণমূলক আউটপুট):
# বিষয়      web-01 (203.0.113.11)   web-02 (203.0.113.12)   Drift?
# Nginx সংস্করণ    1.24.0-2ubuntu7          1.18.0-0ubuntu1          হ্যাঁ — আপডেট হয়নি
# nginx সার্ভিস  active + enabled         active, enabled নয়   হ্যাঁ — Reboot-এ বন্ধ হবে
# খোলা Port       80, 443                  80, 443, 8080          হ্যাঁ — 8080 পরীক্ষার Port
# OpenSSL          3.0.13 (প্যাচ করা)       3.0.2 (পুরনো)          হ্যাঁ — নিরাপত্তা-ঝুঁকি
# ব্যাখ্যা: চারটি লাইনে চারটি ভিন্ন কারণ — ভুলে যাওয়া আপডেট, ভুলে যাওয়া
# enable, ভুলে যাওয়া পরীক্ষার Port, ভুলে যাওয়া প্যাচ। সবগুলোর মূলে একটিই
# কারণ: কোনো লিখিত সংজ্ঞা ছিল না যা সব সার্ভারকে একই অবস্থায় রাখত।
```

২.১২ সাধারণ ভুল

- **ভুল ধারণা:** "আমাদের সার্ভার কম, তাই Drift হবে না।" — Drift সার্ভারের সংখ্যার ওপর নির্ভর করে না, সময়ের ওপর নির্ভর করে। দুটি সার্ভারও ছয় মাসে আলাদা হয়ে যায়।
- **ভুল:** Automation-কে "শুধু সময় বাঁচানোর টুল" ভাবা — আসল মূল্য সামঞ্জস্য, নিরীক্ষা ও পুনরুৎপাদনে; সময় বাঁচানো তার উপজাত মাত্র।
- **ভুল ধারণা:** "ভালো প্রকৌশলী হলে হাতেও ভুল হয় না।" — ভালো প্রকৌশলীও ক্লান্ত হন, ভুলে যান, ছুটিতে যান। পদ্ধতি মানুষের ওপর নির্ভর করবে না — এটাই Engineering।
- **ভুল:** আংশিক Automation — কিছু কাজ Playbook-এ, কিছু কাজ হাতে। এতে Drift আরও খারাপ হয়, কারণ এখন দুটি সত্যের উৎস আছে: কোড ও মানুষের হাত — আর তারা পরস্পরের সঙ্গে মেলে না।
- **ভুল:** Automation লিখে কিন্তু পরীক্ষা না করা — Staging-এ না চালিয়ে সরাসরি Production-এ চালানো। Automation ভুলকে দ্রুত ছড়ায়; পরীক্ষা সেই ছড়ানো আটকায়।
- **ভুল ধারণা:** "Pet Server খারাপ, সব সার্ভার Cattle হতে হবে।" — কিছু সার্ভার সত্যিই বিশেষ (যেমন একটি Legacy ডেটাবেস) — সেগুলোকে Pet হিসেবে যত্ন করা যেতে পারে, তবে সচেতনভাবে, ডকুমেন্টেশনসহ — অভ্যাসবশত নয়।

২.১৩ সমস্যা সমাধান (Troubleshooting)

1. **সমস্যা:** "দুটি সার্ভার একই, তবু একটি কাজ করে, আরেকটি করে না।" — **পরীক্ষা:** সংস্করণ (`nginx -v`), কনফিগারেশন ফাইলের পার্থক্য (`diff`), সার্ভিসের অবস্থা (`systemctl status`) ও খোলা Port (`ss -tlnp`) — এই চারটি তুলনা করুন। **সমাধান:** পার্থক্যটি খুঁজে পেলেই Drift-এর কারণ পাওয়া যায়; স্থায়ী সমাধান হলো একটি Desired State লেখা।
2. **সমস্যা:** "Reboot-এর পর সার্ভিস আর চালু হয় না।" — **পরীক্ষা:** `systemctl is-enabled nginx` — সম্ভবত সার্ভিসটি `start` করা হয়েছিল, কিন্তু `enable` করা হয়নি। **সমাধান:** হাতে করা কাজে এই ভুলটি খুব সাধারণ; Automation-এ `enabled: true` লিখলে দুটি অবস্থাই একসঙ্গে নিশ্চিত হয়।
3. **সমস্যা:** "কে এই Port-টি খুলেছিল, কেউ জানে না।" — **পরীক্ষা:** Shell History, `/var/log/auth.log`-এ কে কখন SSH করেছিলেন, আর ফায়ারওয়াল-নিয়মের সময় দেখুন। **সমাধান:** তদন্তের পর ফায়ারওয়াল-নিয়মগুলোকে কোডে লিখুন — ভবিষ্যতে "কে খুলেছিল" প্রশ্নের উত্তর Git History দেবে।
4. **সমস্যা:** "Automation-এর জন্য সময় নেই, এখনই Production ঠিক করতে হবে।" — **পরীক্ষা:** জরুরি অবস্থায় হাতে ঠিক করা যেতেই পারে — কিন্তু ঠিক করার পরপরই পরিবর্তনটি কোডে লিখে ফেলুন। **সমাধান:** নিয়ম করুন: "হাতে ঠিক করা যাবে, কিন্তু ২৪ ঘণ্টার মধ্যে কোডে লিখতে হবে" — নইলে জরুরি ঠিক করাটিই পরের Drift-এর বীজ হয়।

২.১৪ নিরাপত্তা বিবেচনা

> **WARNING:** হাতে পরিচালিত পরিবেশে নিরাপত্তার সবচেয়ে বড় শত্রু হলো অদৃশ্যতা: কোন সার্ভারে কোন প্যাচ লাগানো হয়েছে, কোন Port খোলা আছে, কোন ব্যবহারকারী-অ্যাকাউন্ট এখনো সক্রিয় — এই প্রশ্নগুলোর উত্তর না জানা মানেই আক্রমণকারীর জন্য সুযোগ। একটি ভুলে যাওয়া সার্ভার, একটি পুরনো OpenSSL, একটি "সাময়িক" Port — এগুলো দিয়েই বড় আক্রমণ শুরু হয়। Automation নিরাপত্তা দেয় এই অর্থে যে Desired State-এ নিরাপত্তা-নীতিগুলো লেখা থাকে (কোন Port খোলা, কোন প্যাকেজের ন্যূনতম সংস্করণ) এবং নিয়মিত Convergence সেগুলো ফিরিয়ে আনে।

তবে Automation-এর নিজেরও নিরাপত্তা-ঝুঁকি আছে, যা এই অধ্যায়েই মনে রাখতে হবে: Automation টুলের হাতে সব সার্ভারের Root/SSH ক্ষমতা থাকে, তাই Control Node-এর নিরাপত্তা পুরো অবকাঠামোর নিরাপত্তা। Control Node-এ SSH Private Key রাখা হয় — সেই Key চুরি গেলে আক্রমণকারী সব সার্ভারে ঢুকতে পারে। তাই Control Node-এ শক্তিশালী প্রমাণীকরণ (MFA যেখানে সম্ভব), Key-এর Passphrase, আর নিয়মিত Key Rotation বাধ্যতামূলক। আর একটি নীতি: Automation-এর জন্য আলাদা, সীমিত-ক্ষমতার ব্যবহারকারী (যেমন `ansible` ব্যবহারকারী, শুধু প্রয়োজনীয় sudo অধিকারসহ) ব্যবহার করুন — Root-এর সরাসরি SSH নয়। বিস্তারিত অধ্যায় ৫ ও ১৪-এ আসবে।

২.১৫ প্রোডাকশন ইঞ্জিনিয়ারিং অনুশীলন

সেরা অনুশীলন: (১) "Code First, Hands Never" — Production-এ কোনো পরিবর্তন হাতে নয়, সব কোড দিয়ে; জরুরি ঠিক করার পর ২৪ ঘণ্টার মধ্যে কোডে লিখুন; (২) প্রতিটি সার্ভারকে Cattle হিসেবে দেখুন — কোনো সার্ভার যেন "অপূরণীয়" না হয়; নিয়মিত একটি সার্ভার মুছে Playbook দিয়ে আবার তৈরি করে পরীক্ষা করুন (একে বলে **Chaos-পরীক্ষা** বা Rebuild Drill); (৩) Inventory ফাইলটি সবসময় হালনাগাদ রাখুন — তালিকায় নেই এমন সার্ভার মানে অদৃশ্য Drift; (৪) ছোট থেকে শুরু করুন — প্রথমে একটি ছোট কাজ Automation করুন (যেমন ফায়ারওয়াল-নিয়ম), সফল হলে পরেরটি; (৫) Automation-এর পরিবর্তনেও পর্যালোচনা (Review) বাধ্যতামূলক করুন — একটি ভুল Playbook ত্রিশটি সার্ভার নষ্ট করতে পারে। **অ্যান্টি-প্যাটার্ন:** "Big Bang Automation" — একদিনে সবকিছু Automation করার চেষ্টা;

"Shadow Manual Work" — Playbook আছে, তবু সবাই হাতে কাজ করে; "Untested Playbook in Production" — পরীক্ষা ছাড়া Production-এ চালানো।

২.১৬ অধ্যায়ের সারাংশ

হাতে সার্ভার পরিচালনা করা মানে জ্ঞানকে মানুষের মাথায় রাখা, ভুলকে অদৃশ্য রাখা, আর Drift-কে অনিবার্য করা। TechNova-র তিনটি সার্ভারের গল্প দেখায় — কেউ খারাপ কাজ না করেও তিনটি সার্ভার তিন রকম হয়ে যায়, কারণ হাতে করা কাজে কোনো একক সংজ্ঞা নেই। Automation সেই সংজ্ঞাটি দেয়: Desired State কোডে লেখা হয়, Git-এ সংরক্ষিত হয়, আর একই Playbook সব সার্ভারে একই ফল দেয়। Pets বনাম Cattle উপমাটি মনে রাখুন: লক্ষ্য হলো এমন সার্ভার তৈরি করা যা যেকোনো সময় মুছে আবার তৈরি করা যায় — আর তা সম্ভব হয় শুধু Automation দিয়ে। তবে মনে রাখবেন: Automation ভুলকেও দ্রুত ছড়ায়, তাই পরীক্ষা ও পর্যালোচনা তার অবিচ্ছেদ্য অংশ।

২.১৭ জ্ঞান যাচাই — প্রশ্ন

1. Snowflake Server বলতে কী বোঝায়? TechNova-র তিনটি সার্ভারের উদাহরণ দিয়ে ব্যাখ্যা করুন।
2. Pets বনাম Cattle উপমাটি ব্যাখ্যা করুন — কোন মডেলে সার্ভার নষ্ট হলে কী করা হয়?
3. হাতে করা প্রশাসনের চারটি খরচ (সময়, মানবিক ভুল, নিরীক্ষার অভাব, পুনরুৎপাদনের অসম্ভবতা) নিজের ভাষায় ব্যাখ্যা করুন।
4. "কাজটি একবারই করতে হবে, তাই Automation লেখার দরকার নেই" — এই যুক্তিটি কেন ভুল?
5. Automation কীভাবে নিরাপত্তা বাড়ায়, আবার কীভাবে নতুন ঝুঁকি তৈরি করে — দুটি দিকই ব্যাখ্যা করুন।

২.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. **Drift নিরীক্ষা:** ২.১০-এর ধাপগুলো অনুসরণ করে দুটি সার্ভার/Container-এর Drift-তালিকা তৈরি করুন ও প্রতিটি পার্থক্যের সম্ভাব্য কারণ লিখুন।
2. **Inventory লেখা:** আপনার নিজের পরিবেশের (ল্যাপটপ, পরীক্ষামূলক সার্ভার, Container) জন্য একটি Inventory YAML লিখুন — প্রতিটি হোস্টের নাম, IP ও ভূমিকাসহ।
3. **খরচের হিসাব:** আপনার পরিবেশে একটি পুনরাবৃত্ত কাজ (যেমন নতুন সার্ভারে Nginx বসানো) হাতে করতে কত সময় লাগে, তার হিসাব করুন; আর Automation লিখতে কত সময় লাগত, তার অনুমান করুন — কতবার পুনরাবৃত্তি হলে Automation লাভজনক হবে, তা বের করুন।

অধ্যায় 3 • PART I — Configuration Management Foundations

Chapter 3 — Declarative Configuration, Idempotency, and Desired State

৩.১ ভূমিকা

দুটি নির্দেশের কথা ভাবুন। প্রথমটি: "দরজাটি বন্ধ করো।" দ্বিতীয়টি: "দরজাটি বন্ধ আছে তা নিশ্চিত করো।" প্রথম নির্দেশটি একটি কাজ — আপনি দরজা বন্ধ করুন, তা আগে থেকে বন্ধ থাকলেও আবার বন্ধ করার চেষ্টা করুন। দ্বিতীয় নির্দেশটি একটি অবস্থা — দরজা বন্ধ থাকলে কিছু করার দরকার নেই, খোলা থাকলে বন্ধ করুন। এই ছোট পার্থক্যটিই Configuration Management-এর হৃদয়: **Declarative** (ঘোষণামূলক) বনাম **Imperative** (নির্দেশমূলক)। আর দ্বিতীয় নির্দেশের একটি বিশেষ গুণ আছে: তা দুবার দিলেও ফল একই থাকে — দরজা বন্ধই থাকে। এই গুণের নাম **Idempotency** — একই কাজ বারবার করলে ফলাফল একই থাকে। এই অধ্যায়ে আমরা এই দুটি ধারণাকে গভীরভাবে শিখব, সঠিক ও ভুল Automation-এর উদাহরণসহ — কারণ Ansible-এর প্রতিটি Module এই দুটি নীতির ওপর দাঁড়িয়ে আছে।

৩.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Declarative ও Imperative পদ্ধতির পার্থক্য উদাহরণসহ ব্যাখ্যা করতে; (২) Idempotency-র সংজ্ঞা দিতে ও গাণিতিকভাবে বোঝাতে (`f(f(x)) = f(x)`); (৩) সঠিক (Idempotent) ও ভুল (Non-Idempotent) Automation-এর উদাহরণ আলাদা করতে — যেমন `lineinfile` বনাম `echo >>`, `apt` Module বনাম `shell: apt-get install`; (৪) Convergence ও Drift Detection কীভাবে Idempotency-র ওপর নির্ভর করে তা ব্যাখ্যা করতে; (৫) একটি Shell Script-কে বিশ্লেষণ করে বলতে তার কোন অংশ Idempotent আর কোন অংশ নয়; (৬) নিজের Playbook-এ Idempotency যাচাই করার পদ্ধতি (`--check` ও দ্বিতীয়বার চালানো) প্রয়োগ করতে।

৩.৩ পূর্বশর্ত ধারণা

অধ্যায় ১ ও ২-এর Desired State, Drift ও Convergence-এর ধারণা; Shell-এ `echo >> file`, `apt-get install`, `mkdir` চালানোর অভিজ্ঞতা; YAML-এর প্রাথমিক ধারণা (Key-Value, তালিকা) — অধ্যায় ১-এর ১.৯ পড়ে থাকলেই চলবে। গণিতের গভীর জ্ঞান প্রয়োজন নেই — `f(f(x)) = f(x)` সূত্রটি একটি উদাহরণ দিয়ে বোঝানো হবে।

৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

Imperative পদ্ধতি বলে কীভাবে করতে হবে — ধাপে ধাপে নির্দেশ: "প্রথমে এই ফাইলটি খোলো, তারপর এই লাইনটি শেষে যোগ করো, তারপর ফাইলটি সংরক্ষণ করো।" **Declarative** পদ্ধতি বলে কী হতে হবে — শেষ অবস্থার বর্ণনা: "এই ফাইলে এই লাইনটি থাকবে।" পার্থক্যটি শুধু ভাষার নয়, দায়িত্বের: Imperative-এ আপনি সিদ্ধান্ত নেন প্রতিটি ধাপ; Declarative-এ টুলটি সিদ্ধান্ত নেয় কীভাবে সেখানে পৌঁছাতে হবে। টুলটি প্রথমে বর্তমান অবস্থা দেখে, তারপর শুধু প্রয়োজনীয় পদক্ষেপ নেয়। এই কারণে Declarative পদ্ধতি স্বাভাবিকভাবেই Idempotent হওয়ার দিকে ঝোঁকে: টুলটি যদি দেখে কাঙ্ক্ষিত অবস্থা ইতিমধ্যে আছে, তাহলে কিছুই করে না।

Idempotency শব্দটি গণিত থেকে এসেছে। একটি ফাংশন `f` Idempotent যদি `f(f(x)) = f(x)` হয় — অর্থাৎ ফাংশনটি একবার প্রয়োগ করার পর আবার প্রয়োগ করলে ফলাফল বদলায় না। দরজার উদাহরণে: "দরজা বন্ধ আছে নিশ্চিত করো" — একবার করলে দরজা বন্ধ; আবার করলে দরজা বন্ধই থাকে। বিপরীতে "দরজাটিকে একটু ঠেলো" — প্রতিবার ঠেললে দরজা আরও সরে যায় — এটি Non-Idempotent। সার্ভারের জগতে: "Nginx ইনস্টল আছে নিশ্চিত করো"

Idempotent — ইনস্টল থাকলে কিছু হয় না; "Nginx ইনস্টল করো" কমান্ডটি Non-Idempotent হতে পারে — কারণ দ্বিতীয়বার চালালে তা আবার ডাউনলোড, আবার ইনস্টল, আবার Restart করার চেষ্টা করতে পারে।

> **CONCEPT:** Idempotency মানে "একবার চালানো আর দশবার চালানো একই কথা"। কেন এটি এত গুরুত্বপূর্ণ? কারণ বাস্তব জীবনে Automation বারবার চলে — কেউ ভুল করে দুবার চালায়, CI Pipeline আবার চালায়, একটি সার্ভার অর্ধেক অবস্থায় থেমে গেলে আবার চালাতে হয়। Automation যদি Idempotent না হয়, তাহলে প্রতিটি পুনরায় চালানো সার্ভারকে আরও খারাপ অবস্থায় নিয়ে যায়। Idempotent Automation নিরাপদে আবার চালানো যায় — আর এই নিরাপত্তাই Configuration Management-কে Script-এর চেয়ে উন্নত করে।

এখানে একটি সূক্ষ্ম পার্থক্য মনে রাখুন: **Declarative হলেই Idempotent হয় না, আর Imperative হলেই Non-Idempotent হয় না।** একটি Declarative বর্ণনা ভুলভাবে বাস্তবায়িত হলে Non-Idempotent হতে পারে; আবার একটি Imperative Script সতর্কভাবে লিখলে Idempotent হতে পারে (যেমন "ফাইলে লাইনটি না থাকলে তবেই যোগ করো")। কিন্তু Declarative টুলগুলো (Ansible, Puppet, Terraform) Idempotency-কে তাদের Module-এর ভেতরে বানিয়ে দেয় — আপনাকে প্রতিবার নিজে ভাবতে হয় না। এটাই তাদের মূল্য।

৩.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Declarative বনাম Imperative — পাশাপাশি

বিষয়	Imperative (নির্দেশমূলক)	Declarative (ঘোষণামূলক)
প্রশ্ন	"কীভাবে করব?" (How)	"কী হতে হবে?" (What)
উদাহরণ	`apt-get install nginx; systemctl start nginx`	"Nginx ইনস্টল থাকবে ও চলবে"
বর্তমান অবস্থা দেখে কি?	সাধারণত না — অন্ধভাবে কমান্ড চালায়	হ্যাঁ — প্রথমে দেখে, তারপর সিদ্ধান্ত নেয়
দুবার চালালে	প্রায়ই ভিন্ন ফল (দ্বিগুণ লাইন, আবার Restart)	একই ফল (দ্বিতীয়বার কিছুই বদলায় না)
ভুল হলে	মাঝপথে থেমে গেলে সার্ভার অর্ধেক অবস্থায় থাকে	আবার চালালে বাকি কাজ শেষ হয় (Convergence)
প্রতিনিধি	Shell Script, Bash	Ansible Playbook, Terraform, Puppet Manifest

Idempotency-র তিনটি স্তর

স্তর ১ — সম্পূর্ণ Idempotent: যেকোনো অবস্থা থেকে চালালে শেষ অবস্থা একই হয়, আর দ্বিতীয়বার চালালে কোনো পরিবর্তনই হয় না (Ansible-এর `changed=0`)। উদাহরণ: `apt` Module দিয়ে Nginx ইনস্টল করা — ইনস্টল থাকলে Module কিছু করে না। **স্তর ২ — আংশিক Idempotent:** শেষ অবস্থা একই হয়, কিন্তু প্রতিবার কিছু পার্শ্ব-প্রতিক্রিয়া (Side Effect) ঘটে — যেমন প্রতিবার সার্ভিস Restart হয়, যদিও কনফিগারেশন বদলায়নি। এটি ক্ষতিকর — কারণ অপ্রয়োজনীয় Restart মানে অপ্রয়োজনীয় বিভ্রাট। **স্তর ৩ — Non-Idempotent:** প্রতিবার চালালে অবস্থা বদলে যায় — যেমন `echo "x" >> file` প্রতিবার একটি করে লাইন যোগ করে। এই স্তরের Automation চালানোই উচিত নয়।

সঠিক বনাম ভুল — চারটি জোড়া উদাহরণ

জোড়া ১ — ফাইলে লাইন যোগ করা: ভুল: `shell: echo "server_name technova.example;" >> /etc/nginx/nginx.conf` — প্রতিবার চালালে লাইনটি আবার যোগ হয়; দশবার চালালে দশটি একই লাইন, আর Nginx কনফিগারেশন ভেঙে যেতে পারে। সঠিক: `lineinfile` Module — এটি প্রথমে দেখে লাইনটি আছে কি না; না থাকলে যোগ করে, থাকলে কিছু করে না। দ্বিতীয়বার চালালে `changed=0`।

জোড়া ২ — প্যাকেজ ইনস্টল করা: ভুল: `shell: apt-get install -y nginx` — প্রতিবার চালালে apt আবার চলে, কখনো কখনো আবার ডাউনলোড করে, আর Playbook-এর প্রতিবেদনে প্রতিবার "changed" দেখায় — ফলে সত্যিকারের পরিবর্তন আর চোখে পড়ে না। সঠিক: `apt` Module (`name: nginx, state: present`) — ইনস্টল থাকলে কিছু করে না, না থাকলে ইনস্টল করে।

জোড়া ৩ — ডিরেক্টরি তৈরি করা: ভুল: `shell: mkdir /var/www/technova` — দ্বিতীয়বার চালালে ত্রুটি হয় ("File exists") — Playbook ব্যর্থ হয়। সঠিক: `file` Module (`path: /var/www/technova, state: directory`) — থাকলে কিছু করে না, না থাকলে তৈরি করে। (Shell-এ `mkdir -p` Idempotent, কিন্তু Module ব্যবহার করাই উত্তম — কারণ Module অনুমতি ও মালিকানাও নিশ্চিত করে।)

জোড়া ৪ — সার্ভিস চালানো: ভুল: `shell: systemctl start nginx` — সার্ভিস বন্ধ থাকলে চালু হয়, কিন্তু Boot-এ চালু হবে কি না তা নিশ্চিত করে না; আর প্রতিবার "changed" দেখায়। সঠিক: `service` Module (`name: nginx, state: started, enabled: true`) — চলছে ও Boot-এ চালু হবে — দুটিই নিশ্চিত করে, আর ইতিমধ্যে ঠিক থাকলে কিছু করে না।

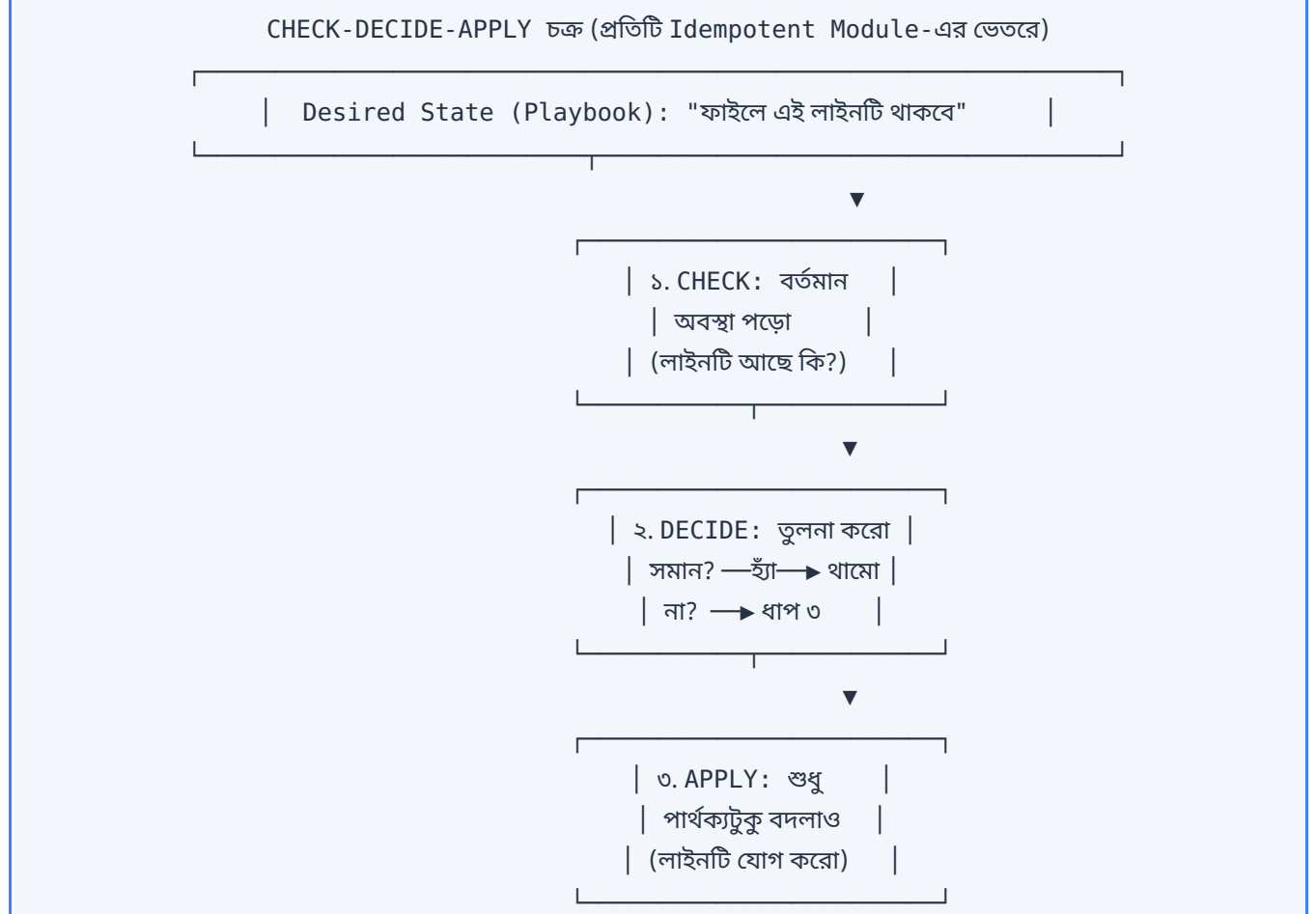
৩.৬ অভ্যন্তরীণ কার্যপ্রণালী

একটি Idempotent Module ভেতরে ভেতরে কী করে? তিনটি ধাপে। **ধাপ ১ — পর্যবেক্ষণ (Check):** Module-টি লক্ষ্য সার্ভারে বর্তমান অবস্থা পড়ে — ফাইলটি আছে কি না, লাইনটি আছে কি না, প্যাকেজটি ইনস্টল আছে কি না, সার্ভিসটি চলছে কি না। এই ধাপে কোনো পরিবর্তন হয় না। **ধাপ ২ — সিদ্ধান্ত (Decide):** বর্তমান অবস্থা ও কাঙ্ক্ষিত অবস্থা মিলিয়ে দেখা হয়। সমান হলে Module থেমে যায় ও জানায় `changed=false` — "কিছু করার ছিল না।" আলাদা হলে Module ঠিক করে কী করতে হবে — শুধু পার্থক্যটুকু। **ধাপ ৩ — প্রয়োগ (Apply):** শুধু প্রয়োজনীয় পরিবর্তনটুকু করা হয় — ফাইলে লাইন যোগ করা, প্যাকেজ ইনস্টল করা, সার্ভিস চালু করা — আর ফলাফল জানানো হয়। এই তিন ধাপের নাম একসঙ্গে **Check-Decide-Apply** চক্র।

এই চক্রটিই Drift Detection-এর ভিত্তি। Ansible-এর `--check` (Dry Run) Mode শুধু প্রথম দুটি ধাপ চালায় — কী বদলাবে তা দেখায়, কিন্তু কিছু বদলায় না। আর Playbook দ্বিতীয়বার চালালে সব Module-ই প্রথম ধাপে দেখে সবকিছু ঠিক আছে, তাই কিছুই বদলায় না — এই ঘটনাটিকে বলে **Convergence-এর প্রমাণ**: দ্বিতীয়বার চালানোতে `changed=0` মানেই সার্ভার Desired State-এ পৌঁছেছে।

> **TIP:** যেকোনো Playbook লেখার পর দুটি পরীক্ষা করুন: (১) `ansible-playbook site.yml --check` দিয়ে দেখুন কী বদলাবে — তালিকাটি আপনার প্রত্যাশার সঙ্গে মেলে কি না; (২) Playbook-টি দুবার চালান — দ্বিতীয়বার সব Task-এ `changed=0` (অথবা `ok`) আসা উচিত। দ্বিতীয়বারেও কিছু Task `changed` দেখালে সেই Task-টি Non-Idempotent — তার কারণ খুঁজে বের করুন। এই দুটি পরীক্ষাই Idempotency যাচাইয়ের সোনালি নিয়ম।

৩.৭ আর্কিটেকচার চিত্র



মনে রাখবেন: Check-Decide-Apply চক্রের প্রথম ধাপ (Check) ছাড়া কোনো Module-ই Idempotent হতে পারে না — অন্ধভাবে কমান্ড চালানো মানেই Non-Idempotent হওয়ার ঝুঁকি।

৩.৮ ব্যবহারিক উদাহরণ

উদাহরণ ১ — দরজা ও আলোর সুইচ: "আলো জ্বালাও" Imperative — আলো জ্বললেও আবার সুইচ টিপতে হয়। "আলো জ্বলা অবস্থায় থাকবে" Declarative — জ্বললে কিছু করতে হয় না। Idempotency-র দৈনন্দিন উদাহরণ: লিফটের বোতাম — একবার টিপলেও যা হয়, দশবার টিপলেও তাই হয় (লিফট একবারই আসে)। বিপরীতে, একটি ভেন্ডিং মেশিনে টাকা ঢোকানো Non-Idempotent — প্রতিবার টাকা ঢোকালে ব্যালান্স বাড়ে।

উদাহরণ ২ — TechNova-র Nginx কনফিগারেশন: ধরুন `/etc/nginx/sites-available/technova.example`-এ `server_name technova.example;` লাইনটি থাকতে হবে। একজন প্রকৌশলী Shell Script-এ লিখলেন `echo "server_name technova.example;" >> ফাইল`। Script-টি CI Pipeline-এ প্রতিটি Deploy-এ চলে — এক মাসে কুড়িবার। ফাইলে এখন কুড়িটি একই লাইন। Nginx শেষ লাইনটি ব্যবহার করে বলে সাইটটি চলছে, কিন্তু ফাইলটি পড়া অসম্ভব, আর একদিন কেউ ফাইলটি সম্পাদনা করতে গিয়ে ভুল লাইন মুছে ফেললেন। `lineinfile` ব্যবহার করলে এই সমস্যাটি কখনোই তৈরি হতো না।

```
# ভুল (NON-IDEMPOTENT) — কখনো Playbook-এ এভাবে লিখবেন না:
- name: Add server_name line (WRONG)
  shell: echo "server_name technova.example;" >> /etc/nginx/sites-available/technova.example
# প্রতিবার চালালে লাইনটি আবার যোগ হয়। ৫ বার চালালে ৫টি লাইন।

# সঠিক (IDEMPOTENT) — Module ব্যবহার করুন:
- name: Add server_name line (CORRECT)
  lineinfile:
    path: /etc/nginx/sites-available/technova.example
    line: "server_name technova.example;"
    state: present
# প্রথমবার: লাইন যোগ হয় (changed=1)। দ্বিতীয়বার: কিছুই হয় না (changed=0)।
```

উদাহরণ ৩ — একটি Script-কে Idempotent করা: ধরুন আপনাকে বাধ্য হয়ে Shell ব্যবহার করতে হচ্ছে (যেমন একটি বিশেষ কমান্ডের কোনো Module নেই)। তখন Script-কে Idempotent করার কৌশল হলো `creates` বা শর্ত (Condition) ব্যবহার করা — যেমন "ফাইলটি না থাকলে তবেই কমান্ড চালাও"। Ansible-এ `shell` Module-এর `creates` প্যারামিটার এই কাজটি করে। তবে মনে রাখবেন: এটি জোড়াতালি — প্রথম পছন্দ সবসময় একটি সত্যিকারের Module।

৩.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচের Playbook-টিতে একই কাজের সঠিক ও ভুল সংস্করণ পাশাপাশি দেখানো হয়েছে — শিক্ষার জন্য। বাস্তব কাজে শুধু সঠিক সংস্করণটি ব্যবহার করবেন। এই ফাইলটি অধ্যায় ৫-এর পরিবেশে (Control Node থেকে) চালানো যাবে।

```
# idempotency-demo.yml — সঠিক বনাম ভুল Automation (শিক্ষামূলক তুলনা)
---
- name: Idempotency demonstration on TechNova web servers
  hosts: webservers
  become: true
  tasks:
    # --- সঠিক (IDEMPOTENT) Task-গুলো ---
    - name: CORRECT - Ensure Nginx is installed
      apt:
        name: nginx
        state: present
        update_cache: true

    - name: CORRECT - Ensure server_name line exists exactly once
```

```
      lineinfile:
        path: /etc/nginx/sites-available/technova.example
        line: "server_name technova.example;"
        state: present
        create: true
        mode: "0644"

    - name: CORRECT - Ensure web directory exists
      file:
        path: /var/www/technova
        state: directory
        owner: www-data
        group: www-data
        mode: "0755"

    - name: CORRECT - Ensure Nginx is running and enabled at boot
      service:
        name: nginx
        state: started
        enabled: true

    # --- ভুল (NON-IDEMPOTENT) Task-গুলো — শুধু দেখার জন্য, চালাবেন না! ---
    # - name: WRONG - Appends the line on every run
    #   shell: echo "server_name technova.example;" >> /etc/nginx/sites-available/
technova.example
    #
    # - name: WRONG - Reinstalls and reports changed on every run
    #   shell: apt-get install -y nginx
    #
    # - name: WRONG - Fails on second run because directory exists
    #   shell: mkdir /var/www/technova
```

৩.১০ ধাপে ধাপে বাস্তবায়ন

অনুশীলন: Idempotency নিজে পরীক্ষা করা (অধ্যায় ৫-এর পরিবেশ তৈরি হওয়ার পর)

1. ৩.৯-এর Playbook-এর শুধু সঠিক অংশটি একটি ফাইলে সংরক্ষণ করুন (`idempotency-demo.yml`)।
2. প্রথমবার চালান: `ansible-playbook idempotency-demo.yml` — লক্ষ্য করুন কোন Task-গুলো `changed` দেখায়।
3. কোনো কিছু না বদলে দ্বিতীয়বার একই কমান্ড চালান — এবার সব Task-এ `ok` (changed=0) আসা উচিত। এটাই Convergence-এর প্রমাণ।
4. এবার একটি সার্ভারে হাতে Drift তৈরি করুন — যেমন `sudo apt remove nginx` বা ফাইল থেকে লাইনটি মুছে ফেলুন — তারপর Playbook আবার চালান। দেখুন শুধু সেই Task-টিই `changed` দেখায়, বাকিগুলো `ok` থাকে। এটাই Drift Detection ও সংশোধন (Remediation)।
5. এবার ভুল সংস্করণের একটি Task (যেমন `echo >>`) আলাদা Playbook-এ লিখে দুবার চালান ও ফাইলে লাইনটি কয়বার যোগ হয়েছে তা গুনুন। পার্থক্যটি নিজের চোখে দেখুন।
6. সবশেষে `--check` Mode-এ চালান: `ansible-playbook idempotency-demo.yml --check` — দেখুন এটি কী বদলাবে তা জানায়, কিন্তু কিছু বদলায় না।

৩.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

সঠিক Playbook-এর প্রথম চালানোতে কিছু Task `changed` দেখাবে (যা যা করতে হয়েছে), দ্বিতীয় চালানোতে সব Task `ok` দেখাবে। নিচের চিত্রণমূলক আউটপুটটি এই পার্থক্য দেখায় — লক্ষ্য করুন `PLAY RECAP`-এ `changed`-এর সংখ্যা কীভাবে শূন্য হয়ে যায়।

```
# প্রথমবার চালানো (চিত্রণমূলক আউটপুট):
# TASK [CORRECT - Ensure Nginx is installed] ****
# changed: [web-01]        <- Nginx ছিল না, ইনস্টল হলো
# TASK [CORRECT - Ensure server_name line exists exactly once] ****
# changed: [web-01]        <- লাইন ছিল না, যোগ হলো
# PLAY RECAP ****
# web-01 : ok=4  changed=4  unreachable=0  failed=0
#
# দ্বিতীয়বার চালানো — কিছুই না বদলে (চিত্রণমূলক আউটপুট):
# TASK [CORRECT - Ensure Nginx is installed] ****
# ok: [web-01]             <- ইনস্টল আছে, কিছু করা হলো না
# TASK [CORRECT - Ensure server_name line exists exactly once] ****
# ok: [web-01]             <- লাইন আছে, কিছু করা হলো না
# PLAY RECAP ****
# web-01 : ok=4  changed=0  unreachable=0  failed=0
# ব্যাখ্যা: দ্বিতীয়বার changed=0 মানেই সার্ভার Desired State-এ আছে।
# ভুল সংস্করণে দ্বিতীয়বারেও changed দেখাত, আর ফাইলে লাইন বাড়ত।
```

৩.১২ সাধারণ ভুল

- **ভুল:** Playbook-এ `shell` বা `command` Module দিয়ে এমন কাজ করা যার জন্য একটি সত্যিকারের Module আছে (যেমন `apt`, `lineinfile`, `file`, `service`)। এটি Non-Idempotency-র সবচেয়ে সাধারণ কারণ। নিয়ম: **প্রথমে Module খুঁজুন, Shell শেষ উপায়।**
- **ভুল ধারণা:** "Playbook একবার চললেই কাজ শেষ।" — না; Playbook-কে বারবার চালানো যায় — এটাই তার উদ্দেশ্য। একবার চালিয়ে ফেলে রাখা মানে Drift-কে আবার বাড়তে দেওয়া।
- **ভুল:** `changed` প্রতিবেদনকে উপেক্ষা করা — প্রতিবার `changed` দেখা মানে হয় Drift ছিল (ভালো), নয় Task-টি Non-Idempotent (খারাপ)। কোনটি তা যাচাই না করলে আসল Drift চোখে পড়ে না।
- **ভুল:** `shell: systemctl restart nginx` প্রতিটি Run-এ চালানো — Restart একটি Side Effect; শুধু কনফিগারেশন বদলালেই Restart হওয়া উচিত (Ansible-এ এর সমাধান হলো Handler — পরের Part-এ বিস্তারিত)।
- **ভুল ধারণা:** "Declarative মানেই স্বয়ংক্রিয়ভাবে Idempotent।" — না; ভুলভাবে লেখা Declarative কোডও Non-Idempotent হতে পারে (যেমন `copy` Module-এ প্রতিবার বদলে যাওয়া Content)। Idempotency যাচাই করতে হয় — দ্বিতীয়বার চালিয়ে।
- **ভুল:** Timestamp বা Random মান কনফিগারেশন ফাইলে লেখা — প্রতিবার চালালে ফাইল বদলে যায়, তাই প্রতিবার `changed` দেখায় ও সার্ভিস Restart হয়। কনফিগারেশনে পরিবর্তনশীল (Volatile) মান রাখবেন না।

৩.১৩ সমস্যা সমাধান (Troubleshooting)

1. **সমস্যা:** "Playbook দ্বিতীয়বার চালালেও কিছু Task `changed` দেখায়।" — **পরীক্ষা:** কোন Task-টি তা দেখুন; সম্ভবত সেটি `shell`/`command` ব্যবহার করছে, বা ফাইলে Timestamp/Random মান লিখছে। **সমাধান:** সেই Task-কে একটি সত্যিকারের Module দিয়ে বদলান, বা `creates`/`removes` শর্ত যোগ করুন।

2. **সমস্যা:** "`lineinfile` ব্যবহার করেও ফাইলে লাইন দুবার আছে।" — **পরীক্ষা:** লাইনটি আগে `echo >>` দিয়ে যোগ করা হয়েছিল কি না — `lineinfile` একটি লাইন নিশ্চিত করে, কিন্তু আগে থেকে থাকা ডুপ্লিকেট সরায় না। **সমাধান:** প্রথমে ডুপ্লিকেট লাইনগুলো হাতে সরিয়ে ফেলুন, তারপর `lineinfile` ব্যবহার করুন; ভবিষ্যতে আর `echo >>` ব্যবহার করবেন না।
3. **সমস্যা:** "Playbook চালালাম, কিন্তু সার্ভার বদলাল না — সব `ok`।" — **পরীক্ষা:** `--check` Mode-এ চালিয়ে দেখুন কী বদলানোর কথা; হয়তো Desired State-টিই ভুল লেখা হয়েছে (যেমন ভুল ফাইলের পথ)। **সমাধান:** Module-এর প্যারামিটার (Path, নাম, সংস্করণ) যাচাই করুন — Idempotent মানে "কিছু করার নেই", কিন্তু ভুল Desired State-ও "কিছু করার নেই" দেখায়।
4. **সমস্যা:** "একটি Task কখনো `changed`, কখনো `ok` দেখায় — অনিয়মিত।" — **পরীক্ষা:** Task-টি কি বাইরের কোনো অবস্থার ওপর নির্ভরশীল (যেমন নেটওয়ার্ক থেকে ডাউনলোড, বা সময়)? **সমাধান:** বাইরের নির্ভরতা কমান; ডাউনলোড করা ফাইলের Checksum যাচাই করুন যাতে একই ফাইল আবার না নামে।

৩.১৪ নিরাপত্তা বিবেচনা

> **WARNING:** Non-Idempotent Automation একটি নিরাপত্তা-ঝুঁকিও বটে। ধরুন একটি Playbook প্রতিবার চালালে ফায়ারওয়াল-নিয়ম আবার যোগ করে — একসময় নিয়মের তালিকা এত বড় হয় যে কেউ আর বুঝতে পারে না কোন নিয়মটি আসল, কোনটি ডুপ্লিকেট। বা ধরুন একটি Script প্রতিবার একটি ব্যবহারকারী তৈরি করে — ব্যর্থ হলে আংশিক ব্যবহারকারী রেখে যায়। Idempotent Module-গুলো এই ঝুঁকি কমায়, কারণ তারা প্রথমে অবস্থা দেখে ও শুধু প্রয়োজনীয় পরিবর্তন করে। আর একটি নিরাপত্তা-নীতি: `shell` Module ব্যবহার করলে কমান্ডে Variable সরাসরি বসাবেন না — Shell Injection-এর ঝুঁকি থাকে; Module-এর নিজস্ব প্যারামিটার ব্যবহার করুন, যেখানে Input যাচাই করা হয়।

আরও একটি বিষয়: `--check` Mode নিরাপত্তা-পর্যালোচনার একটি হাতিয়ার — Production-এ Playbook চালানোর আগে `--check` দিয়ে দেখুন ঠিক কী বদলাবে। যদি তালিকায় এমন কিছু থাকে যা আপনি প্রত্যাশা করেননি (যেমন একটি ফায়ারওয়াল-নিয়ম মুছে যাওয়া), তাহলে থেমে যান ও কারণ খুঁজুন। অপ্রত্যাশিত পরিবর্তনই সবচেয়ে বিপজ্জনক পরিবর্তন।

৩.১৫ প্রোডাকশন ইঞ্জিনিয়ারিং অনুশীলন

সেরা অনুশীলন: (১) প্রতিটি Playbook-এ "দ্বিতীয়বার চালানো" পরীক্ষা CI-তে যোগ করুন — Playbook দুবার চালিয়ে দ্বিতীয়বার `changed=0` না হলে Build ব্যর্থ হবে; (২) `shell`/`command` ব্যবহারের ওপর দলে নিয়ম করুন — প্রতিটি ব্যবহারের পাশে মন্তব্যে (Comment) লিখতে হবে কেন কোনো Module ব্যবহার করা যায়নি; (৩) Handler ব্যবহার করুন Restart-এর জন্য — শুধু কনফিগারেশন বদলালেই সার্ভিস Restart হবে, প্রতিবার নয়; (৪) Playbook-কে নিয়মিত (যেমন প্রতিদিন) চালান — শুধু Deploy-এর সময় নয় — যাতে Drift নিজে থেকে ধরা পড়ে ও সেরে যায়; (৫) `changed` প্রতিবেদন পর্যবেক্ষণ (Monitor) করুন — হঠাৎ `changed` বেড়ে গেলে বুঝবেন কেউ হাতে কিছু বদলেছে। **অ্যান্টি-প্যাটার্ন:** "Run Once and Forget" — Playbook একবার চালিয়ে ভুলে যাওয়া; "Shell Everywhere" — সব কাজ `shell` দিয়ে করা; "Ignore the Recap" — `PLAY RECAP` না পড়া।

৩.১৬ অধ্যায়ের সারাংশ

Declarative পদ্ধতি বলে কী হতে হবে, Imperative বলে কীভাবে করতে হবে। Declarative টুল প্রথমে বর্তমান অবস্থা দেখে, তারপর শুধু পার্থক্যটুকু বদলায় — এই Check-Decide-Apply চক্রই Idempotency-র ভিত্তি। Idempotent মানে একবার চালানো আর দশবার চালানো একই ফল (`f(f(x)) = f(x)`) — আর এই গুণই Automation-কে নিরাপদে বারবার

চালানো যায় এমন করে তোলে, যা Drift Detection ও Convergence-এর পূর্বশর্ত। সঠিক উদাহরণ: `apt`, `lineinfile`, `file`, `service` Module; ভুল উদাহরণ: `echo >>`, `shell: apt-get install`, `shell: mkdir`। সোনালি নিয়ম: প্রথমে Module খুঁজুন, Shell শেষ উপায় — আর প্রতিটি Playbook দুবার চালিয়ে যাচাই করুন।

৩.১৭ জ্ঞান যাচাই — প্রশ্ন

1. Declarative ও Imperative পদ্ধতির পার্থক্য একটি দৈনন্দিন উদাহরণ (যেমন দরজা বন্ধ করা) দিয়ে ব্যাখ্যা করুন।
2. Idempotency-র সংজ্ঞা দিন ও `f(f(x)) = f(x)` সূত্রটি একটি সার্ভারের উদাহরণ দিয়ে বোঝান।
3. `echo "x" >> file` কেন Non-Idempotent, আর `lineinfile` কেন Idempotent — ভেতরের কারণ ব্যাখ্যা করুন।
4. একটি Playbook দ্বিতীয়বার চালালে সব Task `changed=0` দেখানো কেন Convergence-এর প্রমাণ?
5. কখন `shell` Module ব্যবহার করা যুক্তিসঙ্গত, আর তখন Idempotency কীভাবে রক্ষা করবেন?

৩.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. **Idempotency পরীক্ষা:** ৩.১০-এর ধাপগুলো অনুসরণ করে সঠিক Playbook দুবার চালান ও `PLAY RECAP` তুলনা করুন; তারপর হাতে Drift তৈরি করে আবার চালিয়ে দেখুন শুধু সংশ্লিষ্ট Task-টিই `changed` দেখায়।
2. **ভুল খুঁজে বের করা:** নিচের Task-গুলোর মধ্যে কোনটি Non-Idempotent, তা চিহ্নিত করুন ও সঠিক সংস্করণ লিখুন: (ক) `shell: useradd deploy` (খ) `shell: echo "127.0.0.1 db" >> /etc/hosts` (গ) `apt: name=git state=present` (ঘ) `shell: systemctl restart nginx`।
3. **Script বিশ্লেষণ:** আপনার পুরনো কোনো Shell Script নিন ও প্রতিটি লাইনের পাশে লিখুন সেটি Idempotent কি না ও কেন — তারপর Script-টিকে একটি Declarative Playbook-এ রূপান্তরের পরিকল্পনা লিখুন।

অধ্যায় 4 • PART I — Configuration Management Foundations

Chapter 4 — Ansible Architecture — Control Node, Managed Nodes, and Execution Model

৪.১ ভূমিকা

Ansible শেখার আগে একটি প্রশ্নের উত্তর জানা জরুরি: Ansible আসলে কীভাবে কাজ করে? Puppet বা Chef-এর মতো টুলে প্রতিটি সার্ভারে একটি Agent (Daemon) চলে — সেই Agent নিয়মিত Central Server-এর সঙ্গে কথা বলে ও কনফিগারেশন টেনে নেয়। Ansible-এ সেই Agent-টি নেই। কোনো Daemon নেই, কোনো Central Server নেই, কোনো অতিরিক্ত Port খুলতে হয় না। যা আছে তা হলো: একটি **Control Node** — আপনার ল্যাপটপ বা একটি সার্ভার যেখানে Ansible ইনস্টল আছে — আর **Managed Node** — যে সার্ভারগুলোকে আপনি পরিচালনা করতে চান। Control Node SSH দিয়ে Managed Node-এ ঢোকে, একটি ছোট Python প্রোগ্রাম (Module) সেখানে কপি করে, সেটি চালায়, ফলাফল JSON আকারে ফিরিয়ে আনে, আর Module-টি মুছে ফেলে। এই সরলতাই Ansible-এর সবচেয়ে বড় শক্তি — আর এই অধ্যায়ে আমরা এই স্থাপত্যের প্রতিটি অংশ খুলে দেখব।

৪.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Control Node ও Managed Node-এর ভূমিকা ও প্রয়োজনীয়তা ব্যাখ্যা করতে; (২) Agentless SSH স্থাপত্য কী ও কেন এটি Puppet/Chef-এর Agent মডেল থেকে আলাদা, তা বলতে; (৩) Push মডেল বনাম Pull মডেলের পার্থক্য ও প্রতিটির সুবিধা-অসুবিধা তুলনা করতে; (৪) একটি Module-এর Execution Flow — Control Node থেকে SSH, Module কপি, চালানো, JSON ফলাফল ফেরত — ধাপে ধাপে বর্ণনা করতে; (৫) YAML ও JSON Ansible-এ কোথায় কী ভূমিকা পালন করে তা ব্যাখ্যা করতে; (৬) Connection Plugin কী ও কেন SSH-ই ডিফল্ট, তা বলতে; (৭) Ansible-এর সীমাবদ্ধতা (যেমন Windows-এ Python-নির্ভরতা, Scale-এর সীমা) সততার সঙ্গে বর্ণনা করতে।

৪.৩ পূর্বশর্ত ধারণা

SSH কীভাবে কাজ করে তার প্রাথমিক ধারণা (Public Key প্রমাণীকরণ, `ssh user@host`), Python একটি প্রোগ্রামিং ভাষা — এইটুকু জানা, YAML-এর প্রাথমিক ধারণা (অধ্যায় ১ ও ৩), এবং অধ্যায় ৩-এর Idempotency ও Module-এর ধারণা। Puppet/Chef-এর অভিজ্ঞতা প্রয়োজন নেই — তুলনাটি এই অধ্যায়েই বোঝানো হবে।

৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি দূরবর্তী সার্ভারে কাজ করতে হলে দুটি জিনিস লাগে: **যোগাযোগের পথ** (কীভাবে সেখানে পৌঁছাব) ও **কাজ করার উপায়** (সেখানে গিয়ে কীভাবে কাজটি করব)। Puppet ও Chef এই দুটি সমস্যার সমাধান করে একটি স্থায়ী Agent বসিয়ে: Agent-টি সার্ভারে সবসময় চলে, Central Server-এর সঙ্গে কথা বলে, আর নির্দেশ পেলে কাজ করে। এর সুবিধা: Agent নিজে থেকে নিয়মিত কাজ করতে পারে। অসুবিধা: প্রতিটি সার্ভারে Agent ইনস্টল, আপডেট ও সুরক্ষিত রাখতে হয় — আর Agent-টি নিজেই একটি আক্রমণের পৃষ্ঠ (Attack Surface)।

Ansible একটি ভিন্ন প্রশ্ন করে: "সার্ভারে যাওয়ার পথ কি আগে থেকেই নেই?" উত্তর: আছে — **SSH**। প্রায় প্রতিটি Linux সার্ভারে SSH Server চলে, প্রতিটি প্রকৌশলী SSH ব্যবহার করেন। তাহলে নতুন কোনো Agent কেন? Ansible SSH-কেই যোগাযোগের পথ হিসেবে ব্যবহার করে, আর কাজ করার উপায় হিসেবে ব্যবহার করে **Module** — ছোট ছোট Python

প্রোগ্রাম, যা Control Node থেকে SSH দিয়ে Managed Node-এ কপি করা হয়, সেখানে চালানো হয়, আর ফলাফল JSON হিসেবে ফিরিয়ে আনা হয়। কাজ শেষ হলে Module-টি মুছে ফেলা হয় — Managed Node-এ কিছুই থেকে যায় না। এই নীতিকে বলে **Agentless** — কোনো স্থায়ী সফটওয়্যার ছাড়াই পরিচালনা।

> **CONCEPT:** Ansible-এর দর্শনটি এক বাক্যে: "যা আছে, তাই ব্যবহার করো।" SSH আছে — তাই নতুন Port বা Agent লাগে না। Python আছে — তাই Module Python-এ লেখা হয়। YAML মানুষ পড়তে পারে — তাই Playbook YAML-এ লেখা হয়। JSON যন্ত্র পড়তে পারে — তাই ফলাফল JSON-এ ফেরত আসে। এই দর্শনের কারণেই Ansible শেখা সহজ: আপনাকে নতুন কোনো Daemon, নতুন কোনো প্রোটোকল বা নতুন কোনো ভাষা শিখতে হয় না — আপনি যা জানেন, তাই দিয়ে শুরু করতে পারেন।

Push বনাম Pull: Puppet/Chef-এর Agent নিজে Central Server থেকে কনফিগারেশন টেনে নেয় — এটি **Pull Model**। সুবিধা: নতুন সার্ভার নিজে থেকেই কনফিগারেশন নিয়ে নেয়; অসুবিধা: Central Server সবসময় চলতে হয়, আর পরিবর্তন প্রয়োগ হতে Agent-এর পরের চক্র (সাধারণত ৩০ মিনিট) পর্যন্ত অপেক্ষা করতে হয়। Ansible-এ Control Node থেকে Managed Node-এ কাজ ঠেলে দেওয়া হয় — এটি **Push Model**। সুবিধা: পরিবর্তন তৎক্ষণাৎ প্রয়োগ হয়, কোনো Central Server লাগে না; অসুবিধা: নতুন সার্ভার নিজে থেকে কিছু করে না — কেউ Control Node থেকে Playbook চালালে তবেই কাজ হয়। বাস্তবে দুটি মডেলই ব্যবহৃত হয়: Ansible-কে নিয়মিত চালানোর জন্য Cron বা CI Pipeline ব্যবহার করা হয়, যা Push-এর অসুবিধা দূর করে।

৪.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Control Node — যা যা লাগে

Control Node হলো সেই মেশিন যেখানে Ansible ইনস্টল থাকে ও যেখান থেকে আপনি Playbook চালান। এই বইয়ের পরিবেশে Control Node হলো `nahid` ব্যবহারকারীর Ubuntu 24.04 ল্যাপটপ বা সার্ভার, যেখানে Ansible Core 2.17+ ও Python 3.12 আছে। Control Node-এ যা যা থাকে: **Ansible** নিজে (Core ও Collections), **Inventory** (কোন সার্ভারগুলো আছে), **Playbook** (কী করতে হবে), **SSH Key** (Managed Node-এ ঢোকার চাবি), ও **Ansible Configuration** (`ansible.cfg`)। Control Node Windows-এ চলতে পারে না (Ansible Control Node শুধু Linux/macOS-এ সমর্থিত) — তবে Managed Node Windows হতে পারে (WinRM দিয়ে)।

Managed Node — যা যা লাগে

Managed Node হলো সেই সার্ভার যাকে আপনি পরিচালনা করেন — যেমন TechNova-র `web-01`। Managed Node-এ যা লাগে তা অবাক করার মতো কম: **SSH Server** (সাধারণত OpenSSH, যা Ubuntu-তে ডিফল্টভাবে থাকে বা সহজে ইনস্টল করা যায়) ও **Python** (বেশিরভাগ Module-এর জন্য; Ubuntu 24.04-এ Python 3.12 ডিফল্টভাবে থাকে)। কোনো Ansible ইনস্টল করতে হয় না, কোনো Agent চালাতে হয় না, কোনো অতিরিক্ত Port খুলতে হয় না — শুধু SSH Port 22। এই কারণেই Ansible দিয়ে একটি নতুন সার্ভারকে "Adopt" করা এত সহজ: SSH চলে এমন যেকোনো Linux সার্ভারই Managed Node হতে পারে।

Module — কাজের একক

Module হলো Ansible-এর কাজের একক — একটি নির্দিষ্ট কাজ করার ছোট প্রোগ্রাম। যেমন `apt` Module প্যাকেজ ইনস্টল করে, `copy` Module ফাইল কপি করে, `service` Module সার্ভিস পরিচালনা করে, `user` Module ব্যবহারকারী তৈরি করে। প্রতিটি Module-এর তিনটি বৈশিষ্ট্য আছে: (১) এটি **Idempotent** — প্রথমে বর্তমান অবস্থা দেখে, তারপর শুধু

প্রয়োজনীয় পরিবর্তন করে (অধ্যায় ৩); (২) এটি **JSON ফেরত দেয়** — `changed`, `failed`, বার্তা ও বিস্তারিত তথ্যসহ; (৩) এটি **স্বয়ংসম্পূর্ণ (Self-Contained)** — Control Node থেকে কপি করে চালানো যায়, Managed Node-এ আগে থেকে কিছু ইনস্টল থাকার দরকার নেই। Ansible Core-এর সঙ্গে কয়েকশো Module আসে (যেমন `ansible.builtin` Collection-এ), আর আরও হাজারো Module আছে Ansible Galaxy-র Community Collection-গুলোতে।

YAML ও JSON — দুটি ভাষার ভূমিকা

Ansible-এ দুটি ডেটা-ফরম্যাট ব্যবহৃত হয়, দুটি ভিন্ন উদ্দেশ্যে। **YAML** (YAML Ain't Markup Language) হলো মানুষের ভাষা — Playbook, Inventory ও Variable ফাইল YAML-এ লেখা হয়, কারণ YAML মানুষের পড়ার জন্য সহজ (Indentation দিয়ে গঠন বোঝায়)। **JSON** (JavaScript Object Notation) হলো যন্ত্রের ভাষা — Module-গুলো তাদের ফলাফল JSON-এ ফেরত দেয়, আর Control Node সেই JSON পড়ে বোঝে কাজটি সফল হয়েছে কি না, কী বদলেছে। আপনি Playbook লেখেন YAML-এ, আর Ansible ভেতরে ভেতরে JSON-এ কথা বলে — এই বিভাজনটি Ansible-কে একই সঙ্গে মানববান্ধব ও যন্ত্রবান্ধব করে।

Connection Plugin — যোগাযোগের উপায়

Ansible যেভাবে Managed Node-এর সঙ্গে যোগাযোগ করে, তাকে বলে **Connection Plugin**। ডিফল্ট Plugin হলো **SSH** (`ansible.builtin.ssh`) — এটি OpenSSH Client ব্যবহার করে। আরও Plugin আছে: `local` (Control Node-এর নিজের ওপর কাজ করা — `localhost`-এর জন্য), `docker` (Container-এর ভেতরে কাজ করা), `winrm` (Windows Managed Node-এর জন্য), `psrp` (Windows-এর আরেকটি প্রোটোকল)। বেশিরভাগ ব্যবহারকারীর SSH-এর বাইরে কিছু জানার দরকার হয় না — তবে Plugin-এর ধারণাটি জানা থাকলে বোঝা যায় Ansible শুধু SSH-এর মধ্যে সীমাবদ্ধ নয়, তার স্থাপত্য প্রসারণযোগ্য।

বিষয়	Ansible (Agentless, Push)	Puppet / Chef (Agent, Pull)
Managed Node-এ যা লাগে	শুধু SSH + Python	Agent ইনস্টল ও চালু রাখতে হয়
Central Server	লাগে না (Control Node যথেষ্ট)	Master/Server সবসময় চলতে হয়
পরিবর্তন প্রয়োগের সময়	Playbook চালালেই তৎক্ষণাৎ	Agent-এর পরের চক্র পর্যন্ত অপেক্ষা
নতুন সার্ভার Adopt করা	SSH চললেই হয় — মিনিটের কাজ	Agent ইনস্টল, Certificate স্বাক্ষর — ধাপ বেশি
নিয়মিত Convergence	নিজে থেকে হয় না — Cron/CI দিয়ে চালাতে হয়	Agent নিজে থেকে নিয়মিত চালায়
Scale (হাজারো Node)	Push-এ Control Node-এর চাপ বাড়ে; Pull-এর জন্য AWX/Tower ব্যবহার হয়	Master-ভিত্তিক স্থাপত্যে Scale সহজতর
শেখার খরচ	কম — YAML + SSH	বেশি — DSL/Ruby + Agent ব্যবস্থাপনা

৪.৬ অভ্যন্তরীণ কার্যপ্রণালী

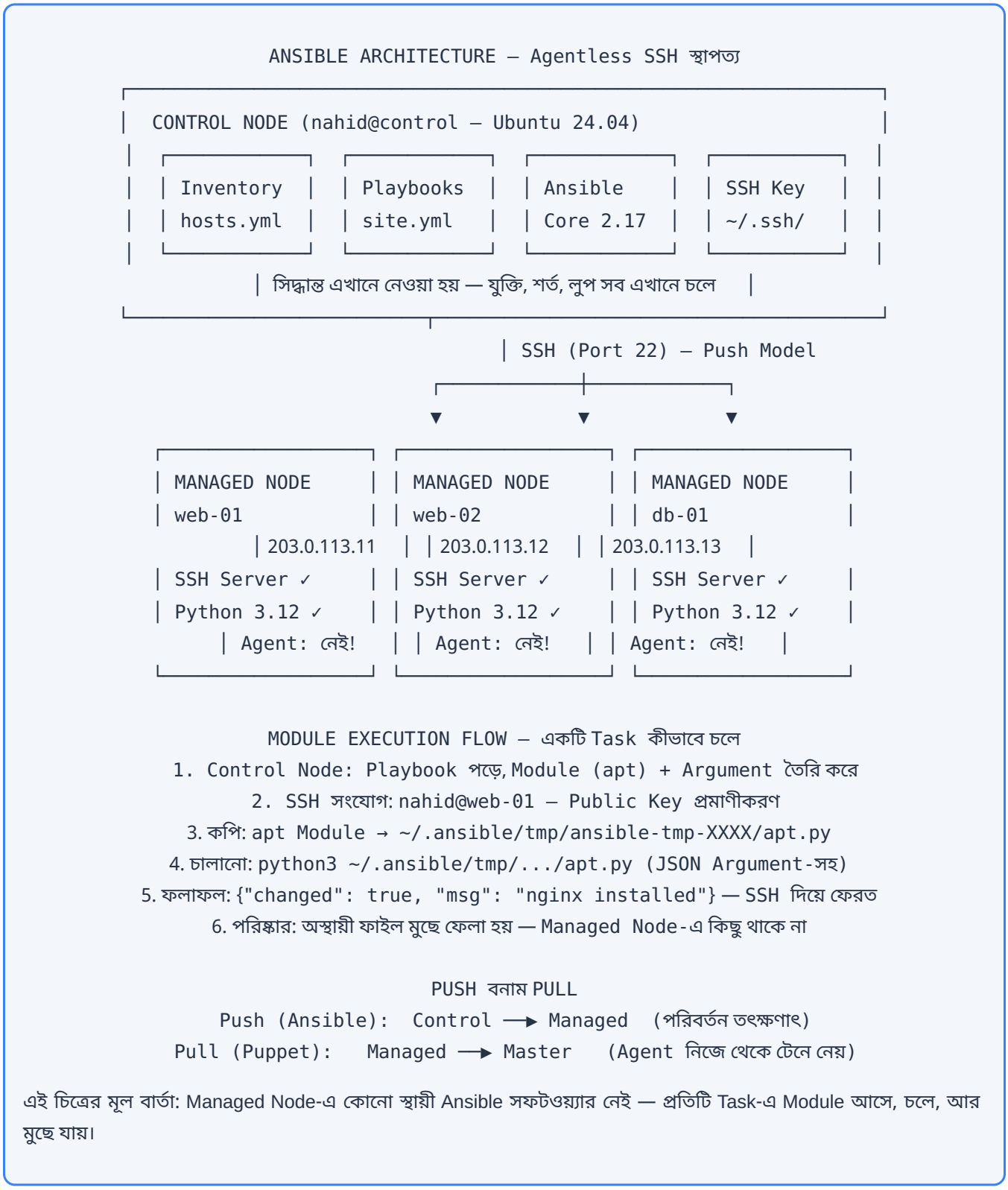
একটি Module চালানো — যেমন `web-01`-এ Nginx ইনস্টল করা — ভেতরে ভেতরে ছয়টি ধাপে ঘটে। **ধাপ ১ — প্রস্তুতি:** Control Node Playbook পড়ে, Inventory থেকে লক্ষ্য সার্ভারের ঠিকানা ও ব্যবহারকারী নেয়, আর Module-এর কোড (Python) ও তার Argument (যেমন `name=nginx, state=present`) তৈরি করে। **ধাপ ২ — সংযোগ:** Control Node

SSH দিয়ে Managed Node-এ সংযোগ করে (Public Key প্রমাণীকরণ দিয়ে)। **ধাপ ৩ — কপি:** Module-এর Python কোডটি Managed Node-এর একটি অস্থায়ী ডিরেক্টরিতে (সাধারণত `~/.ansible/tmp/`) SFTP/SCP দিয়ে কপি করা হয়। **ধাপ ৪ — চালানো:** Managed Node-এ Python দিয়ে Module-টি চালানো হয় — Module সার্ভারের বর্তমান অবস্থা দেখে (Check), সিদ্ধান্ত নেয় (Decide), প্রয়োজনীয় পরিবর্তন করে (Apply)। **ধাপ ৫ — ফলাফল:** Module তার ফলাফল JSON আকারে Standard Output-এ লেখে — যেমন `{"changed": true, "msg": "nginx installed"}` — আর Control Node SSH দিয়ে সেই JSON ফিরিয়ে আনে। **ধাপ ৬ — পরিষ্কার:** অস্থায়ী Module ফাইলটি Managed Node থেকে মুছে ফেলা হয় — কোনো চিহ্ন থাকে না। এই পুরো প্রক্রিয়াটি প্রতি Task-এ, প্রতি সার্ভারে ঘটে — আর এ কারণেই Ansible-কে "SSH-এর ওপরে গড়া" বলা হয়।

এই প্রবাহের একটি গুরুত্বপূর্ণ বৈশিষ্ট্য: **Control Node-ই সব সিদ্ধান্ত নেয়, Managed Node শুধু কাজ করে।** Playbook-এর যুক্তি (Logic), শর্ত (Condition), লুপ (Loop) — সবকিছু Control Node-এ চলে; Managed Node শুধু Module-টি চালায়। এর ফলে Managed Node হালকা থাকে, আর সমস্ত বুদ্ধি এক জায়গায় (Control Node) থাকে — যা পরীক্ষা ও ডিবাগ করা সহজ করে।

> **TIP:** Ansible-এর Execution Flow বোঝার সবচেয়ে ভালো উপায় হলো Verbose Mode: `ansible web-01 -m ping -vvv` চালালে আপনি দেখতে পাবেন SSH কমান্ডটি কী, Module ফাইলটি কোথায় কপি হচ্ছে, আর কী JSON ফিরে আসছে। প্রথমবার `-vvv` দিয়ে দেখুন — Ansible-এর "জাদু" তখন স্বচ্ছ কাচের মতো পরিষ্কার হয়ে যাবে।

৪.৭ আর্কিটেকচার চিত্র



এই চিত্রের মূল বার্তা: Managed Node-এ কোনো স্থায়ী Ansible সফটওয়্যার নেই — প্রতিটি Task-এ Module আসে, চলে, আর মুছে যায়।

৪.৮ ব্যবহারিক উদাহরণ

উদাহরণ ১ — একটি Ad-hoc কমান্ডের প্রবাহ: `ansible webservers -m ping` চালালে যা ঘটে: Control Node Inventory থেকে `webservers` দলের তিনটি সার্ভারের ঠিকানা নেয়, প্রতিটিতে SSH সংযোগ করে, `ping` Module কপি

করে চালায়, আর ফলাফল দেখায় — `web-01 | SUCCESS => {"changed": false, "ping": "pong"}`। `ping` Module আসলে নেটওয়ার্ক Ping নয় — এটি Ansible-এর সংযোগ পরীক্ষা: SSH হচ্ছে কি না, Python চলছে কি না।

উদাহরণ ২ — Facts সংগ্রহ: প্রতিটি Playbook-এর শুরুতে Ansible `setup` Module চালিয়ে Managed Node সম্পর্কে তথ্য সংগ্রহ করে — OS, সংস্করণ, IP ঠিকানা, CPU, মেমরি, Disk — যাকে বলে **Facts**। এই Facts-গুলো Variable হিসেবে Playbook-এ ব্যবহার করা যায় — যেমন শুধু Ubuntu সার্ভারে একটি Task চালানো (`when: ansible_distribution == "Ubuntu"`)। Facts সংগ্রহও একই Execution Flow-তে হয়: Module কপি, চালানো, JSON ফেরত।

```
# উদাহরণ: Facts-এর একটি অংশ (চিত্রণমূলক JSON আউটপুট)
# ansible web-01 -m setup -a "filter=ansible_distribution*"
# web-01 | SUCCESS => {
#     "ansible_facts": {
#         "ansible_distribution": "Ubuntu",
#         "ansible_distribution_version": "24.04",
#         "ansible_distribution_release": "noble",
#         "ansible_hostname": "web-01",
#         "ansible_default_ipv4": {"address": "203.0.113.11"}
#     },
#     "changed": false
# }
# ব্যাখ্যা: এই JSON-ই Ansible-এর ভাষা — Module যা ফেরত দেয়, Control Node
# তা পড়ে সিদ্ধান্ত নেয়। "changed": false মানে Facts সংগ্রহে কিছু বদলায়নি।
```

উদাহরণ ৩ — Parallel Execution: Ansible ডিফল্টভাবে ৫টি সার্ভারে একসঙ্গে কাজ করে (Forks = 5)। তিনটি Web Server থাকলে তিনটিতেই একসঙ্গে Module চলে — তাই Playbook দ্রুত শেষ হয়। Fork-এর সংখ্যা `ansible.cfg`-এ বাড়ানো যায় (`forks = 20`) — তবে বেশি Fork মানে Control Node-এ বেশি চাপ।

৪.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচে দুটি ফাইল দেওয়া হলো: Ansible-এর কনফিগারেশন ফাইল (`ansible.cfg`) ও একটি সম্পূর্ণ Inventory — এই দুটি ফাইলই অধ্যায় ৫-এর পরিবেশে সরাসরি ব্যবহার করা যাবে।

```
# ansible.cfg — Control Node-এর কনফিগারেশন (~/ansible/ansible.cfg)
[defaults]
inventory = ./inventory/hosts.yml
remote_user = nahid
host_key_checking = True
forks = 5
deprecation_warnings = False
# ব্যাখ্যা:
# inventory = Inventory ফাইলের পথ — প্রতিবার -i দিতে হয় না।
# remote_user = SSH-এ ডিফল্ট ব্যবহারকারী (nahid)।
# host_key_checking = True — Host Key যাচাই করা হবে (অধ্যায় ৫-এ বিস্তারিত)।
# forks = একসঙ্গে কতটি সার্ভারে কাজ হবে (ডিফল্ট ৫)।

# inventory/hosts.yml — TechNova-র সম্পূর্ণ Inventory
---
all:
  children:
    webservers:
      hosts:
```

```
        web-01:
          ansible_host: 203.0.113.11
        web-02:
          ansible_host: 203.0.113.12
        web-03:
          ansible_host: 203.0.113.13
      vars:
        ansible_user: nahid
    dbservers:
      hosts:
        db-01:
          ansible_host: 203.0.113.21
      vars:
        ansible_user: nahid
    technova:
      children:
        webservers: {}
        dbservers: {}
# ব্যাখ্যা: Host-গুলো দলে (Group) সাজানো — webservers, dbservers।
# 'technova' দলটি দুটি দলকে একসঙ্গে ধরে — সব সার্ভারে একসঙ্গে কাজ করতে
# চাইলে 'technova' দলকে লক্ষ্য করা যায়। Variable (ansible_user) দলের
# স্তরে লিখলে দলের সব Host-এ প্রযোজ্য হয়।
```

৪.১০ ধাপে ধাপে বাস্তবায়ন

অনুশীলন: Execution Flow নিজের চোখে দেখা (অধ্যায় ৫-এর পরিবেশ তৈরি হওয়ার পর)

1. Control Node থেকে `ansible web-01 -m ping -vvv` চালান — Verbose আউটপুটে SSH কমান্ড, Module-এর অস্থায়ী পথ ও JSON ফলাফল লক্ষ্য করুন।
2. `ansible web-01 -m setup -a "filter=ansible_hostname"` চালিয়ে Facts সংগ্রহ দেখুন।
3. Managed Node-এ SSH করে `ls ~/.ansible/tmp/` দেখুন — Playbook চলার সময় অস্থায়ী ফাইল থাকে, চলা শেষ হলে মুছে যায়।
4. `ansible webservers -m ping` চালিয়ে দেখুন তিনটি সার্ভারে একসঙ্গে কাজ হয় — প্রতিটির `SUCCESS` লাইন আলাদা আসে।
5. `ansible.cfg`-এ `forks = 1` বসিয়ে আবার চালান — এবার সার্ভারগুলো একে একে কাজ করে; পার্থক্যটি সময়ে বোঝা যাবে। তারপর `forks = 5`-এ ফিরিয়ে আনুন।

৪.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

`ansible webservers -m ping`-এর প্রত্যাশিত আউটপুট নিচের মতো — প্রতিটি সার্ভারের জন্য একটি `SUCCESS` লাইন, JSON ফলাফলসহ। কোনো সার্ভারে SSH না হলে `UNREACHABLE` দেখাবে — তার সমস্যা সমাধান অধ্যায় ৫-এ।

```
# ansible webservers -m ping (চিত্রণমূলক আউটপুট):
# web-01 | SUCCESS => {
#     "ansible_facts": {"discovered_interpreter_python": "/usr/bin/python3"},
#     "changed": false,
#     "ping": "pong"
# }
# web-02 | SUCCESS => {
#     "ansible_facts": {"discovered_interpreter_python": "/usr/bin/python3"},
#     "changed": false,
```

```
#     "ping": "pong"
# }
# web-03 | SUCCESS => {
#     "ansible_facts": {"discovered_interpreter_python": "/usr/bin/python3"},
#     "changed": false,
#     "ping": "pong"
# }
# ব্যাখ্যা: "ping": "pong" মানে সংযোগ সফল; "changed": false মানে ping Module
# কিছু বদলায়নি (এটি শুধু পরীক্ষা করে); discovered_interpreter_python মানে
# Ansible Managed Node-এ Python খুঁজে পেয়েছে (/usr/bin/python3)।
```

৪.১২ সাধারণ ভুল

- **ভুল ধারণা:** "Ansible-এর জন্য Managed Node-এ Ansible ইনস্টল করতে হয়।" — না; Managed Node-এ শুধু SSH ও Python লাগে। Ansible শুধু Control Node-এ থাকে।
- **ভুল ধারণা:** "`ping` Module নেটওয়ার্ক Ping করে।" — না; এটি Ansible-এর সংযোগ ও Python পরীক্ষা করে। নেটওয়ার্ক Ping করতে `ping` কমান্ড বা `wait_for` Module ব্যবহার করুন।
- **ভুল:** Control Node-কে Windows-এ চালানোর চেষ্টা করা — Ansible Control Node Windows-এ সমর্থিত নয়। Windows ব্যবহারকারী WSL (Windows Subsystem for Linux) ব্যবহার করতে পারেন।
- **ভুল ধারণা:** "Push Model মানে Drift নিজে থেকে সারে না — তাই Ansible Puppet-এর চেয়ে খারাপ।" — না; Push মানে পরিবর্তন তৎক্ষণাৎ হয়, আর নিয়মিত Convergence-এর জন্য Cron বা CI Pipeline ব্যবহার করা হয় — এটি একটি নকশা-পছন্দ, দুর্বলতা নয়।
- **ভুল:** সব সার্ভারে একসঙ্গে বড় Playbook চালিয়ে Control Node-কে চাপে ফেলা — Fork-এর সংখ্যা না বুঝে বাড়ানো বা কমানো। ছোট দলে শুরু করুন, তারপর Scale করুন।
- **ভুল ধারণা:** "Module মানে Shell কমান্ডের মোড়ক।" — না; Module বর্তমান অবস্থা দেখে ও Idempotentভাবে কাজ করে — Shell কমান্ড অন্ধভাবে চলে। এই পার্থক্যই অধ্যায় ৩-এর মূল শিক্ষা।

৪.১৩ সমস্যা সমাধান (Troubleshooting)

1. **সমস্যা:** "`UNREACHABLE! => Failed to connect to the host via ssh`" — **পরীক্ষা:** হাতে `ssh nahid@203.0.113.11` চালিয়ে দেখুন সংযোগ হয় কি না। **সমাধান:** SSH Key, ব্যবহারকারীর নাম, IP ঠিকানা ও ফায়ারওয়াল যাচাই করুন (বিস্তারিত অধ্যায় ৫)।
2. **সমস্যা:** "`MODULE FAILURE – /usr/bin/python: No module named ...`" বা Python পাওয়া যায়নি। — **পরীক্ষা:** Managed Node-এ `python3 --version` চালান। **সমাধান:** Python ইনস্টল করুন (`sudo apt install -y python3`), বা `ansible_python_interpreter` Variable দিয়ে Python-এর পথ নির্দিষ্ট করুন।
3. **সমস্যা:** "Playbook খুব ধীর — সার্ভারগুলো একে একে কাজ করে।" — **পরীক্ষা:** `ansible.cfg`-এ `forks`-এর মান দেখুন (ডিফল্ট ৫)। **সমাধান:** Fork বাড়ান (যেমন ২০), তবে Control Node-এর CPU ও নেটওয়ার্কের সীমা মনে রাখুন।
4. **সমস্যা:** "Facts সংগ্রহে অনেক সময় লাগে।" — **পরীক্ষা:** `gather_facts` প্রতিটি Play-এর শুরুতে চলে। **সমাধান:** Facts দরকার না হলে Play-এ `gather_facts: false` লিখুন; বা Fact Caching চালু করুন (পরের Part-এ বিস্তারিত)।

৪.১৪ নিরাপত্তা বিবেচনা

> **WARNING:** Ansible-এর Agentless স্থাপত্যের নিরাপত্তা পুরোপুরি SSH-এর নিরাপত্তার ওপর দাঁড়িয়ে আছে। Control Node-এর SSH Private Key-টি হলো পুরো অবকাঠামোর মাস্টার চাবি — যে কেউ সেই Key পেলে সব Managed Node-এ Root ক্ষমতা পেতে পারে (যদি `become` দিয়ে sudo ব্যবহার করা হয়)। তাই Private Key-কে Passphrase দিয়ে সুরক্ষিত করুন, Key-এর অনুমতি `600` রাখুন, Control Node-এ শুধু প্রয়োজনীয় ব্যক্তিদের প্রবেশাধিকার দিন, আর Key Rotation-এর পরিকল্পনা রাখুন। কখনোই Private Key Git-এ Commit করবেন না — এটি সবচেয়ে সাধারণ ও সবচেয়ে বিপজ্জনক ভুল।

আরও দুটি নিরাপত্তা-নীতি: (১) Managed Node-এ Ansible-এর জন্য আলাদা ব্যবহারকারী ব্যবহার করুন (যেমন `ansible`), যার sudo অধিকার শুধু প্রয়োজনীয় কমান্ডে সীমিত — Root-এর সরাসরি SSH বন্ধ করুন (`PermitRootLogin no`); (২) Host Key Verification চালু রাখুন (`host_key_checking = True`) — এটি বন্ধ করলে Man-in-the-Middle আক্রমণের সুযোগ তৈরি হয় (অধ্যায় ৫-এ বিস্তারিত)। মনে রাখবেন: Agentless মানে আক্রমণের পৃষ্ঠ কম — কোনো Agent Port খোলা নেই, কোনো Agent দুর্বলতা নেই — কিন্তু SSH Key-এর সুরক্ষাই তখন একমাত্র প্রতিরক্ষা।

৪.১৫ প্রোডাকশন ইঞ্জিনিয়ারিং অনুশীলন

সেরা অনুশীলন: (১) Control Node-কে একটি নির্দিষ্ট, সুরক্ষিত সার্ভার হিসেবে রাখুন — ব্যক্তিগত ল্যাপটপ থেকে Production Playbook চালাবেন না; (২) Inventory ও Playbook Git-এ রাখুন ও পরিবর্তনে পর্যালোচনা বাধ্যতামূলক করুন; (৩) বড় পরিবেশে (শত শত Node) Ansible-এর Push মডেলের সীমা মাথায় রাখুন — তখন AWX বা Ansible Automation Platform-এর মতো নিয়ন্ত্রিত (Managed) সমাধান ব্যবহার করুন, যেখানে Scheduling, Role-Based Access ও Audit Log থাকে; (৪) Fact Caching চালু করুন যাতে প্রতিটি Play-এ Facts আবার সংগ্রহ না করতে হয়; (৫) Connection-এর জন্য SSH Pipelining চালু করুন (`pipelining = True`) — এতে Module চালানো দ্রুত হয়, তবে `requiretty` থাকা sudo-তে সমস্যা হতে পারে, তাই পরীক্ষা করে চালু করুন। **অ্যান্টি-প্যাটার্ন:** "ল্যাপটপই Control Node" — ল্যাপটপ হারালে বা বন্ধ থাকলে Automation থেমে যায়; "একটি Control Node, কোনো ব্যাকআপ নেই" — Inventory ও Key-এর ব্যাকআপ রাখুন।

৪.১৬ অধ্যায়ের সারাংশ

Ansible-এর স্থাপত্য Agentless ও Push-ভিত্তিক: Control Node (Ansible, Inventory, Playbook, SSH Key) SSH দিয়ে Managed Node-এ (শুধু SSH + Python) সংযোগ করে, Module কপি করে চালায়, JSON ফলাফল ফিরিয়ে আনে, আর অস্থায়ী ফাইল মুছে ফেলে। কোনো Agent নেই, কোনো Central Server নেই, কোনো অতিরিক্ত Port নেই। Playbook YAML-এ লেখা হয় (মানুষের ভাষা), Module-এর ফলাফল JSON-এ আসে (যন্ত্রের ভাষা), আর যোগাযোগের উপায় হলো Connection Plugin — ডিফল্ট SSH। এই সরলতাই Ansible-কে শেখা ও Adopt করা সহজ করেছে — তবে এর নিরাপত্তা SSH Key-এর সুরক্ষার ওপর নির্ভর করে, আর বড় Scale-এ Push মডেলের জন্য AWX-এর মতো সমাধান লাগে।

৪.১৭ জ্ঞান যাচাই — প্রশ্ন

1. Control Node ও Managed Node-এর ভূমিকা ব্যাখ্যা করুন — প্রতিটিতে কী কী থাকতে হয়?
2. Agentless স্থাপত্য কী? Puppet/Chef-এর Agent মডেলের সঙ্গে এর তিনটি পার্থক্য লিখুন।
3. একটি Module চালানোর ছয়টি ধাপ (প্রস্তুতি থেকে পরিষ্কার পর্যন্ত) ক্রমানুসারে বর্ণনা করুন।

4. Push মডেল বনাম Pull মডেল — প্রতিটির একটি সুবিধা ও একটি অসুবিধা লিখুন।
5. Ansible-এ YAML ও JSON-এর ভূমিকা আলাদা কেন — প্রতিটি কোথায় ব্যবহৃত হয়?

৪.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. **Execution Flow পর্যবেক্ষণ:** ৪.১০-এর ধাপগুলো অনুসরণ করে `-vvv` দিয়ে `ping` চালান ও আউটপুট থেকে SSH কমান্ড, অস্থায়ী Module-এর পথ ও JSON ফলাফল চিহ্নিত করুন।
2. **Inventory নকশা:** TechNova-র জন্য একটি Inventory লিখুন যেখানে অন্তত দুটি দল (webservers, dbservers) ও একটি সম্মিলিত দল (technova) আছে — ৪.৯-এর আদলে।
3. **তুলনামূলক বিশ্লেষণ:** আপনার পরিবেশে Ansible-এর Push মডেলের একটি সুবিধা ও একটি অসুবিধা লিখুন — অসুবিধাটি Cron বা CI দিয়ে কীভাবে দূর করা যায়, তার পরিকল্পনা দিন।

অধ্যায় 5 • PART I — Configuration Management Foundations

Chapter 5 — Ansible Installation, SSH Connectivity, and First Automation

৫.১ ভূমিকা

চারটি অধ্যায় ধরে আমরা ধারণা শিখেছি — Configuration Management কী, কেন হাতে করা কাজ ভাঙে, Declarative ও Idempotency কী, আর Ansible-এর স্থাপত্য কেমন। এবার সময় এসেছে হাতে-কলমে করার। এই অধ্যায়ে আমরা Control Node-এ (Ubuntu 24.04, ব্যবহারকারী `nahid`) Ansible Core 2.17+ ইনস্টল করব, SSH Key তৈরি করব, Managed Node-গুলোতে (203.0.113.11–13) Key বসাব, Host Key যাচাইয়ের নিরাপত্তা বুঝব, প্রথম Ad-hoc `ping` চালাব, `ansible.cfg` সাজাব, আর প্রথম Playbook চালিয়ে একটি Web Server-কে Desired State-এ নিয়ে যাব। এই অধ্যায় শেষে আপনার হাতে একটি চলমান Ansible পরিবেশ থাকবে — এই বইয়ের বাকি সব অধ্যায়ের ভিত্তি।

৫.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Ubuntu 24.04-এ `pip` ও `apt` — দুই পদ্ধতিতে Ansible Core ইনস্টল করতে ও তাদের পার্থক্য বলতে; (২) `ansible --version` দিয়ে ইনস্টলেশন যাচাই করতে; (৩) `ssh-keygen` দিয়ে ed25519 Key Pair তৈরি করতে ও `ssh-copy-id` দিয়ে Managed Node-এ Public Key বসাতে; (৪) Host Key Verification কী, কেন এটি গুরুত্বপূর্ণ, আর কেন `StrictHostKeyChecking` বন্ধ করা স্বাভাবিক অভ্যাস হিসেবে নিষিদ্ধ, তা ব্যাখ্যা করতে; (৫) প্রথম Ad-hoc কমান্ড (`ansible -m ping`) চালাতে ও তার আউটপুট পড়তে; (৬) `ansible.cfg`-এর মৌলিক সেটিংস (Inventory, Remote User, Host Key Checking, Forks) লিখতে; (৭) প্রথম Playbook লিখে চালাতে ও `PLAY RECAP` পড়ে ফলাফল যাচাই করতে।

৫.৩ পূর্বশর্ত ধারণা

Ubuntu-তে `apt` দিয়ে প্যাকেজ ইনস্টল করার অভিজ্ঞতা, SSH-এর প্রাথমিক ধারণা (অধ্যায় ৪), YAML-এর প্রাথমিক ধারণা (অধ্যায় ১), আর একটি পরীক্ষামূলক পরিবেশ: একটি Control Node (আপনার ল্যাপটপ বা VM, Ubuntu 24.04) ও অন্তত একটি Managed Node (আরেকটি VM বা Container, IP 203.0.113.11 — ল্যাবের IP; আপনার পরিবেশে আপনার আসল IP ব্যবহার করুন)। Python 3.12 Ubuntu 24.04-এ ডিফল্টভাবে থাকে — আলাদা করে ইনস্টল করতে হয় না।

৫.৪ প্রথম নীতি থেকে ব্যাখ্যা

Ansible ইনস্টল করা মানে আসলে কী? Ansible একটি Python প্রোগ্রাম — তাই ইনস্টল করা মানে Python-এর Package Manager দিয়ে Ansible-এর কোড ডাউনলোড করা ও চালানোর ব্যবস্থা করা। দুটি পথ আছে: **apt** (Ubuntu-র Package Manager) ও **pip** (Python-এর Package Manager)। apt দিয়ে ইনস্টল করলে Ubuntu-র পরীক্ষিত সংস্করণ পাওয়া যায় — স্থিতিশীল, কিন্তু কখনো কখনো পুরনো। pip দিয়ে ইনস্টল করলে সবচেয়ে নতুন সংস্করণ পাওয়া যায় — কিন্তু Python-পরিবেশ ব্যবস্থাপনার দায়িত্ব আপনার। এই বইয়ে আমরা pip-কে প্রাধান্য দেব (Virtual Environment-সহ), কারণ Ansible Core 2.17+ নিশ্চিতভাবে পাওয়া যায় — তবে apt পদ্ধতিটিও দেখানো হবে, যাতে আপনি পার্থক্যটি বুঝতে পারেন।

SSH Key-এর ধারণাটিও প্রথম নীতি থেকে বোঝা যায়: পাসওয়ার্ড দিয়ে প্রমাণীকরণ মানে প্রতিবার একটি গোপন শব্দ পাঠানো — যা ধরা পড়তে পারে, অনুমান করা যায়, আর Automation-এ প্রতিবার টাইপ করা যায় না। **Public Key**

প্রমাণীকরণ মানে একটি Key Pair — একটি Public Key (যা সবাইকে দেওয়া যায়) ও একটি Private Key (যা শুধু আপনার কাছে থাকে)। Managed Node-এ Public Key বসানো হয়; Control Node Private Key দিয়ে প্রমাণ করে "আমিই সেই ব্যক্তি, যার Public Key তুমি রাখো" — Private Key কখনো নেটওয়ার্কে পাঠানো হয় না। এই কারণে Automation-এ Key-ভিত্তিক প্রমাণীকরণই একমাত্র সঠিক পদ্ধতি — পাসওয়ার্ড নয়।

> **CONCEPT:** Host Key Verification হলো SSH-এর "পরিচয় যাচাই" — প্রথমবার কোনো সার্ভারে সংযোগ করলে SSH তার Host Key-এর Fingerprint দেখায় ও জিজ্ঞেস করে "এই সার্ভারটিকে কি বিশ্বাস করব?" আপনি একবার বিশ্বাস করলে Fingerprint `~/.ssh/known_hosts`-এ সংরক্ষিত হয়; পরের বার সার্ভারের Key বদলে গেলে SSH সতর্ক করে — কারণ Key বদলে যাওয়া মানে হয় সার্ভারটি আবার তৈরি হয়েছে, নয়তো কেউ মাঝখানে বসে আপনার সংযোগ ছিনতাই করছে (Man-in-the-Middle)। এই যাচাই বন্ধ করা মানে এই সতর্কতা বন্ধ করা — আক্রমণকারী আপনার সংযোগ ছিনতাই করলেও আপনি জানতে পারবেন না।

৫.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

ইনস্টলেশন — pip বনাম apt

pip পদ্ধতি (প্রস্তাবিত): প্রথমে একটি Python Virtual Environment তৈরি করুন — যাতে Ansible-এর নির্ভরতা (Dependency) সিস্টেমের Python-এর সঙ্গে না মেশে। তারপর `pip install ansible-core` চালান। এই পদ্ধতিতে আপনি Ansible Core 2.17+ পাবেন, আর ভবিষ্যতে `pip install --upgrade ansible-core` দিয়ে আপডেট করতে পারবেন। অসুবিধা: Virtual Environment সক্রিয় (Activate) করতে হয়, বা Ansible-এর পথ (Path) মনে রাখতে হয়। **apt পদ্ধতি:** `sudo apt install ansible` — সহজ, কিন্তু Ubuntu 24.04-এর Repository-তে যে সংস্করণ আছে তা-ই পাবেন; নতুন সংস্করণ চাইলে PPA বা pip-এ যেতে হবে। মনে রাখবেন: `ansible` প্যাকেজে Ansible Core ও অনেক Community Collection থাকে; `ansible-core` প্যাকেজে শুধু Core থাকে — এই বইয়ে আমরা `ansible-core` ব্যবহার করব, কারণ Core-ই মূল; Collection পরে প্রয়োজনমতো যোগ করা যাবে।

SSH Key তৈরি — ed25519

`ssh-keygen -t ed25519` একটি আধুনিক, দ্রুত ও নিরাপদ Key Algorithm ব্যবহার করে। পুরনো RSA (2048-bit) এখনো কাজ করে, কিন্তু ed25519 ছোট Key-তে সমান নিরাপত্তা দেয় ও দ্রুত কাজ করে — আজকের মান। Key তৈরির সময় একটি **Passphrase** দেওয়া উচিত — Private Key চুরি গেলেও Passphrase ছাড়া তা ব্যবহার করা যাবে না। Automation-এ Passphrase অসুবিধা তৈরি করে (প্রতিবার টাইপ করতে হয়) — তার সমাধান হলো `ssh-agent` — একবার Passphrase দিয়ে Agent-এ Key লোড করা, তারপর Agent-ই প্রমাণীকরণ করে। এই অধ্যায়ে আমরা Passphrase-সহ Key তৈরি করব ও `ssh-agent` ব্যবহার করব — নিরাপত্তা ও সুবিধা দুটিই রক্ষা করে।

Host Key Verification — কেন বন্ধ করা যাবে না

অনেক Tutorial-এ দেখবেন `StrictHostKeyChecking=no` বা `host_key_checking = False` লিখতে বলা হয় — "যাতে প্রথমবার সংযোগে প্রশ্ন না আসে"। এটি একটি বিপজ্জনক অভ্যাস। Host Key যাচাই বন্ধ করলে আপনার Control Node যেকোনো সার্ভারকে বিশ্বাস করবে — এমনকি আক্রমণকারীর সার্ভারকেও। আক্রমণকারী আপনার সংযোগ ছিনতাই করে আপনার SSH Key দিয়ে Managed Node-এ ঢুকতে পারে, বা আপনার Playbook-এর তথ্য চুরি করতে পারে। সঠিক পদ্ধতি: প্রথমবার হাতে SSH করে Host Key যাচাই করে `known_hosts`-এ সংরক্ষণ করুন, বা `ssh-keyscan` দিয়ে বিশ্বস্ত উৎস থেকে Key সংগ্রহ করুন — তারপর Ansible চালান। `ansible.cfg`-এ `host_key_checking = True` রাখুন — এটি ডিফল্ট, আর ডিফল্টটিই সঠিক। শুধু অস্থায়ী ল্যাব-পরীক্ষায়, যেখানে সার্ভার বারবার তৈরি ও মুছে ফেলা হয়, এই সেটিং সাময়িকভাবে বদলানো যেতে পারে — তবে Production-এ কখনো নয়।

Ad-hoc কমান্ড বনাম Playbook

Ansible-এ দুইভাবে কাজ করা যায়। **Ad-hoc কমান্ড** (`ansible`) — একটি Module একবার চালানো, যেমন `ansible webservers -m ping` বা `ansible webservers -m apt -a "name=nginx state=present" --become`। এটি দ্রুত পরীক্ষা ও এককালীন কাজের জন্য। **Playbook** (`ansible-playbook`) — YAML ফাইলে লেখা একাধিক Task-এর সমষ্টি, যা বারবার চালানো যায়, Git-এ রাখা যায়, পর্যালোচনা করা যায়। নিয়ম: এককালীন পরীক্ষায় Ad-hoc, পুনরাবৃত্তিযোগ্য কাজে Playbook। এই অধ্যায়ে দুটিই করব — প্রথমে Ad-hoc দিয়ে সংযোগ যাচাই, তারপর Playbook দিয়ে প্রথম Automation।

৫.৬ অভ্যন্তরীণ কার্যপ্রণালী

প্রথম Playbook চালানোর সময় ভেতরে ভেতরে যা ঘটে, তা অধ্যায় ৪-এর Execution Flow-এরই প্রয়োগ। **ধাপ ১:** `ansible-playbook` কমান্ড `ansible.cfg` পড়ে — Inventory-র পথ, Remote User, Fork-এর সংখ্যা জানে। **ধাপ ২:** Inventory পড়ে লক্ষ্য সার্ভারগুলো (web-01, web-02, web-03) নির্ধারণ করে। **ধাপ ৩:** প্রতিটি Play-এর শুরুতে `setup` Module দিয়ে Facts সংগ্রহ করে — OS, IP, Python-এর পথ। **ধাপ ৪:** প্রতিটি Task-এর জন্য Module কপি করে SSH দিয়ে চালায় — যেমন `apt` Module Nginx ইনস্টল করে। **ধাপ ৫:** প্রতিটি Task-এর JSON ফলাফল পড়ে `ok` / `changed` / `failed` হিসেবে দেখায়। **ধাপ ৬:** শেষে `PLAY RECAP` দেয় — কোন সার্ভারে কতটি Task সফল, কতটি বদলেছে, কতটি ব্যর্থ হয়েছে। এই পুরো প্রক্রিয়াটি বোঝা থাকলে যেকোনো ত্রুটির কারণ খুঁজে পাওয়া সহজ হয় — কারণ আপনি জানেন কোন ধাপে কী ঘটার কথা ছিল।

> **TIP:** প্রথমবার Playbook চালানোর আগে সবসময় `--check` (Dry Run) দিয়ে চালান — কী বদলাবে তা দেখুন, তারপর আসল চালানো করুন। আর প্রথমবার Production-এ চালানোর আগে `--limit` দিয়ে একটি সার্ভারে চালান — যেমন `ansible-playbook site.yml --limit web-01` — সফল হলে বাকিগুলোতে চালান। এই দুটি অভ্যাস — Dry Run ও Canary — আপনাকে বড় দুর্ঘটনা থেকে বাঁচাবে।

৫.৭ আর্কিটেকচার চিত্র



লক্ষ্য করুন: Private Key কখনো Control Node ছাড়ে না — শুধু Public Key Managed Node-গুলোতে যায়।

৫.৮ ব্যবহারিক উদাহরণ

উদাহরণ ১ — সংস্করণ যাচাই: `ansible --version` চালালে Ansible Core-এর সংস্করণ, Python-এর সংস্করণ, কনফিগারেশন ফাইলের পথ ও Collection-এর পথ দেখা যায়। এই আউটপুটটি প্রথমেই দেখে নিন — কারণ "Ansible কাজ করছে না" অভিযোগের অনেক কারণই ভুল সংস্করণ বা ভুল কনফিগারেশন ফাইল।

উদাহরণ ২ — Key তৈরি ও বসানো: নিচের ৫.১০-এর ধাপগুলো অনুসরণ করুন। মনে রাখবেন: `ssh-copy-id` প্রথমবার পাসওয়ার্ড চাইবে — এটাই শেষবার পাসওয়ার্ড ব্যবহার; এরপর থেকে Key দিয়ে সংযোগ হবে।

```
# উদাহরণ: একটি Managed Node-এ হাতে সংযোগ পরীক্ষা (Ansible-এর আগে)
ssh nahid@203.0.113.11
# প্রথমবারের আউটপুট (চিত্রণমূলক):
```

```
# The authenticity of host '203.0.113.11 (203.0.113.11)' can't be established.
# ED25519 key fingerprint is SHA256:AbCdEfGhIjKlMnOpQrStUvWxYz1234567890AbCdEf.
# Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
# ব্যাখ্যা: এই প্রশ্নটিই Host Key Verification — 'yes' বললে Fingerprint
# ~/.ssh/known_hosts-এ সংরক্ষিত হয়। পরের বার আর প্রশ্ন আসবে না — যতক্ষণ
# সার্ভারের Key একই থাকে। Fingerprint-টি সার্ভারের Console থেকে যাচাই করে
# নেওয়া উত্তম — বিশেষ করে Production-এ।
```

৫.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচে এই অধ্যায়ের তিনটি মূল ফাইল দেওয়া হলো — `ansible.cfg`, Inventory ও প্রথম Playbook। এগুলো `~/ansible/` ডিরেক্টরিতে রাখুন।

```
# ansible.cfg — ~/ansible/ansible.cfg
[defaults]
inventory = ./inventory/hosts.yml
remote_user = nahid
host_key_checking = True
forks = 5
deprecation_warnings = False
roles_path = ./roles
collections_path = ./collections

[privilege_escalation]
become = True
become_method = sudo
become_ask_pass = False
# ব্যাখ্যা: become = True মানে Task-গুলো ডিফল্টভাবে sudo দিয়ে চলবে;
# become_ask_pass = False মানে sudo-র পাসওয়ার্ড চাওয়া হবে না — কারণ
# nahid ব্যবহারকারীর Passwordless sudo আছে (ল্যাব পরিবেশ)। Production-এ
# sudo-র পাসওয়ার্ড বা সীমিত sudo নিয়ম ব্যবহার করুন।

# inventory/hosts.yml — ~/ansible/inventory/hosts.yml
---
all:
  children:
    webservers:
      hosts:
        web-01:
          ansible_host: 203.0.113.11
        web-02:
          ansible_host: 203.0.113.12
        web-03:
          ansible_host: 203.0.113.13
      vars:
        ansible_user: nahid
# ল্যাবের IP 203.0.113.x — আপনার পরিবেশে আসল IP ব্যবহার করুন।

# site.yml — ~/ansible/site.yml (প্রথম Playbook)
---
- name: Configure TechNova web servers
  hosts: webservers
  become: true
  tasks:
    - name: Ensure Nginx is installed
      apt:
```

```
        name: nginx
        state: present
        update_cache: true

    - name: Ensure Nginx is running and enabled at boot
      service:
        name: nginx
        state: started
        enabled: true

    - name: Allow HTTP and HTTPS through the firewall
      ufw:
        rule: allow
        port: "{{ item }}"
        proto: tcp
      loop:
        - "80"
        - "443"

    - name: Ensure TechNova welcome page exists
      copy:
        dest: /var/www/html/index.html
        content: |
```

```
        owner: www-data
        group: www-data
        mode: "0644"
# ব্যাখ্যা: এই Playbook-টি চারটি কাজ করে — Nginx ইনস্টল, সার্ভিস চালু,
# ফায়ারওয়াল-নিয়ম, ও একটি স্বাগত পাতা। প্রতিটি Task Idempotent — দ্বিতীয়বার
# চালালে changed=0 আসবে (অধ্যায় ৩-এর পরীক্ষা)।
```

৫.১০ ধাপে ধাপে বাস্তবায়ন

অংশ A — Ansible ইনস্টল করা (Control Node-এ, ব্যবহারকারী nahid)

1. সিস্টেম আপডেট করুন: `sudo apt update && sudo apt upgrade -y`
2. Python Virtual Environment-এর জন্য প্রয়োজনীয় প্যাকেজ ইনস্টল করুন: `sudo apt install -y python3-venv python3-pip`
3. একটি ডিরেক্টরি তৈরি করুন: `mkdir -p ~/ansible && cd ~/ansible`
4. Virtual Environment তৈরি ও সক্রিয় করুন: `python3 -m venv .venv && source .venv/bin/activate`
5. Ansible Core ইনস্টল করুন: `pip install ansible-core`
6. যাচাই করুন: `ansible --version` — Ansible Core 2.17+ দেখা উচিত।

অংশ B — SSH Key তৈরি ও বসানো

1. Key তৈরি করুন: `ssh-keygen -t ed25519 -C "nahid@technova.example"` — Passphrase দিন (মনে রাখবেন)।
2. `ssh-agent` চালু করুন ও Key লোড করুন: `eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519`
3. প্রতিটি Managed Node-এ Public Key বসান: `ssh-copy-id nahid@203.0.113.11` (প্রতিটি সার্ভারের জন্য — প্রথমবার পাসওয়ার্ড চাইবে)।
4. পাসওয়ার্ড ছাড়া সংযোগ পরীক্ষা করুন: `ssh nahid@203.0.113.11 "hostname"` — `web-01` দেখা উচিত, পাসওয়ার্ড চাওয়া উচিত নয়।

অংশ C — প্রথম Ad-hoc কমান্ড ও Playbook

1. ৫.৯-এর `ansible.cfg`, Inventory ও `site.yml` ফাইলগুলো `~/ansible/`-এ রাখুন।
2. সংযোগ পরীক্ষা করুন: `ansible webservers -m ping`
3. Dry Run করুন: `ansible-playbook site.yml --check` — কী বদলাবে দেখুন।
4. আসল চালানো করুন: `ansible-playbook site.yml`
5. Idempotency যাচাই করুন: আবার `ansible-playbook site.yml` চালান — এবার সব Task `ok` (changed=0) আসা উচিত।
6. ব্রাউজার বা `curl` দিয়ে যাচাই করুন: `curl http://203.0.113.11/` — "Welcome to technova.example" দেখা উচিত।

```
# অংশ A — ইনস্টলেশন (Control Node-এ চালানোর কমান্ড)
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-venv python3-pip
mkdir -p ~/ansible && cd ~/ansible
python3 -m venv .venv && source .venv/bin/activate
pip install ansible-core
ansible --version
# চিত্রণমূলক আউটপুট:
# ansible [core 2.17.0]
#   config file = /home/nahid/ansible/ansible.cfg
#   configured module search path = ['/home/nahid/.ansible/collections/...']
#   python version = 3.12.3 (main, ...)
#   jinja version = 3.1.4
# ব্যাখ্যা: 'ansible [core 2.17.0]' মানে Ansible Core 2.17 ইনস্টল হয়েছে;
# 'config file' লাইনটি দেখায় কোন ansible.cfg পড়া হচ্ছে — ভুল ফাইল পড়া
# হলে এই লাইনেই ধরা পড়ে।

# অংশ B — SSH Key (Control Node-এ)
ssh-keygen -t ed25519 -C "nahid@technova.example"
# প্রশ্ন: Enter file in which to save the key — Enter চাপুন (ডিফল্ট ~/.ssh/id_ed25519)
# প্রশ্ন: Enter passphrase — একটি শক্তিশালী Passphrase দিন (যেমন CHANGE-ME-STRONG-PASSPHRASE)
eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519
ssh-copy-id nahid@203.0.113.11
ssh-copy-id nahid@203.0.113.12
ssh-copy-id nahid@203.0.113.13
ssh nahid@203.0.113.11 "hostname"
# চিত্রণমূলক আউটপুট: web-01

# apt পদ্ধতি (বিকল্প — pip-এর বদলে, যদি pip ব্যবহার না করতে চান):
```

```
# sudo apt update && sudo apt install -y ansible
# ansible --version
# ব্যাখ্যা: সহজ, কিন্তু সংস্করণ Ubuntu Repository-এর ওপর নির্ভরশীল।
```

৫.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

প্রথম Playbook চালানোর প্রত্যাশিত আউটপুট নিচের মতো। লক্ষ্য করুন প্রতিটি Task-এর `changed` বা `ok`, আর শেষে `PLAY RECAP`।

```
# ansible webservers -m ping (চিত্রণমূলক আউটপুট):
# web-01 | SUCCESS => {"changed": false, "ping": "pong"}
# web-02 | SUCCESS => {"changed": false, "ping": "pong"}
# web-03 | SUCCESS => {"changed": false, "ping": "pong"}
#
# ansible-playbook site.yml (প্রথমবার — চিত্রণমূলক আউটপুট):
# PLAY [Configure TechNova web servers] ****
# TASK [Gathering Facts] ****
# ok: [web-01]
# ok: [web-02]
# ok: [web-03]
# TASK [Ensure Nginx is installed] ****
# changed: [web-01]
# changed: [web-02]
# changed: [web-03]
# TASK [Ensure Nginx is running and enabled at boot] ****
# changed: [web-01]
# changed: [web-02]
# changed: [web-03]
# TASK [Allow HTTP and HTTPS through the firewall] ****
# changed: [web-01]
# changed: [web-02]
# changed: [web-03]
# TASK [Ensure TechNova welcome page exists] ****
# changed: [web-01]
# changed: [web-02]
# changed: [web-03]
# PLAY RECAP ****
# web-01 : ok=5  changed=4  unreachable=0  failed=0
# web-02 : ok=5  changed=4  unreachable=0  failed=0
# web-03 : ok=5  changed=4  unreachable=0  failed=0
# ব্যাখ্যা: Gathering Facts-এ ok=1 (তথ্য সংগ্রহে কিছু বদলায় না); বাকি চারটি
# Task-এ changed=1 — কারণ প্রথমবার সবকিছু করতে হয়েছে। মোট ok=5, changed=4।
# দ্বিতীয়বার চালালে PLAY RECAP হবে: ok=5 changed=0 — এটাই Idempotency-র প্রমাণ।
#
# curl http://203.0.113.11/ (চিত্রণমূলক আউটপুট):
#
#
#
```

```
#
```

৫.১২ সাধারণ ভুল

- **ভুল:** `host_key_checking = False` বা `StrictHostKeyChecking=no` বসিয়ে সমস্যা "সমাধান" করা — এটি নিরাপত্তা-ছিদ্র তৈরি করে (৫.৫ দেখুন)। সঠিক পদ্ধতি: প্রথমবার হাতে SSH করে Host Key যাচাই করা।
- **ভুল:** Private Key-এর অনুমতি ভুল রাখা — SSH অভিযোগ করবে ("UNPROTECTED PRIVATE KEY FILE!")। সঠিক অনুমতি: `chmod 600 ~/.ssh/id_ed25519`, ডিরেক্টরি `chmod 700 ~/.ssh`।
- **ভুল:** Virtual Environment সক্রিয় না করে `ansible` চালানো — "command not found" আসবে। প্রতিবার `source ~/ansible/.venv/bin/activate` করুন, বা Shell-এর Profile-এ যোগ করুন।
- **ভুল:** Inventory-তে ভুল IP বা ভুল ব্যবহারকারী লেখা — `UNREACHABLE` ত্রুটি আসবে। প্রথমে হাতে `ssh` করে সংযোগ যাচাই করুন, তারপর Ansible চালান।
- **ভুল:** Playbook-এর YAML Indentation ভুল করা — YAML Space-সংবেদনশীল (Sensitive); Tab ব্যবহার করবেন না, শুধু Space (২টি) ব্যবহার করুন। ত্রুটি দেখলে প্রথমে Indentation দেখুন।
- **ভুল ধারণা:** "`ansible --version` দেখালেই সব ঠিক আছে।" — না; সংস্করণ ঠিক থাকলেও Inventory-র পথ, SSH Key বা Host Key — যেকোনোটি ভুল থাকতে পারে। `ansible webservers -m ping`-ই আসল পরীক্ষা।
- **ভুল:** পাসওয়ার্ড-ভিত্তিক SSH দিয়ে Automation চালানোর চেষ্টা করা — প্রতিবার পাসওয়ার্ড চাইবে, আর Playbook থেমে যাবে। Key-ভিত্তিক প্রমাণীকরণই Automation-এর জন্য বাধ্যতামূলক।

৫.১৩ সমস্যা সমাধান (Troubleshooting)

1. **সমস্যা:** "`ansible: command not found`" — **পরীক্ষা:** `which ansible` ও `source ~/ansible/.venv/bin/activate` করা হয়েছে কি না দেখুন। **সমাধান:** Virtual Environment সক্রিয় করুন; স্থায়ী সমাধানের জন্য `~/.bashrc`-এ Activation যোগ করুন বা `pipx` দিয়ে ইনস্টল করুন।
2. **সমস্যা:** "`UNREACHABLE! => Failed to connect to the host via ssh: Permission denied (publickey,password)`" — **পরীক্ষা:** হাতে `ssh -v nahid@203.0.113.11` চালিয়ে দেখুন কোন Key ব্যবহার হচ্ছে। **সমাধান:** `ssh-copy-id` আবার চালান; Key-এর অনুমতি যাচাই করুন (`chmod 600`); Managed Node-এ `~/.ssh/authorized_keys`-এর অনুমতি `600` ও মালিকানা সঠিক কি না দেখুন।
3. **সমস্যা:** "`WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!`" — **পরীক্ষা:** সার্ভারটি কি আবার তৈরি করা হয়েছে (নতুন Host Key)? **সমাধান:** যদি সার্ভারটি সত্যিই আবার তৈরি হয়ে থাকে, তাহলে পুরনো Key সরান: `ssh-keygen -R 203.0.113.11` — তারপর আবার সংযোগ করে নতুন Key যাচাই করুন। কখনোই এই সতর্কতা অন্ধভাবে উপেক্ষা করবেন না — এটি Man-in-the-Middle-এর লক্ষণ হতে পারে।
4. **সমস্যা:** "`ERROR! the playbook: site.yml could not be found`" — **পরীক্ষা:** আপনি কোন ডিরেক্টরিতে আছেন (`pwd`) ও ফাইলটি কোথায় আছে দেখুন। **সমাধান:** `cd ~/ansible` করে চালান, বা পুরো পথ দিন: `ansible-playbook ~/ansible/site.yml`।
5. **সমস্যা:** "Playbook চলে, কিন্তু `ufw` Task ব্যর্থ হয়।" — **পরীক্ষা:** Managed Node-এ UFW ইনস্টল আছে কি না (`which ufw`)। **সমাধান:** Playbook-এ UFW ইনস্টল করার Task যোগ করুন (যেমন `apt: name=ufw state=present`) — Task-এর ক্রম মনে রাখুন: ইনস্টল আগে, ব্যবহার পরে।
6. **সমস্যা:** "`sudo: a password is required`" — **পরীক্ষা:** Managed Node-এ `nahid` ব্যবহারকারীর Passwordless sudo আছে কি না (`sudo -n true` চালান)। **সমাধান:** ল্যাবে `sudo visudo` দিয়ে `nahid ALL=(ALL) NOPASSWD:ALL` যোগ করুন; Production-এ `become_ask_pass = True` ব্যবহার করুন বা সীমিত sudo নিয়ম লিখুন।

৫.১৪ নিরাপত্তা বিবেচনা

> **WARNING:** এই অধ্যায়ের প্রতিটি পদক্ষেপের নিরাপত্তা-অর্থ আছে। (১) **Private Key:** কখনোই Git-এ Commit করবেন না, কখনোই অন্যকে পাঠাবেন না, অনুমতি `600` রাখুন। Key চুরি গেলে সব Managed Node-এর Key বদলাতে হবে — তাই Key-এর ব্যাকআপ এনক্রিপ্ট করে রাখুন। (২) **Passphrase:** Key-এ Passphrase দিন — Private Key চুরি গেলেও Passphrase ছাড়া তা অকেজো। `ssh-agent` ব্যবহার করলে প্রতিবার Passphrase টাইপ করতে হয় না। (৩) **Host Key Verification:** কখনোই স্বাভাবিক অভ্যাস হিসেবে বন্ধ করবেন না — এটি Man-in-the-Middle আক্রমণের বিরুদ্ধে আপনার প্রতিরক্ষা। (৪) **Passwordless sudo:** ল্যাবে সুবিধাজনক, কিন্তু Production-এ `nahid` ব্যবহারকারীকে শুধু প্রয়োজনীয় কমান্ডের sudo অধিকার দিন — সব কমান্ডের নয়। (৫) **Secret:** Playbook-এ কখনো আসল পাসওয়ার্ড বা API Key লিখবেন না — এই বইয়ের সব উদাহরণে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে; বাস্তবে Ansible Vault ব্যবহার করুন।

আর একটি গুরুত্বপূর্ণ নীতি: Control Node-এর নিরাপত্তা পুরো অবকাঠামোর নিরাপত্তা। Control Node-এ শক্তিশালী পাসওয়ার্ড বা Key-ভিত্তিক লগইন, নিয়মিত আপডেট, ফায়ারওয়াল, আর শুধু প্রয়োজনীয় ব্যক্তিদের প্রবেশাধিকার — এগুলো বাধ্যতামূলক। Control Node-টি কারো ব্যক্তিগত ল্যাপটপ হলে তা চুরি বা হারানোর ঝুঁকি থাকে — Production-এর Control Node একটি নির্দিষ্ট, সুরক্ষিত সার্ভার হওয়া উচিত (অধ্যায় ৪-এর ৪.১৫ দেখুন)।

৫.১৫ প্রোডাকশন ইঞ্জিনিয়ারিং অনুশীলন

সেরা অনুশীলন: (১) Ansible-এর সংস্করণ নির্দিষ্ট করে রাখুন (Pin) — যেমন `pip install ansible-core==2.17.0` — যাতে দলের সবাই একই সংস্করণ ব্যবহার করে; (২) `~/ansible/` ডিরেক্টরিটি Git Repository হিসেবে রাখুন — Inventory, Playbook ও `ansible.cfg` সবই Version-Controlled হবে; (৩) প্রথমে `--check` ও `--limit` দিয়ে পরীক্ষা করুন, তারপর পুরো পরিবেশে চালান; (৪) Playbook-এর প্রতিটি Run-এর আউটপুট সংরক্ষণ করুন (লগ) — নিরীক্ষার সময় লাগবে; (৫) SSH Key Rotation-এর পরিকল্পনা রাখুন — নিয়মিত Key বদলান ও পুরনো Key সরান; (৬) Control Node-এর ব্যাকআপ রাখুন — Inventory, Playbook ও (এনক্রিপ্ট করা) Key। **অ্যান্টি-প্যাটার্ন:** "ল্যাপটপে Ansible, কোথাও ব্যাকআপ নেই" — ল্যাপটপ নষ্ট হলে সব হারিয়ে যায়; "সবাই Root-এর Key ব্যবহার করে" — কে কী করল, তা জানা যায় না; "Host Key Checking বন্ধ করে রাখা" — সুবিধার জন্য নিরাপত্তা বিসর্জন।

৫.১৬ অধ্যায়ের সারাংশ

এই অধ্যায়ে আমরা একটি চলমান Ansible পরিবেশ তৈরি করেছি: Ubuntu 24.04-এ Python Virtual Environment-এ Ansible Core 2.17+ ইনস্টল, `ansible --version` দিয়ে যাচাই, ed25519 SSH Key তৈরি ও `ssh-copy-id` দিয়ে Managed Node-গুলোতে বসানো, Host Key Verification-এর গুরুত্ব বোঝা (আর কেন তা বন্ধ করা নিষিদ্ধ), প্রথম Ad-hoc `ping` দিয়ে সংযোগ যাচাই, `ansible.cfg` সাজানো, আর প্রথম Playbook চালিয়ে তিনটি Web Server-এ Nginx ইনস্টল, সার্ভিস চালু, ফায়ারওয়াল-নিয়ম ও স্বাগত পাতা বসানো। দ্বিতীয়বার Playbook চালিয়ে `changed=0` দেখে Idempotency-ও যাচাই করেছি। এই পরিবেশই এই বইয়ের বাকি সব অধ্যায়ের ভিত্তি — Part II-এ আমরা এই ভিত্তির ওপর Playbook, Role, Variable ও Template-এর গভীরে যাব।

৫.১৭ জ্ঞান যাচাই — প্রশ্ন

1. pip ও apt — দুই পদ্ধতিতে Ansible ইনস্টল করার পার্থক্য কী? এই বইয়ে pip-কে প্রাধান্য দেওয়া হয়েছে কেন?

2. ed25519 Key Pair কী? Public Key ও Private Key-এর ভূমিকা আলাদা করে ব্যাখ্যা করুন — Private Key কেন কখনো নেটওয়ার্কে পাঠানো হয় না?
3. Host Key Verification কী, আর `StrictHostKeyChecking=no` ব্যবহার করা কেন বিপজ্জনক? সঠিক বিকল্পটি কী?
4. Ad-hoc কমান্ড ও Playbook-এর পার্থক্য কী? কখন কোনটি ব্যবহার করবেন?
5. প্রথম Playbook-এর দ্বিতীয় চালানোতে `changed=0` আসা কেন প্রত্যাশিত — আর তা না এলে কী করবেন?

৫.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. **পরিবেশ তৈরি:** ৫.১০-এর ধাপগুলো অনুসরণ করে আপনার নিজের Control Node ও অন্তত একটি Managed Node-এ সম্পূর্ণ পরিবেশ তৈরি করুন — `ansible webservers -m ping` সফল হওয়া পর্যন্ত।
2. **Playbook সম্প্রসারণ:** ৫.৯-এর `site.yml`-এ একটি নতুন Task যোগ করুন — যেমন `git` প্যাকেজ ইনস্টল করা বা `/etc/motd`-এ TechNova-র বার্তা লেখা — তারপর Playbook দুবার চালিয়ে Idempotency যাচাই করুন।
3. **নিরাপত্তা নিরীক্ষা:** আপনার পরিবেশে যাচাই করুন — (ক) Private Key-এর অনুমতি `600` কি না, (খ) `host_key_checking` চালু আছে কি না, (গ) Private Key Git-এ আছে কি না — প্রতিটি যাচাইয়ের ফল লিখুন।

PART II • DEVOPS ENGINEERING MASTERY

Ansible Core Engineering

Inventory, ad-hoc কমান্ড ও মডিউল, Playbook, variable ও fact, loop ও Jinja2 টেমপ্লেট, error handling ও execution control।

অধ্যায় 6 • PART II — Ansible Inventory ও Execution

Chapter 6 — Ansible Inventory — Static, Dynamic, and Host Groups

৬.১ ভূমিকা

Ansible-এর সবচেয়ে প্রথম প্রশ্নটি খুব সহজ, কিন্তু এর উত্তর ছাড়া আর কিছুই চলে না: "আপনি কোন মেশিনগুলোকে পরিচালনা করতে চান?" এই প্রশ্নের উত্তরই হলো **Inventory**। Inventory হলো Ansible-এর কাছে আপনার অবকাঠামোর মানচিত্র — কোন সার্ভার আছে, তাদের নাম কী, তারা কোন দলে পড়ে, তাদের IP ঠিকানা কী, SSH-এ কীভাবে পৌঁছাতে হবে, আর প্রতিটি দলের জন্য কোন সেটিং প্রযোজ্য। Playbook যত নিখুঁতই হোক, Inventory ভুল হলে Ansible ভুল মেশিনে ভুল কাজ করবে, অথবা কোনো মেশিনেই পৌঁছাতে পারবে না। TechNova-র উদাহরণ নিন: তাদের তিনটি Web সার্ভার (203.0.113.10–12), দুটি API সার্ভার (203.0.113.20–21), একটি ডেটাবেস সার্ভার (203.0.113.30) ও একটি Staging সার্ভার আছে। Inventory ছাড়া প্রতিবার কমান্ডে সাতটি IP টাইপ করতে হতো; Inventory থাকলে শুধু `ansible webservers -m ping` লিখলেই হয়। এই অধ্যায়ে আমরা Static Inventory-র দুটি ফরম্যাট — INI ও YAML — Host Group, `group_vars` / `host_vars`, Children Group, Inventory Variable, আর `ansible-inventory` দিয়ে যাচাই করার পদ্ধতি শিখব। Dynamic Inventory-র ধারণাটিও এখানে পরিচিত হবো; এর গভীর প্রয়োগ অধ্যায় ১৬-এ আসবে।

৬.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) INI ও YAML — দুই ফরম্যাটেই Static Inventory লিখতে; (২) Host Group তৈরি করে একাধিক সার্ভারকে এক নামে ডাকতে; (৩) `group_vars` ও `host_vars` দিয়ে দল ও একক হোস্টের ভেরিয়েবল আলাদা করতে; (৪) Children Group দিয়ে দলের ভেতরে দল সাজাতে; (৫) Inventory Variable যেমন `ansible_host`, `ansible_user`, `ansible_port` ব্যাখ্যা ও ব্যবহার করতে; (৬) `ansible-inventory --list` ও `--graph` দিয়ে Inventory যাচাই করতে; (৭) Dynamic Inventory কী ও কখন তা Static-এর চেয়ে ভালো, তা ব্যাখ্যা করতে।

৬.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১–৫ থেকে Ansible কী, Control Node ও Managed Node কী, SSH Key দিয়ে Password ছাড়া লগইন, YAML-এর Indentation ও `key: value` গঠন — এগুলো জানা থাকতে হবে। Linux-এ ফাইল তৈরি, `ssh` কমান্ড চালানো ও IP ঠিকানার প্রাথমিক ধারণা প্রয়োজন। এই অধ্যায়ের ল্যাবে আমরা Control Node-এ `nahid` ব্যবহারকারী হিসেবে কাজ করব; Managed Node-গুলো হলো 203.0.113.x পরিসরের কাল্পনিক ল্যাব সার্ভার। কোনো আসল সার্ভারে হাত দেওয়ার আগে নিজের ল্যাবে অনুশীলন করুন।

৬.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: একটি অটোমেশন টুলকে কাজ দিতে হলে তাকে তিনটি জিনিস জানাতে হয় — কী করতে হবে (Task), কীভাবে করতে হবে (Module) ও কোথায় করতে হবে (Target)। প্রথম দুটি Playbook-এ থাকে, তৃতীয়টি থাকে Inventory-তে। Inventory ছাড়া Ansible অন্ধ: সে জানে কী করতে হবে, কিন্তু জানে না কার ওপরে করতে হবে। Inventory-র মূল ধারণাটি একটি ঠিকানা-খাতার মতো — প্রতিটি সার্ভারের একটি নাম (Inventory Hostname), একটি ঠিকানা (`ansible_host`), আর কিছু বৈশিষ্ট্য (Variable)। নামটি মানুষের পড়ার জন্য, ঠিকানাটি সংযোগের জন্য, আর বৈশিষ্ট্যগুলো আচরণের জন্য। যেমন `web01` নামটি পড়তে সহজ, তার ঠিকানা 203.0.113.10, আর তার বৈশিষ্ট্য হতে পারে `http_port=80`। Group হলো এই খাতার সূচিপত্র: একই কাজ পাওয়া সার্ভারগুলোকে একসঙ্গে বেঁধে দেওয়া, যাতে "সব

Web সার্ভার" বললেই তিনটি সার্ভার বোঝায়। আর Variable-র স্তরবিন্যাস (Host বনাম Group) হলো উত্তরাধিকারের নিয়ম: যা সবার জন্য প্রযোজ্য তা Group-এ লিখুন, যা একজনের জন্য আলাদা তা Host-এ লিখুন। এই একটি নীতি — "সাধারণটি দলে, বিশেষটি হোস্টে" — মনে রাখলেই Inventory-র প্রায় সব জটিলতা মিটে যায়।

> **CONCEPT:** Inventory Hostname আর আসল DNS নাম এক জিনিস নয়। `web01` শুধু Ansible-এর ভেতরে ব্যবহৃত একটি লেবেল; আসল সংযোগ হয় `ansible_host`-এ দেওয়া IP বা DNS নাম দিয়ে। এই আলাদা করাটাই শক্তি: সার্ভারের IP বদলে গেলে শুধু `ansible_host` বদলালেই হয়, Playbook-এর কোথাও `web01` বদলাতে হয় না। আবার একই সার্ভারকে দুটি Inventory-তে দুটি নামে রাখাও সম্ভব — যেমন Staging-এ `web01-staging` আর Production-এ `web01-prod` — যদিও বাস্তবে তা দুটি আলাদা মেশিন হওয়াই উচিত।

৬.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

INI ফরম্যাট — সহজ ও সরাসরি

INI ফরম্যাট সবচেয়ে পুরনো ও পড়তে সহজ। Group-এর নাম বন্ধনীতে `[webservers]` লিখে তার নিচে হোস্টগুলো লেখা হয়। একই লাইনে হোস্টের Variable দেওয়া যায়। বিশেষ Group `all`-এ সব হোস্ট পড়ে, আর `ungrouped`-এ পড়ে যারা কোনো Group-এ নেই। INI-তে Children Group লেখা হয় `[parent:children]` দিয়ে, আর Group Variable লেখা হয় `[group:vars]` দিয়ে।

YAML ফরম্যাট — কাঠামোবদ্ধ ও সুপারিশকৃত

YAML Inventory-তে সবকিছু Key-Value কাঠামোতে লেখা হয়: `all:`-এর ভেতরে `children:` ও `hosts:`। YAML ফরম্যাট বড় অবকাঠামোয় ভালো, কারণ Variable-গুলো Nested Dictionary হিসেবে সাজানো যায়, আর Playbook-এর YAML-এর সঙ্গে শৈলীর মিল থাকে। Ansible 2.17-এ দুটি ফরম্যাটই সমানভাবে সমর্থিত; নতুন প্রকল্পে YAML-কেই সুপারিশ করা হয়। তবে মনে রাখবেন: YAML Inventory ফাইলের নাম `.yml` বা `.yaml` দিয়ে শেষ হওয়া উচিত, আর প্রথম লাইনে `all:` থাকা চাই।

group_vars ও host_vars — Variable-র ঘর

Inventory ফাইলের ভেতরে Variable লেখা গেলেও বড় প্রকল্পে তা জঞ্জাল তৈরি করে। তাই প্রচলিত নিয়ম হলো Inventory ফাইলের পাশে `group_vars/` ও `host_vars/` ডিরেক্টরি রাখা: `group_vars/webservers.yml`-এ সব Web সার্ভারের Variable, `host_vars/web01.yml`-এ শুধু `web01`-এর Variable। Group-এর নামে ফাইল, হোস্টের নামে ফাইল — এই নাম মিললেই Ansible নিজে থেকে তা পড়ে নেয়। একই Group-এর Variable একাধিক ফাইলে ভাগ করা যায় (`webservers.yml`, `webservers-extra.yml`), তবে নামের সংঘাত এড়াতে একটি ফাইলেই রাখা ভালো।

Inventory Variable — সংযোগের চাবিকাঠি

কয়েকটি Variable Ansible-এর আচরণ সরাসরি নিয়ন্ত্রণ করে: `ansible_host` (সংযোগের IP/DNS), `ansible_port` (SSH পোর্ট, ডিফল্ট 22), `ansible_user` (SSH ব্যবহারকারী), `ansible_ssh_private_key_file` (Private Key-এর পথ), `ansible_python_interpreter` (Managed Node-এ Python-এর পথ, যেমন `/usr/bin/python3`)। এগুলোকে বলে Connection Variable। এছাড়া যেকোনো নিজের Variable — যেমন `http_port`, `app_env` — Playbook-এ ব্যবহার করা যায়। সতর্কতা: Password জাতীয় Variable (`ansible_password`, `ansible_become_password`) কখনো সরল Inventory-তে লিখবেন না; সেগুলোর জন্য Ansible Vault ব্যবহার করুন (অধ্যায় ১৩)।

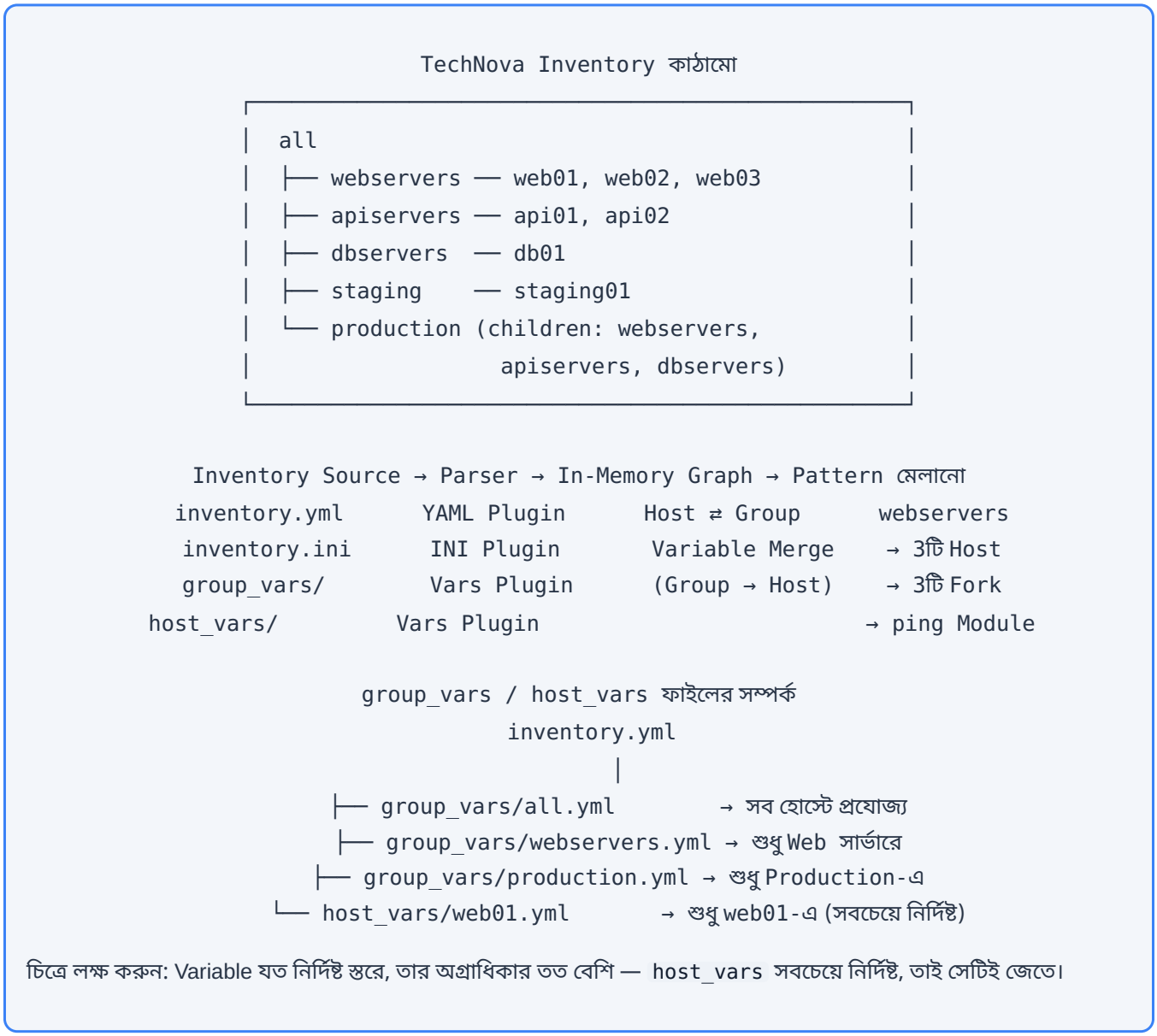
বিষয়	INI ফরম্যাট	YAML ফরম্যাট
Group ঘোষণা	`[webservers]`	`webservers: hosts: ...`
Host Variable	একই লাইনে `key=value`	`hosts: web01: ansible_host: ...`
Children Group	`[parent:children]`	`children:` কী-এর ভেতরে
Group Variable	`[group:vars]` সেকশন	Group-এর ভেতরে `vars:` কী
পড়ার সহজতা	ছোট Inventory-তে খুব সহজ	বড় Inventory-তে কাঠামো পরিষ্কার
Nested Variable	কঠিন, সমতল (Flat) লিখতে হয়	সহজ, Dictionary/List লেখা যায়
সুপারিশ	দ্রুত পরীক্ষা, ছোট ল্যাব	Production ও দলগত প্রকল্প

৬.৬ অভ্যন্তরীণ কার্যপ্রণালি

আপনি যখন `ansible webservers -m ping` চালান, ভেতরে যা ঘটে তা ধাপে ধাপে এরকম: (১) Ansible প্রথমে Inventory Source খুঁজে বের করে — `-i` ফ্ল্যাগে দেওয়া ফাইল/ডিরেক্টরি, অথবা `ansible.cfg`-এ লেখা `inventory` পথ, অথবা ডিফল্ট `/etc/ansible/hosts`। (২) Inventory Plugin ফাইলটি Parse করে: `.ini` দেখলে INI Plugin, `.yml` দেখলে YAML Plugin চালু হয়; ফাইলের Extension-ই Plugin বেছে দেয়। (৩) Parser প্রতিটি Host, Group, Children সম্পর্ক ও Variable পড়ে একটি In-Memory Inventory Graph তৈরি করে — প্রতিটি Host জানে সে কোন Group-গুলোতে আছে, প্রতিটি Group জানে তার সদস্য কারা। (৪) Pattern `webservers` মেলানো হয়: Group-এর সব Host-এর তালিকা বের হয়; Pattern-এ `webservers:!dbservers` বা `webservers[0]`-এর মতো জটিল নির্বাচনও এখানেই সমাধান হয়। (৫) প্রতিটি Host-এর জন্য Variable একত্রিত (Merge) করা হয় — Group Variable-র ওপরে Host Variable বসে; একই Variable দুই Group-এ থাকলে কোনটি জিতবে তা Precedence নিয়মে ঠিক হয় (বিস্তারিত অধ্যায় ৯-এ)। (৬) তারপর প্রতিটি Host-এর জন্য আলাদা Fork-এ সংযোগ তৈরি হয় (ডিফল্ট ৫টি সমান্তরাল Fork) ও Module চালানো হয়। লক্ষ করুন: Inventory Parse করার সময় Ansible কোনো সার্ভারে সংযোগ করে না — ভুল IP থাকলেও Inventory যাচাই সফল হবে, ব্যর্থতা ধরা পড়বে `ping` চালানোর সময়। এজন্য `ansible-inventory --list` দিয়ে গঠন যাচাই আর `ansible all -m ping` দিয়ে সংযোগ যাচাই — দুটি আলাদা ধাপ।

> **WARNING:** Inventory ফাইলে একই Host দুটি Group-এ থাকলে তার Variable দুই Group থেকে আসে — আর Group-এর নামের Alphabetical ক্রমে পরে থাকা Group-এর Variable জেতে। অর্থাৎ `webservers` ও `production` দুটিতেই `http_port` থাকলে `production`-এর মানটি প্রযোজ্য হবে, কারণ 'p' > 'w' নয় — আসলে Alphabetical ক্রমে `production` (p) `webservers` (w)-এর আগে আসে, তাই `webservers`-এর মান জিতবে। এই নিয়ম মনে রাখা কঠিন বলেই একই Variable দুই Group-এ না লেখাই নিরাপদ; বরং `group_vars/all.yml`-এ ডিফল্ট রেখে বিশেষ Group-এ শুধু ব্যতিক্রম লিখুন।

৬.৭ আর্কিটেকচার ডায়াগ্রাম



চিত্রে লক্ষ করুন: Variable যত নির্দিষ্ট স্তরে, তার অগ্রাধিকার তত বেশি — `host_vars` সবচেয়ে নির্দিষ্ট, তাই সেটিই জেতে।

৬.৮ ব্যবহারিক উদাহরণ

নিচে TechNova-র ল্যাবের জন্য INI ও YAML — দুই ফরম্যাটেই একই Inventory লেখা হলো। দুটি ফাইল একসঙ্গে ব্যবহার করবেন না; যেকোনো একটি বেছে নিন।

```
# inventory.ini — INI ফরম্যাট (ছোট ল্যাবের জন্য)
[webservers]
web01 ansible_host=203.0.113.10 ansible_user=nahid
web02 ansible_host=203.0.113.11 ansible_user=nahid
web03 ansible_host=203.0.113.12 ansible_user=nahid

[apiservers]
api01 ansible_host=203.0.113.20 ansible_user=nahid
api02 ansible_host=203.0.113.21 ansible_user=nahid

[dbservers]
db01 ansible_host=203.0.113.30 ansible_user=nahid
```

```
[staging]
staging01 ansible_host=203.0.113.40 ansible_user=nahid

# Children group: production = web + api + db
[production:children]
webservers
apiservers
dbservers

# Group variable: সব webservers-এ http_port 80
[webservers:vars]
http_port=80

# সব হোস্টের জন্য সাধারণ সেটিং
[all:vars]
ansible_python_interpreter=/usr/bin/python3
```

```
# inventory.yml — YAML ফরম্যাট (সুপারিশকৃত)
all:
  children:
    webservers:
      hosts:
        web01:
          ansible_host: 203.0.113.10
        web02:
          ansible_host: 203.0.113.11
        web03:
          ansible_host: 203.0.113.12
      vars:
        http_port: 80
    apiservers:
      hosts:
        api01:
          ansible_host: 203.0.113.20
        api02:
          ansible_host: 203.0.113.21
    dbservers:
      hosts:
        db01:
          ansible_host: 203.0.113.30
    staging:
      hosts:
        staging01:
          ansible_host: 203.0.113.40
    production:
      children:
        webservers:
        apiservers:
        dbservers:
  vars:
    ansible_user: nahid
    ansible_python_interpreter: /usr/bin/python3
```

দুটি ফরম্যাটের ফল একই — সাতটি হোস্ট, পাঁচটি Group। পার্থক্য শুধু লেখার শৈলীতে: INI-তে Group Variable আলাদা `:vars` সেকশনে, YAML-এ Group-এর ভেতরেই `vars:` কী-তে।

৬.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

একটি বাস্তব প্রকল্পের ডিরেক্টরি কাঠামোসহ সম্পূর্ণ Inventory সেটআপ নিচে দেওয়া হলো। `ansible.cfg` দিয়ে Inventory-র অবস্থান স্থির করে দিলে প্রতিবার `-i` লিখতে হয় না।

```
# প্রকল্পের কাঠামো
~/technova-ansible/
├── ansible.cfg
├── inventory.yml
├── group_vars/
│   ├── all.yml
│   ├── webservers.yml
│   ├── apiservers.yml
│   ├── dbservers.yml
│   └── production.yml
└── host_vars/
    ├── web01.yml
    └── db01.yml

# ansible.cfg
[defaults]
inventory = ./inventory.yml
host_key_checking = False
forks = 5
remote_user = nahid

# group_vars/all.yml — সবার জন্য ডিফল্ট
ansible_user: nahid
ansible_python_interpreter: /usr/bin/python3
app_env: production
timezone: Asia/Dhaka

# group_vars/webservers.yml
http_port: 80
https_port: 443
nginx_worker_connections: 1024

# group_vars/apiservers.yml
api_port: 4000
node_env: production

# group_vars/dbservers.yml
db_port: 27017
db_name: technova

# group_vars/production.yml
backup_enabled: true
monitoring_enabled: true

# host_vars/web01.yml — শুধু web01-এর ব্যতিক্রম
http_port: 8080
# ব্যাখ্যা: web01-এ পরীক্ষার জন্য 8080 পোর্ট; বাকি web সার্ভারে 80 থাকবে।

# host_vars/db01.yml
db_name: technova_prod
# ব্যাখ্যা: Group-এর technova নামটি এই হোস্টে technova_prod দিয়ে ওভাররাইড হলো।
```

৬.১০ ধাপে ধাপে বাস্তবায়ন

ধাপ ১ — প্রকল্প ডিরেক্টরি তৈরি: Control Node-এ `mkdir -p ~/technova-ansible/group_vars ~/technova-ansible/host_vars` চালিয়ে কাঠামো তৈরি করুন। **ধাপ ২ — Inventory লিখুন:** উপরের `inventory.yml` ফাইলটি তৈরি করে সাতটি হোস্টের IP লিখুন; IP ভুল হলে পরে সংযোগ ব্যর্থ হবে, তাই দুবার মিলিয়ে নিন। **ধাপ ৩ — ansible.cfg লিখুন:** Inventory-র পথ স্থির করুন, যাতে প্রতিটি কমান্ডে `-i` না লিখতে হয়। **ধাপ ৪ — group_vars/host_vars লিখুন:** প্রতিটি Group-এর Variable আলাদা ফাইলে রাখুন; ফাইলের নাম Group-এর নামের সঙ্গে হুবহু মিলতে হবে। **ধাপ ৫ — গঠন যাচাই:** `ansible-inventory --list` চালিয়ে JSON আউটপুটে সব হোস্ট ও Variable দেখুন; `ansible-inventory --graph` চালিয়ে গাছের মতো কাঠামো দেখুন। **ধাপ ৬ — সংযোগ যাচাই:** `ansible all -m ping` চালান — সব হোস্টে `SUCCESS` আসা উচিত। **ধাপ ৭ — Pattern পরীক্ষা:** `ansible webservers --list-hosts` দিয়ে দেখুন কোন তিনটি হোস্ট মিলছে; `ansible 'production:!dbservers' --list-hosts` দিয়ে বাদ দেওয়ার Pattern পরীক্ষা করুন। কোনো ধাপে ভুল হলে পরের ধাপে যাবেন না — Inventory-র ভুল Playbook-এ গিয়ে বহুগুণ বড় হয়।

৬.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

```
$ ansible-inventory --graph
# চিত্রণমূলক আউটপুট:
@all:
  |--@production:
  |  |--@apiservers:
  |  |  |--api01
  |  |  |--api02
  |  |--@dbservers:
  |  |  |--db01
  |  |--@webservers:
  |  |  |--web01
  |  |  |--web02
  |  |  |--web03
  |--@staging:
  |  |--staging01
  |--@ungrouped:

$ ansible webservers --list-hosts
# চিত্রণমূলক আউটপুট:
  hosts (3):
    web01
    web02
    web03

$ ansible all -m ping
# চিত্রণমূলক আউটপুট:
web01 | SUCCESS => {"changed": false, "ping": "pong"}
web02 | SUCCESS => {"changed": false, "ping": "pong"}
db01  | SUCCESS => {"changed": false, "ping": "pong"}
# ব্যাখ্যা: --graph-এ @ চিহ্ন মানে Group, তার নিচে সদস্য। --list-hosts
# কোনো সংযোগ ছাড়াই Pattern-এর ফল দেখায় — Inventory ভুল থাকলে এখানেই
# ধরা পড়ে। ping-এ "pong" মানে SSH সংযোগ ও Python কাজ করছে।
```

`ansible-inventory --list`-এর JSON আউটপুটে প্রতিটি হোস্টের নিচে তার সব Variable দেখা যায় — এটিই Variable Merge যাচাইয়ের সবচেয়ে নির্ভরযোগ্য উপায়। যেমন `web01`-এর নিচে `http_port: 8080` দেখা গেলে বুঝবেন `host_vars` ঠিকমতো Group Variable-কে ওভাররাইড করেছে।

৬.১২ সাধারণ ভুল

(১) **YAML Indentation ভুল:** `hosts:`-এর নিচে হোস্ট দুই Space বেশি Indent না করলে Ansible ফাইলটি পড়তেই পারবে না; Tab ব্যবহার করবেন না — শুধু Space। (২) **Group নামের সঙ্গে ফাইলের নামের অমিল:** `group_vars/webserver.yml` (একবচন) লিখলে Group `webservers` (বহুবচন) তা পড়বে না — নাম হুবহু মিলতে হবে। (৩) **Inventory-তে Password লেখা:** `ansible_password=secret123` লিখলে তা Git-এ চলে যায় ও সবাই দেখে — Vault ব্যবহার করুন। (৪) **একই Host দুই Inventory ফাইলে ভিন্ন Variable-সহ:** দুটি Inventory Source একসঙ্গে দিলে কোনটি জিতবে তা বোঝা কঠিন হয়; একটি Source-ই রাখুন। (৫) **`all` Group-এ সরাসরি Host লেখা:** `all:`-এর নিচে `hosts:` লিখলে সেই হোস্ট কোনো Group-এ পড়ে না — Staging হোস্ট ভুলে Production Playbook-এ চলে যেতে পারে। (৬) **IP-র বদলে Hostname, অথচ DNS নেই:** ল্যাবে `web01`-কে `ansible_host` ছাড়া লিখলে Ansible DNS খুঁজবে ও ব্যর্থ হবে।

৬.১৩ সমস্যা সমাধান (Troubleshooting)

সমস্যা: `ansible-inventory --list` খালি দেখায়। কারণ সাধারণত `ansible.cfg`-এ Inventory-র পথ ভুল, অথবা ফাইলের Extension অচেনা। সমাধান: `ansible-inventory --list -i inventory.yml -v` চালিয়ে Verbose আউটপুটে কোন ফাইল পড়া হচ্ছে দেখুন। **সমস্যা: `WARNING: Could not match supplied host pattern`।** Pattern-এর Group নামের বানান ভুল, অথবা Inventory ফাইলটি লোডই হয়নি। `ansible-inventory --graph` দিয়ে Group-এর নাম যাচাই করুন। **সমস্যা: Variable-র মান প্রত্যাশিত নয়।** `ansible-inventory --host web01` চালিয়ে ওই হোস্টের চূড়ান্ত Variable দেখুন — কোন স্তর থেকে মানটি এসেছে তা বোঝা যাবে। **সমস্যা: ping-এ `UNREACHABLE`।** এটি Inventory গঠনের ভুল নয়, সংযোগের ভুল — IP, SSH Key, Firewall যাচাই করুন; `ssh nahid@203.0.113.10` হাতে চালিয়ে দেখুন। **সমস্যা: YAML Parse Error।** `python3 -c "import yaml; yaml.safe_load(open('inventory.yml'))"` দিয়ে ফাইলটি আলাদাভাবে যাচাই করুন; সাধারণত Colon-এর পরে Space না থাকা বা Tab থাকা দায়ী।

৬.১৪ নিরাপত্তা বিবেচনা

Inventory ফাইলে কখনো আসল Password, API Key বা Private Key-র বিষয়বস্তু লিখবেন না — এগুলো Git Repository-তে চলে গেলে যে কেউ তা পড়তে পারে। SSH-এর জন্য Password-এর বদলে Key-Based Authentication ব্যবহার করুন, আর Private Key ফাইলের Permission `600` রাখুন। Inventory ফাইলটি Repository-তে রাখা নিরাপদ, কারণ তাতে শুধু IP ও ব্যবহারকারীর নাম থাকে; কিন্তু `host_vars`-এ Secret থাকলে সেই ফাইলটি Ansible Vault দিয়ে Encrypt করুন। আরও একটি বিষয়: Production ও Staging Inventory আলাদা ফাইল বা আলাদা ডিরেক্টরিতে রাখুন (`inventories/production/` বনাম `inventories/staging/`) — একই Inventory-তে দুই পরিবেশ মেশালে ভুল Pattern-এ Staging Playbook Production-এ চলার ঝুঁকি থাকে। `host_key_checking = False` ল্যাবে সুবিধাজনক হলেও Production-এ তা বন্ধ রাখা উচিত নয় — Man-in-the-Middle আক্রমণ ঠেকাতে Host Key যাচাই চালু রাখুন, আর প্রথম সংযোগে Key গ্রহণ করুন।

৬.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

Production-এ Inventory পরিচালনার নিয়মগুলো: (১) **পরিবেশভিত্তিক আলাদা Inventory:** `inventories/production/inventory.yml`, `inventories/staging/inventory.yml` — আলাদা `group_vars` সহ; Deploy-এর সময় `-i inventories/production/` স্পষ্ট করে দিন। (২) **Version Control:** Inventory Git-এ রাখুন; প্রতিটি পরিবর্তন Pull Request-এর মাধ্যমে যাক, যাতে ভুল IP যোগ হওয়ার আগেই আরেকজন দেখে। (৩) **Dynamic**

Inventory-র দিকে যাত্রা: সার্ভারের সংখ্যা ৫০ ছাড়ালে হাতে লেখা Inventory রক্ষণাবেক্ষণ অসম্ভব হয় — তখন Cloud Provider-এর Inventory Plugin (যেমন AWS EC2, Azure) ব্যবহার করুন; অধ্যায় ১৬-এ এর বিস্তারিত আছে। (৪) **নামের নিয়ম:** `<ভূমিকা><সংখ্যা>-<পরিবেশ>` — যেমন `web01-prod` — নাম দেখেই ভূমিকা ও পরিবেশ বোঝা যায়। (৫) **নিয়মিত যাচাই:** CI Pipeline-এ `ansible-inventory --list` ও `ansible all -m ping --check` চালিয়ে Inventory-র সুস্থতা পরীক্ষা করুন; কোনো হোস্ট Unreachable হলে Alert যাক।

৬.১৬ অধ্যায়ের সারাংশ

Inventory হলো Ansible-এর মানচিত্র: কোন সার্ভার, কোন দলে, কী ঠিকানায়, কী বৈশিষ্ট্যসহ। INI ফরম্যাট ছোট ল্যাবে সহজ, YAML ফরম্যাট বড় প্রকল্পে কাঠামোবদ্ধ ও সুপারিশকৃত। Host Group একই কাজের সার্ভারগুলোকে এক নামে বাঁধে, Children Group দলের ভেতরে দল সাজায়, আর `group_vars` / `host_vars` Variable-কে "সাধারণটি দলে, বিশেষটি হোস্টে" নীতিতে ভাগ করে। `ansible-inventory --list` ও `--graph` দিয়ে গঠন যাচাই করা যায়, আর `ping` দিয়ে সংযোগ যাচাই করা যায় — দুটি আলাদা ধাপ। Dynamic Inventory হলো পরের ধাপ: Inventory যখন ফাইল নয়, Cloud থেকে আসা জীবন্ত তালিকা — তার গভীরে যাব অধ্যায় ১৬-এ।

৬.১৭ জ্ঞান যাচাই

১. Inventory Hostname ও `ansible_host`-এর পার্থক্য কী? IP বদলে গেলে কোনটি বদলাতে হয়? ২. INI ফরম্যাটে Children Group কীভাবে লেখা হয়? YAML ফরম্যাটে তার সমতুল্য গঠনটি লিখুন। ৩. `group_vars/webservers.yml` ফাইলটি কখন পড়া হয়, আর নামের বানান ভুল হলে কী হয়? ৪. `ansible-inventory --list` ও `ansible all -m ping` — কোনটি গঠন যাচাই করে, কোনটি সংযোগ যাচাই করে? ৫. একই Variable Group ও Host — দুই স্তরেই থাকলে কোনটি জেতে, আর কেন এই নিয়মটি গুরুত্বপূর্ণ?

৬.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

নিজের ল্যাবে TechNova-র Inventory তৈরি করুন: (ক) YAML ফরম্যাটে `inventory.yml` লিখুন — `webservers` (২টি), `apiservers` (১টি), `dbservers` (১টি) ও `production` Children Group-সহ; (খ) `group_vars/all.yml`-এ `ansible_user: nahid` ও `ansible_python_interpreter: /usr/bin/python3` রাখুন; (গ) `group_vars/webservers.yml`-এ `http_port: 80` ও `host_vars`-এ একটি হোস্টে `http_port: 8080` দিয়ে Override প্রমাণ করুন; (ঘ) `ansible-inventory --graph` ও `ansible-inventory --host <হোস্ট>`-এর আউটপুট সংরক্ষণ করুন; (ঙ) একটি ভুল করুন — Group নামের বানান ভুল লিখে `--graph` চালান — তারপর ঠিক করে দুটি আউটপুটের পার্থক্য নোট করুন। জমা দিন: Inventory ফাইল, `group_vars` / `host_vars` ফাইল ও যাচাইয়ের আউটপুট।

অধ্যায় 7 • PART II — Ansible Inventory ও Execution

Chapter 7 — Ansible Ad-hoc Commands and Modules

৭.১ ভূমিকা

Playbook লেখার আগেই Ansible-কে চেনার সবচেয়ে দ্রুত উপায় হলো **Ad-hoc Command** — এক লাইনের কমান্ড, যা কোনো ফাইল ছাড়াই এক বা একাধিক সার্ভারে একটি কাজ করে ফেলে। সার্ভারটি বেঁচে আছে কি না দেখা, কোনো Package ইনস্টল আছে কি না জানা, একটি ফাইল কপি করা, একটি Service Restart করা — এসব দ্রুত কাজের জন্য Playbook লেখা বাড়তি ঝামেলা; Ad-hoc Command সেখানেই জয়ী। কিন্তু Ad-hoc Command শুধু শর্টকাট নয় — এটি Ansible-এর মূল একক **Module**-কে বোঝার শ্রেণিকক্ষ। প্রতিটি Ad-hoc Command-এর ভেতরে একটি Module চলে, আর Module-ই হলো Ansible-এর আসল কাজের হাতিয়ার: `ping` সংযোগ পরীক্ষা করে, `apt` Package পরিচালনা করে, `copy` ফাইল পাঠায়, `service` Service চালায়। এই অধ্যায়ে আমরা Ad-hoc Command-এর গঠন, সবচেয়ে প্রয়োজনীয় Module-গুলো, `command` বনাম `shell` বনাম `raw`-এর সূক্ষ্ম পার্থক্য, আর `ansible-doc` দিয়ে যেকোনো Module-এর নথি পড়ার অভ্যাস গড়ে তুলব।

৭.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Ad-hoc Command-এর গঠন (`ansible <pattern> -m <module> -a "<args>"`) ব্যাখ্যা ও ব্যবহার করতে; (২) `ping` ও `setup` Module দিয়ে সংযোগ ও তথ্য যাচাই করতে; (৩) `command`, `shell` ও `raw`-এর পার্থক্য বুঝে সঠিকটি বেছে নিতে; (৪) `apt`, `service`, `copy`, `file` ও `user` Module দিয়ে দৈনন্দিন কাজ করতে; (৫) `ansible-doc` দিয়ে কোনো Module-এর Argument, উদাহরণ ও Return Value পড়তে; (৬) Ad-hoc Command কখন যথেষ্ট আর কখন Playbook লেখা উচিত, তা সিদ্ধান্ত নিতে।

৭.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৬-এর Inventory জানা থাকতে হবে — Pattern কীভাবে Host বেছে নেয়, `ansible_user` ও `ansible_host` কী। Linux-এ Package (`apt`), Service (`systemctl`), ফাইলের Permission (`chmod` / `chown`) ও ব্যবহারকারী (`useradd`) পরিচালনার প্রাথমিক অভিজ্ঞতা থাকলে Module-গুলোর কাজ বুঝতে সুবিধা হবে। YAML এখনো গভীরে লাগবে না — Ad-hoc Command-এ Argument সরল `key=value` জোড়ায় লেখা হয়।

৭.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: দূরের সার্ভারে কাজ করাতে হলে তিনটি সমস্যা মেটাতে হয় — সংযোগ (কীভাবে পৌঁছাব), নির্দেশ (কী করতে বলব) ও নিশ্চয়তা (কাজটি হয়েছে কি না জানব কীভাবে)। SSH সংযোগের সমস্যা মেটায়; কিন্তু নির্দেশ যদি হয় কাঁচা Shell কমান্ড (`apt install nginx`), তাহলে দ্বিতীয়বার চালালে কী হবে? প্রথমবার ইনস্টল হবে, দ্বিতীয়বার "already installed" বলবে — কিন্তু Exit Code দেখে আপনি বুঝবেন না কিছু বদলেছে কি না। Module এই সমস্যার সমাধান: Module হলো একটি ঘোষণামূলক (Declarative) নির্দেশ — আপনি বলেন কী অবস্থা চাই (`nginx` Package-টি `present` থাকুক), Module নিজে বর্তমান অবস্থা দেখে সিদ্ধান্ত নেয় কিছু করতে হবে কি না, আর ফিরে জানায় কিছু বদলেছে কি না (`changed: true/false`)। এটাই **Idempotency** — একই নির্দেশ দশবার চালালেও ফল একই থাকে, আর শুধু প্রথমবারই পরিবর্তন ঘটে। Ad-hoc Command হলো এই Module-কে এক লাইনে ডাকার উপায়; Playbook হলো একই Module-গুলোকে ফাইলে সাজিয়ে বারবার চালানোর উপায়। তাই Module বুঝলে Playbook অর্ধেক বোঝা হয়ে যায়।

> **CONCEPT:** Module একটি ছোট Python প্রোগ্রাম, যা Ansible Control Node থেকে Managed Node-এ পাঠিয়ে চালানো হয়। Control Node Module-টির কোড ও Argument একত্র করে Managed Node-এ পাঠায়, সেখানে Python তা চালায়, আর ফল JSON হিসেবে ফিরে আসে। এই কারণেই Managed Node-এ Python 3.12 থাকা জরুরি — Module Python ছাড়া চলতে পারে না। ব্যতিক্রম শুধু `raw` Module, যা SSH দিয়ে সরাসরি কমান্ড চালায় ও Python লাগে না — তাই Python না থাকা বা ভাঙা সার্ভার মেরামতের কাজে `raw` ব্যবহার করা হয়।

৭.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Ad-hoc Command-এর গঠন

সাধারণ গঠন: `ansible <pattern> -m <module> -a "<arguments>"`। Pattern বলে কাদের ওপরে (যেমন `webservers`, `all`, `web01`); `-m` বলে কোন *Module*; `-a` বলে কী *Argument*। Module না দিলে ডিফল্ট Module হলো `command`। দরকারি ফ্ল্যাগ: `-b` (become — sudo দিয়ে চালানো), `-K` (become Password চাওয়া), `-f 10` (Fork সংখ্যা — কয়টি সার্ভারে সমান্তরালে চালাবে), `-v` (Verbose আউটপুট), `--check` (শুধু অনুমান — আসলে কিছু না করে কী বদলাত তা দেখানো)।

ping ও setup — যাচাইয়ের দুই হাতিয়ার

`ping` Module কোনো ICMP Ping নয় — এটি SSH সংযোগ ও Python কাজ করছে কি না পরীক্ষা করে `pong` ফেরত দেয়। `setup` Module Managed Node সম্পর্কে সব তথ্য (Facts) সংগ্রহ করে — OS, IP, মেমরি, CPU, Disk — যা অধ্যায় ৯-এ বিস্তারিত আসবে। `setup -a "filter=ansible_distribution*"` দিয়ে শুধু OS-সংক্রান্ত তথ্য নেওয়া যায়।

command বনাম shell বনাম raw

তিনটি Module-ই কমান্ড চালায়, কিন্তু তাদের ক্ষমতা আলাদা। `command` সরাসরি প্রোগ্রাম চালায় — কোনো Shell নেই, তাই Pipe (`|`), Redirect (`>`), Variable (`$HOME`), Glob (`*`) কাজ করে না; কিন্তু তা নিরাপদ ও অনুমানযোগ্য। `shell` কমান্ডটিকে `/bin/sh`-এর ভেতর দিয়ে চালায় — Pipe, Redirect, `&&` সব কাজ করে; কিন্তু Shell Injection-এর ঝুঁকি থাকে। `raw` SSH-এর কাঁচা চ্যানেল দিয়ে চালায় — Python ছাড়াই চলে, কিন্তু Idempotency নেই ও `changed` সবসময় `true` দেখায়। নিয়ম: প্রথমে ভাবুন কোনো Declarative Module (যেমন `apt`, `copy`) দিয়ে কাজটি হয় কি না; না হলে `command`; Pipe লাগলে `shell`; Python ভাঙা থাকলে শুধু তখনই `raw`।

দৈনন্দিন কাজের Module-গুলো

`apt` (Ubuntu/Debian): `name=nginx state=present update_cache=yes` — Package ইনস্টল করে; `state=absent` সরিয়ে দেয়, `state=latest` আপগ্রেড করে। `service`: `name=nginx state=started enabled=yes` — Service চালু করে ও Boot-এ চালু থাকা নিশ্চিত করে; `state=restarted/reloaded/stopped` অন্য অবস্থাগুলো। `copy`: `src=/local/file dest=/remote/file owner=nahid mode=0644` — Control Node থেকে ফাইল পাঠায়; `content="..."` দিয়ে সরাসরি লেখাও যায়; ফাইল একই থাকলে `changed: false`। `file`: `path=/var/www state=directory mode=0755` — ডিরেক্টরি/ফাইল তৈরি, Permission ও মালিকানা ঠিক করে; `state=absent` মুছে দেয়; `state=link src=...` Symlink বানায়। `user`: `name=deploy state=present groups=www-data shell=/bin/bash` — ব্যবহারকারী তৈরি করে; `state=absent remove=yes` মুছে দেয়।

Module	কাজ	Idempotent?	সাধারণ Argument

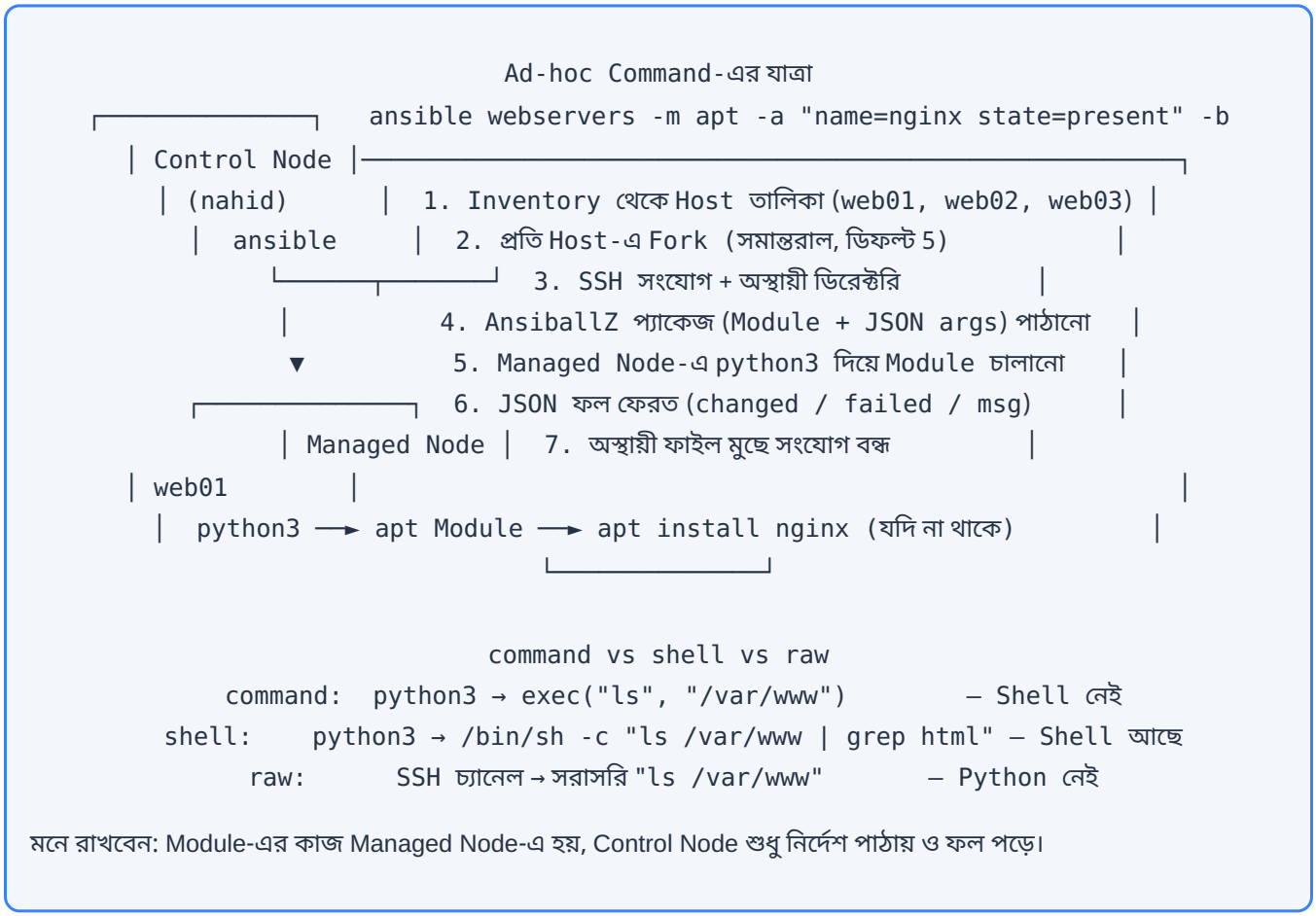
ping	সংযোগ ও Python যাচাই	হ্যাঁ (কিছু বদলায় না)	কোনো Argument লাগে না
setup	Facts সংগ্রহ	হ্যাঁ	`filter=ansible_*`
command	Shell ছাড়া কমান্ড	না (সবসময় changed)	`cmd, chdir, creates, removes`
shell	Shell-এর ভেতরে কমান্ড	না	`cmd, chdir, executable`
raw	Python ছাড়া কাঁচা কমান্ড	না	সরাসরি কমান্ড
apt	Package পরিচালনা	হ্যাঁ	`name, state, update_cache`
service	Service পরিচালনা	হ্যাঁ	`name, state, enabled`
copy	ফাইল পাঠানো	হ্যাঁ	`src, dest, mode, owner`
file	ফাইল/ডিরেক্টরি অবস্থা	হ্যাঁ	`path, state, mode, owner`
user	ব্যবহারকারী পরিচালনা	হ্যাঁ	`name, state, groups, shell`

৭.৬ অভ্যন্তরীণ কার্যপ্রণালি

একটি Ad-hoc Command চালালে ভেতরে যা ঘটে: (১) Ansible Inventory থেকে Pattern-মেলানো Host-গুলোর তালিকা বের করে। (২) প্রতিটি Host-এর জন্য একটি Fork তৈরি হয় (ডিফল্ট ৫টি)। (৩) Control Node SSH দিয়ে Managed Node-এ সংযোগ করে (ডিফল্ট `ssh` Connection Plugin) ও একটি অস্থায়ী ডিরেক্টরি বানায় (যেমন `~/.ansible/tmp/`)। (৪) Module-এর Python কোড (AnsiballZ নামের একটি সংকুচিত প্যাকেজ) ও JSON Argument SFTP/SCP দিয়ে পাঠানো হয়। (৫) Managed Node-এ `python3` দিয়ে Module চালানো হয়; Module নিজে `apt`, `systemctl` বা ফাইল অপারেশন করে, বর্তমান অবস্থা ও কাঙ্ক্ষিত অবস্থার তুলনা করে। (৬) ফল JSON হিসেবে ফিরে আসে — `changed`, `failed`, `msg`, Module-নির্দিষ্ট তথ্যসহ। (৭) Control Node ফলটি মানুষের পড়ার মতো করে সাজিয়ে দেখায়, অস্থায়ী ফাইল মুছে দেয় ও সংযোগ বন্ধ করে। লক্ষ করুন: `command` ও `shell` Module-ও এই একই পথে যায় — পার্থক্য শুধু Managed Node-এ কমান্ডটি কীভাবে চালানো হয় তাতে। আর `raw` Module এই পথটি বাইপাস করে: সে Module পাঠায় না, SSH চ্যানেলে সরাসরি কমান্ড লিখে আউটপুট পড়ে নেয়।

> **TIP:** যেকোনো Module-এর নথি পড়তে `ansible-doc <module>` চালান — যেমন `ansible-doc apt`। এতে Argument-এর তালিকা, কোনটি আবশ্যক, ডিফল্ট মান কী, উদাহরণ ও Return Value — সব থাকে। `ansible-doc -l` দিয়ে সব Module-এর তালিকা দেখা যায়, আর `ansible-doc -s apt` দিয়ে সংক্ষিপ্ত সারাংশ পাওয়া যায়। নতুন Module শেখার সময় প্রথমে `ansible-doc` পড়ুন, তারপর ইন্টারনেট খুঁজুন — নথিটিই সবচেয়ে নির্ভরযোগ্য উৎস।

৭.৭ আর্কিটেকচার ডায়াগ্রাম



মনে রাখবেন: Module-এর কাজ Managed Node-এ হয়, Control Node শুধু নির্দেশ পাঠায় ও ফল পড়ে।

৭.৮ ব্যবহারিক উদাহরণ

```
# 1. সব হোস্ট বেঁচে আছে কি না — ping (কোনো -a লাগে না)
ansible all -m ping
# চিত্রণমূলক আউটপুট: web01 | SUCCESS => {"changed": false, "ping": "pong"}

# 2. শুধু webservers-এর OS তথ্য — setup + filter
ansible webservers -m setup -a "filter=ansible_distribution*"
# ফিরবে: ansible_distribution = "Ubuntu", ansible_distribution_version = "24.04"

# 3. Disk ব্যবহার দেখা — command (Shell ছাড়া, নিরাপদ)
ansible webservers -m command -a "df -h /"
# ব্যাখ্যা: df -h / — Root ফাইলসিস্টেমের ব্যবহার মানুষের পড়ার মতো করে দেখায়।

# 4. Pipe লাগলে shell — কতগুলো .log ফাইল আছে গোনা
ansible webservers -m shell -a "ls /var/log/*.log | wc -l"
# ব্যাখ্যা: Pipe (|) শুধু shell-এ কাজ করে; command-এ এটি আক্ষরিক অক্ষর হিসেবে যাবে।

# 5. শুধু একবার চালানো উচিত এমন কমান্ডে creates ব্যবহার — command-কে Idempotent বানানো
ansible webservers -m command -a "touch /tmp/technova-ready creates=/tmp/technova-ready"
# ব্যাখ্যা: creates= দিয়ে বলা হলো ফাইলটি থাকলে কমান্ড চালাবে না — দ্বিতীয়বার changed: false।

# 6. Nginx ইনস্টল — apt (become দিয়ে sudo)
ansible webservers -m apt -a "name=nginx state=present update_cache=yes" -b
# ব্যাখ্যা: update_cache=yes — apt update চালিয়ে Package তালিকা তাজা করে, তারপর ইনস্টল করে।
```

```
# 7. Nginx চালু ও Boot-এ চালু রাখা — service
ansible webservers -m service -a "name=nginx state=started enabled=yes" -b

# 8. ফাইল পাঠানো — copy (Control Node-এর ফাইল Managed Node-এ)
ansible webservers -m copy -a "src=./index.html dest=/var/www/html/index.html mode=0644" -b
# ব্যাখ্যা: src = Control Node-এর পথ; dest = Managed Node-এর পথ।

# 9. ডিরেক্টরি তৈরি — file
ansible webservers -m file -a "path=/var/www/technova state=directory owner=www-data
mode=0755" -b

# 10. ব্যবহারকারী তৈরি — user
ansible webservers -m user -a "name=deploy state=present groups=www-data shell=/bin/bash" -b

# 11. Python ভাঙা সার্ভারে raw দিয়ে Python ইনস্টল (জরুরি মেরামত)
ansible db01 -m raw -a "apt-get update && apt-get install -y python3" -b
```

৭.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

Ad-hoc Command ফাইলে লেখা হয় না, কিন্তু একই কাজগুলোকে একটি "দ্রুত-চেক" শেল স্ক্রিপ্টে সাজিয়ে রাখলে দলগত কাজে সুবিধা হয়। নিচে TechNova-র দৈনন্দিন স্বাস্থ্য-পরীক্ষার স্ক্রিপ্টটি দেওয়া হলো — এটি Playbook নয়, Ad-hoc Command-এর সংগ্রহ।

```
#!/bin/bash
# File: ~/technova-ansible/health-check.sh — দৈনন্দিন স্বাস্থ্য-পরীক্ষা (Ad-hoc সংগ্রহ)
# ব্যবহার: bash health-check.sh
set -e
cd ~/technova-ansible

echo "== 1. সংযোগ পরীক্ষা =="
ansible all -m ping

echo "== 2. OS সংস্করণ =="
ansible all -m setup -a "filter=ansible_distribution*"

echo "== 3. Disk ব্যবহার =="
ansible all -m command -a "df -h /"

echo "== 4. Nginx অবস্থা =="
ansible webservers -m service -a "name=nginx state=started" -b --check
# ব্যাখ্যা: --check (Check Mode) — আসলে কিছু না করে কী বদলাত তা দেখায়;
# এখানে Service ইতিমধ্যে চললে changed: false দেখাবে।

echo "== 5. Uptime =="
ansible all -m command -a "uptime"
```

৭.১০ ধাপে ধাপে বাস্তবায়ন

ধাপ ১ — Inventory প্রস্তুত: অধ্যায় ৬-এর `inventory.yml` ও `ansible.cfg` তৈরি আছে কি না যাচাই করুন; না থাকলে আগে সেটি করুন। **ধাপ ২ — সংযোগ যাচাই:** `ansible all -m ping` চালান; কোনো হোস্ট Unreachable হলে SSH Key ও IP ঠিক করুন। **ধাপ ৩ — তথ্য দেখুন:** `ansible web01 -m setup -a "filter=ansible_distribution*"` চালিয়ে Facts দেখুন। **ধাপ ৪ — command বনাম shell পরীক্ষা:** `ls /var/`

`log/*.log | wc -l` প্রথমে `command` দিয়ে, তারপর `shell` দিয়ে চালিয়ে পার্থক্যটি নিজের চোখে দেখুন — `command`-এ Pipe কাজ করবে না। **ধাপ ৫ — Package ইনস্টল:** `ansible webservers -m apt -a "name=nginx state=present update_cache=yes" -b` চালান, তারপর আবার একই কমান্ড চালিয়ে দ্বিতীয়বার `changed: false` আসা দেখুন — এটাই Idempotency। **ধাপ ৬ — ফাইল ও Service:** `copy` দিয়ে একটি `index.html` পাঠান, `service` দিয়ে Nginx চালু করুন, ব্রাউজারে `http://203.0.113.10/` খুলে দেখুন। **ধাপ ৭ — নথি পড়া:** `ansible-doc apt` ও `ansible-doc -s copy` চালিয়ে Argument-গুলো পড়ুন।

৭.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

```
$ ansible webservers -m apt -a "name=nginx state=present update_cache=yes" -b
# চিত্রণমূলক আউটপুট (প্রথমবার):
web01 | CHANGED => {"changed": true, "cache_updated": true,
  "stderr": "", "stdout": "Reading package lists... Done"}
web02 | CHANGED => {"changed": true, "cache_updated": true}
web03 | CHANGED => {"changed": true, "cache_updated": true}

$ ansible webservers -m apt -a "name=nginx state=present update_cache=yes" -b
# চিত্রণমূলক আউটপুট (দ্বিতীয়বার):
web01 | SUCCESS => {"changed": false, "cache_updated": false}
web02 | SUCCESS => {"changed": false, "cache_updated": false}
web03 | SUCCESS => {"changed": false, "cache_updated": false}

$ ansible webservers -m command -a "ls /var/log/*.log | wc -l"
# চিত্রণমূলক আউটপুট:
web01 | FAILED => {"msg": "ls: cannot access '/var/log/*.log | wc -l': No such file or
directory"}
# ব্যাখ্যা: command-এ Pipe কাজ করে না — পুরো স্ট্রিংটিকে একটি ফাইলের নাম মনে করা হয়েছে।

$ ansible webservers -m shell -a "ls /var/log/*.log | wc -l"
# চিত্রণমূলক আউটপুট:
web01 | CHANGED => {"changed": true, "stdout": "14", "stderr": ""}
# ব্যাখ্যা: CHANGED দেখালেও আসলে কিছু বদলায়নি — shell/command সবসময় changed: true
# দেয়, যদি না creates/removes বা changed_when দিয়ে নিয়ন্ত্রণ করা হয়।
```

আউটপুটের রঙের অর্থ মনে রাখুন: সবুজ `SUCCESS` মানে কিছু বদলায়নি, হলুদ `CHANGED` মানে কিছু বদলেছে, লাল `FAILED` মানে ত্রুটি। দ্বিতীয়বার `apt` চালিয়ে `SUCCESS` আসাই Idempotency-র প্রমাণ।

৭.১২ সাধারণ ভুল

(১) **সব কাজে `shell` ব্যবহার:** `shell -a "apt install nginx"` লিখলে Idempotency হারায় — দ্বিতীয়বারও `CHANGED` দেখাবে ও Playbook-এ Handler Trigger হবে; `apt` Module ব্যবহার করুন। (২) **`-b` ভুলে যাওয়া:** Package ইনস্টল বা `/var/www`-এ লেখার সময় sudo লাগে; `-b` না দিলে Permission Denied হবে। (৩) **`command`-এ Pipe/Redirect লেখা:** উপরের উদাহরণের মতো ব্যর্থ হবে; Pipe লাগলে `shell` নিন। (৪) **Ad-hoc Command-এ Secret পাস করা:** `-a "password=CHANGE-ME-..."` লিখলে তা Shell History-তে থেকে যায় — Vault বা Prompt ব্যবহার করুন। (৫) **Pattern ভুল:** `ansible webserver -m ping` (একবচন) লিখলে Group মিলবে না; `--list-hosts` দিয়ে আগে যাচাই করুন। (৬) **`state=latest` অকারণে ব্যবহার:** প্রতিবার চালালে Package আপগ্রেড হতে পারে — Production-এ অপ্রত্যাশিত পরিবর্তন ঘটে; সাধারণত `state=present`-ই যথেষ্ট।

৭.১৩ সমস্যা সমাধান (Troubleshooting)

সমস্যা: `UNREACHABLE! => {"msg": "Failed to connect to the host via ssh"}`। SSH Key, IP, Firewall যাচাই করুন; হাতে `ssh nahid@203.0.113.10` চালান। **সমস্যা: `MODULE FAILURE – /usr/bin/python3: No such file`।** Managed Node-এ Python নেই বা পথ ভুল; `raw` দিয়ে Python ইনস্টল করুন অথবা `ansible_python_interpreter` ঠিক করুন। **সমস্যা: `Permission denied (publickey)`।** Control Node-এর Key Managed Node-এর `authorized_keys`-এ নেই; `ssh-copy-id` দিয়ে Key পাঠান। **সমস্যা: `FAILED => {"msg": "No package matching 'ngnix' is available"}`।** Package নামের বানান ভুল; `apt search` বা `ansible-doc apt` দেখুন। **সমস্যা: আউটপুটে কিছু দেখা যায় না।** `-v` (বা `-vvv`) যোগ করে Verbose আউটপুট নিন — SSH-এর বিস্তারিত ত্রুটি দেখা যাবে। **সমস্যা: `WARNING: Module did not set no_log`।** Module-এর আউটপুটে সংবেদনশীল তথ্য থাকতে পারে — Playbook-এ `no_log: true` ব্যবহার করুন।

৭.১৪ নিরাপত্তা বিবেচনা

`shell` Module-এ ব্যবহারকারীর দেওয়া তথ্য (যেমন ফাইলের নাম) সরাসরি বসাবেন না — Shell Injection হতে পারে; প্রয়োজনে `command` ব্যবহার করুন অথবা ইনপুট যাচাই করুন। Ad-hoc Command Shell History-তে থেকে যায় — Password, Token বা API Key কখনো `-a`-তে লিখবেন না। `-b` (become) দিয়ে সব কাজ root হিসেবে চালানো নিরাপদ নয়: শুধু যে কাজে root লাগে সেটিতেই `-b` দিন; ফাইল পড়া বা তথ্য দেখার কাজে sudo লাগে না। `raw` Module Python-এর নিরাপত্তা-পরীক্ষা বাইপাস করে — শুধু মেরামতের সময় ব্যবহার করুন, নিয়মিত কাজে নয়। আর মনে রাখবেন: Ad-hoc Command-এর কোনো Version Control থাকে না — Production-এ গুরুত্বপূর্ণ পরিবর্তন Ad-hoc দিয়ে না করে Playbook দিয়ে করুন, যাতে পরিবর্তনটি Git-এ লিপিবদ্ধ থাকে।

৭.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

Production-এ Ad-hoc Command-এর ভূমিকা তিনটি: (১) **যাচাই ও অনুসন্ধান:** Deploy-এর আগে `ping`, Deploy-এর পরে `service`-এর অবস্থা দেখা; (২) **জরুরি মেরামত:** একটি Service হঠাৎ বন্ধ হলে দ্রুত Restart করা — তবে তারপর সেই মেরামতটি Playbook-এ লিপিবদ্ধ করুন, যাতে আবার হলে Playbook-ই চালানো যায়; (৩) **তথ্য সংগ্রহ:** `setup` দিয়ে Facts নিয়ে সমস্যার কারণ খোঁজা। নিয়ম: Ad-hoc Command দিয়ে Production-এ কখনো স্থায়ী পরিবর্তন (Package ইনস্টল, ফাইল বদল, ব্যবহারকারী তৈরি) করবেন না — সেসব Playbook-এর কাজ। Ad-hoc Command চালানোর আগে সবসময় `--check` বা `--list-hosts` দিয়ে দেখুন কমান্ডটি কোন হোস্টগুলোতে যাবে; একবার ভুল Pattern-এ `all`-এর বদলে Production-এর সব সার্ভারে ভুল কমান্ড চলে যাওয়া একটি ক্লাসিক দুর্ঘটনা। দলগত কাজে দরকারি Ad-hoc Command-গুলো `health-check.sh`-এর মতো স্ক্রিপ্টে সংরক্ষণ করুন, যাতে সবাই একইভাবে পরীক্ষা করে।

৭.১৬ অধ্যায়ের সারাংশ

Ad-hoc Command হলো Ansible-এর এক লাইনের কাজের উপায়: `ansible <pattern> -m <module> -a "<args>"`। Module হলো Declarative হাতিয়ার — কাঙ্ক্ষিত অবস্থা বলুন, Module বর্তমান অবস্থা দেখে সিদ্ধান্ত নেবে ও `changed` জানাবে; এটাই Idempotency। `ping` ও `setup` যাচাই করে, `apt` / `service` / `copy` / `file` / `user` দৈনন্দিন কাজ করে। `command` নিরাপদ কিন্তু Shell ছাড়া, `shell` Pipe-সহ কিন্তু ঝুঁকিপূর্ণ, `raw` শুধু Python-হীন মেরামতে। যেকোনো Module-এর সত্যিকারের নথি হলো `ansible-doc` — সেটি পড়ার অভ্যাসই একজন ভালো Ansible Engineer-এর প্রথম লক্ষণ।

৭.১৭ জ্ঞান যাচাই

১. Ad-hoc Command-এর তিনটি অংশ (Pattern, Module, Argument) কী কী? `-m` না দিলে কোন Module চলে? ২. `command` দিয়ে `ls /var/log/*.log | wc -l` চালালে কেন ব্যর্থ হয়, আর `shell` দিয়ে কেন সফল হয়? ৩. `apt` Module দ্বিতীয়বার চালালে `changed: false` আসে কেন, কিন্তু `shell` Module-এ সবসময় `changed: true` আসে কেন? ৪. `ansible-doc apt` ও `ansible-doc -s apt`-এর পার্থক্য কী? ৫. Python ইনস্টল নেই এমন সার্ভারে কোন Module দিয়ে মেরামত করবেন, আর কেন অন্য Module-গুলো সেখানে চলবে না?

৭.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

নিজের ল্যাবে: (ক) `ansible all -m ping` ও `ansible all -m setup -a "filter=ansible_distribution*"` চালিয়ে আউটপুট সংরক্ষণ করুন; (খ) `command` ও `shell` দিয়ে একই Pipe-যুক্ত কমান্ড চালিয়ে পার্থক্য নোট করুন; (গ) `apt` দিয়ে `nginx` ইনস্টল করে দুবার চালিয়ে Idempotency প্রমাণ করুন (প্রথমবার CHANGED, দ্বিতীয়বার SUCCESS); (ঘ) `copy` দিয়ে একটি নিজের লেখা `index.html` পাঠিয়ে `service` দিয়ে Nginx চালু করুন ও ব্রাউজারে যাচাই করুন; (ঙ) `ansible-doc file` পড়ে `state=link` দিয়ে একটি Symlink তৈরি করুন। জমা দিন: প্রতিটি কমান্ড ও তার আউটপুট, Idempotency-র প্রমাণসহ।

অধ্যায় 8 • PART II — Ansible Inventory ও Execution

Chapter 8 — Playbooks, Plays, Tasks, and Handlers

৮.১ ভূমিকা

Ad-hoc Command একবারের কাজের জন্য চমৎকার, কিন্তু TechNova-র Web সার্ভার সেটআপ করতে হলে একবারে অনেক কাজ করতে হয়: Nginx ইনস্টল করা, কনফিগারেশন ফাইল পাঠানো, Firewall খোলা, Service চালু করা, আর কনফিগারেশন বদলালে Service Restart করা। এই কাজগুলো প্রতিবার Ad-hoc Command দিয়ে করলে ক্রম ভুল হবে, কোনো ধাপ বাদ যাবে, আর কেউ জানবে না ঠিক কী করা হয়েছিল। **Playbook** এই সমস্যার সমাধান: এটি একটি YAML ফাইল, যেখানে পুরো সেটআপের ধাপগুলো ক্রমানুসারে, পাঠযোগ্যভাবে লেখা থাকে — যা বারবার চালানো যায়, Git-এ রাখা যায়, আর দলের সবাই একইভাবে বুঝতে পারে। Playbook-এর ভেতরে থাকে **Play** (কোন হোস্টে, কী প্রসঙ্গে), **Task** (একটি কাজ) ও **Handler** (শুধু প্রয়োজনে চলা বিশেষ Task)। এই অধ্যায়ে আমরা Playbook-এর গঠন, Task-এর নামকরণ, `notify` ও `flush_handlers`, Check Mode, Diff Mode ও Tags-এর পরিচয় শিখব — আর শেষে Nginx ইনস্টলের একটি সম্পূর্ণ, চালানো-যায় এমন Playbook তৈরি করব।

৮.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) YAML Playbook-এর গঠন — Play, Task, Handler — ব্যাখ্যা ও লিখতে; (২) Play ও Task-এর পার্থক্য বলতে; (৩) `notify` দিয়ে Handler Trigger করতে ও `flush_handlers` দিয়ে মাঝখানে Handler চালাতে; (৪) Task-এর স্পষ্ট নাম লেখার নিয়ম মেনে চলতে; (৫) `--check` (Check Mode) ও `--diff` (Diff Mode) দিয়ে নিরাপদে পরিবর্তন যাচাই করতে; (৬) Tags-এর প্রাথমিক ব্যবহার বুঝতে; (৭) Nginx ইনস্টল ও চালুর একটি সম্পূর্ণ Idempotent Playbook লিখে চালাতে।

৮.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৬-এর Inventory ও অধ্যায় ৭-এর Module-গুলো (`apt`, `service`, `copy`, `file`) জানা থাকতে হবে — Playbook আসলে সেই Module-গুলোকে ফাইলে সাজানো। YAML-এর Indentation, List (`-` দিয়ে) ও Dictionary (`key: value`) গঠন জানা চাই। `ansible-playbook` কমান্ডটি এই অধ্যায়ে প্রথম ব্যবহৃত হবে; এর আগে শুধু `ansible` (Ad-hoc) ব্যবহার করা হয়েছে।

৮.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: একটি সার্ভার সেটআপ করা আসলে একটি রেসিপি — উপকরণ (Package, ফাইল) ও ধাপ (ইনস্টল, কনফিগার, চালু) নির্দিষ্ট ক্রমে সাজানো। রেসিপি মুখে মুখে বললে প্রতিবার আলাদা হয়; কাগজে লিখে রাখলে প্রতিবার একই হয়। Playbook সেই কাগজ। এর তিনটি স্তর আছে। **Play** বলে: "এই রেসিপিটি এই দলের সার্ভারে চালাও, এই ব্যবহারকারী হিসেবে, এই Variable-সহ।" একটি Playbook-এ একাধিক Play থাকতে পারে — যেমন প্রথম Play Web সার্ভারে, দ্বিতীয় Play ডেটাবেস সার্ভারে। **Task** হলো রেসিপির একটি ধাপ — "Nginx ইনস্টল করো" — যা একটি Module কল। Task-গুলো ওপর থেকে নিচে ক্রমানুসারে চলে; একটি Task ব্যর্থ হলে পরের Task-গুলো (সেই Host-এ) থেমে যায়। **Handler** হলো বিশেষ Task, যা সাধারণত চলে না — শুধু তখনই চলে যখন কোনো Task `notify` করে বলে "কিছু বদলেছে, এখন তুমি চলো।" যেমন কনফিগারেশন ফাইল বদলালে Handler Nginx Restart করে; ফাইল না বদলালে Restart হয় না — এতে অকারণে Service বন্ধ হয় না। এই তিনটি স্তর — Play (কোথায়), Task (কী ধাপে), Handler (কখন প্রতিক্রিয়া) — মিলেই Playbook-এর পুরো দর্শন।

> **CONCEPT:** Playbook Declarative, Procedural নয়। আপনি ধাপের ক্রম লেখেন ঠিকই, কিন্তু প্রতিটি Task বলে কাঙ্ক্ষিত অবস্থা — "Nginx `present` থাকুক", "Service `started` থাকুক" — কীভাবে সেখানে পৌঁছাতে হবে তা Module ঠিক করে। ফলে Playbook দ্বিতীয়বার চালালে যেসব Task-এর অবস্থা ইতিমধ্যে ঠিক আছে, সেগুলো কিছু না করে `ok` দেখায়। এই Idempotency-ই Playbook-কে নিরাপদে বারবার চালানো যায় এমন হাতিয়ার করে তোলে — আর এজন্যই Playbook-কে "Runbook" বা "Documentation" হিসেবেও ব্যবহার করা যায়: ফাইলটি পড়লেই বোঝা যায় সার্ভারটি কী অবস্থায় থাকার কথা।

৮.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Playbook-এর গঠন

একটি Playbook হলো Play-এর List। প্রতিটি Play-তে থাকে: `name` (Play-এর নাম), `hosts` (কোন Pattern-এ চালাবে), `become` (sudo ব্যবহার করবে কি না), `vars` (Play-এর Variable), `tasks` (Task-এর List), `handlers` (Handler-এর List)। প্রতিটি Task-তে থাকে: `name` (মানুষের পড়ার নাম), Module-এর নাম ও তার Argument (যেমন `apt: name=nginx state=present` অথবা YAML Dictionary আকারে), আর ঐচ্ছিক `notify`, `when`, `tags`। Task-এর নাম বাধ্যতামূলক নয়, কিন্তু ছাড়া Playbook পড়া যায় না — আউটপুটে শুধু Module-এর নাম দেখায়।

notify ও Handler-এর নিয়ম

কোনো Task `notify: Restart Nginx` লিখলে, Task-টি `changed` হলেই Handler-টি Trigger হয়; Task `ok` থাকলে Handler চলে না। একই Handler একাধিক Task থেকে Notify হলেও Play শেষে Handler মাত্র একবার চলে — এতে বারবার Restart এড়ানো যায়। Handler-গুলো Play-এর সব Task শেষে চলে, তবে `flush_handlers` Task লিখলে সেখানেই চলমান Trigger-গুলো কার্যকর হয়। Handler-এর নাম হুবহু মিলতে হবে; Ansible 2.17-এ `listen` দিয়ে একাধিক নামে একই Handler ডাকাও যায়। Handler নিজে ব্যর্থ হলে Play ব্যর্থ হয় — Restart ব্যর্থ মানে Deploy ব্যর্থ, যা সঠিক আচরণ।

Check Mode, Diff Mode ও Tags

`--check` (Check Mode, Dry Run) দিয়ে Playbook চালালে Ansible আসলে কিছু বদলায় না — শুধু অনুমান করে কোন Task `changed` দেখাত। তবে সতর্কতা: `command` / `shell` Task Check Mode-এ Skip হয়, আর কিছু Module-এর অনুমান ভুল হতে পারে; তাই Check Mode নির্দেশক, নিশ্চয়তা নয়। `--diff` (Diff Mode) ফাইল বদলের সময় পুরনো ও নতুন বিষয়বস্তুর পার্থক্য (Unified Diff) দেখায় — কনফিগারেশন বদলের আগে কী বদলাবে তা দেখার সেরা উপায়। `--check --diff` একসঙ্গে চালানো সবচেয়ে নিরাপদ যাচাই। Tags দিয়ে Task-গুলোকে চিহ্নিত করা যায়: `tags: [nginx, install]` লিখে `--tags nginx` দিয়ে শুধু সেই Task-গুলো, অথবা `--skip-tags install` দিয়ে সেই Task-গুলো বাদ দিয়ে চালানো যায়। Tags-এর গভীর প্রয়োগ অধ্যায় ১১-এ আসবে।

উপাদান	ভূমিকা	কখন চলে	উদাহরণ
Playbook	Play-এর List; পুরো ফাইল	`ansible-playbook` চালালে	`site.yml`
Play	কোন হোস্টে, কী প্রসঙ্গে	ক্রমানুসারে, একটির পর একটি	`hosts: webservers`
Task	একটি কাজ (Module কল)	Play-এর ভেতরে ক্রমানুসারে	`apt: name=nginx`
Handler	প্রতিক্রিয়ামূলক Task	শুধু Notify হলে, Play শেষে	`Restart Nginx`

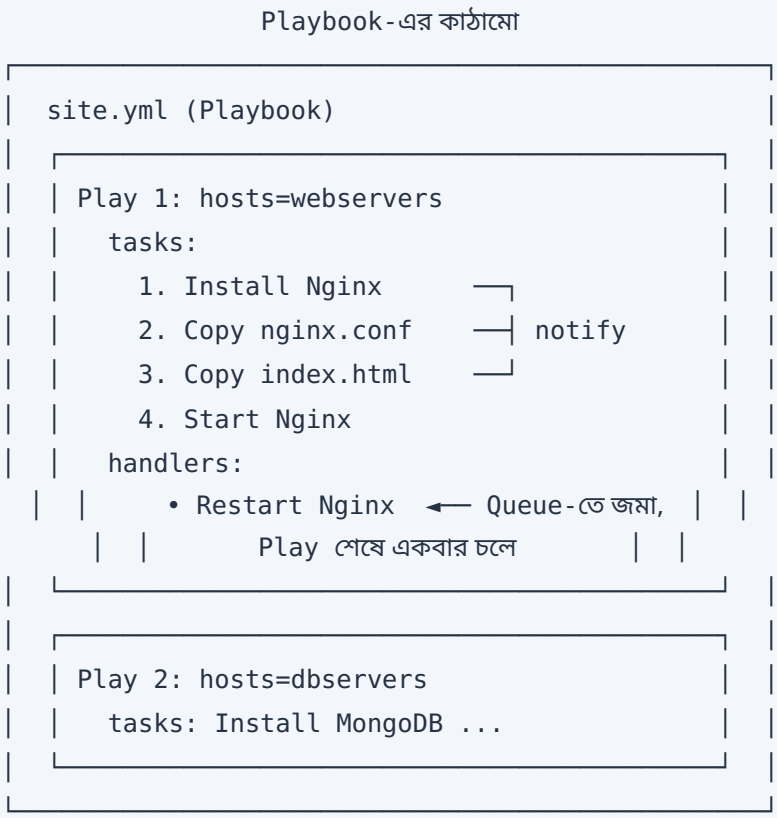
notify	Handler Trigger করা	Task `changed` হলে	`notify: Restart Nginx`
flush_handlers	Trigger হওয়া Handler এখনই চালানো	ওই Task-এ পৌঁছালে	`meta: flush_handlers`

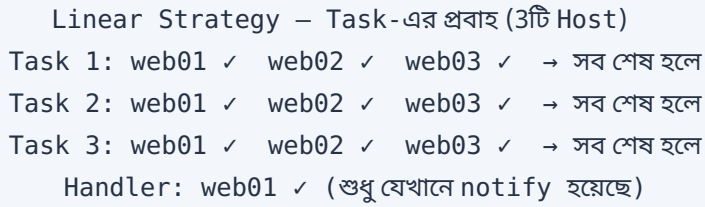
৮.৬ অভ্যন্তরীণ কার্যপ্রণালি

`ansible-playbook site.yml` চালালে ভেতরে যা ঘটে: (১) Playbook ফাইলটি YAML হিসেবে Parse হয়; Syntax ভুল থাকলে এখানেই ত্রুটি দেখায়। (২) Inventory থেকে প্রতিটি Play-এর `hosts` Pattern-মেলানো Host-গুলো বের করা হয়। (৩) প্রতিটি Play-এর জন্য Task-গুলো ক্রমানুসারে চলতে শুরু করে — ডিফল্ট Strategy `linear`: একটি Task সব Host-এ শেষ হলে পরের Task শুরু হয়। (৪) প্রতিটি Task প্রতিটি Host-এ অধ্যায় ৭-এর মতো Module চালায় — SSH, AnsiballZ, JSON ফল। (৫) Task `changed` হলে তার `notify`-তে লেখা Handler-গুলো একটি Queue-তে জমা হয়, কিন্তু তখনই চলে না। (৬) Play-এর সব Task শেষ হলে Queue-তে জমা Handler-গুলো একবার করে চলে (যে ক্রমে Handler লেখা আছে সেই ক্রমে, Notify-র ক্রমে নয়)। (৭) কোনো Task ব্যর্থ হলে সেই Host-এর জন্য Play থেমে যায়, কিন্তু অন্য Host-গুলো চলতে থাকে — যদি না `any_errors_fatal` সেট করা থাকে (অধ্যায় ১১)। (৮) সব Play শেষ হলে Ansible একটি Recap দেখায়: প্রতিটি Host-এ কয়টি `ok`, `changed`, `failed` হয়েছে। লক্ষ করুন: Handler-এর এই "শেষে একবার" নিয়মটিই কনফিগারেশন বদলের পরে একাধিকবার Restart এড়ায় — Playbook-এর সবচেয়ে গুরুত্বপূর্ণ কার্যকারিতাগত সিদ্ধান্তগুলোর একটি।

> **WARNING:** Handler শুধু তখনই চলে যখন Play-টি স্বাভাবিকভাবে শেষ হয়। Play-এর মাঝখানে কোনো Task ব্যর্থ হয়ে Play থেমে গেলে, তার আগে Notify হওয়া Handler-গুলো চলবে না — যদি না আপনি `force_handlers: true` সেট করেন। এর অর্থ: কনফিগারেশন ফাইল বদলে গেল, কিন্তু পরের Task ব্যর্থ হওয়ায় Nginx Restart হলো না — সার্ভারটি পুরনো কনফিগারেশন নিয়েই চলতে থাকল। গুরুত্বপূর্ণ Play-তে `force_handlers: true` লেখার অভ্যাস করুন, অথবা `flush_handlers` দিয়ে গুরুত্বপূর্ণ Handler আগেই চালিয়ে নিন।

৮.৭ আর্কিটেকচার ডায়াগ্রাম





Linear Strategy-তে একটি Task সব Host-এ শেষ না হওয়া পর্যন্ত পরের Task শুরু হয় না — এতে ক্রম নিশ্চিত থাকে, কিন্তু একটি ধীর Host সবাইকে আটকে রাখতে পারে। Free Strategy (অধ্যায় ১১) এই বাধা দূর করে।

৮.৮ ব্যবহারিক উদাহরণ

একটি ছোট Playbook দিয়ে শুরু করি — শুধু Nginx ইনস্টল ও চালু করা — তারপর ধাপে ধাপে Handler ও কনফিগারেশন যোগ করব।

```
# উদাহরণ 1 — সবচেয়ে ছোট Playbook (nginx-basic.yml)
---
- name: Install Nginx on webservers
  hosts: webservers
  become: true
  tasks:
    - name: Install Nginx package
      apt:
        name: nginx
```

```
        state: present
        update_cache: true
    - name: Ensure Nginx is started and enabled
      service:
        name: nginx
        state: started
        enabled: true

# চালানো: ansible-playbook nginx-basic.yml
# ব্যাখ্যা: --- দিয়ে YAML ডকুমেন্ট শুরু; - name: দিয়ে Play শুরু;
# hosts: বলে কাদের ওপরে; become: true বলে sudo ব্যবহার করবে;
# tasks: List-এর প্রতিটি আইটেম একটি Task।

# উদাহরণ 2 — Handler-সহ Playbook (কনফিগারেশন বদলালে Restart)
---
- name: Configure Nginx with custom page
  hosts: webservers
  become: true
  tasks:
    - name: Copy custom index page
      copy:
        src: files/index.html
        dest: /var/www/html/index.html
        mode: "0644"
      notify: Restart Nginx
  handlers:
    - name: Restart Nginx
      service:
        name: nginx
        state: restarted
# ব্যাখ্যা: index.html বদলালেই (changed) Handler চলবে; না বদলালে চলবে না।

# উদাহরণ 3 — flush_handlers: কনফিগারেশন যাচাইয়ের আগে Restart নিশ্চিত করা
    - name: Copy Nginx configuration
      copy:
        src: files/nginx.conf
        dest: /etc/nginx/nginx.conf
        mode: "0644"
      notify: Restart Nginx
    - name: Flush handlers now
      meta: flush_handlers
    - name: Check Nginx is responding
      command: curl -fs http://localhost/
      changed_when: false
# ব্যাখ্যা: meta: flush_handlers — কোনো Module নয়, Ansible-এর নিজের নির্দেশ;
# এখানে পৌঁছালে জমা থাকা Handler এখনই চলে, তারপর curl যাচাই করে।
```

৮.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচে TechNova-র Web সার্ভারের জন্য সম্পূর্ণ, চালানো-যায় এমন Playbook — Nginx ইনস্টল, কনফিগারেশন, ওয়েব পেজ, Firewall ও Service — একসঙ্গে। এর সঙ্গে লাগা ফাইলগুলোও দেওয়া হলো।

```
# File: ~/technova-ansible/site.yml — সম্পূর্ণ Nginx Playbook
---
- name: Set up Nginx webservers for TechNova
  hosts: webservers
```

```
  become: true
  force_handlers: true
  vars:
    http_port: 80
    server_name: technova.example
  tasks:
    - name: Update apt cache
      apt:
        update_cache: true
        cache_valid_time: 3600
      tags: [nginx, install]

    - name: Install Nginx package
      apt:
        name: nginx
        state: present
      tags: [nginx, install]

    - name: Create web root directory
      file:
        path: /var/www/technova
        state: directory
        owner: www-data
        group: www-data
        mode: "0755"
      tags: [nginx, config]

    - name: Deploy Nginx site configuration
      copy:
        src: files/technova.conf
        dest: /etc/nginx/sites-available/technova.conf
        owner: root
        group: root
        mode: "0644"
      notify: Reload Nginx
      tags: [nginx, config]

    - name: Enable TechNova site
      file:
        src: /etc/nginx/sites-available/technova.conf
        dest: /etc/nginx/sites-enabled/technova.conf
        state: link
      notify: Reload Nginx
      tags: [nginx, config]

    - name: Remove default Nginx site
      file:
        path: /etc/nginx/sites-enabled/default
        state: absent
      notify: Reload Nginx
      tags: [nginx, config]

    - name: Deploy homepage
      copy:
        src: files/index.html
        dest: /var/www/technova/index.html
        owner: www-data
        group: www-data
        mode: "0644"
```

```
      tags: [nginx, content]

    - name: Allow HTTP through firewall
      command: ufw allow 80/tcp
      args:
        creates: /etc/ufw/applications.d/technova
      tags: [nginx, firewall]
      # ব্যাখ্যা: creates= দিয়ে Idempotency আনা হয়েছে; ভালো উপায় হলো
      # community.general.ufw Module ব্যবহার করা (অধ্যায় ১২-এ Collection)।

    - name: Ensure Nginx is started and enabled
      service:
        name: nginx
        state: started
        enabled: true
      tags: [nginx, service]

  handlers:
    - name: Reload Nginx
      service:
        name: nginx
        state: reloaded
    - name: Restart Nginx
      service:
        name: nginx
        state: restarted

# File: files/technova.conf
# server {
#     listen 80;
#     server_name technova.example;
#     root /var/www/technova;
#     index index.html;
#     location / {
#         try_files $uri $uri/ =404;
#     }
# }

# File: files/index.html
# <!DOCTYPE html>
# <html><head><title>TechNova</title></head>
# <body><h1>Welcome to TechNova — served by Ansible</h1></body></html>
```

৮.১০ ধাপে ধাপে বাস্তবায়ন

ধাপ ১ — ফাইল তৈরি: `~/technova-ansible/`-এ `site.yml`, `files/technova.conf` ও `files/index.html` তৈরি করুন। **ধাপ ২ — Syntax যাচাই:** `ansible-playbook site.yml --syntax-check` চালান — YAML বা গঠনের ভুল থাকলে এখানেই ধরা পড়বে। **ধাপ ৩ — Dry Run:** `ansible-playbook site.yml --check --diff` চালিয়ে কী বদলাবে দেখুন; কিছু বদলাবে না, শুধু অনুমান দেখাবে। **ধাপ ৪ — আসল চালানো:** `ansible-playbook site.yml` চালান ও প্রতিটি Task-এর `ok/changed` দেখুন। **ধাপ ৫ — Idempotency যাচাই:** আবার একই কমান্ড চালান — এবার সব Task `ok` দেখাবে, Handler চলবে না। **ধাপ ৬ — পরিবর্তন পরীক্ষা:** `index.html`-এ একটি লাইন বদলে আবার চালান — শুধু Deploy Homepage Task `changed` হবে, কিন্তু Handler (Reload) চলবে না, কারণ Homepage Task-এ `notify` নেই; তারপর `technova.conf` বদলে চালান — এবার Reload Handler চলবে। **ধাপ ৭ — ব্রাউজার যাচাই:** `http://203.0.113.10/` খুলে "Welcome to TechNova" দেখুন।

৮.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

```
$ ansible-playbook site.yml
# চিত্রণমূলক আউটপুট (প্রথমবার):
PLAY [Set up Nginx webservers for TechNova] ************************
TASK [Gathering Facts] *********************************************
ok: [web01]
ok: [web02]
ok: [web03]
TASK [Install Nginx package] ***************************************
changed: [web01]
changed: [web02]
changed: [web03]
TASK [Deploy Nginx site configuration] *****************************
changed: [web01]
changed: [web02]
changed: [web03]
RUNNING HANDLER [Reload Nginx] *************************************
changed: [web01]
changed: [web02]
changed: [web03]
PLAY RECAP *********************************************************
web01 : ok=8 changed=5 unreachable=0 failed=0
web02 : ok=8 changed=5 unreachable=0 failed=0
web03 : ok=8 changed=5 unreachable=0 failed=0

# ব্যাখ্যা: "Gathering Facts" প্রতিটি Play-এর শুরুতে স্বয়ংক্রিয়ভাবে চলে
# (setup Module) — Facts অধ্যায় ৯-এ। Handler শুধু Play শেষে চলেছে,
# যদিও তিনটি Task তাকে Notify করেছিল — তবু একবারই চলেছে। PLAY RECAP-এ
# ok/changed-এর হিসাব দেখা যায়। দ্বিতীয়বার চালালে changed=0 হবে।
```

`--check --diff`-এর আউটপুটে ফাইল বদলের ক্ষেত্রে `--- before` ও `+++ after` দিয়ে পার্থক্য দেখায় — Deploy-এর আগে কী বদলাবে তা পড়ে নেওয়ার অভ্যাস করুন।

৮.১২ সাধারণ ভুল

(১) **Task-এর নাম না লেখা:** নাম ছাড়া আউটপুটে শুধু `apt` দেখায় — কোন Task ব্যর্থ হলো বোঝা যায় না। (২) **Handler-এর নামের বানান ভুল:** `notify: Reload Nginx` লিখে Handler-এর নাম `Reload nginx` (ছোট h) লিখলে Handler চলবে না, আর কোনো ত্রুটিও দেখাবে না — Ansible শুধু সতর্কতা দেয়। (৩) **Play-তে `become: true` ভুলে যাওয়া:** `apt` Task Permission Denied হয়ে ব্যর্থ হবে। (৪) **YAML Indentation ভুল:** `tasks:`-এর নিচে Task দুই Space Indent না করলে Syntax Error হয়। (৫) **`service state=restarted` সরাসরি Task-এ লেখা:** প্রতিবার Playbook চালালে Service Restart হবে — Handler-এ রাখুন, যাতে শুধু প্রয়োজনে Restart হয়। (৬) **এক Playbook-এ সব কিছু ঢোকানো:** Web, DB, Monitoring — সব এক Playbook-এ লিখলে ফাইলটি পড়া যায় না; Play আলাদা করুন বা ফাইল ভাগ করুন (অধ্যায় ১২-এ Role)।

৮.১৩ সমস্যা সমাধান (Troubleshooting)

সমস্যা: `ERROR! Syntax Error while loading YAML`। YAML Indentation বা Colon-এর পরে Space-এর ভুল; `yamllint` বা Python YAML Parser দিয়ে যাচাই করুন। **সমস্যা: `ERROR! couldn't resolve module/action 'aptt'`।** Module নামের বানান ভুল; `ansible-doc -l | grep apt` দিয়ে সঠিক নাম দেখুন। **সমস্যা: Handler চলছে**

না। প্রথমে দেখুন Notify করা Task-টি `changed` হয়েছে কি না — `ok` থাকলে Handler চলবে না, এটাই সঠিক আচরণ; নামের বানান মিলিয়ে নিন; `--list-tasks` ও `--list-handlers` দিয়ে তালিকা দেখুন। **সমস্যা: Playbook মাঝখানে থেমে গেল।** ব্যর্থ Task-টির ত্রুটি পড়ুন; `-v` যোগ করুন; শুধু একটি Host-এ পরীক্ষা করতে `--limit web01` ব্যবহার করুন। **সমস্যা: `FAILED => {"msg": "Failed to reload nginx: Job for nginx.service failed"}`।** কনফিগারেশনে Syntax ভুল — Managed Node-এ `nginx -t` চালিয়ে দেখুন; ভালো উপায় হলো কনফিগারেশন Deploy-এর আগে `validate: /usr/sbin/nginx -t -c %s` ব্যবহার করা (অধ্যায় ১০-এর Template-এ দেখানো হবে)।

৮.১৪ নিরাপত্তা বিবেচনা

Playbook Git-এ রাখা হয় — তাই Playbook-এ কখনো আসল Password, Token বা Private Key লিখবেন না; Secret-এর জন্য Ansible Vault (অধ্যায় ১৩) ব্যবহার করুন। `become: true` Play-স্তরে দিলে সব Task root হিসেবে চলে; নিরাপদ উপায় হলো শুধু যে Task-গুলোতে root লাগে সেগুলোতে `become: true` দেওয়া। Playbook-এ `command` দিয়ে `curl ... | bash` ধরনের কাজ করবেন না — তাতে যাচাই ছাড়া দূরের কোড root হিসেবে চলে। আর `site.yml`-এর মতো Playbook চালানোর আগে `--check --diff` দিয়ে দেখে নিন কী বদলাবে — Production-এ অন্ধভাবে Playbook চালানো একটি গুরুতর ঝুঁকি।

৮.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

Production-এ Playbook-এর নিয়মগুলো: (১) **একটি Playbook, একটি দায়িত্ব:** `site.yml` সব Play অন্তর্ভুক্ত করলেও প্রতিটি সেবার Playbook আলাদা ফাইলে রাখুন (`web.yml`, `db.yml`), আর `site.yml` থেকে `import_playbook` দিয়ে সেগুলো ডাকুন। (২) **Version Control ও Review:** প্রতিটি Playbook পরিবর্তন Pull Request-এর মাধ্যমে যাক; `--syntax-check` ও `--check` CI-তে চালান। (৩) **Staging-এ আগে:** Production-এ চালানোর আগে একই Playbook Staging Inventory-তে চালিয়ে দেখুন। (৪) **সীমিত চালানো:** প্রথমে `--limit web01` দিয়ে একটি সার্ভারে চালান, ঠিক থাকলে বাকিগুলোতে — একে বলে Canary Deploy। (৫) **নামকরণ:** Task-এর নাম Verb দিয়ে শুরু করুন ("Install Nginx", "Deploy configuration") — আউটপুট পড়ে যেন গল্পের মতো বোঝা যায় কী হচ্ছে। (৬) **Idempotency পরীক্ষা:** প্রতিটি Playbook দুবার চালিয়ে দ্বিতীয়বার `changed=0` আসা নিশ্চিত করুন; না এলে কোনো Task Idempotent নয় — সেটি ঠিক করুন।

৮.১৬ অধ্যায়ের সারাংশ

Playbook হলো সার্ভার সেটআপের লিখিত রেসিপি: Play বলে কোথায়, Task বলে কী ধাপে, Handler বলে কখন প্রতিক্রিয়া জানাতে হবে। Task ক্রমানুসারে চলে, Handler শুধু Notify হলে Play শেষে একবার চলে, আর `flush_handlers` দিয়ে তাকে আগেও চালানো যায়। `--check --diff` দিয়ে নিরাপদে যাচাই করা যায়, Tags দিয়ে অংশবিশেষ চালানো যায়। Nginx-এর সম্পূর্ণ Playbookটি দেখিয়েছে: ইনস্টল, কনফিগারেশন, সাইট চালু, Firewall ও Service — সব এক ফাইলে, Idempotentভাবে, বারবার চালানো যায় এমনভাবে। পরের অধ্যায়ে আমরা Variable, Facts ও Conditionals দিয়ে এই Playbook-কে নমনীয় করব।

৮.১৭ জ্ঞান যাচাই

১. Play, Task ও Handler-এর পার্থক্য কী? প্রতিটি কখন চলে? ২. তিনটি Task একই Handler-কে Notify করলে Handler কয়বার চলে, আর কেন? ৩. `flush_handlers` কখন প্রয়োজন হয়? একটি উদাহরণ দিন। ৪. `--check` ও `--diff`

একসঙ্গে চালালে কী সুবিধা হয়? Check Mode-এর সীমাবদ্ধতা কী? ৫. কনফিগারেশন ফাইল বদলালে সরাসরি Task-এ `service state=restarted` লেখা কেন ভুল, Handler-এ রাখা কেন সঠিক?

৮.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

নিজের ল্যাবে: (ক) অধ্যায়ের `site.yml` সম্পূর্ণ Playbook-টি তৈরি করে `--syntax-check` ও `--check --diff` চালান; (খ) আসলভাবে চালিয়ে ব্রাউজারে যাচাই করুন; (গ) দ্বিতীয়বার চালিয়ে `changed=0` প্রমাণ করুন; (ঘ) `index.html` বদলে চালিয়ে দেখুন Handler চলে কি না, তারপর `technova.conf` বদলে চালিয়ে Reload Handler চলা দেখুন; (ঙ) একটি Task-এ ইচ্ছাকৃতভাবে Handler-এর নামের বানান ভুল লিখে কী হয় তা পর্যবেক্ষণ করুন ও ঠিক করুন। জমা দিন: Playbook ফাইল, দুবার চালানোর Recap ও Handler পরীক্ষার নোট।

অধ্যায় 9 • PART II — Ansible Inventory ও Execution

Chapter 9 — Variables, Facts, Registers, and Conditionals

৯.১ ভূমিকা

অধ্যায় ৮-এর Nginx Playbook-টি কাজ করে, কিন্তু তাতে একটি সমস্যা আছে: সবকিছু হার্ডকোড করা। `http_port: 80`, `server_name: technova.example` — Staging-এ পোর্ট 8080 চাইলে পুরো Playbook কপি করে বদলাতে হবে, আর তখন দুটি কপি আলাদা পথে চলতে শুরু করবে। **Variable** এই সমস্যার সমাধান: মানগুলোকে নাম দিয়ে রাখুন, Playbook-এ নামটি ব্যবহার করুন, আর মানটি Inventory, Group, Host বা Command Line থেকে দিন — একই Playbook Staging ও Production — দুই জায়গায় চলবে। **Facts** হলো Ansible-এর নিজে সংগ্রহ করা Variable — সার্ভারের OS, IP, মেমরি — যা না লিখেই পাওয়া যায়। **Register** হলো Task-এর ফল ধরে রাখার উপায়, আর **Conditional** (`when`) হলো "শুধু তখনই চালাও যখন..." বলার উপায়। এই চারটি মিলে Playbook-কে স্থির রেসিপি থেকে বুদ্ধিমান প্রোগ্রামে বদলে দেয়। এই অধ্যায়ে আমরা Variable-র ধরন ও অবস্থান, সম্পূর্ণ Precedence ক্রম, Facts, Register, `set_fact`, `when`, `changed_when` ও `failed_when` শিখব।

৯.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Variable-র ধরন (String, Number, Boolean, List, Dictionary) ও অবস্থান (Inventory, Play, Task, Command Line) ব্যাখ্যা করতে; (২) সম্পূর্ণ Variable Precedence ক্রমটি টেবিলসহ বলতে ও প্রয়োগ করতে; (৩) `ansible_facts` দিয়ে সার্ভারের তথ্য পড়ে Task-এ ব্যবহার করতে; (৪) `register` দিয়ে Task-এর ফল ধরে রেখে পরের Task-এ ব্যবহার করতে; (৫) `set_fact` দিয়ে নিজের Fact তৈরি করতে; (৬) `when` দিয়ে শর্তসাপেক্ষে Task চালাতে; (৭) `changed_when` ও `failed_when` দিয়ে `changed` / `failed` -এর অর্থ নিয়ন্ত্রণ করতে।

৯.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৬-এর `group_vars` / `host_vars` ও অধ্যায় ৮-এর Playbook গঠন জানা থাকতে হবে। YAML-এ List ও Dictionary লেখা, Jinja2-এর `{{ variable }}` গঠনের প্রাথমিক ধারণা থাকলে ভালো — Jinja2-র গভীরে যাব অধ্যায় ১০-এ। Linux-এ OS, IP ঠিকানা ও মেমরির ধারণা থাকলে Facts বুঝতে সুবিধা হবে।

৯.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: একটি Playbook-এ দুই ধরনের তথ্য থাকে — যা আপনি সিদ্ধান্ত নেন (যেমন পোর্ট 80, সার্ভারের নাম) ও যা সার্ভার নিজে জানে (যেমন তার OS Ubuntu 24.04, তার IP 203.0.113.10, তার মেমরি 4 GB)। প্রথমটি Variable, দ্বিতীয়টি Fact। Variable-কে যত দেরিতে (Late) ও যত নির্দিষ্টভাবে দেওয়া যায়, তত ভালো — কারণ দেরিতে দেওয়া মান আগের মানকে ওভাররাইড করে, আর নির্দিষ্ট মান সাধারণ মানকে ওভাররাইড করে। এটাই Precedence-র মূল নীতি: "সাধারণ ডিফল্ট আগে, নির্দিষ্ট ব্যতিক্রম পরে।" Register হলো তৃতীয় ধরনের তথ্য — যা *Task* চালিয়ে জানা যায় (যেমন একটি কমান্ডের আউটপুট, একটি ফাইল আছে কি না)। আর Conditional হলো সিদ্ধান্ত: "এই Task চালাব কি না, তা তথ্যের ওপরে নির্ভর করুক।" যেমন: "শুধু Ubuntu-তে `apt` চালাও, শুধু মেমরি 2 GB-র বেশি হলে Cache চালু করো, শুধু আগের Task বদলে থাকলে পরের Task চালাও।" এই চারটি ধারণা — Variable, Fact, Register, Conditional — একসঙ্গে মিলে Playbook-কে পরিবেশ-সচেতন করে তোলে।

> **CONCEPT:** Variable Precedence মনে রাখার সহজ সূত্র: "যত কাছে, তত জোরালো।" Command Line (`-e`) সবচেয়ে কাছে — তাই সবচেয়ে জোরালো; তারপর Task-এর `vars` , তারপর Play-এর `vars` , তারপর `host_vars` , তারপর `group_vars` , আর সবচেয়ে দূরে Role-এর ডিফল্ট। ব্যতিক্রম আছে — `host_vars` Group-এর চেয়ে জোরালো হলেও `group_vars/all` সবচেয়ে দুর্বল Group; আর Facts সবচেয়ে জোরালোদের মধ্যে পড়ে না, কারণ Facts-কে Variable দিয়ে ওভাররাইড করা যায় না — Facts-এর নামে Variable লিখলে তা উপেক্ষিত হয়। এই সূত্রটি মনে রাখলে Precedence টেবিলটি আর মুখস্থ করতে হয় না।

৯.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Variable-র ধরন ও অবস্থান

Variable পাঁচ ধরনের হতে পারে: String (`server_name: technova.example`), Number (`http_port: 80`), Boolean (`ssl_enabled: true`), List (`packages: [nginx, curl, git]`), Dictionary (`db: {name: technova, port: 27017}`)। অবস্থান অনুযায়ী: (১) Inventory-তে — Host লাইনে, `group_vars` / `host_vars` -এ; (২) Playbook-এ — Play-এর `vars:` , Task-এর `vars:` ; (৩) Command Line-এ — `-e "http_port=8080"` বা `-e @vars.yml` দিয়ে ফাইল থেকে; (৪) `vars_files:` দিয়ে বাইরের YAML ফাইল থেকে; (৫) `set_fact` বা `register` দিয়ে চলমান Play-এর ভেতরে। ব্যবহারের সময় Jinja2 গঠন: `{{ http_port }}` , Dictionary-তে `{{ db.name }}` বা `{{ db['name'] }}` ।

Variable Precedence — সম্পূর্ণ ক্রম

Ansible-এর Precedence ক্রম (কম থেকে বেশি অগ্রাধিকার) নিচের টেবিলে দেওয়া হলো। মনে রাখবেন: নিচের দিকের (বেশি অগ্রাধিকারের) মান ওপরের মানকে ওভাররাইড করে। এই তালিকাটি Ansible Core 2.17-এর নথি অনুযায়ী সরলীকৃত — পূর্ণ তালিকায় ২০টির বেশি স্তর আছে, কিন্তু বাস্তবে এই ১২টি স্তরেই ৯৯% কাজ হয়।

ক্রম (কম → বেশি)	উৎস	উদাহরণ
1 (সবচেয়ে কম)	Role Default (`defaults/main.yml`)	Role-এর ডিফল্ট মান
2	Inventory `group_vars/all`	সবার জন্য ডিফল্ট
3	Inventory `group_vars/<group>`	`group_vars/webservers.yml`
4	Inventory `host_vars/<host>`	`host_vars/web01.yml`
5	Playbook `vars_files`	`vars_files: [common.yml]`
6	Play-এর `vars`	Play-তে `vars: http_port: 80`
7	Task-এর `vars`	Task-তে `vars: http_port: 8080`
8	`set_fact` / `register`	চলমান Play-এ তৈরি মান
9	Role ও Include-এর `vars`	`include_role: vars: ...`
10	Block-এর `vars`	`block: vars: ...`
11	Command Line `-e` (Extra Vars)	`-e "http_port=8080"`

12 (সবচেয়ে বেশি)	Connection Variable / `ansible_*`	`ansible_user`, `ansible_host`

দুটি গুরুত্বপূর্ণ ব্যতিক্রম: (ক) একই স্তরে একাধিক Group থাকলে Alphabetical ক্রমে শেষের Group-এর মান জেতে (অধ্যায় ৬-এর সতর্কতা); (খ) `host_vars`-এর মান `group_vars`-এর চেয়ে জোরালো, কিন্তু Play-এর `vars` আবার `host_vars`-এর চেয়ে জোরালো — অর্থাৎ Playbook-এ লেখা মান Inventory-র মানকে ওভাররাইড করে। এজন্যই পরিবেশভেদে বদলানো মান Playbook-এ হার্ডকোড না করে Inventory বা Command Line-এ রাখা ভালো।

Facts — সার্ভার নিজে যা জানে

প্রতিটি Play-এর শুরুতে Ansible স্বয়ংক্রিয়ভাবে `setup` Module চালিয়ে Facts সংগ্রহ করে — যদি না `gather_facts: false` লেখা হয়। Facts পাওয়া যায় `ansible_facts` Dictionary-তে: `ansible_facts['distribution']` (Ubuntu), `ansible_facts['distribution_version']` (24.04), `ansible_facts['default_ipv4']['address']` (203.0.113.10), `ansible_facts['memtotal_mb']` (4096), `ansible_facts['processor_count']`, `ansible_facts['hostname']`। পুরনো শৈলীতে `ansible_distribution` সরাসরি লেখা যেত; Ansible 2.17-এ `ansible_facts` Dictionary-ই সুপারিশকৃত। Facts সংগ্রহ বন্ধ করতে চাইলে `gather_facts: false` লিখুন — এতে Playbook দ্রুত চলে, কিন্তু Facts-নির্ভর Task কাজ করবে না।

Register, set_fact, when, changed_when, failed_when

`register: result` লিখলে Task-এর ফল (JSON) `result` Variable-এ জমা হয়: `result.stdout` (আউটপুট), `result.rc` (Exit Code), `result.changed` (বদলেছে কি না), `result.failed`। `set_fact: my_var: "value"` দিয়ে চলমান Play-এ নিজের Fact তৈরি করা যায় — পরের Task-গুলো তা ব্যবহার করতে পারে। `when: ansible_facts['distribution'] == "Ubuntu"` লিখলে Task শুধু শর্ত মিললে চলে; শর্ত না মিললে Task `skipped` দেখায়। `changed_when: false` দিয়ে `command`-এর মতো Module-কে বলা যায় "তুমি কিছু বদলাওনি" — যাতে শুধু তথ্য দেখার কমান্ড `CHANGED` না দেখায়। `failed_when: result.rc != 0` দিয়ে ব্যর্থতার সংজ্ঞা বদলানো যায় — যেমন কোনো কমান্ড Exit Code 1 দিলেও তা স্বাভাবিক হলে ব্যর্থ ধরা হবে না।

৯.৬ অভ্যন্তরীণ কার্যপ্রণালি

Variable কীভাবে সমাধান (Resolve) হয়, তা বুঝলে Precedence-র রহস্য মিটে যায়। Playbook চালু হলে Ansible প্রতিটি Host-এর জন্য একটি Variable Scope তৈরি করে — একটি স্তরবিন্যস্ত অভিধান, যেখানে নিচের স্তরের মান ওপরের স্তরের মানকে ঢেকে দেয়। Play-এর শুরুতে Facts সংগ্রহ হয় ও Scope-এ যোগ হয়। তারপর প্রতিটি Task চালানোর আগে Ansible Task-এর Jinja2 Expression-গুলো (`{{ ... }}`) মূল্যায়ন করে: Variable-র নাম খুঁজতে Scope-এর স্তরগুলো ওপর থেকে নিচে দেখা হয় — যে স্তরে প্রথম মান পাওয়া যায়, সেটিই ব্যবহৃত হয়। `register` করলে Task-এর ফল Scope-এ নতুন Variable হিসেবে যোগ হয় — তাই পরের Task-এ `{{ result.stdout }}` ব্যবহার করা যায়, কিন্তু আগের Task-এ নয়। `set_fact` একইভাবে Scope-এ মান যোগ করে, তবে তা `register`-এর চেয়ে স্থায়ী — একই Play-এর সব পরের Task-এ থাকে। `when` শর্তটি Task চালানোর আগেই মূল্যায়ন হয়: শর্ত মিথ্যা হলে Module-টি চালানোই হয় না, Task `skipped` হয়। আর `changed_when` / `failed_when` মূল্যায়ন হয় Task চালানোর পরে, ফলের ওপরে — তাই `command`-এর Exit Code দেখে সিদ্ধান্ত নেওয়া যায়। লক্ষ করুন: Variable-র মান Task চালানোর সময় নির্ধারিত হয়, Playbook Parse করার সময় নয় — এজন্যই `register`-এর মান পরের Task-এ ব্যবহার করা যায়।

> **TIP:** কোনো Variable-র চূড়ান্ত মান কী হবে তা যাচাই করতে `debug` Module ব্যবহার করুন: `- debug: var=http_port` বা `- debug: msg="Port is {{ http_port }}"`। `debug` Module কিছু বদলায় না, শুধু মানটি দেখায় — Precedence নিয়ে সন্দেহ হলে এটিই সবচেয়ে দ্রুত যাচাই। আর `ansible-inventory --host web01` দিয়ে Inventory-স্তরের মান দেখা যায়, কিন্তু Play-স্তরের মান দেখা যায় না — সেটি দেখতে `debug`-ই ভরসা।

৯.৭ আর্কিটেকচার ডায়াগ্রাম



চিত্রে লক্ষ করুন: `register` -এর মান শুধু পরের Task-এ পাওয়া যায়, আর `when` শর্ত Task চালানোর আগেই পরীক্ষা হয়।

৯.৮ ব্যবহারিক উদাহরণ

```
# উদাহরণ 1 — Facts ব্যবহার করে OS অনুযায়ী Package ইনস্টল
- name: Install web server based on OS
  hosts: all
  tasks:
    - name: Install Nginx on Ubuntu
      apt:
        name: nginx
        state: present
      when: ansible_facts['distribution'] == "Ubuntu"
    - name: Show server information
      debug:
        msg: "Host {{ ansible_facts['hostname'] }} has {{ ansible_facts['memtotal_mb'] }} MB
RAM, IP {{ ansible_facts['default_ipv4']['address'] }}"

# উদাহরণ 2 — register দিয়ে কমান্ডের ফল ধরা
- name: Check Nginx version
  command: nginx -v
  register: nginx_version
  changed_when: false
  failed_when: false
# ব্যাখ্যা: nginx -v আউটপুট stderr-এ দেয়; register-এ stdout ও stderr দুটিই থাকে।
# changed_when: false — শুধু সংস্করণ দেখা, কিছু বদলায়নি।
# failed_when: false — Nginx না থাকলেও Play থামবে না; পরে when দিয়ে পরীক্ষা করা যাবে।

- name: Show Nginx version
  debug:
    msg: "Nginx version output: {{ nginx_version.stderr }}"
  when: nginx_version.rc == 0

# উদাহরণ 3 — set_fact দিয়ে গণনা করা মান তৈরি
- name: Set cache size based on memory
  set_fact:
    cache_size: "{{ 1024 if ansible_facts['memtotal_mb'] > 4096 else 256 }}"
# ব্যাখ্যা: মেমরি 4 GB-র বেশি হলে Cache 1024 MB, না হলে 256 MB।
# set_fact-এর মানটি পরের সব Task-এ {{ cache_size }} হিসেবে পাওয়া যাবে।

# উদাহরণ 4 — একাধিক শর্ত (and/or) ও List-এর শর্ত
- name: Install monitoring tools
  apt:
    name: htop
    state: present
  when:
    - ansible_facts['distribution'] == "Ubuntu"
    - ansible_facts['memtotal_mb'] > 1024
# ব্যাখ্যা: List-এর সব শর্ত সত্য হলে (AND) তবেই Task চলে।
# OR চাইলে: when: ansible_facts['distribution'] == "Ubuntu" or ansible_facts['distribution'] ==
"Debian"

# উদাহরণ 5 — Command Line থেকে Variable ওভাররাইড
# ansible-playbook site.yml -e "http_port=8080 app_env=staging"
# ব্যাখ্যা: -e (Extra Vars) সবচেয়ে জোরালো — Inventory ও Play-এর মানকে ওভাররাইড করে।
```

৯.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচে Variable, Facts, Register ও Conditional — সব একসঙ্গে ব্যবহার করা একটি সম্পূর্ণ Playbook দেওয়া হলো। এটি Staging ও Production — দুই পরিবেশেই চলে; পার্থক্য শুধু Inventory-র Variable-এ।

```
# File: ~/technova-ansible/variables-demo.yml
---
- name: Demonstrate variables, facts, registers and conditionals
  hosts: webservers
  become: true
  gather_facts: true
  vars:
    http_port: 80
    packages:
      - nginx
      - curl
      - git
  tasks:
    - name: Show effective variables
      debug:
        msg: "HTTP port is {{ http_port }}, server is {{ server_name |
default('technova.example') }}"
    - name: Install base packages
      apt:
        name: "{{ packages }}"
        state: present
        update_cache: true
    - name: Check if Nginx config exists
      stat:
        path: /etc/nginx/nginx.conf
      register: nginx_conf
    - name: Report config status
      debug:
        msg: "Nginx config exists: {{ nginx_conf.stat.exists }}, size
{{ nginx_conf.stat.size }} bytes"
    - name: Set cache size from memory
      set_fact:
        cache_size: "{{ 1024 if ansible_facts['memtotal_mb'] > 4096 else 256 }}"
    - name: Show cache decision
      debug:
        msg: "Memory {{ ansible_facts['memtotal_mb'] }} MB, cache set to {{ cache_size }} MB"
    - name: Install htop only on large servers
      apt:
        name: htop
        state: present
      when: ansible_facts['memtotal_mb'] > 2048
    - name: Check Nginx is responding
      command: curl -fs http://localhost:{{ http_port }}/
      register: curl_result
      changed_when: false
      failed_when: curl_result.rc != 0
    - name: Report website status
      debug:
        msg: "Website responded with HTTP {{ curl_result.rc }} (0 means success)"

# File: group_vars/webservers.yml (এই Playbook-এর জন্য)
# http_port: 80
# server_name: technova.example
```

```
# File: group_vars/staging.yml (Staging Inventory-তে)
# http_port: 8080
# server_name: staging.technova.example
# ব্যাখ্যা: একই Playbook, শুধু Inventory বদলে Staging-এ পোর্ট 8080-এ চলে।
```

৯.১০ ধাপে ধাপে বাস্তবায়ন

ধাপ ১ — Playbook তৈরি: উপরের `variables-demo.yml` ফাইলটি তৈরি করুন। **ধাপ ২ — Inventory Variable সাজান:** `group_vars/webservers.yml`-এ `http_port: 80` লিখুন। **ধাপ ৩ — চালান ও দেখুন:** `ansible-playbook variables-demo.yml` চালিয়ে `debug` আউটপুটে মানগুলো দেখুন। **ধাপ ৪ — Override পরীক্ষা:** `ansible-playbook variables-demo.yml -e "http_port=8080"` চালিয়ে দেখুন পোর্ট বদলে গেছে — এটাই Precedence-র প্রমাণ। **ধাপ ৫ — Host Override পরীক্ষা:** `host_vars/web01.yml`-এ `http_port: 8081` লিখে আবার চালান; দেখুন `web01`-এ 8081, বাকিদের 80 আসে। **ধাপ ৬ — Facts দেখুন:** `ansible web01 -m setup` চালিয়ে Facts-এর পুরো তালিকা দেখুন ও `ansible_facts['memtotal_mb']`-এর মান মিলিয়ে নিন। **ধাপ ৭ — Conditional পরীক্ষা:** একটি Task-এ `when: false` লিখে চালিয়ে `skipped` দেখুন, তারপর শর্তটি ঠিক করুন।

৯.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

```
$ ansible-playbook variables-demo.yml
# চিত্রণমূলক আউটপুট:
TASK [Show effective variables] ************************************
ok: [web01] => {"msg": "HTTP port is 80, server is technova.example"}
TASK [Check if Nginx config exists] ********************************
ok: [web01] => {"changed": false}
TASK [Report config status] ****************************************
ok: [web01] => {"msg": "Nginx config exists: True, size 1234 bytes"}
TASK [Install htop only on large servers] **************************
skipping: [web01]
# ব্যাখ্যা: skipping মানে when শর্ত মেলেনি — web01-এর মেমরি 2048 MB-র কম।
# এটি ব্যর্থতা নয়; Playbook স্বাভাবিকভাবে এগিয়ে যায়।

$ ansible-playbook variables-demo.yml -e "http_port=8080"
# চিত্রণমূলক আউটপুট:
TASK [Show effective variables] ************************************
ok: [web01] => {"msg": "HTTP port is 8080, server is technova.example"}
# ব্যাখ্যা: -e দিয়ে দেওয়া 8080 Play-এর 80-কে ওভাররাইড করেছে —
# Precedence-র সবচেয়ে জোরালো স্তর কাজ করেছে।
```

`stat` Module-এর `register` করা ফলে `stat.exists`, `stat.size`, `stat.mode` — অনেক তথ্য থাকে; `debug: var=nginx_conf` দিয়ে পুরো JSON দেখে নিন — Register-এর গঠন বোঝার সেরা উপায়।

৯.১২ সাধারণ ভুল

(১) **`when`-এ `{{ }}` ব্যবহার:** `when: "{{ ansible_facts['distribution'] == 'Ubuntu' }}"` লিখলে ভুল হবে — `when`-এ সরাসরি Expression লিখতে হয়, Jinja2 বন্ধনী নয়। (২) **Undefined Variable:** `{{ server_name }}` ব্যবহার করলেন, কিন্তু কোথাও `server_name` লেখা নেই — Playbook ব্যর্থ হবে; `{{ server_name | default('technova.example') }}` দিয়ে ডিফল্ট দিন (Filter-এর বিস্তারিত অধ্যায় ১০-এ)। (৩) **Boolean-কে String লেখা:** YAML-এ `ssl_enabled: "true"` (উদ্ধৃতিসহ) String, আর `ssl_enabled: true` Boolean — `when:`

`ssl_enabled`-এ String "false" সত্য হিসেবে গণ্য হয়! (৪) **Precedence না বুঝে একই Variable সব জায়গায় লেখা:** Inventory, Play ও Command Line — তিন জায়গায় `http_port` লিখলে কোনটি জিতবে তা না জেনে বিভ্রান্তি হয়; একটি জায়গায় রাখুন। (৫) **`register`-এর মান একই Task-এ ব্যবহার:** `register` করা Variable সেই Task-এর `when`-এ পাওয়া যায় না — শুধু পরের Task-এ। (৬) **`changed_when: false` ভুলে যাওয়া:** শুধু তথ্য দেখার `command` প্রতিবার `CHANGED` দেখিয়ে Handler Trigger করে ফেলে।

৯.১৩ সমস্যা সমাধান (Troubleshooting)

সমস্যা: `ERROR! 'server_name' is undefined`। Variable-টি কোথাও সংজ্ঞায়িত হয়নি; `debug: var=server_name` দিয়ে পরীক্ষা করুন, অথবা `default` Filter ব্যবহার করুন। **সমস্যা: `when` শর্ত সবসময় Skip হচ্ছে।** শর্তটি `debug` দিয়ে দেখুন — যেমন `ansible_facts['distribution']`-এর আসল মান "Ubuntu" কি না, না "ubuntu" (ছোট হাতের); String তুলনা Case-Sensitive। **সমস্যা: `-e` দিয়ে মান বদলাচ্ছে না।** Variable-টির নামের বানান মিলিয়ে নিন; আর মনে রাখুন `-e` শুধু Playbook-এর Variable ওভাররাইড করে, `ansible_host`-এর মতো Connection Variable-কে নয় — সেটি Inventory-তেই ঠিক করতে হয়। **সমস্যা: `register`-এর `stdout` খালি।** অনেক কমান্ড (যেমন `nginx -v`) আউটপুট `stderr`-এ দেয়; `debug: var=nginx_version` দিয়ে পুরো ফল দেখুন। **সমস্যা: Facts সংগ্রহ ধীর।** `gather_facts: false` লিখে Facts বন্ধ করুন, আর শুধু যে Facts লাগে তা `setup` Module-এর `filter` দিয়ে নিন।

৯.১৪ নিরাপত্তা বিবেচনা

Variable-তে Secret (Password, API Key, Token) কখনো সরলভাবে লিখবেন না — `group_vars`-এ লেখা Secret Git-এ চলে যায়। Secret-এর জন্য Ansible Vault ব্যবহার করুন (অধ্যায় ১৩), আর Playbook-এ `no_log: true` দিয়ে Secret-যুক্ত Task-এর আউটপুট লগে দেখানো বন্ধ করুন। `debug` Module দিয়ে পুরো `register` ফল দেখানোর সময় সতর্ক থাকুন — তাতে Secret থাকতে পারে। Command Line-এ `-e "password=CHANGE-ME-..."` লিখলে তা Shell History-তে থেকে যায়; বরং `-e @secrets.yml` দিয়ে Vault-এনক্রিপ্ট করা ফাইল ব্যবহার করুন। আর Facts-এ সার্ভারের বিস্তারিত তথ্য (IP, ব্যবহারকারী, Package তালিকা) থাকে — Facts-এর আউটপুট শেয়ার করার সময় সংবেদনশীল অংশ মুছে নিন।

৯.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

Production-এ Variable পরিচালনার নিয়মগুলো: (১) **ডিফল্ট Role-এ, ব্যতিক্রম Inventory-তে:** সাধারণ মান `group_vars/all.yml`-এ, পরিবেশের মান `group_vars/production.yml`-এ, হোস্টের ব্যতিক্রম `host_vars`-এ — এই স্তরবিন্যাস মেনে চলুন। (২) **নামের নিয়ম:** Variable-র নাম ছোট হাতের অক্ষর ও Underscore-এ লিখুন (`http_port`, `server_name`); Ansible-এর নিজের `ansible_*` নাম দিয়ে নিজের Variable নাম রাখবেন না। (৩) **`-e` শুধু জরুরি Override-এ:** নিয়মিত মান Command Line-এ দিলে Playbook-এর পুনরুৎপাদনযোগ্যতা (Reproducibility) নষ্ট হয় — কেউ জানবে না আসলে কোন মান দিয়ে চালানো হয়েছিল। (৪) **Facts-নির্ভর Conditional:** OS বা মেমরির ওপরে শর্ত লিখলে একই Playbook ভিন্ন সার্ভারে চলে — এটাই Portable Playbook-এর চাবিকাঠি। (৫) **`changed_when` / `failed_when` সব `command` Task-এ:** প্রতিটি `command` Task-এ এই দুটি লিখুন, নাহলে Idempotency ও ত্রুটি-শনাক্তকরণ — দুটিই ভেঙে যায়।

৯.১৬ অধ্যায়ের সারাংশ

Variable হলো Playbook-এর নমনীয়তা: একই Playbook ভিন্ন মান দিয়ে ভিন্ন পরিবেশে চলে। Precedence-র নিয়ম — "যত কাছে ও যত নির্দিষ্ট, তত জোরালো" — মনে রাখলে কোন মান জিতবে তা অনুমান করা যায়; Command Line-এর `-e` সবচেয়ে জোরালো। Facts হলো সার্ভারের নিজের তথ্য, Register হলো Task-এর ফল, `set_fact` হলো নিজের তৈরি Fact, আর `when` হলো শর্ত। `changed_when` ও `failed_when` দিয়ে `command`-এর মতো Module-কে Idempotent ও সঠিকভাবে ব্যর্থ-শনাক্তকারী বানানো যায়। পরের অধ্যায়ে আমরা Loop, Filter ও Template দিয়ে এই নমনীয়তাকে আরও এক ধাপ এগিয়ে নেব।

৯.১৭ জ্ঞান যাচাই

১. Variable Precedence ক্রমে `host_vars`, Play-এর `vars` ও Command Line-এর `-e` — কোনটি সবচেয়ে জোরালো, আর কেন? ২. `ansible_facts['distribution']` ও `ansible_facts['default_ipv4']['address']` দিয়ে কী জানা যায়? Facts সংগ্রহ বন্ধ করতে কী লিখতে হয়? ৩. `register` করা Variable একই Task-এর `when`-এ ব্যবহার করা যায় কি না, আর কেন? ৪. `when: "{{ x == 1 }}"` লেখা কেন ভুল? সঠিক গঠনটি কী? ৫. `changed_when: false` ও `failed_when: false`-এর পার্থক্য কী? প্রতিটির একটি করে ব্যবহারের উদাহরণ দিন।

৯.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

নিজের ল্যাবে: (ক) অধ্যায়ের `variables-demo.yml` চালিয়ে `debug` আউটপুট সংরক্ষণ করুন; (খ) `-e "http_port=8080"` দিয়ে Override প্রমাণ করুন; (গ) `host_vars/web01.yml`-এ ভিন্ন পোর্ট দিয়ে Host Override প্রমাণ করুন; (ঘ) একটি নতুন Task যোগ করুন যা শুধু মেমরি 2 GB-র বেশি হলে `htop` ইনস্টল করে, আর একটি Task যোগ করুন যা `curl`-এর ফল `register` করে শুধু সফল হলে বার্তা দেখায়; (ঙ) ইচ্ছাকৃতভাবে একটি Undefined Variable ব্যবহার করে ত্রুটিটি দেখুন, তারপর `default` Filter দিয়ে ঠিক করুন। জমা দিন: Playbook, তিনটি ভিন্ন মানের `debug` আউটপুট ও Precedence-র ব্যাখ্যা।

অধ্যায় 10 • PART II — Ansible Inventory ও Execution

Chapter 10 — Loops, Filters, Templates, and Jinja2

১০.১ ভূমিকা

TechNova-র Web সার্ভারে পাঁচটি Package ইনস্টল করতে হলে কি পাঁচটি Task লিখতে হবে? তিনটি ব্যবহারকারী তৈরি করতে তিনটি Task? আর প্রতিটি সার্ভারের জন্য আলাদা Nginx কনফিগারেশন ফাইল লিখতে হবে — যেখানে শুধু সার্ভারের নাম ও পোর্ট আলাদা? এই পুনরাবৃত্তি এড়াতেই **Loop**, **Filter** ও **Template**। Loop একই Task-কে একটি তালিকার প্রতিটি আইটেমের জন্য চালায়। Filter মানকে রূপান্তর করে — ডিফল্ট মান দেয়, List-কে String-এ জোড়ে, Dictionary-কে JSON-এ বদলায়। Template হলো Jinja2 দিয়ে লেখা ফাইল, যেখানে `{{ variable }}` ও `{% if %}` লিখে প্রতিটি সার্ভারের জন্য আলাদা কনফিগারেশন তৈরি করা যায়। এই তিনটি মিলে Playbook সংক্ষিপ্ত, পাঠযোগ্য ও পরিবেশ-সচেতন হয়। এই অধ্যায়ে আমরা `loop`, পুরনো `with_items`, `loop_control`, Jinja2 Expression, দরকারি Filter-গুলো, `template` Module ও একটি বাস্তব `nginx.conf.j2` উদাহরণ শিখব।

১০.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) `loop` দিয়ে একটি Task-কে List-এর প্রতিটি আইটেমে চালাতে; (২) `with_items`-এর পুরনো শৈলী চিনতে ও `loop`-এ রূপান্তর করতে; (৩) `loop_control` দিয়ে Loop Variable-র নাম ও আউটপুটের লেবেল নিয়ন্ত্রণ করতে; (৪) Jinja2 Expression (`{{ }}`, `{% %}`) লিখতে; (৫) `default`, `join`, `to_json`, `upper`, `length`-সহ দরকারি Filter ব্যবহার করতে; (৬) `template` Module দিয়ে Jinja2 Template থেকে কনফিগারেশন ফাইল তৈরি করতে; (৭) একটি বাস্তব `nginx.conf.j2` Template লিখে ভিন্ন সার্ভারে ভিন্ন কনফিগারেশন Deploy করতে।

১০.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৯-এর Variable, Facts ও Register জানা থাকতে হবে — Loop-এর আইটেম প্রায়ই Variable-র List হয়, আর Template-এ Facts ব্যবহার করা হয়। YAML-এ List ও Dictionary লেখা, আর `copy` Module-এর ব্যবহার জানা চাই। Python-এর প্রাথমিক ধারণা (List, Dictionary, `if`) থাকলে Jinja2 বুঝতে সুবিধা হবে, তবে আবশ্যক নয় — Jinja2-র গঠন এই অধ্যায়েই শেখানো হবে।

১০.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: পুনরাবৃত্তি হলো ভুলের উৎস। পাঁচটি Package-র জন্য পাঁচটি Task লিখলে ষষ্ঠ Package যোগ করতে ষষ্ঠ Task লিখতে হয়, আর কোনো Task-এ বানান ভুল হলে শুধু সেই Package-টি বাদ যায় — নীরবে। Loop এই পুনরাবৃত্তিকে ডেটায় বদলে দেয়: Task একটি, ডেটা একটি List। List বদলালে কাজ বদলায়, Task বদলাতে হয় না। Template একই নীতি ফাইলের ক্ষেত্রে প্রয়োগ করে: কনফিগারেশন ফাইলকে দুটি অংশে ভাগ করুন — যা সব সার্ভারে এক (স্থির লেখা) ও যা সার্ভারভেদে আলাদা (Variable)। স্থির অংশ Template-এ লিখুন, আলাদা অংশ `{{ }}`-এ লিখুন; Ansible প্রতিটি সার্ভারের Variable দিয়ে Template পূরণ করে আলাদা ফাইল বানিয়ে দেবে। Filter হলো এই প্রক্রিয়ার ছোট ছোট যন্ত্র: মান না থাকলে ডিফল্ট দেওয়া, List-কে কমা দিয়ে জোড়া, সংখ্যাকে String-এ বদলানো — এসব ছোট রূপান্তর ছাড়া Template লেখা যায় না। এই তিনটি — Loop (কাজের পুনরাবৃত্তি), Filter (মানের রূপান্তর), Template (ফাইলের পুনরাবৃত্তি) — মিলে Playbook-কে "একটি কাজ, অনেক সার্ভার" থেকে "একটি Playbook, অনেক পরিবেশ"-এ নিয়ে যায়।

> **CONCEPT:** Jinja2 হলো একটি Templating ভাষা — Python-এর লেখকদের তৈরি, Ansible-এর নিজের ভাষা নয়। Ansible শুধু Jinja2-কে ডেকে Template পূরণ করে। Jinja2-র তিনটি গঠন মনে রাখুন: `{{ expression }}` — মান ছাপানো (যেমন `{{ http_port }}`); `{% statement %}` — নিয়ন্ত্রণ (যেমন `{% if ssl_enabled %}...{% endif %}`, `{% for pkg in packages %}...{% endfor %}`); `{# comment #}` — মন্তব্য, যা চূড়ান্ত ফাইলে আসে না। Ansible-এর `when` শর্তেও Jinja2 Expression চলে — তবে সেখানে `{{ }}` লেখা যায় না, কারণ `when` নিজেই একটি Expression প্রসঙ্গ।

১০.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

loop — আধুনিক পুনরাবৃত্তি

`loop` একটি Task-কে List-এর প্রতিটি আইটেমের জন্য চালায়; প্রতিটি পুনরাবৃত্তিতে বর্তমান আইটেমটি পাওয়া যায় `item` Variable-এ। উদাহরণ: `loop: [nginx, curl, git]` দিয়ে `apt: name={{ item }}` লিখলে তিনবার `apt` চলবে। Dictionary-র List-ও Loop করা যায়: `loop: [{name: deploy, shell: /bin/bash}, {name: backup, shell: /bin/sh}]` দিয়ে `user: name={{ item.name }} shell={{ item.shell }}`। Loop-এর ফল `register` করলে `results` List পাওয়া যায় — প্রতিটি আইটেমের ফল আলাদাভাবে থাকে।

with_items — পুরনো শৈলী

Ansible 2.5-এর আগে Loop লেখা হতো `with_items` দিয়ে। এটি এখনো কাজ করে, কিন্তু নতুন কোডে `loop`-ই সুপারিশকৃত — কারণ `loop` সরল, আর `with_*` পরিবারের অন্য সদস্যরা (`with_dict`, `with_fileglob`) ধীরে ধীরে `loop` + Filter-এ রূপান্তরিত হচ্ছে। পুরনো Playbook পড়তে গেলে `with_items` চিনতে হবে, কিন্তু নতুন Playbook-এ `loop` লিখুন। রূপান্তরটি সহজ: `with_items: [a, b]` মানেই `loop: [a, b]`।

loop_control — Loop-কে নিয়ন্ত্রণ

`loop_control` দিয়ে তিনটি জিনিস নিয়ন্ত্রণ করা যায়: (১) `loop_var: pkg` — `item`-এর বদলে নিজের নাম দেওয়া; Nested Loop-এ এটি আবশ্যক, কারণ ভেতরের Loop বাইরের `item`-কে ঢেকে দেয়। (২) `label: "{{ pkg }}"` — আউটপুটে প্রতিটি পুনরাবৃত্তির পাশে কী দেখাবে তা ঠিক করা; Dictionary-র List Loop করলে ডিফল্ট আউটপুট বিশাল হয়, `label` দিয়ে তা সংক্ষিপ্ত করা যায়। (৩) `pause: 2` — প্রতিটি পুনরাবৃত্তির মাঝে কয়েক সেকেন্ড বিরতি; API Rate Limit এড়াতে দরকারি।

Filter — মানের রূপান্তর

Filter লেখা হয় Pipe (`|`) দিয়ে: `{{ value | filter_name }}`। দরকারি Filter-গুলো: `default('...')` — মান না থাকলে ডিফল্ট; `join(', ')` — List-কে String-এ জোড়া; `to_json` / `from_json` — JSON-এ রূপান্তর; `upper` / `lower` — বড়/ছোট হাতের অক্ষর; `length` — List বা String-এর দৈর্ঘ্য; `int` / `string` — ধরন বদল; `trim` — সামনে-পেছনের ফাঁকা জায়গা মোছা; `basename` / `dirname` — পথের অংশ নেওয়া; `dict2items` / `items2dict` — Dictionary ও List-এর মধ্যে রূপান্তর। Filter একসঙ্গে জোড়া যায়: `{{ server_name | default('technova.example') | upper }}`।

template Module — Jinja2 থেকে ফাইল

`template` Module `copy`-এর মতো, তবে `src`-এ দেওয়া ফাইলটি Jinja2 Template হিসেবে পূরণ করে তারপর পাঠায়। Template ফাইলের নাম সাধারণত `.j2` দিয়ে শেষ হয় (যেমন `nginx.conf.j2`)। `template`-এ `validate` Argument

দিয়ে ফাইলটি পাঠানোর আগে যাচাই করা যায়: `validate: /usr/sbin/nginx -t -c %s` — `%s`-এর জায়গায় অস্থায়ী ফাইলের পথ বসে; যাচাই ব্যর্থ হলে ফাইলটি পাঠানোই হয় না। Template-এ `{% if %}` দিয়ে শর্তসাপেক্ষ লাইন, `{% for %}` দিয়ে পুনরাবৃত্ত লাইন লেখা যায় — যেমন প্রতিটি Backend সার্ভারের জন্য একটি `server` লাইন।

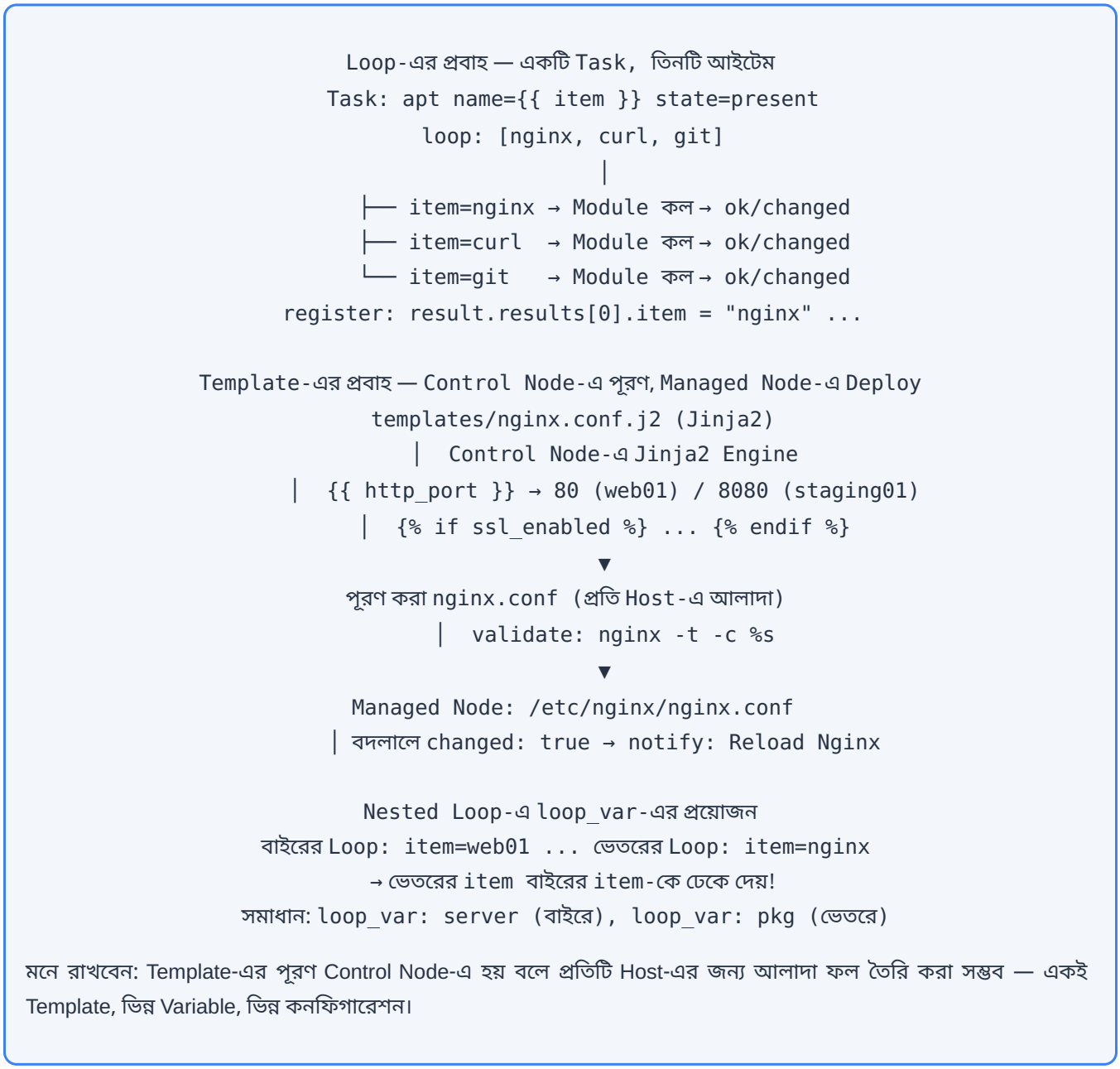
Filter	কাজ	উদাহরণ	ফল
default	মান না থাকলে ডিফল্ট	`{{ port | default(80) }}`	80
join	List-কে String-এ জোড়া	`{{ ['a','b'] | join(', ') }}`	a, b
to_json	JSON String-এ রূপান্তর	`{{ db | to_json }}`	{"name": "technova"}
upper / lower	বড়/ছোট হাতের অক্ষর	`{{ 'TechNova' | lower }}`	technova
length	দৈর্ঘ্য গোনা	`{{ packages | length }}`	3
int	সংখ্যায় রূপান্তর	`{{ "8080" | int }}`	8080
dict2items	Dictionary-কে List-এ বদলানো	`{{ db | dict2items }}`	[{key, value}, ...]

১০.৬ অভ্যন্তরীণ কার্যপ্রণালি

Loop-যুক্ত Task চালালে ভেতরে যা ঘটে: Ansible Task-টি একবার Parse করে, তারপর `loop`-এর List-এর প্রতিটি আইটেমের জন্য আলাদাভাবে Module কল করে — প্রতিটি আইটেম একটি আলাদা Task-ফল হিসেবে দেখায় (`ok: [web01] => (item=nginx)`)। কোনো আইটেম ব্যর্থ হলে বাকি আইটেমগুলোও থেমে যায় (যদি না `ignore_errors` থাকে — অধ্যায় ১১)। Template চালালে: (১) Control Node-এ Jinja2 Engine Template ফাইলটি পড়ে, `{{ }}` ও `{% %}` মূল্যায়ন করে — Variable-গুলো সেই Host-এর Scope থেকে নেওয়া হয়, তাই প্রতিটি Host-এর জন্য আলাদা ফল তৈরি হয়। (২) পূরণ করা ফাইলটি অস্থায়ীভাবে Control Node-এ সংরক্ষিত হয়। (৩) `validate` থাকলে যাচাই কমান্ডটি Control Node-এ (বা `validate`-এর ক্ষেত্রে Managed Node-এ পাঠিয়ে) চালানো হয় — ব্যর্থ হলে প্রক্রিয়া থেমে যায়। (৪) তারপর `copy`-এর মতো করে ফাইলটি Managed Node-এ পাঠানো হয়; ফাইলের বিষয়বস্তু আগের মতো থাকলে `changed: false`, বদলে থাকলে `changed: true` ও `notify` Trigger হয়। লক্ষ করুন: Template পূরণ Control Node-এ হয়, Managed Node-এ নয় — তাই Template-এ Control Node-এর Variable ব্যবহার করা যায় না, শুধু Host-এর Variable ও Facts ব্যবহার করা যায়।

> **WARNING:** Template-এ Jinja2-র `{{ }}` আর Nginx-এর মতো সফটওয়্যারের নিজের `$variable` বা `{{ }}` গঠন গুলিয়ে ফেলবেন না। Nginx কনফিগারেশনে `$uri`, `$host` — এগুলো Nginx-এর Variable, Jinja2-র নয় — Jinja2 এগুলো ছোঁয় না, কারণ সেগুলো `{{ }}`-এর ভেতরে নেই। কিন্তু কোনো কনফিগারেশনে যদি আক্ষরিক `{{ }}` লিখতে হয় (যেমন কিছু JavaScript Template), তাহলে Jinja2-কে বলতে হয় `{% raw %}{{ ... }}{% endraw %}` দিয়ে — নাহলে Jinja2 সেটিকে Variable মনে করে ত্রুটি দেখাবে।

১০.৭ আর্কিটেকচার ডায়াগ্রাম



মনে রাখবেন: Template-এর পূরণ Control Node-এ হয় বলে প্রতিটি Host-এর জন্য আলাদা ফল তৈরি করা সম্ভব — একই Template, ভিন্ন Variable, ভিন্ন কনফিগারেশন।

১০.৮ ব্যবহারিক উদাহরণ

```
# উদাহরণ 1 — Loop দিয়ে একাধিক Package ইনস্টল
- name: Install base packages
  apt:
    name: "{{ item }}"
    state: present
  loop:
    - nginx
    - curl
    - git
    - htop
# ব্যাখ্যা: একটি Task, চারটি Module কল; নতুন Package যোগ করতে শুধু List-এ যোগ করুন।

# উদাহরণ 2 — Dictionary-র List Loop করে ব্যবহারকারী তৈরি
```

```
- name: Create application users
  user:
    name: "{{ item.name }}"
    shell: "{{ item.shell }}"
    groups: "{{ item.groups }}"
  loop:
    - {name: deploy, shell: /bin/bash, groups: www-data}
    - {name: backup, shell: /bin/sh, groups: backup}
  loop_control:
    label: "{{ item.name }}"
# ব্যাখ্যা: label দিয়ে আউটপুটে শুধু ব্যবহারকারীর নাম দেখায়,
# পুরো Dictionary দেখায় না।

# উদাহরণ 3 — with_items (পুরনো শৈলী — চিনে রাখুন, নতুন কোডে loop লিখুন)
- name: Install packages (legacy style)
  apt:
    name: "{{ item }}"
    state: present
  with_items:
    - nginx
    - curl
# ব্যাখ্যা: with_items আর loop-এর ফল একই; নতুন Playbook-এ loop ব্যবহার করুন।

# উদাহরণ 4 — Filter-এর ব্যবহার
- name: Show transformed values
  debug:
    msg: "Servers: {{ ['web01', 'web02'] | join(', ') }}, count: {{ packages | length }}, env:
{{ app_env | default('production') | upper }}"
# ব্যাখ্যা: join List-কে String করে, length গোনে, default না থাকলে মান দেয়,
# upper বড় হাতের অক্ষরে বদলায়।

# উদাহরণ 5 — Template দিয়ে কনফিগারেশন
- name: Deploy Nginx configuration from template
  template:
    src: templates/nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    mode: "0644"
    validate: /usr/sbin/nginx -t -c %s
  notify: Reload Nginx
# ব্যাখ্যা: validate ব্যর্থ হলে ভুল কনফিগারেশন কখনো Deploy হবে না —
# Production-এ Template ব্যবহারের সবচেয়ে গুরুত্বপূর্ণ নিরাপত্তা।
```

১০.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচে একটি বাস্তব `nginx.conf.j2` Template ও তা ব্যবহার করা সম্পূর্ণ Playbook দেওয়া হলো। Template-টিতে শর্তসাপেক্ষ SSL, Loop দিয়ে Upstream সার্ভার, আর Variable দিয়ে Worker সংখ্যা — সব আছে।

```
# File: templates/nginx.conf.j2 — বাস্তব Nginx Template
# {{ ansible_managed }} — এই লাইনটি Ansible নিজে যোগ করে: ফাইলটি কে বানিয়েছে তা জানায়।
user www-data;
worker_processes {{ ansible_facts['processor_count'] | default(2) }};
pid /run/nginx.pid;
events {
    worker_connections {{ nginx_worker_connections | default(1024) }};
}
http {
```

```
    sendfile on;
    tcp_nopush on;
    keepalive_timeout 65;
    types_hash_max_size 2048;
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    # Upstream: API সার্ভারগুলো Loop দিয়ে তৈরি
    upstream api_backend {
{% for server in groups['apiservers'] %}
        server {{ hostvars[server]['ansible_host'] }}:{{ api_port | default(4000) }};
{% endfor %}
    }

    server {
        listen {{ http_port | default(80) }}{% if ssl_enabled | default(false) %} ssl{% endif
%};
        server_name {{ server_name | default('technova.example') }};
        root /var/www/technova;
        index index.html;

        location / {
            try_files $uri $uri/ =404;
        }

        location /api/ {
            proxy_pass http://api_backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
{% if ssl_enabled | default(false) %}
        ssl_certificate /etc/ssl/certs/technova.example.crt;
        ssl_certificate_key /etc/ssl/private/technova.example.key;
{% endif %}
    }
}

# File: ~/technova-ansible/nginx-template.yml — Template ব্যবহার করা Playbook
---
- name: Deploy Nginx from template
  hosts: webservers
  become: true
  vars:
    http_port: 80
    server_name: technova.example
    nginx_worker_connections: 1024
    ssl_enabled: false
    api_port: 4000
  tasks:
    - name: Install Nginx
      apt:
        name: nginx
        state: present
        update_cache: true
    - name: Deploy Nginx configuration from template
      template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        owner: root
```

```
        group: root
        mode: "0644"
        validate: /usr/sbin/nginx -t -c %s
      notify: Reload Nginx
    - name: Ensure Nginx is started and enabled
      service:
        name: nginx
        state: started
        enabled: true
  handlers:
    - name: Reload Nginx
      service:
        name: nginx
        state: reloaded
```

এই Template-এর বিশেষত্ব লক্ষ করুন: `groups['apiservers']` দিয়ে Inventory-র API Group-এর সব সার্ভারের IP নিয়ে Upstream বানানো হয়েছে — API সার্ভার যোগ হলে Template বদলাতে হয় না, Inventory বদলালেই হয়। `hostvars[server]['ansible_host']` দিয়ে অন্য Host-এর Variable পড়া যায় — এটাই `hostvars` Dictionary-র কাজ।

১০.১০ ধাপে ধাপে বাস্তবায়ন

ধাপ ১ — Template তৈরি: `templates/nginx.conf.j2` ফাইলটি তৈরি করুন; `{{ }}` ও `{% %}` গঠনগুলো মিলিয়ে নিন। **ধাপ ২ — Playbook তৈরি:** উপরের `nginx-template.yml` তৈরি করুন। **ধাপ ৩ — Dry Run:** `ansible-playbook nginx-template.yml --check --diff` চালিয়ে দেখুন কী কনফিগারেশন তৈরি হবে — Diff-এ পূরণ করা ফাইলটি দেখা যাবে। **ধাপ ৪ — আসল চালানো:** `ansible-playbook nginx-template.yml` চালান; Handler Reload চলবে। **ধাপ ৫ — ভিন্ন মান পরীক্ষা:** `-e "http_port=8080 ssl_enabled=false"` দিয়ে চালিয়ে দেখুন পোর্ট বদলে গেছে; তারপর `group_vars/webservers.yml`-এ `ssl_enabled: true` লিখে (Certificate ছাড়া) কী হয় দেখুন — `validate` ব্যর্থ হয়ে Deploy থেমে যাবে, এটাই প্রত্যাশিত। **ধাপ ৬ — Loop পরীক্ষা:** একটি Loop Task-এ ইচ্ছাকৃতভাবে একটি ভুল Package নাম দিয়ে দেখুন কোন আইটেমে ব্যর্থ হয়। **ধাপ ৭ — যাচাই:** Managed Node-এ `cat /etc/nginx/nginx.conf` চালিয়ে দেখুন Template-এর `{{ }}` জায়গায় আসল মান বসেছে।

১০.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

```
$ ansible-playbook nginx-template.yml --check --diff
# চিত্রণমূলক আউটপুট (অংশবিশেষ):
TASK [Deploy Nginx configuration from template] ********************
--- before: /etc/nginx/nginx.conf
+++ after: /tmp/ansible-template/nginx.conf
@@ -1,5 +1,5 @@
 user www-data;
-worker_processes auto;
+worker_processes 2;
 events {
-    worker_connections 768;
+    worker_connections 1024;
 }
     upstream api_backend {
+        server 203.0.113.20:4000;
+        server 203.0.113.21:4000;
     }
```

```
# ব্যাখ্যা: Diff-এ দেখা যাচ্ছে Template কীভাবে পূরণ হয়েছে —
# worker_processes-এ CPU সংখ্যা, Upstream-এ দুটি API সার্ভারের IP।
# ssl_enabled false থাকায় ssl_certificate লাইনগুলো আসেনি।

$ ansible-playbook nginx-template.yml
# চিত্রণমূলক আউটপুট:
TASK [Deploy Nginx configuration from template] ********************
changed: [web01]
RUNNING HANDLER [Reload Nginx] *************************************
changed: [web01]
# ব্যাখ্যা: Template বদলেছে বলে changed, আর Handler Reload চলেছে।
# দ্বিতীয়বার চালালে changed: false হবে — Template-এর ফল একই থাকবে।
```

Loop-এর আউটপুটে প্রতিটি আইটেম আলাদা লাইনে দেখায়: `ok: [web01] => (item=nginx)` — কোন আইটেমে কী হলো তা স্পষ্ট থাকে। `loop_control.label` না দিলে Dictionary-র List-এর ক্ষেত্রে পুরো Dictionary ছাপা হয়, যা পড়া কঠিন।

১০.১২ সাধারণ ভুল

(১) **Template-এ `when`-এর মতো `{{ }}` ভুলে যাওয়া বা বাড়তি দেওয়া:** Template-এ মান ছাপাতে `{{ }}` লাগে, কিন্তু `{% if %}`-এর ভেতরে Variable-র নাম সরাসরি লিখতে হয় — `{% if {{ ssl_enabled }} %}` ভুল। (২) **`item`-কে Nested Loop-এ ব্যবহার:** ভেতরের Loop বাইরের `item`-কে ঢেকে দেয়; `loop_var` দিয়ে আলাদা নাম দিন। (৩) **`loop`-এ String দেওয়া:** `loop: nginx` লিখলে Jinja2 String-টিকে অক্ষরের List মনে করে প্রতিটি অক্ষরে Loop চালাবে; সবসময় List দিন: `loop: [nginx]`। (৪) **`validate` না দেওয়া:** Template-এ ভুল থাকলে ভুল কনফিগারেশন Deploy হয়ে Service বন্ধ হয়ে যায়; `validate` দিয়ে যাচাই করুন। (৫) **Filter-এ বন্ধনীর ভুল:** `{{ port | default 80 }}` ভুল — Filter-এর Argument বন্ধনীতে লিখতে হয়: `default(80)`। (৬) **`with_items` ও `loop` একসঙ্গে লেখা:** একটি Task-এ দুটি Loop পদ্ধতি একসঙ্গে লেখা যায় না — একটিই বেছে নিন।

১০.১৩ সমস্যা সমাধান (Troubleshooting)

সমস্যা: `ERROR! TemplateSyntaxError: expected token 'end of print statement', got '...'`। Template-এ Jinja2 গঠনের ভুল — সাধারণত `{{ }}`-এর ভেতরে উদ্ধৃতি বা বন্ধনীর অমিল; `jinja2` Parser দিয়ে আলাদাভাবে যাচাই করুন, অথবা `--check --diff` দিয়ে লাইনটি খুঁজুন। **সমস্যা: `ERROR! 'dict object' has no attribute 'ansible_host'`।** `hostvars[server]`-এ সার্ভারের নাম ভুল, অথবা সেই Host Facts সংগ্রহ করেনি; `groups['apiservers']`-এর নাম মিলিয়ে নিন। **সমস্যা: Template-এর মান খালি আসছে।** Variable-টির নামের বানান বা Scope যাচাই করুন; `debug: var=http_port` দিয়ে মান দেখুন। **সমস্যা: Loop-এর একটি আইটেম ব্যর্থ হয়ে পুরো Task থেমে গেল।** এটাই ডিফল্ট আচরণ; কোন আইটেম ব্যর্থ হয়েছে তা আউটপুটে `(item=...)` দেখে চিহ্নিত করুন; সব আইটেম চালাতে চাইলে অধ্যায় ১১-এর `ignore_errors` বা `failed_when` ব্যবহার করুন। **সমস্যা: `validate` ব্যর্থ হচ্ছে।** `%s` ভুলে গেলে যাচাই কমান্ড ফাইলের পথ পায় না; আর যাচাই কমান্ডটি Managed Node-এ থাকতে হবে — Control Node-এ নয়।

১০.১৪ নিরাপত্তা বিবেচনা

Template-এ Secret (Password, Private Key) সরাসরি লিখবেন না — Template ফাইলটি Git-এ থাকে; Secret Vault থেকে Variable হিসেবে আনুন ও Template-এ `{{ db_password }}` হিসেবে ব্যবহার করুন। `validate` ছাড়া Template Deploy করবেন না — ভুল কনফিগারেশন Service বন্ধ করে দিতে পারে, আর ভুল SSL কনফিগারেশন নিরাপত্তা-ঝুঁকি

তৈরি করে। Template-এ `hostvars` দিয়ে অন্য Host-এর Variable পড়ার সময় সতর্ক থাকুন — কোনো Host-এর Secret অন্য Host-এর কনফিগারেশনে চলে যেতে পারে; শুধু প্রয়োজনীয় Variable-ই Template-এ ব্যবহার করুন। আর Loop-এ ব্যবহারকারীর দেওয়া তালিকা (যেমন বাইরের API থেকে আসা) সরাসরি ব্যবহার করবেন না — ইনপুট যাচাই না করে Loop চালালে অপ্রত্যাশিত ব্যবহারকারী বা ফাইল তৈরি হতে পারে।

১০.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

Production-এ Loop, Filter ও Template-এর নিয়মগুলো: (১) **ডেটা Variable-তে, যুক্তি Task-এ:** Package-র তালিকা, ব্যবহারকারীর তালিকা, Upstream সার্ভার — সব `group_vars`-এ List হিসেবে রাখুন; Task-এ শুধু Loop লিখুন। এতে নতুন সার্ভার বা Package যোগ করতে Playbook ছুঁতে হয় না। (২) **Template-এ `validate` বাধ্যতামূলক:** Nginx, Apache, PostgreSQL — প্রতিটি কনফিগারেশন Template-এ সংশ্লিষ্ট যাচাই কমান্ড লিখুন। (৩) **`ansible_managed` মন্তব্য:** Template-এর প্রথম লাইনে `# {{ ansible_managed }}` লিখুন — যাতে কেউ হাতে ফাইলটি বদলাতে গেলে বুঝতে পারে এটি Ansible-এর তৈরি, হাতে বদলালে পরের Deploy-এ মুছে যাবে। (৪) **Loop-এর আকার সীমিত রাখুন:** শত শত আইটেমের Loop ধীর হয়; বড় তালিকার জন্য `loop_control.pause` বা ব্যাচে ভাগ করার কথা ভাবুন। (৫) **`with_*` থেকে `loop`-এ স্থানান্তর:** পুরনো Playbook-এ `with_items` থাকলে ধীরে ধীরে `loop`-এ বদলান — Ansible-এর ভবিষ্যৎ সংস্করণে `with_*` সরিয়ে দেওয়া হতে পারে।

১০.১৬ অধ্যায়ের সারাংশ

Loop একই Task-কে তালিকার প্রতিটি আইটেমে চালায় — `item` দিয়ে বর্তমান আইটেম পাওয়া যায়, `loop_control` দিয়ে নাম ও লেবেল নিয়ন্ত্রণ করা যায়; পুরনো `with_items` চিনে রাখুন, নতুন কোডে `loop` লিখুন। Filter মানকে রূপান্তর করে — `default`, `join`, `to_json` সবচেয়ে দরকারি। Template হলো Jinja2 দিয়ে লেখা কনফিগারেশন: `{{ }}` মান ছাপায়, `{% if %}` ও `{% for %}` শর্ত ও পুনরাবৃত্তি করে; `template` Module তা পূরণ করে `validate` দিয়ে যাচাই করে Deploy করে। `nginx.conf.j2` দেখিয়েছে: একই Template, ভিন্ন Variable — ভিন্ন সার্ভারে ভিন্ন কনফিগারেশন, আর Inventory বদলালেই Upstream বদলে যায়। পরের অধ্যায়ে আমরা ত্রুটি-পরিচালনা, Tags ও Execution নিয়ন্ত্রণ শিখব।

১০.১৭ জ্ঞান যাচাই

১. `loop` ও `with_items`-এর পার্থক্য কী? নতুন Playbook-এ কোনটি ব্যবহার করা উচিত, আর কেন? ২. Nested Loop-এ `loop_var` কেন প্রয়োজন হয়? একটি উদাহরণ দিন। ৩. `{{ packages | join(', ') }}` ও `{{ server_name | default('technova.example') | upper }}` — প্রতিটি Filter কী করে? ৪. `template` Module ও `copy` Module-এর পার্থক্য কী? `validate` কেন গুরুত্বপূর্ণ? ৫. Template-এ আক্ষরিক `{{ }}` লিখতে হলে কী করতে হয়, আর কেন?

১০.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

নিজের ল্যাবে: (ক) অধ্যায়ের `nginx.conf.j2` Template ও `nginx-template.yml` Playbook তৈরি করে `--check --diff` চালিয়ে Diff সংরক্ষণ করুন; (খ) আসলভাবে চালিয়ে Managed Node-এ তৈরি `nginx.conf` দেখে Template-এর মানগুলো মিলিয়ে নিন; (গ) `-e "http_port=8080"` দিয়ে চালিয়ে পোর্ট বদল প্রমাণ করুন; (ঘ) Loop দিয়ে তিনটি Package ইনস্টল ও দুটি ব্যবহারকারী তৈরি করার Task যোগ করুন; (ঙ) Template-এ `{% if %}` দিয়ে একটি শর্তসাপেক্ষ লাইন যোগ করুন (যেমন `ssl_enabled` true হলে একটি মন্তব্য লাইন) ও দুই মানেই পরীক্ষা করুন। জমা দিন: Template, Playbook, Diff আউটপুট ও তৈরি হওয়া কনফিগারেশন ফাইল।

অধ্যায় 11 • PART II — Ansible Inventory ও Execution

Chapter 11 — Error Handling, Tags, Blocks, and Execution Control

১১.১ ভূমিকা

এতদিন আমরা ধরে নিয়েছি সব Task সফল হবে। বাস্তব Production-এ তা হয় না: Package Repository সাময়িকভাবে বন্ধ থাকে, একটি সার্ভারের Disk ভরে যায়, কনফিগারেশন যাচাই ব্যর্থ হয়, নেটওয়ার্ক মাঝখানে ছিঁড়ে যায়। প্রশ্ন হলো: একটি Task ব্যর্থ হলে Ansible কী করবে — পুরো Playbook থামিয়ে দেবে, নাকি এগিয়ে যাবে? আর আপনি কি তা নিয়ন্ত্রণ করতে পারবেন? এই অধ্যায়ের বিষয় সেই নিয়ন্ত্রণ: **Block/Rescue/Always** দিয়ে "চেষ্টা করো, ব্যর্থ হলে উদ্ধার করো, যাই হোক পরিষ্কার করো" লেখা; `ignore_errors`, `failed_when`, `any_errors_fatal`, `max_fail_percentage` দিয়ে ব্যর্থতার অর্থ ঠিক করা; **Tags** দিয়ে Playbook-এর অংশবিশেষ চালানো; আর `serial`, `strategy` (Linear বনাম Free) ও `throttle` দিয়ে কতগুলো সার্ভারে, কী ক্রমে, কত দ্রুত কাজ চলবে তা নিয়ন্ত্রণ করা। এগুলো না জানলে Production Deploy হয় ভাগ্যের খেলা; জানলে তা হয় নিয়ন্ত্রিত প্রকৌশল।

১১.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) `block` / `rescue` / `always` দিয়ে ত্রুটি-উদ্ধারের কাঠামো লিখতে; (২) `ignore_errors` ও `failed_when`-এর পার্থক্য ব্যাখ্যা ও প্রয়োগ করতে; (৩) `any_errors_fatal` ও `max_fail_percentage` দিয়ে কখন পুরো Play থামবে তা ঠিক করতে; (৪) Tags দিয়ে `--tags` ও `--skip-tags` ব্যবহার করে Playbook-এর অংশবিশেষ চালাতে; (৫) `serial` দিয়ে Rolling Update করতে; (৬) Linear ও Free Strategy-র পার্থক্য বুঝে সঠিকটি বেছে নিতে; (৭) `throttle` দিয়ে সমান্তরাল কাজের হার সীমিত করতে।

১১.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৮-এর Playbook, Task ও Handler, অধ্যায় ৯-এর `when`, `changed_when` ও `failed_when`, আর অধ্যায় ১০-এর Loop জানা থাকতে হবে — Block-এর ভেতরে Task, Loop ও Conditional — সব একসঙ্গে ব্যবহৃত হয়। Rolling Update-এর ধারণা (একটি সার্ভার আপডেট করে যাচাই, তারপর পরেরটি) বুঝতে Production Deploy-এর প্রাথমিক অভিজ্ঞতা থাকলে ভালো।

১১.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: যেকোনো জটিল কাজে তিন ধরনের ব্যর্থতা থাকে — প্রত্যাশিত ব্যর্থতা (যেমন ফাইলটি নাও থাকতে পারে — তা স্বাভাবিক), পুনরুদ্ধারযোগ্য ব্যর্থতা (যেমন Service Restart ব্যর্থ হলে পুরনো কনফিগারেশনে ফিরে যাওয়া যায়) ও মারাত্মক ব্যর্থতা (যেমন ডেটাবেস Migration ব্যর্থ হলে আর এগোনো উচিত নয়)। প্রতিটির জন্য আলাদা প্রতিক্রিয়া দরকার: প্রত্যাশিত ব্যর্থতাকে ব্যর্থতা ধরা উচিত নয় (`failed_when`); পুনরুদ্ধারযোগ্য ব্যর্থতার জন্য বিকল্প পথ দরকার (`rescue`); মারাত্মক ব্যর্থতায় সব থামিয়ে দেওয়া উচিত (`any_errors_fatal`)। আর Execution নিয়ন্ত্রণের নীতিটি হলো বিস্ফোরণের ব্যাসার্ধ সীমিত করা (Blast Radius): একসঙ্গে সব সার্ভারে Deploy করলে ভুল হলে সব সার্ভার একসঙ্গে ভাঙে; একবারে একটি সার্ভারে করলে (`serial: 1`) ভুল হলে শুধু একটি সার্ভার ভাঙে, বাকিগুলো বাঁচে। Tags, Serial, Strategy ও Throttle — সবই এই এক নীতির প্রয়োগ: পরিবর্তনকে ছোট ছোট, নিয়ন্ত্রিত ধাপে ভাগ করা।

> **CONCEPT:** Ansible-এর ডিফল্ট আচরণ হলো "একটি Host ব্যর্থ হলে সেই Host-এর Play থামাও, বাকি Host-গুলো চালিয়ে যাও।" এটি অনেকের কাছে অপ্রত্যাশিত মনে হয় — Playbook-টি "সবুজ" দেখালেও একটি Host ব্যর্থ থাকতে পারে। PLAY RECAP-এ `failed=1` দেখলেই বুঝবেন কোনো Host বাদ পড়েছে। `any_errors_fatal: true` এই আচরণ বদলে দেয়: একটি Host ব্যর্থ হলে সব Host-এর Play থেমে যায়। কোনটি সঠিক তা নির্ভর করে কাজের ওপর: স্বাধীন Web সার্ভারে একটি ব্যর্থ হলে বাকিগুলো চলতে পারে; কিন্তু ডেটাবেস Migration-এ একটি ব্যর্থ হলে বাকিগুলো চালানো বিপজ্জনক।

১১.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

block / rescue / always — চেষ্টা, উদ্ধার, পরিষ্কার

`block` হলো Task-এর একটি দল, যার ওপরে একসঙ্গে `when`, `tags`, `become` প্রয়োগ করা যায়। `block`-এর কোনো Task ব্যর্থ হলে `rescue` সেকশনের Task-গুলো চলে — যেমন পুরনো কনফিগারেশন ফিরিয়ে আনা, Service আগের অবস্থায় ফেরানো, Alert পাঠানো। `always` সেকশনের Task-গুলো সফল-ব্যর্থ নির্বিশেষে সবসময় চলে — যেমন অস্থায়ী ফাইল মোছা, লগ লেখা। `rescue`-এর Task-ও ব্যর্থ হলে Play ব্যর্থ হয়; `always` ব্যর্থ হলেও Play-এর ফল বদলায় না (তবে সতর্কতা দেখায়)। Block-এর ভেতরে `block` লেখা যায় (Nested Block), তবে গভীরতা বাড়লে পড়া কঠিন হয় — দুই স্তরের বেশি না লেখাই ভালো।

ignore_errors, failed_when, any_errors_fatal, max_fail_percentage

`ignore_errors: true` লিখলে Task ব্যর্থ হলেও Play এগিয়ে যায় — তবে সতর্কতা: এটি ব্যর্থতাকে লুকিয়ে ফেলে; শুধু তখনই ব্যবহার করুন যখন ব্যর্থতা প্রত্যাশিত ও নিরীহ (যেমন একটি ঐচ্ছিক ফাইল মুছে ফেলা)। `failed_when` ব্যর্থতার সংজ্ঞা বদলায়: `failed_when: result.rc != 0 and "not found" not in result.stderr` — "not found" থাকলে ব্যর্থ ধরা হবে না। `any_errors_fatal: true` Play-স্তরে লিখলে যেকোনো Host-এ যেকোনো ব্যর্থতায় পুরো Play থেমে যায়। `max_fail_percentage: 30` লিখলে ৩০%-এর বেশি Host ব্যর্থ হলে বাকি Host-গুলোর Play থেমে যায় — যেমন ১০টি সার্ভারের মধ্যে ৪টি ব্যর্থ হলে আর এগোনো হবে না; এটি Rolling Update-এ "খারাপ Deploy তাড়াতাড়ি থামানো"-র উপায়।

Tags — অংশবিশেষ চালানো

Task, Block, Play বা Role-এ `tags: [nginx, install]` লিখে চিহ্নিত করা যায়। `ansible-playbook site.yml --tags nginx` দিয়ে শুধু `nginx` Tag-যুক্ত Task চলে; `--skip-tags install` দিয়ে `install` Tag-যুক্ত Task বাদ যায়। বিশেষ Tag: `always` — সবসময় চলে, `--tags` দিলেও বাদ যায় না (যদি না `--skip-tags always` দেওয়া হয়); `never` — কখনো চলে না, শুধু `--tags never` স্পষ্টভাবে দিলে চলে। Handler-এও Tag দেওয়া যায়, তবে Handler শুধু Notify হলেই চলে — Tag দিয়ে Handler সরাসরি চালানো যায় না। `--list-tags` দিয়ে Playbook-এর সব Tag দেখা যায়।

serial, strategy, throttle — কতগুলো, কী ক্রমে, কত দ্রুত

`serial: 1` Play-স্তরে লিখলে Host-গুলো একবারে একটি করে প্রক্রিয়া করা হয় — Rolling Update: প্রথম সার্ভার আপডেট হয়ে যাচাই হলে পরেরটি। `serial: 2` বা `serial: "30%"` দিয়ে ব্যাচের আকার ঠিক করা যায়; List দিয়ে ভিন্ন আকারের ব্যাচ দেওয়া যায়: `serial: [1, 2, 5]` — প্রথমে ১টি, তারপর ২টি, তারপর ৫টি। `strategy: linear` (ডিফল্ট) — একটি Task সব Host-এ শেষ হলে পরের Task; `strategy: free` — প্রতিটি Host নিজের গতিতে এগিয়ে যায়, কোনো Task-এ কেউ কারো জন্য অপেক্ষা করে না; Free দ্রুত, কিন্তু Task-এর ক্রম Host-ভেদে আলাদা হয় — তাই

Free শুধু স্বাধীন Task-এ ব্যবহার করুন। `throttle: 1` Task-স্তরে লিখলে ওই Task একবারে একটি Host-এ চলে — যেমন API Rate Limit থাকা কাজে; এটি `serial`-এর মতো পুরো Play-কে নয়, শুধু ওই Task-কে সীমিত করে।

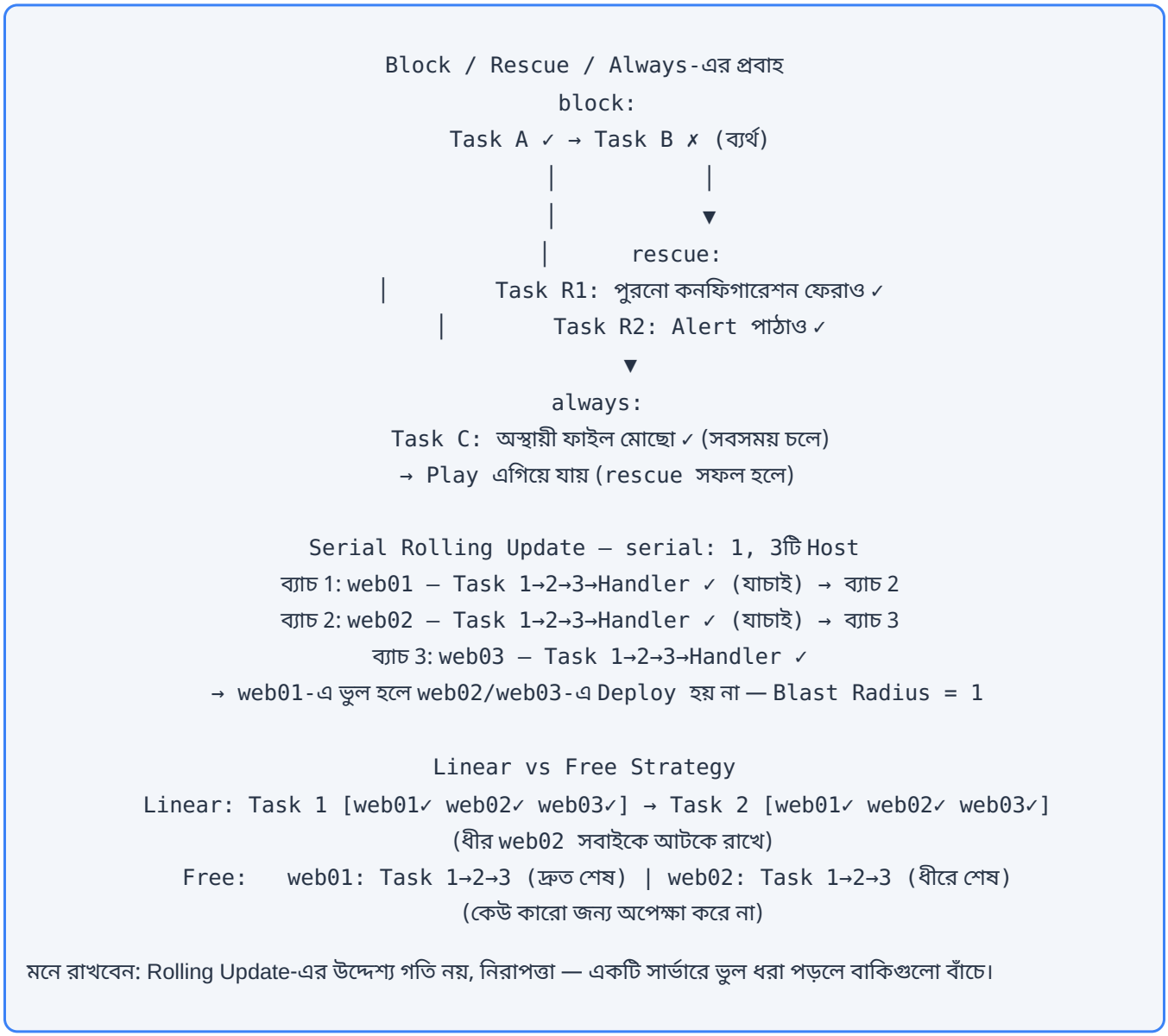
নিয়ন্ত্রণ	স্তর	কাজ	উদাহরণ
block/rescue/always	Task দল	চেষ্টা, উদ্ধার, পরিষ্কার	`rescue:` পুরনো কনফিগারেশন ফেরানো
ignore_errors	Task	ব্যর্থতা উপেক্ষা করে এগোনো	`ignore_errors: true`
failed_when	Task	ব্যর্থতার সংজ্ঞা বদলানো	`failed_when: rc != 0`
any_errors_fatal	Play	একটি ব্যর্থতায় সব থামানো	`any_errors_fatal: true`
max_fail_percentage	Play	নির্দিষ্ট হারে ব্যর্থতায় থামানো	`max_fail_percentage: 30`
--tags / --skip-tags	কমান্ড লাইন	অংশবিশেষ চালানো/বাদ দেওয়া	`--tags nginx`
serial	Play	ব্যাচে ব্যাচে Rolling Update	`serial: 1`
strategy: free	Play	প্রতিটি Host নিজের গতিতে	`strategy: free`
throttle	Task	একটি Task-এর সমান্তরালতা সীমিত করা	`throttle: 1`

১১.৬ অভ্যন্তরীণ কার্যপ্রণালি

Block চালালে Ansible ভেতরে Task-গুলোকে একটি দল হিসেবে দেখে: Block-এর কোনো Task ব্যর্থ হলে Ansible বাকি Block Task-গুলো Skip করে `rescue`-এ যায়; `rescue` শেষ হলে `always` চলে, তারপর Play এগিয়ে যায় — যেন Block-টি সফল হয়েছে (যদি `rescue` সফল হয়)। `serial` চালালে Ansible Host-গুলোকে ব্যাচে ভাগ করে: প্রথম ব্যাচের সব Host-এ পুরো Play শেষ হলে দ্বিতীয় ব্যাচ শুরু হয় — প্রতিটি ব্যাচে Facts সংগ্রহ থেকে Handler পর্যন্ত সব চলে। Linear Strategy-তে Task Scheduler প্রতিটি Task-এর জন্য সব Host-এর ফল না আসা পর্যন্ত অপেক্ষা করে; Free Strategy-তে প্রতিটি Host-এর জন্য আলাদা Worker থাকে, যারা Task-এর তালিকা ধরে নিজের গতিতে এগিয়ে যায় — একটি ধীর Host আর কাউকে আটকায় না। `throttle` একটি Task-এর জন্য একটি Semaphore-এর মতো কাজ করে: একবারে নির্দিষ্ট সংখ্যক Host-ই ওই Task চালাতে পারে, বাকিরা অপেক্ষা করে। আর `max_fail_percentage` প্রতিটি Task-এর পরে ব্যর্থ Host-এর হার গুনে: হার সীমা ছাড়ালে Scheduler বাকি Host-গুলোর Task বাতিল করে দেয়। লক্ষ করুন: এই সব নিয়ন্ত্রণ Control Node-এ হয় — Managed Node জানে না সে কোন ব্যাচে আছে বা Strategy কী।

> **WARNING:** `ignore_errors: true` সবচেয়ে বেশি অপব্যবহার হওয়া নির্দেশ। এটি লিখলে ব্যর্থ Task-টি `ok` নয়, `failed` হিসেবেই চিহ্নিত হয় — কিন্তু Play থামে না। সমস্যা হলো: পরের Task-গুলো ধরে নেয় আগের Task সফল হয়েছে। যেমন Nginx ইনস্টল ব্যর্থ হওয়ার পরও কনফিগারেশন Deploy-এর Task চলতে থাকে, আর শেষে PLAY RECAP-এ `failed=1` দেখে কেউ খেয়ালই করে না। নিয়ম: `ignore_errors` শুধু তখনই ব্যবহার করুন যখন ব্যর্থতা সত্যিই নিরীহ ও প্রত্যাশিত; অন্যথায় `failed_when` দিয়ে ব্যর্থতার সংজ্ঞা ঠিক করুন, অথবা `block/rescue` দিয়ে উদ্ধারের পথ লিখুন।

১১.৭ আর্কিটেকচার ডায়াগ্রাম



মনে রাখবেন: Rolling Update-এর উদ্দেশ্য গতি নয়, নিরাপত্তা — একটি সার্ভারে ভুল ধরা পড়লে বাকিগুলো বাঁচে।

১১.৮ ব্যবহারিক উদাহরণ

```
# উদাহরণ 1 — block/rescue/always দিয়ে নিরাপদ কনফিগারেশন Deploy
- name: Deploy Nginx config safely
  hosts: webservers
  become: true
  tasks:
    - block:
        - name: Backup current config
          copy:
            src: /etc/nginx/nginx.conf
            dest: /etc/nginx/nginx.conf.bak
            remote_src: true
        - name: Deploy new config from template
          template:
            src: templates/nginx.conf.j2
            dest: /etc/nginx/nginx.conf
            validate: /usr/sbin/nginx -t -c %s
```

```
          notify: Reload Nginx
      rescue:
        - name: Restore backup config
          copy:
            src: /etc/nginx/nginx.conf.bak
            dest: /etc/nginx/nginx.conf
            remote_src: true
          notify: Reload Nginx
        - name: Report rollback
          debug:
            msg: "Config deploy failed on {{ inventory_hostname }}, restored backup"
      always:
        - name: Remove backup file
          file:
            path: /etc/nginx/nginx.conf.bak
            state: absent
# ব্যাখ্যা: remote_src: true — src ফাইলটি Managed Node-এই আছে, Control Node-এ নয়।

# উদাহরণ 2 — failed_when দিয়ে ব্যর্থতার সংজ্ঞা
- name: Check if a user exists
  command: id deploy
  register: user_check
  changed_when: false
  failed_when: user_check.rc != 0 and "no such user" not in user_check.stderr
# ব্যাখ্যা: ব্যবহারকারী না থাকলে (rc=1) তা ব্যর্থতা নয় — পরে when দিয়ে
# user_check.rc != 0 পরীক্ষা করে ব্যবহারকারী তৈরি করা যাবে।

# উদাহরণ 3 — Tags দিয়ে অংশবিশেষ চালানো
- name: Install Nginx
  apt:
    name: nginx
    state: present
  tags: [nginx, install]
- name: Deploy homepage
  copy:
    src: files/index.html
    dest: /var/www/technova/index.html
  tags: [nginx, content]
# চালানো: ansible-playbook site.yml --tags content   (শুধু Homepage)
# চালানো: ansible-playbook site.yml --skip-tags install (ইনস্টল বাদে সব)

# উদাহরণ 4 — Rolling update: serial + max_fail_percentage
- name: Rolling update of webservers
  hosts: webservers
  become: true
  serial: 1
  max_fail_percentage: 0
  tasks:
    - name: Update Nginx to latest
      apt:
        name: nginx
        state: latest
    - name: Verify website responds
      command: curl -fs http://localhost/
      changed_when: false
# ব্যাখ্যা: serial: 1 — একবারে একটি সার্ভার; max_fail_percentage: 0 —
# একটি ব্যর্থ হলেই থামো; যাচাই Task-টি প্রতিটি ব্যাচে চলে।
```

১১.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচে এই অধ্যায়ের সব নিয়ন্ত্রণ একসঙ্গে ব্যবহার করা একটি সম্পূর্ণ Production-ধাঁচের Playbook দেওয়া হলো — Rolling Update, Block/Rescue, Tags ও Throttle-সহ।

```
# File: ~/technova-ansible/rolling-update.yml — Production Rolling Update
---
- name: Rolling update of TechNova webservers
  hosts: webservers
  become: true
  serial: 1
  max_fail_percentage: 0
  any_errors_fatal: false
  force_handlers: true
  vars:
    http_port: 80
  tasks:
    - name: Pre-check — website is up before update
      command: curl -fs http://localhost:{{ http_port }}/
      changed_when: false
      tags: [check]

    - block:
        - name: Update apt cache
          apt:
            update_cache: true
            cache_valid_time: 3600
          tags: [nginx, install]

        - name: Upgrade Nginx package
          apt:
            name: nginx
            state: latest
          notify: Reload Nginx
          tags: [nginx, install]

        - name: Deploy Nginx configuration from template
          template:
            src: templates/nginx.conf.j2
            dest: /etc/nginx/nginx.conf
            mode: "0644"
            validate: /usr/sbin/nginx -t -c %s
          notify: Reload Nginx
          tags: [nginx, config]

        - name: Flush handlers before verification
          meta: flush_handlers
          tags: [nginx, config]

        - name: Verify website responds after update
          command: curl -fs http://localhost:{{ http_port }}/
          changed_when: false
          tags: [check]
      rescue:
        - name: Roll back Nginx package (best effort)
          apt:
            name: nginx
            state: present
```

```
          ignore_errors: true
          tags: [nginx, rollback]
        - name: Report rollback for this host
          debug:
            msg: "ROLLBACK on {{ inventory_hostname }} — update failed, check manually"
          tags: [nginx, rollback]
      always:
        - name: Record update attempt in log
          lineinfile:
            path: /var/log/technova-update.log
            line: "{{ ansible_date_time.iso8601 }} update attempted on
{{ inventory_hostname }}"
            create: true
            mode: "0644"
          tags: [always]

    - name: Notify monitoring (rate-limited)
      command: curl -fs -X POST https://monitor.technova.example/api/deploy -d
"host={{ inventory_hostname }}"
      changed_when: false
      failed_when: false
      throttle: 1
      tags: [notify]

  handlers:
    - name: Reload Nginx
      service:
        name: nginx
        state: reloaded
    - name: Restart Nginx
      service:
        name: nginx
        state: restarted
```

এই Playbook-টি লক্ষ করুন: `serial: 1` দিয়ে একবারে একটি সার্ভার, `max_fail_percentage: 0` দিয়ে একটি ব্যর্থতায় থামা, `block/rescue` দিয়ে উদ্ধার, `always` দিয়ে লগ লেখা, `throttle: 1` দিয়ে Monitoring API-তে একবারে একটি কল — প্রতিটি নিয়ন্ত্রণের একটি করে সুনির্দিষ্ট কাজ আছে।

১১.১০ ধাপে ধাপে বাস্তবায়ন

ধাপ ১ — Playbook তৈরি: উপরের `rolling-update.yml` ও `templates/nginx.conf.j2` তৈরি করুন। **ধাপ ২ — Tag তালিকা দেখুন:** `ansible-playbook rolling-update.yml --list-tags` চালিয়ে সব Tag দেখুন। **ধাপ ৩ — শুধু যাচাই চালান:** `ansible-playbook rolling-update.yml --tags check` চালিয়ে দেখুন শুধু যাচাই Task-গুলো চলে। **ধাপ ৪ — Dry Run:** `ansible-playbook rolling-update.yml --check --diff --limit web01` দিয়ে একটি সার্ভারে কী হবে দেখুন। **ধাপ ৫ — আসল Rolling Update:** `ansible-playbook rolling-update.yml` চালান ও লক্ষ করুন: web01 শেষ হলে তবে web02 শুরু হয়। **ধাপ ৬ — ব্যর্থতা পরীক্ষা:** Template-এ ইচ্ছাকৃতভাবে ভুল লিখে (যেমন `{{ undefined_var }}`) চালিয়ে দেখুন `rescue` চলে কি না ও Play থেমে যায় কি না। **ধাপ ৭ — Free Strategy পরীক্ষা:** Play-এ `strategy: free` যোগ করে আবার চালিয়ে সময়ের পার্থক্য লক্ষ করুন।

১১.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

```
$ ansible-playbook rolling-update.yml
# চিত্রণমূলক আউটপুট (অংশবিশেষ):
PLAY [Rolling update of TechNova webservers] ***********************
TASK [Pre-check — website is up before update] *********************
ok: [web01]
TASK [Upgrade Nginx package] ***************************************
changed: [web01]
TASK [Verify website responds after update] ************************
ok: [web01]
PLAY [Rolling update of TechNova webservers] ***********************
TASK [Pre-check — website is up before update] *********************
ok: [web02]
# ব্যাখ্যা: PLAY লাইনটি web01-এর জন্য একবার, web02-এর জন্য আবার দেখা যাচ্ছে —
# serial: 1-এর কারণে প্রতিটি ব্যাচে পুরো Play আলাদাভাবে চলে। PLAY RECAP-ও
# প্রতিটি ব্যাচের জন্য আলাদাভাবে দেখায়।

$ ansible-playbook rolling-update.yml --tags check
# চিত্রণমূলক আউটপুট:
TASK [Pre-check — website is up before update] *********************
ok: [web01]
TASK [Verify website responds after update] ************************
skipping: [web01]
# ব্যাখ্যা: দ্বিতীয় যাচাই Task-টি Block-এর ভেতরে থাকায় --tags check দিলেও
# Block-এর অন্য Task-গুলো Skip হয়েছে, আর Verify Task-টি Block-এর ভেতরে
# থাকায় Block-এর শর্তে Skip হয়েছে — Tag Block-স্তরে কাজ করে, ভেতরের
# Task-এর Tag-কে ঢেকে দিতে পারে। এজন্য Block-এ Tag দেওয়ার সময় সতর্ক থাকুন।

# Rescue-এর আউটপুট (Template ভুল থাকলে):
TASK [Deploy Nginx configuration from template] ********************
fatal: [web01]: FAILED! => {"msg": "'undefined_var' is undefined"}
TASK [Report rollback for this host] *******************************
ok: [web01] => {"msg": "ROLLBACK on web01 — update failed, check manually"}
# ব্যাখ্যা: Block-এর Task ব্যর্থ হওয়ায় Rescue চলেছে; web02/web03-এ Deploy
# হয়নি — max_fail_percentage: 0-এর কারণে Play থেমে গেছে।
```

১১.১২ সাধারণ ভুল

(১) **Block-এ Tag দিয়ে ভেতরের Task-এর Tag ঢেকে দেওয়া:** উপরের উদাহরণের মতো — Block-স্তরের Tag ভেতরের Task-গুলোতে প্রযোজ্য হয়; `--tags` দিয়ে ভেতরের একটি Task চালাতে চাইলে Block-এ Tag দেবেন না। (২) **`ignore_errors: true` Play-স্তরে দেওয়া:** সব ব্যর্থতা লুকিয়ে যায়; শুধু নির্দিষ্ট Task-এ দিন। (৩) **`serial` ছাড়া Production Deploy:** সব সার্ভারে একসঙ্গে Deploy করলে ভুল হলে সব সার্ভার একসঙ্গে ভাঙে — সবসময় `serial` ব্যবহার করুন। (৪) **Free Strategy-তে Handler-নির্ভর Task:** Free Strategy-তে Host-গুলো ভিন্ন গতিতে চলে, তাই একটি Host-এর Handler চলার আগেই অন্য Host-এর Task তার ওপর নির্ভর করতে পারে — Handler-নির্ভর কাজে Linear-ই নিরাপদ। (৫) **`max_fail_percentage`-কে শতাংশ না বুঝে লেখা:** `max_fail_percentage: 30` মানে ৩০% ব্যর্থ হলে থামবে — ছোট Inventory-তে (যেমন ৩টি সার্ভার) ১টি ব্যর্থ মানেই ৩৩%, তাই Play থেমে যাবে; ছোট Inventory-তে `0` বা `any_errors_fatal` স্পষ্ট। (৬) **`rescue`-এ আবার একই ভুল Task লেখা:** যে Task ব্যর্থ হয়েছে, Rescue-এ সেই একই Task আবার চালালে আবার ব্যর্থ হবে; Rescue-এ বিকল্প পথ লিখুন (Backup ফেরানো, পুরনো সংস্করণ ইনস্টল করা)।

১১.১৩ সমস্যা সমাধান (Troubleshooting)

সমস্যা: `--tags` দিলে প্রত্যাশিত Task চলছে না। `--list-tags` দিয়ে Tag-গুলো দেখুন; মনে রাখুন Block-স্তরের Tag ভেতরের Task-কে প্রভাবিত করে; আর `always` Tag-যুক্ত Task সবসময় চলে — তা বাদ দিতে `--skip-tags always` দিন। **সমস্যা: Rescue চলছে না।** ব্যর্থ Task-টি Block-এর ভেতরে আছে কি না যাচাই করুন; Block-এর বাইরের Task ব্যর্থ হলে Rescue চলে না। আর `ignore_errors: true` থাকলে Task ব্যর্থ ধরা হয় না — Rescue Trigger হয় না। **সমস্যা: Serial Deploy খুব ধীর।** `serial: 1`-এর বদলে `serial: "25%"` বা `serial: [1, 2, 5]` দিয়ে ব্যাচ বড় করুন; প্রথম ব্যাচ ছোট রেখে (Canary) পরের ব্যাচগুলো বড় করা ভালো অভ্যাস। **সমস্যা: Free Strategy-তে Handler সময়মতো চলছে না।** Free Strategy-তে প্রতিটি Host নিজের Play শেষে Handler চালায়; Handler-নির্ভর যাচাই Task-এর আগে `flush_handlers` লিখুন। **সমস্যা: `max_fail_percentage` কাজ করছে না।** এটি শুধু Play-স্তরে কাজ করে, Task-স্তরে নয়; আর `any_errors_fatal: true` থাকলে সেটি আগে কার্যকর হয়।

১১.১৪ নিরাপত্তা বিবেচনা

`ignore_errors: true` নিরাপত্তা-সংক্রান্ত Task-এ (যেমন Firewall নিয়ম, SSL কনফিগারেশন, ব্যবহারকারীর Permission) কখনো ব্যবহার করবেন না — ব্যর্থতা লুকিয়ে গেলে সার্ভারটি অনিরাপদ অবস্থায় চলতে থাকে, আর কেউ জানতেও পারে না। Rescue-এ লেখা Rollback পথটি নিজেই নিরাপদ কি না যাচাই করুন: পুরনো কনফিগারেশন ফেরানোর সময় যদি তাতে জানা দুর্বলতা থাকে, তাহলে Rollback-ই ঝুঁকি তৈরি করে। `serial` ব্যবহারের সময় মনে রাখুন: Rolling Update চলাকালীন কিছু সার্ভার নতুন সংস্করণে, কিছু পুরনো সংস্করণে থাকে — এই মিশ্র অবস্থায় API-র সংস্করণ-অসামঞ্জস্য (Version Skew) হতে পারে; তাই Update-এর আগে সামঞ্জস্য যাচাই করুন। আর `throttle: 1` দিয়ে API কল সীমিত করা শুধু Rate Limit-এর জন্য নয় — একসঙ্গে অনেক সার্ভার থেকে একই API-তে আক্রমণের মতো ট্রাফিক যাওয়া নিরাপত্তা-সিস্টেমে Alert তৈরি করতে পারে।

১১.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

Production-এ ত্রুটি-পরিচালনা ও Execution নিয়ন্ত্রণের নিয়মগুলো: (১) **সবসময় Rolling Update:** `serial: 1` বা `serial: "25%"` ছাড়া Production Deploy করবেন না; প্রথম ব্যাচে একটি সার্ভার (Canary) যাচাই করে তারপর বাকিগুলো। (২) **প্রতিটি Deploy-এ Pre-check ও Post-check:** Update-এর আগে ও পরে Website সাড়া দিচ্ছে কি না যাচাই করুন — যাচাই ব্যর্থ হলে `rescue` চলবে। (৩) **Tags-এর নামকরণ:** Tag-গুলো কাজের নামে দিন (`install`, `config`, `content`, `check`) — যাতে `--tags config` দিয়ে শুধু কনফিগারেশন Deploy করা যায়। (৪) **`max_fail_percentage` সব Production Play-তে:** একটি সীমা ঠিক করুন (সাধারণত 0 বা 10) — যাতে খারাপ Deploy নিজে থেমে যায়। (৫) **Rescue পথটি পরীক্ষা করুন:** Rollback কখনো পরীক্ষা না করে রাখবেন না — Staging-এ ইচ্ছাকৃতভাবে ব্যর্থতা ঘটিয়ে Rescue চলে কি না যাচাই করুন। (৬) **Free Strategy শুধু স্বাধীন কাজে:** Package ইনস্টল বা তথ্য সংগ্রহের মতো স্বাধীন Task-এ Free দ্রুত; কিন্তু Handler-নির্ভর বা ক্রম-নির্ভর কাজে Linear-ই নিরাপদ।

১১.১৬ অধ্যায়ের সারাংশ

Production-এ ব্যর্থতা অনিবার্য — প্রশ্ন হলো আপনি তার জন্য প্রস্তুত কি না। `block/rescue/always` দিয়ে চেষ্টা, উদ্ধার ও পরিষ্কারের কাঠামো লেখা যায়; `failed_when` দিয়ে ব্যর্থতার সংজ্ঞা ঠিক করা যায়; `ignore_errors` শুধু প্রত্যাশিত ও নিরীহ ব্যর্থতায় ব্যবহার করা উচিত; `any_errors_fatal` ও `max_fail_percentage` দিয়ে কখন পুরো Play থামবে তা ঠিক করা যায়। Tags দিয়ে Playbook-এর অংশবিশেষ চালানো যায়, `serial` দিয়ে Rolling Update করা যায়, Linear Strategy ক্রম নিশ্চিত করে আর Free Strategy গতি দেয়, আর `throttle` একটি Task-এর সমান্তরালতা সীমিত করে। এই অধ্যায় দিয়ে PART II শেষ হলো — আপনি এখন Inventory লিখতে, Ad-hoc Command চালাতে, Playbook তৈরি

করতে, Variable ও Template ব্যবহার করতে, আর ব্যর্থতা সামলে নিয়ন্ত্রিতভাবে Deploy করতে পারেন। PART III-এ আমরা এই ভিত্তির ওপরে Role, Collection, Vault ও Cloud Provisioning-এর মতো Production-স্তরের বিষয়গুলো গড়ে তুলব।

১১.১৭ জ্ঞান যাচাই

১. `block`-এর একটি Task ব্যর্থ হলে কী হয়? `rescue` ও `always` কখন চলে? ২. `ignore_errors: true` ও `failed_when: false`-এর পার্থক্য কী? কোনটি কখন ব্যবহার করা উচিত? ৩. `--tags config` ও `--skip-tags install` — প্রতিটি কী করে? `always` ও `never` Tag-এর বিশেষত্ব কী? ৪. `serial: 1` ও `max_fail_percentage: 0` একসঙ্গে ব্যবহার করলে Rolling Update-এ কী ঘটে? ৫. Linear ও Free Strategy-র পার্থক্য কী? Handler-নির্ভর কাজে কোনটি নিরাপদ, আর কেন?

১১.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

নিজের ল্যাবে: (ক) অধ্যায়ের `rolling-update.yml` তৈরি করে `--list-tags` ও `--tags check` চালিয়ে Tag-এর আচরণ নোট করুন; (খ) `--limit web01` দিয়ে একটি সার্ভারে Rolling Update চালান; (গ) Template-এ ইচ্ছাকৃত ভুল লিখে Rescue চলা ও Play থেমে যাওয়া পর্যবেক্ষণ করুন, তারপর Template ঠিক করে আবার চালান; (ঘ) একটি Task-এ `ignore_errors: true` ও আরেকটি Task-এ `failed_when` লিখে দুটির আউটপুটের পার্থক্য নোট করুন; (ঙ) Play-এ `strategy: free` যোগ করে চালিয়ে Linear-এর সঙ্গে সময়ের পার্থক্য মাপুন। জমা দিন: Playbook, Tag পরীক্ষা, Rescue পরীক্ষা ও Strategy তুলনার নোট — আর PART II-এর ছয়টি অধ্যায়ের একটি এক-পৃষ্ঠার সারাংশ নিজের ভাষায় লিখুন।

PART III • DEVOPS ENGINEERING MASTERY

Advanced Ansible Engineering

Role, Collection ও Galaxy, Ansible Vault ও secret ব্যবস্থাপনা, SSH ও privilege escalation, dynamic inventory ও API অটোমেশন।

অধ্যায় 12 • PART III — Roles, Collections, Vault & Cloud

Chapter 12 — Roles: Reusable Automation Architecture

১২.১ ভূমিকা

TechNova-র প্রথম দিকের Ansible Playbook-গুলো দেখলে একটি পরিচিত ছবি চোখে পড়ে: `site.yml` নামের একটি বিশাল ফাইলে তিনশো লাইন Task — Nginx ইনস্টল, Node.js বসানো, SSL সার্টিফিকেট কপি করা, ফায়ারওয়াল নিয়ম, লগ রোটেশন — সব একসঙ্গে। প্রথম সার্ভারে কাজটি চলেছিল, দ্বিতীয় সার্ভারে একই কাজের কিছু অংশ দরকার পড়তেই কেউ ফাইলটি কপি করে কিছু লাইন মুছে দিলেন, তৃতীয় সার্ভারে আবার কপি-পেস্ট, চতুর্থটিতে ভুল সংস্করণ। ছয় মাস পরে কেউ জানে না কোন ফাইলে আসল সত্য লেখা আছে — কোনটির Nginx কনফিগারেশন নতুন, কোনটিতে পুরনো নিরাপত্তা-ত্রুটি রয়ে গেছে। এই সমস্যার নাম **Copy-Paste Automation** — অটোমেশন আছে, কিন্তু পুনঃব্যবহারের কাঠামো নেই। এই অধ্যায়ের বিষয় **Role** সেই সমস্যারই সমাধান। Role হলো Ansible-এর পুনঃব্যবহারযোগ্য (Reusable) একক: একটি নির্দিষ্ট কাজ — যেমন Nginx বসানো বা Node.js অ্যাপ Deploy করা — তার সব Task, Handler, Variable, File ও Template একটিমাত্র ডিরেক্টরিতে গুছিয়ে রাখা, যাতে যেকোনো Playbook, যেকোনো পরিবেশ, যেকোনো প্রজেক্ট থেকে তাকে ডাকা যায়। Role শিখলে আপনি আর Playbook লেখেন না — আপনি Role-এর লেগো-টুকরো জোড়া দিয়ে Playbook সাজান। আর সেটাই প্রোডাকশন-মানের অটোমেশনের প্রথম শর্ত।

১২.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Role-এর ডিরেক্টরি কাঠামো — `tasks/`, `handlers/`, `defaults/`, `vars/`, `files/`, `templates/`, `meta/` — প্রতিটি ডিরেক্টরির কাজ ব্যাখ্যা করতে; (২) `ansible-galaxy init` দিয়ে নতুন Role তৈরি করতে; (৩) Role Dependency (একটি Role অন্য Role-এর ওপর নির্ভরশীল হওয়া) ঘোষণা ও ব্যবহার করতে; (৪) Role, Playbook, Collection, Module, Task ও Plugin-এর পার্থক্য সারণিতে দেখাতে; (৫) `defaults` বনাম `vars`-এর Precedence (অগ্রাধিকার) ব্যাখ্যা করে সঠিক জায়গায় Variable রাখতে; (৬) যেকোনো Playbook থেকে Role ডেকে TechNova-র Web, API ও Database স্তর একই Role দিয়ে বারবার তৈরি করতে।

১২.৩ পূর্বশর্ত জ্ঞান

এই অধ্যায়ের জন্য Volume 09-এর আগের অংশের জ্ঞান প্রয়োজন: Playbook ও Task লেখা, YAML-এর মৌলিক গঠন (Indentation, List, Dictionary), Inventory ও Variable-এর ধারণা, Handler ও Template (Jinja2)-এর প্রাথমিক ব্যবহার, এবং `ansible-playbook` চালানোর অভিজ্ঞতা। Linux ডিরেক্টরি কাঠামো ও `ansible-galaxy` কমান্ডের নাম শুনে থাকলে ভালো, না থাকলেও এই অধ্যায়ে প্রতিটি ধাপ হাতে-কলমে দেখানো হবে। Git-এর প্রাথমিক ধারণা (Commit, Branch) থাকলে Role শেয়ার করার অংশটি সহজে বোঝা যাবে।

১২.৪ প্রথম নীতি থেকে ব্যাখ্যা

যেকোনো বড় কাজ গোছানোর প্রথম নীতি একটাই — **একটি কাজ, একটি জায়গা** (Single Responsibility)। রান্নাঘরে যেমন মসলা এক জায়গায়, বাসন আরেক জায়গায় থাকে, তেমনি অটোমেশনেও প্রতিটি দায়িত্বের নিজের জায়গা থাকা উচিত। ধরুন TechNova-র Nginx বসানোর কাজে যা যা লাগে: প্যাকেজ ইনস্টল করা (Task), কনফিগারেশন ফাইল লেখা (Template), সার্ভিস Restart করা (Handler), ডিফল্ট Port নম্বর (Variable), আর অতিরিক্ত ফাইল (যেমন SSL সার্টিফিকেট)। এই পাঁচটি জিনিস যদি পাঁচটি আলাদা Playbook-এ ছড়িয়ে থাকে, তবে একটি পরিবর্তন করতে পাঁচ জায়গায়

যেতে হয় — আর একটি জায়গা ভুলে গেলেই পরিবেশগুলো আলাদা হয়ে যায়, যাকে বলে **Configuration Drift**। Role এই পাঁচটি জিনিসকে একটি ডিরেক্টরিতে বেঁধে দেয়, যার নামই তার কাজ — যেমন `technova_nginx`। এরপর Playbook-এর কাজ শুধু বলা: "এই সার্ভারে nginx Role চালাও, এই Variable-গুলো দিয়ে।" Role-এর ভেতরে কীভাবে কাজটি হয়, তা Playbook-এর জানার দরকার নেই — এটাই **Abstraction**, আর এটাই Role-এর প্রথম নীতি। দ্বিতীয় নীতি হলো **পুনঃব্যবহার**: একবার লেখা Role Development, Staging ও Production — তিন পরিবেশে, দশটি সার্ভারে, একইভাবে চলে; পার্থক্য শুধু Variable-এর মানে। তৃতীয় নীতি হলো **স্পষ্ট চুক্তি**: Role-এর `defaults/main.yml` হলো তার "ব্যবহার-নির্দেশিকা" — কোন Variable বদলানো যায়, ডিফল্ট মান কী — তা পড়েই যে কেউ Role ব্যবহার করতে পারে, ভেতরের Task না পড়েই।

> **CONCEPT:** Role হলো Ansible-এর "ফাংশন" — যেমন প্রোগ্রামিং-এ একই কোড বারবার লিখতে ফাংশন বানানো হয়, তেমনি Ansible-এ একই Task-গুচ্ছ বারবার লিখতে Role বানানো হয়। Playbook হলো "মেইন প্রোগ্রাম", Role হলো তার ভেতরে ডাকা ফাংশন। ফাংশনের যেমন প্যারামিটার (Variable) থাকে, Role-এরও তেমনি `defaults` ও `vars` থাকে। আর ফাংশন যেমন অন্য ফাংশন ডাকতে পারে, Role-ও তেমনি অন্য Role-এর ওপর নির্ভর করতে পারে (Dependency)। এই উপমাটি মনে রাখলে Role-এর প্রতিটি নিয়ম নিজে নিজেই বোঝা যায়।

১২.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Role-এর ডিরেক্টরি কাঠামো

`ansible-galaxy init technova_nginx` চালালে যে কাঠামো তৈরি হয়, তা এইরকম:

```
technova_nginx/
├── defaults/
│   └── main.yml        # বদলানো যায় এমন ডিফল্ট Variable (সবচেয়ে দুর্বল Precedence)
├── vars/
│   └── main.yml        # Role-এর নিজের স্থির Variable (শক্ত Precedence, বাইরে থেকে বদলানো কঠিন)
├── tasks/
│   └── main.yml        # মূল Task-তালিকা; এখান থেকেই Role শুরু হয়
├── handlers/
│   └── main.yml        # Notify করা Handler (যেমন: Restart Nginx)
├── files/
│   └── ...             # যেমন আছে তেমন কপি করার ফাইল (Static File)
├── templates/
│   └── ...             # Jinja2 Template (.j2) — Variable বসিয়ে Render হয়
├── meta/
│   └── main.yml        # Role-এর পরিচয়, লেখক, লাইসেন্স, Dependency, সমর্থিত প্ল্যাটফর্ম
└── README.md           # Role-এর ব্যবহার-নির্দেশিকা
```

প্রতিটি ডিরেক্টরির একটি নির্দিষ্ট চুক্তি আছে। `tasks/main.yml` হলো Role-এর প্রবেশদ্বার (Entry Point) — Role ডাকলেই Ansible এই ফাইলটি পড়ে। এখান থেকে `include_tasks` দিয়ে আরও ফাইলে ভাগ করা যায় (যেমন `install.yml`, `configure.yml`)। `handlers/main.yml`-এর Handler-গুলো শুধু `notify` করলেই চলে, এবং Play শেষে একবারই চলে — যতবারই Notify করা হোক না কেন। `defaults/main.yml`-এ থাকে সেই Variable-গুলো, যা ব্যবহারকারী চাইলে Playbook বা Inventory থেকে বদলে দিতে পারেন — যেমন `nginx_port: 80`। `vars/main.yml`-এ থাকে সেই Variable-গুলো, যা Role-এর ভেতরের যুক্তির অংশ — যেমন প্যাকেজের নামের তালিকা বা অভ্যন্তরীণ পাথ — ব্যবহারকারীর বদলানো উচিত নয়, কারণ বদলালে Role ভেঙে যেতে পারে। `files/`-এর ফাইল `copy` মডিউল দিয়ে যেমন আছে তেমন কপি হয়; `templates/`-এর `.j2` ফাইল `template` মডিউল দিয়ে Variable বসিয়ে Render হয়। `meta/`

`main.yml`-এ থাকে Role-এর পরিচয় (Author, Description, License), সমর্থিত প্ল্যাটফর্ম (যেমন Ubuntu 24.04), এবং `dependencies` — এই Role চালানোর আগে কোন Role-গুলো চালাতে হবে।

Role বনাম Playbook বনাম Collection বনাম Module বনাম Task বনাম Plugin

ধারণা	কী	স্তর	উদাহরণ	পুনঃব্যবহার
Task	একটি মাত্র কাজ — একটি Module-কে ডাকা	সবচেয়ে ছোট একক	`apt: name=nginx`	একই Playbook-এর ভেতরে
Module	একটি কাজ করার কোড (Plugin-এর একটি ধরন)	Task-এর ইঞ্জিন	`ansible.builtin.apt`, `copy`	সব Playbook-এ
Playbook	Task-এর তালিকা — কোন হোস্টে, কী ক্রমে, কী হবে	অর্কেস্ট্রেশন	`site.yml`	সীমিত — কপি করতে হয়
Role	একটি কাজের সম্পূর্ণ প্যাকেজ — Task, Handler, Variable, File, Template	পুনঃব্যবহারের একক	`technova_nginx`	যেকোনো Playbook, যেকোনো প্রজেক্ট
Collection	অনেক Role, Module ও Plugin-এর সংগ্রহ — একটি Namespace-সহ প্যাকেজ	বিতরণের একক	`community.general`, `ansible.posix`	Galaxy থেকে ইনস্টল করে সব প্রজেক্টে
Plugin	Ansible-এর আচরণ বাড়ানোর কোড — Connection, Lookup, Callback, Filter, Inventory	এক্সটেনশন	`aws_ec2` Inventory Plugin, `password` Lookup	সব Playbook-এ, Collection-এর ভেতরে

সহজে মনে রাখার উপায়: Task হলো একটি বাক্য, Module হলো ক্রিয়াপদ, Playbook হলো একটি অনুচ্ছেদ, Role হলো একটি অধ্যায়, Collection হলো একটি বই, আর Plugin হলো বইয়ের পেছনের পরিশিষ্ট (Appendix) — যা বইয়ের ক্ষমতা বাড়ায়। একজন নতুন শিক্ষার্থী প্রায়ই Role আর Collection গুলিয়ে ফেলেন — মনে রাখবেন, Role একটি কাজের প্যাকেজ, Collection অনেক Role ও Module-এর বড় প্যাকেজ। আর Playbook কখনোই Role-এর বিকল্প নয় — Playbook বলে "কোথায়, কখন", Role বলে "কীভাবে"।

defaults বনাম vars — Precedence-এর নিয়ম

Ansible-এ Variable-এর অগ্রাধিকার (Precedence) একটি সিঁড়ির মতো: যত ওপরে, তত শক্তিশালী। Role-এর `defaults` হলো সিঁড়ির সবচেয়ে নিচের ধাপ — প্রায় যেকোনো জায়গা থেকে তাকে হারানো যায়: Playbook-এর `vars`, Inventory Variable, `group_vars`, এমনকি কমান্ড লাইনের `-e` দিয়েও। এটাই উদ্দেশ্য: ডিফল্ট মান এমন হবে যা "বেশিরভাগ ক্ষেত্রে ঠিক", আর ব্যতিক্রম হলে ব্যবহারকারী সহজে বদলে দিতে পারবেন। Role-এর `vars` হলো সিঁড়ির অনেক ওপরের ধাপ — তাকে হারাতে পারে শুধু কমান্ড লাইনের `-e` বা Playbook-এর `vars`-এর কিছু বিশেষ রূপ। তাই `vars/main.yml`-এ কখনোই এমন মান রাখবেন না, যা পরিবেশভেদে বদলাতে পারে (যেমন Port নম্বর, ডোমেইন, পাসওয়ার্ড) — সেগুলো `defaults`-এ যায়। আর `vars`-এ রাখবেন Role-এর অভ্যন্তরীণ ধ্রুবক (Constant) — যেমন প্যাকেজের নাম, কনফিগারেশন ফাইলের পাথ। এই ভুলটি — সবকিছু `vars`-এ লেখা — নতুন Role লেখকদের সবচেয়ে সাধারণ ভুল, এবং এর ফলে Role পুনঃব্যবহারের অযোগ্য হয়ে যায়।

Role Dependency

TechNova-র `technova_app` Role-এর Node.js দরকার, আর Node.js বসানোর কাজটি `technova_nodejs` Role-এই আছে। তখন `technova_app/meta/main.yml`-এ লেখা হয়:

```
# roles/technova_app/meta/main.yml
galaxy_info:
  author: TechNova DevOps Team
  description: Deploy TechNova Node.js API application
  license: MIT
  min_ansible_version: "2.15"
  platforms:
    - name: Ubuntu
      versions:
        - noble   # Ubuntu 24.04 LTS
dependencies:
  - role: technova_nodejs
    vars:
      nodejs_version: "20"
  - role: technova_nginx
    vars:
      nginx_port: 80
```

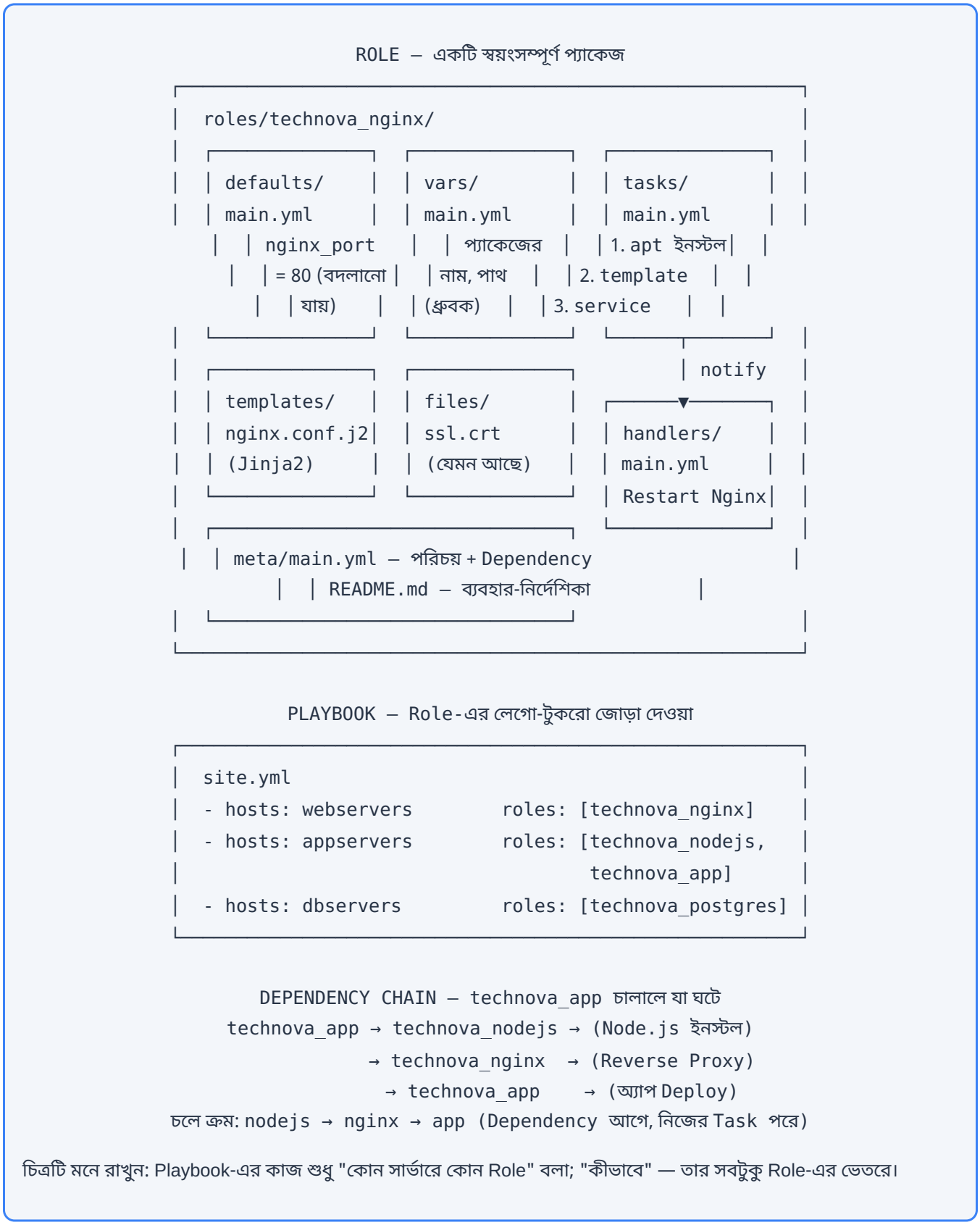
Dependency-র অর্থ: `technova_app` চালানোর আগে Ansible প্রথমে `technova_nodejs` ও `technova_nginx` চালাবে, সেই ক্রমেই যেভাবে তালিকায় লেখা আছে। সতর্কতা: Dependency চক্র (Circular Dependency) — A নির্ভর করে B-এর ওপর, B আবার A-এর ওপর — তৈরি করবেন না; Ansible তাতে আটকে যাবে বা অপ্রত্যাশিতভাবে চলবে। আর Dependency-তে Variable পাস করলে তা শুধু সেই Role-এর জন্য প্রযোজ্য হয়।

১২.৬ অভ্যন্তরীণ কার্যপ্রণালী

আপনি যখন Playbook-এ `roles: [technova_nginx]` লেখেন, Ansible ভেতরে যা করে তা এই ক্রমে ঘটে। প্রথমে Ansible Role-টি খোঁজে — খোঁজার ক্রম: Playbook-এর পাশের `roles/` ডিরেক্টরি, তারপর `ansible.cfg`-এর `roles_path`, তারপর `~/.ansible/roles`, তারপর `/usr/share/ansible/roles`। Role পাওয়া গেলে Ansible তার `meta/main.yml` পড়ে Dependency-গুলো বের করে, এবং Dependency-গুলোকে Play-এর Task-তালিকার সামনে বসায় — অর্থাৎ Dependency প্রথমে চলে। তারপর Ansible Role-এর `defaults/main.yml` ও `vars/main.yml` লোড করে Variable-গুলোকে Precedence অনুযায়ী সাজায়। এরপর `tasks/main.yml`-এর Task-গুলো একে একে চলে — প্রতিটি Task-এর জন্য Ansible সংশ্লিষ্ট Module-এর কোড Control Node-এ তৈরি করে, SSH দিয়ে Managed Node-এ পাঠায়, সেখানে চালায়, এবং ফলাফল (Changed, OK, Failed) ফিরিয়ে আনে। Task-এর `notify` থাকলে Handler-টি একটি তালিকায় জমা হয়, কিন্তু তখনই চলে না — Play-এর সব Task শেষ হলে, Flush করা Handler-গুলো একবার করে চলে। এই "শেষে একবার" নিয়মটিই Handler-এর মূল যুক্তি: কনফিগারেশন দশবার বদলালেও Nginx একবারই Restart হয়। Role-এর ভেতরে `files/` বা `templates/`-এর ফাইল খুঁজতে Ansible প্রথমে Role-এর নিজের ডিরেক্টরি দেখে — তাই Task-এ শুধু ফাইলের নাম লিখলেই চলে, পূর্ণ পাথ লাগে না। এই খোঁজার ক্রমটিই Role-কে স্বয়ংসম্পূর্ণ (Self-contained) করে।

> **TIP:** Role-এর নাম সবসময় ছোট হাতের অক্ষর, সংখ্যা ও আন্ডারস্কোর দিয়ে লিখুন — যেমন `technova_nginx`, `technova_nodejs`। হাইফেন বা বড় হাতের অক্ষর Ansible Galaxy-তে সমস্যা করতে পারে। আর Role-এর নামে কোম্পানির নামের উপসর্গ (Prefix) রাখলে নামের সংঘাত (Collision) এড়ানো যায় — Galaxy-তে হাজার হাজার `nginx` Role আছে, কিন্তু `technova_nginx` শুধু আপনার।

১২.৭ আর্কিটেকচার চিত্র



চিত্রটি মনে রাখুন: Playbook-এর কাজ শুধু "কোন সার্ভারে কোন Role" বলা; "কীভাবে" — তার সবটুকু Role-এর ভেতরে।

১২.৮ ব্যবহারিক উদাহরণ

TechNova-র Web সার্ভারে Nginx বসানোর জন্য একটি ছোট কিন্তু সম্পূর্ণ Role-এর মূল অংশগুলো দেখুন। প্রথমে Task-ফাইল — এখানে তিনটি Task: প্যাকেজ ইনস্টল, Template থেকে কনফিগারেশন লেখা, সার্ভিস চালু করা:

```
# roles/technova_nginx/tasks/main.yml
---
- name: Nginx প্যাকেজ ইনস্টল করুন
  ansible.builtin.apt:
    name: nginx
    state: present
    update_cache: true
  become: true

- name: Nginx কনফিগারেশন Template থেকে বসান
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/sites-available/technova.conf
    owner: root
    group: root
    mode: "0644"
  become: true
  notify: Restart Nginx

- name: ডিফল্ট সাইট বন্ধ করে TechNova সাইট চালু করুন
  ansible.builtin.file:
    src: /etc/nginx/sites-available/technova.conf
    dest: /etc/nginx/sites-enabled/technova.conf
    state: link
  become: true
  notify: Restart Nginx

- name: Nginx সার্ভিস চালু ও Boot-এ চালু রাখুন
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: true
  become: true
```

এই চারটি Task-এর প্রতিটিই **Idempotent** — অর্থাৎ দ্বিতীয়বার চালালে কোনো পরিবর্তন হয় না, যদি সবকিছু আগেই ঠিক থাকে। `apt` মডিউল দেখে প্যাকেজটি আগেই ইনস্টল আছে কি না, `template` মডিউল দেখে ফাইলের বিষয়বস্তু বদলেছে কি না। Role লেখার সময় এই Idempotency-ই সবচেয়ে বড় পরীক্ষা — একটি Role দুবার চালালে দ্বিতীয়বার সব Task `ok` দেখাবে, `changed` নয়। আর Handler-টি দেখুন:

```
# roles/technova_nginx/handlers/main.yml
---
- name: Restart Nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
  become: true

# roles/technova_nginx/defaults/main.yml
---
nginx_port: 80
```

```
nginx_server_name: app.technova.example
nginx_worker_processes: auto

# roles/technova_nginx/templates/nginx.conf.j2
server {
    listen {{ nginx_port }};
    server_name {{ nginx_server_name }};
    location / {
        proxy_pass http://127.0.0.1:4000;
        proxy_set_header Host $host;
    }
}
```

লক্ষ্য করুন, Template-এ Variable-গুলো Jinja2 সিনট্যাক্সে (`{{ ... }}`) লেখা — Role ব্যবহারকারী শুধু `nginx_server_name` বদলে দিলেই কনফিগারেশন বদলে যায়, Template ছোঁয়ার দরকার নেই।

১২.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

এবার পুরো Role-টি একসঙ্গে — TechNova-র Node.js অ্যাপ Deploy করার Role `technova_app`, যেখানে Dependency, Variable, Task, Handler, Template সব আছে:

```
# ── প্রজেক্ট কাঠামো ──
# /home/nahid/technova-ansible/
# ├── ansible.cfg
# ├── inventory/hosts.yml
# ├── site.yml
# └── roles/
#     ├── technova_nodejs/
#     ├── technova_nginx/
#     └── technova_app/

# ── ansible.cfg ──
[defaults]
inventory = inventory/hosts.yml
roles_path = roles
host_key_checking = True
retry_files_enabled = False

# ── inventory/hosts.yml ──
all:
  children:
    webservers:
      hosts:
        web01.technova.example:
          ansible_host: 203.0.113.10
    appservers:
      hosts:
        app01.technova.example:
          ansible_host: 203.0.113.11
        app02.technova.example:
          ansible_host: 203.0.113.12

# ── roles/technova_app/defaults/main.yml ──
---
app_name: technova-api
app_user: technova
```

```
app_dir: /opt/technova-api
app_port: 4000
app_env: production
app_repo: https://github.com/technova/api.git
app_version: main

# ── roles/technova_app/tasks/main.yml ──
---
- name: অ্যাপ ব্যবহারকারী তৈরি করুন
  ansible.builtin.user:
    name: "{{ app_user }}"
    system: true
    create_home: false
  become: true

- name: অ্যাপ ডিরেক্টরি তৈরি করুন
  ansible.builtin.file:
    path: "{{ app_dir }}"
    state: directory
    owner: "{{ app_user }}"
    group: "{{ app_user }}"
    mode: "0755"
  become: true

- name: Git থেকে অ্যাপের কোড টানুন
  ansible.builtin.git:
    repo: "{{ app_repo }}"
    dest: "{{ app_dir }}"
    version: "{{ app_version }}"
  become: true
  become_user: "{{ app_user }}"
  notify: Restart Technova API

- name: npm নির্ভরতা ইনস্টল করুন
  community.general.npm:
    path: "{{ app_dir }}"
    state: present
  become: true
  become_user: "{{ app_user }}"

- name: systemd সার্ভিস ফাইল বসান
  ansible.builtin.template:
    src: technova-api.service.j2
    dest: /etc/systemd/system/technova-api.service
    mode: "0644"
  become: true
  notify:
    - Reload systemd
    - Restart Technova API

# ── roles/technova_app/handlers/main.yml ──
---
- name: Reload systemd
  ansible.builtin.systemd:
    daemon_reload: true
  become: true

- name: Restart Technova API
  ansible.builtin.systemd:
```

```
    name: technova-api
    state: restarted
    enabled: true
  become: true

# ── roles/technova_app/templates/technova-api.service.j2 ──
[Unit]
Description=TechNova API
After=network.target

[Service]
User={{ app_user }}
WorkingDirectory={{ app_dir }}
ExecStart=/usr/bin/node {{ app_dir }}/server.js
Environment=NODE_ENV={{ app_env }}
Environment=PORT={{ app_port }}
Restart=always

[Install]
WantedBy=multi-user.target

# ── site.yml — সব Role একসঙ্গে ডাকা ──
---
- name: Web স্তর তৈরি করুন
  hosts: webservers
  roles:
    - technova_nginx

- name: অ্যাপ স্তর তৈরি করুন
  hosts: appservers
  roles:
    - role: technova_app
      vars:
        app_version: v1.4.2
```

এই ফাইলগুলো একসঙ্গে একটি সম্পূর্ণ Deploy-এর নকশা: `site.yml` বলে কোথায় কী, Role-গুলো বলে কীভাবে। `app_version` Playbook থেকে পাস করা হয়েছে — এটাই Role-কে নমনীয় করে: একই Role দিয়ে `main` ব্রাঞ্চও Deploy করা যায়, `v1.4.2` ট্যাগও।

১২.১০ ধাপে ধাপে বাস্তবায়ন

1. **প্রজেক্ট ডিরেক্টরি তৈরি করুন:** Control Node-এ (ব্যবহারকারী `nahid`) `mkdir -p ~/technova-ansible/roles` চালিয়ে কাজের জায়গা বানান।
2. **Role তৈরি করুন:** `cd ~/technova-ansible/roles && ansible-galaxy init technova_nginx` চালান। Ansible Role-এর পুরো কাঠামো (defaults, vars, tasks, handlers, files, templates, meta, README) বানিয়ে দেবে।
3. **কাঠামো যাচাই করুন:** `find technova_nginx -type f | sort` দিয়ে দেখুন কোন ফাইল কোথায় তৈরি হলো। অপ্রয়োজনীয় ফাইল (যেমন `tests/` ডিরেক্টরি) পরে মুছে ফেলতে পারেন।
4. **Variable লিখুন:** `defaults/main.yml`-এ `nginx_port` ও `nginx_server_name` লিখুন — যা ব্যবহারকারী বদলাতে পারবেন। `vars/main.yml`-এ শুধু ধ্রুবক রাখুন।

5. **Task লিখুন:** `tasks/main.yml`-এ ইনস্টল, কনফিগার ও সার্ভিস-চালুর Task লিখুন — প্রতিটি Task-এ `name` দিন, যাতে আউটপুট পড়া যায়।
6. **Template ও Handler লিখুন:** `templates/nginx.conf.j2`-এ Jinja2 Variable ব্যবহার করুন; `handlers/main.yml`-এ Restart Handler লিখুন ও Task থেকে `notify` করুন।
7. **Syntax যাচাই করুন:** `ansible-playbook site.yml --syntax-check` চালান — YAML ভুল থাকলে এখানেই ধরা পড়বে।
8. **Dry Run করুন:** `ansible-playbook site.yml --check --diff` চালান — কোনো পরিবর্তন না করে কী বদলাবে তা দেখুন।
9. **আসল Run করুন:** `ansible-playbook site.yml` চালান। প্রথমবার `changed` দেখাবে।
10. **Idempotency পরীক্ষা করুন:** একই কমান্ড আবার চালান — এবার সব Task `ok` দেখাবে, `changed=0` হবে। না হলে কোনো Task Idempotent নয় — সেটি ঠিক করুন।
11. **Galaxy-তে প্রকাশের প্রস্তুতি:** `meta/main.yml`-এ লেখক, লাইসেন্স, বর্ণনা ও প্ল্যাটফর্ম পূরণ করুন; `README.md`-এ ব্যবহারের উদাহরণ লিখুন।

১২.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

```
$ ansible-playbook site.yml
# চিত্রণমূলক আউটপুট:
PLAY [Web স্তর তৈরি করুন] ***********************************************

TASK [Gathering Facts] ***************************************************
ok: [web01.technova.example]

TASK [technova_nginx : Nginx প্যাকেজ ইনস্টল করুন] **********************
changed: [web01.technova.example]

TASK [technova_nginx : Nginx কনফিগারেশন Template থেকে বসান] **********
changed: [web01.technova.example]

TASK [technova_nginx : Nginx সার্ভিস চালু ও Boot-এ চালু রাখুন] *******
ok: [web01.technova.example]

RUNNING HANDLER [technova_nginx : Restart Nginx] *********************
changed: [web01.technova.example]

PLAY RECAP ***************************************************************
web01.technova.example : ok=5  changed=3  unreachable=0  failed=0
```

আউটপুট পড়ার নিয়ম: Task-এর নামের আগে Role-এর নাম দেখায় — `technova_nginx : ...` — এতে বোঝা যায় কোন Task কোন Role থেকে এসেছে। `changed` মানে সার্ভারে আসলেই কিছু বদলেছে, `ok` মানে আগেই ঠিক ছিল। Handler আলাদা অংশে চলে — `RUNNING HANDLER` — এবং শুধু তখনই চলে যখন কোনো Task তাকে Notify করেছে। `PLAY RECAP`-এ `failed=0` ও `unreachable=0` মানেই সফল Run। দ্বিতীয়বার চালালে একই আউটপুটে `changed=0` হবে — সেটাই Idempotency-এর প্রমাণ। যদি দ্বিতীয়বারও `changed` দেখায়, তবে কোনো Task প্রতিবার কিছু বদলে দিচ্ছে — যেমন `shell` দিয়ে তারিখ লেখা বা `command` দিয়ে সবসময় ফাইল তৈরি করা — সেটি ঠিক করতে হবে।

১২.১২ সাধারণ ভুল

- **সবকিছু `vars/`-এ লেখা:** ব্যবহারকারী বদলাতে পারবেন না — Port, ডোমেইন, সংস্করণ সব `defaults/`-এ রাখুন।
- **Role-এর নামে বড় হাতের অক্ষর বা হাইফেন:** Galaxy ও কিছু টুলে সমস্যা হয় — ছোট হাতের অক্ষর ও আন্ডারস্কোর ব্যবহার করুন।
- **Task-এ `name` না দেওয়া:** আউটপুটে শুধু মডিউলের নাম দেখাবে — ভুল খুঁজে পাওয়া কঠিন হয়।
- **Role-এর ভেতরে `hosts` লেখা:** Role কখনো `hosts` ঠিক করে না — সেটি Playbook-এর কাজ। Role শুধু "কীভাবে" বলে।
- **Dependency চক্র তৈরি করা:** A নির্ভর করে B-এর ওপর, B আবার A-এর ওপর — Ansible আটকে যাবে বা ভুল ক্রমে চলবে।
- **`shell` / `command` দিয়ে সবকিছু করা:** এগুলো Idempotent নয় — `apt`, `template`, `service`-এর মতো নির্দিষ্ট Module ব্যবহার করুন।
- **Role-এ Secret লেখা:** পাসওয়ার্ড বা API Key কখনো Role-এর `defaults` বা `vars`-এ লিখবেন না — তার জন্য Ansible Vault (অধ্যায় ১৪) ব্যবহার করুন।
- **`meta/main.yml` খালি রাখা:** Dependency ও প্ল্যাটফর্ম না লিখলে অন্যরা Role ব্যবহার করতে গিয়ে ভুল পরিবেশে চালাবে।

১২.১৩ সমস্যা সমাধান (Troubleshooting)

> **REAL WORLD — TechNova:** একবার TechNova-র Staging-এ `technova_app` Role চালাতে গিয়ে Ansible বলল `ERROR! the role 'technova_app' was not found`। কারণ ছিল `ansible.cfg`-এ `roles_path` ভুল ডিরেক্টরির দিকে নির্দেশ করছিল — Role ছিল `~/technova-ansible/roles/`-এ, কিন্তু `roles_path` বলছিল `/etc/ansible/roles`। `ansible-config dump | grep -i roles_path` দিয়ে আসল পাথ দেখে ঠিক করা হয়েছিল। শিক্ষা: Role খুঁজে না পেলে প্রথমে পাথ দেখুন, Role-এর নাম নয়।

1. **লক্ষণ:** `role not found` — **পরীক্ষা:** `ansible-config dump | grep roles_path` ও `ls roles/` — Role ডিরেক্টরি ঠিক পাথে আছে কি না। **সমাধান:** `ansible.cfg`-এ `roles_path` ঠিক করুন বা Playbook-এর পাশে `roles/` ডিরেক্টরি রাখুন।
2. **লক্ষণ:** Variable-এর মান প্রত্যাশিত নয় — **পরীক্ষা:** `ansible-playbook site.yml -e "nginx_port=8080" --check` দিয়ে দেখুন কোন মান জিতছে; Precedence-এর সিঁড়ি মনে রাখুন — `defaults` সবচেয়ে দুর্বল। **সমাধান:** মানটি `vars/main.yml`-এ আটকে আছে কি না দেখুন — থাকলে `defaults`-এ সরান।
3. **লক্ষণ:** Handler চলছে না — **পরীক্ষা:** Handler-এর নাম ও `notify`-এর নাম অক্ষরে অক্ষরে মিলছে কি না; Role-এর Handler-এর নামে Role-এর উপসর্গ লাগে না, শুধু Handler-এর নামই লিখতে হয়। **সমাধান:** নাম মিলিয়ে নিন; প্রয়োজনে `--force-handlers` দিয়ে পরীক্ষা করুন।
4. **লক্ষণ:** দ্বিতীয়বার চালালেও `changed` দেখায় — **পরীক্ষা:** `--diff` দিয়ে দেখুন কোন Task বদলাচ্ছে; সাধারণত `template`-এ সময়-নির্ভর মান বা `shell` Task এর কারণ। **সমাধান:** Idempotent Module ব্যবহার করুন।
5. **লক্ষণ:** Dependency দুবার চলছে — **পরীক্ষা:** দুটি Role-ই একই Dependency ঘোষণা করেছে কি না। **সমাধান:** Ansible একই Dependency একবারই চালায়, যদি `allow_duplicates: false` (ডিফল্ট) থাকে; `meta/main.yml`-এ `allow_duplicates: true` লেখা থাকলে মুছে দিন।

১২.১৪ নিরাপত্তা বিবেচনা

> **WARNING:** Galaxy থেকে Role ডাউনলোড করা মানে অন্যের লেখা কোড আপনার সার্ভারে `root` হয়ে চালানো — কারণ বেশিরভাগ Role-এ `become: true` থাকে। কোনো Role ইনস্টল করার আগে তার `tasks/` পড়ুন: `shell` বা `command` দিয়ে সন্দেহজনক কিছু (যেমন বাইরের URL থেকে স্ক্রিপ্ট ডাউনলোড করে চালানো — `curl ... | bash`) আছে কি না দেখুন। অপরিচিত লেখকের Role প্রোডাকশনে চালাবেন না; প্রয়োজনে Role-টি নিজের Repository-এ Fork করে পর্যালোচনা করে ব্যবহার করুন।

Role-এ Secret রাখা সবচেয়ে বিপজ্জনক অভ্যাস — Git-এ Push করা Role-এর `vars/main.yml`-এ পাসওয়ার্ড লিখলে তা ইতিহাসে চিরদিন থেকে যায়, এমনকি পরে মুছে ফেললেও। Secret-এর জন্য Ansible Vault (অধ্যায় ১৪) ব্যবহার করুন, আর Role-এর `defaults`-এ শুধু প্লেসহোল্ডার রাখুন — যেমন `db_password: "CHANGE-ME-DB-PASSWORD"`। আরও একটি বিষয়: Role-এর `files/`-এ Private Key বা সার্টিফিকেট রাখলে তা Git-এ যাওয়ার আগেই `.gitignore`-এ রাখুন, আর প্রোডাকশনে সার্টিফিকেট Vault বা সার্টিফিকেট ম্যানেজমেন্ট সিস্টেম থেকে আনুন। Role-এর File Permission-ও পরীক্ষা করুন — `template` ও `copy` মডিউলে সবসময় `mode` দিন, যাতে গোপন ফাইল ভুল করে সবার পড়ার মতো না হয়।

১২.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

প্রোডাকশনে Role ব্যবহারের মানদণ্ডগুলো এইরকম: **একটি Role, একটি দায়িত্ব** — Nginx Role শুধু Nginx বসাবে, ডেটাবেসও বসাবে না; **Version Pin করা** — `requirements.yml`-এ Galaxy Role-এর সংস্করণ নির্দিষ্ট করে দিন (`geerlingguy.nginx, version: 3.1.0`), যাতে হঠাৎ আপডেটে Deploy না ভাঙে; **Role-কে Git Repository-তে রাখা** — প্রতিটি Role-এর নিজের Repository বা একটি Monorepo-তে `roles/` ডিরেক্টরি, আর পরিবর্তন Code Review-এর মাধ্যমে যাবে; **Testing** — `molecule` দিয়ে Role-কে Docker Container-এ পরীক্ষা করুন, যাতে প্রোডাকশনে যাওয়ার আগেই ভুল ধরা পড়ে; **README লেখা** — প্রতিটি Role-এর README-তে Variable-এর তালিকা, ডিফল্ট মান ও ব্যবহারের উদাহরণ থাকবে, যাতে নতুন সদস্য না পড়ে Task বুঝতে পারেন; **Idempotency পরীক্ষা** — CI Pipeline-এ Role দুবার চালিয়ে দ্বিতীয়বার `changed=0` নিশ্চিত করুন; **পরিবেশ-ভিত্তিক Variable আলাদা রাখা** — `group_vars/production.yml` ও `group_vars/staging.yml`-এ পরিবেশের মান রাখুন, Role-এর ভেতরে নয়। TechNova এই নিয়মগুলো মেনে তাদের Role-গুলোকে এমনভাবে সাজিয়েছে যে নতুন সার্ভার তৈরি করতে এখন শুধু Inventory-তে একটি লাইন যোগ করলেই হয় — বাকিটা Role-ই করে দেয়।

১২.১৬ অধ্যায়ের সারাংশ

Role হলো Ansible-এর পুনঃব্যবহারযোগ্য একক — Task, Handler, Variable, File ও Template-এর একটি স্বয়ংসম্পূর্ণ প্যাকেজ, যার কাঠামো `ansible-galaxy init` দিয়ে তৈরি হয়। `defaults/main.yml`-এ থাকে বদলানো যায় এমন ডিফল্ট মান, `vars/main.yml`-এ থাকে Role-এর অভ্যন্তরীণ ধ্রুবক; `tasks/main.yml` হলো প্রবেশদ্বার, `handlers/main.yml`-এ থাকে Notify-করা Handler, `meta/main.yml`-এ থাকে পরিচয় ও Dependency। Role, Playbook, Collection, Module, Task ও Plugin — প্রতিটি আলাদা স্তরের ধারণা: Task একটি কাজ, Module তার ইঞ্জিন, Playbook বলে কোথায়-কখন, Role বলে কীভাবে, Collection অনেক Role-এর বিতরণ-প্যাকেজ, Plugin Ansible-এর ক্ষমতা বাড়ায়। Dependency দিয়ে একটি Role অন্য Role-কে ডাকতে পারে, তবে চক্র তৈরি করা যাবে না। ভালো Role Idempotent হয়, Secret রাখে না, README-সহ আসে, এবং Version Pin করে ব্যবহৃত হয়।

১২.১৭ জ্ঞান যাচাই — প্রশ্ন

1. Role-এর `defaults/main.yml` ও `vars/main.yml`-এর পার্থক্য কী? কোনটিতে Port নম্বর রাখবেন এবং কেন?
2. Task, Module, Playbook, Role, Collection ও Plugin — এই ছয়টির মধ্যে কোনটি পুনঃব্যবহারের একক, আর কোনটি বিতরণের একক? ব্যাখ্যা করুন।
3. `technova_app` Role `technova_nodejs`-এর ওপর নির্ভরশীল — এই Dependency কোথায়, কীভাবে ঘোষণা করবেন? চালানোর ক্রম কী হবে?
4. একটি Role দুবার চালালে দ্বিতীয়বার সব Task `ok` দেখানো উচিত কেন? `changed` দেখালে কী বুঝবেন?
5. Galaxy থেকে ডাউনলোড করা Role প্রোডাকশনে চালানোর আগে কী কী পরীক্ষা করবেন? Secret নিয়ে কী সতর্কতা নেবেন?

১২.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. **নিজের Role তৈরি করুন:** `ansible-galaxy init technova_postgres` দিয়ে একটি Role বানান, যাতে PostgreSQL ইনস্টল, কনফিগারেশন Template, ও সার্ভিস-চালুর Task থাকে। `defaults/main.yml`-এ `postgres_port` ও `postgres_db` রাখুন।
2. **Dependency যোগ করুন:** আপনার `technova_app` Role-এর `meta/main.yml`-এ `technova_postgres`-কে Dependency হিসেবে যোগ করুন, এবং `site.yml` চালিয়ে ক্রমটি যাচাই করুন।
3. **Idempotency পরীক্ষা:** আপনার Role দুবার চালান; দ্বিতীয়বারের `PLAY RECAP`-এ `changed=0` নিশ্চিত করুন। না হলে কোন Task-টি দায়ী তা `--diff` দিয়ে খুঁজে বের করুন।
4. **README লিখুন:** আপনার Role-এর README-তে Variable-এর তালিকা, ডিফল্ট মান ও একটি ব্যবহারের উদাহরণ লিখুন — যাতে একজন নতুন সদস্য Task না পড়েই Role ব্যবহার করতে পারেন।

অধ্যায় 13 • PART III — Roles, Collections, Vault & Cloud

Chapter 13 — Collections, Plugins, and Ansible Galaxy

১৩.১ ভূমিকা

অধ্যায় ১২-এ আপনি শিখেছেন Role কীভাবে একটি কাজকে প্যাকেজ করে। কিন্তু TechNova যখন AWS-এ সার্ভার তৈরি করতে চাইল, Azure-এ DNS রেকর্ড বদলাতে চাইল, আর PostgreSQL-এর ব্যাকআপ নিতে চাইল — তখন দেখা গেল Role একা যথেষ্ট নয়। এই কাজগুলোর জন্য দরকার এমন Module — যেমন `amazon.aws.ec2_instance` বা `community.postgresql.postgresql_db` — যা Ansible-এর মূল (Core) প্যাকেজে নেই। আগে, Ansible-এর পুরনো সংস্করণে, সব Module একটিমাত্র বড় প্যাকেজে থাকত — নতুন Module যোগ হলে পুরো Ansible আপডেট করতে হতো। Ansible 2.10 থেকে এই সমস্যার সমাধান হলো **Collection** — Module, Role ও Plugin-এর নাম-দেওয়া (Namespaced) প্যাকেজ, যা আলাদাভাবে ইনস্টল, আপডেট ও শেয়ার করা যায়। আর এই Collection-গুলো খুঁজে পাওয়া ও বিতরণের জায়গার নাম **Ansible Galaxy** — কমিউনিটির লেখা হাজার হাজার Role ও Collection-এর ভান্ডার। এই অধ্যায়ে আপনি শিখবেন Collection কীভাবে কাজ করে, Fully Qualified Collection Name (FQCN) কেন লিখতে হয়, Plugin কত ধরনের হয়, এবং Galaxy থেকে নিরাপদে কীভাবে কোড আনতে হয়।

১৩.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Collection-এর ধারণা ও নামকরণ — `namespace.collection` — ব্যাখ্যা করতে; (২) `ansible-galaxy collection install` দিয়ে Collection ইনস্টল, আপডেট ও তালিকা দেখতে; (৩) FQCN লেখার নিয়ম ও `ansible.builtin` বনাম `community.general`-এর পার্থক্য বলতে; (৪) Plugin-এর পাঁচটি প্রধান ধরন — Connection, Lookup, Callback, Inventory ও Filter — প্রতিটির কাজ ও উদাহরণ দিতে; (৫) `requirements.yml` দিয়ে প্রজেক্টের সব Collection Version Pin করে পুনরুৎপাদনযোগ্য (Reproducible) পরিবেশ বানাতে; (৬) Galaxy থেকে Collection আনার আগে তার নিরাপত্তা ও মান যাচাই করতে।

১৩.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১২-এর Role-এর কাঠামো, Module ডাকার অভিজ্ঞতা (`ansible.builtin.apt`, `copy`, `template`), YAML-এ Task লেখা, এবং কমান্ড লাইনে `ansible-galaxy` চালানোর অভিজ্ঞতা প্রয়োজন। Python প্যাকেজ ম্যানেজার `pip`-এর ধারণা থাকলে Collection-কে বুঝতে সুবিধা হবে — Collection অনেকটা Python-এর Package-এর মতো, আর Galaxy অনেকটা PyPI-এর মতো। ক্লাউডের নাম (AWS, Azure, GCP) শুনে থাকলেই চলবে — এই অধ্যায়ে ক্লাউড Module-এর গভীরে যাওয়া হবে না, তা অধ্যায় ১৬-এর বিষয়।

১৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি বড় সফটওয়্যারকে ভাগ করার প্রথম নীতি হলো **নাম দিয়ে আলাদা করা** (Namespacing)। ধরুন দুটি ভিন্ন দল দুটি ভিন্ন `user` Module লেখে — একটি Linux ব্যবহারকারী বানায়, আরেকটি ডেটাবেস ব্যবহারকারী বানায়। নাম এক হলে সংঘাত (Collision) অনিবার্য। Collection এই সংঘাত মেটায় নামের আগে মালিকের নাম জুড়ে দিয়ে: `ansible.builtin.user` (Ansible-এর নিজের), `community.general.user` (কমিউনিটির), `amazon.aws.ec2_instance` (AWS-এর)। পুরো নামটিকেই বলে **Fully Qualified Collection Name (FQCN)** — তিনটি অংশ: Namespace (কার লেখা), Collection (কোন প্যাকেজ), Module (কোন কাজ)। দ্বিতীয় নীতি হলো **আলাদা**

জীবনচক্র (Independent Lifecycle): প্রতিটি Collection-এর নিজের সংস্করণ (Version) থাকে — `community.general` আপডেট হলে `ansible-core` আপডেট করতে হয় না, আর `ansible-core` আপডেট হলে Collection ভাঙে না। তৃতীয় নীতি হলো **এক্সটেনশনের জায়গা**: Ansible-এর মূল অংশ ছোট রাখা হয় — শুধু সবচেয়ে সাধারণ Module-গুলো (`ansible.builtin`) — আর বাকি সবকিছু Collection-এ থাকে। Plugin-ও একই নীতিতে কাজ করে: Ansible-এর মূল আচরণ (যেমন কীভাবে SSH-এ যুক্ত হবে, কোথা থেকে Variable আসবে) বদলাতে চাইলে Plugin লিখতে হয় — Plugin হলো Ansible-এর "প্লাগ-ইন করা" ক্ষমতা, যেমন ব্রাউজারে Extension লাগানো।

> **CONCEPT:** FQCN লেখা মানে ঠিকানা পুরো লেখা — শুধু "রহিম" বললে কোন রহিম বোঝা যায় না, "ঢাকার মিরপুরের রহিম" বললে বোঝা যায়। তেমনি শুধু `apt` লিখলে Ansible অনুমান করে নেয় আপনি `ansible.builtin.apt` বোঝাচ্ছেন, কিন্তু `community.general.npm`-এর মতো নামে শুধু `npm` লিখলে ভুল Module ডাকা হতে পারে। প্রোডাকশন Playbook-এ সবসময় FQCN লিখুন — এটি দীর্ঘ, কিন্তু দ্ব্যর্থহীন (Unambiguous)।

১৩.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Collection-এর কাঠামো

একটি Collection ডিস্কে দেখতে এইরকম:

```
~/.ansible/collections/ansible_collections/
├── community/
│   └── general/                # Collection: community.general
│       ├── META/
│       │   └── runtime.yml     # কোন Ansible সংস্করণ দরকার, Redirect নিয়ম
│       ├── plugins/
│       │   ├── modules/        # Module-গুলো (npm, apk, pacman...)
│       │   ├── lookup/         # Lookup Plugin
│       │   ├── filter/         # Filter Plugin
│       │   ├── callback/       # Callback Plugin
│       │   └── inventory/      # Inventory Plugin
│       ├── roles/              # Collection-এর ভেতরের Role-গুলো
│       ├── galaxy.yml          # Collection-এর পরিচয় ও সংস্করণ
│       └── README.md
```

`galaxy.yml`-এ থাকে Collection-এর নাম, সংস্করণ, লেখক, লাইসেন্স ও নির্ভরতা (Dependency) — যেমন `community.general` সংস্করণ 9.x-এর জন্য `ansible-core >= 2.15` দরকার। `META/runtime.yml`-এ থাকে Redirect নিয়ম — পুরনো নামের Module নতুন নামে সরিয়ে নেওয়ার ঘোষণা — যাতে পুরনো Playbook হঠাৎ না ভাঙে। Collection ইনস্টল হয় `~/.ansible/collections`-এ (ব্যবহারকারী-স্তরে) অথবা `/usr/share/ansible/collections`-এ (সিস্টেম-স্তরে); পাথটি `ansible.cfg`-এর `collections_path` দিয়ে বদলানো যায়।

builtin বনাম community বনাম vendor Collection

Collection	কার লেখা	কী থাকে	উদাহরণ
`ansible.builtin`	Ansible Core দল	সবচেয়ে সাধারণ Module ও Plugin — Ansible Core-এর সঙ্গে আসে, আলাদা ইনস্টল লাগে না	`apt`, `copy`, `template`, `service`, `user`, `debug`

`ansible.posix`	Ansible দল	POSIX সিস্টেমের বিশেষ কাজ — Firewall, Mount, ACL	`firewalld`, `mount`, `acl`
`community.general`	কমিউনিটি	সবচেয়ে বড় সাধারণ Collection — শত শত Module	`npm`, `composer`, `ufw`, `timezone`
`community.postgresql`	কমিউনিটি	PostgreSQL-এর সব কাজ	`postgresql_db`, `postgresql_user`
`amazon.aws`	AWS দল	AWS-এর সব সার্ভিস	`ec2_instance`, `s3_bucket`
`azure.azcollection`	Microsoft দল	Azure-এর সব সার্ভিস	`azure_rm_virtualmachine`
`google.cloud`	Google দল	GCP-এর সার্ভিস	`gcp_compute_instance`
`kubernetes.core`	কমিউনিটি	Kubernetes অবজেক্ট পরিচালনা	`k8s`, `helm`

নিয়মটি এইরকম: সাধারণ Linux কাজে `ansible.builtin`, বিশেষ কোনো সফটওয়্যারের কাজে তার নিজের Collection, আর ক্লাউডের কাজে সংশ্লিষ্ট Vendor-এর Collection। ভুল Collection বেছে নিলে হয় Module পাওয়া যাবে না, নয়তো পুরনো, অরক্ষিত Module ব্যবহার করা হবে।

Plugin-এর ধরন

Plugin হলো Ansible-এর আচরণ বাড়ানোর কোড। পাঁচটি প্রধান ধরন জানলেই চলবে:

Plugin-এর ধরন	কী করে	উদাহরণ
Connection Plugin	Managed Node-এ কীভাবে যুক্ত হবে তা ঠিক করে	`ssh` (ডিফল্ট), `local`, `docker`, `kubectl`
Lookup Plugin	বাইরের উৎস থেকে Variable-এর মান আনে — Jinja2-এ `lookup()` দিয়ে ডাকা হয়	`file`, `env`, `password`, `url`, `ansible.builtin.vault`
Callback Plugin	Playbook চলার ঘটনা (শুরু, Task-এর ফলাফল, শেষ) ধরে আউটপুট বদলায় বা কোথাও পাঠায়	`default`, `yaml`, `profile_tasks`, `slack`
Inventory Plugin	বাইরের উৎস (ক্লাউড, CMDB) থেকে হোস্টের তালিকা তৈরি করে	`aws_ec2`, `azure_rm`, `constructed`, `yaml`
Filter Plugin	Jinja2 Template-এ মান বদলায় — `|` চিহ্নের পরে ব্যবহার হয়	`to_json`, `to_yaml`, `b64encode`, `regex_replace`

আরও ধরন আছে — Action Plugin, Test Plugin, Vars Plugin, Strategy Plugin — কিন্তু উপরের পাঁচটিই প্রতিদিনের কাজে লাগে। মনে রাখবেন: Module একটি Task চালায়, Plugin Ansible-এর আচরণ বদলায় — Module Playbook-এ ডাকা হয়, Plugin কনফিগারেশন বা Template-এ ব্যবহৃত হয়।

Ansible Galaxy — ভান্ডার ও তার ব্যবহার

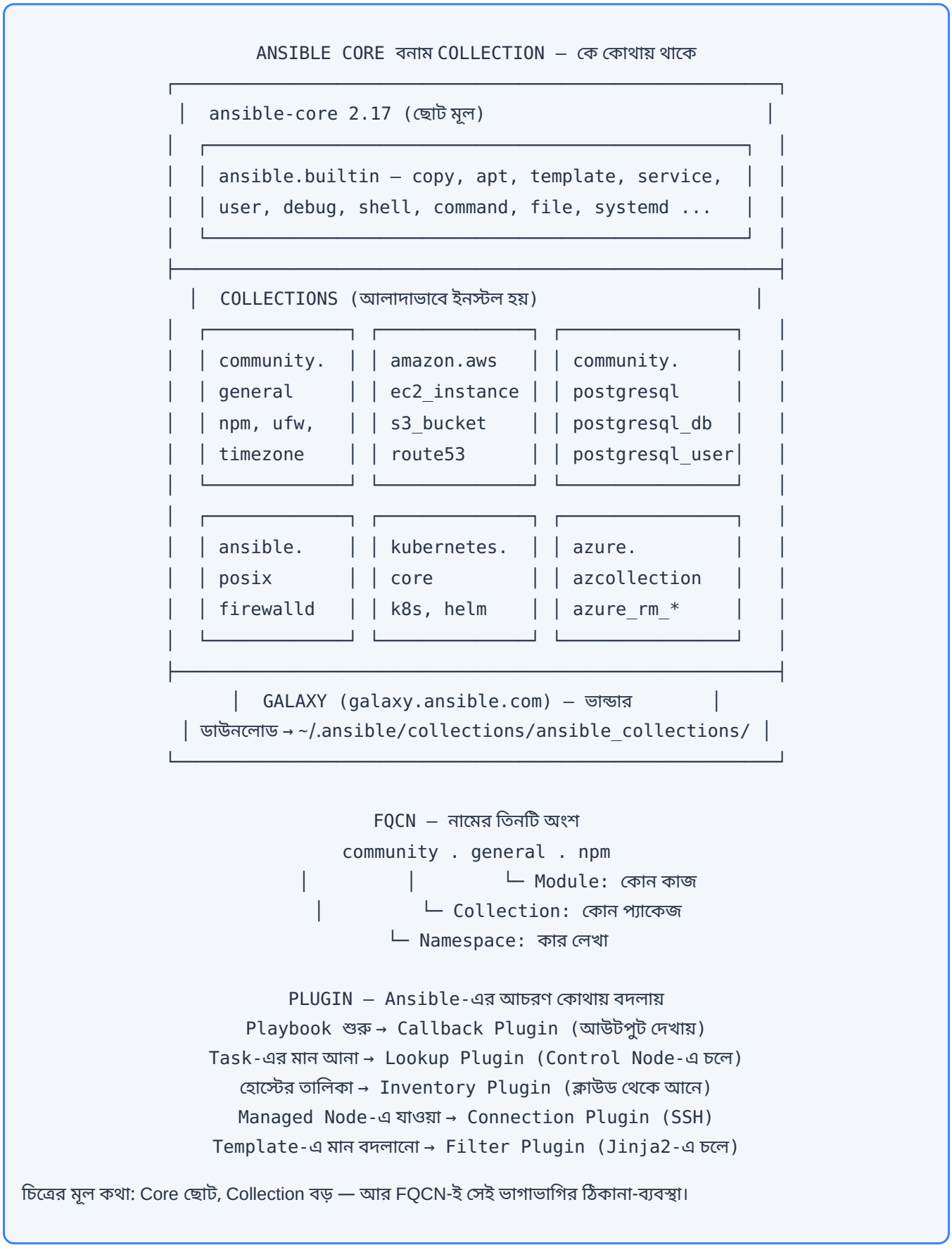
Galaxy (galaxy.ansible.com) হলো Role ও Collection-এর সরকারি ভান্ডার। এখানে প্রতিটি আইটেমের পাতায় দেখা যায়: ডাউনলোড সংখ্যা, তারকা (Star), শেষ আপডেটের তারিখ, সমর্থিত প্ল্যাটফর্ম, ও README। Galaxy-র বিকল্প হিসেবে **Automation Hub** আছে — Red Hat-এর যাচাই করা (Certified) Collection-এর ভান্ডার, যা এন্টারপ্রাইজ পরিবেশে বেশি ব্যবহৃত হয়। Galaxy থেকে কিছু নেওয়ার আগে তিনটি জিনিস দেখুন: (১) শেষ আপডেট কবে — এক বছরের বেশি পুরনো হলে সাবধানে ব্যবহার করুন; (২) ডাউনলোড ও ব্যবহারকারীর সংখ্যা — জনপ্রিয় মানেই পরীক্ষিত; (৩) কোডটি পড়ুন — বিশেষ করে `shell` Task ও বাইরের URL আছে কি না।

১৩.৬ অভ্যন্তরীণ কার্যপ্রণালী

আপনি যখন `ansible-galaxy collection install community.general` চালান, ভেতরে যা ঘটে: Galaxy-র API-তে অনুরোধ যায়, Collection-এর সর্বশেষ সংস্করণের `.tar.gz` ফাইল ডাউনলোড হয়, তার ভেতরের `galaxy.yml` পড়ে Dependency-গুলো বের করা হয়, এবং প্রতিটি Dependency-ও একইভাবে ডাউনলোড হয়। এরপর ফাইলগুলো `~/.ansible/collections/ansible_collections/community/general/`-এ বসানো হয়। Playbook চলার সময় আপনি যখন `community.general.npm` লেখেন, Ansible প্রথমে FQCN-টি ভেঙে দেখে — Namespace `community`, Collection `general`, Module `npm` — তারপর `collections_path`-এর প্রতিটি জায়গায় সেই Collection খোঁজে, Module-এর Python ফাইলটি লোড করে, আর Task-এর আর্গুমেন্টসহ তা Managed Node-এ চালায়। FQCN না লিখে শুধু `npm` লিখলে Ansible একটি খোঁজার ক্রম অনুসরণ করে: প্রথমে `ansible.builtin`, তারপর Playbook-এর `collections` কী-তে তালিকাভুক্ত Collection-গুলো, তারপর অন্যান্য ইনস্টল করা Collection। এই অনুমান-নির্ভর খোঁজার কারণেই শুধু নাম লেখা ঝুঁকিপূর্ণ — দুটি Collection-এ একই নামের Module থাকলে ভুলটি ডাকা হতে পারে। Lookup Plugin-এর ক্ষেত্রে কাজটি আরও সূক্ষ্ম: `{{ lookup('file', '/etc/motd') }}` লেখা হলে Lookup-টি Control Node-এই চলে — Managed Node-এ নয় — কারণ Lookup-এর কাজ Control Node-এর ফাইল বা পরিবেশ থেকে মান আনা। আর Filter Plugin Jinja2 Template Render করার সময় চলে — যেমন `{{ db_password | b64encode }}` লেখা হলে Template Render-এর মুহূর্তেই মানটি Base64-এ বদলে যায়।

> **TIP:** `ansible-galaxy collection list` দিয়ে ইনস্টল করা Collection-গুলোর সংস্করণ দেখুন, আর `ansible-doc community.general.npm` দিয়ে যেকোনো Module-এর ডকুমেন্টেশন টার্মিনালেই পড়ুন। `ansible-doc` হলো সবচেয়ে কম ব্যবহৃত কিন্তু সবচেয়ে দরকারি কমান্ড — ব্রাউজার খুলতে হয় না, উদাহরণসহ সব আর্গুমেন্ট দেখা যায়।

১৩.৭ আর্কিটেকচার চিত্র



চিত্রের মূল কথা: Core ছোট, Collection বড় — আর FQCN-ই সেই ভাগাভাগির ঠিকানা-ব্যবস্থা।

১৩.৮ ব্যবহারিক উদাহরণ

TechNova-র অ্যাপ Deploy-এ `community.general`-এর তিনটি Module লাগে — `npm`, `ufw` (ফায়ারওয়াল) ও `timezone`। প্রথমে Collection ইনস্টল করুন:

```
# Collection ইনস্টল করা
ansible-galaxy collection install community.general
# চিত্রণমূলক আউটপুট:
# Starting galaxy collection install process
# Process install dependency map
# Starting collection install process
# Downloading https://galaxy.ansible.com/.../community-general-9.5.0.tar.gz
# Installing 'community.general:9.5.0' to '/home/nahid/.ansible/collections/...'

# ইনস্টল করা Collection-এর তালিকা দেখুন
ansible-galaxy collection list
# চিত্রণমূলক আউটপুট:
# Collection        Version
# ---------------- -------
# community.general 9.5.0

# একটি Module-এর ডকুমেন্টেশন দেখুন
ansible-doc community.general.npm
# আউটপুটে দেখাবে: Module-এর বর্ণনা, প্রতিটি আর্গুমেন্ট, উদাহরণ
```

এবার Playbook-এ FQCN দিয়ে Module ডাকা:

```
---
- name: TechNova অ্যাপ সার্ভার প্রস্তুত করুন
  hosts: appservers
  become: true
  tasks:
    - name: সময় অঞ্চল Asia/Dhaka করুন
      community.general.timezone:
        name: Asia/Dhaka

    - name: শুধু প্রয়োজনীয় Port খুলুন (UFW)
      community.general.ufw:
        rule: allow
        port: "22"
        proto: tcp

    - name: npm নির্ভরতা ইনস্টল করুন
      community.general.npm:
        path: /opt/technova-api
        state: present
      become_user: technova
```

Lookup ও Filter Plugin-এর উদাহরণ:

```
# Lookup — Control Node-এর ফাইল থেকে মান আনা
- name: SSH পাবলিক কী দেখান
  ansible.builtin.debug:
    msg: "{{ lookup('ansible.builtin.file', '/home/nahid/.ssh/id_ed25519.pub') }}"
```

```
# Lookup — পরিবেশ Variable থেকে মান আনা
- name: API টোকেন পরিবেশ থেকে নিন
  ansible.builtin.debug:
    msg: "{{ lookup('ansible.builtin.env', 'TECHNOVA_API_TOKEN') }}"
  no_log: true

# Filter — Template-এ মান বদলানো
# {{ app_name | upper }}            → TECHNOVA-API
# {{ db_config | to_json }}         → JSON স্ট্রিং
# {{ 'hello world' | regex_replace(' ', '_') }} → hello_world
# {{ app_port | string }}           → "4000" (স্ট্রিং হিসেবে)
```

লক্ষ্য করুন, Lookup Control Node-এ চলে — তাই `file` Lookup-এ Control Node-এর পাথ লিখতে হয়, Managed Node-এর পাথ নয়। এটি নতুনদের সবচেয়ে বেশি গুলিয়ে ফেলার জায়গা।

১৩.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

`requirements.yml` হলো প্রজেক্টের Collection-এর "তালিকা" — নতুন কেউ প্রজেক্টটি পেলে একটি কমান্ডেই সব Collection ইনস্টল করতে পারেন:

```
# ── requirements.yml — Collection ও Role-এর Version Pin ──
---
collections:
  - name: community.general
    version: "9.5.0"
  - name: community.postgresql
    version: "3.10.0"
  - name: amazon.aws
    version: "8.1.0"
  - name: ansible.posix
    version: "1.6.2"

roles:
  - name: geerlingguy.nginx
    version: "3.1.0"

# ── ইনস্টল করার কমান্ড ──
# ansible-galaxy install -r requirements.yml
# Collection ইনস্টল হবে ~/.ansible/collections-এ
# Role ইনস্টল হবে ~/.ansible/roles-এ

# ── ansible.cfg — Collection পাথ ও Callback Plugin ──
[defaults]
inventory = inventory/hosts.yml
roles_path = roles
collections_path = ~/.ansible/collections:/usr/share/ansible/collections
# profile_tasks Callback চালু করলে প্রতিটি Task-এর সময় দেখা যায়
callbacks_enabled = ansible.builtin.profile_tasks
stdout_callback = yaml
# stdout_callback = yaml দিলে আউটপুট YAML আকারে সাজানো দেখায় — পড়তে সহজ

# ── site.yml — FQCN দিয়ে লেখা সম্পূর্ণ Playbook ──
---
- name: ডেটাবেস সার্ভার প্রস্তুত করুন
  hosts: dbservers
```

```
  become: true
  collections:
    - community.postgresql
  tasks:
    - name: PostgreSQL ডেটাবেস তৈরি করুন
      community.postgresql.postgresql_db:
        name: technova_production
        state: present
      become_user: postgres

    - name: অ্যাপের ডেটাবেস ব্যবহারকারী তৈরি করুন
      community.postgresql.postgresql_user:
        name: technova_app
        password: "CHANGE-ME-DB-PASSWORD"
        db: technova_production
        priv: "ALL"
      become_user: postgres
      no_log: true
```

একটি সতর্কতা: উপরের উদাহরণে পাসওয়ার্ডটি সরাসরি লেখা হয়েছে শুধু গঠন দেখানোর জন্য — বাস্তবে এটি Ansible Vault-এ থাকবে (অধ্যায় ১৪)। `collections` কী-টি Play-এর শুরুতে লিখলে সেই Play-এর ভেতরে ছোট নাম (`postgresql_db`) ব্যবহার করা যায় — তবে প্রোডাকশনে FQCN-ই লেখা ভালো, কারণ ছোট নাম পড়ে বোঝা যায় না Module-টি কোন Collection-এর।

১৩.১০ ধাপে ধাপে বাস্তবায়ন

1. **বর্তমান অবস্থা দেখুন:** `ansible-galaxy collection list` চালিয়ে দেখুন কোন Collection আগে থেকে ইনস্টল আছে।
2. **একটি Collection ইনস্টল করুন:** `ansible-galaxy collection install community.general` চালান। ডাউনলোড ও ইনস্টলের আউটপুট পড়ুন।
3. **নির্দিষ্ট সংস্করণ ইনস্টল করুন:** `ansible-galaxy collection install community.general==9.5.0` — সংস্করণ নির্দিষ্ট না করলে সর্বশেষটি আসে, যা হঠাৎ বদলে যেতে পারে।
4. **Module-এর ডকুমেন্টেশন পড়ুন:** `ansible-doc community.general.ufw` চালিয়ে আর্গুমেন্ট ও উদাহরণ দেখুন।
5. **`requirements.yml` লিখুন:** প্রজেক্টের সব Collection নাম ও সংস্করণসহ লিখুন (উপরের উদাহরণ দেখুন)।
6. **Requirements থেকে ইনস্টল করুন:** `ansible-galaxy install -r requirements.yml` চালান — নতুন মেশিনে এটাই প্রথম কমান্ড।
7. **Playbook-এ FQCN ব্যবহার করুন:** `community.general.timezone` দিয়ে সময় অঞ্চল বদলানোর Task লিখুন ও `--check` দিয়ে পরীক্ষা করুন।
8. **Callback Plugin চালু করুন:** `ansible.cfg`-এ `callbacks_enabled = ansible.builtin.profile_tasks` লিখে Playbook চালান — প্রতিটি Task-এর সময় দেখা যাবে।
9. **Lookup পরীক্ষা করুন:** `ansible localhost -m debug -a "msg={{ lookup('ansible.builtin.env', 'HOME') }}"` চালিয়ে দেখুন Lookup Control Node-এ চলে।
10. **আপডেটের নিয়ম ঠিক করুন:** Collection আপডেট করার আগে `ansible-galaxy collection install community.general --force`-এর পরিবর্তে `requirements.yml`-এর সংস্করণ বদলে পরীক্ষামূলক পরিবেশে (Staging) চালান।

১৩.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

```
$ ansible-galaxy collection install community.general
# চিত্রণমূলক আউটপুট:
# Starting galaxy collection install process
# Process install dependency map
# Starting collection install process
# Downloading https://galaxy.ansible.com/api/v3/.../community-general-9.5.0.tar.gz
# Installing 'community.general:9.5.0' to '/home/nahid/.ansible/collections/
ansible_collections/community/general'
# community.general:9.5.0 was installed successfully

$ ansible-playbook site.yml
# চিত্রণমূলক আউটপুট (profile_tasks Callback চালু থাকলে):
# TASK [সময় অঞ্চল Asia/Dhaka করুন] ***********************************
# changed: [db01.technova.example]
#
# PLAY RECAP *********************************************************
# db01.technova.example : ok=3  changed=1  unreachable=0  failed=0
#
# Playbook run took 0 days, 0 hours, 0 minutes, 12 seconds
# ===============================================================================
# সময় অঞ্চল Asia/Dhaka করুন ---------------------------------- 4.21s
# PostgreSQL ডেটাবেস তৈরি করুন ------------------------------- 2.03s
```

ইনস্টলের আউটপুটে তিনটি জিনিস দেখুন: কোন সংস্করণ ডাউনলোড হলো, কোথায় ইনস্টল হলো, আর Dependency কিছু ইনস্টল হলো কি না। `profile_tasks` Callback-এর আউটপুটে প্রতিটি Task-এর সময় দেখা যায় — কোন Task ধীর, তা চিহ্নিত করতে এটি অমূল্য। `PLAY RECAP`-এর নিচে সময়ের সারাংশ আসে — পুরো Playbook কতক্ষণ চলল। যদি ইনস্টলের সময় `ERROR: Failed to download` দেখায়, তবে প্রথমে ইন্টারনেট সংযোগ ও Galaxy-র ঠিকানা দেখুন — কখনো কখনো Galaxy সাময়িকভাবে বন্ধ থাকে, তখন `--force` নয়, পরে আবার চেষ্টা করাই সঠিক।

১৩.১২ সাধারণ ভুল

- **FQCN না লেখা:** শুধু `npm` লিখলে ভুল Collection-এর Module ডাকা হতে পারে — সবসময় পুরো নাম লিখুন।
- **সংস্করণ Pin না করা:** `requirements.yml`-এ সংস্করণ না লিখলে ছয় মাস পরে নতুন মেশিনে অন্য সংস্করণ ইনস্টল হয়ে Playbook ভাঙতে পারে।
- **Lookup-কে Managed Node-এর কাজ ভাবা:** `lookup('file', ...)` Control Node-এর ফাইল পড়ে — Managed Node-এর ফাইল পড়তে `slurp` বা `fetch` Module লাগে।
- **Galaxy থেকে যাচাই না করে Role নেওয়া:** এক বছরের বেশি আপডেট হয়নি, এমন Role প্রোডাকশনে চালানো বিপজ্জনক।
- **Collection-এর ভেতরে নিজের Role লেখা:** নিজের Role `roles/`-এ রাখুন; Collection বানানো হয় শেয়ার করার জন্য, একটিমাত্র প্রজেক্টের জন্য নয়।
- **`ansible.builtin` ও `community.general`-এর একই নামের Module গুলিয়ে ফেলা:** যেমন `ufw` শুধু `community.general`-এ আছে, `builtin`-এ নেই — ডকুমেন্টেশন না পড়ে অনুমান করবেন না।
- **Callback Plugin চালু করে আউটপুটে Secret ফাঁস করা:** `yaml` Callback বিস্তারিত আউটপুট দেখায় — Secret-যুক্ত Task-এ `no_log: true` না দিলে লগে Secret ছাপা হতে পারে।

১৩.১৩ সমস্যা সমাধান (Troubleshooting)

REAL WORLD — TechNova: TechNova-র CI Pipeline হঠাৎ ভাঙল — `community.postgresql.postgresql_db` Module পাওয়া যাচ্ছে না বলে ত্রুটি। কারণ ছিল CI-এর নতুন Runner-এ `requirements.yml` থেকে Collection ইনস্টল করার ধাপটি বাদ পড়েছিল — Playbook-টি Control Node-এ চলছিল, কিন্তু Collection ইনস্টলই হয়নি। `ansible-galaxy collection list` দিয়ে দেখা গেল তালিকা খালি। সমাধান: CI-তে `ansible-galaxy install -r requirements.yml` ধাপটি Playbook চালানোর আগে বাধ্যতামূলক করা হলো। শিক্ষা: "Module পাওয়া যাচ্ছে না" মানে প্রায়ই Collection ইনস্টল হয়নি — Module-এর দোষ নয়।

1. **লক্ষণ:** `ERROR! couldn't resolve module/action 'npm'` — **পরীক্ষা:** `ansible-galaxy collection list | grep community` — Collection ইনস্টল আছে কি না। **সমাধান:** `ansible-galaxy collection install community.general` চালান, অথবা FQCN লিখুন।
2. **লক্ষণ:** ইনস্টলের সময় `Failed to download` — **পরীক্ষা:** ইন্টারনেট সংযোগ ও Galaxy-র অবস্থা; প্রক্সি থাকলে `https_proxy` পরিবেশ Variable দেখুন। **সমাধান:** কিছুক্ষণ পরে আবার চেষ্টা করুন; প্রয়োজনে `--force` দিয়ে পুনরায় ইনস্টল করুন।
3. **লক্ষণ:** ভুল সংস্করণের Module-এর আচরণ — **পরীক্ষা:** `ansible-galaxy collection list` দিয়ে ইনস্টল করা সংস্করণ ও `requirements.yml`-এর সংস্করণ মিলিয়ে দেখুন। **সমাধান:** `ansible-galaxy collection install community.general==9.5.0 --force` দিয়ে নির্দিষ্ট সংস্করণে ফিরুন।
4. **লক্ষণ:** Lookup-এ ফাইলের মান খালি আসছে — **পরীক্ষা:** পাথটি Control Node-এর কি না; Managed Node-এর পাথ হলে Lookup কাজ করবে না। **সমাধান:** Managed Node-এর ফাইল পড়তে `slurp` Module ব্যবহার করুন।
5. **লক্ষণ:** Callback Plugin-এর আউটপুট দেখা যাচ্ছে না — **পরীক্ষা:** `ansible.cfg`-এ `callbacks_enabled`-এর বানান ও Plugin-এর নাম FQCN-সহ লেখা আছে কি না। **সমাধান:** `ansible-doc -t callback -l` দিয়ে উপলব্ধ Callback-এর তালিকা দেখুন।

১৩.১৪ নিরাপত্তা বিবেচনা

WARNING: Galaxy একটি খোলা ভান্ডার — যে কেউ Collection প্রকাশ করতে পারে। একটি Collection ইনস্টল করা মানে তার Python কোড আপনার Control Node-এ চালানো, আর তার Module Managed Node-এ `root` হয়ে চলতে পারে। তাই Collection-এর `plugins/modules/` ডিরেক্টরি অন্তত চোখ বুলিয়ে দেখুন: `subprocess`, `os.system`, বা বাইরের URL থেকে ডাউনলোডের মতো সন্দেহজনক কোড আছে কি না। এন্টারপ্রাইজ পরিবেশে শুধু Automation Hub-এর যাচাই করা Collection ব্যবহার করাই নিরাপদ নীতি।

আরও দুটি নিরাপত্তা-নিয়ম: প্রথম, Collection-এর সংস্করণ Pin করুন — সর্বশেষ সংস্করণ স্বয়ংক্রিয়ভাবে নিলে একদিন একটি দূষিত (Compromised) সংস্করণ ঢুকে যেতে পারে; Pin করা সংস্করণ বদলানোর সময় Code Review করুন। দ্বিতীয়, Lookup Plugin দিয়ে Secret আনার সময় `no_log: true` দিন — নইলে `debug` বা Callback-এর আউটপুটে Secret ছাপা হয়ে লগে থেকে যাবে। তৃতীয়, `community.general.npm`-এর মতো Module-এ বাইরের প্যাকেজ ইনস্টল হয় — প্যাকেজের উৎস (Registry) বিশ্বাসযোগ্য কি না, তা যাচাই করুন। Collection নিজে নিরাপদ হলেও তার মাধ্যমে আসা তৃতীয় পক্ষের প্যাকেজ অনিরাপদ হতে পারে।

১৩.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

প্রোডাকশনে Collection ব্যবহারের মানদণ্ড: **সবকিছু `requirements.yml`-এ** — Collection, Role, সংস্করণ — সব Git-এ থাকবে, যাতে যেকোনো মেশিনে একই পরিবেশ তৈরি হয়; **CI-তে Collection ইনস্টল বাধ্যতামূলক** — Playbook চালানোর আগের ধাপে `ansible-galaxy install -r requirements.yml` চলবে; **সংস্করণ আপডেটের প্রক্রিয়া** — নতুন সংস্করণ প্রথমে Staging-এ পরীক্ষা করে, তারপর Production-এর `requirements.yml` বদলানো হবে; **FQCN বাধ্যতামূলক** — Code Review-এ ছোট নামের Module দেখলে ফেরত পাঠানো হবে; **নিজের Collection শুধু প্রয়োজনে** — একাধিক প্রজেক্টে একই Role ও Module বারবার লাগলে তবেই নিজের Collection বানান, একটি প্রজেক্টের জন্য Role-ই যথেষ্ট; **Galaxy-র বিকল্প ভান্ডার** — কোম্পানির ভেতরে নিজস্ব Galaxy Server বা Automation Hub ব্যবহার করলে বাইরের নির্ভরতা কমে; **নিয়মিত অডিট** — প্রতি মাসে `ansible-galaxy collection list` দিয়ে দেখুন কোন Collection পুরনো হয়েছে, আর তার Changelog পড়ে আপডেট করবেন কি না সিদ্ধান্ত নিন।

১৩.১৬ অধ্যায়ের সারাংশ

Collection হলো Module, Role ও Plugin-এর নাম-দেওয়া প্যাকেজ — `namespace.collection` আকারে নামকরণ করা হয়, Galaxy থেকে `ansible-galaxy collection install` দিয়ে ইনস্টল করা হয়, আর `requirements.yml`-এ সংস্করণ Pin করে পুনরুৎপাদনযোগ্য রাখা হয়। FQCN — যেমন `community.general.npm` — লেখাই সঠিক অভ্যাস, কারণ ছোট নামে ভুল Module ডাকা হতে পারে। `ansible.builtin`-এ আছে সবচেয়ে সাধারণ Module, `community.general`-এ আছে শত শত সাধারণ Module, আর ক্লাউডের কাজে Vendor-এর Collection (`amazon.aws`, `azure.azcollection`) ব্যবহার করা হয়। Plugin পাঁচ ধরনের — Connection, Lookup, Callback, Inventory ও Filter — প্রতিটি Ansible-এর একটি নির্দিষ্ট আচরণ বাড়ায়। Galaxy থেকে কিছু নেওয়ার আগে তার জনপ্রিয়তা, শেষ আপডেট ও কোড যাচাই করা বাধ্যতামূলক, কারণ Collection-এর কোড আপনার সার্ভারে উচ্চ অধিকার নিয়ে চলে।

১৩.১৭ জ্ঞান যাচাই — প্রশ্ন

1. FQCN কী? `community.general.npm`-এর তিনটি অংশ আলাদা করে ব্যাখ্যা করুন।
2. `ansible.builtin`, `community.general` ও `amazon.aws`-এর মধ্যে পার্থক্য কী? প্রতিটির একটি করে ব্যবহারের উদাহরণ দিন।
3. Lookup Plugin কোথায় চলে — Control Node-এ না Managed Node-এ? `lookup('file', ...)` দিয়ে Managed Node-এর ফাইল পড়া যায় না কেন?
4. Connection, Lookup, Callback, Inventory ও Filter Plugin — প্রতিটির একটি করে উদাহরণসহ কাজ ব্যাখ্যা করুন।
5. `requirements.yml`-এ সংস্করণ Pin করা কেন জরুরি? Pin না করলে ছয় মাস পরে কী সমস্যা হতে পারে?

১৩.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. **Collection ইনস্টল ও পরীক্ষা:** `community.general` ইনস্টল করে `ansible-doc community.general.ufw` পড়ুন, তারপর একটি Playbook লিখুন যা UFW দিয়ে শুধু Port 22 ও 80 খোলে।
2. **requirements.yml বানান:** আপনার TechNova প্রজেক্টের জন্য `requirements.yml` লিখুন — অন্তত তিনটি Collection সংস্করণসহ — এবং নতুন ডিরেক্টরিতে `ansible-galaxy install -r requirements.yml` চালিয়ে যাচাই করুন।

3. **Lookup ও Filter:** একটি Playbook লিখুন যা `lookup('ansible.builtin.env', 'HOME')` দিয়ে Control Node-এর Home ডিরেক্টরি দেখায়, আর একটি Variable-কে `| upper` ও `| to_json` Filter দিয়ে বদলে দেখায়।
4. **Galaxy যাচাই:** Galaxy-তে একটি জনপ্রিয় Role-এর পাতা খুঁজে তার শেষ আপডেটের তারিখ, ডাউনলোড সংখ্যা ও README পড়ে লিখুন — আপনি কি এটি প্রোডাকশনে ব্যবহার করবেন? কেন বা কেন নয়?

অধ্যায় 14 • PART III — Roles, Collections, Vault & Cloud

Chapter 14 — Ansible Vault and Secrets Management

১৪.১ ভূমিকা

TechNova-র এক জুনিয়র ইঞ্জিনিয়ার একদিন Git-এ একটি Playbook Push করলেন — তাতে ডেটাবেসের পাসওয়ার্ড সরাসরি লেখা ছিল। কয়েক ঘণ্টার মধ্যে কেউ একজন সেই Repository-টি দেখে পাসওয়ার্ডটি নিয়ে Production ডেটাবেসে ঢুকে পড়ল। ক্ষতি সামাল দিতে পাসওয়ার্ড বদলানো, সব সার্ভারের কী ঘোরানো (Rotation), আর গ্রাহকদের জানানো — সব মিলিয়ে এক সপ্তাহের কাজ। অথচ সমাধানটি ছিল সহজ — **Ansible Vault**। Vault হলো Ansible-এর নিজস্ব গোপন-তথ্য (Secret) সুরক্ষার ব্যবস্থা: পাসওয়ার্ড, API Key, Private Key — যেকোনো সংবেদনশীল মান AES-256 দিয়ে এনক্রিপ্ট (Encrypt) করে ফাইলে রাখা যায়, আর Playbook চলার সময় শুধু সঠিক Vault পাসওয়ার্ড দিলেই তা খুলে ব্যবহার করা যায়। এই অধ্যায়ে আপনি শিখবেন Vault দিয়ে ফাইল ও একক মান (String) কীভাবে এনক্রিপ্ট করতে হয়, Vault পাসওয়ার্ড কীভাবে নিরাপদে রাখতে হয়, Vault ID দিয়ে একাধিক পরিবেশের আলাদা Secret কীভাবে পরিচালনা করতে হয়, আর সবচেয়ে গুরুত্বপূর্ণ — কী করলে Secret আর কখনো Git-এ ফাঁস হবে না।

১৪.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) `ansible-vault create`, `edit`, `encrypt`, `decrypt` ও `rekey` কমান্ড দিয়ে Vault ফাইল তৈরি ও পরিচালনা করতে; (২) Vault পাসওয়ার্ড ফাইল ও `--ask-vault-pass` ব্যবহার করে Playbook চালাতে; (৩) `encrypt_string` দিয়ে একটি মাত্র Variable-এর মান এনক্রিপ্ট করে Playbook-এর ভেতরে রাখতে; (৪) Vault ID দিয়ে Development, Staging ও Production-এর আলাদা Secret পরিচালনা করতে; (৫) `no_log: true` দিয়ে সংবেদনশীল Task-এর আউটপুট লুকাতে; (৬) Secret Rotation-এর ধারণা ব্যাখ্যা করে একটি নিরাপদ Rotation-পরিকল্পনা লিখতে।

১৪.৩ পূর্বশর্ত জ্ঞান

Variable ও `group_vars` / `host_vars`-এর ধারণা, YAML-এ Variable লেখা, Playbook চালানোর অভিজ্ঞতা, এবং Git-এর প্রাথমিক ধারণা (Commit, Push, `.gitignore`) প্রয়োজন। এনক্রিপশনের গভীর গণিত জানার দরকার নেই — শুধু এইটুকু জানলেই চলবে: এনক্রিপশন মানে পাঠযোগ্য মানকে এমনভাবে বদলে দেওয়া, যা শুধু সঠিক চাবি (Key) থাকলেই আবার পড়া যায়। Linux-এ ফাইল Permission (`chmod 600`) ও পরিবেশ Variable-এর ধারণা থাকলে Vault পাসওয়ার্ড ফাইলের অংশটি সহজে বোঝা যাবে।

১৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

Secret পরিচালনার প্রথম নীতি হলো **আলাদা করা** (Separation): কোড আর Secret এক জায়গায় থাকবে না। কোড Git-এ যায়, সবার দেখার জন্য; Secret যায় শুধু তাদের কাছে, যাদের সত্যিই দরকার। দ্বিতীয় নীতি হলো **এনক্রিপশন**: Secret যদি কোনোভাবে ফাঁসও হয়, তবে তা যেন পড়া না যায় — শুধু চাবি থাকলেই পড়া যাবে। তৃতীয় নীতি হলো **সর্বনিম্ন অধিকার** (Least Privilege): যে Secret যার দরকার নেই, সে তা দেখতে পাবে না — Development-এর Secret Production-এর Secret থেকে আলাদা থাকবে। Ansible Vault এই তিনটি নীতিকেই একসঙ্গে প্রয়োগ করে: Secret-যুক্ত ফাইল AES-256 দিয়ে এনক্রিপ্ট করা হয় — এনক্রিপ্ট করা ফাইলটি Git-এ রাখা যায়, কারণ তা পড়া যায় না; Playbook চলার সময় Vault পাসওয়ার্ড দিলে Ansible ফাইলটি মেমরিতে খুলে মানগুলো ব্যবহার করে, কিন্তু ডিস্কে কখনো প্লেইনটেক্সট (Plaintext —

এনক্রিপ্ট না করা, সরাসরি পড়া যায় এমন লেখা) লেখে না। চাবি (Vault পাসওয়ার্ড) আর তালা (এনক্রিপ্ট করা ফাইল) আলাদা জায়গায় থাকে — এটাই পুরো ব্যবস্থার মূল। আর একটি কথা মনে রাখবেন: Vault পাসওয়ার্ড হারিয়ে গেলে এনক্রিপ্ট করা Secret আর কখনো উদ্ধার করা যায় না — কোনো "Forgot Password" বোতাম নেই। তাই পাসওয়ার্ডের নিরাপদ সংরক্ষণ Vault ব্যবহারেরই অংশ।

> **CONCEPT:** Ansible Vault একটি তালা দেওয়া বাক্সের মতো: বাক্সটি (এনক্রিপ্ট করা ফাইল) আপনি সবার সামনে রাখতে পারেন — Git-এ, Slack-এ, যেকোনো জায়গায় — কারণ বাক্স না খুলে ভেতরের জিনিস দেখা যায় না। চাবিটি (Vault পাসওয়ার্ড) শুধু আপনার কাছে থাকে। কিন্তু বাক্সের গায়ে চাবি ঝুলিয়ে রাখলে (যেমন Vault পাসওয়ার্ড ফাইলটি Git-এ Push করলে) তালার কোনো মূল্য থাকে না। Vault-এর নিরাপত্তা তাই ফাইলের এনক্রিপশনে নয়, চাবির ব্যবস্থাপনায় — চাবি কোথায় থাকে, কে জানে, কীভাবে ঘোরানো হয় — সেটাই আসল নিরাপত্তা।

১৪.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Vault কমান্ডসমূহ

কমান্ড	কাজ	কখন ব্যবহার করবেন
`ansible-vault create secrets.yml`	নতুন এনক্রিপ্ট করা ফাইল তৈরি করে এডিটরে খোলে	প্রথমবার Secret ফাইল বানাতে
`ansible-vault edit secrets.yml`	এনক্রিপ্ট করা ফাইল খুলে সম্পাদনা করে, আবার এনক্রিপ্ট করে বন্ধ করে	Secret বদলাতে
`ansible-vault encrypt secrets.yml`	প্লেইনটেক্সট ফাইলকে এনক্রিপ্ট করে	আগে থেকে লেখা ফাইল সুরক্ষিত করতে
`ansible-vault decrypt secrets.yml`	এনক্রিপ্ট করা ফাইলকে প্লেইনটেক্সট করে	শুধু সাময়িকভাবে — সাবধানে, কারণ ফাইলটি তখন খোলা থাকে
`ansible-vault view secrets.yml`	ডিক্রিপ্ট না করে শুধু বিষয়বস্তু দেখায়	Secret দেখতে, কিন্তু ফাইল না খুলতে
`ansible-vault rekey secrets.yml`	Vault পাসওয়ার্ড বদলায় (পুরনো পাসওয়ার্ড জানতে হয়)	পাসওয়ার্ড ফাঁস হলে বা নিয়মিত Rotation-এ
`ansible-vault encrypt_string 'মান' --name 'var'`	একটি মাত্র স্ট্রিং এনক্রিপ্ট করে YAML আকারে দেয়	একটি Variable Playbook-এর ভেতরেই রাখতে

একটি এনক্রিপ্ট করা ফাইল দেখতে এইরকম — প্রথম লাইনে `$ANSIBLE_VAULT;1.1;AES256` লেখা থাকে, যা বলে দেয় এটি Vault ফাইল, সংস্করণ 1.1, আর এনক্রিপশন AES-256:

```
$ANSIBLE_VAULT;1.1;AES256
36383364386636613638396533306462313635346435383130386463346538313
39616262376235323833336565336162623364616231363532656632386465663
... (এনক্রিপ্ট করা বিষয়বস্তু — পড়া যায় না)
```

Vault পাসওয়ার্ড দেওয়ার উপায়

Playbook চালানোর সময় Vault পাসওয়ার্ড চারভাবে দেওয়া যায়। প্রথম, `--ask-vault-pass` — চালানোর সময় টার্মিনালে পাসওয়ার্ড চাওয়া হয়; সবচেয়ে নিরাপদ, কারণ পাসওয়ার্ড কোথাও লেখা থাকে না, তবে CI Pipeline-এ চলে না। দ্বিতীয়, `--vault-password-file ~/.ansible/vault-pass.txt` — পাসওয়ার্ড একটি ফাইলে রাখা হয়; ফাইলটির Permission `chmod 600` হতে হবে, আর ফাইলটি কখনো Git-এ যাবে না। তৃতীয়, `ansible.cfg`-এ `vault_password_file` লিখে দেওয়া — প্রতিবার ফ্ল্যাগ লিখতে হয় না। চতুর্থ, **Vault ID** — একাধিক পাসওয়ার্ডের জন্য: `--vault-id dev@prompt` বা `--vault-id prod@~/.ansible/vault-pass-prod.txt` — `@`-এর আগে লেবেল, পরে উৎস। Vault ID-র ফাইলে প্রথম লাইনে লেবেল লেখা থাকে — যেমন `$ANSIBLE_VAULT;1.1;AES256;prod` — যাতে Ansible জানে কোন পাসওয়ার্ড দিয়ে কোন ফাইল খুলতে হবে।

encrypt_string — একটি মানের এনক্রিপশন

পুরো ফাইল এনক্রিপ্ট না করে একটি মাত্র Variable-এর মান এনক্রিপ্ট করতে `encrypt_string` ব্যবহার করা হয়। এটি YAML-এর একটি ব্লক তৈরি করে, যা Playbook বা `group_vars` ফাইলে সরাসরি বসানো যায় — ফাইলের বাকি অংশ প্লেইনটেক্সট থাকে, শুধু Secret-টুকু এনক্রিপ্ট করা থাকে। এটি Code Review-এর জন্য সুবিধাজনক — পর্যালোচক দেখতে পান কোন Variable Secret, কিন্তু মানটি দেখতে পান না।

no_log — আউটপুটে Secret লুকানো

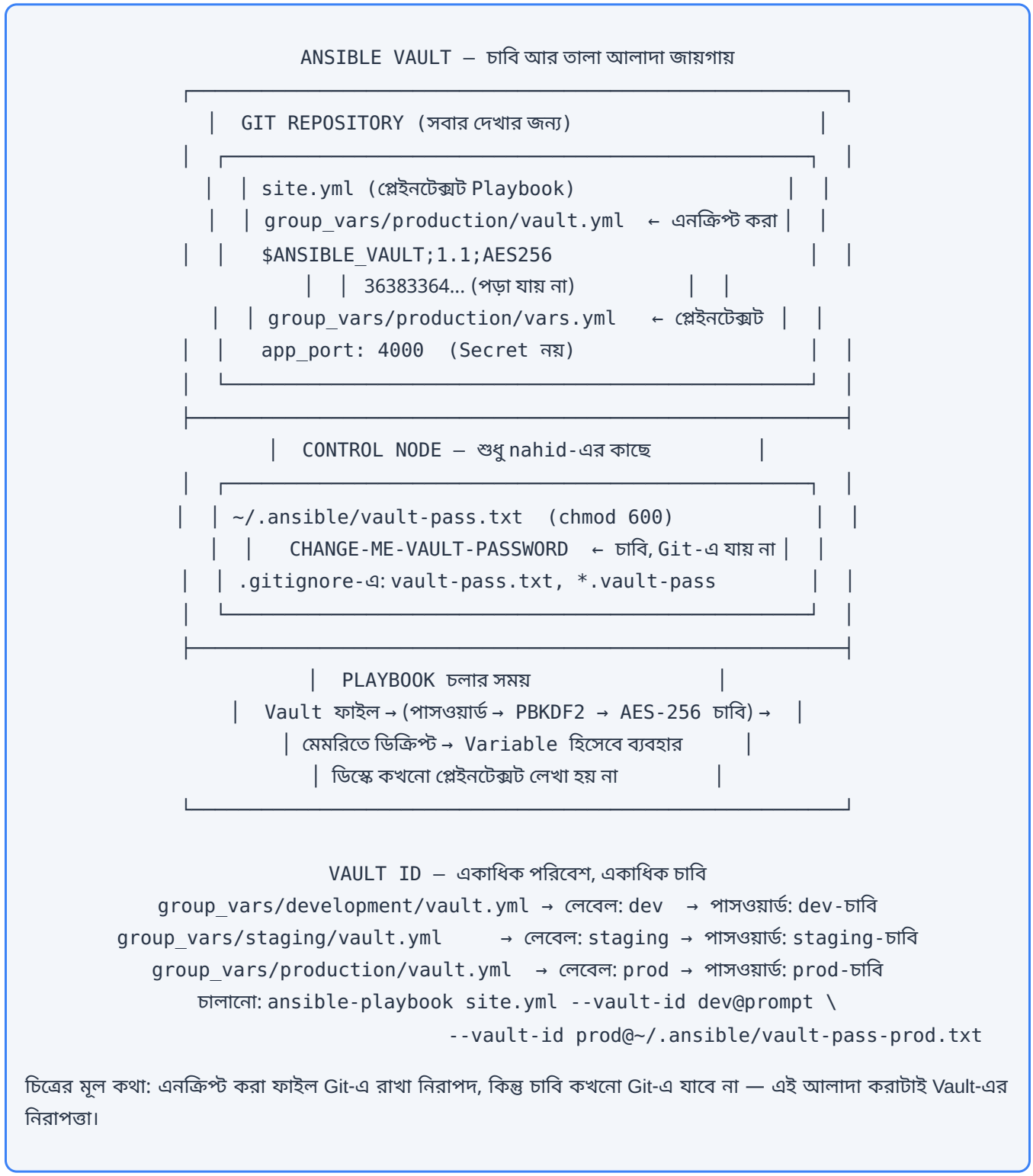
Vault দিয়ে Secret এনক্রিপ্ট করলেও আরেকটি ফাঁসের পথ থাকে — Playbook-এর আউটপুট। কোনো Task-এ Secret-যুক্ত Variable ব্যবহার করলে Ansible ডিফল্টভাবে Task-এর আর্গুমেন্ট আউটপুটে দেখায় না, কিন্তু `debug` Task বা ত্রুটির বার্তায় Secret ছাপা হয়ে যেতে পারে। তাই সংবেদনশীল Task-এ `no_log: true` লিখুন — তাতে সেই Task-এর আউটপুট সম্পূর্ণ লুকানো থাকে। সতর্কতা: `no_log: true` দিলে Task ব্যর্থ হলেও কারণ দেখা যায় না — তাই প্রথমে `no_log` ছাড়া পরীক্ষা করে, ঠিকঠাক চললে তবেই `no_log: true` যোগ করুন।

১৪.৬ অভ্যন্তরীণ কার্যপ্রণালী

Vault-এর ভেতরের কাজটি এইরকম: আপনি যখন `ansible-vault create` চালান, Ansible আপনার দেওয়া পাসওয়ার্ড থেকে **PBKDF2** (Password-Based Key Derivation Function 2) দিয়ে একটি এনক্রিপশন চাবি তৈরি করে — পাসওয়ার্ডটিকে হাজার বার হ্যাশ করে এমন একটি চাবি বানানো হয়, যা থেকে পাসওয়ার্ডটি আর উদ্ধার করা যায় না, আর Brute Force আক্রমণ (সব সম্ভাব্য পাসওয়ার্ড চেষ্টা করা) ধীর হয়ে যায়। এরপর ফাইলের বিষয়বস্তু **AES-256** (Advanced Encryption Standard, 256-bit চাবি) দিয়ে এনক্রিপ্ট করা হয় — সঙ্গে একটি এলোমেলো Salt ও Initialization Vector (IV) যোগ করা হয়, যাতে একই বিষয়বস্তু দুবার এনক্রিপ্ট করলেও ভিন্ন ফলাফল আসে। Playbook চলার সময় Ansible Vault ফাইলটি পড়ে, পাসওয়ার্ড থেকে একই পদ্ধতিতে চাবি তৈরি করে, ফাইলটি মেমরিতে ডিক্রিপ্ট করে, আর Variable-গুলোকে সাধারণ Variable-এর মতো ব্যবহার করে — ডিস্কে কখনো প্লেইনটেক্সট লেখা হয় না। Vault ID থাকলে Ansible ফাইলের লেবেল দেখে সঠিক পাসওয়ার্ডটি বেছে নেয়; লেবেল না মিললে সে প্রতিটি দেওয়া পাসওয়ার্ড চেষ্টা করে দেখে। একটি গুরুত্বপূর্ণ সীমা মনে রাখবেন: Vault ফাইল এনক্রিপ্ট করে, কিন্তু চলমান প্রক্রিয়ার মেমরি বা লগ ফাইল সুরক্ষিত করে না — তাই `no_log: true` ও লগের নিরাপত্তা আলাদাভাবে দেখতে হয়। আর Vault পাসওয়ার্ড নিজে কোথাও এনক্রিপ্ট করা থাকে না — সেটি একটি ফাইলে প্লেইনটেক্সট থাকে, বা মানুষের মনে থাকে — তাই পাসওয়ার্ডের সংরক্ষণই Vault-এর সবচেয়ে দুর্বল জায়গা।

WARNING: Vault পাসওয়ার্ড হারিয়ে গেলে এনক্রিপ্ট করা Secret আর কখনো উদ্ধার করা যায় না — Ansible-এর কোনো ব্যাকডোর (পেছনের দরজা) নেই। তাই পাসওয়ার্ডটি অন্তত দুটি নিরাপদ জায়গায় রাখুন — যেমন দলের পাসওয়ার্ড ম্যানেজার (1Password, Bitwarden) ও একজন বিশ্বস্ত সদস্যের কাছে — আর `rekey` করার আগে পুরনো পাসওয়ার্ড দিয়ে ফাইলটি খোলা যায় কি না, তা পরীক্ষা করে নিন।

১৪.৭ আর্কিটেকচার চিত্র



চিত্রের মূল কথা: এনক্রিপ্ট করা ফাইল Git-এ রাখা নিরাপদ, কিন্তু চাবি কখনো Git-এ যাবে না — এই আলাদা করাটাই Vault-এর নিরাপত্তা।

১৪.৮ ব্যবহারিক উদাহরণ

TechNova-র Production ডেটাবেসের পাসওয়ার্ড Vault-এ রাখার পুরো প্রক্রিয়া:

```
# 1. Vault পাসওয়ার্ড ফাইল তৈরি করুন (শুধু আপনার মেশিনে, Git-এর বাইরে)
echo 'CHANGE-ME-VAULT-PASSWORD' > ~/.ansible/vault-pass.txt
chmod 600 ~/.ansible/vault-pass.txt
# ব্যাখ্যা: chmod 600 মানে শুধু ফাইলের মালিক পড়তে পারবেন — অন্য কেউ নয়।

# 2. .gitignore-এ পাসওয়ার্ড ফাইল যোগ করুন — যাতে ভুলেও Git-এ না যায়
cat >> ~/technova-ansible/.gitignore <<'EOF'
vault-pass.txt
*.vault-pass
.vault-pass*
EOF

# 3. নতুন Vault ফাইল তৈরি করুন
ansible-vault create group_vars/production/vault.yml \
  --vault-password-file ~/.ansible/vault-pass.txt
# এডিটর খুলবে — তাতে লিখুন:
# db_password: "CHANGE-ME-DB-PASSWORD"
# api_secret_key: "CHANGE-ME-API-SECRET"
# jwt_secret: "CHANGE-ME-JWT-SECRET"

# 4. ফাইলটি দেখুন — এনক্রিপ্ট করা অবস্থায়
cat group_vars/production/vault.yml
# চিত্রণমূলক আউটপুট:
# $ANSIBLE_VAULT;1.1;AES256
# 36383364386636613638396533306462313635346435383130386463346538 31
# ...

# 5. ডিক্রিপ্ট না করে শুধু বিষয়বস্তু দেখুন
ansible-vault view group_vars/production/vault.yml \
  --vault-password-file ~/.ansible/vault-pass.txt

# 6. একটি মাত্র স্ট্রিং এনক্রিপ্ট করুন (Playbook-এর ভেতরে রাখার জন্য)
ansible-vault encrypt_string 'CHANGE-ME-DB-PASSWORD' --name 'db_password' \
  --vault-password-file ~/.ansible/vault-pass.txt
# চিত্রণমূলক আউটপুট:
# db_password: !vault |
#   $ANSIBLE_VAULT;1.1;AES256
#   36383364386636613638396533306462313635346435383130386463346538 31
#   ...
```

এবার Playbook-এ Vault Variable ব্যবহার করা:

```
# group_vars/production/vars.yml (প্লেইনটেক্সট — Secret নয়)
db_name: technova_production
db_user: technova_app
db_host: 203.0.113.12

# group_vars/production/vault.yml (এনক্রিপ্ট করা — Secret)
# db_password: "CHANGE-ME-DB-PASSWORD"  ← এনক্রিপ্ট করা অবস্থায় থাকে

# Playbook-এ ব্যবহার — সাধারণ Variable-এর মতোই
- name: ডেটাবেস ব্যবহারকারী তৈরি করুন
```

```
  community.postgresql.postgresql_user:
    name: "{{ db_user }}"
    password: "{{ db_password }}"
    db: "{{ db_name }}"
  become_user: postgres
  no_log: true   # ← পাসওয়ার্ড আউটপুটে না দেখাতে

# Playbook চালানো
ansible-playbook site.yml --vault-password-file ~/.ansible/vault-pass.txt
# অথবা পাসওয়ার্ড হাতে লিখতে:
ansible-playbook site.yml --ask-vault-pass
```

১৪.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

```
# ── প্রজেক্ট কাঠামো ──
# /home/nahid/technova-ansible/
# ├── ansible.cfg
# ├── .gitignore
# ├── requirements.yml
# ├── site.yml
# ├── inventory/hosts.yml
# └── group_vars/
#     ├── all/
#     │   └── vars.yml
#     ├── development/
#     │   ├── vars.yml
#     │   └── vault.yml          # Vault ID: dev
#     ├── staging/
#     │   ├── vars.yml
#     │   └── vault.yml          # Vault ID: staging
#     └── production/
#         ├── vars.yml           # প্লেইনটেক্সট Variable
#         └── vault.yml          # Vault ID: prod (এনক্রিপ্ট করা)

# ── ansible.cfg ──
[defaults]
inventory = inventory/hosts.yml
roles_path = roles
# Vault পাসওয়ার্ড ফাইলের ডিফল্ট পাথ — প্রতিবার ফ্ল্যাগ লিখতে হয় না
vault_password_file = ~/.ansible/vault-pass.txt
# একাধিক Vault ID থাকলে:
# vault_identity_list = dev@~/.ansible/vault-pass-dev.txt, prod@~/.ansible/vault-pass-prod.txt

# ── .gitignore ──
vault-pass.txt
*.vault-pass
.vault-pass*
*.retry
__pycache__/

# ── group_vars/production/vars.yml ──
---
db_name: technova_production
db_user: technova_app
db_host: 203.0.113.12
app_domain: app.technova.example
```

```
# ── group_vars/production/vault.yml ── (এনক্রিপ্ট করা ফাইল)
# ভেতরে যা থাকে (ডিক্রিপ্ট করলে):
# ---
# db_password: "CHANGE-ME-DB-PASSWORD"
# api_secret_key: "CHANGE-ME-API-SECRET"
# jwt_secret: "CHANGE-ME-JWT-SECRET"
# ssl_private_key_passphrase: "CHANGE-ME-SSL-PASSPHRASE"

# ── site.yml — Vault Variable ব্যবহার ──
---
- name: Production ডেটাবেস প্রস্তুত করুন
  hosts: dbservers
  become: true
  tasks:
    - name: অ্যাপের ডেটাবেস ব্যবহারকারী তৈরি করুন
      community.postgresql.postgresql_user:
        name: "{{ db_user }}"
        password: "{{ db_password }}"
        db: "{{ db_name }}"
        priv: "ALL"
      become_user: postgres
      no_log: true

# ── Vault ID দিয়ে চালানো (তিনটি পরিবেশ) ──
# ansible-playbook site.yml \
#   --vault-id dev@~/.ansible/vault-pass-dev.txt \
#   --vault-id staging@~/.ansible/vault-pass-staging.txt \
#   --vault-id prod@~/.ansible/vault-pass-prod.txt
```

এই কাঠামোর সুবিধা: প্রতিটি পরিবেশের Secret আলাদা Vault ফাইলে, আলাদা পাসওয়ার্ডে — Development-এর পাসওয়ার্ড জানলেও Production-এর Secret খোলা যায় না। আর `vars.yml` প্লেইনটেক্সট থাকায় Code Review-এ সবাই দেখতে পান কোন Variable-গুলো Secret নয়।

১৪.১০ ধাপে ধাপে বাস্তবায়ন

1. **পাসওয়ার্ড ঠিক করুন:** একটি শক্ত Vault পাসওয়ার্ড বানান — অন্তত ২০ অক্ষর, বড়-ছোট হাতের অক্ষর, সংখ্যা ও চিহ্ন মিশিয়ে; পাসওয়ার্ড ম্যানেজারে সংরক্ষণ করুন।
2. **পাসওয়ার্ড ফাইল তৈরি করুন:** `echo 'CHANGE-ME-VAULT-PASSWORD' > ~/.ansible/vault-pass.txt && chmod 600 ~/.ansible/vault-pass.txt` চালান।
3. **.gitignore আপডেট করুন:** `vault-pass.txt` ও `*.vault-pass` যোগ করুন, এবং `git status` দিয়ে নিশ্চিত হোন ফাইলটি Git-এ যাচ্ছে না।
4. **Vault ফাইল তৈরি করুন:** `ansible-vault create group_vars/production/vault.yml --vault-password-file ~/.ansible/vault-pass.txt` চালিয়ে Secret-গুলো লিখুন।
5. **এনক্রিপশন যাচাই করুন:** `cat group_vars/production/vault.yml` দিয়ে দেখুন — প্লেইনটেক্সট পাসওয়ার্ড দেখা যাচ্ছে কি না; দেখা গেলে কিছু ভুল হয়েছে।
6. **Playbook-এ Variable ব্যবহার করুন:** `{{ db_password }}`-এর মতো করে Secret Variable ডাকুন, আর Task-এ `no_log: true` দিন।
7. **Playbook চালান:** `ansible-playbook site.yml --vault-password-file ~/.ansible/vault-pass.txt` — সফল হলে Secret ঠিকভাবে খুলছে।

8. **encrypt_string চেষ্টা করুন:** একটি Variable-এর মান `ansible-vault encrypt_string` দিয়ে এনক্রিপ্ট করে `group_vars/all/vars.yml`-এ বসান।
9. **Rotation অনুশীলন করুন:** `ansible-vault rekey group_vars/production/vault.yml` দিয়ে পাসওয়ার্ড বদলে দেখুন — পুরনো পাসওয়ার্ড দিয়ে আর খুলছে না কি না, তা যাচাই করুন।
10. **CI-এর জন্য প্রস্তুতি:** CI Pipeline-এ Vault পাসওয়ার্ড ফাইল হিসেবে না রেখে CI-এর Secret Store (যেমন GitHub Actions Secrets) থেকে পরিবেশ Variable-এ এনে ব্যবহারের পরিকল্পনা লিখুন।

১৪.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

```
$ ansible-vault create group_vars/production/vault.yml \
    --vault-password-file ~/.ansible/vault-pass.txt
# চিত্রণমূলক আউটপুট: (এডিটর খুলবে; সংরক্ষণ করে বেরিয়ে এলে কোনো বার্তা নয়)

$ cat group_vars/production/vault.yml
# চিত্রণমূলক আউটপুট:
# $ANSIBLE_VAULT;1.1;AES256
# 36383364386636613638396533306462313635346435383130386463346538 31
# 39616262376235323833336565336162623364616231363532656632386465 66

$ ansible-playbook site.yml --vault-password-file ~/.ansible/vault-pass.txt
# চিত্রণমূলক আউটপুট:
# PLAY [Production ডেটাবেস প্রস্তুত করুন] ****************************
#
# TASK [অ্যাপের ডেটাবেস ব্যবহারকারী তৈরি করুন] ***********************
# changed: [db01.technova.example]
#
# PLAY RECAP *********************************************************
# db01.technova.example : ok=2  changed=1  unreachable=0  failed=0

$ ansible-vault rekey group_vars/production/vault.yml
# চিত্রণমূলক আউটপুট:
# Vault password (old): ********
# New Vault password: ********
# Confirm New Vault password: ********
# Rekey successful
```

আউটপুট পড়ার নিয়ম: `cat` করলে শুধু এনক্রিপ্ট করা লেখা দেখা যাবে — প্লেইনটেক্সট পাসওয়ার্ড দেখা গেলে Vault কাজ করেনি। Playbook চালানোর আউটপুটে `no_log: true` থাকায় Secret-যুক্ত Task-এর বিস্তারিত দেখা যাবে না — শুধু `changed` বা `ok` দেখাবে, এটাই প্রত্যাশিত। ভুল পাসওয়ার্ড দিলে ত্রুটি আসবে — `ERROR! Decryption failed` — তখন পাসওয়ার্ড ফাইলের বিষয়বস্তু ও `chmod 600` Permission পরীক্ষা করুন। `rekey` সফল হলে পুরনো পাসওয়ার্ড দিয়ে আর ফাইল খুলবে না — নতুন পাসওয়ার্ডটি পাসওয়ার্ড ম্যানেজারে সংরক্ষণ করুন, তারপরেই পুরনোটি মুছুন।

১৪.১২ সাধারণ ভুল

- **Vault পাসওয়ার্ড ফাইল Git-এ Push করা:** সবচেয়ে ভয়াবহ ভুল — এনক্রিপশনের কোনো মূল্য থাকে না। `.gitignore`-এ রাখুন ও `git status` দিয়ে যাচাই করুন।
- **Secret সরাসরি Playbook-এ লেখা:** `password: "আসল-পাসওয়ার্ড"` — কখনো নয়; সবসময় Vault Variable ব্যবহার করুন।
- **Vault পাসওয়ার্ড দুর্বল রাখা:** `password123` বা কোম্পানির নাম দিয়ে Vault বানালে Brute Force-এ খুলে যাবে।

- **`decrypt` করে ফাইলটি প্লেইনটেক্সট রেখে দেওয়া:** `ansible-vault decrypt` করার পর ফাইলটি খোলা থাকে — কাজ শেষে আবার `encrypt` করুন, অথবা `edit` ব্যবহার করুন।
- **`no_log: true` না দেওয়া:** Secret-যুক্ত Task-এর আউটপুট লগে ছাপা হয়ে যায় — বিশেষ করে `debug` Task-এ।
- **একটাই Vault পাসওয়ার্ড সব পরিবেশে:** Development-এর পাসওয়ার্ড ফাঁস হলে Production-ও খুলে যায় — Vault ID দিয়ে আলাদা করুন।
- **Vault পাসওয়ার্ড শুধু একজনের কাছে রাখা:** সেই ব্যক্তি ছুটিতে গেলে বা চলে গেলে Secret আর খোলা যায় না — দলের পাসওয়ার্ড ম্যানেজারে সংরক্ষণ করুন।
- **Secret Rotation না করা:** একই পাসওয়ার্ড বছরের পর বছর ব্যবহার করা — ফাঁস হলে কেউ জানতেও পারবে না।

১৪.১৩ সমস্যা সমাধান (Troubleshooting)

> **REAL WORLD — TechNova:** TechNova-র Staging Deploy হঠাৎ ব্যর্থ হলো — `ERROR! Decryption failed (no vault secrets were found that could decrypt)`। তদন্তে দেখা গেল, একজন ইঞ্জিনিয়ার Staging-এর Vault ফাইলটি Production-এর পাসওয়ার্ড দিয়ে এনক্রিপ্ট করেছিলেন — ফাইলের লেবেল ছিল `staging`, কিন্তু চাবি ছিল Production-এর। সমাধান: ফাইলটি সঠিক পাসওয়ার্ড দিয়ে `rekey` করে Staging-এর পাসওয়ার্ডে আনা হলো, আর নিয়ম করা হলো — Vault ফাইল তৈরি করার সময় `--vault-id staging@...` লিখে লেবেলসহ তৈরি করতে হবে। শিক্ষা: Vault-এর ত্রুটি প্রায়ই ভুল পাসওয়ার্ডের কারণে হয় — প্রথমে লেবেল ও পাসওয়ার্ডের মিল দেখুন।

1. **লক্ষণ:** `Decryption failed` — **পরীক্ষা:** পাসওয়ার্ড ফাইলের বিষয়বস্তু (`cat ~/.ansible/vault-pass.txt`) ও ফাইলের প্রথম লাইনের লেবেল (`head -1 vault.yml`) মিলছে কি না। **সমাধান:** সঠিক পাসওয়ার্ড দিন; লেবেল থাকলে `--vault-id লেবেল@উৎস` ব্যবহার করুন।
2. **লক্ষণ:** `ERROR! input is not vault encrypted data` — **পরীক্ষা:** ফাইলটি আসলেই Vault ফাইল কি না (`head -1`-এ `$ANSIBLE_VAULT` আছে কি না)। **সমাধান:** প্লেইনটেক্সট ফাইল `edit` বা `view` দিয়ে খোলা যায় না — প্রথমে `encrypt` করুন।
3. **লক্ষণ:** Playbook-এ Variable `undefined` দেখাচ্ছে — **পরীক্ষা:** Vault ফাইলটি সঠিক `group_vars` ডিরেক্টরিতে আছে কি না, আর Playbook-টি Vault পাসওয়ার্ড দিয়ে চালানো হচ্ছে কি না। **সমাধান:** `ansible-inventory --graph` দিয়ে দেখুন কোন Group-এ হোস্টটি আছে।
4. **লক্ষণ:** `rekey`-এর পর ফাইল খুলছে না — **পরীক্ষা:** নতুন পাসওয়ার্ডটি ঠিকভাবে টাইপ হয়েছিল কি না; `rekey` করার সময় Caps Lock চালু ছিল কি না। **সমাধান:** পাসওয়ার্ড ম্যানেজার থেকে কপি করে `view` দিয়ে পরীক্ষা করুন; পুরনো পাসওয়ার্ডের ব্যাকআপ থাকলে তা দিয়ে `rekey` করে আবার চেষ্টা করুন।
5. **লক্ষণ:** CI-তে Vault পাসওয়ার্ড পাওয়া যাচ্ছে না — **পরীক্ষা:** CI-এর Secret Store-এ পাসওয়ার্ডটি আছে কি না, আর Pipeline-এ তা ফাইলে লেখা হচ্ছে কি না। **সমাধান:** CI-তে পাসওয়ার্ডটি পরিবেশ Variable থেকে একটি অস্থায়ী ফাইলে লিখে `--vault-password-file` দিয়ে ব্যবহার করুন, আর Pipeline শেষে ফাইলটি মুছে দিন।

১৪.১৪ নিরাপত্তা বিবেচনা

> **WARNING:** Vault ফাইল Git-এ রাখা নিরাপদ — কিন্তু শুধু তখনই, যখন পাসওয়ার্ড শক্তিশালী হয় এবং পাসওয়ার্ডটি Git-এ না থাকে। দুর্বল পাসওয়ার্ড দিয়ে এনক্রিপ্ট করা Vault ফাইল Git-এ Push করা মানে Secret-টি "পরে খোলা যাবে" অবস্থায় সবার সামনে রাখা — কারণ Git-এর ইতিহাস থেকে ফাইলটি কখনো মুছে যায় না। পাসওয়ার্ডটি অন্তত ২০ অক্ষরের করুন, আর নিয়মিত `rekey` করুন।

Vault-এর সীমা জানা জরুরি: Vault শুধু ফাইল এনক্রিপ্ট করে — এটি কোনো পূর্ণাঙ্গ Secret ম্যানেজমেন্ট সিস্টেম নয়। বড় দলে HashiCorp Vault, AWS Secrets Manager বা Azure Key Vault-এর মতো সিস্টেম ব্যবহার করা হয়, যেখানে Secret-এর অ্যাক্সেস লগ হয়, Rotation স্বয়ংক্রিয় হয়, আর প্রতিটি অ্যাপ শুধু নিজের Secret-টুকুই পায়। Ansible Vault সেই সিস্টেমগুলোর বিকল্প নয় — এটি ছোট দলের জন্য, Git-এ Secret রাখার নিরাপদ উপায়। আরও কয়েকটি নিয়ম: Vault পাসওয়ার্ড ফাইলের Permission সবসময় `600` রাখুন; পাসওয়ার্ডটি কখনো Chat, ইমেইল বা টিকিটে লিখবেন না; Secret-যুক্ত ফাইল `ansible-vault view` দিয়ে দেখুন, `decrypt` করে নয়; আর Git-এর ইতিহাসে কখনো প্লেইনটেক্সট Secret চলে গেলে শুধু ফাইল মুছলেই হবে না — Secret-টি বদলে ফেলতে হবে, কারণ ইতিহাসে তা থেকেই যাবে।

১৪.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

প্রোডাকশনে Secret পরিচালনার মানদণ্ড: **প্রতি পরিবেশে আলাদা Vault** — Development, Staging, Production-এর Secret আলাদা ফাইলে, আলাদা পাসওয়ার্ডে, Vault ID দিয়ে চিহ্নিত; **Secret Rotation-এর সময়সূচি** — ডেটাবেস পাসওয়ার্ড প্রতি ৯০ দিনে, API Key প্রতি ৬০ দিনে বদলানো; Rotation-এর ধাপ: নতুন Secret তৈরি → Vault-এ লেখা → Playbook চালিয়ে সার্ভারে বসানো → পুরনো Secret বাতিল; **Rotation-এর সময় Downtime এড়ানো** — নতুন পাসওয়ার্ড বসানোর আগে পুরনোটি কিছুক্ষণ বৈধ রাখুন (যদি সিস্টেম তা সমর্থন করে), যাতে Deploy-এর মাঝখানে অ্যাপ না ভাঙে; **পাসওয়ার্ড ম্যানেজার** — Vault পাসওয়ার্ড দলের পাসওয়ার্ড ম্যানেজারে, অন্তত দুজন বিশ্বস্ত সদস্যের অ্যাক্সেসসহ; **Git Pre-commit Hook** — `git-secrets` বা `detect-secrets` দিয়ে Commit-এর আগে পরীক্ষা করা, যাতে প্লেইনটেক্সট Secret ভুলেও Commit না হয়; **অডিট** — কে, কখন, কোন Vault ফাইল বদলেছে, তা Git-এর ইতিহাসে দেখা যায় — নিয়মিত পর্যালোচনা করুন; **বড় হলে Vault ছাড়ুন** — দল বড় হলে HashiCorp Vault বা ক্লাউড Secret Manager-এ যান, Ansible Vault শুধু ছোট দলের জন্য।

১৪.১৬ অধ্যায়ের সারাংশ

Ansible Vault হলো Secret এনক্রিপ্ট করে রাখার ব্যবস্থা — AES-256 দিয়ে ফাইল বা একক স্ট্রিং এনক্রিপ্ট করা যায়, আর Playbook চলার সময় পাসওয়ার্ড দিলে তা মেমরিতে খুলে ব্যবহার করা যায়। `create`, `edit`, `encrypt`, `decrypt`, `view`, `rekey` ও `encrypt_string` — এই সাতটি কমান্ডেই Vault-এর পুরো কাজ হয়। Vault পাসওয়ার্ড ফাইলে (`chmod 600`) রাখা যায়, তবে ফাইলটি কখনো Git-এ যাবে না; Vault ID দিয়ে প্রতিটি পরিবেশের আলাদা Secret পরিচালনা করা যায়। এনক্রিপ্ট করা Vault ফাইল Git-এ রাখা নিরাপদ — তবে শুধু শক্তিশালী পাসওয়ার্ড ও আলাদা পাসওয়ার্ড-ব্যবস্থাপনা থাকলে। সংবেদনশীল Task-এ `no_log: true` দিতে হয়, আর Secret নিয়মিত Rotation করতে হয়। Vault ছোট দলের জন্য যথেষ্ট; বড় দলে পূর্ণাঙ্গ Secret Manager ব্যবহার করা উচিত।

১৪.১৭ জ্ঞান যাচাই — প্রশ্ন

1. Vault-এনক্রিপ্ট করা ফাইল Git-এ রাখা কেন নিরাপদ, আর Vault পাসওয়ার্ড ফাইল Git-এ রাখা কেন বিপজ্জনক? দুটির পার্থক্য ব্যাখ্যা করুন।

2. `ansible-vault edit` ও `ansible-vault decrypt`-এর মধ্যে পার্থক্য কী? Secret বদলাতে কোনটি ব্যবহার করবেন এবং কেন?
3. Vault ID কী? Development ও Production-এর Secret আলাদা রাখতে Vault ID কীভাবে সাহায্য করে?
4. `encrypt_string` কখন ব্যবহার করবেন, আর পুরো ফাইল এনক্রিপ্ট করা কখন ভালো? দুটির সুবিধা-অসুবিধা লিখুন।
5. Secret Rotation বলতে কী বোঝায়? একটি ডেটাবেস পাসওয়ার্ড Rotation-এর ধাপগুলো লিখুন — যাতে Deploy-এর সময় অ্যাপ না ভাঙে।

১৪.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. **Vault ফাইল বানান:** আপনার TechNova প্রজেক্টে `group_vars/production/vault.yml` তৈরি করুন — তাতে ডেটাবেস পাসওয়ার্ড, API Secret ও JWT Secret রাখুন (সবগুলো `CHANGE-ME-...` প্লেসহোল্ডার দিয়ে)।
2. **encrypt_string:** একটি API Key `encrypt_string` দিয়ে এনক্রিপ্ট করে `group_vars/all/vars.yml`-এ বসান, এবং Playbook চালিয়ে দেখুন মানটি ঠিকভাবে ব্যবহৃত হচ্ছে কি না।
3. **Vault ID:** Development ও Production-এর জন্য আলাদা Vault পাসওয়ার্ড ও আলাদা Vault ফাইল বানান, এবং `--vault-id` দিয়ে দুটি পরিবেশেই Playbook চালান।
4. **ফাঁস পরীক্ষা:** আপনার Repository-তে `git log --all -p | grep -i password` চালিয়ে দেখুন ইতিহাসে কোথাও প্লেইনটেক্সট Secret আছে কি না; থাকলে তা Rotation করার পরিকল্পনা লিখুন।

অধ্যায় 15 • PART III — Roles, Collections, Vault & Cloud

Chapter 15 — SSH, Privilege Escalation, and Secure Remote Execution

১৫.১ ভূমিকা

Ansible-এর সবচেয়ে সুন্দর বৈশিষ্ট্য হলো — Managed Node-এ কোনো Agent ইনস্টল করতে হয় না। কিন্তু তাহলে Ansible সার্ভারে পৌঁছায় কীভাবে? উত্তর: **SSH** (Secure Shell) — যে প্রোটোকল দিয়ে আপনি টার্মিনালে সার্ভারে ঢোকেন, Ansible সেই একই পথে ঢুকে, কিন্তু মানুষের বদলে স্বয়ংক্রিয়ভাবে। এই সুবিধার একটি মূল্য আছে: SSH-ই যদি দুর্বল হয়, তবে পুরো অটোমেশন ব্যবস্থা দুর্বল। দুর্বল পাসওয়ার্ড, পাসওয়ার্ড ছাড়া Private Key, Host Key যাচাই বন্ধ করে দেওয়া, আর সব কাজে `root` ব্যবহার — এই চারটি ভুলই TechNova-র মতো দলকে বারবার বিপদে ফেলেছে। এই অধ্যায়ে আপনি শিখবেন Ansible-এ SSH কীভাবে কাজ করে, `ControlPersist` কেন গতি বাড়ায়, `become` দিয়ে নিরাপদে অধিকার (Privilege) বাড়ানো যায় কীভাবে, SSH Key কীভাবে সুরক্ষিত রাখতে হয়, আর সবচেয়ে গুরুত্বপূর্ণ — কেন `StrictHostKeyChecking=no` প্রোডাকশনে কখনো লেখা যাবে না।

১৫.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Ansible-এ SSH-এর ভূমিকা ও সংযোগের ধাপগুলো ব্যাখ্যা করতে; (২) `ssh_args` ও `ControlPersist` কনফিগার করে Playbook-এর গতি বাড়াতে; (৩) `become`, `become_user` ও `become_method` (`sudo` / `su`) দিয়ে Least Privilege নীতি অনুযায়ী অধিকার বাড়াতে; (৪) ed25519 Key তৈরি, Passphrase দেওয়া ও `ssh-agent` ব্যবহার করে Key সুরক্ষিত রাখতে; (৫) Host Key Verification-এর কাজ ও Man-in-the-Middle (MITM) আক্রমণের ঝুঁকি ব্যাখ্যা করতে; (৬) `shell` / `command` Module-এর বিপজ্জনক ব্যবহার চিহ্নিত করে নিরাপদ বিকল্প বেছে নিতে; (৭) `ansible_user` ও `ansible_password` Inventory-তে লেখার ঝুঁকি ব্যাখ্যা করতে।

১৫.৩ পূর্বশর্ত জ্ঞান

SSH দিয়ে সার্ভারে ঢোকার অভিজ্ঞতা (`ssh user@host`), Public/Private Key-এর প্রাথমিক ধারণা, Linux-এ `sudo` ব্যবহারের অভিজ্ঞতা, Ansible Playbook ও Inventory লেখা, এবং ফাইল Permission (`chmod`)-এর ধারণা প্রয়োজন। ক্রিপ্টোগ্রাফির গভীর জ্ঞান দরকার নেই — শুধু এইটুকু জানলেই চলবে: Public Key সবার সামনে রাখা যায়, Private Key শুধু মালিকের কাছে থাকে।

১৫.৪ প্রথম নীতি থেকে ব্যাখ্যা

দূরবর্তী (Remote) কাজের প্রথম নীতি হলো **পরিচয় যাচাই, দুই দিক থেকে** (Mutual Authentication)। আপনি যখন সার্ভারে SSH করেন, দুটি প্রশ্নের উত্তর দরকার: সার্ভারটি কি সত্যিই সেই সার্ভার, যাকে আপনি ভাবছেন? আর আপনি কি সত্যিই সেই ব্যবহারকারী, যাকে আপনি দাবি করছেন? প্রথম প্রশ্নের উত্তর দেয় **Host Key** — সার্ভারের নিজের পরিচয়পত্র, যা প্রথমবার সংযোগের সময় আপনার `~/.ssh/known_hosts` ফাইলে সংরক্ষিত হয়; পরের বার সার্ভারের Key বদলে গেলে SSH সতর্ক করে দেয় — কারণ মাঝখানে কেউ বসে থাকতে পারে (Man-in-the-Middle)। দ্বিতীয় প্রশ্নের উত্তর দেয় **Key Authentication** — আপনার Private Key দিয়ে একটি চ্যালেঞ্জে স্বাক্ষর করা হয়, সার্ভার আপনার Public Key দিয়ে তা যাচাই করে; পাসওয়ার্ড পাঠানো হয় না, তাই পাসওয়ার্ড চুরি হওয়ার ঝুঁকি নেই। দ্বিতীয় নীতি হলো **সর্বনিম্ন অধিকার** (Least Privilege): Ansible সাধারণ ব্যবহারকারী (`nahid`) হিসেবে ঢুকবে, আর শুধু যে Task-গুলোতে `root` দরকার, সেগুলোতেই `sudo` দিয়ে অধিকার বাড়াবে — সবসময় `root` হিসেবে ঢোকা মানে একটি ভুলেই পুরো সিস্টেম ধ্বংসের

সুযোগ দেওয়া। তৃতীয় নীতি হলো **কমান্ডের সীমা**: Ansible-এর নির্দিষ্ট Module (`apt`, `file`, `template`) ব্যবহার করা নিরাপদ, কারণ তারা Idempotent ও পরীক্ষিত; `shell` দিয়ে যেকোনো কমান্ড চালানো মানে সেই সুরক্ষা নিজে হাতে ভেঙে দেওয়া।

> **CONCEPT:** SSH Key Authentication-কে ভাবুন তালা-চাবির মতো, তবে উল্টো: তালাটি (Public Key) সার্ভারে থাকে, সবার দেখার জন্য; চাবিটি (Private Key) শুধু আপনার কাছে থাকে। সার্ভার আপনাকে একটি ধাঁধা (Challenge) দেয় — চাবি ছাড়া তার উত্তর দেওয়া যায় না। পাসওয়ার্ড Authentication-এর উল্টো: পাসওয়ার্ডটি সার্ভারে পাঠাতে হয়, আর পথে কেউ তা শুনে ফেলতে পারে। তাই Key Authentication পাসওয়ার্ডের চেয়ে শক্তিশালী — যদি Private Key সুরক্ষিত থাকে। Private Key-টি পাসওয়ার্ড ছাড়া, সবার পড়ার মতো Permission-এ রাখলে সেই শক্তি শূন্য হয়ে যায়।

১৫.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Ansible-এ SSH কীভাবে কাজ করে

Ansible ডিফল্টভাবে `ssh` Connection Plugin ব্যবহার করে — এটি OpenSSH ক্লায়েন্টের ওপর ভিত্তি করে। একটি Task চালানোর সময় Ansible যা করে: প্রথমে Control Node-এ Module-এর Python কোড তৈরি করে, তারপর SSH দিয়ে Managed Node-এ সংযোগ করে, কোডটি `/tmp`-এর নিচে একটি অস্থায়ী ফাইলে পাঠায়, সেখানে Python দিয়ে চালায়, ফলাফল (JSON) ফিরিয়ে আনে, আর অস্থায়ী ফাইলটি মুছে দেয়। এই প্রক্রিয়ায় Managed Node-এ শুধু Python ও SSH দরকার — কোনো Agent নয়। সংযোগের সেটিংস `ansible.cfg`-এর `[ssh_connection]` অংশে ও Inventory Variable-এ ঠিক করা যায়:

```
# ansible.cfg — SSH সংযোগের সেটিংস
[ssh_connection]
# ControlPersist — একটি সংযোগ বারবার ব্যবহার করা (Multiplexing)
ssh_args = -o ControlMaster=auto -o ControlPersist=60s -o ControlPath=~/.ansible/cp/%r@%h-%p
# ControlPersist=60s মানে: একটি সংযোগ ৬০ সেকেন্ড খোলা থাকবে,
# তার মধ্যে অন্য Task-গুলো নতুন করে সংযোগ না করে একই পথ ব্যবহার করবে।
pipelining = True
# Pipelining চালু হলে Module-এর কোড SSH-এর ভেতর দিয়েই পাঠানো হয়,
# আলাদা করে SFTP/SCP করতে হয় না — গতি বাড়ে, তবে sudo-তে requiretty থাকলে সমস্যা হয়।

# Inventory-তে সংযোগের Variable
# ansible_host: 203.0.113.10
# ansible_user: nahid
# ansible_port: 22
# ansible_ssh_private_key_file: ~/.ssh/id_ed25519
# ansible_ssh_common_args: '-o StrictHostKeyChecking=yes'
```

`ControlPersist` বড় Playbook-এ বিশাল পার্থক্য তৈরি করে: শত শত Task-এর প্রতিটিতে নতুন SSH সংযোগ করতে কয়েক সেকেন্ড লাগে, আর Multiplexing-এ একবার সংযোগ করলেই বাকি Task-গুলো সেই পথে চলে যায়। `ControlPath`-এর ডিরেক্টরি (`~/.ansible/cp/`) আগে থেকে তৈরি থাকতে হয়, নইলে Multiplexing কাজ করবে না।

Privilege Escalation — become

সেটিং	কাজ	উদাহরণ

`become: true`	অধিকার বাড়ানো চালু করে	Task বা Play-এ লিখুন
`become_user: root`	কার অধিকার নেবেন তা ঠিক করে (ডিফল্ট: `root`)	`become_user: postgres` — ডেটাবেসের কাজে
`become_method: sudo`	কীভাবে অধিকার বাড়াবেন (ডিফল্ট: `sudo`)	`su`, `sudo`, `doas`
`ansible_become_password`	`sudo`-এর পাসওয়ার্ড (Vault-এ রাখতে হয়!)	Inventory Variable হিসেবে — কখনো প্লেইনটেক্সট নয়

সেরা অভ্যাস: Play-এর শুরুতে `become: true` না লিখে শুধু যে Task-গুলোতে দরকার, সেগুলোতে লিখুন — যেমন প্যাকেজ ইনস্টল বা সিস্টেম ফাইল লেখায়। অ্যাপের কোড Git থেকে টানা বা `npm install` চালানোর মতো কাজে `root` দরকার নেই — সেখানে `become_user: technova` ব্যবহার করুন। `sudo`-এর জন্য Managed Node-এ `/etc/sudoers.d/nahid` ফাইলে লেখা থাকে: `nahid ALL=(ALL) NOPASSWD: ALL` — তবে প্রোডাকশনে `NOPASSWD`-এর বদলে পাসওয়ার্ড-সহ `sudo` ও Vault-এ রাখা `ansible_become_password` ব্যবহার করাই নিরাপদ।

SSH Key নিরাপত্তা

```
# ed25519 Key তৈরি করুন (RSA-এর চেয়ে ছোট, দ্রুত ও নিরাপদ)
ssh-keygen -t ed25519 -C "nahid@technova.example" -f ~/.ssh/id_ed25519
# Passphrase দিতে বলবে — অবশ্যই দিন। Passphrase ছাড়া Key মানে:
# Key ফাইলটি চুরি গেলেই যে কেউ আপনার সার্ভারে ঢুকতে পারবে।

# Key-এর Permission ঠিক করুন
chmod 600 ~/.ssh/id_ed25519        # Private Key — শুধু মালিক পড়তে পারবেন
chmod 644 ~/.ssh/id_ed25519.pub    # Public Key — সবাই পড়তে পারেন
chmod 700 ~/.ssh                   # .ssh ডিরেক্টরি — শুধু মালিক ঢুকতে পারবেন

# ssh-agent — Passphrase একবার দিয়ে Key মেমরিতে রাখা
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
# এরপর প্রতিবার Passphrase লিখতে হয় না — Agent মেমরিতে Key ধরে রাখে।
# তবে Agent-এর মেমরিতে থাকা Key ব্যবহার করতে পারবে সেই মেশিনে ঢোকা যে কেউ —
# তাই শেয়ার করা মেশিনে ssh-agent ব্যবহার করবেন না, আর কাজ শেষে ssh-add -D দিয়ে মুছুন।

# Public Key সার্ভারে বসানো
ssh-copy-id -i ~/.ssh/id_ed25519.pub nahid@203.0.113.10
# এটি Public Key-টি সার্ভারের ~/.ssh/authorized_keys ফাইলে যোগ করে।
```

Host Key Verification — কখনো বন্ধ করবেন না

> **WARNING — StrictHostKeyChecking বন্ধ করা প্রোডাকশনে নিষিদ্ধ:** অনেক টিউটোরিয়ালে প্রথমবারের সংযোগের ঝামেলা এড়াতে `StrictHostKeyChecking=no` লিখতে বলা হয়। এটি একটি বিপজ্জনক অভ্যাস। Host Key যাচাই বন্ধ করলে Ansible যেকোনো সার্ভারকে — এমনকি আক্রমণকারীর সাজানো ভুয়া সার্ভারকেও — আসল সার্ভার বলে মেনে নেবে, আর আপনার Secret, পাসওয়ার্ড ও কোড সেই ভুয়া সার্ভারে চলে যাবে। এটাই **Man-in-the-Middle (MITM)** আক্রমণ: মাঝখানে বসে থাকা আক্রমণকারী নিজেকে আসল সার্ভার বলে পরিচয় দেয়, আপনার সব তথ্য পড়ে, তারপর আসল সার্ভারে পাঠিয়ে দেয় — আপনি কিছুই টের পান না। প্রোডাকশনে সবসময় `StrictHostKeyChecking=yes` (ডিফল্ট) রাখুন।

তাহলে নতুন সার্ভার প্রথমবার কীভাবে যোগ করবেন? সঠিক পদ্ধতি তিনটি: প্রথম, হাতে একবার SSH করে Host Key `known_hosts`-এ সংরক্ষণ করুন (`ssh nahid@203.0.113.10` চালিয়ে `yes` লিখুন — তবে Fingerprint-টি সার্ভারের কনসোল বা বিশ্বস্ত উৎস থেকে মিলিয়ে নিন)। দ্বিতীয়, `ssh-keyscan` দিয়ে বিশ্বস্ত নেটওয়ার্ক থেকে Key সংগ্রহ করুন: `ssh-keyscan -t ed25519 203.0.113.10 >> ~/.ssh/known_hosts`। তৃতীয়, Ansible-এর `known_hosts` Module দিয়ে Playbook-এই Host Key যোগ করুন — তবে Key-টি আগে থেকেই জানা থাকতে হবে। `host_key_checking = False` শুধু অস্থায়ী ল্যাবে, বিচ্ছিন্ন নেটওয়ার্কে, পরীক্ষার জন্য ব্যবহার করা যায় — আর তাও `ansible.cfg`-এ স্থায়ীভাবে লেখা যাবে না।

বিপজ্জনক shell/command ব্যবহারের সীমা

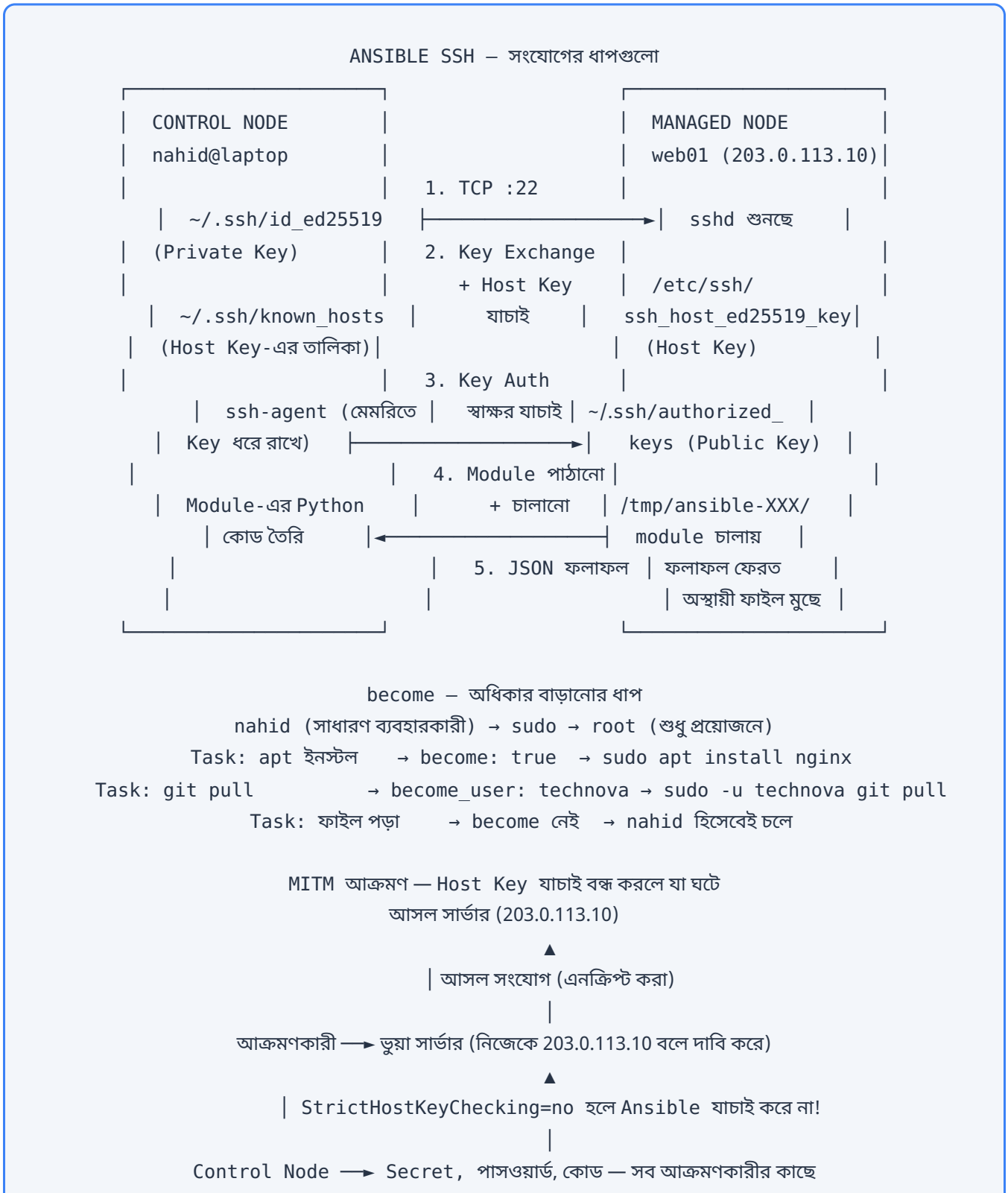
`shell` ও `command` Module যেকোনো কমান্ড চালাতে পারে — এটাই তাদের বিপদ। `shell: rm -rf /opt/old-app` লেখা Task-এ একটি Variable ভুল হলেই পুরো ফাইলসিস্টেম মুছে যেতে পারে। নিয়মগুলো: প্রথম, নির্দিষ্ট Module থাকলে সেটিই ব্যবহার করুন — ফাইল মুছতে `file` Module, প্যাকেজ বসাতে `apt`, সার্ভিস চালাতে `service`। দ্বিতীয়, `shell` ব্যবহার করলে `creates` বা `removes` আর্গুমেন্ট দিন — যাতে Task Idempotent হয় (ফাইলটি থাকলে আর না চলে)। তৃতীয়, ব্যবহারকারীর দেওয়া মান কখনো `shell` কমান্ডে সরাসরি বসাবেন না — Shell Injection আক্রমণে তা দিয়ে যেকোনো কমান্ড চালানো যায়। চতুর্থ, `shell` Task-এ `executable: /bin/bash` নির্দিষ্ট করে দিন, যাতে Shell-এর আচরণ পরিবেশভেদে না বদলায়।

১৫.৬ অভ্যন্তরীণ কার্যপ্রণালী

Ansible-এর SSH সংযোগের ভেতরের ধাপগুলো এই ক্রমে ঘটে। প্রথমে Control Node-এর OpenSSH ক্লায়েন্ট Managed Node-এর Port 22-এ TCP সংযোগ করে। তারপর **Key Exchange** হয় — দুই পক্ষ একটি গোপন Session Key-তে সম্মত হয়, যা দিয়ে পুরো সংযোগ এনক্রিপ্ট করা হয়; এই ধাপে সার্ভার তার Host Key দিয়ে নিজের পরিচয় প্রমাণ করে, আর ক্লায়েন্ট `known_hosts`-এর সঙ্গে তা মিলিয়ে দেখে। এরপর **Authentication** হয় — ক্লায়েন্ট তার Private Key দিয়ে একটি চ্যালেঞ্জে স্বাক্ষর করে, সার্ভার `authorized_keys`-এর Public Key দিয়ে তা যাচাই করে; সফল হলে একটি এনক্রিপ্ট করা Channel তৈরি হয়। তারপর Ansible Module-এর কোড SFTP বা Pipelining দিয়ে পাঠিয়ে চালায়। `become: true` থাকলে Ansible Managed Node-এ `sudo` কমান্ডের ভেতরে Module-টি চালায় — `sudo` তখন `/etc/sudoers` দেখে এই ব্যবহারকারী `root` হতে পারে কি না, তা যাচাই করে; পাসওয়ার্ড লাগলে Ansible `ansible_become_password` (Vault থেকে আসা) `sudo`-এর Prompt-এ পাঠায়। `ControlPersist` চালু থাকলে প্রথম সংযোগের পরে একটি Unix Socket (`ControlPath`-এ) তৈরি হয় — পরের Task-গুলো নতুন TCP সংযোগ না করে সেই Socket দিয়েই Channel খোলে, যাতে Key Exchange ও Authentication-এর খরচ আর দিতে হয় না। এই Socket ফাইলটি Control Node-এর ডিস্কে থাকে — তাই তার Permission ও অবস্থান নিরাপদ রাখতে হয়; শেয়ার করা ডিরেক্টরিতে `ControlPath` রাখবেন না।

> **TIP:** SSH সংযোগের সমস্যা নির্ণয়ে `-vvv` ফ্ল্যাগ ব্যবহার করুন: `ansible-playbook site.yml -vvv` চালালে SSH-এর বিস্তারিত ডিবাগ আউটপুট দেখা যায় — কোন Key চেষ্টা করা হচ্ছে, Host Key মিলছে কি না, `sudo` কোথায় আটকাচ্ছে — সব দেখা যায়। তবে `-vvv` আউটপুটে Secret থাকতে পারে — তাই তা কারও সঙ্গে শেয়ার করার আগে পরীক্ষা করে নিন।

১৫.৭ আর্কিটেকচার চিত্র



চিত্রের মূল কথা: Host Key যাচাই হলো সার্ভারের পরিচয়পত্র দেখা — তা বন্ধ করা মানে অপরিচিত কাউকে চোখ বন্ধ করে বিশ্বাস করা।

১৫.৮ ব্যবহারিক উদাহরণ

TechNova-র নিরাপদ SSH সেটআপ — শুরু থেকে শেষ পর্যন্ত:

```
# ── 1. Control Node-এ Key তৈরি ──
ssh-keygen -t ed25519 -C "nahid@technova.example" -f ~/.ssh/id_ed25519
# Passphrase দিন — যেমন: CHANGE-ME-SSH-PASSPHRASE (পাসওয়ার্ড ম্যানেজারে রাখুন)

# ── 2. Managed Node-এ Public Key বসানো ──
ssh-copy-id -i ~/.ssh/id_ed25519.pub nahid@203.0.113.10
# প্রথমবার Host Key-এর Fingerprint দেখাবে — সার্ভারের কনসোল থেকে মিলিয়ে নিন, তারপর yes লিখুন।

# ── 3. পাসওয়ার্ড Authentication বন্ধ করা (Managed Node-এ) ──
# /etc/ssh/sshd_config-এ:
#   PasswordAuthentication no
#   PubkeyAuthentication yes
#   PermitRootLogin no
# তারপর: sudo systemctl restart sshd
# ব্যাখ্যা: পাসওয়ার্ড দিয়ে ঢোকা বন্ধ হলে Brute Force আক্রমণ প্রায় অসম্ভব হয়;
# আর root-এ সরাসরি SSH বন্ধ হলে আক্রমণকারীকে প্রথমে nahid হয়ে, তারপর sudo করতে হয়।

# ── 4. sudoers ফাইল — nahid-কে সীমিত sudo দেওয়া ──
# /etc/sudoers.d/nahid (visudo দিয়ে সম্পাদনা করুন):
nahid ALL=(ALL) ALL
# ব্যাখ্যা: nahid sudo করতে পারবেন, তবে পাসওয়ার্ড লাগবে।
# NOPASSWD লিখবেন না — পাসওয়ার্ড ছাড়া sudo মানে Key চুরি গেলেই root।

# ── 5. Ansible দিয়ে সংযোগ পরীক্ষা ──
ansible webservers -m ping
# চিত্রণমূলক আউটপুট:
# web01.technova.example | SUCCESS => {"ping": "pong"}
```

Playbook-এ `become`-এর সঠিক ব্যবহার:

```
---
- name: Web সার্ভার প্রস্তুত করুন
  hosts: webservers
  # Play-এ become নেই — শুধু প্রয়োজনে Task-এ দেওয়া হবে
  tasks:
    - name: Nginx ইনস্টল করুন (root দরকার)
      ansible.builtin.apt:
        name: nginx
        state: present
      become: true

    - name: অ্যাপের কোড টানুন (technova ব্যবহারকারী হিসেবে)
      ansible.builtin.git:
        repo: https://github.com/technova/api.git
        dest: /opt/technova-api
        version: main
      become: true
      become_user: technova

    - name: অ্যাপের লগ দেখুন (সাধারণ ব্যবহারকারী হিসেবেই)
      ansible.builtin.command: tail -n 20 /opt/technova-api/app.log
      register: app_log
```

```
      changed_when: false
      # ব্যাখ্যা: শুধু পড়ার কাজে become লাগে না — Least Privilege।
```

১৫.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

```
# ── ansible.cfg — নিরাপদ SSH সেটিংস ──
[defaults]
inventory = inventory/hosts.yml
roles_path = roles
# Host Key যাচাই সবসময় চালু (ডিফল্ট True — কখনো False করবেন না)
host_key_checking = True
# অস্থায়ী ফাইল Control Node-এ কোথায় থাকবে
remote_tmp = /tmp/.ansible/tmp

[ssh_connection]
# ControlPersist — সংযোগ পুনঃব্যবহার (গতি)
ssh_args = -o ControlMaster=auto -o ControlPersist=60s -o ControlPath=~/.ansible/cp/%r@%h-%p
# Pipelining — Module SSH-এর ভেতর দিয়ে পাঠানো
pipelining = True
# সংযোগের সময়সীমা (সেকেন্ড)
timeout = 30

[privilege_escalation]
# ডিফল্ট become সেটিংস — Playbook-এ না লিখলেও চলবে, তবে Task-এ লেখাই ভালো
become = False
become_method = sudo
become_user = root
become_ask_pass = False

# ── inventory/hosts.yml — সংযোগের Variable (Secret নয়) ──
all:
  vars:
    ansible_user: nahid
    ansible_port: 22
    ansible_ssh_private_key_file: ~/.ssh/id_ed25519
    # কখনো এখানে ansible_password বা ansible_become_password লিখবেন না!
    # সেগুলো Vault-এ থাকে (group_vars/all/vault.yml)।
  children:
    webservers:
      hosts:
        web01.technova.example:
          ansible_host: 203.0.113.10
    appservers:
      hosts:
        app01.technova.example:
          ansible_host: 203.0.113.11

# ── group_vars/all/vault.yml ── (এনক্রিপ্ট করা)
# ansible_become_password: "CHANGE-ME-SUDO-PASSWORD"

# ── Managed Node-এ sshd_config-এর নিরাপদ অংশ ──
# /etc/ssh/sshd_config
# Port 22
# PasswordAuthentication no
# PubkeyAuthentication yes
# PermitRootLogin no
```

```
# MaxAuthTries 3
# LoginGraceTime 30
# AllowUsers nahid technova
# ব্যাখ্যা: AllowUsers দিয়ে শুধু নির্দিষ্ট ব্যবহারকারীদেরই SSH করতে দেওয়া হয়।
```

এই কনফিগারেশনের প্রতিটি লাইনের একটি নিরাপত্তা-কারণ আছে: `host_key_checking = True` MITM ঠেকায়, `PasswordAuthentication no` Brute Force ঠেকায়, `PermitRootLogin no` সরাসরি root আক্রমণ ঠেকায়, আর `AllowUsers` আক্রমণের পৃষ্ঠ (Attack Surface) ছোট করে।

১৫.১০ ধাপে ধাপে বাস্তবায়ন

1. **Key তৈরি করুন:** `ssh-keygen -t ed25519 -C "nahid@technova.example"` চালান; শক্ত Passphrase দিন।
2. **Permission ঠিক করুন:** `chmod 700 ~/.ssh`, `chmod 600 ~/.ssh/id_ed25519` চালান।
3. **Public Key সার্ভারে বসান:** `ssh-copy-id -i ~/.ssh/id_ed25519.pub nahid@203.0.113.10` — Fingerprint মিলিয়ে নিয়ে `yes` লিখুন।
4. **Key দিয়ে ঢোকা পরীক্ষা করুন:** `ssh nahid@203.0.113.10` — পাসওয়ার্ড না চেয়ে Passphrase চাইলে Key কাজ করছে।
5. **ssh-agent চালু করুন:** `eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519` — Passphrase একবার দিন।
6. **Ansible দিয়ে পরীক্ষা করুন:** `ansible webservers -m ping` — `SUCCESS` আসা উচিত।
7. **সার্ভারের SSH শক্ত করুন:** `sshd_config`-এ `PasswordAuthentication no` ও `PermitRootLogin no` লিখে `sudo systemctl restart sshd` চালান — তবে একটি টার্মিনাল খোলা রেখে পরীক্ষা করুন, যাতে ভুল হলে আটকে না যান।
8. **`ansible.cfg` লিখুন:** উপরের উদাহরণ অনুযায়ী `ssh_args`, `ControlPersist` ও `host_key_checking = True` লিখুন।
9. **ControlPath ডিরেক্টরি বানান:** `mkdir -p ~/.ansible/cp` — নইলে Multiplexing কাজ করবে না।
10. **গতি মাপুন:** `time ansible-playbook site.yml` চালান — ControlPersist চালু করার আগে ও পরে সময়ের পার্থক্য দেখুন।
11. **become পরীক্ষা করুন:** একটি Task-এ `become: true` দিয়ে `apt` চালান, আর একটি Task-এ `become_user: technova` দিয়ে `git` চালান — দুটিই সফল হওয়া উচিত।

১৫.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

```
$ ansible webservers -m ping
# চিত্রণমূলক আউটপুট:
# web01.technova.example | SUCCESS => {
#     "ansible_facts": {
#         "discovered_interpreter_python": "/usr/bin/python3"
#     },
#     "changed": false,
#     "ping": "pong"
# }

$ ansible-playbook site.yml -vvv
# চিত্রণমূলক আউটপুট (অংশবিশেষ):
```

```
# <203.0.113.10> ESTABLISH SSH CONNECTION FOR USER: nahid
# <203.0.113.10> SSH: EXEC ssh -C -o ControlMaster=auto
#   -o ControlPersist=60s -o ControlPath=~/.ansible/cp/nahid@203.0.113.10-22
#   -o StrictHostKeyChecking=yes -o User=nahid ...
# <203.0.113.10> (0, b'{"changed": true, ...}', b'')

$ ssh -v nahid@203.0.113.10
# চিত্রণমূলক আউটপুট (অংশবিশেষ):
# debug1: Offering public key: /home/nahid/.ssh/id_ed25519 ED25519 SHA256:...
# debug1: Server accepts key: /home/nahid/.ssh/id_ed25519 ED25519 SHA256:...
# debug1: Authentication succeeded (publickey).
```

আউটপুট পড়ার নিয়ম: `ping` মডিউলের `SUCCESS` মানে SSH সংযোগ, Authentication ও Python — তিনটিই ঠিক আছে। `-vvv` আউটপুটে `StrictHostKeyChecking=yes` দেখা যাবে — এটাই প্রমাণ যে Host Key যাচাই চালু আছে। `ssh -v`-এর আউটপুটে `Authentication succeeded (publickey)` মানে Key Authentication কাজ করছে — `password` লেখা থাকলে বুঝবেন পাসওয়ার্ড দিয়ে ঢোকা হচ্ছে, যা বন্ধ করা উচিত। যদি `Permission denied (publickey)` আসে, তবে Public Key সার্ভারের `authorized_keys`-এ আছে কি না, আর `~/.ssh`-এর Permission ঠিক আছে কি না, তা দেখুন।

১৫.১২ সাধারণ ভুল

- **`StrictHostKeyChecking=no` লেখা:** প্রোডাকশনে সবচেয়ে বিপজ্জনক ভুল — MITM আক্রমণের দরজা খুলে দেওয়া।
- **Passphrase ছাড়া Private Key:** Key ফাইল চুরি গেলেই সার্ভারে ঢোকা যায় — সবসময় Passphrase দিন।
- **Private Key-এর Permission `644` রাখা:** SSH তখন Key ব্যবহার করতেই অস্বীকার করবে — `chmod 600` বাধ্যতামূলক।
- **সব কাজে `root` হিসেবে SSH করা:** `PermitRootLogin yes` ও `ansible_user: root` — দুটিই এড়িয়ে চলুন।
- **`ansible_password` Inventory-তে প্লেইনটেক্সট লেখা:** Git-এ গেলে সবাই দেখতে পাবে — Key Authentication ব্যবহার করুন, পাসওয়ার্ড নয়।
- **`NOPASSWD: ALL` দিয়ে sudo দেওয়া:** Key চুরি গেলেই root — পাসওয়ার্ড-সহ sudo ব্যবহার করুন।
- **`shell` দিয়ে সবকিছু করা:** Idempotency হারায়, আর Shell Injection-এর ঝুঁকি থাকে — নির্দিষ্ট Module ব্যবহার করুন।
- **`ControlPath` ডিরেক্টরি না বানানো:** Multiplexing চুপচাপ কাজ করবে না, আর Playbook ধীর চলবে — কারণ বোঝা যাবে না।

১৫.১৩ সমস্যা সমাধান (Troubleshooting)

REAL WORLD — TechNova: একবার TechNova-র Deploy হঠাৎ ব্যর্থ হলো — `Permission denied (publickey,password)`। কারণ ছিল নতুন সার্ভারে `ssh-copy-id` করা হয়েছিল, কিন্তু সার্ভারের `~/.ssh` ডিরেক্টরির Permission ছিল `755` -এর বদলে `777` — OpenSSH নিরাপত্তার জন্য এমন ডিরেক্টরির `authorized_keys` পড়তে অস্বীকার করে। `chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys` দিয়ে ঠিক করা হয়েছিল। শিক্ষা: SSH-এর ত্রুটি প্রায়ই Permission-এর কারণে হয় — প্রথমে `~/.ssh` -এর Permission দেখুন, Key-এর দোষ ধরার আগে।

1. **লক্ষণ:** `Permission denied (publickey)` — **পরীক্ষা:** `ssh -v nahid@203.0.113.10` দিয়ে দেখুন কোন Key চেষ্টা করা হচ্ছে; `ls -la ~/.ssh/` দিয়ে Permission দেখুন। **সমাধান:** `chmod 700 ~/.ssh` , `chmod 600 ~/.ssh/id_ed25519` , `chmod 600 ~/.ssh/authorized_keys` (সার্ভারে)।
2. **লক্ষণ:** `Host key verification failed` — **পরীক্ষা:** সার্ভারটি নতুন করে ইনস্টল হয়েছে কি না, বা IP ঠিকানা বদলেছে কি না। **সমাধান:** সার্ভারের পরিচয় নিশ্চিত করে `ssh-keygen -R 203.0.113.10` দিয়ে পুরনো Key মুছে, আবার `ssh-keyscan` দিয়ে নতুন Key যোগ করুন — কখনো `StrictHostKeyChecking=no` দিয়ে এড়িয়ে যাবেন না।
3. **লক্ষণ:** `sudo: a password is required` — **পরীক্ষা:** `ansible_become_password` Vault-এ আছে কি না, আর Playbook Vault পাসওয়ার্ড দিয়ে চালানো হচ্ছে কি না। **সমাধান:** Vault ফাইলে পাসওয়ার্ড যোগ করুন, অথবা `--ask-become-pass` দিয়ে চালান।
4. **লক্ষণ:** Playbook অস্বাভাবিক ধীর — **পরীক্ষা:** `~/.ansible/cp/` ডিরেক্টরি আছে কি না, আর `ssh_args` -এ `ControlPersist` লেখা আছে কি না। **সমাধান:** ডিরেক্টরি বানিয়ে `ansible.cfg` -এ `ControlPersist` যোগ করুন।
5. **লক্ষণ:** `Missing sudo password` বা `Incorrect sudo password` — **পরীক্ষা:** `sudo -l` দিয়ে সার্ভারে সরাসরি দেখুন `nahid` -এর sudo অধিকার আছে কি না। **সমাধান:** `/etc/sudoers.d/nahid` ফাইলটি `visudo` দিয়ে ঠিক করুন।

১৫.১৪ নিরাপত্তা বিবেচনা

WARNING: SSH Key-ই আপনার সার্ভারের চাবি — Private Key-টি কখনো ইমেইল, Chat, Git বা স্ক্রিনশটে পাঠাবেন না। Key চুরি গেছে বলে সন্দেহ হলে সঙ্গে সঙ্গে সার্ভারের `authorized_keys` থেকে সেই Public Key মুছে ফেলুন, নতুন Key তৈরি করুন, আর সব সার্ভারে নতুন Public Key বসান। একটি চুরি যাওয়া Key দিয়ে আক্রমণকারী যতক্ষণ `authorized_keys` -এ Public Key আছে, ততক্ষণ ঢুকতে পারবে — Key বদলানো নয়, Public Key মুছে ফেলাই সমাধান।

আরও কয়েকটি নিরাপত্তা-নিয়ম: প্রথম, `ansible_password` ও `ansible_become_password` কখনো Inventory-তে প্লেইনটেক্সট লিখবেন না — সেগুলো Vault-এ থাকে (অধ্যায় ১৪)। দ্বিতীয়, Control Node-এর `~/.ansible/cp/` (ControlPersist Socket) ও `/tmp/.ansible/` ডিরেক্টরি শেয়ার করা জায়গায় রাখবেন না — Socket দিয়ে অন্য কেউ আপনার সংযোগ ব্যবহার করতে পারে। তৃতীয়, `shell` Task-এ ব্যবহারকারীর দেওয়া মান বসাবেন না — Shell Injection-এর ঝুঁকি থাকে। চতুর্থ, SSH-এর লগ (`/var/log/auth.log`) নিয়মিত দেখুন — ব্যর্থ লগইন প্রচেষ্টা বাড়লে Fail2ban-এর

মতো ব্যবস্থা নিন। পঞ্চম, পুরনো Key-এর ধরন (যেমন RSA 1024-bit, DSA) ব্যবহার করবেন না — ed25519 বা RSA 4096-bit ব্যবহার করুন।

১৫.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

প্রোডাকশনে SSH ও Privilege Escalation-এর মানদণ্ড: **শুধু Key Authentication** — সব সার্ভারে `PasswordAuthentication no`; **প্রতিটি ইঞ্জিনিয়ারের আলাদা Key** — শেয়ার করা Key নয়, যাতে কেউ চলে গেলে শুধু তার Key মুছে ফেলা যায়; **Key Rotation** — প্রতি বছর Key বদলানো, আর চলে যাওয়া সদস্যের Key সঙ্গে সঙ্গে মুছে ফেলা; **Bastion Host** — Production সার্ভারে সরাসরি SSH না করে একটি Bastion (Jump) Host দিয়ে ঢোকা, যাতে সব সংযোগ এক জায়গা থেকে লগ হয়; **সীমিত sudo** — `ALL=(ALL) ALL`-এর বদলে শুধু প্রয়োজনীয় কমান্ডের sudo দেওয়া (যেমন `nahid ALL=(ALL) /usr/bin/apt, /bin/systemctl restart nginx`); **SSH লগ ও অ্যালার্ট** — ব্যর্থ লগইন ও নতুন Key যোগ করার ঘটনায় অ্যালার্ট; **ControlPersist ও Pipelining** — বড় Playbook-এ গতি বাড়াতে চালু রাখা; **Host Key-এর তালিকা Git-এ** — দলের `known_hosts` ফাইলটি সংস্করণ নিয়ন্ত্রণে রাখা, যাতে নতুন সদস্যের মেশিনে ভুয়া Host Key ঢুকতে না পারে।

১৫.১৬ অধ্যায়ের সারাংশ

Ansible Agent ছাড়াই SSH দিয়ে Managed Node-এ কাজ করে — Module-এর কোড SSH-এ পাঠিয়ে চালিয়ে ফলাফল ফিরিয়ে আনে। `ControlPersist` সংযোগ পুনঃব্যবহার করে গতি বাড়ায়, `Pipelining` Module পাঠানোর ধাপ কমায়। `become`, `become_user` ও `become_method` দিয়ে শুধু প্রয়োজনীয় Task-এ অধিকার বাড়ানো হয় — এটাই Least Privilege নীতি। SSH Key-এর ক্ষেত্রে ed25519, Passphrase, `chmod 600` ও `ssh-agent` — এই চারটি নিয়ম মানতে হয়। Host Key Verification কখনো বন্ধ করা যাবে না — তা MITM আক্রমণের দরজা খুলে দেয়; নতুন Host Key `ssh-keyscan` বা `known_hosts` Module দিয়ে নিরাপদে যোগ করতে হয়। `shell`/`command`-এর বদলে নির্দিষ্ট Module ব্যবহার করতে হয়, আর `ansible_user`/`ansible_password` Inventory-তে প্লেইনটেক্সট লেখা যাবে না — পাসওয়ার্ড থাকলে তা Vault-এ থাকবে।

১৫.১৭ জ্ঞান যাচাই — প্রশ্ন

1. Host Key Verification কী? `StrictHostKeyChecking=no` লিখলে Man-in-the-Middle আক্রমণ কীভাবে সম্ভব হয়, তা ধাপে ধাপে ব্যাখ্যা করুন।
2. `become`, `become_user` ও `become_method`-এর কাজ আলাদা করে লিখুন। ডেটাবেসের কাজে `become_user: postgres` কেন ব্যবহার করা হয়?
3. Passphrase ছাড়া Private Key কেন বিপজ্জনক? `ssh-agent` Passphrase-এর ঝামেলা কীভাবে কমায়, আর তার ঝুঁকি কী?
4. `ControlPersist` কীভাবে Playbook-এর গতি বাড়ায়? `ControlPath` ডিরেক্টরি না থাকলে কী হয়?
5. `ansible_password` Inventory-তে লেখা কেন বিপজ্জনক? পাসওয়ার্ড-ভিত্তিক Authentication-এর বদলে কী ব্যবহার করবেন?

১৫.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. **নিরাপদ Key সেটআপ:** ed25519 Key তৈরি করে Passphrase দিন, Permission ঠিক করুন, `ssh-copy-id` দিয়ে সার্ভারে বসান, এবং `ansible -m ping` দিয়ে যাচাই করুন।

2. **SSH শক্ত করুন:** একটি পরীক্ষামূলক সার্ভারে `PasswordAuthentication no` ও `PermitRootLogin no` লিখে SSH Restart করুন — একটি টার্মিনাল খোলা রেখে পরীক্ষা করুন, যাতে আটকে না যান।
3. **become অনুশীলন:** একটি Playbook লিখুন যেখানে একটি Task `become: true` দিয়ে প্যাকেজ ইনস্টল করে, একটি Task `become_user: technova` দিয়ে Git থেকে কোড টানে, আর একটি Task কোনো `become` ছাড়াই ফাইল পড়ে।
4. **Host Key পরীক্ষা:** `ssh-keyscan -t ed25519 203.0.113.10` চালিয়ে Host Key সংগ্রহ করুন, আর `ssh-keygen -R` দিয়ে পুরনো Key মুছে আবার যোগ করার প্রক্রিয়াটি অনুশীলন করুন।

অধ্যায় 16 • PART III — Roles, Collections, Vault & Cloud

Chapter 16 — Dynamic Inventory, Cloud Integration, and API Automation

১৬.১ ভূমিকা

TechNova যখন ক্লাউডে গেল, তখন একটি নতুন সমস্যা দেখা দিল: সার্ভারের সংখ্যা আর স্থির নয়। সকালে তিনটি সার্ভার, বিকেলে ট্রাফিক বাড়লে সাতটি, রাতে আবার তিনটি — Auto Scaling-এ সার্ভার তৈরি হয়, আবার মুছে যায়। Static Inventory ফাইলে (`hosts.yml`) IP ঠিকানা লিখে রাখলে তা কয়েক ঘণ্টার মধ্যেই ভুল হয়ে যায় — নতুন সার্ভারের IP তালিকায় নেই, মুছে যাওয়া সার্ভারের IP তালিকায় রয়ে গেছে, আর Playbook ভুল সার্ভারে বা অস্তিত্বহীন সার্ভারে কাজ করার চেষ্টা করে। এই সমস্যার সমাধান হলো **Dynamic Inventory** — Inventory ফাইল হাতে লেখার বদলে ক্লাউডের API থেকে সরাসরি হোস্টের তালিকা আনা। আর একই API-দক্ষতা দিয়ে Ansible শুধু সার্ভার কনফিগার করে না — সার্ভার তৈরি, DNS রেকর্ড বদলানো, লোড ব্যালান্সার আপডেট করা, এমনকি যেকোনো REST API-কে স্বয়ংক্রিয়ভাবে ডাকা যায়। এই অধ্যায়ে আপনি শিখবেন Dynamic Inventory Plugin (`aws_ec2`, `azure_rm`, `gcp_compute`, `constructed`) কীভাবে কাজ করে, Inventory Caching কীভাবে API-কলের খরচ কমায়, `uri` Module দিয়ে REST API কীভাবে ডাকতে হয়, আর ক্লাউডের Credential কীভাবে নিরাপদে রাখতে হয়।

১৬.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Static বনাম Dynamic Inventory-এর পার্থক্য ব্যাখ্যা করে কখন কোনটি ব্যবহার করবেন তা বলতে; (২) `aws_ec2`, `azure_rm` ও `gcp_compute` Inventory Plugin কনফিগার করতে; (৩) `constructed` ও `compose` দিয়ে ক্লাউডের Tag ও Metadata থেকে Group ও Variable তৈরি করতে; (৪) Inventory Caching চালু করে API-কলের সংখ্যা ও Playbook-এর সময় কমাতে; (৫) `uri` Module দিয়ে REST API-তে GET, POST, PUT ও DELETE অনুরোধ পাঠাতে; (৬) ক্লাউড Authentication পরিবেশ Variable দিয়ে করতে, আর Hardcoded Key-এর ঝুঁকি ব্যাখ্যা করতে।

১৬.৩ পূর্বশর্ত জ্ঞান

Static Inventory (`hosts.yml`) ও Group Variable-এর ধারণা, YAML-এ Dictionary ও List লেখা, অধ্যায় ১৩-এর Collection ও FQCN-এর ধারণা, REST API-এর প্রাথমিক ধারণা (GET/POST, JSON, HTTP Status Code), এবং পরিবেশ Variable-এর ব্যবহার প্রয়োজন। ক্লাউডের (AWS, Azure, GCP) গভীর জ্ঞান দরকার নেই — শুধু এইটুকু জানলেই চলবে: ক্লাউডে সার্ভারকে Instance বলে, আর প্রতিটি Instance-এ Tag (যেমন `Role=web`) লাগানো যায়।

১৬.৪ প্রথম নীতি থেকে ব্যাখ্যা

Inventory-এর প্রথম নীতি হলো **সত্যের একটিমাত্র উৎস** (Single Source of Truth)। Static Inventory-তে সত্যের উৎস হলো আপনার হাতে লেখা ফাইল — আর ক্লাউডে সত্যের উৎস হলো ক্লাউডের নিজের ডেটাবেস, যেখানে প্রতিটি চলমান Instance-এর IP, Tag ও অবস্থা লেখা থাকে। দুটি উৎস থাকলে তারা আলাদা হয়ে যায় — এটাই Drift। Dynamic Inventory এই Drift দূর করে: Inventory-এর উৎসই হলো ক্লাউডের API — Ansible প্রতিবার চালানোর সময় API-কে জিজ্ঞেস করে "এখন কোন সার্ভারগুলো চলছে?", আর সেই উত্তরই Inventory হয়। দ্বিতীয় নীতি হলো **Tag দিয়ে শ্রেণিবিন্যাস**: ক্লাউডে সার্ভারকে Group-এ ভাগ করা হয় IP দিয়ে নয়, Tag দিয়ে — `Role=web` Tag-যুক্ত সব Instance-ই `webservers` Group-এ পড়ে, তাদের IP যাই হোক না কেন। তৃতীয় নীতি হলো **API-ই হলো নিয়ন্ত্রণের ভাষা**: ক্লাউডের প্রায়

সবকিছু — সার্ভার তৈরি, বন্ধ করা, DNS বদলানো — একটি HTTP API দিয়ে করা যায়; Ansible-এর `uri` Module সেই API-কে ডাকার সাধারণ উপায়, আর ক্লাউড Collection-এর Module-গুলো সেই API-কে ডাকার বিশেষ উপায়। এই তিনটি নীতি একসঙ্গে মিলে ক্লাউড অটোমেশনের ভিত্তি তৈরি করে: সত্য API থেকে আসে, শ্রেণিবিন্যাস Tag থেকে আসে, আর পরিবর্তন API-কলের মাধ্যমে হয়।

> **CONCEPT:** Static Inventory হলো ছাপানো ফোনবই — একবার ছাপা হলে নতুন নম্বর তাতে থাকে না। Dynamic Inventory হলো ফোনবইয়ের অনলাইন সংস্করণ — প্রতিবার খুললেই সর্বশেষ তথ্য দেখা যায়। ছোট, স্থির পরিবেশে (যেমন তিনটি স্থায়ী সার্ভার) ছাপানো ফোনবই যথেষ্ট; কিন্তু সার্ভার যখন ঘণ্টায় ঘণ্টায় তৈরি হয় আর মুছে যায়, তখন অনলাইন সংস্করণ ছাড়া উপায় নেই। আর একটি কথা: Dynamic Inventory মানে Inventory ফাইলটি মুছে ফেলা নয় — Static ও Dynamic একসঙ্গে চলতে পারে, যেমন TechNova তাদের স্থায়ী ডেটাবেস সার্ভার Static-এ রাখে, আর Auto Scaling-এর Web সার্ভার Dynamic-এ আনে।

১৬.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Dynamic Inventory Plugin — aws_ec2

AWS-এ চলমান EC2 Instance-গুলো থেকে Inventory তৈরি করতে `amazon.aws.aws_ec2` Inventory Plugin ব্যবহার করা হয়। কনফিগারেশন ফাইলটি YAML আকারে লেখা হয়, আর `.aws_ec2.yml` দিয়ে শেষ হয়:

```
# inventory/aws_ec2.yml — AWS Dynamic Inventory
---
plugin: amazon.aws.aws_ec2
regions:
  - ap-south-1
# শুধু চলমান (running) Instance নিন
filters:
  instance-state-name: running
# Tag দিয়ে Group তৈরি করুন: Tag "Role=web" হলে Group "webservers"
keyed_groups:
  - key: tags.Role
    prefix: role
    separator: ""
# Tag "Environment=production" হলে Group "production"
  - key: tags.Environment
    prefix: env
    separator: ""
# Instance-এর Private IP দিয়ে SSH করুন
hostnames:
  - private-ip-address
# Tag থেকে Variable তৈরি করুন
compose:
  ansible_user: "'nahid'"
  app_role: tags.Role
```

এই ফাইল দিয়ে Ansible চালালে `tags.Role=web` যুক্ত Instance-গুলো `roleweb` Group-এ পড়ে, আর `tags.Environment=production` যুক্তগুলো `envproduction` Group-এ পড়ে। `keyed_groups`-এর `prefix` ও `separator` দিয়ে Group-এর নাম নিয়ন্ত্রণ করা যায় — যেমন `prefix: ""` দিলে Group-এর নাম হবে শুধু Tag-এর মান (`web`)। `compose` দিয়ে Tag বা Metadata থেকে Variable তৈরি করা যায় — যেমন প্রতিটি হোস্টে `app_role` Variable-এ তার Tag-এর মান বসে যায়।

Azure ও GCP Inventory Plugin

ক্লাউড	Plugin	Collection	Group তৈরির উপায়	প্রমাণীকরণ (Authentication)
AWS	`amazon.aws.aws_ec2`	`amazon.aws`	Tag (`keyed_groups` দিয়ে)	`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY` পরিবেশ Variable
Azure	`azure.azcollection.azure_rm`	`azure.azcollection`	Tag ও Resource Group	`AZURE_CLIENT_ID`, `AZURE_SECRET`, `AZURE_TENANT`, `AZURE_SUBSCRIPTION_ID`
GCP	`google.cloud.gcp_compute`	`google.cloud`	Label ও Zone	Service Account JSON ফাইল (`GCP_SERVICE_ACCOUNT_FILE`)

তিনটি ক্লাউডেই নীতিটি এক: Plugin ক্লাউডের API-তে অনুরোধ পাঠায়, চলমান Instance-গুলোর তালিকা আনে, আর Tag/Label দিয়ে Group বানায়। পার্থক্য শুধু প্রমাণীকরণের উপায়ে — AWS-এ Access Key, Azure-এ Service Principal, GCP-এ Service Account JSON। কোনো ক্ষেত্রেই Credential Playbook বা Inventory ফাইলে লেখা যাবে না — সেগুলো পরিবেশ Variable বা Vault-এ থাকে (নিচে বিস্তারিত)।

constructed Inventory Plugin — Group ও Variable বানানো

`constructed` Plugin-এর কাজ হলো অন্য Inventory-এর ওপর নিয়ম লাগিয়ে নতুন Group ও Variable তৈরি করা — যেমন "যেসব হোস্টের `app_role` হলো `web`, তারা `webservers` Group-এ পড়বে":

```
# inventory/constructed.yml
---
plugin: constructed
groups:
  webservers: app_role == "web"
  appservers: app_role == "api"
  production: env == "production"
compose:
  ansible_port: 22
```

এই ফাইলটি `aws_ec2.yml`-এর সঙ্গে একই Inventory ডিরেক্টরিতে রাখলে Ansible প্রথমে AWS থেকে হোস্ট আনে, তারপর `constructed`-এর নিয়ম লাগিয়ে Group বানায়। এতে Playbook-এ `hosts: webservers` লেখা যায় — Tag-এর নাম মনে রাখতে হয় না।

Inventory Caching

Dynamic Inventory প্রতিবার চালানোর সময় API-কে ডাকে — বড় পরিবেশে এটি ধীর ও ব্যয়বহুল হতে পারে। **Inventory Caching** এই সমস্যার সমাধান: প্রথমবার API থেকে আনা তালিকা একটি Cache ফাইলে রাখা হয়, আর নির্দিষ্ট সময়ের মধ্যে আবার চালালে Cache থেকেই তালিকা নেওয়া হয়:

```
# ansible.cfg — Inventory Cache
[inventory]
# Cache চালু করুন
```

```
cache = True
# Cache কোথায় রাখবেন — JSON ফাইল হিসেবে
cache_plugin = ansible.builtin.jsonfile
cache_connection = /tmp/ansible_inventory_cache
# Cache কতক্ষণ বৈধ থাকবে (সেকেন্ডে) — ৩৬০০ = ১ ঘণ্টা
cache_timeout = 3600

# Cache জোর করে বাতিল করে নতুন করে আনতে:
# ansible-playbook site.yml --flush-cache
```

Cache-এর সময়সীমা ঠিক করা একটি বাণিজ্যিক সিদ্ধান্ত (Trade-off): সময়সীমা বেশি হলে Playbook দ্রুত চলে, কিন্তু নতুন সার্ভার এক ঘণ্টা পর্যন্ত তালিকায় আসবে না; সময়সীমা কম হলে তালিকা সবসময় নতুন থাকে, কিন্তু API-কল বেশি হয়। TechNova Production-এ ১৫ মিনিট (৯০০ সেকেন্ড) ব্যবহার করে — নতুন সার্ভার ১৫ মিনিটের মধ্যে তালিকায় আসে, আর API-কলের খরচও কম থাকে।

uri Module — REST API অটোমেশন

যেকোনো REST API-কে ডাকতে `ansible.builtin.uri` Module ব্যবহার করা হয়। এটি HTTP অনুরোধ পাঠায় — GET, POST, PUT, DELETE — আর উত্তরের Status Code ও JSON Body ফিরিয়ে আনে:

```
# GET — তথ্য পড়া
- name: TechNova API-এর স্বাস্থ্য পরীক্ষা করুন
  ansible.builtin.uri:
    url: https://api.technova.example/health
    method: GET
    return_content: true
    status_code: 200
  register: health_check

# POST — নতুন কিছু তৈরি করা (যেমন Deploy Hook ডাকা)
- name: Deploy Hook ডাকুন
  ansible.builtin.uri:
    url: https://api.technova.example/deploy
    method: POST
    headers:
      Content-Type: application/json
      Authorization: "Bearer CHANGE-ME-API-TOKEN"
    body_format: json
    body:
      version: "v1.4.2"
      environment: production
    status_code: 202
  no_log: true   # ← Token আউটপুটে না দেখাতে
```

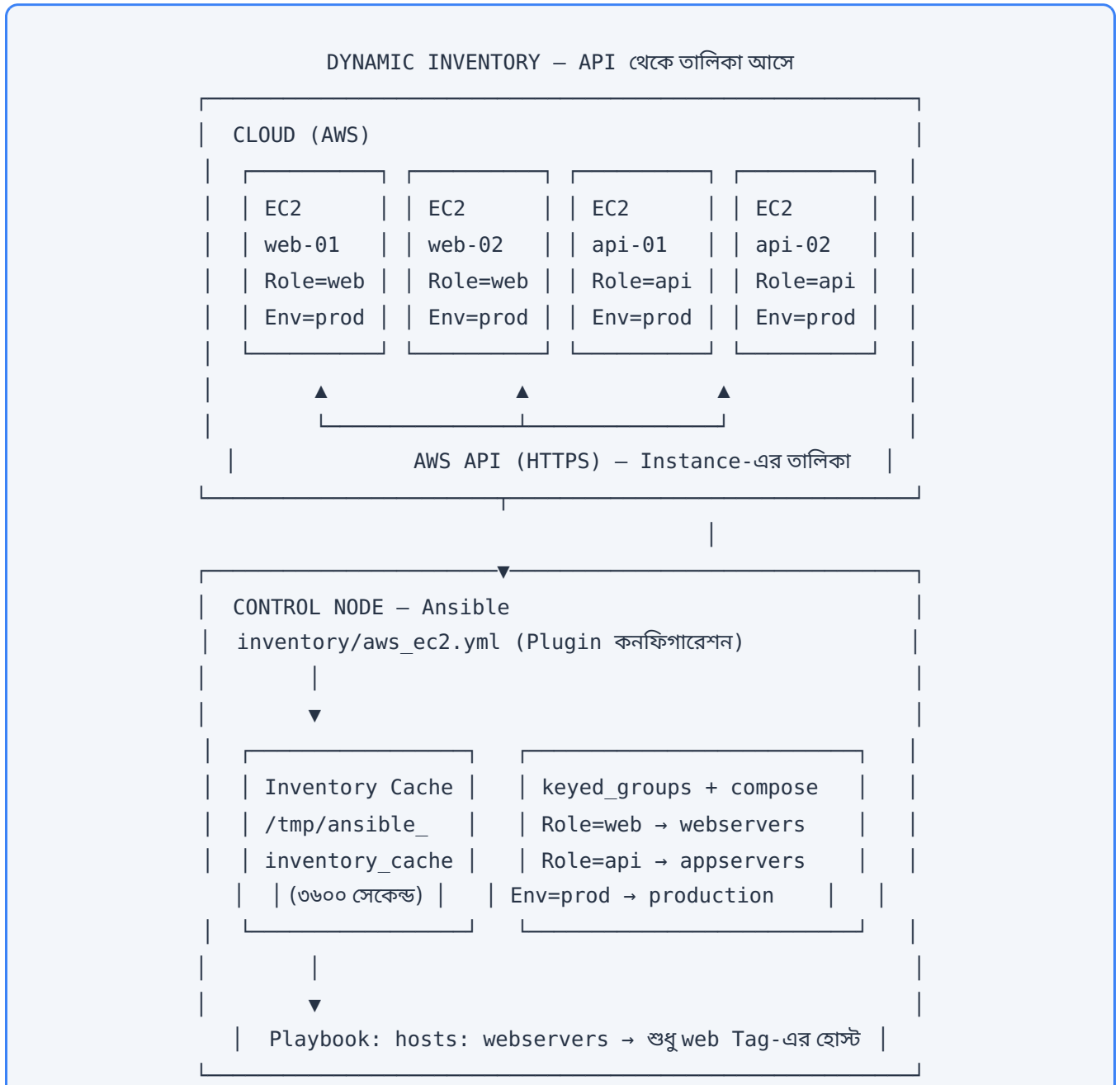
`uri` Module-এর শক্তি হলো তার সাধারণতা (Generality) — যেকোনো API-কে ডাকা যায়, এমনকি যার জন্য কোনো বিশেষ Module নেই। তবে বিশেষ Module থাকলে সেটিই ব্যবহার করা ভালো — যেমন AWS-এর জন্য `amazon.aws.ec2_instance`, DNS-এর জন্য `community.general.cloudflare_dns` — কারণ বিশেষ Module Idempotent হয়, আর `uri` দিয়ে লেখা Task Idempotency নিজে হাতে নিশ্চিত করতে হয়।

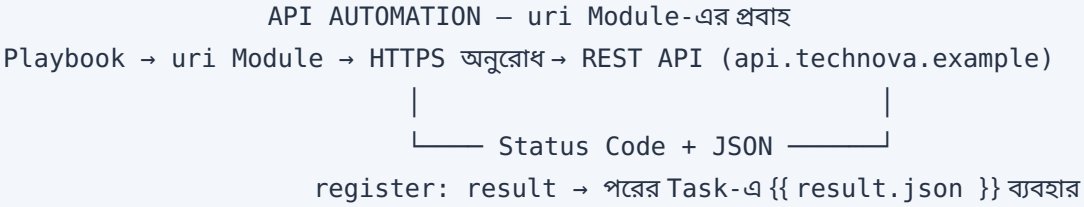
১৬.৬ অভ্যন্তরীণ কার্যপ্রণালী

Dynamic Inventory চালানোর সময় Ansible ভেতরে যা করে, তার ক্রম এইরকম। প্রথমে Ansible Inventory-এর উৎসগুলো পড়ে — `ansible.cfg`-এর `inventory` পাথে থাকা সব ফাইল ও ডিরেক্টরি। `.aws_ec2.yml` দেখলে সে বুঝতে পারে এটি একটি Inventory Plugin-এর কনফিগারেশন, আর Plugin-টি লোড করে। Plugin-টি প্রথমে Cache দেখে — Cache বৈধ থাকলে API-কে না ডেকে Cache থেকেই তালিকা ফিরিয়ে দেয়। Cache না থাকলে Plugin ক্লাউডের API-তে অনুরোধ পাঠায় — পরিবেশ Variable থেকে Credential নিয়ে, HTTPS-এ স্বাক্ষরিত অনুরোধ পাঠিয়ে, চলমান Instance-গুলোর তালিকা আনে। এরপর Plugin প্রতিটি Instance-এর Tag, IP, Zone ও Metadata পড়ে `keyed_groups` ও `compose`-এর নিয়ম লাগায় — Group তৈরি করে, Variable বসায়। তারপর `constructed` Plugin (যদি থাকে) সেই তালিকার ওপর আরও নিয়ম লাগায়। শেষে Ansible-এর হাতে আসে একটি সম্পূর্ণ Inventory — যেন কেউ হাতে `hosts.yml` লিখে দিয়েছে। এরপর Playbook স্বাভাবিকভাবে চলে — Ansible জানে না (আর জানার দরকারও নেই) যে Inventory ফাইল থেকে আসেনি, API থেকে এসেছে। `uri` Module-এর ক্ষেত্রে কাজটি আরও সরাসরি: Module-টি Control Node থেকে নয়, Managed Node থেকে (অথবা `delegate_to: localhost` দিলে Control Node থেকে) HTTP অনুরোধ পাঠায়, উত্তরের Status Code `status_code`-এর সঙ্গে মিলিয়ে দেখে, আর JSON Body-কে Variable-এ রূপান্তর করে — যাতে পরের Task-এ `{{ health_check.json.status }}`-এর মতো করে ব্যবহার করা যায়।

> **TIP:** Dynamic Inventory-এর তালিকা দেখতে `ansible-inventory -i inventory/aws_ec2.yml --graph` চালান — Group ও হোস্টের গাছের মতো চিত্র দেখা যাবে। আর `ansible-inventory -i inventory/aws_ec2.yml --list` চালালে পুরো Inventory JSON আকারে দেখা যাবে — কোন হোস্ট কোন Group-এ, কোন Variable কী — সব যাচাই করা যায়, Playbook না চালিয়েই।

১৬.৭ আর্কিটেকচার চিত্র





CREDENTIAL — কোথায় থাকে, কোথায় থাকে না

✅ পরিবেশ Variable: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY

✅ Vault ফাইল: group_vars/all/vault.yml (এনক্রিপ্ট করা)

✅ CI Secret Store: GitHub Actions Secrets, GitLab CI Variables

❌ Playbook-এ সরাসরি লেখা — কখনো নয়

❌ Inventory ফাইলে লেখা — কখনো নয়

❌ Git-এ Push করা ফাইলে লেখা — কখনো নয়

চিত্রের মূল কথা: Inventory-এর সত্য ক্লাউডের API-তে থাকে, আর Credential-এর জায়গা পরিবেশ Variable বা Vault-এ — কখনো কোডে নয়।

১৬.৮ ব্যবহারিক উদাহরণ

TechNova-র AWS পরিবেশে Dynamic Inventory ও API অটোমেশনের সম্পূর্ণ উদাহরণ:

```
# ── 1. AWS Credential পরিবেশ Variable-এ দিন (টার্মিনালে, ফাইলে নয়) ──
export AWS_ACCESS_KEY_ID="CHANGE-ME-AWS-ACCESS-KEY"
export AWS_SECRET_ACCESS_KEY="CHANGE-ME-AWS-SECRET-KEY"
export AWS_DEFAULT_REGION="ap-south-1"
# ব্যাখ্যা: export করা মান শুধু এই টার্মিনাল সেশনে থাকে — ফাইলে লেখা হয় না।
# স্থায়ীভাবে রাখতে চাইলে ~/.bashrc-এ নয়, পাসওয়ার্ড ম্যানেজার বা CI Secret Store ব্যবহার করুন।

# ── 2. Dynamic Inventory-এর তালিকা দেখুন ──
ansible-inventory -i inventory/aws_ec2.yml --graph
# চিত্রণমূলক আউটপুট:
# @all:
#   |--@roleweb:
#   |  |--web-01.technova.example
#   |  |--web-02.technova.example
#   |--@roleapi:
#   |  |--api-01.technova.example
#   |  |--api-02.technova.example
#   |--@ungrouped:

# ── 3. Dynamic Inventory দিয়ে Playbook চালান ──
ansible-playbook -i inventory/aws_ec2.yml site.yml
# ব্যাখ্যা: -i দিয়ে Inventory-এর উৎস বলা হয় — Static ফাইলের বদলে Dynamic Plugin।

# ── 4. AWS-এ নতুন Instance তৈরি করা (amazon.aws Collection) ──
- name: নতুন Web সার্ভার তৈরি করুন
  amazon.aws.ec2_instance:
    name: technova-web-03
    instance_type: t3.small
    image_id: ami-0a1b2c3d4e5f67890   # Ubuntu 24.04 LTS AMI (উদাহরণ)
    region: ap-south-1
    key_name: technova-key
    vpc_subnet_id: subnet-0123456789abcdef0
    security_group: technova-web-sg
    network:
      assign_public_ip: false
    tags:
      Role: web
      Environment: production
    state: present
  register: new_instance
  delegate_to: localhost
  # ব্যাখ্যা: delegate_to: localhost মানে Task-টি Managed Node-এ নয়,
  # Control Node-এ চলে — কারণ AWS API-কে ডাকতে Managed Node-এ যাওয়ার দরকার নেই।
```

REST API অটোমেশনের আরও উদাহরণ — Deploy-এর পরে স্বাস্থ্য পরীক্ষা ও Slack-এ জানানো:

```
# Deploy-এর পরে API-এর স্বাস্থ্য পরীক্ষা করুন
- name: API স্বাস্থ্য পরীক্ষা করুন (পুনরায় চেষ্টাসহ)
  ansible.builtin.uri:
    url: https://api.technova.example/health
    method: GET
    status_code: 200
  register: health
  until: health.status == 200
  retries: 10
  delay: 15
  # ব্যাখ্যা: until/retries/delay মানে — ২০০ না আসা পর্যন্ত ১৫ সেকেন্ড পরপর ১০ বার চেষ্টা করবে।
  # নতুন Deploy-এর পরে অ্যাপ চালু হতে সময় লাগে — তাই একবার চেষ্টা করলেই হবে না।

# স্বাস্থ্য পরীক্ষার ফলাফল দেখান
- name: স্বাস্থ্য পরীক্ষার ফলাফল দেখান
  ansible.builtin.debug:
    msg: "API status: {{ health.json.status }}, version: {{ health.json.version }}"
```

১৬.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

```
# ── প্রজেক্ট কাঠামো ──
# /home/nahid/technova-ansible/
# ├── ansible.cfg
# ├── requirements.yml
# ├── site.yml
# ├── inventory/
# │   ├── hosts.yml            # Static — স্থায়ী সার্ভার (ডেটাবেস)
# │   ├── aws_ec2.yml          # Dynamic — AWS EC2 Instance
# │   └── constructed.yml      # Group তৈরির নিয়ম
# └── group_vars/
#     └── all/
#         └── vars.yml

# ── requirements.yml ──
---
collections:
  - name: amazon.aws
    version: "8.1.0"
  - name: community.general
    version: "9.5.0"

# ── inventory/aws_ec2.yml ──
---
plugin: amazon.aws.aws_ec2
regions:
  - ap-south-1
filters:
  instance-state-name: running
  tag:Environment: production
keyed_groups:
  - key: tags.Role
    prefix: ""
    separator: ""
hostnames:
  - private-ip-address
compose:
```

```
  ansible_user: "'nahid'"
  ansible_ssh_private_key_file: "'~/.ssh/id_ed25519'"

# ── inventory/constructed.yml ──
---
plugin: constructed
groups:
  webservers: Role == "web"
  appservers: Role == "api"
  production: Environment == "production"

# ── inventory/hosts.yml ── (Static অংশ — স্থায়ী ডেটাবেস সার্ভার)
all:
  children:
    dbservers:
      hosts:
        db01.technova.example:
          ansible_host: 203.0.113.12

# ── ansible.cfg ──
[defaults]
inventory = inventory/
roles_path = roles
host_key_checking = True

[inventory]
cache = True
cache_plugin = ansible.builtin.jsonfile
cache_connection = /tmp/ansible_inventory_cache
cache_timeout = 900

# ── site.yml — Dynamic Inventory দিয়ে Deploy ──
---
- name: Web সার্ভারে Nginx বসান
  hosts: webservers
  roles:
    - technova_nginx

- name: API সার্ভারে অ্যাপ Deploy করুন
  hosts: appservers
  roles:
    - role: technova_app
      vars:
        app_version: v1.4.2

- name: Deploy-এর পরে স্বাস্থ্য পরীক্ষা করুন
  hosts: localhost
  gather_facts: false
  tasks:
    - name: API স্বাস্থ্য পরীক্ষা করুন
      ansible.builtin.uri:
        url: https://api.technova.example/health
        method: GET
        status_code: 200
      register: health
      until: health.status == 200
      retries: 10
      delay: 15
```

এই কাঠামোতে `inventory/` ডিরেক্টরিটি Ansible-এর Inventory-এর উৎস — Ansible ডিরেক্টরির ভেতরের সব ফাইল পড়ে: Static `hosts.yml`, Dynamic `aws_ec2.yml`, আর `constructed.yml`-এর নিয়ম — তিনটি মিলে একটি সম্পূর্ণ Inventory তৈরি হয়। `hosts: localhost` দিয়ে লেখা শেষ Play-টি Control Node-এই চলে — স্বাস্থ্য পরীক্ষার জন্য কোনো Managed Node-এ যাওয়ার দরকার নেই।

১৬.১০ ধাপে ধাপে বাস্তবায়ন

1. **Collection ইনস্টল করুন:** `ansible-galaxy collection install amazon.aws` চালান — AWS Module ও Inventory Plugin এতে আসে।
2. **Credential পরিবেশ Variable-এ দিন:** `export AWS_ACCESS_KEY_ID="CHANGE-ME-AWS-ACCESS-KEY"` ও `export AWS_SECRET_ACCESS_KEY="CHANGE-ME-AWS-SECRET-KEY"` চালান।
3. **Inventory Plugin ফাইল লিখুন:** `inventory/aws_ec2.yml` তৈরি করুন — Region, Filter, `keyed_groups` ও `compose` লিখুন।
4. **তালিকা যাচাই করুন:** `ansible-inventory -i inventory/aws_ec2.yml --graph` চালিয়ে দেখুন Group ও হোস্ট ঠিকভাবে আসছে কি না।
5. **constructed নিয়ম যোগ করুন:** `inventory/constructed.yml` লিখে আবার `--graph` চালান — নতুন Group-গুলো দেখা যাবে।
6. **Cache চালু করুন:** `ansible.cfg`-এর `[inventory]` অংশে Cache সেটিংস লিখুন, আর দুবার `ansible-inventory --list` চালিয়ে দ্বিতীয়বারের গতি মাপুন।
7. **Playbook চালান:** `ansible-playbook -i inventory/ site.yml` চালান — Dynamic Inventory দিয়ে Deploy হবে।
8. **Cache বাতিল করা শিখুন:** নতুন সার্ভার তৈরি হওয়ার পরে `ansible-playbook site.yml --flush-cache` চালিয়ে দেখুন নতুন হোস্ট তালিকায় আসছে কি না।
9. **uri Module পরীক্ষা করুন:** `https://api.technova.example/health`-এ GET অনুরোধ পাঠিয়ে `register` করা ফলাফল `debug` দিয়ে দেখান।
10. **নিরাপত্তা যাচাই করুন:** `grep -r "CHANGE-ME-AWS" ~/technova-ansible/ --include="*.yml" --include="*.cfg"` চালিয়ে নিশ্চিত হোন কোনো Credential ফাইলে লেখা নেই।

১৬.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

```
$ ansible-inventory -i inventory/aws_ec2.yml --graph
# চিত্রণমূলক আউটপুট:
# @all:
#   |--@aws_ec2:
#   |  |--web-01.technova.example
#   |  |--web-02.technova.example
#   |  |--api-01.technova.example
#   |  |--api-02.technova.example
#   |--@roleweb:
#   |  |--web-01.technova.example
#   |  |--web-02.technova.example
#   |--@roleapi:
#   |  |--api-01.technova.example
#   |  |--api-02.technova.example
#   |--@ungrouped:
```

```
$ ansible-playbook -i inventory/ site.yml
# চিত্রণমূলক আউটপুট:
# PLAY [Web সার্ভারে Nginx বসান] ************************************
#
# TASK [Gathering Facts] *********************************************
# ok: [web-01.technova.example]
# ok: [web-02.technova.example]
#
# TASK [technova_nginx : Nginx প্যাকেজ ইনস্টল করুন] ******************
# ok: [web-01.technova.example]
# ok: [web-02.technova.example]
#
# PLAY [Deploy-এর পরে স্বাস্থ্য পরীক্ষা করুন] ***********************
#
# TASK [API স্বাস্থ্য পরীক্ষা করুন] **********************************
# ok: [localhost]
#
# PLAY RECAP *********************************************************
# web-01.technova.example : ok=4  changed=0  unreachable=0  failed=0
# web-02.technova.example : ok=4  changed=0  unreachable=0  failed=0
# localhost               : ok=1  changed=0  unreachable=0  failed=0
```

আউটপুট পড়ার নিয়ম: `--graph`-এ Group-এর গাছ দেখা যায় — `roleweb` Group-এ দুটি Web সার্ভার, `roleapi` Group-এ দুটি API সার্ভার। Playbook-এর আউটপুটে হোস্টের নামগুলো ক্লাউডের Instance-এর নাম — IP ঠিকানা কোথাও লেখা নেই, কারণ Inventory API থেকে এসেছে। শেষ Play-টি `localhost`-এ চলে — স্বাস্থ্য পরীক্ষার জন্য কোনো Managed Node লাগে না। যদি `--graph`-এ কোনো হোস্ট না দেখায়, তবে প্রথমে Filter দেখুন — `instance-state-name: running` Filter-টি চলমান Instance ছাড়া বাকিদের বাদ দেয়; বন্ধ থাকা Instance তালিকায় আসবে না, এটাই প্রত্যাশিত।

১৬.১২ সাধারণ ভুল

- **Credential Playbook বা Inventory-তে লেখা:** সবচেয়ে বিপজ্জনক ভুল — Git-এ গেলে Key চুরি হয়; পরিবেশ Variable বা Vault ব্যবহার করুন।
- **Tag না দিয়ে Instance তৈরি করা:** Tag ছাড়া Dynamic Inventory Group বানাতে পারে না — প্রতিটি Instance-এ `Role` ও `Environment` Tag বাধ্যতামূলক করুন।
- **Cache-এর সময়সীমা বেশি রাখা:** নতুন সার্ভার ঘণ্টার পর ঘণ্টা তালিকায় আসবে না — Production-এ ১৫ মিনিটের বেশি রাখবেন না।
- **`delegate_to: localhost` না দেওয়া:** ক্লাউড Module Managed Node-এ চালালে সেখানে Credential থাকতে হয় — API-কলের Task Control Node-এ চালান।
- **`uri` দিয়ে Idempotent Task না লেখা:** POST অনুরোধ প্রতিবার নতুন কিছু তৈরি করে — Idempotency নিশ্চিত করতে প্রথমে GET দিয়ে দেখুন জিনিসটি আছে কি না।
- **Secret `uri`-এর আউটপুটে ফাঁস করা:** Token-যুক্ত Task-এ `no_log: true` না দিলে লগে Token ছাপা হয়।
- **Static ও Dynamic Inventory-এর নামের সংঘাত:** দুটি উৎসে একই Group নাম থাকলে Group মিশে যায় — নামের নিয়ম ঠিক করে নিন।
- **API-এর Rate Limit না মানা:** Cache ছাড়া বারবার API ডাকলে ক্লাউড অনুরোধ বন্ধ করে দিতে পারে — Cache চালু রাখুন।

১৬.১৩ সমস্যা সমাধান (Troubleshooting)

REAL WORLD — TechNova: TechNova-র Auto Scaling-এ নতুন Web সার্ভার তৈরি হলো, কিন্তু Playbook তাকে খুঁজে পাচ্ছিল না। কারণ ছিল দুটি: প্রথম, নতুন Instance-এ `Role=web` Tag লাগানো হয়নি — Launch Template-এ Tag দেওয়ার ধাপটি বাদ পড়েছিল। দ্বিতীয়, Inventory Cache-এর সময়সীমা ছিল এক ঘণ্টা — Cache-এ পুরনো তালিকা ছিল। সমাধান: Launch Template-এ Tag বাধ্যতামূলক করা হলো, Cache-এর সময়সীমা ১৫ মিনিটে নামানো হলো, আর Deploy Pipeline-এ `--flush-cache` যোগ করা হলো। শিক্ষা: Dynamic Inventory-এর সমস্যা প্রায়ই Inventory-এর নয় — Tag বা Cache-এর।

1. **লক্ষণ:** `ansible-inventory --graph`-এ কোনো হোস্ট নেই — **পরীক্ষা:** `export AWS_ACCESS_KEY_ID` দেওয়া আছে কি না (`env | grep AWS`); Filter-এর Tag ঠিক আছে কি না; Instance চলমান কি না। **সমাধান:** Credential দিন, Filter শিথিল করে পরীক্ষা করুন, আর `aws ec2 describe-instances` দিয়ে সরাসরি API-এর উত্তর দেখুন।
2. **লক্ষণ:** `ERROR! Failed to parse inventory source` — **পরীক্ষা:** ফাইলের নাম `.aws_ec2.yml` দিয়ে শেষ হচ্ছে কি না; প্রথম লাইনে `plugin: amazon.aws.aws_ec2` লেখা আছে কি না। **সমাধান:** ফাইলের নাম ও Plugin-এর নাম ঠিক করুন — নাম ভুল হলে Ansible ফাইলটিকে Static Inventory ভেবে পড়তে চেষ্টা করে।
3. **লক্ষণ:** নতুন হোস্ট তালিকায় আসছে না — **পরীক্ষা:** Cache বৈধ আছে কি না; Tag লাগানো আছে কি না। **সমাধান:** `--flush-cache` দিয়ে Cache বাতিল করুন, আর Instance-এর Tag যাচাই করুন।
4. **লক্ষণ:** `uri` Task-এ `401 Unauthorized` — **পরীক্ষা:** Token-টি Vault থেকে ঠিকভাবে আসছে কি না; Header-এর নাম `Authorization` ঠিক আছে কি না। **সমাধান:** `no_log: true` সরিয়ে সাময়িকভাবে (পরীক্ষার পরিবেশে) Header দেখুন, অথবা `curl` দিয়ে হাতে পরীক্ষা করুন।
5. **লক্ষণ:** `uri` Task-এ `502 Bad Gateway` বা সময়সীমা (Timeout) — **পরীক্ষা:** API সার্ভারটি চলছে কি না; URL-টি ঠিক আছে কি না। **সমাধান:** `timeout` আর্গুমেন্ট বাড়ান, আর `until/retries/delay` দিয়ে পুনরায় চেষ্টা যোগ করুন।

১৬.১৪ নিরাপত্তা বিবেচনা

WARNING: ক্লাউড Credential হলো আপনার ক্লাউড অ্যাকাউন্টের চাবি — AWS Secret Key ফাঁস হলে আক্রমণকারী আপনার নামে সার্ভার তৈরি করে বিল বাড়াতে পারে, ডেটা চুরি করতে পারে, এমনকি পুরো অ্যাকাউন্ট মুছে দিতে পারে। Credential কখনো Playbook, Inventory, Git, Chat বা টিকিটে লিখবেন না। সবসময় পরিবেশ Variable, Vault, বা CI Secret Store ব্যবহার করুন — আর Credential-এর অধিকার (Permission) সীমিত রাখুন: Ansible-এর Key দিয়ে শুধু যা দরকার, তাই করা যাবে — যেমন শুধু EC2 Instance দেখা ও তৈরি করা, IAM ব্যবহারকারী বানানো নয়। এটাই ক্লাউডে Least Privilege।

আরও কয়েকটি নিরাপত্তা-নিয়ম: প্রথম, ক্লাউড Credential নিয়মিত Rotation করুন — AWS-এ Access Key প্রতি ৯০ দিনে বদলান। দ্বিতীয়, `uri` Module দিয়ে API ডাকার সময় সবসময় HTTPS ব্যবহার করুন — HTTP-এ Token প্লেইনটেক্সটে যায়। তৃতীয়, API Token-যুক্ত Task-এ `no_log: true` দিন, যাতে লগে Token না থাকে। চতুর্থ, Dynamic Inventory-এর Cache ফাইল (`/tmp/ansible_inventory_cache`) শেয়ার করা জায়গায় রাখবেন না — তাতে Instance-এর IP ও Tag-এর তথ্য থাকে। পঞ্চম, ক্লাউডে তৈরি সার্ভারের Security Group-এ শুধু প্রয়োজনীয় Port খুলুন — Dynamic Inventory দিয়ে সার্ভার তৈরি করা সহজ বলে Security Group-এর নিয়ম শিথিল করবেন না।

১৬.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

প্রোডাকশনে Dynamic Inventory ও ক্লাউড অটোমেশনের মানদণ্ড: **Tag-এর নীতি** — প্রতিটি Instance-এ `Role`, `Environment`, `Owner` ও `CostCenter` Tag বাধ্যতামূলক, আর Launch Template-এ Tag স্বয়ংক্রিয়ভাবে লাগানো; **Cache-এর সময়সীমা** — Production-এ ১৫ মিনিট, Development-এ ১ ঘণ্টা; **Deploy Pipeline-এ `--flush-cache`** — নতুন সার্ভার তৈরি হওয়ার পরে Cache বাতিল করা; **Credential-এর ব্যবস্থাপনা** — CI-তে Secret Store, স্থানীয়ভাবে পরিবেশ Variable, কখনো ফাইলে নয়; **Idempotent ক্লাউড Task** — `ec2_instance`-এর মতো Module ব্যবহার করা, `uri` দিয়ে সার্ভার তৈরি না করা; **পরিবর্তনের আগে `--check`** — ক্লাউডে কিছু তৈরি বা বদলানোর আগে Dry Run করে দেখা; **খরচের নজরদারি** — Dynamic Inventory দিয়ে তৈরি সার্ভারের সংখ্যা ও খরচ নিয়মিত দেখা, যাতে Auto Scaling-এর ভুলে বিল না বাড়ে; **Static + Dynamic একসঙ্গে** — স্থায়ী সার্ভার Static-এ, পরিবর্তনশীল সার্ভার Dynamic-এ, দুটির নামের নিয়ম আলাদা রাখা।

১৬.১৬ অধ্যায়ের সারাংশ

Dynamic Inventory হলো ক্লাউডের API থেকে হোস্টের তালিকা আনার ব্যবস্থা — Static Inventory-এর বদলে, বা তার সঙ্গে। `aws_ec2`, `azure_rm` ও `gcp_compute` Plugin ক্লাউডের Instance-গুলো থেকে Tag দিয়ে Group বানায়, আর `constructed` Plugin সেই Group-এর ওপর আরও নিয়ম লাগায়। Inventory Caching API-কলের সংখ্যা কমায়, তবে সময়সীমা ঠিক করতে হয় — Production-এ ১৫ মিনিট একটি ভালো মান। `uri` Module দিয়ে যেকোনো REST API-কে GET, POST, PUT, DELETE অনুরোধ পাঠানো যায় — তবে বিশেষ Module থাকলে সেটিই ব্যবহার করা উচিত, কারণ তা Idempotent হয়। ক্লাউড Credential পরিবেশ Variable, Vault বা CI Secret Store-এ থাকে — কখনো Playbook, Inventory বা Git-এ নয়। Tag-এর নীতি, Cache-এর সময়সীমা ও Credential-এর Rotation — এই তিনটি ঠিক থাকলে ক্লাউড অটোমেশন নিরাপদ ও নির্ভরযোগ্য হয়।

১৬.১৭ জ্ঞান যাচাই — প্রশ্ন

1. Static বনাম Dynamic Inventory — Auto Scaling পরিবেশে Static Inventory কেন ভুল হয়ে যায়? Single Source of Truth বলতে কী বোঝায়?
2. `keyed_groups` ও `compose`-এর কাজ কী? `tags.Role=web` থেকে `webservers` Group কীভাবে তৈরি করবেন?
3. Inventory Caching কী? Cache-এর সময়সীমা বেশি রাখলে কী সমস্যা হয়, আর কম রাখলে কী সমস্যা হয়?
4. `uri` Module দিয়ে POST অনুরোধ পাঠাতে কী কী লাগে? Idempotency নিশ্চিত করতে কী করবেন?
5. ক্লাউড Credential Playbook-এ লেখা কেন বিপজ্জনক? Credential রাখার তিনটি নিরাপদ জায়গার নাম লিখুন।

১৬.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. **Dynamic Inventory:** `inventory/aws_ec2.yml` লিখে `ansible-inventory --graph` চালান — Group ও হোস্টের তালিকা যাচাই করুন। (AWS অ্যাকাউন্ট না থাকলে ফাইলটি লিখে তার গঠন ব্যাখ্যা করুন।)
2. **constructed নিয়ম:** `inventory/constructed.yml` লিখুন যা `Role` Tag থেকে `webservers` ও `appservers` Group বানায়, এবং `--graph` দিয়ে যাচাই করুন।
3. **Cache পরীক্ষা:** `ansible.cfg`-এ Cache চালু করে দুবার `ansible-inventory --list` চালান — দ্বিতীয়বারের সময় মাপুন ও Cache ফাইলটি দেখুন।
4. **API অটোমেশন:** `uri` Module দিয়ে একটি পাবলিক API-তে (যেমন `https://api.github.com`) GET অনুরোধ পাঠিয়ে ফলাফল `register` করে `debug` দিয়ে দেখান।

PART IV • DEVOPS ENGINEERING MASTERY

Server and Cloud Configuration Automation

Linux সার্ভার provisioning, Nginx/Node.js/ডেটাবেস ডেপ্লয়মেন্ট, নিরাপত্তা hardening, AWS/Azure/GCP/IBM Cloud অটোমেশন, Docker ও Kubernetes পরিবেশ কনফিগারেশন।

অধ্যায় 17 • PART IV — Server Provisioning & Configuration

Chapter 17 — Linux Server Provisioning and Configuration: Baseline Server Setup

১৭.১ ভূমিকা

একটি সার্ভার হাতে পাওয়ার পর প্রথম প্রশ্নটি কখনোই "অ্যাপটি কীভাবে চালাব" নয় — প্রথম প্রশ্নটি হলো "এই মেশিনটি কি বিশ্বাসযোগ্য অবস্থায় আছে?" একটি নতুন Ubuntu Server 24.04 LTS মেশিন প্রথমে থাকে কাঁচা অবস্থায়: Hostname হয়তো ডিফল্ট, Timezone UTC, প্রয়োজনীয় প্যাকেজ নেই, Kernel Parameter ডিফল্ট, আর কোনো ফাইল কোথায় থাকবে তার কোনো নিশ্চয়তা নেই। একজন প্রকৌশলী হাতে-কলমে এই কাজগুলো করলে দশটি সার্ভারে দশ রকম ফল হয় — কোথাও Timezone ভুল, কোথাও একটি প্যাকেজ বাদ পড়ে, কোথাও MOTD-তে পুরনো তথ্য। এই অধ্যায়ে আমরা শিখব কীভাবে Ansible দিয়ে একটি **Baseline** — অর্থাৎ প্রতিটি সার্ভারের জন্য অভিন্ন, যাচাইযোগ্য, পুনরাবৃত্তিযোগ্য একটি প্রাথমিক অবস্থা — তৈরি করা যায়। Baseline হলো সেই ভিত্তি যার ওপরে পরের অধ্যায়গুলোর Nginx, Node.js, ডেটাবেস ও নিরাপত্তা-কনফিগারেশন দাঁড়াবে। TechNova-র ক্ষেত্রে Baseline মানে: প্রতিটি সার্ভারের Hostname পরিকল্পনা অনুযায়ী, Timezone Asia/Dhaka, প্রয়োজনীয় প্যাকেজ ইনস্টল, Kernel Parameter নির্দিষ্ট মান অনুযায়ী, আর গুরুত্বপূর্ণ ফাইলগুলো Template থেকে তৈরি। একবার Baseline Playbook লেখা হলে নতুন সার্ভার এলেই একটিমাত্র কমান্ডে তাকে Production-উপযুক্ত প্রাথমিক অবস্থায় আনা যায় — এবং একই Playbook দ্বিতীয়বার চালালেও কিছু বদলায় না, কারণ Ansible-এর প্রতিটি কাজ **Idempotent** — অর্থাৎ একই অবস্থায় একাধিকবার চালালেও ফল একই থাকে। এই অধ্যায়ের শেষে আপনি নিজের ল্যাবে একটি সম্পূর্ণ Baseline Playbook লিখে তিনটি সার্ভারে চালাতে পারবেন।

> **CONCEPT:** Baseline মানে "সব সার্ভারে একই ন্যূনতম অবস্থা"। এটি কোনো অ্যাপ ইনস্টল করা নয় — এটি সেই মাটি তৈরি করা যেখানে অ্যাপ দাঁড়াবে। Baseline-এ সাধারণত থাকে পরিচয় (Hostname, Timezone), প্যাকেজ, Kernel সেটিং, মৌলিক ফাইল ও লগের ব্যবস্থা। Baseline ছাড়া Configuration Management হয় এলোমেলো: প্রতিটি সার্ভার আলাদা ইতিহাস বহন করে, আর সমস্যা হলে কেউ বলতে পারে না "এই সার্ভারটি কী অবস্থায় থাকার কথা ছিল"।

১৭.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Baseline Server Setup বলতে কী বোঝায় তা ব্যাখ্যা করতে এবং TechNova-র জন্য একটি Baseline-এর তালিকা তৈরি করতে; (২) `ansible.builtin.hostname` ও `community.general.timezone` মডিউল দিয়ে Hostname ও Timezone নির্ধারণ করতে; (৩) `apt` মডিউল দিয়ে Cache আপডেট ও প্যাকেজ ইনস্টল করতে এবং Cache-এর বৈধতা (Cache Validity) বোঝাতে; (৪) `sysctl` মডিউল দিয়ে Kernel Parameter স্থায়ীভাবে নির্ধারণ করতে এবং `sysctl -p`-এর সঙ্গে এর পার্থক্য বলতে; (৫) `copy`, `template`, `lineinfile` ও `blockinfile` — চারটি ফাইল-মডিউলের পার্থক্য, ব্যবহারক্ষেত্র ও Idempotency আচরণ ব্যাখ্যা করতে; (৬) MOTD ও অন্যান্য সিস্টেম ফাইল Template থেকে তৈরি করতে; (৭) `--check` ও `--diff` দিয়ে Playbook চালানোর আগে পরিবর্তন যাচাই করতে।

১৭.৩ পূর্বশর্ত ধারণা

এই অধ্যায়ের জন্য আপনার প্রয়োজন: Ansible Core 2.17+ ইনস্টল করা একটি Control Node (ব্যবহারকারী `nahid`, Python 3.12), একটি Inventory ফাইলে অন্তত একটি Ubuntu 24.04 Managed Node (ল্যাব IP যেমন 203.0.113.10), SSH Key দিয়ে Password-ছাড়া লগইন, আর Playbook, Task, Module ও Handler-এর প্রাথমিক ধারণা — যা এই Volume-এর আগের Part-গুলোতে এসেছে। Linux পক্ষ থেকে জানা দরকার: Hostname কী, Timezone কীভাবে কাজ করে (`timedatectl`), `apt` Cache কী, Kernel Parameter (`/proc/sys/`) কী, এবং `/etc/motd` কী। Jinja2 Template-এর প্রাথমিক ধারণা (`{{ variable }}`) থাকলে ভালো, না থাকলে এই অধ্যায়ের উদাহরণ দেখেই বোঝা যাবে। ল্যাবের জন্য তিনটি সার্ভার ব্যবহার করব: `web01 (203.0.113.10)`, `api01 (203.0.113.11)`, `db01 (203.0.113.12)` — সবাই `technova.example` ডোমেইনের অধীনে।

১৭.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি সার্ভারকে ভাবুন একটি নতুন অফিস-কক্ষ হিসেবে। প্রথমে কক্ষটির নামফলক (Hostname) লাগে, ঘড়িটি স্থানীয় সময়ে (Timezone) মেলাতে হয়, দরকারি যন্ত্রপাতি (Package) আনতে হয়, আর ঘরের কিছু নিয়ম — যেমন কতজন একসঙ্গে ঢুকতে পারবে, দরজা কতক্ষণ খোলা থাকবে (Kernel Parameter) — লিখে রাখতে হয়। হাতে করলে প্রতিটি কক্ষ আলাদা হয়; ছাঁচ (Template) ব্যবহার করলে প্রতিটি কক্ষ একই নিয়মে সাজে। Ansible ঠিক এই কাজটিই করে, তবে একটি মূলনীতি মেনে: Ansible কখনো "কী করতে হবে" তার নির্দেশের তালিকা চালায় না, সে বলে "শেষ অবস্থাটি কেমন হবে" — একে বলে **Declarative** পদ্ধতি। আপনি বলেন "Hostname হবে web01", Ansible নিজে পরীক্ষা করে Hostname ইতিমধ্যে web01 কি না; হলে কিছু করে না, না হলে বদলায়। এই পরীক্ষা-তারপর-পরিবর্তন চক্রের নাম **Idempotency**, আর এটাই Configuration Management-কে Shell Script থেকে আলাদা করে। একটি Shell Script দ্বিতীয়বার চালালে একটি লাইন দুবার যোগ হতে পারে, একটি ফাইল দুবার কপি হতে পারে; একটি Idempotent Playbook দ্বিতীয়বার চালালে বলে "ok — কিছু বদলায়নি"। এই অধ্যায়ের প্রতিটি মডিউল এই নীতি মেনে চলে, তবে প্রতিটির Idempotency-র সীমা আলাদা — বিশেষ করে `lineinfile` ও `blockinfile` নিয়ে পরে বিস্তারিত আলোচনা আসবে।

১৭.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Hostname ও Timezone — সার্ভারের পরিচয়

Hostname সার্ভারের নাম; এটি তিন জায়গায় থাকে — Kernel-এর Hostname, `/etc/hostname` ফাইল, ও `/etc/hosts`-এ নিজের নামের রেকর্ড। `ansible.builtin.hostname` মডিউল তিনটি জায়গাই সামলায় এবং `hostnamectl` ব্যবহার করে, তাই হাতে `echo web01 > /etc/hostname` লেখার দরকার নেই — সেটি করলে চলমান Kernel-এর Hostname বদলাত না। Timezone-এর জন্য `community.general.timezone` মডিউল `timedatectl set-timezone Asia/Dhaka`-এর কাজ করে; লগের Timestamp, Cron Job ও SSL সার্টিফিকেটের মেয়াদ — সবকিছু Timezone-এর ওপর নির্ভর করে বলে Baseline-এ এটি বাধ্যতামূলক। মনে রাখবেন: সার্ভারের System Time সবসময় UTC-তে রাখা ও Application-স্তরে স্থানীয় সময় দেখানো একটি ভালো অভ্যাস, তবে TechNova-র ল্যাবে সরলতার জন্য আমরা System Timezone-ই Asia/Dhaka রাখছি এবং NTP-র জন্য `systemd-timesyncd` চালু রাখছি।

apt Cache ও প্যাকেজ — কেন Cache আপডেট আলাদা Task

Ubuntu-তে `apt` প্যাকেজের তালিকা রাখে একটি স্থানীয় Cache-এ (`/var/lib/apt/lists/`)। Cache পুরনো হলে `apt install nginx` বলতে পারে "প্যাকেজ পাওয়া যায়নি", যদিও Repository-তে তা আছে। তাই Baseline-এ প্রথমে `apt: update_cache=yes cache_valid_time=3600` চালানো হয় — `cache_valid_time` মানে Cache এক ঘণ্টার মধ্যে আপডেট হয়ে থাকলে আবার আপডেট করবে না, ফলে Playbook দ্রুত চলে। প্যাকেজ ইনস্টলে `state: present` মানে

"ইনস্টল থাকতে হবে", `state: latest` মানে "সর্বশেষ সংস্করণ চাই" — Baseline-এ সাধারণত `present` ব্যবহার করা হয়, কারণ `latest` প্রতিবার নতুন সংস্করণ টেনে এনে সার্ভারগুলোর মধ্যে পার্থক্য তৈরি করতে পারে। প্রয়োজনীয় Baseline প্যাকেজের একটি ছোট তালিকা: `curl`, `git`, `vim`, `htop`, `ufw`, `fail2ban`, `unattended-upgrades` — এগুলো ডিবাগ ও নিরাপত্তার জন্য দরকার, তবে প্রতিটি প্যাকেজেরই আক্রমণের পৃষ্ঠ (Attack Surface) বাড়ে বলে অপ্রয়োজনীয় কিছু ইনস্টল করা উচিত নয়।

sysctl — Kernel Parameter স্থায়ীভাবে নির্ধারণ

Kernel Parameter হলো Kernel-এর চলমান সেটিং, যেমন `net.ipv4.ip_forward` বা `vm.swappiness`। হাতে `sysctl -w net.ipv4.ip_forward=1` দিলে তা Reboot-এ মুছে যায়; `/etc/sysctl.conf`-এ লিখলে তা থাকে, কিন্তু লেখা ও প্রয়োগ দুটি আলাদা ধাপ। `ansible.builtin.sysctl` মডিউল দুটি কাজই একসঙ্গে করে: মানটি `/etc/sysctl.d/99-technova.conf`-এ লিখে রাখে (`sysctl_file` প্যারামিটার দিয়ে) এবং সঙ্গে সঙ্গে প্রয়োগ করে (`reload: yes`)। Baseline-এ আমরা সাধারণত তিনটি Parameter রাখি: `vm.swappiness=10` (Swap কম ব্যবহার করা — ডেটাবেস সার্ভারে গুরুত্বপূর্ণ), `net.ipv4.ip_forward=0` (সাধারণ সার্ভারে Forwarding বন্ধ), এবং `fs.file-max=65535` (অনেক সংযোগ সামলাতে ফাইল-সীমা বাড়ানো)। প্রতিটি Parameter-এর অর্থ ও ঝুঁকি না বুঝে মান বদলাবেন না — ভুল sysctl মান সার্ভারকে অচল করে দিতে পারে।

চারটি ফাইল-মডিউল — copy, template, lineinfile, blockinfile

`copy` মডিউল Control Node থেকে একটি ফাইল হুবহু Managed Node-এ পাঠায় — কোনো Variable নয়, কোনো পরিবর্তন নয়; যেমন একটি স্থির Banner ফাইল। `template` মডিউল Jinja2 Template প্রসেস করে পাঠায় — ফাইলের ভেতরে `{{ inventory_hostname }}` বা `{{ technova_domain }}` বসানো যায়, তাই প্রতিটি সার্ভারে ফাইলের বিষয়বস্তু আলাদা হতে পারে; MOTD-র জন্য এটিই সঠিক। `lineinfile` একটি ফাইলের একটি মাত্র লাইন নিশ্চিত করে — যেমন `/etc/hosts`-এ একটি এন্ট্রি, বা কোনো Config ফাইলে একটি সেটিং; এটি Regular Expression (`regexp`) দিয়ে পুরনো লাইনটি খুঁজে বদলায়, তাই একই লাইন দুবার যোগ হয় না। `blockinfile` একটি লাইনের বদলে একটি গোটা Block যোগ করে, Marker (`# BEGIN ANSIBLE MANAGED BLOCK` ... `# END ANSIBLE MANAGED BLOCK`) দিয়ে ঘেরা — Marker থাকলে Ansible বুঝতে পারে Block-টি তার নিজের, এবং পরেরবার একই Block আবার যোগ করে না। নিয়মটি সহজ: পুরো ফাইলটি আপনার নিয়ন্ত্রণে থাকলে `template`; ফাইলটি অন্য সফটওয়্যারের, শুধু একটি লাইন বা Block বদলাতে চাইলে `lineinfile` বা `blockinfile`; আর কখনোই ফাইল এডিট করতে `shell` দিয়ে `sed` বা `echo >>` ব্যবহার করবেন না — তাতে Idempotency ভেঙে যায়।

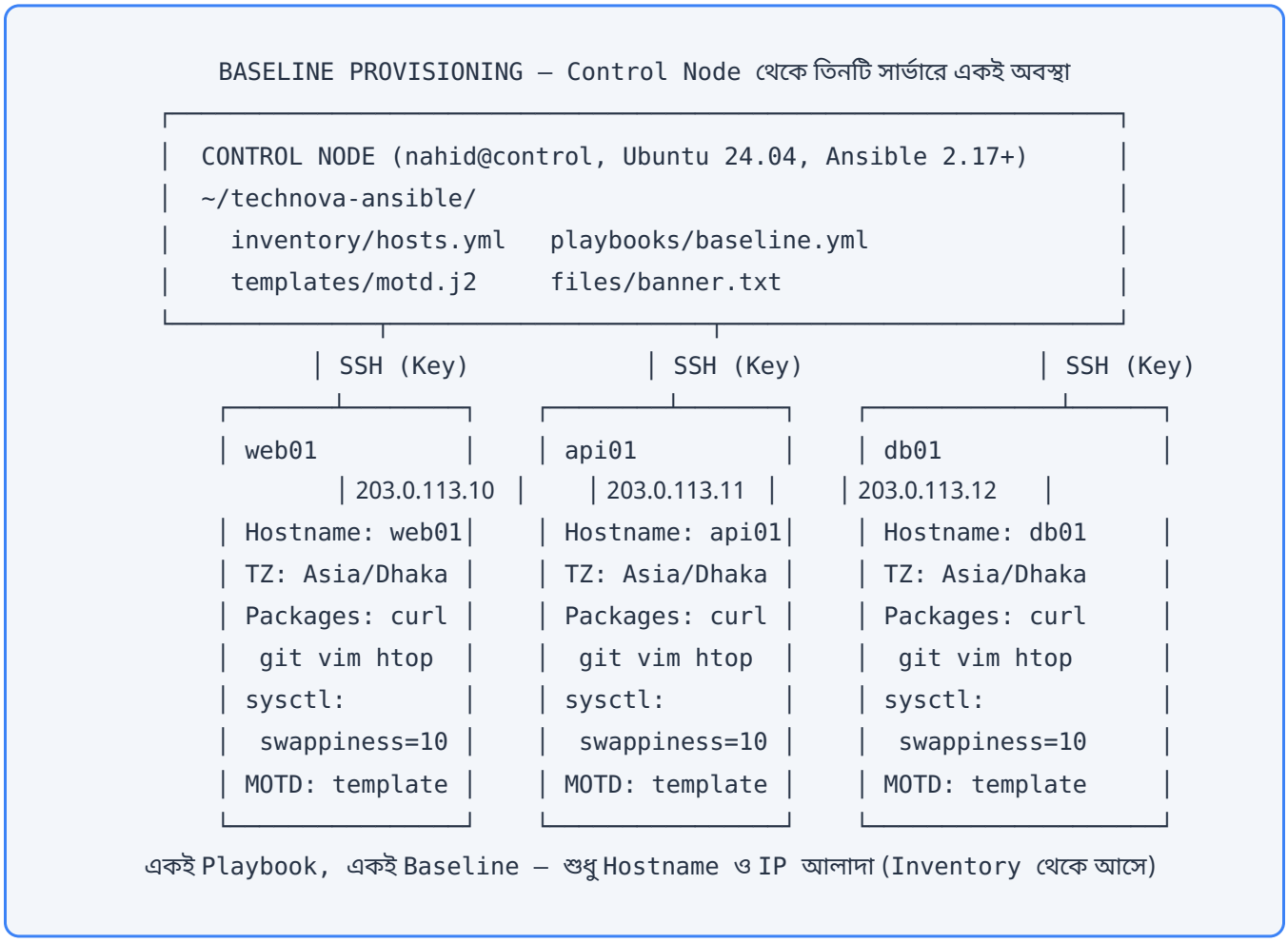
মডিউল	কী করে	কখন ব্যবহার করবেন	Idempotency
`copy`	ফাইল হুবহু পাঠায়	স্থির ফাইল, কোনো Variable নেই	Checksum মিললে কিছু করে না
`template`	Jinja2 প্রসেস করে পাঠায়	প্রতি হোস্টে বিষয়বস্তু আলাদা হবে	Rendered ফলাফল মিললে কিছু করে না
`lineinfile`	একটি লাইন নিশ্চিত করে	অন্যের ফাইলে একটি সেটিং	regexp মিললে লাইন বদলায়, না মিললে যোগ করে
`blockinfile`	Marker-ঘেরা Block নিশ্চিত করে	অন্যের ফাইলে একাধিক লাইনের অংশ	Marker দেখে Block চেনে, দুবার যোগ করে না

১৭.৬ অভ্যন্তরীণ কার্যপ্রণালী

Playbook চালালে Control Node প্রথমে Inventory পড়ে, তারপর প্রতিটি Managed Node-এ SSH দিয়ে Python স্ক্রিপ্ট আকারে Module পাঠায় — একে বলে Module Execution। `template`-এর ক্ষেত্রে কাজটি দুই ধাপে হয়: প্রথমে Control Node-এ Jinja2 Template-টি Variable দিয়ে Render করা হয়, তারপর Rendered ফাইলটি Managed Node-এ পাঠানো হয় এবং Checksum তুলনা করা হয় — মিললে Task "ok", না মিললে "changed"। `lineinfile`-এর ক্ষেত্রে Module Managed Node-এ ফাইলটি পড়ে, `regexp` দিয়ে লাইনটি খোঁজে, তারপর সিদ্ধান্ত নেয়: লাইনটি ইতিমধ্যে সঠিক থাকলে কিছু করে না; লাইনটি ভিন্ন মান নিয়ে থাকলে সেই লাইনটি বদলায়; লাইনটি না থাকলে ফাইলের শেষে যোগ করে। `sysctl` মডিউল প্রথমে `/proc/sys/…` থেকে বর্তমান মান পড়ে, তারপর `sysctl_file`-এ মানটি লেখে ও `sysctl -w` দিয়ে প্রয়োগ করে। `apt` মডিউল প্রথমে Cache-এর সময় দেখে — `cache_valid_time`-এর মধ্যে আপডেট হয়ে থাকলে `apt-get update` এড়িয়ে যায়, ফলে Playbook দ্রুত ও নেটওয়ার্ক-সাশ্রয়ী হয়। সব Module-ই শেষে একটি JSON ফলাফল ফেরত পাঠায় যাতে থাকে `changed` (সত্যি কিছু বদলেছে কি না) ও `failed` — Ansible এই ফলাফল দেখেই রঙিন সারাংশ দেখায়: সবুজ মানে ok, হলুদ মানে changed, লাল মানে failed। একটি গুরুত্বপূর্ণ বিষয়: `check_mode` (`--check`) চালালে Module-গুলো আসল পরিবর্তন না করে শুধু জানায় "কী বদলাত" — তবে `command`/`shell` মডিউল Check Mode-এ আসলেই চলে, তাই Baseline-এ সেগুলো এড়ানোই ভালো।

> **TIP:** প্রতিটি Baseline Playbook-এ `gather_facts: true` রাখুন — Ansible তখন প্রতিটি হোস্টের `ansible_hostname`, `ansible_distribution`, `ansible_default_ipv4.address`-এর মতো তথ্য সংগ্রহ করে, যা Template-এ ব্যবহার করা যায়। তবে Fact সংগ্রহে সময় লাগে; অনেক হোস্টে `gather_subset` দিয়ে শুধু দরকারি Fact নেওয়া যায়।

১৭.৭ আর্কিটেকচার ও চিত্র



চিত্রটির মূল কথা: Baseline-এর সব তথ্যের উৎস দুটি — Inventory (কোন সার্ভার, কোন IP, কোন গ্রুপ) ও Variable (কোন Timezone, কোন প্যাকেজ, কোন sysctl মান)। Playbook নিজে কোনো সার্ভারের নাম হার্ডকোড করে না; সে Inventory থেকে নেয়। নতুন সার্ভার যোগ করতে শুধু Inventory-তে একটি লাইন যোগ করলেই হয় — Playbook বদলাতে হয় না।

১৭.৮ ব্যবহারিক উদাহরণ

প্রথমে Inventory ও Variable ফাইল তৈরি করুন। Inventory-তে তিনটি হোস্ট ও তাদের গ্রুপ, আর `group_vars/all.yml`-এ Baseline-এর সব মান — যাতে মান বদলাতে Playbook ছুঁতে না হয়:

```
# inventory/hosts.yml — TechNova ল্যাব Inventory
all:
  children:
    webservers:
      hosts:
        web01:
          ansible_host: 203.0.113.10
    apiservers:
      hosts:
        api01:
          ansible_host: 203.0.113.11
    dbservers:
```

```
      hosts:
        db01:
          ansible_host: 203.0.113.12
  vars:
    ansible_user: nahid
    ansible_ssh_private_key_file: ~/.ssh/technova_lab
    technova_domain: technova.example

# group_vars/all.yml — Baseline-এর সব মান এক জায়গায়
technova_timezone: Asia/Dhaka
technova_baseline_packages:
  - curl
  - git
  - vim
  - htop
  - ufw
  - fail2ban
  - unattended-upgrades
technova_sysctl:
  vm.swappiness: 10
  net.ipv4.ip_forward: 0
  fs.file-max: 65535
```

এরপর MOTD Template — `templates/motd.j2` — যেখানে প্রতিটি সার্ভারের নিজের Hostname ও IP বসবে:

```
# templates/motd.j2 — Jinja2 Template; {{ }}-এর ভেতর Variable বসে
Welcome to TechNova — {{ inventory_hostname }}.{{ technova_domain }}
Managed by Ansible — do not edit this file by hand.
IP: {{ ansible_default_ipv4.address | default('203.0.113.x') }}
Timezone: {{ technova_timezone }}
Environment: lab (203.0.113.0/24)
```

আর `/etc/hosts`-এ সব সার্ভারের নাম যোগ করতে `lineinfile`-এর সঙ্গে Loop ব্যবহার করুন — Loop-এর প্রতিটি Item একটি লাইন নিশ্চিত করে, তাই Playbook দ্বিতীয়বার চালালে লাইনগুলো দুবার যোগ হয় না:

```
# /etc/hosts-এ ল্যাবের তিনটি সার্ভারের এন্ট্রি (lineinfile + loop)
- name: Ensure lab hosts are in /etc/hosts
  ansible.builtin.lineinfile:
    path: /etc/hosts
    regexp: "^{{ item.ip }}\\s+{{ item.name }}\\."
    line: "{{ item.ip }} {{ item.name }}.{{ technova_domain }} {{ item.name }}"
    state: present
  loop:
    - { name: web01, ip: 203.0.113.10 }
    - { name: api01, ip: 203.0.113.11 }
    - { name: db01, ip: 203.0.113.12 }
```

১৭.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচে Baseline-এর সম্পূর্ণ Playbook — `playbooks/baseline.yml`। এটি একটি ফাইলেই Hostname, Timezone, apt Cache, প্যাকেজ, sysctl, MOTD, `/etc/hosts` ও একটি স্বাগত-বার্তা (Banner) — সবকিছু করে। কোনো Secret এখানে নেই; শুধু Configuration মান আছে।

```
---
# playbooks/baseline.yml — TechNova Baseline Server Setup
# চালানোর কমান্ড: ansible-playbook -i inventory/hosts.yml playbooks/baseline.yml
- name: Baseline setup for all TechNova servers
  hosts: all
  become: true
  gather_facts: true

  tasks:
    - name: Set hostname from inventory name
      ansible.builtin.hostname:
        name: "{{ inventory_hostname }}"

    - name: Set timezone to Asia/Dhaka
      community.general.timezone:
        name: "{{ technova_timezone }}"

    - name: Update apt cache if older than one hour
      ansible.builtin.apt:
        update_cache: true
        cache_valid_time: 3600

    - name: Install baseline packages
      ansible.builtin.apt:
        name: "{{ technova_baseline_packages }}"
        state: present

    - name: Apply kernel parameters (sysctl)
      ansible.builtin.sysctl:
        name: "{{ item.key }}"
        value: "{{ item.value }}"
        sysctl_file: /etc/sysctl.d/99-technova.conf
        state: present
        reload: true
      loop: "{{ technova_sysctl | dict2items }}"

    - name: Deploy MOTD from template
      ansible.builtin.template:
        src: motd.j2
        dest: /etc/motd
        owner: root
        group: root
        mode: "0644"

    - name: Ensure lab hosts are in /etc/hosts
      ansible.builtin.lineinfile:
        path: /etc/hosts
        regexp: "^{{ item.ip }}\\s+{{ item.name }}\\."
        line: "{{ item.ip }} {{ item.name }}.{{ technova_domain }} {{ item.name }}"
        state: present
      loop:
        - { name: web01, ip: 203.0.113.10 }
        - { name: api01, ip: 203.0.113.11 }
        - { name: db01, ip: 203.0.113.12 }

    - name: Add managed banner block to /etc/profile.d
      ansible.builtin.blockinfile:
        path: /etc/profile.d/technova.sh
```

```
        create: true
        mode: "0644"
        block: |
          # TechNova lab server — managed by Ansible
          echo "TechNova lab — managed by Ansible. Do not edit by hand."
        marker: "# {mark} ANSIBLE MANAGED BLOCK - technova"
```

১৭.১০ ধাপে ধাপে বাস্তবায়ন

1. Control Node-এ প্রজেক্ট কাঠামো তৈরি করুন: `mkdir -p ~/technova-ansible/{inventory,playbooks,templates,group_vars}` এবং উপরের Inventory, `group_vars/all.yml` ও `templates/motd.j2` ফাইলগুলো লিখুন।
2. সংযোগ যাচাই করুন: `ansible all -i inventory/hosts.yml -m ping` — প্রতিটি হোস্টে `SUCCESS` আসতে হবে; না এলে SSH Key ও `ansible_host` পরীক্ষা করুন।
3. Syntax যাচাই করুন: `ansible-playbook --syntax-check playbooks/baseline.yml`।
4. Dry Run করুন: `ansible-playbook -i inventory/hosts.yml playbooks/baseline.yml --check --diff` — কী বদলাবে তা দেখুন, কিন্তু কিছু বদলাবে না।
5. আসল Run করুন: `ansible-playbook -i inventory/hosts.yml playbooks/baseline.yml`। প্রথম Run-এ অনেক Task "changed" দেখাবে — এটাই স্বাভাবিক।
6. দ্বিতীয়বার একই কমান্ড চালান — এবার প্রায় সব Task "ok" দেখাবে; এটিই Idempotency-র প্রমাণ। কোনো Task বারবার "changed" দেখালে তার কারণ খুঁজুন (১৭.১২ দেখুন)।
7. একটি সার্ভারে লগইন করে যাচাই করুন: `hostname`, `timedatectl`, `cat /etc/motd`, `sysctl vm.swappiness` ও `dpkg -l | grep -E 'curl|git|htop'` চালান।

১৭.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

প্রথম Run-এর সারাংশটি দেখতে এরকম হবে (চিত্রণমূলক আউটপুট):

```
$ ansible-playbook -i inventory/hosts.yml playbooks/baseline.yml
PLAY [Baseline setup for all TechNova servers] ****************************

TASK [Set hostname from inventory name] ***********************************
changed: [web01]
changed: [api01]
changed: [db01]

TASK [Set timezone to Asia/Dhaka] *****************************************
changed: [web01]
changed: [api01]
changed: [db01]

TASK [Update apt cache if older than one hour] ****************************
changed: [web01]

TASK [Install baseline packages] ******************************************
changed: [web01]
ok: [api01]

TASK [Apply kernel parameters (sysctl)] ***********************************
```

```
changed: [web01]

TASK [Deploy MOTD from template] ***************************************
changed: [web01]

PLAY RECAP *************************************************************
web01 : ok=8  changed=7  unreachable=0  failed=0
api01 : ok=8  changed=6  unreachable=0  failed=0
db01  : ok=8  changed=6  unreachable=0  failed=0
```

ব্যাখ্যা: `PLAY RECAP`-এ প্রতিটি হোস্টের হিসাব থাকে — `ok` মানে Task চলেছে ও অবস্থা ঠিক ছিল, `changed` মানে Task সত্যিই কিছু বদলেছে, `unreachable` মানে SSH সংযোগ হয়নি, `failed` মানে Task ভুল করেছে। লক্ষ্য করুন `api01`-এ প্যাকেজ Task "ok" — কারণ সেই সার্ভারে প্যাকেজগুলো আগে থেকেই ছিল; Ansible অপ্রয়োজনে কিছু করেনি। দ্বিতীয় Run-এ `changed=0` হওয়া উচিত — না হলে কোনো Task Idempotent নয়, সেটি ১৭.১২-এর ভুলগুলোর একটি।

১৭.১২ সাধারণ ভুল

- **`shell` দিয়ে ফাইল এডিট করা:** `echo "203.0.113.10 web01" >> /etc/hosts` প্রতিবার চালালে লাইনটি বারবার যোগ হয়। সমাধান: `lineinfile` ব্যবহার করুন।
- **`regexp` না দিয়ে `lineinfile` চালানো:** Regexp না থাকলে Module শুধু লাইনটির উপস্থিতি দেখে; পুরনো মান ভিন্ন হলেও তা বদলায় না, ফলে ফাইলে দুটি লাইন জমে যায়। সবসময় সুনির্দিষ্ট `regexp` দিন।
- **`blockinfile`-এর Marker বদলে ফেলা:** Marker বদলালে Ansible পুরনো Block চিনতে পারে না এবং নতুন Block যোগ করে — ফাইলে একই Block দুবার জমে। Marker একবার ঠিক করে আর বদলাবেন না।
- **`apt`-এ `state: latest` অকারণে ব্যবহার:** প্রতিবার নতুন সংস্করণ এনে সার্ভারগুলোর মধ্যে পার্থক্য তৈরি করে। Baseline-এ `present` ব্যবহার করুন; আপগ্রেডের জন্য আলাদা, পরিকল্পিত Playbook চালান।
- **Hostname-এ Inventory নামের বদলে হার্ডকোড করা মান:** Playbook-এ `name: web01` লিখলে তা সব সার্ভারে একই Hostname বসিয়ে দেয়। সবসময় `{{ inventory_hostname }}` ব্যবহার করুন।
- **Template-এ `ansible_default_ipv4` Fact না থাকা:** Fact সংগ্রহ বন্ধ (`gather_facts: false`) থাকলে Template Render-এ Variable না পেয়ে ভুল করে। হয় Fact চালু রাখুন, নয়তো `default()` ফিল্টার দিন।
- **sysctl মান না বুঝে কপি করা:** ইন্টারনেট থেকে নেওয়া sysctl মান অন্ধভাবে বসালে নেটওয়ার্ক বা মেমরি-আচরণ বদলে যেতে পারে। প্রতিটি মানের অর্থ ও ঝুঁকি যাচাই করে তবেই Baseline-এ রাখুন।

১৭.১৩ সমস্যা সমাধান (Troubleshooting)

1. **লক্ষণ:** `UNREACHABLE! – Failed to connect to the host via ssh` — **পরীক্ষা:** `ssh -i ~/.ssh/technova_lab nahid@203.0.113.10` হাতে চালান। **সমাধান:** Key-এর Permission (600) ও `ansible_host` ঠিক আছে কি না দেখুন; সার্ভারে `nahid` ব্যবহারকারীর `~/.ssh/authorized_keys` পরীক্ষা করুন।
2. **লক্ষণ:** `Permission denied (publickey)` — **পরীক্ষা:** Control Node-এ Key ফাইলের পথ ও সার্ভারে Key-এর উপস্থিতি। **সমাধান:** `ssh-copy-id` দিয়ে Key আবার পাঠান; Key ফাইলের Owner ও Permission ঠিক করুন।
3. **লক্ষণ:** `become` Task-এ `sudo: a password is required` — **পরীক্ষা:** সার্ভারে `sudo -n true` চালান। **সমাধান:** `nahid`-কে Password-ছাড়া sudo দিতে হবে (অধ্যায় ১৯-এ বিস্তারিত), অথবা `ansible-playbook --ask-become-pass` ব্যবহার করুন।

4. **লক্ষণ:** দ্বিতীয় Run-ও কোনো Task "changed" দেখায় — **পরীক্ষা:** `--diff` দিয়ে দেখুন কী বদলাচ্ছে। **সমাধান:** Template-এ Timestamp বা Random মান থাকলে প্রতিবার ফলাফল বদলায় — Template থেকে সময়-নির্ভর মান সরান; `lineinfile`-এর `regexp` সঠিক কি না দেখুন।
5. **লক্ষণ:** `apt` Task-এ `Failed to lock /var/lib/dpkg/lock` — **পরীক্ষা:** সার্ভারে অন্য কোনো `apt` প্রসেস চলছে কি না (`ps aux | grep apt`)। **সমাধান:** `unattended-upgrades` বা অন্য প্রসেস শেষ হওয়া পর্যন্ত অপেক্ষা করুন, তারপর আবার চালান।
6. **লক্ষণ:** `community.general.timezone` মডিউল পাওয়া যায়নি — **পরীক্ষা:** `ansible-galaxy collection list | grep community.general`। **সমাধান:** `ansible-galaxy collection install community.general` চালান; Collection-এর সংস্করণ Ansible Core 2.17-এর সঙ্গে সামঞ্জস্যপূর্ণ কি না দেখুন।

১৭.১৪ নিরাপত্তা বিবেচনা

> **WARNING:** Baseline Playbook-এ কখনোই Password, API Key বা Private Key লিখবেন না — এমনকি "শুধু ল্যাবের জন্য" বলেও নয়। ল্যাবে লেখা অভ্যাসই প্রোডাকশনে গিয়ে দুর্ঘটনা ঘটায়। Secret দরকার হলে Ansible Vault ব্যবহার করুন (এই Volume-এর আগের Part-এ বর্ণিত), আর Repository-তে কখনোই Plain-Text Secret Commit করবেন না। `git` History থেকে Secret মোছা অত্যন্ত কঠিন — একবার Commit হলে তা চিরদিনের জন্য ফাঁস বলে ধরে নিন।

Baseline-এর নিরাপত্তা-দিকগুলো: প্রথমত, শুধু প্রয়োজনীয় প্যাকেজ ইনস্টল করুন — প্রতিটি বাড়তি প্যাকেজ একটি বাড়তি আক্রমণের পৃষ্ঠ। দ্বিতীয়ত, `/etc/motd` বা Banner-এ সার্ভারের সংস্করণ, Kernel বা অভ্যন্তরীণ তথ্য বেশি লিখবেন না — MOTD লগইনকারী যে কেউ দেখে। তৃতীয়ত, sysctl-এ `net.ipv4.ip_forward=0` রাখা মানে সার্ভারটি Router হিসেবে ব্যবহৃত হবে না — এটি ডিফল্ট-নিরাপদ অবস্থা। চতুর্থত, Playbook ফাইলগুলোর Permission পরীক্ষা করুন — Inventory-তে Private Key-এর পথ থাকলেও Key ফাইলটি যেন শুধু `nahid` পড়তে পারে (600)। পঞ্চমত, Baseline চালানোর আগে `--check --diff` দিয়ে পরিবর্তন দেখে নেওয়া একটি নিরাপত্তা-অভ্যাস: ভুল Variable-এর কারণে ভুল সার্ভারে ভুল Hostname বসে যাওয়া থেকে এটি বাঁচায়।

১৭.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

প্রোডাকশনে Baseline মানে শুধু একটি Playbook নয় — একটি প্রক্রিয়া। প্রথমত, Baseline Playbook-টি Git Repository-তে রাখুন, প্রতিটি পরিবর্তন Pull Request ও Review-এর মধ্য দিয়ে যাক, এবং প্রতিটি Run-এর `PLAY RECAP` সংরক্ষণ করুন — কে, কখন, কোন সার্ভারে কী বদলেছে, তার হিসাব (Audit Trail) ছাড়া প্রোডাকশন চলে না। দ্বিতীয়ত, Baseline-কে ভূমিকা (Role) অনুযায়ী ভাগ করুন: সব সার্ভারের জন্য `common` Role, আর Web, API, Database-এর জন্য আলাদা Role — এতে নতুন ধরনের সার্ভার যোগ করা সহজ হয়। তৃতীয়ত, Baseline নিয়মিত চালান (যেমন সাপ্তাহিক) — একে বলে **Drift Correction**: কেউ হাতে সার্ভারে কিছু বদলে ফেললে পরের Run-এ তা আবার Baseline-এ ফিরে আসে। চতুর্থত, Staging-এ প্রথমে চালিয়ে তারপর Production-এ চালান; কখনোই প্রথমবার সরাসরি Production-এ নতুন Baseline চালাবেন না। পঞ্চমত, `cache_valid_time`-এর মতো মানগুলো পরিবেশ অনুযায়ী ঠিক করুন — ধীর নেটওয়ার্কে Cache বেশি সময় বৈধ রাখা যায়, দ্রুত পরিবর্তনশীল পরিবেশে কম। সবশেষে, Baseline-এর প্রতিটি মানের পাশে একটি মন্তব্য (Comment) লিখুন — ছয় মাস পরে কেউ যেন বুঝতে পারে `vm.swappiness=10` কেন রাখা হয়েছিল।

১৭.১৬ অধ্যায়ের সারাংশ

Baseline হলো প্রতিটি সার্ভারের অভিন্ন, যাচাইযোগ্য প্রাথমিক অবস্থা — Hostname, Timezone, প্যাকেজ, Kernel Parameter ও মৌলিক ফাইল। Ansible এই অবস্থা Declarative ও Idempotentভাবে তৈরি করে: অবস্থা ঠিক থাকলে কিছু করে না, না থাকলে ঠিক করে। `hostname` ও `timezone` মডিউল পরিচয় নির্ধারণ করে, `apt` মডিউল Cache-এর বৈধতা মেনে প্যাকেজ ইনস্টল করে, `sysctl` মডিউল Kernel Parameter স্থায়ীভাবে লেখে ও প্রয়োগ করে, আর `copy`, `template`, `lineinfile`, `blockinfile` — চারটি ফাইল-মডিউল পুরো ফাইল থেকে একটিমাত্র লাইন পর্যন্ত সব স্তরে কাজ করে। `--check --diff` দিয়ে পরিবর্তন আগেই দেখা যায়, আর দ্বিতীয় Run-এ `changed=0` হওয়াই Idempotency-র প্রমাণ। এই Baseline-এর ওপরেই পরের অধ্যায়গুলো অ্যাপ, নিরাপত্তা ও ক্লাউড-কনফিগারেশন বসাবে।

১৭.১৭ জ্ঞান যাচাই

1. Baseline বলতে কী বোঝায়, এবং কেন প্রতিটি সার্ভারে একই Baseline থাকা জরুরি?
2. `apt` মডিউলে `cache_valid_time: 3600`-এর অর্থ কী? এটি না থাকলে কী সমস্যা হতো?
3. `sysctl -w` দিয়ে মান বদলানো আর `ansible.builtin.sysctl` মডিউল ব্যবহারের পার্থক্য কী?
4. একটি ফাইলের শুধু একটি লাইন বদলাতে চাইলে `template`-এর বদলে `lineinfile` কেন ভালো? দুটি ঝুঁকি লিখুন যা `template` ব্যবহারে হতো।
5. দ্বিতীয় Run-এ একটি Task বারবার "changed" দেখালে আপনি প্রথমে কী পরীক্ষা করবেন, এবং কেন?

১৭.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. **নিজের Baseline:** ল্যাবে একটি Ubuntu 24.04 সার্ভারে উপরের সম্পূর্ণ Playbook চালান; প্রথম ও দ্বিতীয় Run-এর `PLAY RECAP` সংরক্ষণ করুন এবং দ্বিতীয় Run-এ `changed=0` হয়েছে কি না লিখুন।
2. **ভাঙা Idempotency খুঁজুন:** ইচ্ছে করে `lineinfile` Task থেকে `regexp` সরিয়ে Playbook দুবার চালান; `/etc/hosts`-এ কী ঘটে তা লিখুন, তারপর `regexp` ফিরিয়ে ঠিক করুন।
3. **নতুন Role:** Baseline-কে একটি Ansible Role-এ (`roles/common/`) রূপান্তর করুন — Tasks, Templates ও Variables আলাদা ফাইলে সাজিয়ে Playbook থেকে Role-টি কল করুন।

অধ্যায় 18 • PART IV — Application Deployment

Chapter 18 — Nginx, Node.js, Database, and Application Deployment: MERN App Deployment

১৮.১ ভূমিকা

Baseline তৈরি হলে এবার সার্ভারে আসল কাজটি বসে: অ্যাপ্লিকেশন চালানো। TechNova-র MERN অ্যাপ — MongoDB, Express, React/Next.js, Node.js — চারটি স্তরে চলে: ব্রাউজার প্রথমে Nginx-কে ডাকে, Nginx অনুরোধটি Node.js-এর Express API-তে পাঠায় (Reverse Proxy), API MongoDB থেকে ডেটা নিয়ে উত্তর দেয়, আর React ফ্রন্টএন্ড ব্যবহারকারীর পর্দায় দেখায়। হাতে-কলমে এই Deploy করতে গেলে ধাপগুলো অনেক: Node.js ইনস্টল, কোড টানা, `npm install`, Environment Variable বসানো, systemd Service লেখা, Nginx Config লেখা, ডেটাবেস চালু করা, তারপর সবকিছু Restart করে দেখা আদৌ চলছে কি না। একটি ধাপ ভুল হলে অ্যাপ বন্ধ থাকে, আর কেউ বলতে পারে না কোন ধাপে ভুল হয়েছে। এই অধ্যায়ে আমরা পুরো Deploy-টি একটি Ansible Playbook-এ বাঁধব — Nginx-এর Config Template থেকে, Node.js 20 ইনস্টল, Express API-র systemd Service Unit, MongoDB বা PostgreSQL-এর ইনস্টল ও কনফিগারেশন, Environment ফাইল, Health Check, এবং Zero-Downtime Restart-এর প্যাটার্ন পর্যন্ত। লক্ষ্য শুধু "অ্যাপ চলছে" নয় — লক্ষ্য হলো অ্যাপটি এমনভাবে চলা যাতে নতুন সংস্করণ এলে পুরনোটি বন্ধ না করে নতুনটি চালু করা যায়, কোনো অনুরোধ হারিয়ে না যায়, আর কোনো ধাপ ব্যর্থ হলে তা পরিষ্কারভাবে ধরা পড়ে।

> **CONCEPT:** Reverse Proxy মানে ব্যবহারকারী সরাসরি অ্যাপ-সার্ভারের সঙ্গে কথা বলে না — মাঝখানে Nginx বসে। Nginx বাইরের 80/443 Port-এ শোনে, আর ভেতরের 4000 Port-এ চলা Express API-তে অনুরোধ পাঠায়। এর ফলে API-কে ইন্টারনেটে খোলা রাখতে হয় না, একটিমাত্র জায়গায় TLS (HTTPS) শেষ করা যায়, Static ফাইল দ্রুত দেওয়া যায়, আর পরে একাধিক API সার্ভারে Load ভাগ করা যায়। Nginx বন্ধ থাকলে অ্যাপ চললেও ব্যবহারকারী কিছু দেখতে পায় না — তাই Deploy-এ Nginx ও API — দুটিকেই যাচাই করতে হয়।

১৮.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) MERN অ্যাপের চারটি স্তর (Nginx, Node.js/Express, ডেটাবেস, ফ্রন্টএন্ড) ও তাদের মধ্যে অনুরোধের প্রবাহ ব্যাখ্যা করতে; (২) Template দিয়ে Nginx Reverse Proxy Config তৈরি করতে ও `nginx -t` দিয়ে যাচাই করতে; (৩) Node.js 20 ইনস্টল করতে (NodeSource Repository বা Package Manager পদ্ধতিতে) এবং সংস্করণ পিন করার গুরুত্ব বলতে; (৪) Express API-র জন্য systemd Service Unit লিখতে ও `systemctl` দিয়ে পরিচালনা করতে; (৫) MongoDB বা PostgreSQL ইনস্টল ও প্রাথমিক কনফিগার করতে এবং দুটির মধ্যে কখন কোনটি বেছে নেবেন তা বলতে; (৬) Environment ফাইল (`.env`) নিরাপদে পরিচালনা করতে; (৭) Health Check Endpoint দিয়ে Deploy যাচাই করতে; (৮) Zero-Downtime Restart-এর প্যাটার্ন ব্যাখ্যা করতে ও একটি সরল রূপ বাস্তবায়ন করতে।

১৮.৩ পূর্বশর্ত ধারণা

অধ্যায় ১৭-এর Baseline Playbook আপনার ল্যাবে চালানো থাকতে হবে — Hostname, Timezone ও প্যাকেজ ঠিক থাকা চাই। আরও প্রয়োজন: systemd Service-এর প্রাথমিক ধারণা (`systemctl start/enable/status`), Nginx-এর প্রাথমিক ধারণা (Server Block, `proxy_pass`), Node.js ও `npm`-এর অভিজ্ঞতা, MongoDB বা PostgreSQL-এর যেকোনো একটির প্রাথমিক ধারণা, এবং একটি MERN অ্যাপের Repository — TechNova-র ল্যাবে আমরা `https://git.technova.example/technova/mern-app.git` (কাল্পনিক) ব্যবহার করব, যার ভেতরে `server/` (Express API,

Port 4000) ও `client/` (React ফ্রন্টএন্ড) আছে। Environment Variable-এর ধারণা (`PORT`, `DATABASE_URL`, `JWT_SECRET`) জানা থাকতে হবে; Secret-এর মান কখনো Playbook-এ লেখা যাবে না — Ansible Vault বা Environment-এর মাধ্যমে দিতে হবে।

১৮.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি ওয়েব অনুরোধকে ভাবুন একটি চিঠি হিসেবে। চিঠিটি প্রথমে পৌঁছায় বাড়ির দরজায় — সেটি Nginx। দরজার রক্ষী চিঠিটি পড়ে দেখে কোন ঘরে যাবে: `/api/` দিয়ে শুরু হলে API-র ঘরে (Express, Port 4000), অন্য কিছু হলে ফ্রন্টএন্ডের ঘরে (React, Port 3000)। API-র ঘরে গিয়ে চিঠিটি আরও এক ধাপ যায় — ডেটাবেসের কাছে (MongoDB, Port 27017) — ডেটা এনে উত্তর লেখে, আর সেই উত্তর একই পথে ফিরে আসে। এই পুরো পথটির প্রতিটি ধাপ একটি আলাদা Process: Nginx একটি Process, Node.js আরেকটি, MongoDB আরেকটি। কোনো Process মারা গেলে তার পরের ধাপটি ভেঙে যায় — তাই প্রতিটি Process-কে এমনভাবে চালাতে হয় যাতে সে মারা গেলে আবার চালু হয় (systemd-এর `Restart=always`), আর চালু হয়েছে কি না তা পরীক্ষা করা যায় (Health Check)। Deploy মানে আসলে এই তিনটি Process-কে সঠিক Config, সঠিক Environment ও সঠিক ক্রমে চালু করা: প্রথমে ডেটাবেস, তারপর API, তারপর Nginx — কারণ API ডেটাবেস ছাড়া Health Check-এ ফেল করবে, আর Nginx API ছাড়া 502 (Bad Gateway) দেবে। Ansible এই ক্রমটি Playbook-এর Task-এর ক্রম দিয়ে নিশ্চিত করে, আর Handler দিয়ে শুধু তখনই Restart করে যখন Config সত্যিই বদলেছে।

১৮.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Nginx Reverse Proxy — Template দিয়ে Server Block

Nginx-এর Config থাকে `/etc/nginx/sites-available/`-এ, আর চালু হয় `sites-enabled/`-এর Symlink দিয়ে। Ansible-এ আমরা Template (`technova.conf.j2`) দিয়ে Config লিখি, যাতে Domain (`app.technova.example`) ও API Port Variable থেকে আসে। মূল নির্দেশগুলো: `listen 80`, `server_name app.technova.example`, `location /api/ { proxy_pass http://127.0.0.1:4000; }`, আর `proxy_set_header` দিয়ে আসল IP ও Host API-তে পাঠানো — না হলে API সব অনুরোধ একই IP থেকে আসছে বলে মনে করবে, আর Rate Limit ও লগ ভুল হবে। Config বদলের পর কখনোই সরাসরি `systemctl restart nginx` দেবেন না — প্রথমে `nginx -t` দিয়ে Syntax যাচাই করুন; Ansible-এ এটি একটি Task হিসেবে চলে, আর ভুল Config হলে Playbook সেখানেই থেমে যায়, ফলে ভাঙা Config নিয়ে Nginx Restart হয় না। TLS (HTTPS) এই অধ্যায়ের মূল বিষয় নয়, তবে প্রোডাকশনে Port 443 ও সার্টিফিকেট বাধ্যতামূলক — সার্টিফিকেটের ব্যবস্থা (যেমন Let's Encrypt) আলাদা অধ্যায়ের বিষয়।

Node.js 20 ইনস্টল — সংস্করণ পিন করা

Ubuntu 24.04-এর ডিফল্ট Repository-তে Node.js-এর যে সংস্করণ থাকে তা TechNova-র অ্যাপের দরকারি সংস্করণের সঙ্গে নাও মিলতে পারে। তাই আমরা NodeSource Repository যোগ করে Node.js 20 ইনস্টল করি — `apt_repository` মডিউল দিয়ে Repository যোগ করা হয়, তারপর `apt` দিয়ে `nodejs` প্যাকেজ ইনস্টল। গুরুত্বপূর্ণ: সংস্করণটি Variable-এ রাখুন (`nodejs_version: "20"`), যাতে পরে 22-এ যেতে শুধু একটি মান বদলাতে হয়। `npm install` চালাতে `npm` মডিউল বা `command` ব্যবহার করা যায়, তবে Production-এ `npm ci` ভালো — এটি `package-lock.json` অনুযায়ী হুবহু একই নির্ভরতা ইনস্টল করে, যেখানে `npm install` নতুন সংস্করণ টেনে আনতে পারে। `npm ci` চালানোর আগে কোডটি `git` মডিউল দিয়ে টানতে হয় — `git` মডিউল Repository Clone ও নির্দিষ্ট Branch/Tag Checkout — দুটিই সামলায়, আর Idempotent: কোড ইতিমধ্যে সেই Commit-এ থাকলে কিছু করে না।

systemd Service Unit — API-কে একটি সার্ভিস হিসেবে চালানো

Node.js অ্যাপকে টার্মিনালে `node server.js` দিয়ে চালালে টার্মিনাল বন্ধ হলেই অ্যাপ মারা যায়। systemd Service Unit (`/etc/systemd/system/technova-api.service`) অ্যাপকে একটি সার্ভিস বানায়: `[Unit]`-এ বর্ণনা ও নির্ভরতা (`After=network.target mongod.service`), `[Service]`-এ চালানোর ব্যবহারকারী (`User=technova` — কখনোই Root দিয়ে অ্যাপ চালাবেন না), Working Directory, Environment ফাইল (`EnvironmentFile=/opt/technova/api/.env`), ExecStart (`/usr/bin/node server.js`), আর `Restart=always` ও `RestartSec=5` — মারা গেলে ৫ সেকেন্ড পর আবার চালু হবে। `[Install]`-এ `WantedBy=multi-user.target` মানে Boot-এ চালু হবে। Service ফাইল বদলালে `systemctl daemon-reload` দিতে হয় — Ansible-এ এটি Handler হিসেবে চলে, শুধু তখনই যখন Template সত্যিই বদলেছে।

ডেটাবেস — MongoDB নাকি PostgreSQL

TechNova-র MERN অ্যাপের মূল ডেটাবেস MongoDB (Document Database, Port 27017) — অ্যাপের ডেটা JSON-এর মতো Document আকারে থাকে বলে MongoDB স্বাভাবিক পছন্দ। তবে অ্যাপে যদি কঠোর সম্পর্ক (Relation) ও লেনদেন (Transaction) দরকার হয়, PostgreSQL (Relational Database, Port 5432) ভালো। এই অধ্যায়ে আমরা MongoDB-কে মূল ধরে এগোব, আর PostgreSQL-এর ইনস্টল-পদ্ধতিও দেখাব — দুটিতেই নীতি এক: Repository যোগ করা, প্যাকেজ ইনস্টল, Service চালু করা, আর `bindIp` (MongoDB) বা `listen_addresses` (PostgreSQL) দিয়ে শুধু প্রয়োজনীয় Interface-এ শোনা — ডেটাবেস কখনোই ইন্টারনেটে খোলা রাখা যাবে না। MongoDB-তে প্রাথমিক ব্যবহারকারী ও Password তৈরি করতে `community.mongodb.mongodb_user` মডিউল ব্যবহার করা যায়; Password-এর মান Ansible Vault থেকে আসতে হবে, কখনোই Plain-Text নয়।

Environment ফাইল ও Health Check

`.env` ফাইলে থাকে `PORT=4000`, `DATABASE_URL=mongodb://127.0.0.1:27017/technova`, `JWT_SECRET=CHANGE-ME-...` — এই ফাইলটি Template দিয়ে তৈরি করা হয়, Owner অ্যাপ-ব্যবহারকারী, Permission 600 — যাতে অন্য কেউ পড়তে না পারে। Health Check মানে API-তে একটি ছোট Endpoint — `GET /api/health` — যা উত্তর দেয় `{"status": "ok"}` এবং ডেটাবেস সংযোগও পরীক্ষা করে। Deploy-এর শেষ Task-টি `uri` মডিউল দিয়ে এই Endpoint-কে ডাকে; 200 না এলে Playbook ব্যর্থ হয় — এটাই Deploy-এর সত্যিকারের যাচাই, শুধু "Service চালু আছে" দেখা নয়।

স্তর	সফটওয়্যার	Port	Ansible-এ পরিচালনা
Reverse Proxy	Nginx	80 (443 পরে)	template + nginx -t + handler reload
API	Node.js 20 + Express	4000	git + npm ci + systemd service
ফ্রন্টএন্ড	React / Next.js	3000	Build করে Nginx দিয়ে Static পরিবেশন
ডেটাবেস	MongoDB 7 / PostgreSQL 16	27017 / 5432	apt + service + bindIp সীমিত করা

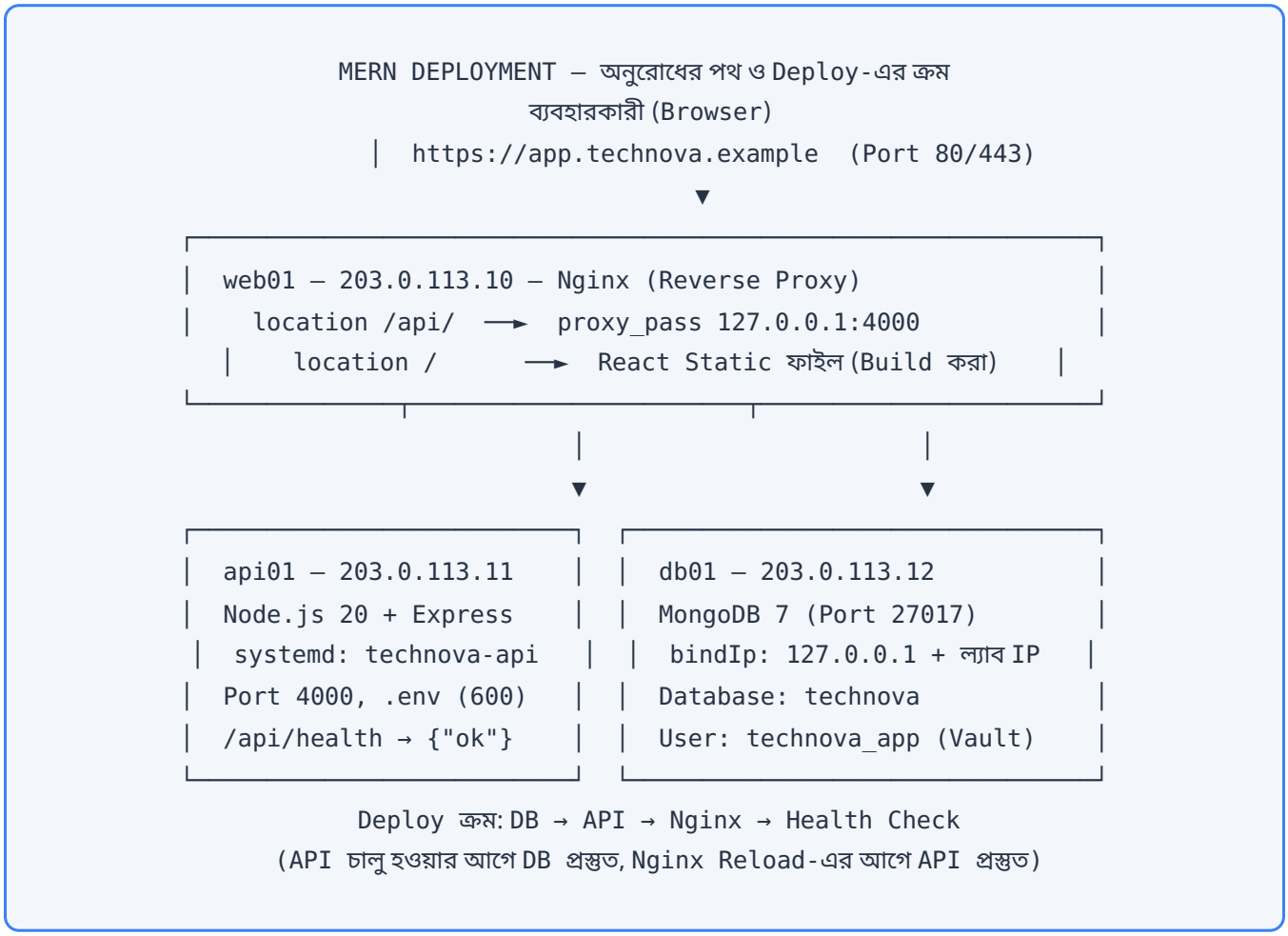
১৮.৬ অভ্যন্তরীণ কার্যপ্রণালী

Deploy Playbook-এর ভেতরের ক্রমটি বোঝা জরুরি, কারণ ক্রম ভুল হলে সবকিছু ঠিক থাকলেও অ্যাপ চলবে না। প্রথমে Package ও Repository Task-গুলো চলে — এগুলো সার্ভারের অবস্থা বদলায়, অ্যাপ ছোঁয় না। তারপর `git` Task কোড টানে; কোড বদলে গেলে পরের Task-গুলো জানে নতুন Deploy হচ্ছে। তারপর `npm ci` চলে — এটি সবচেয়ে ধীর Task, কারণ নির্ভরতা ডাউনলোড হয়; `npm ci` শুধু তখনই চালানো উচিত যখন কোড বদলেছে — Ansible-এ এটি করা যায় `git`

Task-এর ফলাফল (`register` করা Variable) দেখে, অথবা `creates` নয়, বরং Handler-এর মাধ্যমে। তারপর Template Task-গুলো Nginx Config, systemd Unit ও `.env` লেখে; কোনো Template বদলালে সংশ্লিষ্ট Handler-টি Playbook-এর শেষে চলে — Nginx-এর জন্য `reload nginx`, API-র জন্য `restart technova-api`। Handler-এর মূল গুণ: একাধিক Task একই Handler-কে Notify করলেও Handler একবারই চলে। শেষে Health Check Task `uri` মডিউল দিয়ে `http://127.0.0.1:4000/api/health`-কে ডাকে, কয়েকবার Retry করে (`until`, `retries`, `delay`) — কারণ API চালু হতে কয়েক সেকেন্ড লাগে; প্রথম চেষ্টাতেই ব্যর্থ হওয়া মানেই Deploy ব্যর্থ নয়, তবে নির্দিষ্ট সংখ্যক চেষ্টার পরেও 200 না এলে Playbook থেমে যায় ও ভুলটি স্পষ্ট হয়। Zero-Downtime Restart-এর সরল রূপটি হলো: নতুন কোড Deploy করে API Restart করার আগে Nginx-এ একটি সাময়িক রক্ষণাবেক্ষণ পৃষ্ঠা (Maintenance Page) নয়, বরং API-কে এমনভাবে Restart করা যাতে পুরনো Process নতুন Process চালু হওয়ার পর বন্ধ হয় — systemd-এর একটিমাত্র Service-এ এটি পুরোপুরি সম্ভব নয়; বাস্তব Zero-Downtime-এর জন্য দুটি API Instance (Blue/Green) বা Load Balancer-এর পেছনে Rolling Restart দরকার, যা ১৮.১০-এর প্যাটার্নে ব্যাখ্যা করা হয়েছে।

> **TIP:** `npm ci` চালানোর আগে `node_modules`-এর Cache নষ্ট করবেন না — প্রতিবার পুরো নির্ভরতা নতুন করে টানা ধীর ও ভঙ্গুর। আর `npm install`-এর বদলে `npm ci` ব্যবহার করুন: `package-lock.json` থাকলে দুটি একই ফল দেবে, কিন্তু `npm ci` Lock ফাইল থেকে একচুলও সরবে না — Production-এ এই নিশ্চয়তাই দরকার।

১৮.৭ আর্কিটেকচার ও চিত্র



ল্যাবে তিনটি সার্ভার আলাদা হলেও ছোট Deploy-এ সবকিছু একটিমাত্র সার্ভারেও চলতে পারে — তখন Nginx, API ও MongoDB একই হোস্টে থাকে, শুধু Port আলাদা। আর্কিটেকচারের নীতি একই থাকে: স্তরগুলো Port ও Process দিয়ে আলাদা, আর প্রতিটি স্তরের Config Template থেকে আসে।

১৮.৮ ব্যবহারিক উদাহরণ

Nginx Template — `templates/technova.conf.j2`:

```
# templates/technova.conf.j2 — Nginx Reverse Proxy (TechNova)
server {
    listen 80;
    server_name {{ technova_app_domain }};

    # API অনুরোধ Express-এ পাঠানো
    location /api/ {
        proxy_pass http://127.0.0.1:{{ technova_api_port }};
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }

    # React ফ্রন্টএন্ডের Static ফাইল
    location / {
        root /opt/technova/client/build;
        try_files $uri /index.html;
    }
}
```

systemd Service Unit — `templates/technova-api.service.j2`:

```
# templates/technova-api.service.j2 — Express API Service Unit
[Unit]
Description=TechNova Express API
After=network.target mongod.service
Wants=mongod.service

[Service]
Type=simple
User=technova
Group=technova
WorkingDirectory=/opt/technova/api
EnvironmentFile=/opt/technova/api/.env
ExecStart=/usr/bin/node server.js
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Health Check Task — API চালু হওয়ার জন্য অপেক্ষা করা:

```
# Health Check — API 200 না দেওয়া পর্যন্ত Retry করা
- name: Wait for API health endpoint
  ansible.builtin.uri:
    url: "http://127.0.0.1:{{ technova_api_port }}/api/health"
    status_code: 200
    return_content: true
  register: health
  until: health.status == 200
  retries: 12
  delay: 5
  # অর্থ: ৫ সেকেন্ড পরপর ১২ বার চেষ্টা — মোট ৬০ সেকেন্ড পর্যন্ত অপেক্ষা
```

১৮.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচে MERN Deploy-এর মূল Playbook — `playbooks/deploy-mern.yml`। এটি Database, API ও Nginx — তিনটি স্তরকে তিনটি Play-এ ভাগ করেছে, যাতে ক্রমটি স্পষ্ট থাকে। Secret-এর মান এখানে নেই — `.env` Template-এর Secret Variable Ansible Vault থেকে আসে।

```
---
# playbooks/deploy-mern.yml — TechNova MERN App Deployment
# চালানোর কমান্ড: ansible-playbook -i inventory/hosts.yml playbooks/deploy-mern.yml
- name: Install and configure MongoDB
  hosts: dbservers
  become: true
  tasks:
    - name: Install MongoDB 7
      ansible.builtin.apt:
        name: mongodb-org
        state: present
        update_cache: true
      # বাস্তবে MongoDB-র নিজস্ব Repository আগে যোগ করতে হয় (apt_repository)

    - name: Ensure MongoDB listens only on local and lab interfaces
      ansible.builtin.lineinfile:
        path: /etc/mongod.conf
        regexp: '^  bindIp:'
        line: '  bindIp: 127.0.0.1,203.0.113.12'
      notify: Restart MongoDB

    - name: Start and enable MongoDB
      ansible.builtin.service:
        name: mongod
        state: started
        enabled: true

  handlers:
    - name: Restart MongoDB
      ansible.builtin.service:
        name: mongod
        state: restarted

- name: Deploy Express API
  hosts: apiservers
  become: true
  vars:
```

```
    technova_api_port: 4000
    technova_app_domain: app.technova.example
  tasks:
    - name: Create app user and directory
      ansible.builtin.user:
        name: technova
        system: true
        create_home: false
        shell: /usr/sbin/nologin

    - name: Clone MERN app repository
      ansible.builtin.git:
        repo: https://git.technova.example/technova/mern-app.git
        dest: /opt/technova/api
        version: main
        force: false
      register: app_code

    - name: Install Node.js dependencies (npm ci)
      ansible.builtin.command:
        cmd: npm ci --omit=dev
        chdir: /opt/technova/api/server
      when: app_code.changed
      changed_when: true

    - name: Deploy environment file from template
      ansible.builtin.template:
        src: env.j2
        dest: /opt/technova/api/.env
        owner: technova
        group: technova
        mode: "0600"
      notify: Restart API

    - name: Deploy systemd service unit
      ansible.builtin.template:
        src: technova-api.service.j2
        dest: /etc/systemd/system/technova-api.service
        mode: "0644"
      notify:
        - Reload systemd
        - Restart API

    - name: Start and enable API service
      ansible.builtin.systemd:
        name: technova-api
        state: started
        enabled: true
        daemon_reload: true

    - name: Wait for API health endpoint
      ansible.builtin.uri:
        url: "http://127.0.0.1:4000/api/health"
        status_code: 200
      register: health
      until: health.status == 200
      retries: 12
      delay: 5
```

```
  handlers:
    - name: Reload systemd
      ansible.builtin.systemd:
        daemon_reload: true
    - name: Restart API
      ansible.builtin.systemd:
        name: technova-api
        state: restarted

- name: Configure Nginx reverse proxy
  hosts: webservers
  become: true
  vars:
    technova_app_domain: app.technova.example
    technova_api_port: 4000
  tasks:
    - name: Install Nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true

    - name: Deploy Nginx site config from template
      ansible.builtin.template:
        src: technova.conf.j2
        dest: /etc/nginx/sites-available/technova.conf
        mode: "0644"
      notify: Reload Nginx

    - name: Enable TechNova site
      ansible.builtin.file:
        src: /etc/nginx/sites-available/technova.conf
        dest: /etc/nginx/sites-enabled/technova.conf
        state: link
      notify: Reload Nginx

    - name: Remove default Nginx site
      ansible.builtin.file:
        path: /etc/nginx/sites-enabled/default
        state: absent
      notify: Reload Nginx

    - name: Test Nginx configuration syntax
      ansible.builtin.command: nginx -t
      changed_when: false

    - name: Start and enable Nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true

  handlers:
    - name: Reload Nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
```

Environment Template — `templates/env.j2` (Secret Vault থেকে আসে, এখানে শুধু Placeholder):

```
# templates/env.j2 — API Environment File (Permission 0600)
PORT=4000
NODE_ENV=production
DATABASE_URL=mongodb://technova_app:CHANGE-ME-DB-PASSWORD@203.0.113.12:27017/technova
JWT_SECRET=CHANGE-ME-JWT-SECRET
# বাস্তবে CHANGE-ME-... মানগুলো Ansible Vault-এর Variable দিয়ে বদলান — কখনোই Plain-Text লিখবেন না।
```

১৮.১০ ধাপে ধাপে বাস্তবায়ন

1. Baseline Playbook চালিয়ে তিনটি সার্ভার প্রস্তুত করুন (অধ্যায় ১৭)।
2. MongoDB-র Repository যোগ করুন ও `playbooks/deploy-mern.yml`-এর প্রথম Play চালান — শুধু Database Play: `ansible-playbook -i inventory/hosts.yml playbooks/deploy-mern.yml --tags db` নয়, বরং Play-ভিত্তিক সীমা ব্যবহার করতে চাইলে প্রথমে আলাদা করে Database Playbook চালান; এই বইয়ের উদাহরণে তিনটি Play এক ফাইলে আছে, তাই পুরো ফাইল চালানোর আগে `--check --diff` দিয়ে দেখুন।
3. Vault ফাইল তৈরি করুন: `ansible-vault create group_vars/all/vault.yml` — তাতে `technova_db_password` ও `technova_jwt_secret` রাখুন, আর `env.j2`-এ Placeholder-এর বদলে সেই Variable ব্যবহার করুন।
4. Deploy চালান: `ansible-playbook -i inventory/hosts.yml playbooks/deploy-mern.yml --ask-vault-pass`।
5. যাচাই করুন: `curl http://127.0.0.1:4000/api/health` (API সার্ভারে), তারপর ব্রাউজার বা `curl` দিয়ে `http://app.technova.example/api/health` (Nginx দিয়ে)।
6. Zero-Downtime প্যাটার্ন (সরল রূপ): নতুন কোড Deploy করার সময় প্রথমে নতুন সংস্করণটি আলাদা Port-এ (যেমন 4001) চালু করুন, Health Check পাস হলে Nginx-এর `proxy_pass` Template-এ Port বদলে Reload করুন (Reload-এ চলমান সংযোগ ভাঙে না, Restart-এ ভাঙে), তারপর পুরনো Process বন্ধ করুন। একটিমাত্র সার্ভারে এই Blue/Green প্যাটার্নটি হাতে করা যায়; একাধিক সার্ভারে Load Balancer-এর পেছনে একবারে একটি সার্ভার করে Rolling Restart করুন — কখনোই সব সার্ভার একসঙ্গে Restart করবেন না।
7. Rollback অনুশীলন: `git` Task-এর `version`-এ আগের Tag বসিয়ে Playbook আবার চালান — কোড আগের সংস্করণে ফিরবে, আর Health Check তা নিশ্চিত করবে।

> **REAL WORLD — TechNova:** TechNova একবার Deploy-এর সময় সরাসরি `systemctl restart technova-api` দিয়ে API বন্ধ করে নতুন কোড চালু করেছিল — Restart-এর ১০ সেকেন্ডে আসা সব অনুরোধ 502 পেয়েছিল, আর একজন গ্রাহকের পেমেন্ট-অনুরোধ মাঝপথে হারিয়ে গিয়েছিল। তারপর থেকে তাদের নিয়ম: Deploy-এর আগে Health Check-এর Retry, Nginx Reload (Restart নয়), আর নতুন সংস্করণ চালু হওয়ার পর পুরনোটি বন্ধ করা। শিক্ষা: Deploy-এর গতি নয়, অনুরোধ না হারানোই আসল মাপকাঠি।

১৮.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

সফল Deploy-এর পর যাচাইগুলো এরকম দেখাবে (চিত্রণমূলক আউটপুট):

```
$ curl http://127.0.0.1:4000/api/health
{"status": "ok", "db": "connected", "uptime": 42}
```

```
# ব্যাখ্যা: API চলছে, ডেটাবেসের সঙ্গে সংযোগ আছে, ৪২ সেকেন্ড ধরে চলছে।

$ systemctl status technova-api --no-pager
● technova-api.service - TechNova Express API
   Loaded: loaded (/etc/systemd/system/technova-api.service; enabled)
   Active: active (running) since Mon 2026-11-09 10:15:00 +06; 42s ago
   Main PID: 1234 (node)
# ব্যাখ্যা: Service চালু, Boot-এ চালু হওয়ার জন্য enabled, Main Process node।

$ nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
# ব্যাখ্যা: Nginx Config-এর Syntax ঠিক আছে — Reload করা নিরাপদ।

$ curl -s http://app.technova.example/api/health
{"status": "ok", "db": "connected", "uptime": 42}
# ব্যাখ্যা: Nginx দিয়ে বাইরে থেকে API-তে পৌঁছানো যাচ্ছে — Reverse Proxy কাজ করছে।
```

Playbook-এর `PLAY RECAP`-এ সব হোস্টে `failed=0` থাকতে হবে। Health Check Task-এ `retries`-এর কারণে প্রথম দু-একবার চেষ্টা ব্যর্থ হয়ে পরে সফল হওয়া স্বাভাবিক — এটি ভুল নয়, এটি API-র Start হতে সময় লাগার প্রমাণ। কিন্তু Health Check-এর ১২ বার চেষ্টার পরেও 200 না এলে Playbook থেমে যাবে — তখন ১৮.১৩-এর ধাপগুলো অনুসরণ করুন।

১৮.১২ সাধারণ ভুল

- **Nginx Reload-এর বদলে Restart:** Restart চলমান সংযোগ ভেঙে দেয়; Config বদলে শুধু Reload করুন। Handler-এ `state: reloaded` ব্যবহার করুন।
- **`nginx -t` ছাড়া Config Deploy:** ভুল Config নিয়ে Nginx Reload হলে Nginx বন্ধ হয়ে যেতে পারে। সবসময় `nginx -t` Task রাখুন।
- **অ্যাপ Root দিয়ে চালানো:** `User=root` দিয়ে API চালালে অ্যাপের কোনো দুর্বলতা পুরো সার্ভারকে ঝুঁকিতে ফেলে। সবসময় আলাদা System User (`technova`) ব্যবহার করুন।
- **`.env` ফাইলের Permission 644:** অন্য ব্যবহারকারী Secret পড়তে পারে। সবসময় Owner অ্যাপ-ব্যবহারকারী ও Mode 600 রাখুন।
- **`npm install` দিয়ে Production Deploy:** Lock ফাইল থেকে সরে গিয়ে ভিন্ন নির্ভরতা আসতে পারে। Production-এ `npm ci` ব্যবহার করুন।
- **ডেটাবেস `bindIp` খোলা রাখা:** `bindIp: 0.0.0.0` মানে ডেটাবেস সব Interface-এ শোনে — ইন্টারনেটে খোলা থাকলে আক্রমণের লক্ষ্য হয়। শুধু প্রয়োজনীয় IP-তে সীমিত করুন।
- **Health Check ছাড়া Deploy:** Service "active" দেখালেই অ্যাপ ঠিক আছে — এই ধারণা ভুল। API চালু হয়েও ডেটাবেস সংযোগে ব্যর্থ হতে পারে; শুধু Health Check-ই তা ধরে।
- **সব সার্ভার একসঙ্গে Restart:** Rolling Restart না করে সব API একসঙ্গে বন্ধ করলে পুরো অ্যাপ বন্ধ থাকে। একবারে একটি সার্ভার করে Restart করুন।

১৮.১৩ সমস্যা সমাধান (Troubleshooting)

1. **লক্ষণ:** ব্রাউজারে 502 Bad Gateway — **পরীক্ষা:** `systemctl status technova-api` ও `curl http://127.0.0.1:4000/api/health`। **সমাধান:** API বন্ধ থাকলে লগ দেখুন (`journalctl -u technova-api -n 50`); API চললে Nginx-এর `proxy_pass` Port মিলছে কি না দেখুন।
2. **লক্ষণ:** API চালু হয়েই মারা যায় (Crash Loop) — **পরীক্ষা:** `journalctl -u technova-api -n 100`-এ ত্রুটি পড়ুন। **সমাধান:** সাধারণ কারণ: `.env`-এ ভুল `DATABASE_URL`, Port ইতিমধ্যে ব্যবহৃত হচ্ছে, বা `node_modules` অসম্পূর্ণ — `npm ci` আবার চালান।
3. **লক্ষণ:** `nginx -t` ব্যর্থ — `unknown directive` বা `invalid port` — **পরীক্ষা:** Template-এর Variable-এর মান (`technova_api_port`) ঠিক আছে কি না। **সমাধান:** Variable-এর মান ঠিক করে Template আবার Deploy করুন; কখনোই হাতে Config এডিট করে ঠিক করবেন না — পরের Run-এ তা মুছে যাবে।
4. **লক্ষণ:** MongoDB-তে সংযোগ হয় না — `MongoServerSelectionError` — **পরীক্ষা:** db01-এ `systemctl status mongod`, `ss -tlnp | grep 27017`, আর API সার্ভার থেকে `curl -v telnet://203.0.113.12:27017`। **সমাধান:** `bindIp` ও Firewall (অধ্যায় ১৯) — দুটিই পরীক্ষা করুন; ডেটাবেসের ব্যবহারকারী ও Password Vault থেকে ঠিক আসছে কি না দেখুন।
5. **লক্ষণ:** `npm ci` ব্যর্থ — `package-lock.json` নেই — **পরীক্ষা:** Repository-তে Lock ফাইল Commit করা আছে কি না। **সমাধান:** Lock ফাইল Commit করুন; সাময়িকভাবে `npm install` ব্যবহার করবেন না — কারণটি ঠিক করুন।
6. **লক্ষণ:** Health Check Timeout — **পরীক্ষা:** API-র লগ ও `ss -tlnp | grep 4000`। **সমাধান:** API ধীরে Start হলে `retries` ও `delay` বাড়ান; কিন্তু বারবার Timeout হলে আসল কারণ খুঁজুন — Retry দিয়ে আসল সমস্যা ঢাকা যায় না।

১৮.১৪ নিরাপত্তা বিবেচনা

> **WARNING:** ডেটাবেস কখনোই ইন্টারনেটে খোলা রাখবেন না। MongoDB-র `bindIp` শুধু প্রয়োজনীয় IP-তে সীমিত করুন, শক্তিশালী ব্যবহারকারী-Password ব্যবহার করুন, আর ডেটাবেসের Port (27017/5432) Firewall দিয়ে শুধু অ্যাপ-সার্ভার থেকে পৌঁছানো যায় এমন করুন। অতীতে ইন্টারনেটে খোলা MongoDB-তে Ransomware হামলা হয়েছে — কারণ অনেকেই Authentication ছাড়াই ডেটাবেস চালিয়েছিলেন। Authentication ছাড়া MongoDB চালানো প্রোডাকশনে সম্পূর্ণ নিষিদ্ধ।

Deploy-এর নিরাপত্তা-দিকগুলো: প্রথমত, Secret (`JWT_SECRET`, ডেটাবেস Password) শুধু Ansible Vault থেকে আসবে, আর `.env` ফাইলের Permission 600 হবে। দ্বিতীয়ত, অ্যাপ কখনোই Root দিয়ে চলবে না — আলাদা System User, আর সেই User-এর শুধু `/opt/technova`-এ লেখার অধিকার। তৃতীয়ত, Nginx-এর মাধ্যমে বাইরে শুধু 80/443 Port খোলা থাকবে — API-র 4000 ও ডেটাবেসের 27017 Port বাইরে থেকে বন্ধ থাকবে (অধ্যায় ১৯-এর UFW নিয়ম দেখুন)। চতুর্থত, `X-Powered-By`-এর মতো Header যা সফটওয়্যারের সংস্করণ ফাঁস করে, তা বন্ধ করুন — আক্রমণকারী সংস্করণ জেনে দুর্বলতা খোঁজে। পঞ্চমত, Deploy-এর সময় পুরনো `.env` বা Log ফাইল Repository-তে Commit হয়নি তা যাচাই করুন — `.gitignore`-এ `.env` ও `node_modules` থাকা বাধ্যতামূলক।

১৮.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

প্রোডাকশন Deploy-এর নিয়মগুলো: প্রথমত, Deploy সবসময় Tag করা সংস্করণ থেকে করুন — `version: main` নয়, `version: v1.4.2` — যাতে কোন সংস্করণ চলছে তা নিশ্চিতভাবে বলা যায়, আর Rollback মানে আগের Tag-এ ফিরে যাওয়া। দ্বিতীয়ত, Deploy-এর আগে Staging পরিবেশে একই Playbook চালান — Staging-এ যা কাজ করে, Production-এও তা কাজ করার সম্ভাবনা বেশি। তৃতীয়ত, প্রতিটি Deploy-এর লগ ও Health Check-এর ফলাফল সংরক্ষণ করুন; Deploy-এর সময়, সংস্করণ ও দায়ী ব্যক্তি — এই তিনটি তথ্য ছাড়া সমস্যা হলে কারণ খুঁজে পাওয়া যায় না। চতুর্থত, ডেটাবেস Migration আলাদা ধাপ হিসেবে চালান — কোড Deploy ও Migration একসঙ্গে করলে Rollback জটিল হয়; Migration সবসময় Backward-Compatible রাখুন (পুরনো কোড নতুন স্কিমায় চলতে পারে, এমনভাবে)। পঞ্চমত, Zero-Downtime-এর জন্য একাধিক API সার্ভার ও Load Balancer ব্যবহার করুন — একটিমাত্র সার্ভারে পুরোপুরি Zero-Downtime সম্ভব নয়, এটি স্বীকার করে পরিকল্পনা করুন। সবশেষে, Deploy-এর পর Smoke Test চালান — শুধু Health Check নয়, Login ও একটি মূল API অনুরোধও পরীক্ষা করুন।

১৮.১৬ অধ্যায়ের সারাংশ

MERN Deploy মানে চারটি স্তরকে সঠিক ক্রমে, সঠিক Config দিয়ে চালু করা: ডেটাবেস প্রথমে, তারপর Node.js/Express API systemd Service হিসেবে, তারপর Nginx Reverse Proxy, আর সবশেষে Health Check দিয়ে যাচাই। Nginx Template থেকে Config পায় ও `nginx -t` দিয়ে যাচাই হয়; Node.js 20 সংস্করণ পিন করে ইনস্টল হয়; `npm ci` Lock ফাইল অনুযায়ী নির্ভরতা বসায়; `.env` ফাইল Vault থেকে Secret নিয়ে 600 Permission-এ থাকে; আর Handler শুধু Config বদলালেই Reload/Restart করে। Zero-Downtime মানে Restart নয় — নতুন সংস্করণ চালু করে, Health Check পাস হলে Traffic সরানো, তারপর পুরনোটি বন্ধ করা। এই প্যাটার্নই পরের অধ্যায়ের নিরাপত্তা-কঠোরকরণের ভিত্তি।

১৮.১৭ জ্ঞান যাচাই

1. Reverse Proxy কী, এবং কেন ব্যবহারকারী সরাসরি Express API-র সঙ্গে কথা বলে না?
2. `nginx -t` Task-টি কেন Handler-এর আগে রাখা হয়? ভুল Config নিয়ে Reload হলে কী হতে পারে?
3. `npm install`-এর বদলে Production Deploy-এ `npm ci` কেন ভালো?
4. systemd Service Unit-এ `Restart=always` ও `User=technova` — দুটি লাইনের কাজ আলাদাভাবে ব্যাখ্যা করুন।
5. একটিমাত্র সার্ভারে `systemctl restart` দিয়ে Deploy করলে কেন কিছু অনুরোধ হারায়, আর Blue/Green প্যাটার্ন কীভাবে তা এড়ায়?

১৮.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. **সম্পূর্ণ Deploy:** ল্যাবে উপরের Playbook দিয়ে MERN অ্যাপ Deploy করুন; `curl` দিয়ে Nginx পথে ও সরাসরি API পথে — দুভাবেই Health Check-এর আউটপুট সংরক্ষণ করুন।
2. **PostgreSQL রূপ:** MongoDB-র বদলে PostgreSQL 16 ইনস্টল করে একই API-র জন্য `DATABASE_URL` বদলে Deploy করুন; `bindIp`-এর বদলে `listen_addresses` কীভাবে সীমিত করলেন তা লিখুন।
3. **Zero-Downtime অনুশীলন:** API-র নতুন সংস্করণ 4001 Port-এ চালু করে Health Check পাস হলে Nginx-এর `proxy_pass` বদলে Reload করুন; Deploy চলাকালীন প্রতি সেকেন্ডে Health Check ডেকে দেখুন কোনো অনুরোধ ব্যর্থ হয় কি না, ফলাফল লিখুন।

অধ্যায় 19 • PART IV — Security Hardening

Chapter 19 — Users, SSH, Firewall, Packages, and Security Hardening

১৯.১ ভূমিকা

অ্যাপ চালু হলেই কাজ শেষ নয় — এবার প্রশ্ন হলো সার্ভারটি কতটা নিরাপদ। একটি নতুন সার্ভারে সাধারণত Root লগইন খোলা থাকে, Password দিয়ে SSH করা যায়, সব Port খোলা থাকে, আর কোনো আপডেট-নীতি থাকে না। এই অবস্থায় ইন্টারনেটে রাখা সার্ভার কয়েক ঘণ্টার মধ্যেই স্বয়ংক্রিয় আক্রমণের (Automated Attack) লক্ষ্য হয়: Bot-গুলো ক্রমাগত SSH-এ Password চেষ্টা করে, খোলা Port খোঁজে, পুরনো প্যাকেজের দুর্বলতা কাজে লাগায়। হাতে-কলমে নিরাপত্তা-কঠোরকরণ (Hardening) করতে গেলে ভুলের ঝুঁকি সবচেয়ে বেশি — একটি ভুল `sshd_config` লাইন লিখলেই নিজেই আর সার্ভারে ঢুকতে পারবেন না, আর একটি ভুল Firewall নিয়মে পুরো অ্যাপ বন্ধ হয়ে যায়। এই অধ্যায়ে আমরা নিরাপত্তাকে একটি Ansible Playbook-এ বাঁধব: ব্যবহারকারী ও গ্রুপ তৈরি, SSH Key বসানো, sudoers ফাইল `visudo` দিয়ে যাচাই করে লেখা, SSH Server-এর কঠোরকরণ (Password লগইন বন্ধ, Root লগইন বন্ধ), UFW Firewall-এর নিয়ম, fail2ban-এর ধারণা, স্বয়ংক্রিয় নিরাপত্তা-আপডেট (unattended-upgrades), আর CIS-শৈলীর Baseline-এর নোট। মূল নীতি: নিরাপত্তা একটি কাজ নয়, একটি অবস্থা — আর সেই অবস্থা Playbook-এ লিখে রাখলে তা যাচাই করা যায়, পুনরাবৃত্তি করা যায়, আর কেউ হাতে বদলে ফেললে পরের Run-এ তা আবার ঠিক হয়ে যায়।

> **CONCEPT:** Hardening মানে আক্রমণের পৃষ্ঠ (Attack Surface) ছোট করা — যা দরকার নেই তা বন্ধ করা, যা দরকার তা শক্ত করা। প্রতিটি খোলা Port, প্রতিটি Password-লগইন, প্রতিটি অপ্রয়োজনীয় ব্যবহারকারী — একটি করে দরজা। Hardening-এর প্রশ্ন সবসময় একটিই: "এই দরজাটি কি সত্যিই দরকার, আর দরকার হলে তার চাবি কি যথেষ্ট শক্ত?" Password-এর বদলে SSH Key, Root-এর বদলে sudo, সব Port-এর বদলে শুধু প্রয়োজনীয় Port — এই তিনটি বদলেই অধিকাংশ স্বয়ংক্রিয় আক্রমণ থেমে যায়।

১৯.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Linux-এ User, Group ও sudo-র ভূমিকা ব্যাখ্যা করতে এবং `user` ও `group` মডিউল দিয়ে তা তৈরি করতে; (২) `authorized_key` মডিউল দিয়ে SSH Public Key বসাতে ও Key-ভিত্তিক লগইনের সুবিধা বলতে; (৩) sudoers ফাইল Template দিয়ে লিখতে ও `validate: 'visudo -cf %s'` দিয়ে যাচাইয়ের গুরুত্ব ব্যাখ্যা করতে; (৪) `sshd_config` কঠোর করতে — `PasswordAuthentication no`, `PermitRootLogin no`, `PubkeyAuthentication yes` — এবং পরিবর্তনের পর SSH-এর পরীক্ষা-পদ্ধতি বলতে; (৫) UFW দিয়ে শুধু প্রয়োজনীয় Port খুলতে ও নিয়মের ক্রম বোঝাতে; (৬) fail2ban কীভাবে বারবার ব্যর্থ লগইন আটকায় তা ব্যাখ্যা করতে; (৭) unattended-upgrades দিয়ে নিরাপত্তা-আপডেট স্বয়ংক্রিয় করতে; (৮) CIS Benchmark-শৈলীর Baseline বলতে কী বোঝায় তা ব্যাখ্যা করতে ও TechNova-র জন্য একটি ছোট Checklist তৈরি করতে।

১৯.৩ পূর্বশর্ত ধারণা

অধ্যায় ১৭ ও ১৮-এর Playbook আপনার ল্যাবে চালানো থাকতে হবে — বিশেষ করে Baseline ও MERN Deploy। আরও প্রয়োজন: Linux User ও Permission-এর ধারণা (`chmod`, `chown`, `sudo`), SSH Key জোড়ার ধারণা (Public Key সার্ভারে, Private Key নিজের কাছে), `sshd_config` ফাইলের প্রাথমিক ধারণা, Firewall ও Port-এর ধারণা, আর Ansible

Vault-এর ব্যবহার — কারণ এই অধ্যায়ে কোনো Password Plain-Text লেখা হবে না। সতর্কতা: এই অধ্যায়ের Playbook চালানোর আগে নিশ্চিত হয়ে নিন আপনার SSH Key দিয়ে লগইন কাজ করে — Password লগইন বন্ধ করার পর Key ছাড়া আর ঢোকা যাবে না। ল্যাবে প্রথমে একটি অতিরিক্ত Terminal খোলা রাখুন যাতে ভুল হলে সার্ভারে ঢুকে ঠিক করা যায়; প্রোডাকশনে এই Playbook প্রথমে Staging-এ চালান।

১৯.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি সার্ভারের নিরাপত্তাকে ভাবুন একটি বাড়ির নিরাপত্তা হিসেবে। বাড়িতে ঢোকার দরজা (SSH Port 22) আছে; দরজায় তালা (Password বা Key) আছে; বাড়ির ভেতরে ঘরগুলো (User Account) আছে; আর কিছু ঘরে শুধু মালিক ঢুকতে পারেন (Root/sudo)। Password-ভিত্তিক তালা মানে যে কেউ চাবির অনুমান করে ঢোকার চেষ্টা করতে পারে — আর কম্পিউটার সেকেন্ডে হাজারো অনুমান করতে পারে, তাই দুর্বল Password কয়েক ঘণ্টায় ভেঙে যায়। SSH Key মানে তালাটি এমন যে চাবিটি অনুমান করা যায় না — Private Key ছাড়া দরজা খোলে না, আর Private Key কখনো নেটওয়ার্কে পাঠানো হয় না। Root লগইন বন্ধ করা মানে বাড়ির মালিকের ঘরের দরজায় সরাসরি ঢোকা বন্ধ — মালিককে প্রথমে নিজের ঘরে (নিজের User) ঢুকে তারপর sudo দিয়ে মালিকের কাজ করতে হয়, আর সেই প্রতিটি কাজ লগে লেখা থাকে (`/var/log/auth.log`) — কে, কখন, কী করল, তার হিসাব থাকে। Firewall মানে বাড়ির চারপাশের দেওয়াল — শুধু যে দরজাগুলো দরকার (80, 443, 22 — আর তাও শুধু নির্দিষ্ট IP থেকে, যদি সম্ভব হয়) সেগুলো খোলা, বাকি সব বন্ধ। fail2ban মানে একজন পাহারাদার যে দেখে কেউ বারবার ভুল চাবি চেষ্টা করছে, আর তাকে কিছুক্ষণের জন্য গেটের বাইরে আটকে দেয়। এই প্রতিটি স্তর আলাদাভাবে ছোট, কিন্তু একসঙ্গে তারা Defense in Depth — একটি স্তর ভাঙলেও পরের স্তরটি থাকে।

১৯.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

User ও Group — কে কী করতে পারবে

`ansible.builtin.user` মডিউল ব্যবহারকারী তৈরি করে: `name`, `groups`, `shell`, `create_home`, `system` (System User — যেমন অ্যাপ চালানোর `technova` ব্যবহারকারী, যার লগইন দরকার নেই) — এই প্যারামিটারগুলো দিয়ে। `ansible.builtin.group` মডিউল গ্রুপ তৈরি করে — যেমন `deploy` গ্রুপ, যার সদস্যরা Deploy করতে পারে, বা `docker` গ্রুপ (অধ্যায় ২১-এ আসবে)। ব্যবহারকারী মুছতে `state: absent` ও `remove: true` ব্যবহার করা হয় — তবে ব্যবহারকারী মুছলে তার ফাইলগুলোর মালিকানা অনাথ (Orphan) হয়ে যায়, তাই মুছার আগে ফাইলগুলোর ব্যবস্থা করুন। একটি গুরুত্বপূর্ণ নীতি: প্রতিটি মানুষের আলাদা ব্যবহারকারী থাকবে — সবাই মিলে একটি `nahid` ব্যবহারকারী ভাগ করে নেওয়া যাবে না, কারণ তাহলে কে কী করেছে তা লগে বোঝা যায় না।

authorized_key — Password ছাড়া লগইন

`ansible.builtin.authorized_key` মডিউল ব্যবহারকারীর `~/.ssh/authorized_keys` ফাইলে Public Key বসায়। Key-টি Variable থেকে আসে — যেমন `technova_admin_keys` তালিকা — আর Key-এর মান Ansible Vault বা Inventory Variable-এ রাখা যায়, তবে Public Key গোপন নয় — এটি সার্ভারে থাকার জন্যই তৈরি, তাই Plain-Text রাখলেও ক্ষতি নেই; গোপন হলো Private Key, যা কখনো সার্ভারে বা Repository-তে যাবে না। `exclusive: true` দিলে Module ফাইলে শুধু তালিকার Key-গুলোই রাখে, অন্য Key মুছে দেয় — এতে পুরনো কর্মীর Key স্বয়ংক্রিয়ভাবে মুছে যায়। Key বসানোর পর সবসময় নতুন Terminal থেকে Key দিয়ে লগইন পরীক্ষা করুন, তারপরেই Password বন্ধ করুন — এই ক্রমটি ভুল করলে নিজেই বাইরে আটকে যাবেন।

sudoers ও visudo যাচাই — সবচেয়ে বিপজ্জনক ফাইল

`/etc/sudoers` ফাইলে একটি Syntax ভুল থাকলে পুরো sudo ব্যবস্থা ভেঙে যায় — তখন কেউই Root হতে পারে না, আর সার্ভার ঠিক করতে Console-এ যেতে হয়। তাই sudoers ফাইল কখনোই সরাসরি `copy` দিয়ে লেখা উচিত নয় —

`template` মডিউলের `validate` প্যারামিটার ব্যবহার করুন: `validate: 'visudo -cf %s'` — Ansible প্রথমে ফাইলটি একটি অস্থায়ী জায়গায় লিখে `visudo -c` দিয়ে যাচাই করে, যাচাই পাস হলে তবেই আসল জায়গায় বসায়, না হলে Playbook থেমে যায় ও আসল ফাইলটি অক্ষত থাকে। সেরা অভ্যাস: মূল `/etc/sudoers` ফাইল না ছুঁয়ে `/etc/sudoers.d/`-এ আলাদা ফাইল লেখা — যেমন `/etc/sudoers.d/technova-deploy` — যাতে একটি ভুলে পুরো sudo না ভাঙে। sudoers-এর একটি সাধারণ লাইন: `nahid ALL=(ALL) NOPASSWD: ALL` — এর অর্থ `nahid` সব কমান্ড Password ছাড়া sudo দিয়ে চালাতে পারে। ল্যাবে এটি সুবিধাজনক, তবে প্রোডাকশনে Password-ছাড়া sudo সীমিত রাখা উচিত — শুধু নির্দিষ্ট কমান্ডের জন্য, যেমন `nahid ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart technova-api` — এতে Deploy করা যায়, কিন্তু পুরো Root পাওয়া যায় না।

sshd_config কঠোরকরণ

SSH Server-এর Config `/etc/ssh/sshd_config`-এ থাকে; Ansible-এ `lineinfile` দিয়ে প্রতিটি সেটিং নিশ্চিত করা হয়, আর পরিবর্তনের পর `sshd -t` দিয়ে যাচাই করে Handler দিয়ে Service Reload করা হয় — Restart নয়, Reload, কারণ Restart-এ চলমান সংযোগ ভেঙে যেতে পারে। মূল সেটিংগুলো: `PasswordAuthentication no` (Password দিয়ে লগইন বন্ধ — শুধু Key), `PermitRootLogin no` (Root সরাসরি লগইন বন্ধ), `PubkeyAuthentication yes` (Key লগইন চালু), `PermitEmptyPasswords no`, `X11Forwarding no`, `MaxAuthTries 3` (তিনবার ভুল চেষ্টার পর সংযোগ কাটা), `LoginGraceTime 30` (৩০ সেকেন্ডে লগইন না হলে সংযোগ কাটা)। একটি সতর্কতা: `sshd_config`-এ `Include` নির্দেশ থাকলে `/etc/ssh/sshd_config.d/`-এর ফাইলগুলোও পড়া হয় — শুধু মূল ফাইল বদলালেই যথেষ্ট নাও হতে পারে; `sshd -T` দিয়ে কার্যকর (Effective) সেটিং দেখে নিন।

UFW Firewall — শুধু প্রয়োজনীয় Port

UFW (Uncomplicated Firewall) হলো `iptables`-এর সহজ আবরণ। Ansible-এ `community.general.ufw` মডিউল দিয়ে নিয়ম লেখা হয়: `rule: allow`, `port: 22`, `proto: tcp` — আর `state: enabled` দিয়ে Firewall চালু করা হয়। TechNova-র নিয়ম: SSH (22) — শুধু নির্দিষ্ট IP থেকে, যদি সম্ভব হয়; HTTP (80) ও HTTPS (443) — সবার জন্য (Web সার্ভারে); API (4000) ও MongoDB (27017) — শুধু ল্যাব নেটওয়ার্ক (203.0.113.0/24) থেকে, বাইরে থেকে নয়। নিয়মের ক্রম গুরুত্বপূর্ণ: প্রথমে SSH-এর Allow নিয়ম, তারপর Firewall চালু করা — উল্টো করলে নিজেই আটকে যাবেন। Ansible-এ `ufw` Task-এ `rule: allow` ও `port` দিয়ে প্রতিটি নিয়ম আলাদা Task হিসেবে লেখা হয়, আর শেষে `state: enabled` ও `policy: deny` (ডিফল্ট: সবকিছু বন্ধ) দিয়ে Firewall চালু করা হয়।

fail2ban, unattended-upgrades ও CIS Baseline

fail2ban লগ ফাইল (`/var/log/auth.log`) দেখে — কোনো IP থেকে বারবার ব্যর্থ SSH লগইন হলে সেই IP-কে Firewall দিয়ে কিছুক্ষণের জন্য আটকে দেয় (Ban)। এটি আক্রমণ থামায় না, তবে স্বয়ংক্রিয় Bot-এর চেষ্টা ধীর করে দেয়। Ansible-এ fail2ban ইনস্টল করে তার Jail Config (`/etc/fail2ban/jail.local`) Template দিয়ে লেখা হয় — যেমন `maxretry = 5`, `bantime = 3600`। unattended-upgrades স্বয়ংক্রিয়ভাবে নিরাপত্তা-আপডেট ইনস্টল করে — `/etc/apt/apt.conf.d/50unattended-upgrades`-এ শুধু Security Repository চালু রাখা হয়, আর Reboot-এর প্রয়োজন হলে তা নির্দিষ্ট সময়ে করা হয়; প্রোডাকশনে স্বয়ংক্রিয় Reboot বিপজ্জনক হতে পারে — তাই `Unattended-Upgrade::Automatic-Reboot` সেটিংটি সচেতনভাবে ঠিক করুন। CIS Benchmark (Center for Internet Security) হলো সার্ভার-কঠোরকরণের একটি প্রমিত Checklist — যেমন "Root-এর umask 027 হওয়া উচিত", "অপ্রয়োজনীয় সার্ভিস বন্ধ থাকা উচিত" — TechNova-র জন্য আমরা CIS-এর প্রতিটি নিয়ম অন্ধভাবে অনুসরণ করি না, তবে CIS-কে Checklist হিসেবে ব্যবহার করে নিজেদের Baseline-এর সঙ্গে মিলিয়ে দেখি কোন নিয়মটি আমাদের পরিবেশে প্রযোজ্য।

নিয়ন্ত্রণ	Ansible মডিউল / ফাইল	লক্ষ্য অবস্থা

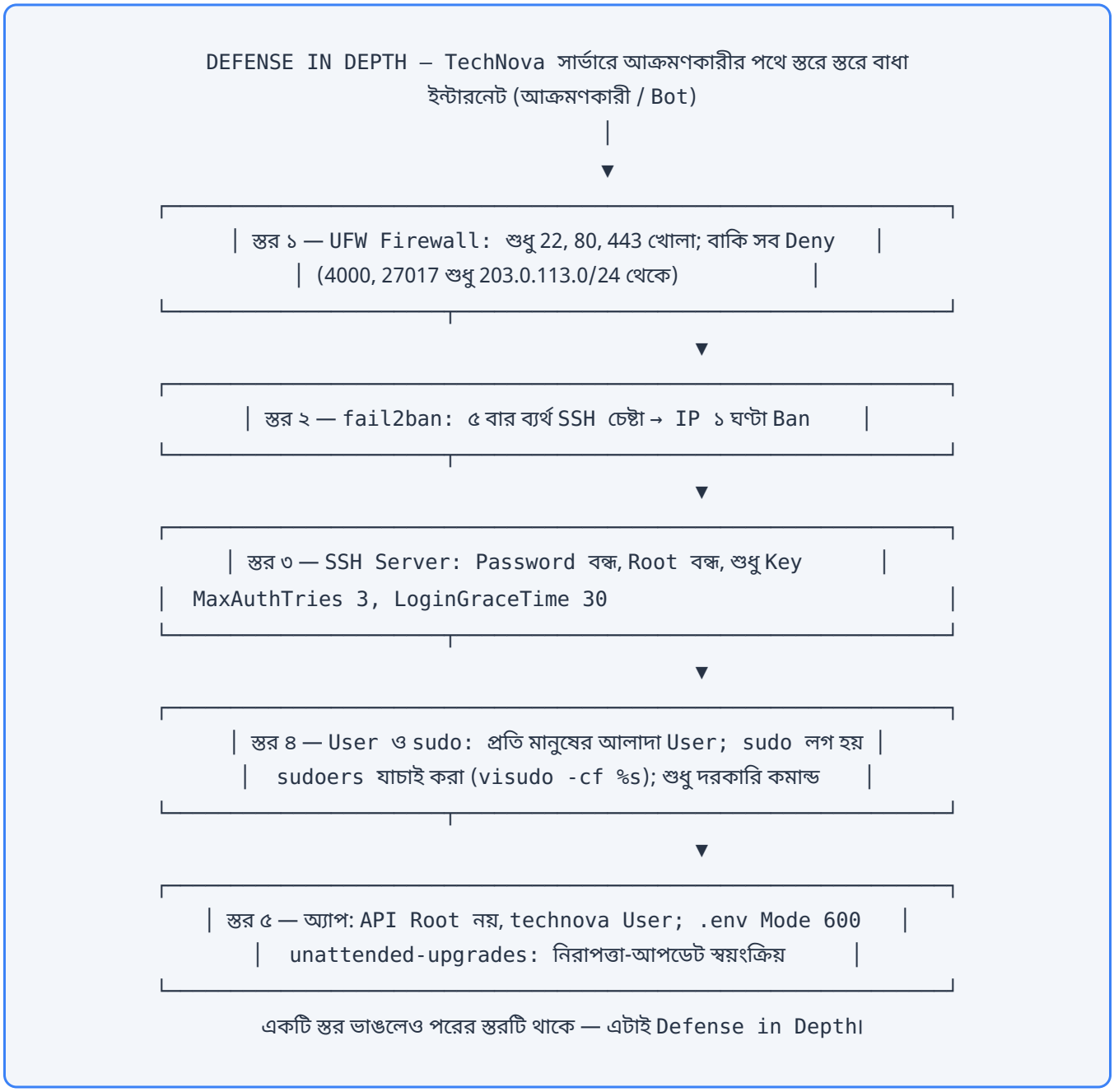
ব্যবহারকারী	`user`, `group`	প্রতি মানুষের আলাদা User; অ্যাপের জন্য System User
SSH Key	`authorized_key`	শুধু তালিকার Key; `exclusive: true`
sudo	`template` + `validate: visudo -cf %s`	`/etc/sudoers.d/`-এ আলাদা ফাইল, যাচাই করা
SSH Server	`lineinfile` + `sshd -t`	Password বন্ধ, Root বন্ধ, Key চালু
Firewall	`community.general.ufw`	ডিফল্ট Deny; শুধু 22/80/443 খোলা
Brute-Force প্রতিরোধ	fail2ban + `jail.local`	৫ বার ব্যর্থ হলে ১ ঘণ্টা Ban
আপডেট	unattended-upgrades	নিরাপত্তা-আপডেট স্বয়ংক্রিয়

১৯.৬ অভ্যন্তরীণ কার্যপ্রণালী

Hardening Playbook-এর ভেতরের ক্রমটি নিরাপত্তার জন্য সবচেয়ে গুরুত্বপূর্ণ — ক্রম ভুল হলে আপনি নিজেই সার্ভারের বাইরে আটকে যাবেন। সঠিক ক্রম: প্রথমে ব্যবহারকারী তৈরি ও SSH Key বসানো, তারপর sudoers লেখা ও যাচাই করা, তারপর Key দিয়ে লগইন পরীক্ষা করা (একটি আলাদা Task — `ansible.builtin.command: ssh -o BatchMode=yes ...` নয়, বরং Playbook-এর বাইরে হাতে পরীক্ষা করা নিরাপদ), তারপর sshd_config বদলানো, তারপর Firewall-এর Allow নিয়ম বসিয়ে Firewall চালু করা, তারপর fail2ban ও unattended-upgrades। `authorized_key` মডিউল কীভাবে কাজ করে: এটি `~/.ssh/authorized_keys` ফাইলটি পড়ে, তালিকার Key ইতিমধ্যে আছে কি না দেখে, না থাকলে যোগ করে, আর `exclusive: true` থাকলে তালিকার বাইরের Key মুছে দেয়। `validate` যেভাবে কাজ করে: Template মডিউল Rendered ফাইলটি প্রথমে অস্থায়ী ফাইলে লেখে, `visudo -cf /tmp/অস্থায়ী-ফাইল` চালায়, Exit Code 0 হলে তবেই আসল জায়গায় বসায় — Exit Code 0 না হলে Playbook ব্যর্থ হয় ও আসল ফাইল অক্ষত থাকে। `sshd -t` একইভাবে Config-এর Syntax যাচাই করে — ভুল Config নিয়ে SSH Reload হলে নতুন সংযোগ আর হবে না, তাই যাচাই ছাড়া Reload করা নিষিদ্ধ। UFW-এর ক্ষেত্রে Module প্রথমে নিয়মটি আছে কি না দেখে — একই নিয়ম দুবার যোগ হয় না — আর `state: enabled` দেওয়ার সময় সতর্কবার্তা দেয় যে SSH-এর নিয়ম না থাকলে সংযোগ হারাতে পারেন। fail2ban-এর ভেতরের কাজ: এটি `auth.log` পড়ে ব্যর্থ লগইনের প্যাটার্ন মেলায়, আর সীমা ছাড়ালে `iptables` দিয়ে IP-টি আটকায় — Ban-এর মেয়াদ শেষ হলে নিয়মটি নিজে থেকেই মুছে যায়।

> **WARNING:** এই অধ্যায়ের Playbook চালানোর আগে একটি Terminal খোলা রাখুন যেখানে আপনি ইতিমধ্যে সার্ভারে লগইন করে আছেন — যাতে ভুল হলে সেই Terminal থেকে ঠিক করা যায়। Password লগইন বন্ধ করার আগে নতুন Terminal থেকে Key দিয়ে লগইন পরীক্ষা করুন। এই দুটি সতর্কতা না মানলে একটি ভুল `sshd_config` লাইন আপনাকে সার্ভারের বাইরে আটকে দেবে, আর ঠিক করতে Console (ভৌত বা ক্লাউড Console) লাগবে — ল্যাবে তা কষ্টকর, প্রোডাকশনে তা বিপর্যয়।

১৯.৭ আর্কিটেকচার ও চিত্র



চিত্রটির মূল কথা: নিরাপত্তা একটি দরজা নয়, একটি গলি — আক্রমণকারীকে প্রতিটি মোড়ে একটি করে বাধা পার হতে হয়। কোনো একটি স্তরকে "যথেষ্ট" ভেবে বাকিগুলো বাদ দিলে সেই গলিটি খোলা রাস্তা হয়ে যায়।

১৯.৮ ব্যবহারিক উদাহরণ

sudoers Template — `templates/sudoers-technova.j2` (যাচাইসহ Deploy করা হয়):

```
# templates/sudoers-technova.j2 — /etc/sudoers.d/technova-deploy-এ বসবে
# nahid: Deploy-এর জন্য শুধু দরকারি কমান্ড Password ছাড়া
nahid ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart technova-api, /usr/bin/systemctl reload
nginx, /usr/bin/systemctl status technova-api
# deploy গ্রুপ: API Service পরিচালনা করতে পারে
%deploy ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart technova-api
```

```
# ব্যাখ্যা: সব কমান্ডের জন্য NOPASSWD: ALL ল্যাবে সহজ, কিন্তু প্রোডাকশনে শুধু
# নির্দিষ্ট কমান্ড দিন — উপরের মতো। কমান্ডের তালিকা বাড়লে আলাদা লাইন যোগ করুন।
```

sshd_config কঠোরকরণ — `lineinfile` দিয়ে প্রতিটি সেটিং:

```
# sshd_config কঠোরকরণ — প্রতিটি সেটিং একটি Task (বা Loop)
- name: Harden SSH server configuration
  ansible.builtin.lineinfile:
    path: /etc/ssh/sshd_config
    regexp: "^{{ item.key }}\\s"
    line: "{{ item.key }} {{ item.value }}"
    state: present
    validate: /usr/sbin/sshd -t -f %s
  loop:
    - { key: PasswordAuthentication, value: "no" }
    - { key: PermitRootLogin, value: "no" }
    - { key: PubkeyAuthentication, value: "yes" }
    - { key: PermitEmptyPasswords, value: "no" }
    - { key: X11Forwarding, value: "no" }
    - { key: MaxAuthTries, value: "3" }
    - { key: LoginGraceTime, value: "30" }
  notify: Reload SSH
# সতর্কতা: এই Task চালানোর আগে Key দিয়ে লগইন পরীক্ষা করুন (১৯.৬ দেখুন)।
# validate এখানে sshd -t দিয়ে Config যাচাই করে — ভুল Config বসবে না।
```

UFW নিয়ম — শুধু প্রয়োজনীয় Port:

```
# UFW — প্রথমে SSH Allow, তারপর Firewall চালু (ক্রমটি গুরুত্বপূর্ণ!)
- name: Allow SSH from lab network only
  community.general.ufw:
    rule: allow
    port: "22"
    proto: tcp
    src: 203.0.113.0/24

- name: Allow HTTP and HTTPS for everyone
  community.general.ufw:
    rule: allow
    port: "{{ item }}"
    proto: tcp
  loop: ["80", "443"]

- name: Allow API port from lab network only
  community.general.ufw:
    rule: allow
    port: "4000"
    proto: tcp
    src: 203.0.113.0/24

- name: Enable UFW with default deny policy
  community.general.ufw:
    state: enabled
    policy: deny
```

১৯.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচে নিরাপত্তা-কঠোরকরণের সম্পূর্ণ Playbook — `playbooks/hardening.yml`। ক্রমটি লক্ষ্য করুন: ব্যবহারকারী ও Key প্রথমে, sudoers যাচাইসহ, তারপর SSH, তারপর Firewall, তারপর fail2ban ও আপডেট।

```
---
# playbooks/hardening.yml — TechNova Security Hardening
# চালানোর কমান্ড: ansible-playbook -i inventory/hosts.yml playbooks/hardening.yml
# আগে: Key দিয়ে লগইন পরীক্ষা করুন ও একটি Terminal খোলা রাখুন (১৯.৬ দেখুন)।
- name: Harden all TechNova servers
  hosts: all
  become: true
  vars:
    technova_admin_keys:
      - "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExamplePublicKeyForNahid nahid@control"
      # বাস্তবে এখানে আপনার আসল Public Key বসান — Public Key গোপন নয়।
      # Private Key কখনো এখানে বা Repository-তে লিখবেন না।
  tasks:
    - name: Create deploy group
      ansible.builtin.group:
        name: deploy
        state: present

    - name: Create admin user nahid
      ansible.builtin.user:
        name: nahid
        groups: deploy,sudo
        append: true
        shell: /bin/bash
        create_home: true

    - name: Install admin SSH public keys
      ansible.builtin.authorized_key:
        user: nahid
        key: "{{ item }}"
        state: present
        exclusive: true
      loop: "{{ technova_admin_keys }}"

    - name: Deploy sudoers file with visudo validation
      ansible.builtin.template:
        src: sudoers-technova.j2
        dest: /etc/sudoers.d/technova-deploy
        owner: root
        group: root
        mode: "0440"
        validate: 'visudo -cf %s'

    - name: Harden SSH server configuration
      ansible.builtin.lineinfile:
        path: /etc/ssh/sshd_config
        regexp: "^{{ item.key }}\\s"
        line: "{{ item.key }} {{ item.value }}"
        state: present
        validate: /usr/sbin/sshd -t -f %s
      loop:
        - { key: PasswordAuthentication, value: "no" }
```

```
        - { key: PermitRootLogin, value: "no" }
        - { key: PubkeyAuthentication, value: "yes" }
        - { key: PermitEmptyPasswords, value: "no" }
        - { key: X11Forwarding, value: "no" }
        - { key: MaxAuthTries, value: "3" }
        - { key: LoginGraceTime, value: "30" }
      notify: Reload SSH

    - name: Install fail2ban and unattended-upgrades
      ansible.builtin.apt:
        name:
          - fail2ban
          - unattended-upgrades
        state: present
        update_cache: true

    - name: Configure fail2ban SSH jail
      ansible.builtin.template:
        src: jail.local.j2
        dest: /etc/fail2ban/jail.local
        mode: "0644"
      notify: Restart fail2ban

    - name: Allow SSH from lab network only
      community.general.ufw:
        rule: allow
        port: "22"
        proto: tcp
        src: 203.0.113.0/24

    - name: Allow HTTP and HTTPS
      community.general.ufw:
        rule: allow
        port: "{{ item }}"
        proto: tcp
      loop: ["80", "443"]

    - name: Enable UFW with default deny policy
      community.general.ufw:
        state: enabled
        policy: deny

  handlers:
    - name: Reload SSH
      ansible.builtin.service:
        name: ssh
        state: reloaded
    - name: Restart fail2ban
      ansible.builtin.service:
        name: fail2ban
        state: restarted
```

fail2ban Jail Template — `templates/jail.local.j2`:

```
# templates/jail.local.j2 — fail2ban SSH Jail
[sshd]
enabled = true
port = ssh
```

```
filter = sshd
logpath = /var/log/auth.log
maxretry = 5
bantime = 3600
findtime = 600
# অর্থ: ১০ মিনিটে ৫ বার ব্যর্থ লগইন হলে IP-টি ১ ঘণ্টার জন্য Ban হবে।
```

১৯.১০ ধাপে ধাপে বাস্তবায়ন

1. প্রথমে একটি Terminal খোলা রাখুন যেখানে আপনি ইতিমধ্যে সার্ভারে লগইন করে আছেন — এটি আপনার নিরাপত্তা-জাল (Safety Net)।
2. নতুন Terminal থেকে Key দিয়ে লগইন পরীক্ষা করুন: `ssh -i ~/.ssh/technova_lab nahid@203.0.113.10` — Password চাইলে এগোবেন না; Key-এর সমস্যা ঠিক করুন।
3. Syntax যাচাই: `ansible-playbook --syntax-check playbooks/hardening.yml`, তারপর Dry Run: `ansible-playbook -i inventory/hosts.yml playbooks/hardening.yml --check --diff`।
4. Playbook চালান: `ansible-playbook -i inventory/hosts.yml playbooks/hardening.yml`।
5. পরীক্ষা করুন: নতুন Terminal থেকে Key দিয়ে লগইন করুন (সফল হওয়া উচিত); Password দিয়ে লগইন চেষ্টা করুন (ব্যর্থ হওয়া উচিত); `ssh root@203.0.113.10` চেষ্টা করুন (ব্যর্থ হওয়া উচিত)।
6. Firewall যাচাই করুন: `sudo ufw status verbose` — শুধু প্রয়োজনীয় নিয়ম দেখা উচিত; বাইরে থেকে `curl http://203.0.113.10:4000/api/health` চেষ্টা করুন (ব্যর্থ হওয়া উচিত, কারণ 4000 শুধু ল্যাব নেটওয়ার্ক থেকে খোলা)।
7. fail2ban যাচাই করুন: `sudo fail2ban-client status sshd` — Jail চালু আছে কি না দেখুন।
8. দ্বিতীয়বার Playbook চালান — সব Task "ok" হওয়া উচিত (Idempotency)।

১৯.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

কঠোরকরণের পর যাচাইগুলো এরকম দেখাবে (চিত্রণমূলক আউটপুট):

```
$ ssh -i ~/.ssh/technova_lab nahid@203.0.113.10
Welcome to TechNova — web01.technova.example
# ব্যাখ্যা: Key দিয়ে লগইন সফল — Password চাওয়া হয়নি।

$ ssh root@203.0.113.10
Permission denied (publickey).
# ব্যাখ্যা: Root লগইন বন্ধ — সঠিক আচরণ।

$ sudo ufw status verbose
Status: active
Default: deny (incoming), allow (outgoing)
To                         Action      From
--                         ------      ----
22/tcp                     ALLOW       203.0.113.0/24
80/tcp                     ALLOW       Anywhere
443/tcp                    ALLOW       Anywhere
4000/tcp                   ALLOW       203.0.113.0/24
# ব্যাখ্যা: শুধু প্রয়োজনীয় Port খোলা; বাকি সব Deny।

$ sudo fail2ban-client status sshd
Status for the jail: sshd
```

```
|- Filter: currently failed: 0, total failed: 3
`- Actions: currently banned: 0, total banned: 0
# ব্যাখ্যা: Jail চালু; এখনো কেউ Ban হয়নি।

$ sudo visudo -cf /etc/sudoers.d/technova-deploy
/etc/sudoers.d/technova-deploy: parsed OK
# ব্যাখ্যা: sudoers ফাইলের Syntax ঠিক আছে।
```

একটি গুরুত্বপূর্ণ পরীক্ষা: Password দিয়ে লগইন চেষ্টা করলে সার্ভার `Permission denied (publickey)` বলবে — এটি ভুল নয়, এটি কঠোরকরণের প্রমাণ। যদি Key দিয়ে লগইন না হয়, তাহলে খোলা রাখা Terminal থেকে `sshd_config` পরীক্ষা করুন — `PasswordAuthentication` সাময়িকভাবে `yes` করে সমস্যা ঠিক করুন, তারপর আবার `no` করুন।

১৯.১২ সাধারণ ভুল

- **Key পরীক্ষা না করে Password বন্ধ করা:** সবচেয়ে সাধারণ ও বিপজ্জনক ভুল। সবসময় আগে Key লগইন পরীক্ষা করুন, তারপর Password বন্ধ করুন।
- **SSH Allow নিয়মের আগে Firewall চালু করা:** UFW চালু হলে SSH-ও বন্ধ হয়ে যায়। সবসময় প্রথমে Allow নিয়ম, তারপর Enable।
- **sudoers ফাইলে `validate` না দেওয়া:** একটি Syntax ভুলে পুরো sudo ভেঙে যায়। সবসময় `validate: 'visudo -cf %s'` ব্যবহার করুন।
- **মূল `/etc/sudoers` ফাইল এডিট করা:** ভুল হলে পুরো sudo ভাঙে। `/etc/sudoers.d/`-এ আলাদা ফাইল লিখুন।
- **`authorized_key`-এ `exclusive: true` না দেওয়া:** পুরনো কর্মীর Key ফাইলে থেকে যায়। সদস্য পরিবর্তন হলে তালিকা থেকে Key সরালেই তা মুছে যাওয়া উচিত — এজন্য `exclusive: true` দিন।
- **সবাইকে একটি User ভাগ করে দেওয়া:** লগে কে কী করেছে বোঝা যায় না। প্রতি মানুষের আলাদা User রাখুন।
- **fail2ban ইনস্টল করে Jail চালু না করা:** ডিফল্ট Jail সবসময় চালু থাকে না — `jail.local`-এ `enabled = true` লিখুন ও Service Restart করুন।
- **Firewall-এ 4000/27017 সবার জন্য খুলে দেওয়া:** API ও ডেটাবেসের Port শুধু ল্যাব নেটওয়ার্ক থেকে খোলা থাকবে — বাইরে থেকে নয়।

১৯.১৩ সমস্যা সমাধান (Troubleshooting)

1. **লক্ষণ:** Playbook-এর পর আর লগইন করা যাচ্ছে না — **পরীক্ষা:** খোলা রাখা Terminal থেকে `sudo sshd -T | grep -i passwordauthentication` ও `cat ~/.ssh/authorized_keys` দেখুন। **সমাধান:** Key ফাইলে Key ঠিক আছে কি না দেখুন; `~/.ssh`-এর Permission 700 ও `authorized_keys`-এর Permission 600 কি না দেখুন; প্রয়োজনে সাময়িকভাবে `PasswordAuthentication yes` করে Key ঠিক করুন।
2. **লক্ষণ:** `sudo: unable to resolve host` — **পরীক্ষা:** `/etc/hosts`-এ নিজের Hostname আছে কি না। **সমাধান:** অধ্যায় ১৭-এর `lineinfile` Task দিয়ে `/etc/hosts` ঠিক করুন — এটি sudo-র ত্রুটি নয়, Hostname-র সমস্যা।
3. **লক্ষণ:** `visudo: ... parse error` — Playbook থেমে গেছে — **পরীক্ষা:** Template-এর লাইনটি পড়ুন; sudoers-এ `%` চিহ্ন গ্রুপ বোঝায়, ভুল জায়গায় থাকলে Syntax ভাঙে। **সমাধান:** Template ঠিক করুন; আসল ফাইলটি অক্ষত আছে — `validate` সেটি নিশ্চিত করেছে।

4. **লক্ষণ:** UFW চালু করার পর Ansible সংযোগ হারিয়েছে — **পরীক্ষা:** Console থেকে `ufw status` দেখুন; SSH-এর Allow নিয়ম আছে কি না। **সমাধান:** Console থেকে SSH Allow নিয়ম যোগ করুন; ভবিষ্যতে Playbook-এ SSH নিয়মটি Enable-এর আগে রাখুন।
5. **লক্ষণ:** fail2ban নিজের IP-কে Ban করেছে — **পরীক্ষা:** `fail2ban-client status sshd`। **সমাধান:** `fail2ban-client set sshd unbanip <আপনার-IP>` দিয়ে Unban করুন, আর `jail.local`-এ `ignoreip = 127.0.0.1 203.0.113.0/24` যোগ করুন যাতে ল্যাবের IP Ban না হয়।
6. **লক্ষণ:** `community.general.ufw` মডিউল পাওয়া যায়নি — **পরীক্ষা:** `ansible-galaxy collection list | grep community.general`। **সমাধান:** Collection ইনস্টল করুন; না হলে `ufw` কমান্ড `command` মডিউল দিয়ে চালানো যায়, তবে তা Idempotent নয় — Collection ব্যবহারই সঠিক।

১৯.১৪ নিরাপত্তা বিবেচনা

> **WARNING:** Password-ছাড়া sudo (`NOPASSWD: ALL`) মানে যে কেউ আপনার User-এ ঢুকলেই Root পেয়ে যায় — Key চুরি হলে আর কোনো বাধা থাকে না। ল্যাবে এটি সুবিধাজনক হলেও প্রোডাকশনে শুধু নির্দিষ্ট কমান্ডের জন্য `NOPASSWD` দিন, আর বাকি sudo-তে Password চান। আর Private Key কখনোই Password ছাড়া রাখবেন না — Key-তে Passphrase দিন, যাতে ল্যাপটপ চুরি হলেও Key ব্যবহার করা না যায়।

এই অধ্যায়ের নিরাপত্তা-দিকগুলো একসঙ্গে দেখুন: প্রথমত, Key-ভিত্তিক লগইন Password-এর চেয়ে শক্তিশালী, তবে Key-এর নিরাপত্তা আপনার ল্যাপটপের নিরাপত্তার ওপর নির্ভর করে — Private Key-এর Permission 600 ও Passphrase বাধ্যতামূলক। দ্বিতীয়ত, `exclusive: true` দিয়ে পুরনো Key মুছে দেওয়া মানে কর্মী চলে গেলে তার প্রবেশাধিকার স্বয়ংক্রিয়ভাবে শেষ হয় — এটি একটি প্রক্রিয়া, শুধু একটি Task নয়। তৃতীয়ত, Firewall-এর নিয়মগুলো নিয়মিত পর্যালোচনা করুন — অস্থায়ীভাবে খোলা Port ভুলে খোলা থেকে যায়, এটাই সবচেয়ে সাধারণ Firewall-ভুল। চতুর্থত, fail2ban আক্রমণ থামায় না, শুধু ধীর করে — আসল প্রতিরক্ষা হলো Key-লগইন ও Firewall। পঞ্চমত, unattended-upgrades স্বয়ংক্রিয় আপডেট দিলেও Kernel আপডেটের পর Reboot দরকার — Reboot-এর পরিকল্পনা ছাড়া আপডেট অসম্পূর্ণ থাকে। সবশেষে, CIS Benchmark-এর প্রতিটি নিয়ম অন্ধভাবে প্রয়োগ করবেন না — কিছু নিয়ম আপনার অ্যাপ ভেঙে দিতে পারে; প্রতিটি নিয়মের পেছনের কারণ বুঝে তবেই প্রয়োগ করুন।

১৯.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

প্রোডাকশনে নিরাপত্তা-কঠোরকরণের নিয়ম: প্রথমত, Hardening Playbook Git-এ রাখুন ও প্রতিটি পরিবর্তন Review করুন — নিরাপত্তা-পরিবর্তনে একজন Reviewer বাধ্যতামূলক। দ্বিতীয়ত, Hardening নিয়মিত চালান (Drift Correction) — কেউ হাতে Firewall-এ Port খুললে পরের Run-এ তা বন্ধ হয়ে যাবে। তৃতীয়ত, SSH-এর জন্য শুধু নির্দিষ্ট IP (যেমন অফিসের IP বা VPN) থেকে Allow করুন — ইন্টারনেটের সবার জন্য Port 22 খোলা রাখা প্রোডাকশনে এড়িয়ে চলুন। চতুর্থত, sudo-র লগ (`/var/log/auth.log`) নিয়মিত পর্যালোচনা করুন — কে, কখন, কী sudo করেছে, তা দেখা নিরাপত্তা-পর্যালোচনার অংশ। পঞ্চমত, Key Rotation-এর পরিকল্পনা করুন — নির্দিষ্ট সময় পরপর Key বদলান, আর কর্মী চলে গেলে সঙ্গে সঙ্গে Key মুছে ফেলুন। ষষ্ঠত, CIS Benchmark-এর বিরুদ্ধে নিয়মিত Audit চালান — Ansible-এর জন্য CIS-ভিত্তিক Role আছে, তবে তা প্রয়োগের আগে Staging-এ পরীক্ষা করুন। সবশেষে, Hardening-এর পর সবসময় একটি সম্পূর্ণ Smoke Test করুন — অ্যাপের Health Check, লগইন, ও Deploy — তিনটিই কাজ করছে কি না দেখুন; নিরাপত্তা বাড়াতে গিয়ে অ্যাপ বন্ধ করে দেওয়া সবচেয়ে সাধারণ প্রোডাকশন-ভুল।

১৯.১৬ অধ্যায়ের সারাংশ

Hardening মানে আক্রমণের পৃষ্ঠ ছোট করা — স্তরে স্তরে: ব্যবহারকারী ও গ্রুপ, SSH Key, যাচাই করা sudoers, কঠোর sshd_config, UFW Firewall, fail2ban, আর স্বয়ংক্রিয় নিরাপত্তা-আপডেট। ক্রমটি গুরুত্বপূর্ণ: Key প্রথমে, Password বন্ধ পরে; SSH Allow প্রথমে, Firewall চালু পরে; sudoers-এ `validate: 'visudo -cf %s'` সবসময়। `PasswordAuthentication no` ও `PermitRootLogin no` — দুটি লাইনেই অধিকাংশ স্বয়ংক্রিয় আক্রমণ থেমে যায়, তবে আসল নিরাপত্তা আসে Defense in Depth থেকে — একটি স্তর ভাঙলেও পরের স্তরটি থাকে। CIS Benchmark একটি Checklist, অন্ধ নিয়ম নয় — প্রতিটি নিয়মের কারণ বুঝে প্রয়োগ করুন।

১৯.১৭ জ্ঞান যাচাই

1. `validate: 'visudo -cf %s'` কেন অপরিহার্য? এটি না থাকলে একটি Syntax ভুলে কী ঘটত?
2. Password লগইনের বদলে SSH Key কেন নিরাপদ — দুটি কারণ লিখুন।
3. UFW-এ SSH Allow নিয়মের আগে Firewall চালু করলে কী ঘটে, এবং কেন?
4. `authorized_key`-এ `exclusive: true`-এর অর্থ কী, আর কর্মী চলে গেলে এটি কীভাবে সাহায্য করে?
5. fail2ban আক্রমণ থামায় না — তাহলে এটি কেন ব্যবহার করা হয়? এর সীমা কী?

১৯.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. **সম্পূর্ণ Hardening:** ল্যাবে উপরের Playbook চালান; Key লগইন, Password লগইন ব্যর্থতা, Root লগইন ব্যর্থতা, ও UFW Status — চারটি যাচাইয়ের আউটপুট সংরক্ষণ করুন।
2. **নিজেকে আটকে ফেলুন (নিরাপদে):** একটি অতিরিক্ত ল্যাব সার্ভারে ইচ্ছে করে ভুল `sshd_config` লাইন লিখে দেখুন `validate` কীভাবে Playbook থামিয়ে দেয়; তারপর `validate` ছাড়া একই কাজ করলে কী ঘটত তা লিখুন (শুধু ল্যাবে, Console-এর ব্যবস্থা রেখে)।
3. **CIS Checklist:** CIS Benchmark-এর (সরকারি ডকুমেন্টেশন থেকে) ১০টি নিয়ম বেছে TechNova-র জন্য একটি Checklist বানান — প্রতিটি নিয়মের পাশে লিখুন এটি আপনার Playbook-এ আছে কি না, আর না থাকলে কেন।

অধ্যায় 20 • PART IV — Cloud Automation

Chapter 20 — AWS, Azure, GCP, and IBM Cloud Automation: Dynamic Inventory and Cloud VM Configuration

২০.১ ভূমিকা

এতদিন আমরা জানতাম সার্ভারগুলো কোথায় আছে — ল্যাবের তিনটি IP ঠিক লেখা ছিল Inventory-তে। ক্লাউডে এই নিশ্চয়তা থাকে না: Virtual Machine (VM) তৈরি হয়, মুছে যায়, IP বদলায়, নতুন Region-এ নতুন VM আসে — আর প্রতিবার Inventory ফাইলে হাতে IP লেখা অসম্ভব। এই অধ্যায়ের বিষয় হলো ক্লাউড-অটোমেশন: Ansible কীভাবে ক্লাউড থেকে সরাসরি জানে কোন VM চলছে (Dynamic Inventory), কীভাবে সেই VM-গুলোতে আগের অধ্যায়গুলোর Baseline, Deploy ও Hardening Playbook চালানো যায়, আর কীভাবে প্রমাণপত্র (Credential) নিরাপদে পরিচালনা করা যায়। একটি গুরুত্বপূর্ণ সীমানা প্রথমেই পরিষ্কার করে নিই: Ansible VM তৈরি করার জন্য নয় — VM তৈরি করা (Provisioning) Terraform-এর কাজ, আর VM তৈরি হওয়ার পর তার ভেতরে সফটওয়্যার বসানো (Configuration) Ansible-এর কাজ। এই বইয়ে আমরা সেই সীমানা মেনে চলব: Provisioning-এর উদাহরণে সঠিক Module-এর নাম দেওয়া হবে, তবে মূল কাজ হবে তৈরি হওয়া VM-কে Dynamic Inventory দিয়ে খুঁজে তার ওপরে Configuration চালানো। AWS, Azure, GCP ও IBM Cloud — চারটি ক্লাউডেই নীতি এক, শুধু প্রমাণপত্রের পদ্ধতি ও Inventory Plugin-এর নাম আলাদা। আর একটি সতর্কতা: ক্লাউড রিসোর্সে চার্জ হয় — ল্যাবের VM চালু রেখে ভুলে গেলে বিল আসবে; ল্যাব শেষে সবসময় রিসোর্স মুছে ফেলুন। এই অধ্যায়ে কোনো মূল্য বা সেবার সক্ষমতা উদ্ভাবন করা হয়নি — সুনির্দিষ্ট মূল্য, সীমা ও সক্ষমতার জন্য সংশ্লিষ্ট ক্লাউডের সরকারি (Official) ডকুমেন্টেশনই চূড়ান্ত কর্তৃপক্ষ।

> **CONCEPT:** Static Inventory মানে "IP লিখে রাখা" — সার্ভার স্থির থাকলে চলে। Dynamic Inventory মানে "ক্লাউডকে জিজ্ঞেস করা" — Ansible চালানোর সময় ক্লাউড API থেকে জানে কোন VM চলছে, তাদের IP কী, কোন Tag আছে, কোন Region-এ আছে। VM বদলালে Inventory নিজে থেকেই বদলায় — ফাইল এডিট করতে হয় না। Dynamic Inventory Plugin হলো সেই জিজ্ঞেস করার উপায়: AWS-এর জন্য `aws_ec2`, Azure-এর জন্য `azure_rm`, GCP-র জন্য `gcp_compute`।

২০.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Static ও Dynamic Inventory-এর পার্থক্য ব্যাখ্যা করতে এবং ক্লাউডে Dynamic Inventory কেন দরকার তা বলতে; (২) AWS-এর `aws_ec2`, Azure-এর `azure_rm` ও GCP-র `gcp_compute` Plugin-এর নাম, Collection ও প্রাথমিক Config লিখতে; (৩) ক্লাউড প্রমাণপত্র Environment Variable ও Credentials ফাইলের মাধ্যমে দিতে এবং Hardcode করার বিপদ ব্যাখ্যা করতে; (৪) Provisioning (Terraform) ও Configuration (Ansible)-এর সীমানা ব্যাখ্যা করতে এবং `amazon.aws.ec2_instance`-কে সঠিক প্রসঙ্গে স্থাপন করতে; (৫) ক্লাউড VM-এ Baseline ও Hardening Playbook চালানোর Workflow বর্ণনা করতে; (৬) IAM/Access Permission-এর মূলনীতি (Least Privilege) ব্যাখ্যা করতে; (৭) IBM Cloud-এর ক্ষেত্রে Dynamic Inventory-এর সীমা ও বিকল্প (Constructed Inventory) বর্ণনা করতে; (৮) ল্যাব-খরচের ঝুঁকি ব্যাখ্যা করতে ও রিসোর্স মুছে ফেলার Checklist তৈরি করতে।

২০.৩ পূর্বশর্ত ধারণা

এই অধ্যায়ের জন্য আপনার প্রয়োজন: অধ্যায় ১৭–১৯-এর Playbook (Baseline, Deploy, Hardening) — কারণ ক্লাউড VM-এ এগুলোই চালানো হবে; Inventory ও Group Variable-এর ধারণা; Environment Variable-এর ধারণা; আর ক্লাউডের প্রাথমিক ধারণা — Region, Instance/VM, Tag, Security Group/Firewall, IAM (Identity and Access Management — কে কী করতে পারবে)। কোনো ক্লাউড অ্যাকাউন্ট এই অধ্যায়ের জন্য বাধ্যতামূলক নয় — উদাহরণগুলো পড়ে ও Dry Run-এর ধারণা নিয়ে বোঝা যাবে; ল্যাব করতে চাইলে যেকোনো একটি ক্লাউডের Free Tier বা Trial ব্যবহার করুন, আর সরকারি ডকুমেন্টেশন থেকে সীমা ও শর্ত দেখে নিন। Terraform-এর প্রাথমিক ধারণা থাকলে ভালো, না থাকলে এই অধ্যায়ে শুধু এটুকু জানলেই চলবে: Terraform Infrastructure তৈরি করে, Ansible তার ভেতরে Configuration করে।

২০.৪ প্রথম নীতি থেকে ব্যাখ্যা

ক্লাউডকে ভাবুন একটি ভাড়া-বাড়ির বাজার হিসেবে। আপনি একটি বাড়ি (VM) ভাড়া নেন — তার ঠিকানা (IP) ভাড়া নেওয়ার সময় জানা যায়, ভাড়া শেষ হলে বাড়িটি ফেরত যায়, আর নতুন বাড়ি নিলে নতুন ঠিকানা আসে। আপনার অতিথি-তালিকা (Inventory) যদি কাগজে লেখা ঠিকানার তালিকা হয়, তাহলে প্রতিবার বাড়ি বদলালে তালিকা হাতে বদলাতে হয় — ভুল হবেই। Dynamic Inventory মানে বাজারের অফিসকে জিজ্ঞেস করা: "এই মুহূর্তে আমার নামে কোন বাড়ি ভাড়া আছে?" — অফিসের খাতা (Cloud API) থেকে ঠিকানাগুলো এনে দেওয়া হয়। প্রমাণপত্র (Credential) মানে আপনার পরিচয়পত্র — অফিসকে দেখাতে হয় আপনি কে, নইলে অফিস আপনার বাড়ির ঠিকানা দেবে না। পরিচয়পত্রটি যদি আপনি চিঠিতে (Playbook-এ) লিখে রাখেন, তাহলে চিঠিটি যে পড়বে সে-ই আপনার নামে বাড়ি ভাড়া নিতে পারবে — তাই পরিচয়পত্র কখনো চিঠিতে লেখা যায় না; এটি থাকে আপনার পকেটে (Environment Variable) বা তালাবদ্ধ বাক্সে (Credentials ফাইল), আর চিঠিতে শুধু লেখা থাকে "পরিচয়পত্রটি পকেট থেকে নাও"। IAM মানে অফিসের নিয়ম: আপনার পরিচয়পত্র দিয়ে কী কী করা যায় — শুধু বাড়ির ঠিকানা দেখা যায়, নাকি বাড়ি ভাড়া নেওয়া ও ভাঙাও যায়? Least Privilege মানে পরিচয়পত্রে শুধু যতটুকু দরকার ততটুকু অধিকার দেওয়া — Ansible-এর পরিচয়পত্রে VM-এর তথ্য পড়া ও VM-এ SSH করার অধিকার থাকলেই চলে, VM মুছে ফেলার অধিকার থাকা উচিত নয়।

২০.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

AWS — aws_ec2 Dynamic Inventory

AWS-এ Dynamic Inventory-র জন্য `amazon.aws` Collection-এর `aws_ec2` Plugin ব্যবহার করা হয়। Config ফাইলটি YAML আকারে লেখা হয় (`inventory/aws_ec2.yml`), যাতে থাকে `plugin: amazon.aws.aws_ec2`, `regions` (যেমন `ap-south-1`), আর `filters` — যেমন Tag দিয়ে ছাঁকা: `tag:Environment: lab`। Ansible চালানোর সময় Plugin AWS API-কে জিজ্ঞেস করে চলমান EC2 Instance-গুলোর তালিকা আনে, আর প্রতিটি Instance-কে একটি Host বানায় — Host-এর নাম হয় Instance ID বা Tag থেকে নেওয়া নাম, আর `ansible_host` হয় Public বা Private IP। `keyed_groups` দিয়ে Tag-এর ভিত্তিতে গ্রুপ তৈরি করা যায় — যেমন `tag:Role: web` থাকা Instance-গুলো `webservers` গ্রুপে যায়, ফলে আগের অধ্যায়ের Playbook-গুলো (`hosts: webservers`) কোনো পরিবর্তন ছাড়াই চলে। প্রমাণপত্রের জন্য AWS-এ Access Key ID ও Secret Access Key Environment Variable-এ দেওয়া হয় (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`) অথবা `~/.aws/credentials` ফাইলে রাখা হয় — কখনোই Playbook বা Inventory ফাইলে নয়।

Azure — azure_rm Dynamic Inventory

Azure-এ Dynamic Inventory-র জন্য `azure.azcollection` Collection-এর `azure_rm` Plugin ব্যবহার করা হয়। Config-এ থাকে `plugin: azure.azcollection.azure_rm`, Subscription ID, আর Resource Group বা Tag দিয়ে

ছাঁকা। Azure-এ প্রমাণপত্রের জন্য Service Principal ব্যবহার করা হয় — একটি অ্যাপ্লিকেশন-পরিচয় যার Client ID, Client Secret ও Tenant ID থাকে; এগুলো Environment Variable-এ দেওয়া হয় (`AZURE_CLIENT_ID`, `AZURE_SECRET`, `AZURE_TENANT`, `AZURE_SUBSCRIPTION_ID`) অথবা `~/.azure/credentials` ফাইলে রাখা হয়। Azure-এ VM-এর Network Security Group (NSG) হলো Firewall-এর সমতুল্য — Ansible দিয়ে VM কনফিগার করার আগে NSG-তে SSH (22) খোলা আছে কি না তা যাচাই করুন; NSG-তে Port খোলা না থাকলে Ansible সংযোগই করতে পারবে না। Azure-এর Tag-ভিত্তিক Grouping AWS-এর মতোই `keyed_groups` দিয়ে করা যায়।

GCP — gcp_compute Dynamic Inventory

GCP-তে Dynamic Inventory-র জন্য `google.cloud` Collection-এর `gcp_compute` Plugin ব্যবহার করা হয়। Config-এ থাকে `plugin: google.cloud.gcp_compute`, Project ID, আর Zone-এর তালিকা। GCP-তে প্রমাণপত্রের জন্য Service Account Key (JSON ফাইল) ব্যবহার করা হয় — ফাইলের পথ Environment Variable-এ দেওয়া হয় (`GCP_SERVICE_ACCOUNT_FILE` বা `GOOGLE_APPLICATION_CREDENTIALS`), আর Key ফাইলটির Permission 600 রাখা হয়। Service Account-এর অধিকারও Least Privilege মেনে দেওয়া হয় — শুধু Compute Instance-এর তথ্য পড়ার অধিকার। GCP-তে Firewall Rule আলাদা Resource — VM-এ SSH করার আগে Firewall-এ Port 22 খোলা আছে কি না যাচাই করুন।

IBM Cloud — সীমা ও বিকল্প

IBM Cloud-এর জন্য Ansible Core-এ AWS/Azure/GCP-এর মতো কোনো আদর্শ Dynamic Inventory Plugin নেই — এটি একটি সত্য, আর এই সত্যটি স্বীকার করাই সঠিক। IBM Cloud-এ দুটি বাস্তবসম্মত পথ আছে। প্রথম পথ: `ibm.cloud` Collection (যদি আপনার পরিবেশে উপলব্ধ থাকে) ও তার Inventory Plugin — ব্যবহারের আগে Collection-এর সরকারি ডকুমেন্টেশন থেকে Plugin-এর নাম, সংস্করণ ও সমর্থন যাচাই করুন। দ্বিতীয় পথ — যা বেশি নির্ভরযোগ্য: **Constructed Inventory** — IBM Cloud CLI (`ibmcloud`) বা API দিয়ে চলমান Instance-গুলোর তালিকা JSON আকারে এনে, সেই তালিকা থেকে একটি Inventory ফাইল তৈরি করা (একটি ছোট Script দিয়ে), তারপর Ansible-কে সেই Inventory দিয়ে চালানো। এই পথে Inventory ফাইলটি "Static" হলেও তা মানুষ হাতে লেখে না — মেশিন তৈরি করে, তাই ভুল কম হয়। IBM Cloud-এর প্রমাণপত্রের জন্য API Key ব্যবহার করা হয় (`IBMCLOUD_API_KEY` Environment Variable), আর Key-টি কখনোই Playbook-এ লেখা যাবে না।

Provisioning বনাম Configuration — সীমানাটি কেন গুরুত্বপূর্ণ

`amazon.aws.ec2_instance` একটি Provisioning Module — এটি EC2 Instance তৈরি, চালু, বন্ধ ও মুছে ফেলতে পারে। এটি দিয়ে VM তৈরি করা যায়, তবে এই বইয়ের সুপারিশ হলো Provisioning-এর জন্য Terraform ব্যবহার করা — কারণ Terraform Infrastructure-এর সম্পূর্ণ অবস্থা (State) ধরে রাখে, পরিবর্তনের পরিকল্পনা (`terraform plan`) দেখায়, আর Infrastructure-কে কোড হিসেবে পরিচালনা করে। Ansible-এর ভূমিকা শুরু হয় VM তৈরি হওয়ার পর: Dynamic Inventory দিয়ে VM খুঁজে, তার ওপরে Baseline, Deploy ও Hardening Playbook চালানো। একটি সাধারণ Workflow: Terraform VM তৈরি করে ও তার IP/Tag Output দেয় → Ansible Dynamic Inventory দিয়ে VM খুঁজে পায় → Ansible Configuration চালায়। এই সীমানা মেনে চললে দুটি টুলের কাজ আলাদা থাকে, আর কোনো একটি টুল দিয়ে অন্যটির কাজ করতে গিয়ে জটিলতা তৈরি হয় না।

IAM ও Access Permission — Least Privilege

IAM (Identity and Access Management) মানে "কে, কী, কোথায় করতে পারবে" তার নিয়ম। Ansible-এর জন্য একটি আলাদা পরিচয় (AWS IAM User/Role, Azure Service Principal, GCP Service Account) তৈরি করুন, আর তাকে শুধু যতটুকু দরকার ততটুকু অধিকার দিন: Instance-এর তথ্য পড়া (Read), Tag পড়া, আর VM-এ SSH করার জন্য Key-এর অধিকার — VM তৈরি বা মোছার অধিকার নয়। অধিকার বেশি দিলে একটি ভুল Playbook বা চুরি হওয়া Credential

পুরো ক্লাউড-অ্যাকাউন্ট মুছে ফেলতে পারে। প্রতিটি ক্লাউডে অধিকারের নাম ও পদ্ধতি আলাদা — সুনির্দিষ্ট Policy-র নাম ও ধাপের জন্য সংশ্লিষ্ট ক্লাউডের সরকারি IAM ডকুমেন্টেশন দেখুন; এই বইয়ে কোনো Policy-র নাম উদ্ভাবন করা হয়নি।

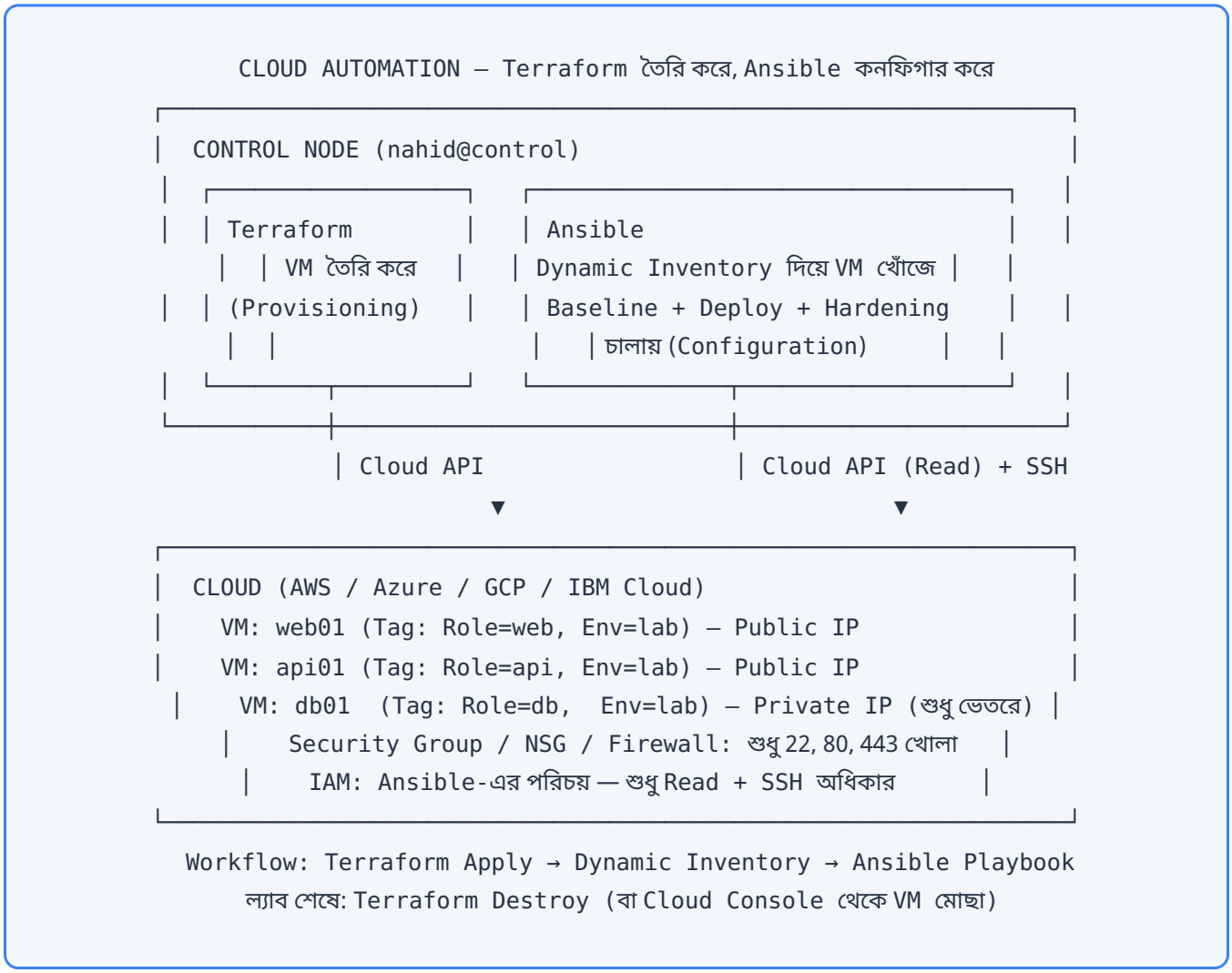
ক্লাউড	Dynamic Inventory Plugin	Collection	প্রমাণপত্রের পদ্ধতি
AWS	`aws_ec2`	`amazon.aws`	Env Var (`AWS_ACCESS_KEY_ID` ইত্যাদি) বা `~/.aws/credentials`
Azure	`azure_rm`	`azure.azcollection`	Service Principal Env Var বা `~/.azure/credentials`
GCP	`gcp_compute`	`google.cloud`	Service Account JSON ফাইল (Env Var-এ পথ)
IBM Cloud	আদর্শ Plugin নেই — Constructed Inventory	`ibm.cloud` (যাচাই করে ব্যবহার করুন)	API Key Env Var (`IBMCLOUD_API_KEY`)

২০.৬ অভ্যন্তরীণ কার্যপ্রণালী

Dynamic Inventory চালানোর সময় ভেতরে যা ঘটে: প্রথমে Ansible Inventory Plugin-টি লোড করে ও Config ফাইলটি পড়ে; তারপর Plugin ক্লাউড API-কে একটি অনুরোধ পাঠায় — "এই Region-এ, এই Filter মেনে, কোন Instance চলছে?" — প্রমাণপত্রটি Environment Variable বা Credentials ফাইল থেকে নিয়ে অনুরোধের Header-এ বসানো হয়। ক্লাউড API Instance-গুলোর তালিকা JSON আকারে ফেরত পাঠায় — প্রতিটি Instance-এর ID, IP (Public/Private), Tag, State (running/stopped), Region ও Zone। Plugin সেই তালিকা থেকে Ansible Host তৈরি করে: Host-এর নাম সাধারণত Instance ID বা `Name` Tag, `ansible_host` হয় IP, আর Tag-গুলো থেকে Group তৈরি হয় (`keyed_groups`)। এরপর Ansible স্বাভাবিকভাবে চলে — প্রতিটি Host-এ SSH করে Fact সংগ্রহ করে, Playbook-এর Task চালায়। একটি গুরুত্বপূর্ণ বিষয়: Dynamic Inventory প্রতিবার Ansible চালানোর সময় নতুন করে API-কে জিজ্ঞেস করে — তাই VM তৈরি বা মোছার পরের Run-এ Inventory নিজে থেকেই আপডেট থাকে। Cache ব্যবহার করলে (`cache: true` ও `cache_plugin`) API-কে বারবার জিজ্ঞেস করা কমে, তবে Cache পুরনো হলে নতুন VM দেখা যায় না — ল্যাবে Cache বন্ধ রাখাই নিরাপদ। আরেকটি বিষয়: API অনুরোধে সময় লাগে — শত শত Instance থাকলে Inventory লোড হতে কয়েক সেকেন্ড থেকে কয়েক মিনিট লাগতে পারে; `filters` দিয়ে তালিকা ছোট রাখলে তা দ্রুত হয়।

> **WARNING — খরচ:** ক্লাউড রিসোর্সে চার্জ হয় — VM চালু থাকলেই বিল আসে, আপনি ব্যবহার করুন বা না করুন। ল্যাবের VM তৈরি করার আগে সংশ্লিষ্ট ক্লাউডের সরকারি মূল্য-পৃষ্ঠা (Pricing Page) দেখে নিন, সবচেয়ে ছোট আকারের VM বেছে নিন, আর ল্যাব শেষে VM মুছে ফেলুন — শুধু বন্ধ (Stop) করলেই যথেষ্ট নয়, কারণ বন্ধ VM-এর Disk ও IP-র জন্যও চার্জ হতে পারে (সুনির্দিষ্ট নিয়মের জন্য সরকারি ডকুমেন্টেশন দেখুন)। এই বইয়ে কোনো মূল্য উল্লেখ করা হয়নি — কারণ মূল্য পরিবর্তনশীল ও Region-ভেদে আলাদা।

২০.৭ আর্কিটেকচার ও চিত্র



চিত্রটির মূল কথা: দুটি টুলের সীমানা স্পষ্ট — Terraform VM-এর অস্তিত্বের দায়িত্বে, Ansible VM-এর ভেতরের অবস্থার দায়িত্বে। Dynamic Inventory হলো দুটির সংযোগ-সেতু — Terraform যা তৈরি করে, Ansible তা খুঁজে পায়।

২০.৮ ব্যবহারিক উদাহরণ

AWS Dynamic Inventory — `inventory/aws_ec2.yml`:

```
# inventory/aws_ec2.yml — AWS Dynamic Inventory (amazon.aws.aws_ec2)
plugin: amazon.aws.aws_ec2
regions:
  - ap-south-1
filters:
  tag:Environment: lab
  instance-state-name: running
keyed_groups:
  - key: tags.Role
    prefix: role
    separator: ""
hostnames:
  - tag:Name
compose:
```

```
  ansible_host: public_ip_address
# চালানোর আগে: export AWS_ACCESS_KEY_ID=CHANGE-ME-...
#               export AWS_SECRET_ACCESS_KEY=CHANGE-ME-...
# যাচাই: ansible-inventory -i inventory/aws_ec2.yml --list
```

Azure Dynamic Inventory — `inventory/azure_rm.yml`:

```
# inventory/azure_rm.yml — Azure Dynamic Inventory (azure.azcollection.azure_rm)
plugin: azure.azcollection.azure_rm
include_vm_resource_groups:
  - technova-lab-rg
auth_source: env
keyed_groups:
  - key: tags.Role
    prefix: role
    separator: ""
hostnames:
  - name
# চালানোর আগে: export AZURE_CLIENT_ID=CHANGE-ME-...
#               export AZURE_SECRET=CHANGE-ME-...
#               export AZURE_TENANT=CHANGE-ME-...
#               export AZURE_SUBSCRIPTION_ID=CHANGE-ME-...
# যাচাই: ansible-inventory -i inventory/azure_rm.yml --list
```

GCP Dynamic Inventory — `inventory/gcp_compute.yml`:

```
# inventory/gcp_compute.yml — GCP Dynamic Inventory (google.cloud.gcp_compute)
plugin: google.cloud.gcp_compute
projects:
  - technova-lab-project
zones:
  - asia-south1-a
filters:
  - labels.environment = lab
  - status = RUNNING
keyed_groups:
  - key: labels.role
    prefix: role
    separator: ""
hostnames:
  - name
compose:
  ansible_host: networkInterfaces[0].accessConfigs[0].natIP
# চালানোর আগে: export GOOGLE_APPLICATION_CREDENTIALS=/home/nahid/.gcp/technova-lab.json
# যাচাই: ansible-inventory -i inventory/gcp_compute.yml --list
```

Provisioning Module-এর সঠিক প্রসঙ্গ — `amazon.aws.ec2_instance` (উদাহরণ, সুপারিশ নয়):

```
# উদাহরণ — amazon.aws.ec2_instance দিয়ে EC2 তৈরি করা যায়, তবে এই বইয়ে
# Provisioning-এর জন্য Terraform সুপারিশ করা হয়েছে (২০.৫ দেখুন)।
# নিচের Task-টি শুধু Module-এর নাম ও আকৃতি দেখানোর জন্য — ল্যাবে চালানোর
# আগে সরকারি ডকুমেন্টেশন থেকে প্যারামিটার যাচাই করুন ও খরচ দেখুন।
- name: Create a lab EC2 instance (example only)
  amazon.aws.ec2_instance:
    name: technova-lab-web01
    instance_type: t3.micro
```

```
    image_id: ami-CHANGE-ME
    region: ap-south-1
    security_group: technova-lab-sg
    key_name: technova-lab
    tags:
      Environment: lab
      Role: web
    state: present
  # সতর্কতা: এই Task চালালে AWS-এ চার্জ হবে; ল্যাব শেষে Instance মুছে ফেলুন।
```

২০.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচে ক্লাউড VM-এ Baseline চালানোর সম্পূর্ণ Workflow ফাইল — `playbooks/cloud-baseline.yml` — যা Dynamic Inventory-র গ্রুপের ওপরে অধ্যায় ১৭-এর Baseline Role চালায়। Role-টি আগের অধ্যায়ের `roles/common/` থেকে নেওয়া — Playbook-টি নিজে কোনো সার্ভারের নাম জানে না, শুধু গ্রুপের নাম জানে।

```
---
# playbooks/cloud-baseline.yml — Cloud VM-এ Baseline চালানো
# চালানোর কমান্ড (AWS): ansible-playbook -i inventory/aws_ec2.yml playbooks/cloud-baseline.yml
# চালানোর আগে প্রমাণপত্র Environment Variable-এ দিন (২০.৮ দেখুন)।
- name: Baseline for cloud VMs (all roles)
  hosts: all
  become: true
  gather_facts: true
  roles:
    - common

- name: Harden cloud VMs
  hosts: all
  become: true
  roles:
    - hardening
  # common ও hardening Role দুটি অধ্যায় ১৭ ও ১৯-এর Playbook থেকে নেওয়া।
  # Role-এ রূপান্তর না করে থাকলে Playbook-টি সরাসরি Task দিয়েও লেখা যায়।

# IBM Cloud — Constructed Inventory তৈরির Script (ধারণাগত)
# ibmcloud is list --output JSON > /tmp/ibm-instances.json
# তারপর একটি ছোট Script JSON থেকে Ansible Inventory YAML তৈরি করে:
#   all:
#     hosts:
#       ibm-web01:
#         ansible_host: 203.0.113.20
# এই Inventory দিয়ে একই Playbook চালানো যায় — Playbook বদলাতে হয় না।
```

পরিবেশ-ভিত্তিক প্রমাণপত্র পরিচালনার উদাহরণ — `.env` নয়, Shell-এর Environment:

```
# ~/.bashrc বা ল্যাব-সেশনে — প্রমাণপত্র কখনো ফাইলে Commit করবেন না
export AWS_ACCESS_KEY_ID="CHANGE-ME-AWS-ACCESS-KEY-ID"
export AWS_SECRET_ACCESS_KEY="CHANGE-ME-AWS-SECRET-ACCESS-KEY"
export AWS_DEFAULT_REGION="ap-south-1"
# Azure-এর জন্য: AZURE_CLIENT_ID, AZURE_SECRET, AZURE_TENANT, AZURE_SUBSCRIPTION_ID
# GCP-এর জন্য: GOOGLE_APPLICATION_CREDENTIALS="/home/nahid/.gcp/technova-lab.json"
# IBM Cloud-এর জন্য: IBMCLOUD_API_KEY="CHANGE-ME-IBM-API-KEY"
```

```
# যাচাই: env | grep -E 'AWS_|AZURE_|GOOGLE_|IBMCLOUD_' — মান দেখা গেলে ঠিক আছে।
# সতর্কতা: Terminal-এর History-তে Secret লেখা এড়াতে export কমান্ডটি History থেকে মুছুন।
```

২০.১০ ধাপে ধাপে বাস্তবায়ন

1. একটি ক্লাউড বেছে নিন (যেমন AWS) ও তার সরকারি ডকুমেন্টেশন থেকে Free Tier/Trial-এর সীমা ও মূল্য দেখুন।
2. Collection ইনস্টল করুন: `ansible-galaxy collection install amazon.aws` (AWS), `azure.azcollection` (Azure), বা `google.cloud` (GCP)।
3. Least Privilege পরিচয় তৈরি করুন: AWS-এ IAM User/Role (শুধু EC2 Read অধিকার), Azure-এ Service Principal, GCP-তে Service Account — সুনির্দিষ্ট ধাপের জন্য সংশ্লিষ্ট ক্লাউডের সরকারি IAM ডকুমেন্টেশন দেখুন।
4. প্রমাণপত্র Environment Variable-এ দিন (২০.৯ দেখুন) — Playbook বা Inventory ফাইলে নয়।
5. Dynamic Inventory যাচাই করুন: `ansible-inventory -i inventory/aws_ec2.yml --list` — চলমান VM-গুলোর তালিকা দেখা উচিত; খালি তালিকা মানে Filter বা Region ভুল, অথবা কোনো VM চলছে না।
6. সংযোগ যাচাই করুন: `ansible all -i inventory/aws_ec2.yml -m ping` — SSH Key ও Security Group/NSG/Firewall-এ Port 22 খোলা আছে কি না তা এখানে ধরা পড়ে।
7. Baseline চালান: `ansible-playbook -i inventory/aws_ec2.yml playbooks/cloud-baseline.yml --check --diff` দিয়ে প্রথমে দেখুন, তারপর আসল Run করুন।
8. ল্যাব শেষে রিসোর্স মুছুন: Terraform ব্যবহার করলে `terraform destroy`; হাতে তৈরি করলে Cloud Console থেকে VM, Disk ও IP মুছে ফেলুন; তারপর Cloud Console-এর Billing পৃষ্ঠায় দেখুন আর কোনো চলমান রিসোর্স নেই।

২০.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

Dynamic Inventory-এর যাচাই-আউটপুট এরকম দেখাবে (চিত্রণমূলক আউটপুট):

```
$ ansible-inventory -i inventory/aws_ec2.yml --list
{
    "_meta": {
        "hostvars": {
            "technova-lab-web01": {
                "ansible_host": "203.0.113.20",
                "tags": {"Environment": "lab", "Role": "web", "Name": "technova-lab-web01"}
            },
            "technova-lab-api01": {
                "ansible_host": "203.0.113.21",
                "tags": {"Environment": "lab", "Role": "api", "Name": "technova-lab-api01"}
            }
        }
    },
    "roleweb": {
        "hosts": ["technova-lab-web01"]
    },
    "roleapi": {
        "hosts": ["technova-lab-api01"]
    }
}
# ব্যাখ্যা: Plugin AWS API থেকে দুটি চলমান Instance এনেছে; Tag থেকে গ্রুপ
# তৈরি হয়েছে (roleweb, roleapi); ansible_host হলো Public IP।
# এই গ্রুপের নাম আপনার Playbook-এর hosts:-এর সঙ্গে মিলিয়ে নিন।
```

```
$ ansible all -i inventory/aws_ec2.yml -m ping
technova-lab-web01 | SUCCESS => {"ping": "pong"}
technova-lab-api01 | SUCCESS => {"ping": "pong"}
# ব্যাখ্যা: দুটি VM-এই SSH সংযোগ কাজ করছে — Baseline চালানো যাবে।
```

একটি সাধারণ বিভ্রান্তি: `ansible-inventory --list`-এ VM দেখা গেলেও `ping` ব্যর্থ হতে পারে — এর অর্থ Inventory ঠিক আছে, কিন্তু SSH-এর পথ বন্ধ (Security Group-এ Port 22 বন্ধ, Key ভুল, বা VM-এর User ভুল)। আরেকটি বিভ্রান্তি: `keyed_groups`-এর গ্রুপের নাম (যেমন `roleweb`) আপনার Playbook-এর গ্রুপের নামের (`webservers`) সঙ্গে না মিললে Playbook "No hosts matched" বলবে — তখন গ্রুপের নাম মিলিয়ে নিন অথবা `groups` দিয়ে নাম বদলান।

২০.১২ সাধারণ ভুল

- **প্রমাণপত্র Playbook-এ লেখা:** Access Key বা Secret Playbook, Inventory বা Git-এ লিখলে তা ফাঁস হয়। সবসময় Environment Variable বা Credentials ফাইল ব্যবহার করুন।
- **Ansible দিয়ে Provisioning করা:** `ec2_instance` দিয়ে VM তৈরি করা যায়, তবে Infrastructure-এর State Terraform-এ রাখাই সঠিক — Ansible দিয়ে Provisioning করলে State-এর হিসাব থাকে না।
- **Filter না দিয়ে Dynamic Inventory চালানো:** সব Region-এর সব Instance এনে Inventory বিশাল হয়ে যায়, আর ভুল VM-এ Playbook চলে যেতে পারে। সবসময় Tag ও State দিয়ে ছাঁকুন।
- **Database VM-কে Public IP দেওয়া:** db01-এর Public IP থাকলে তা ইন্টারনেট থেকে পৌঁছানো যায়। ডেটাবেস VM শুধু Private IP-তে রাখুন, আর Ansible-কে Private IP দিয়ে সংযোগ করতে দিন (Bastion Host বা VPN দিয়ে)।
- **ল্যাবের VM মুছতে ভুলে যাওয়া:** VM বন্ধ করলেই বিল থামে না — Disk ও IP-র চার্জ চলতে পারে। ল্যাব শেষে VM মুছে Billing পৃষ্ঠায় যাচাই করুন।
- **একটি পরিচয়ে সব অধিকার দেওয়া:** Ansible-এর পরিচয়ে VM মোছার অধিকার থাকলে একটি ভুল Task পুরো পরিবেশ মুছে ফেলতে পারে। Least Privilege মেনে চলুন।
- **Region ভুলে যাওয়া:** Inventory-তে Region `ap-south-1` লেখা, কিন্তু VM তৈরি হয়েছে অন্য Region-এ — তখন Inventory খালি দেখাবে। Region সবসময় মিলিয়ে নিন।
- **IBM Cloud-এ আদর্শ Plugin ধরে নেওয়া:** IBM Cloud-এর জন্য AWS-এর মতো Plugin নেই — Constructed Inventory বা Collection-এর ডকুমেন্টেশন যাচাই না করে এগোবেন না।

২০.১৩ সমস্যা সমাধান (Troubleshooting)

1. **লক্ষণ:** `ansible-inventory --list` খালি তালিকা দেয় — **পরীক্ষা:** Filter (Tag, State, Region) ও প্রমাণপত্র। **সমাধান:** Cloud Console-এ দেখুন VM সত্যিই `running` কি না, Tag ঠিক আছে কি না; `ANSIBLE_DEBUG=1` দিয়ে Plugin-এর অনুরোধ দেখুন।
2. **লক্ষণ:** `Failed to authenticate / InvalidClientTokenId` — **পরীক্ষা:** Environment Variable-এর মান ও Region। **সমাধান:** Key-টি সক্রিয় কি না Cloud Console-এ দেখুন; Secret-এ অতিরিক্ত Space বা Newline আছে কি না দেখুন; Key কখনোই Playbook-এ লিখে ঠিক করবেন না — Environment-এ ঠিক করুন।
3. **লক্ষণ:** Inventory-তে VM আছে, কিন্তু `ping` ব্যর্থ — `UNREACHABLE` — **পরীক্ষা:** Security Group/NSG/Firewall-এ Port 22, SSH Key, ও `ansible_user`। **সমাধান:** Cloud Console থেকে Security Group-এ নিজের IP থেকে Port 22 খুলুন; VM-এর User (AWS-এ সাধারণত `ubuntu`, Azure-এ আপনার তৈরি User) ঠিক আছে কি না দেখুন।
4. **লক্ষণ:** `No hosts matched` — **পরীক্ষা:** `ansible-inventory --list`-এর গ্রুপের নাম ও Playbook-এর `hosts:`। **সমাধান:** `keyed_groups`-এর `prefix` ও `separator` দেখে গ্রুপের নাম মিলিয়ে নিন।

5. **লক্ষণ:** Collection বা Plugin পাওয়া যায়নি — `Failed to parse inventory` — **পরীক্ষা:** `ansible-galaxy collection list`। **সমাধান:** Collection ইনস্টল করুন ও সংস্করণটি Ansible Core 2.17-এর সঙ্গে সামঞ্জস্যপূর্ণ কি না দেখুন।
6. **লক্ষণ:** ল্যাব শেষে বিল আসছে — **পরীক্ষা:** Cloud Console-এর Billing ও Resource তালিকা। **সমাধান:** VM ছাড়াও Disk, Snapshot, Elastic IP ও Load Balancer-ও মুছে ফেলুন — এগুলো আলাদা Resource, VM মুছলেই সব মুছে যায় না।

২০.১৪ নিরাপত্তা বিবেচনা

> **WARNING:** ক্লাউড প্রমাণপত্র (Access Key, Service Principal Secret, Service Account JSON) হলো আপনার ক্লাউড-অ্যাকাউন্টের চাবি — এটি ফাঁস হলে আক্রমণকারী আপনার নামে VM তৈরি করে বিল বাড়াতে পারে বা ডেটা চুরি করতে পারে। প্রমাণপত্র কখনো Git, Chat, স্ক্রিনশট বা Playbook-এ দেবেন না; Key ফাইলের Permission 600 রাখুন; আর প্রমাণপত্র ফাঁস হয়েছে সন্দেহ হলে সঙ্গে সঙ্গে তা বাতিল (Revoke) করে নতুনটি তৈরি করুন — ফাঁস হওয়া Key "পরে বদলাব" বলে রেখে দেওয়া যাবে না।

ক্লাউড-নিরাপত্তার আরও দিক: প্রথমত, Ansible-এর পরিচয়ে Least Privilege — শুধু Read অধিকার, VM মোছা বা IAM বদলানোর অধিকার নয়। দ্বিতীয়ত, Database VM শুধু Private IP-তে রাখুন — Public IP দিলে তা ইন্টারনেটে খোলা থাকে। তৃতীয়ত, Security Group/NSG-তে SSH শুধু নিজের IP থেকে খুলুন — সবার জন্য (0.0.0.0/0) Port 22 খোলা রাখা প্রোডাকশনে নিষিদ্ধ। চতুর্থত, প্রমাণপত্র নিয়মিত ঘোরান (Rotate) — দীর্ঘদিন একই Key ব্যবহার করবেন না। পঞ্চমত, Cloud-এর Audit Log (AWS CloudTrail, Azure Activity Log, GCP Cloud Audit Logs) চালু রাখুন — কে, কখন, কী করেছে, তার হিসাব ছাড়া নিরাপত্তা-ঘটনার কারণ খুঁজে পাওয়া যায় না। সবশেষে, Terraform State ফাইলে Secret থাকতে পারে — State ফাইলটি কখনো Git-এ Commit করবেন না, Remote Backend-এ Encrypted অবস্থায় রাখুন।

২০.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

প্রোডাকশনে ক্লাউড-অটোমেশনের নিয়ম: প্রথমত, Terraform ও Ansible-এর দায়িত্ব আলাদা রাখুন — Terraform Infrastructure, Ansible Configuration; দুটির কাজ একটি টুলে মেশাবেন না। দ্বিতীয়ত, Dynamic Inventory-র Filter সবসময় Environment Tag দিয়ে ছাঁকুন — `lab` ও `production` VM একই Inventory-তে এলে ভুল Playbook ভুল পরিবেশে চলতে পারে; Production Inventory আলাদা রাখুন। তৃতীয়ত, Production-এ Playbook চালানোর আগে `--check --diff` দিয়ে পরিবর্তন দেখুন, আর পরিবর্তনটি Review করুন — ক্লাউডে একটি ভুল Task শত শত VM-এ একসঙ্গে চলতে পারে। চতুর্থত, খরচের সীমা (Budget Alert) চালু করুন — ক্লাউডের Billing Alert দিয়ে নির্দিষ্ট সীমা ছাড়ালে বিজ্ঞপ্তি পান; ল্যাবের জন্য সবচেয়ে ছোট VM ব্যবহার করুন। পঞ্চমত, Tag-এর নীতি (Tagging Policy) ঠিক করুন — প্রতিটি VM-এ Environment, Role, Owner ও Cost-Center Tag থাকা বাধ্যতামূলক করুন; Tag ছাড়া VM খুঁজে পাওয়া যায় না, আর বিলের হিসাবও করা যায় না। সবশেষে, ল্যাব-পরিবেশের জন্য একটি ধ্বংস-Checklist রাখুন — VM, Disk, IP, Snapshot, Security Group — প্রতিটি Resource মুছে ফেলার পর Billing পৃষ্ঠায় যাচাই করুন।

২০.১৬ অধ্যায়ের সারাংশ

ক্লাউডে সার্ভার স্থির নয় — তাই Inventory-ও স্থির হতে পারে না। Dynamic Inventory (AWS-এ `aws_ec2`, Azure-এ `azure_rm`, GCP-তে `gcp_compute`) ক্লাউড API থেকে চলমান VM-এর তালিকা এনে Tag-ভিত্তিক গ্রুপ তৈরি করে, আর সেই গ্রুপের ওপরে আগের অধ্যায়ের Baseline, Deploy ও Hardening Playbook কোনো পরিবর্তন ছাড়াই চলে। প্রমাণপত্র সবসময় Environment Variable বা Credentials ফাইলে থাকে — কখনো Playbook-এ নয়। Provisioning Terraform-

এর কাজ, Configuration Ansible-এর কাজ — `amazon.aws.ec2_instance` একটি Provisioning Module, এটি দিয়ে VM তৈরি করা যায়, তবে এই বইয়ের Workflow-তে Terraform VM তৈরি করে ও Ansible তা কনফিগার করে। IBM Cloud-এর জন্য আদর্শ Dynamic Inventory Plugin নেই — সেখানে Constructed Inventory বা Collection-এর ডকুমেন্টেশন যাচাই করে এগোতে হয়। আর সবচেয়ে গুরুত্বপূর্ণ: ক্লাউড রিসোর্সে চার্জ হয় — ল্যাব শেষে সবকিছু মুছে ফেলুন, আর সুনির্দিষ্ট মূল্য ও সক্ষমতার জন্য সরকারি ডকুমেন্টেশন দেখুন।

২০.১৭ জ্ঞান যাচাই

1. Static Inventory ক্লাউডে কেন অচল, আর Dynamic Inventory কীভাবে এই সমস্যা সমাধান করে?
2. AWS, Azure ও GCP-র Dynamic Inventory Plugin-এর নাম ও Collection লিখুন।
3. `amazon.aws.ec2_instance` কী ধরনের Module, এবং এই বইয়ে Provisioning-এর জন্য Terraform কেন সুপারিশ করা হয়েছে?
4. প্রমাণপত্র Playbook-এ লিখলে কী বিপদ হয় — দুটি পরিণতি লিখুন।
5. IBM Cloud-এর ক্ষেত্রে Dynamic Inventory-এর সীমা কী, এবং Constructed Inventory কীভাবে কাজ করে?

২০.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. **Dynamic Inventory:** যেকোনো একটি ক্লাউডের Free Tier/Trial-এ একটি VM তৈরি করুন (Terraform বা Console দিয়ে), তারপর Dynamic Inventory Config লিখে `ansible-inventory --list` দিয়ে VM-টি দেখান; তারপর VM-টি মুছে Billing পৃষ্ঠায় যাচাই করুন।
2. **ক্লাউড Baseline:** Dynamic Inventory দিয়ে VM-এ অধ্যায় ১৭-এর Baseline Playbook চালান; Static Inventory দিয়ে চালানোর সঙ্গে পার্থক্য কী তা লিখুন।
3. **খরচ-Checklist:** আপনার বেছে নেওয়া ক্লাউডের সরকারি ডকুমেন্টেশন থেকে ল্যাবের জন্য একটি ধ্বংস-Checklist বানান — VM, Disk, IP, Snapshot — প্রতিটি Resource মুছে ফেলার ধাপসহ।

অধ্যায় 21 • PART IV — Container Platform Configuration

Chapter 21 — Docker and Kubernetes Environment Configuration

২১.১ ভূমিকা

আগের অধ্যায়গুলোতে আমরা সার্ভারে সরাসরি অ্যাপ চালিয়েছি — systemd দিয়ে Node.js, Nginx দিয়ে Proxy, MongoDB দিয়ে ডেটা। এবার আমরা সেই অ্যাপকে Container-এর ভেতরে চালানোর পরিবেশ তৈরি করব — আর Kubernetes-এর Node হিসেবে সার্ভারকে প্রস্তুত করব। একটি গুরুত্বপূর্ণ সীমানা প্রথমেই পরিষ্কার করে নিই: এই অধ্যায়টি **Configuration**-এর অধ্যায় — Docker Host তৈরি করা, Container Deploy করা, আর Kubernetes Node-এর পূর্বশর্ত (Prerequisite) বসানো। এটি Cluster পরিচালনা (Cluster Administration) বা Kubernetes-এর ভেতরের কাজ (Pod, Deployment, Service পরিচালনা) শেখায় না — সেগুলো Kubernetes-এর নিজস্ব অধ্যায়ের বিষয়। কেন এই সীমানা? কারণ Docker Host-এর Config ভুল হলে Container চলবে না, আর Kubernetes Node-এর পূর্বশর্ত (Swap বন্ধ, Kernel Parameter, containerd Config) ভুল হলে kubelet চালুই হবে না — এই ভিত্তিটি Ansible দিয়ে সঠিকভাবে বসানোই এই অধ্যায়ের কাজ। TechNova-র ক্ষেত্রে লক্ষ্যটি স্পষ্ট: একটি Ubuntu 24.04 সার্ভারে Docker Engine ইনস্টল করে `technova-api` Container চালানো, আর আরেকটি সার্ভারকে Kubernetes Node হিসেবে প্রস্তুত করা — যাতে পরের Volume-এ Cluster তৈরি করার সময় শুধু `kubeadm init` ও `kubeadm join` বাকি থাকে।

> **CONCEPT:** Docker Host মানে একটি সার্ভার যেখানে Docker Engine চলে — Container চালানোর জন্য প্রস্তুত মেশিন। Kubernetes Node মানে একটি সার্ভার যেখানে kubelet, kubeadm ও containerd চলে — Cluster-এর সদস্য হওয়ার জন্য প্রস্তুত মেশিন। দুটি ভিন্ন ভূমিকা, তবে ভিত্তি এক: সঠিক Kernel Parameter, সঠিক Container Runtime, আর সঠিক ব্যবহারকারী-অধিকার। Ansible দুটি ভূমিকাই একই Playbook-কাঠামো দিয়ে তৈরি করে — শুধু Role আলাদা।

২১.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Docker Host-এর উপাদান (Docker Engine, containerd, Docker CLI) ব্যাখ্যা করতে; (২) Ansible দিয়ে Docker-এর Repository যোগ করে Docker CE ইনস্টল করতে ও `docker` গ্রুপে ব্যবহারকারী যোগ করতে; (৩) `daemon.json` দিয়ে Docker Daemon কনফিগার করতে (Log Driver, Storage Driver, Registry Mirror-এর ধারণাসহ); (৪) `community.docker` Collection-এর `docker_image`, `docker_container` ও `docker_compose_v2` মডিউল দিয়ে Container Deploy করতে; (৫) Kubernetes Node-এর পূর্বশর্ত ব্যাখ্যা করতে — Swap বন্ধ, Kernel Module (`overlay`, `br_netfilter`), sysctl Parameter, containerd Config; (৬) kubelet, kubeadm ও kubectl ইনস্টল করতে (Repository দিয়ে); (৭) Docker Host ও Kubernetes Node-এর Config-এর পার্থক্য ব্যাখ্যা করতে; (৮) এই অধ্যায়ের সীমানা (Configuration, Cluster Administration নয়) ব্যাখ্যা করতে ও পরের ধাপের নাম বলতে।

২১.৩ পূর্বশর্ত ধারণা

এই অধ্যায়ের জন্য আপনার প্রয়োজন: অধ্যায় ১৭-এর Baseline ও অধ্যায় ১৯-এর Hardening আপনার ল্যাব সার্ভারে চালানো থাকতে হবে; Container-এর প্রাথমিক ধারণা (Image, Container, Port, Volume) — যা Volume 06-এ এসেছে;

systemd Service-এর ধারণা; আর `docker` কমান্ডের প্রাথমিক অভিজ্ঞতা (থাকলে ভালো, না থাকলেও এই অধ্যায়ের উদাহরণ দেখে বোঝা যাবে)। Kubernetes-এর পূর্ব অভিজ্ঞতা প্রয়োজন নেই — এই অধ্যায়ে শুধু Node প্রস্তুত করা শেখানো হবে, Cluster তৈরি করা নয়। ল্যাবের জন্য দুটি সার্ভার ব্যবহার করব: `docker01 (203.0.113.30)` — Docker Host, আর `k8s-node01 (203.0.113.31)` — Kubernetes Node-এর প্রার্থী। দুটি সার্ভারেই Ubuntu 24.04, Ansible Core 2.17+, Python 3.12 — একই পরিবেশ, যা এই বইয়ের শুরু থেকে চলে আসছে।

২১.৪ প্রথম নীতি থেকে ব্যাখ্যা

Container-কে ভাবুন একটি বাক্স হিসেবে যাতে অ্যাপ ও তার সব নির্ভরতা ভরা আছে — বাক্সটি যেকোনো টেবিলে (Host) রাখলেই চলে, তবে টেবিলটিতে বাক্স রাখার ব্যবস্থা থাকতে হয়। Docker Engine হলো সেই ব্যবস্থা: একটি Daemon (পেছনে চলা সার্ভিস) যা বাক্সটি খোলে, চালায়, থামায়, আর বাক্সের ভেতরে কী আছে তার হিসাব রাখে। Daemon-এর নিচে থাকে containerd — আসল Runtime, যে বাক্সটি চালায়; Docker Engine হলো তার ওপরে একটি সহজ আবরণ (API ও CLI)। Kubernetes-এর জগতে একই ভূমিকা পালন করে kubelet — প্রতিটি Node-এ চলা একটি Agent, যে Cluster-এর Control Plane-এর নির্দেশ অনুযায়ী Container চালায়। তবে kubelet নিজে Container চালায় না — সে containerd-কে বলে Container চালাতে। তাই Kubernetes Node প্রস্তুত করার মানে আসলে containerd-কে সঠিকভাবে বসানো, Kernel-কে Container-এর জন্য প্রস্তুত করা (Swap বন্ধ, নির্দিষ্ট Kernel Module ও sysctl), আর kubelet ও kubeadm ইনস্টল করা। একটি সাধারণ ভুল ধারণা: "Docker ইনস্টল করলেই Kubernetes চলবে" — না; Kubernetes-এর containerd-এর Config Docker-এর Config থেকে আলাদা (বিশেষ করে `SystemdCgroup` সেটিং), আর Docker-এর সঙ্গে আসা containerd-কে Kubernetes-এর জন্য আলাদাভাবে কনফিগার করতে হয়। এই অধ্যায়ে আমরা দুটি ভূমিকার Config আলাদাভাবে শিখব, যাতে একটির Config অন্যটির সঙ্গে গুলিয়ে না যায়।

২১.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Docker Host — Repository, Package, Group

Ubuntu 24.04-এ Docker ইনস্টল করার সঠিক পথ হলো Docker-এর নিজস্ব Repository (`download.docker.com`) যোগ করে `docker-ce`, `docker-ce-cli`, `containerd.io` ও `docker-compose-plugin` ইনস্টল করা — Ubuntu-র ডিফল্ট Repository-তে Docker-এর পুরনো সংস্করণ থাকতে পারে। Ansible-এ `apt_key` (বা আধুনিক পদ্ধতিতে `ansible.builtin.get_url` দিয়ে Keyring ফাইলে Key রাখা) ও `apt_repository` দিয়ে Repository যোগ করা হয়, তারপর `apt` দিয়ে প্যাকেজ ইনস্টল। ইনস্টলের পর `nahid` ব্যবহারকারীকে `docker` গ্রুপে যোগ করা হয় (`user` মডিউলে `groups: docker, append: true`) — যাতে `sudo` ছাড়া `docker` কমান্ড চালানো যায়। সতর্কতা: `docker` গ্রুপে থাকা মানে কার্যত Root-এর সমান অধিকার — কারণ Docker Daemon Root দিয়ে চলে, আর `docker run -v /:/host` দিয়ে Host-এর পুরো ফাইলসিস্টেম দেখা যায়। তাই শুধু বিশ্বস্ত ব্যবহারকারীকেই `docker` গ্রুপে রাখুন — এটি অধ্যায় ১৯-এর Least Privilege নীতিরই অংশ।

daemon.json — Docker Daemon-এর Config

Docker Daemon-এর Config থাকে `/etc/docker/daemon.json`-এ — একটি JSON ফাইল, যা Ansible-এ Template বা `copy` দিয়ে লেখা হয়। মূল সেটিংগুলো: `log-driver` (যেমন `json-file`) ও `log-opts` (যেমন `max-size: 10m`, `max-file: "3"`) — Log ফাইল যাতে Disk ভরে না ফেলে; `storage-driver` (Ubuntu-তে সাধারণত `overlay2`); `exec-opts`-এ `native.cgroupdriver=systemd` — systemd-এর সঙ্গে Cgroup Driver মেলানো, যা Kubernetes-এর জন্যও গুরুত্বপূর্ণ। `daemon.json` বদলালে Docker Daemon Restart করতে হয় — Handler দিয়ে, শুধু তখনই যখন ফাইলটি সত্যিই বদলেছে। একটি ভুল JSON লিখলে Docker Daemon চালুই হবে না — তাই Template-এ JSON-এর

Syntax যাচাই করুন (`python3 -m json.tool` দিয়ে হাতে পরীক্ষা করা যায়), আর Playbook-এ `validate` ব্যবহার করা যায়।

community.docker মডিউল — Container Deploy

`community.docker` Collection-এর তিনটি মূল মডিউল: `docker_image` — Image টানা (`name: technova-api: 1.0.0`, `source: pull`) বা Build করা (`source: build`, `build.path`); `docker_container` — Container চালানো (`name`, `image`, `ports`, `env`, `volumes`, `restart_policy: always`, `state: started`); আর `docker_compose_v2` — একটি Compose ফাইল (`compose.yml`) দিয়ে একাধিক Container একসঙ্গে চালানো (`project_src`, `state: present`)। `docker_container` মডিউল Idempotent: Container ইতিমধ্যে সঠিক Config-এ চললে কিছু করে না; Config বদলালে Container নতুন করে তৈরি করে (Recreate) — এই আচরণটি জানা জরুরি, কারণ Config-এর ছোট পরিবর্তনেও Container Restart হয়। `docker_container`-এ `env` দিয়ে Environment Variable দেওয়া যায়, তবে Secret-এর মান Vault থেকে আসতে হবে — Compose ফাইল বা Playbook-এ Plain-Text Secret লেখা যাবে না।

Kubernetes Node প্রস্তুতি — Swap, Kernel Module, sysctl, containerd

Kubernetes-এর kubelet Swap চালু থাকলে কাজ করে না — তাই Swap বন্ধ করতে হয়: `swapoff -a` (চলমান Swap বন্ধ) ও `/etc/fstab`-এর Swap লাইন মন্তব্য করে দেওয়া (Reboot-এর পরেও বন্ধ থাকার জন্য) — Ansible-এ `lineinfile` বা `replace` দিয়ে করা হয়। Kernel Module `overlay` ও `br_netfilter` লোড করতে হয় — `community.general.modprobe` মডিউল দিয়ে, আর `/etc/modules-load.d/k8s.conf`-এ লিখে Reboot-এর পরেও লোড নিশ্চিত করা হয়। sysctl Parameter তিনটি বাধ্যতামূলক: `net.bridge.bridge-nf-call-iptables=1`, `net.bridge.bridge-nf-call-ip6tables=1`, ও `net.ipv4.ip_forward=1` — এগুলো `ansible.builtin.sysctl` মডিউল দিয়ে `/etc/sysctl.d/k8s.conf`-এ লেখা হয়। containerd-এর Config (`/etc/containerd/config.toml`) তৈরি করতে হয় `containerd config default` দিয়ে, তারপর `SystemdCgroup = true` সেট করতে হয় — এটি `lineinfile` বা Template দিয়ে করা হয়; ভুল Cgroup Driver-এ kubelet চালু হবে না, এটি সবচেয়ে সাধারণ Kubernetes Node-ভুল। তারপর kubelet, kubeadm ও kubectl ইনস্টল করা হয় Kubernetes-এর Repository (`pkgs.k8s.io`) থেকে — সংস্করণটি Variable-এ রাখুন (যেমন `kubernetes_version: "1.31"`), যাতে সব Node-এ একই সংস্করণ থাকে।

সীমানা — Configuration, Cluster Administration নয়

এই অধ্যায় শেষে আপনার Node প্রস্তুত থাকবে — `kubeadm init` বা `kubeadm join` চালানোর জন্য। কিন্তু Cluster তৈরি করা, Pod/Deployment/Service পরিচালনা করা, Network Plugin (CNI) বসানো, বা Cluster Upgrade করা — এগুলো এই অধ্যায়ের বাইরে। কেন? কারণ Cluster Administration-এর নিজস্ব ধারণা (Control Plane, etcd, CNI, RBAC) আছে, যা একটি পূর্ণ অধ্যায় দাবি করে। এই সীমানা মেনে চললে এই অধ্যায়ের Playbook-গুলো ছোট ও যাচাইযোগ্য থাকে, আর পরের Volume-এ Cluster-অধ্যায়ে শুধু Cluster-এর কাজটুকুই বাকি থাকে।

কাজ	Ansible মডিউল / ফাইল	লক্ষ্য অবস্থা
Docker Repository	`apt_repository` + Keyring	`download.docker.com` যোগ করা
Docker ইনস্টল	`apt`: `docker-ce`, `containerd.io`	Service চালু ও Enabled
Docker গ্রুপ	`user` (`groups: docker`)	

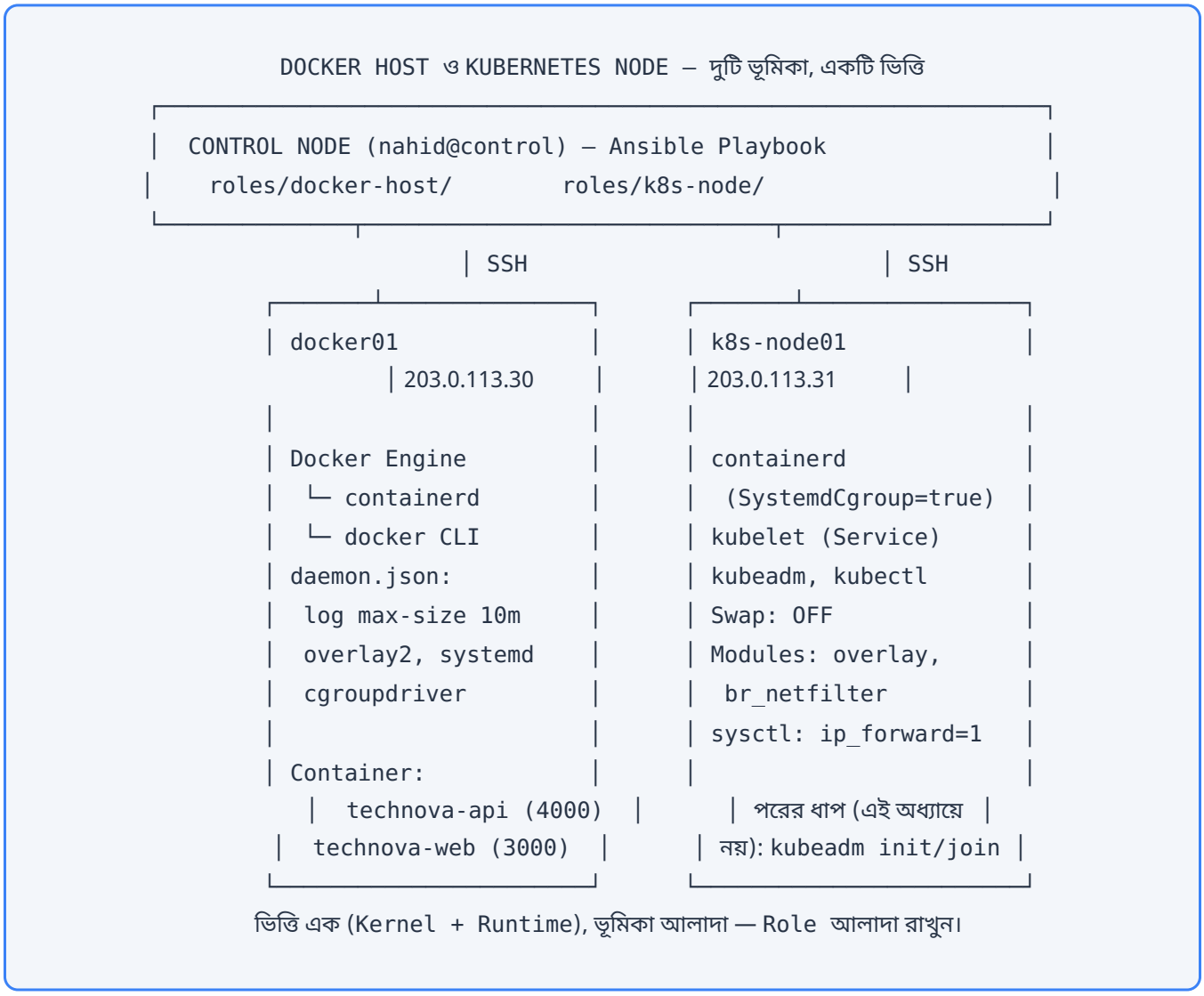
		`nahid` sudo ছাড়া docker চালাতে পারে
Daemon Config	`/etc/docker/daemon.json`	Log সীমা, `overlay2`, systemd Cgroup
Container Deploy	`docker_image`, `docker_container`, `docker_compose_v2`	Container চলছে, Restart Policy Always
K8s পূর্বশর্ত	`swapoff`, `modprobe`, `sysctl`	Swap বন্ধ, Module লোড, IP Forward চালু
containerd (K8s)	`/etc/containerd/config.toml`	`SystemdCgroup = true`
K8s টুল	`apt` : kubelet, kubeadm, kubectl	একই সংস্করণ, Service Enabled (kubelet)

২১.৬ অভ্যন্তরীণ কার্যপ্রণালী

Docker Host Playbook-এর ভেতরের ক্রম: প্রথমে Repository ও Key, তারপর প্যাকেজ, তারপর `daemon.json`, তারপর Docker Service চালু, তারপর ব্যবহারকারীকে গ্রুপে যোগ করা — গ্রুপে যোগ করার পর ব্যবহারকারীকে আবার লগইন করতে হয়, নইলে নতুন গ্রুপ কার্যকর হয় না; Ansible-এর SSH সংযোগে এটি একটি পরিচিত বিষয় — Playbook-এর পরের Task-গুলো `become: true` দিয়ে চলে বলে তাতে সমস্যা হয় না, তবে হাতে `docker ps` চালাতে গেলে নতুন করে লগইন করতে হয়। `docker_container` মডিউল কীভাবে কাজ করে: এটি Docker Daemon-এর API-কে জিজ্ঞেস করে নামের Container আছে কি না, থাকলে তার Config-এর সঙ্গে চাওয়া Config মেলায় — মিললে কিছু করে না, না মিললে Container থামিয়ে মুছে নতুন করে তৈরি করে। এই Recreate আচরণের কারণে `docker_container`-এ `restart: true` না দিলেও Config বদলে Container Restart হয় — এটি জানা থাকলে Deploy-এর সময় অবাক হতে হয় না। `docker_compose_v2` মডিউল ভেতরে `docker compose` কমান্ড চালায় — Compose ফাইলে বদল থাকলে শুধু বদলানো Service-গুলো নতুন করে তৈরি হয়। Kubernetes Node প্রস্তুতিতে `modprobe` মডিউল Kernel Module লোড করে, আর `sysctl` মডিউল মানটি ফাইলে লিখে সঙ্গে সঙ্গে প্রয়োগ করে — দুটি কাজই Reboot-এর পরেও টিকে থাকে, কারণ মানগুলো ফাইলে লেখা থাকে। containerd-এর `SystemdCgroup = true` সেটিংটির ভেতরের অর্থ: containerd ও kubelet — দুজনেই Cgroup পরিচালনার জন্য systemd-কে ব্যবহার করবে — একজন Cgroupfs ও অন্যজন systemd ব্যবহার করলে kubelet চালু হবে না, এটি একটি ক্লাসিক ভুল।

> **TIP:** `docker_container` মডিউল ব্যবহারের আগে Control Node-এ Docker SDK for Python ইনস্টল থাকা দরকার (`pip install docker`) — কারণ `community.docker` Collection Daemon-এর সঙ্গে কথা বলতে এই লাইব্রেরি ব্যবহার করে। Managed Node-এ নয়, Control Node-এ — এটি ভুলে গেলে মডিউল "Docker SDK not found" বলে ব্যর্থ হবে। একইভাবে `docker_compose_v2`-এর জন্য Managed Node-এ `docker-compose-plugin` থাকা দরকার।

২১.৭ আর্কিটেকচার ও চিত্র



চিত্রটির মূল কথা: Docker Host ও Kubernetes Node — দুটি আলাদা Role, দুটি আলাদা Playbook — কিন্তু দুটির ভিত্তি এক: সঠিক Runtime ও Kernel সেটিং। একটি Role-এর Config অন্যটিতে কপি করবেন না — বিশেষ করে containerd-এর Config দুটি ভূমিকায় আলাদা।

২১.৮ ব্যবহারিক উদাহরণ

Docker Host-এ Container Deploy — `docker_container` দিয়ে API চালানো:

```
# Docker Host-এ technova-api Container চালানো (community.docker)
- name: Pull TechNova API image
  community.docker.docker_image:
    name: technova-api
    tag: "1.0.0"
    source: pull
    # বাস্তবে Image-টি Registry থেকে আসবে (যেমন technova.example-এর Registry)।

- name: Run TechNova API container
  community.docker.docker_container:
    name: technova-api
```

```
    image: technova-api:1.0.0
    state: started
    restart_policy: always
    ports:
      - "4000:4000"
    env:
      NODE_ENV: production
      PORT: "4000"
      DATABASE_URL: "mongodb://203.0.113.12:27017/technova"
    volumes:
      - /opt/technova/api/logs:/app/logs
    healthcheck:
      test: ["CMD", "curl", "-f", "http://127.0.0.1:4000/api/health"]
      interval: 30s
      timeout: 5s
      retries: 3
  # ব্যাখ্যা: restart_policy always মানে Host Reboot-এ Container আবার চালু হবে;
  # healthcheck মানে Docker নিজেই API-র স্বাস্থ্য পরীক্ষা করবে।
```

Compose দিয়ে একাধিক Container — `docker_compose_v2` :

```
# Compose ফাইল Deploy করে একসঙ্গে Web + API চালানো
- name: Deploy TechNova stack with Compose v2
  community.docker.docker_compose_v2:
    project_src: /opt/technova/compose
    state: present
    # /opt/technova/compose/compose.yml ফাইলটি Template দিয়ে আগে বসানো হয়।
    # Compose ফাইলে Secret-এর মান Vault থেকে আসবে — Plain-Text নয়।

# compose.yml-এর ধারণাগত আকৃতি (Template থেকে তৈরি):
# services:
#   api:
#     image: technova-api:1.0.0
#     ports: ["4000:4000"]
#     restart: always
#   web:
#     image: technova-web:1.0.0
#     ports: ["3000:3000"]
#     restart: always
```

Kubernetes Node প্রস্তুতি — Swap বন্ধ ও Kernel Module:

```
# Swap বন্ধ করা — চলমান Swap ও fstab — দুটোই
- name: Disable swap immediately
  ansible.builtin.command: swapoff -a
  when: ansible_swaptotal_mb > 0
  changed_when: true

- name: Remove swap from /etc/fstab (so it stays off after reboot)
  ansible.builtin.lineinfile:
    path: /etc/fstab
    regexp: '^([^#].*\s+swap\s+.*)$'
    line: '# \1 (disabled for Kubernetes)'
    backrefs: true

# Kernel Module লোড করা ও Reboot-এর পরেও লোড নিশ্চিত করা
- name: Load overlay and br_netfilter kernel modules
```

```
    community.general.modprobe:
      name: "{{ item }}"
      state: present
    loop: [overlay, br_netfilter]

  - name: Ensure kernel modules load on boot
    ansible.builtin.copy:
      content: "overlay\nbr_netfilter\n"
      dest: /etc/modules-load.d/k8s.conf
      mode: "0644"
```

২১.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচে দুটি সম্পূর্ণ Playbook — প্রথমটি Docker Host, দ্বিতীয়টি Kubernetes Node প্রস্তুতি। দুটি আলাদা ফাইল, যাতে ভূমিকা গুলিয়ে না যায়।

```
---
# playbooks/docker-host.yml — TechNova Docker Host Setup
# চালানোর কমান্ড: ansible-playbook -i inventory/hosts.yml playbooks/docker-host.yml
# আগে: Control Node-এ pip install docker (২১.৬ দেখুন)।
- name: Set up Docker host
  hosts: dockerhosts
  become: true
  tasks:
    - name: Install prerequisites for Docker repository
      ansible.builtin.apt:
        name:
          - ca-certificates
          - curl
          - gnupg
        state: present
        update_cache: true

    - name: Add Docker GPG key to keyring
      ansible.builtin.get_url:
        url: https://download.docker.com/linux/ubuntu/gpg
        dest: /etc/apt/keyrings/docker.asc
        mode: "0644"

    - name: Add Docker repository
      ansible.builtin.apt_repository:
        repo: "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.asc] https://
download.docker.com/linux/ubuntu noble stable"
        state: present

    - name: Install Docker CE and Compose plugin
      ansible.builtin.apt:
        name:
          - docker-ce
          - docker-ce-cli
          - containerd.io
          - docker-compose-plugin
        state: present
        update_cache: true

    - name: Deploy Docker daemon config
      ansible.builtin.template:
```

```
        src: daemon.json.j2
        dest: /etc/docker/daemon.json
        mode: "0644"
      notify: Restart Docker

    - name: Add nahid to docker group
      ansible.builtin.user:
        name: nahid
        groups: docker
        append: true

    - name: Start and enable Docker
      ansible.builtin.service:
        name: docker
        state: started
        enabled: true

    - name: Pull TechNova API image
      community.docker.docker_image:
        name: technova-api
        tag: "1.0.0"
        source: pull

    - name: Run TechNova API container
      community.docker.docker_container:
        name: technova-api
        image: technova-api:1.0.0
        state: started
        restart_policy: always
        ports:
          - "4000:4000"

  handlers:
    - name: Restart Docker
      ansible.builtin.service:
        name: docker
        state: restarted
```

```
---
# playbooks/k8s-node.yml — Kubernetes Node Preparation (Configuration only)
# চালানোর কমান্ড: ansible-playbook -i inventory/hosts.yml playbooks/k8s-node.yml
# এটি Cluster তৈরি করে না — শুধু Node প্রস্তুত করে (২১.৫ দেখুন)।
- name: Prepare Kubernetes node
  hosts: k8snodes
  become: true
  vars:
    kubernetes_version: "1.31"
  tasks:
    - name: Disable swap immediately
      ansible.builtin.command: swapoff -a
      when: ansible_swaptotal_mb > 0
      changed_when: true

    - name: Remove swap from /etc/fstab
      ansible.builtin.lineinfile:
        path: /etc/fstab
        regexp: '^([^#].*\s+swap\s+.*)$'
        line: '# \1 (disabled for Kubernetes)'
```

```
        backrefs: true

    - name: Load overlay and br_netfilter kernel modules
      community.general.modprobe:
        name: "{{ item }}"
        state: present
      loop: [overlay, br_netfilter]

    - name: Ensure kernel modules load on boot
      ansible.builtin.copy:
        content: "overlay\nbr_netfilter\n"
        dest: /etc/modules-load.d/k8s.conf
        mode: "0644"

    - name: Apply Kubernetes sysctl parameters
      ansible.builtin.sysctl:
        name: "{{ item.key }}"
        value: "{{ item.value }}"
        sysctl_file: /etc/sysctl.d/k8s.conf
        state: present
        reload: true
      loop:
        - { key: net.bridge.bridge-nf-call-iptables, value: 1 }
        - { key: net.bridge.bridge-nf-call-ip6tables, value: 1 }
        - { key: net.ipv4.ip_forward, value: 1 }

    - name: Install containerd
      ansible.builtin.apt:
        name: containerd.io
        state: present
        update_cache: true

    - name: Generate default containerd config
      ansible.builtin.command: containerd config default
      register: containerd_default
      changed_when: false

    - name: Deploy containerd config
      ansible.builtin.template:
        src: containerd-config.toml.j2
        dest: /etc/containerd/config.toml
        mode: "0644"
      notify: Restart containerd

    - name: Add Kubernetes repository key
      ansible.builtin.get_url:
        url: https://pkgs.k8s.io/core:/stable:/v1.31/deb/Release.key
        dest: /etc/apt/keyrings/kubernetes-apt-keyring.gpg
        mode: "0644"

    - name: Add Kubernetes repository
      ansible.builtin.apt_repository:
        repo: "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://
pkgs.k8s.io/core:/stable:/v1.31/deb/ /"
        state: present

    - name: Install kubelet, kubeadm and kubectl (pinned version)
      ansible.builtin.apt:
        name:
```

```
          - "kubelet={{ kubernetes_version }}.0-1.1"
          - "kubeadm={{ kubernetes_version }}.0-1.1"
          - "kubectl={{ kubernetes_version }}.0-1.1"
        state: present
        update_cache: true

    - name: Hold Kubernetes packages at this version
      ansible.builtin.dpkg_selections:
        name: "{{ item }}"
        selection: hold
      loop: [kubelet, kubeadm, kubectl]

    - name: Enable kubelet service
      ansible.builtin.service:
        name: kubelet
        enabled: true
        state: started
      # সতর্কতা: Cluster তৈরি না হওয়া পর্যন্ত kubelet CrashLoop-এ থাকতে পারে —
      # এটি স্বাভাবিক; Node প্রস্তুত, Cluster তৈরি এখনো বাকি (২১.১১ দেখুন)।

  handlers:
    - name: Restart containerd
      ansible.builtin.service:
        name: containerd
        state: restarted
```

`daemon.json` Template — `templates/daemon.json.j2`:

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  },
  "storage-driver": "overlay2",
  "exec-opts": ["native.cgroupdriver=systemd"]
}
```

২১.১০ ধাপে ধাপে বাস্তবায়ন

1. Inventory-তে দুটি নতুন গ্রুপ যোগ করুন: `dockerhosts` (docker01, 203.0.113.30) ও `k8snodes` (k8s-node01, 203.0.113.31)।
2. Control Node-এ Docker SDK ইনস্টল করুন: `pip install docker` — আর Collection ইনস্টল করুন: `ansible-galaxy collection install community.docker community.general`।
3. Baseline ও Hardening Playbook দুটি নতুন সার্ভারে চালান (অধ্যায় ১৭ ও ১৯)।
4. Docker Host Playbook চালান: `ansible-playbook -i inventory/hosts.yml playbooks/docker-host.yml --check --diff` দিয়ে প্রথমে দেখুন, তারপর আসল Run করুন।
5. যাচাই করুন: `docker ps` (নতুন করে লগইন করার পর, গ্রুপ কার্যকর করতে), `docker images`, ও `curl http://127.0.0.1:4000/api/health`।
6. Kubernetes Node Playbook চালান: `ansible-playbook -i inventory/hosts.yml playbooks/k8s-node.yml`।

7. যাচাই করুন: `swapoff --show` (খালি থাকা উচিত), `lsmod | grep -E 'overlay|br_netfilter'`, `sysctl net.ipv4.ip_forward` (1 হওয়া উচিত), `systemctl status containerd`, ও `kubeadm version`।
8. মনে রাখুন: এই ধাপের পর Node প্রস্তুত, কিন্তু Cluster তৈরি হয়নি — `kubelet` CrashLoop-এ থাকতে পারে, এটি স্বাভাবিক (২১.১১ দেখুন)।

২১.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

Docker Host-এর যাচাই-আউটপুট এরকম দেখাবে (চিত্রণমূলক আউটপুট):

```
$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES          STATUS         PORTS
technova-api   Up 5 minutes   0.0.0.0:4000->4000/tcp
# ব্যাখ্যা: Container চলছে, Host-এর 4000 Port Container-এর 4000 Port-এ যুক্ত।

$ curl http://127.0.0.1:4000/api/health
{"status": "ok", "db": "connected", "uptime": 300}
# ব্যাখ্যা: Container-এর ভেতরের API Health Check-এ সাড়া দিচ্ছে।

$ docker logs technova-api --tail 5
TechNova API listening on port 4000
Connected to MongoDB at 203.0.113.12:27017
# ব্যাখ্যা: Container-এর লগে API চালু ও ডেটাবেস সংযোগ দেখা যাচ্ছে।
```

Kubernetes Node-এর যাচাই-আউটপুট এরকম দেখাবে (চিত্রণমূলক আউটপুট):

```
$ swapoff --show
# (কোনো আউটপুট নেই — Swap বন্ধ, এটাই সঠিক।)

$ sysctl net.ipv4.ip_forward net.bridge.bridge-nf-call-iptables
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-iptables = 1
# ব্যাখ্যা: Kubernetes-এর জন্য দরকারি Kernel Parameter চালু আছে।

$ systemctl status containerd --no-pager | head -n 3
● containerd.service - containerd container runtime
   Active: active (running)
# ব্যাখ্যা: containerd চলছে।

$ kubeadm version
kubeadm version: &version.Info{Major:"1", Minor:"31", GitVersion:"v1.31.0"}
# ব্যাখ্যা: kubeadm ইনস্টল আছে ও সংস্করণ 1.31।

$ systemctl status kubelet --no-pager | head -n 5
● kubelet.service - kubelet: The Kubernetes Node Agent
   Active: active (running)
# সতর্কতা: Cluster তৈরি না হওয়া পর্যন্ত kubelet-এর লগে ত্রুটি দেখা যেতে পারে
# (যেমন "node not found") — এটি স্বাভাবিক; Node প্রস্তুত, Cluster এখনো বাকি।
# Cluster তৈরি করা এই অধ্যায়ের বাইরে — পরের Volume-এ আসবে।
```

একটি গুরুত্বপূর্ণ পার্থক্য: Docker Host-এ `docker ps`-এ Container দেখা যাওয়া মানেই কাজ শেষ; Kubernetes Node-এ kubelet চালু থাকা মানেই কাজ শেষ নয় — Node প্রস্তুত, কিন্তু Cluster-এ যোগ দেওয়া হয়নি। দুটি অবস্থাকে গুলিয়ে ফেলবেন না।

২১.১২ সাধারণ ভুল

- **Docker SDK না ইনস্টল করে `community.docker` চালানো:** মডিউল "Docker SDK not found" বলে ব্যর্থ হয়। Control Node-এ `pip install docker` করুন।
- **`docker` গ্রুপে সবাইকে যোগ করা:** Docker গ্রুপ মানে কার্যত Root অধিকার। শুধু বিশ্বস্ত ব্যবহারকারীকে গ্রুপে রাখুন।
- **`daemon.json`-এ ভুল JSON:** একটি কমা ভুল হলেই Docker Daemon চালু হবে না। Template লেখার পর `python3 -m json.tool` দিয়ে যাচাই করুন।
- **Container-এর Log সীমা না দেওয়া:** `max-size` না থাকলে Log ফাইল Disk ভরে ফেলে। `daemon.json`-এ সবসময় Log সীমা রাখুন।
- **Swap বন্ধ না করে kubelet চালানো:** kubelet Swap চালু থাকলে ব্যর্থ হয়। `swapoff -a` ও `/etc/fstab` — দুটোই করুন।
- **containerd-এ `SystemdCgroup = false` রাখা:** kubelet ও containerd-এর Cgroup Driver না মিললে kubelet চালু হবে না। Kubernetes Node-এ সবসময় `SystemdCgroup = true` রাখুন।
- **Kubernetes প্যাকেজের সংস্করণ না পিন করা:** সব Node-এ ভিন্ন সংস্করণ থাকলে Cluster-এ সমস্যা হয়। সংস্করণ Variable-এ রাখুন ও `dpkg_selections` দিয়ে Hold করুন।
- **Docker Host ও Kubernetes Node-এর Config গুলিয়ে ফেলা:** Docker-এর `daemon.json` ও Kubernetes-এর containerd Config আলাদা — একটির মান অন্যটিতে কপি করবেন না।

২১.১৩ সমস্যা সমাধান (Troubleshooting)

1. **লক্ষণ:** `docker ps`-এ `permission denied` — **পরীক্ষা:** `groups nahid` — `docker` গ্রুপ আছে কি না। **সমাধান:** গ্রুপে যোগ করার পর নতুন করে লগইন করুন (গ্রুপ তখনই কার্যকর হয়); অথবা `sudo docker ps` দিয়ে পরীক্ষা করুন।
2. **লক্ষণ:** Docker Daemon চালু হচ্ছে না — `journalctl -u docker -n 50`-এ JSON ত্রুটি — **পরীক্ষা:** `python3 -m json.tool /etc/docker/daemon.json`। **সমাধান:** Template-এর JSON ঠিক করুন; কখনোই হাতে `daemon.json` এডিট করে ঠিক করবেন না — পরের Run-এ তা মুছে যাবে।
3. **লক্ষণ:** `docker_container` Task-এ `Failed to import docker` — **পরীক্ষা:** Control Node-এ `python3 -c "import docker"`। **সমাধান:** `pip install docker` করুন; Ansible যে Python ব্যবহার করে সেই Python-এই SDK থাকতে হবে — `ansible_python_interpreter` দেখুন।
4. **লক্ষণ:** Container চালু হয়েই থেমে যায় — **পরীক্ষা:** `docker logs technova-api`। **সমাধান:** সাধারণ কারণ: ভুল `DATABASE_URL`, Port ইতিমধ্যে ব্যবহৃত হচ্ছে, বা Image-এ `curl` না থাকায় Healthcheck ব্যর্থ — লগ পড়ে কারণটি ঠিক করুন।
5. **লক্ষণ:** kubelet ব্যর্থ — `failed to run Kubelet: misconfiguration: kubelet cgroup driver: "systemd" is different from docker cgroup driver: "cgroupfs"` — **পরীক্ষা:** `/etc/containerd/config.toml`-এ `SystemdCgroup`। **সমাধান:** `SystemdCgroup = true` সেট করে containerd Restart করুন, তারপর kubelet Restart করুন।
6. **লক্ষণ:** kubelet ব্যর্থ — `running with swap on is not supported` — **পরীক্ষা:** `swapon --show` ও `/etc/fstab`। **সমাধান:** `swapoff -a` চালান ও `/etc/fstab`-এর Swap লাইন মন্তব্য করুন; Reboot-এর পরেও Swap বন্ধ আছে কি না যাচাই করুন।

7. **লক্ষণ:** `kubeadm` প্যাকেজ পাওয়া যায়নি — **পরীক্ষা:** Repository-এর URL ও Key। **সমাধান:** `pkgs.k8s.io`-এর সরকারি ডকুমেন্টেশন থেকে Repository-এর সঠিক URL ও সংস্করণ যাচাই করুন; সংস্করণ Variable-টি ঠিক আছে কি না দেখুন।

২১.১৪ নিরাপত্তা বিবেচনা

> **WARNING:** Docker Socket (`/var/run/docker.sock`) কখনোই Container-এর ভেতরে Mount করবেন না — এটি Container-কে Host-এর Docker Daemon-এর চাবি দিয়ে দেওয়া, যার মাধ্যমে Container Host-এ Root হতে পারে। একইভাবে `--privileged` Container চালাবেন না, আর Host-এর `/` ডিরেক্টরি Container-এ Mount করবেন না। এই তিনটি ভুলের যেকোনো একটি Container-এর Isolation ভেঙে দেয় — Volume 06-এ এই সতর্কতা বিস্তারিত এসেছে।

Container-পরিবেশের নিরাপত্তা-দিকগুলো: প্রথমত, শুধু বিশ্বস্ত Registry থেকে Image টানুন, আর Image-এর Digest যাচাই করুন — Tag (`latest`) বদলে যেতে পারে, Digest বদলায় না। দ্বিতীয়ত, Container Root দিয়ে চালাবেন না — `docker_container`-এ `user` প্যারামিটার দিয়ে আলাদা User দিন, আর Image-এ `USER` নির্দেশ থাকা উচিত। তৃতীয়ত, Container-এর Secret Environment Variable-এ Plain-Text নয় — Vault থেকে আসবে, আর `docker inspect` দিয়ে Secret দেখা যায় বলে Secret-কে ফাইলে Mount করা (Docker Secret বা Volume) ভালো। চতুর্থত, Docker Host-এর Firewall-এ শুধু প্রয়োজনীয় Port খোলা থাকবে — Container-এর Port Host-এ Publish করলেই তা বাইরে থেকে পৌঁছানো যায়, তাই `127.0.0.1:4000:4000` দিয়ে শুধু Localhost-এ বাঁধা নিরাপদ, যখন বাইরে থেকে সরাসরি পৌঁছানোর দরকার নেই। পঞ্চমত, Kubernetes Node-এ kubelet-এর Port (10250) শুধু Control Plane থেকে পৌঁছানো উচিত — সবার জন্য খোলা রাখবেন না। সবশেষে, Image নিয়মিত আপডেট করুন — পুরনো Image-এ পুরনো দুর্বলতা থাকে; Image Scan করা প্রোডাকশন-অভ্যাস।

২১.১৫ প্রোডাকশন প্রকৌশল অনুশীলন

প্রোডাকশনে Container-পরিবেশের নিয়ম: প্রথমত, Docker Host ও Kubernetes Node — দুটি ভূমিকার Playbook আলাদা রাখুন, আর প্রতিটি Role-এর পরিবর্তন Staging-এ পরীক্ষা করে Production-এ চালান। দ্বিতীয়ত, Image-এর সংস্করণ সবসময় Tag দিয়ে পিন করুন — `latest` ব্যবহার করবেন না, কারণ `latest` কখন কী বোঝায় তা নিশ্চিত নয়; `technova-api:1.0.0`-এর মতো সুনির্দিষ্ট Tag ব্যবহার করুন। তৃতীয়ত, Container-এর Log ও Resource সীমা দিন — `docker_container`-এ `memory` ও `cpu_shares` দিয়ে সীমা বেঁধে দিন, যাতে একটি Container পুরো Host-এর RAM খেয়ে না ফেলে। চতুর্থত, Container-এর ডেটা Volume-এ রাখুন — Container মুছে গেলে তার ভেতরের ফাইল মুছে যায়; ডেটাবেসের ডেটা কখনোই Container-এর ভেতরে রাখবেন না। পঞ্চমত, Kubernetes Node-এর সংস্করণ সব Node-এ একই রাখুন — একটি Node-এ নতুন সংস্করণ, আরেকটিতে পুরনো — এই অবস্থা Cluster-এ সমস্যা তৈরি করে। ষষ্ঠত, Docker Host-এর Disk পর্যবেক্ষণ করুন — পুরনো Image ও থেমে যাওয়া Container Disk ভরে ফেলে; নিয়মিত `docker system prune` (সতর্কতার সঙ্গে, শুধু ল্যাবে) বা Image পরিষ্কারের নীতি রাখুন। সবশেষে, মনে রাখুন: এই অধ্যায়ের কাজ হলো পরিবেশ প্রস্তুত করা — Cluster পরিচালনা, Pod পরিচালনা ও Upgrade — সেগুলো পরের Volume-এর কাজ, আর সেখানে এই অধ্যায়ের Node-গুলোই ব্যবহৃত হবে।

২১.১৬ অধ্যায়ের সারাংশ

Docker Host তৈরি মানে Docker-এর Repository যোগ করে Docker CE ইনস্টল করা, `daemon.json` দিয়ে Daemon কনফিগার করা (Log সীমা, `overlay2`, systemd Cgroup Driver), ব্যবহারকারীকে `docker` গ্রুপে যোগ করা, আর

`community.docker` Collection-এর `docker_image`, `docker_container` ও `docker_compose_v2` দিয়ে Container Deploy করা। Kubernetes Node প্রস্তুত মানে Swap বন্ধ করা, `overlay` ও `br_netfilter` Module লোড করা, তিনটি sysctl Parameter বসানো, containerd-এ `SystemdCgroup = true` সেট করা, আর kubelet, kubeadm ও kubectl একই সংস্করণে ইনস্টল করা। এই অধ্যায়টি Configuration-এর অধ্যায় — Cluster তৈরি ও পরিচালনা এর বাইরে। এই ভিত্তির ওপরেই Part IV শেষ হয়: Baseline থেকে Deploy, Hardening থেকে Cloud, আর Container-পরিবেশ — TechNova-র পুরো Provisioning ও Configuration চক্রটি এখন একটি Playbook-এর সেটে বাঁধা।

২১.১৭ জ্ঞান যাচাই

1. Docker Host ও Kubernetes Node — দুটি ভূমিকার ভিত্তি কী এক, আর Config কী আলাদা? দুটি পার্থক্য লিখুন।
2. `daemon.json`-এ Log সীমা (`max-size`, `max-file`) কেন দরকার?
3. `docker_container` মডিউল Config বদলালে Container-এর কী হয়, এবং কেন?
4. kubelet Swap চালু থাকলে কেন কাজ করে না, আর Swap বন্ধ করতে দুটি ধাপ কেন দরকার?
5. containerd-এ `SystemdCgroup = true` না থাকলে kubelet-এ কী ত্রুটি দেখা যায়, এবং কেন?

২১.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

1. **Docker Host:** ল্যাবে Docker Host Playbook চালিয়ে `technova-api` Container চালান; `docker ps`, `docker logs` ও Health Check-এর আউটপুট সংরক্ষণ করুন।
2. **Compose Stack:** Web + API — দুটি Container-এর একটি Compose ফাইল Template দিয়ে তৈরি করে `docker_compose_v2` দিয়ে Deploy করুন; একটি Service-এর Image Tag বদলে দেখুন শুধু সেই Service-টি নতুন করে তৈরি হয় কি না।
3. **Kubernetes Node:** ল্যাবে Kubernetes Node Playbook চালিয়ে Swap, Kernel Module, sysctl, containerd Config ও kubelet সংস্করণ — পাঁচটি যাচাইয়ের আউটপুট সংরক্ষণ করুন; kubelet-এর লগে "node not found" জাতীয় ত্রুটি দেখা গেলে তা কেন স্বাভাবিক তা লিখুন।

PART V • DEVOPS ENGINEERING MASTERY

Production Configuration Management

টেস্টিং ও ট্রাবলশুটিং, Ansible CI/CD, AWX ও Automation Controller, এন্টারপ্রাইজ গভর্ন্যান্স ও প্রোডাকশন অপারেশন — সাথে Terraform ও Ansible integration।

অধ্যায় ২২ • PART V — Testing, Validation & Troubleshooting

Chapter 22 — Ansible Testing, Validation, and Troubleshooting

২২.১ ভূমিকা

Ansible-এর সবচেয়ে বড় শক্তি — মানুষের পড়ার মতো YAML দিয়ে শত শত সার্ভার একসঙ্গে বদলে ফেলা — একই সঙ্গে তার সবচেয়ে বড় ঝুঁকিও। একটি ভুল ইন্ডেন্টেশন, একটি ভুল ভেরিয়েবল নাম, বা একটি টাস্কের ভুল `when` শর্ত মুহূর্তের মধ্যে সব সার্ভারে ভুল কনফিগারেশন ছড়িয়ে দিতে পারে। হাতে-লেখা শেল স্ক্রিপ্টে ভুল হলে সাধারণত একটি সার্ভার নষ্ট হয়; Ansible-এ ভুল হলে ইনভেন্টরির সব সার্ভার একসঙ্গে নষ্ট হয়। তাই প্রোডাকশনে Ansible চালানোর আগে পরীক্ষা (Testing), যাচাই (Validation) ও সমস্যা সমাধান (Troubleshooting) শেখা কোনো বিলাসিতা নয় — এটি টিকে থাকার শর্ত। এই অধ্যায়ে আপনি শিখবেন কীভাবে Playbook চালানোর আগেই ভুল ধরা যায় (`--syntax-check`, `--check`, `--diff`), কীভাবে `ansible-lint` ও `yamllint` দিয়ে কোডের মান ধরে রাখা যায়, কীভাবে `molecule` ও `ansible-test` দিয়ে Role-এর স্বয়ংক্রিয় পরীক্ষা লেখা যায়, কীভাবে `assert` মডিউল দিয়ে Playbook-এর ভেতরেই শর্ত যাচাই করা যায়, এবং কীভাবে `-v` থেকে `-vvvv` পর্যন্ত Verbose মোড, `debug` মডিউল ও `register` ব্যবহার করে ব্যর্থতার মূল কারণ খুঁজে বের করা যায়। TechNova-র প্রোডাকশন সার্ভারগুলোতে (203.0.113.10–203.0.113.12) কিছু বদলানোর আগে আমরা প্রতিটি ধাপ ল্যাবে পরীক্ষা করব — কারণ প্রোডাকশনে "চেষ্টা করে দেখি" বলার কোনো সুযোগ নেই।

২২.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) `ansible-lint` ও `yamllint` ইনস্টল করে Playbook ও Role-এর মান যাচাই করতে; (২) `--syntax-check`, `--check` (Dry Run) ও `--diff`-এর পার্থক্য ব্যাখ্যা করতে এবং সঠিক পরিস্থিতিতে প্রতিটি ব্যবহার করতে; (৩) `molecule` দিয়ে Role-এর জন্য স্বয়ংক্রিয় পরীক্ষার কাঠামো বোঝাতে; (৪) `ansible-test`-এর ভূমিকা ব্যাখ্যা করতে; (৫) `assert` মডিউল দিয়ে Playbook-এর ভেতরে প্রত্যাশা (Expectation) পরীক্ষা করতে; (৬) `-v` থেকে `-vvvv` পর্যন্ত Verbose স্তরের পার্থক্য বলতে; (৭) `UNREACHABLE`, `FAILED` ও `changed=false` বিভ্রান্তির কারণ নির্ণয় করতে; (৮) `debug` মডিউল ও `register` দিয়ে চলমান Playbook-এর ভেতরের অবস্থা দেখতে; (৯) ব্যর্থ Playbook-এর জন্য একটি নিয়মতান্ত্রিক Troubleshooting পদ্ধতি অনুসরণ করতে।

২২.৩ পূর্বশর্ত ধারণা

এই অধ্যায় বুঝতে আপনার জানা থাকা দরকার: Playbook, Role, Inventory ও Variable-এর মৌলিক ধারণা (আগের অধ্যায়গুলো থেকে), YAML-এর ইন্ডেন্টেশন নিয়ম, Linux টার্মিনালে কমান্ড চালানো ও লগ পড়া, এবং SSH কীভাবে কাজ করে তার প্রাথমিক ধারণা। Python-এর অভিজ্ঞতা থাকলে `molecule` ও `ansible-test` বুঝতে সুবিধা হবে, তবে আবশ্যক নয় — প্রতিটি ধারণা এখানে প্রথম থেকে ব্যাখ্যা করা হয়েছে। Control Node-এ Ansible Core 2.17+ ও Python 3.12 ইনস্টল করা আছে ধরে নেওয়া হচ্ছে; TechNova-র ল্যাব ইনভেন্টরিতে তিনটি সার্ভার আছে: `web01.technova.example` (203.0.113.10), `api01.technova.example` (203.0.113.11) ও `db01.technova.example` (203.0.113.12)।

২২.৪ প্রথম নীতি থেকে ব্যাখ্যা

পরীক্ষার মূল নীতিটি খুব সহজ: **ভুল যত আগে ধরা যায়, তার খরচ তত কম।** সফটওয়্যার প্রকৌশলে একে বলে Shift-Left Testing — পরীক্ষাকে যতটা সম্ভব বাম দিকে, অর্থাৎ কাজের শুরুতে সরিয়ে নেওয়া। একটি YAML ইন্ডেন্টেশন ভুল লেখার

সময় ধরা গেলে খরচ শূন্য; Playbook চালানোর সময় ধরা গেলে খরচ কয়েক মিনিট; প্রোডাকশন সার্ভারে ভুল কনফিগারেশন ছড়িয়ে যাওয়ার পর ধরা গেলে খরচ ঘণ্টার পর ঘণ্টা ডাউনটাইম ও গ্রাহকের আস্থা। Ansible-এর পরীক্ষার স্তরগুলো ঠিক এই ধাপগুলো অনুসরণ করে: প্রথমে **Static Analysis** — কোড না চালিয়ে শুধু পড়ে ভুল ধরা (yamllint, ansible-lint, --syntax-check); তারপর **Dry Run** — আসলে কিছু না বদলে কী বদলাত তা দেখা (--check, --diff); তারপর **Isolated Test** — আসল সার্ভারের মতো একটি বিচ্ছিন্ন পরিবেশে (Container বা VM) পুরো Role চালিয়ে দেখা (molecule); তারপর **Runtime Assertion** — Playbook চলার সময়েই প্রতিটি ধাপের ফল যাচাই করা (assert); এবং শেষে **Debugging** — কিছু ভুল হলে তার মূল কারণ খুঁজে বের করা (verbose, debug, register)। এই স্তরগুলো একে অপরের বিকল্প নয়, পরিপূরক: প্রতিটি স্তর আগের স্তর ধরতে না পারা এক ধরনের ভুল ধরে।

> **CONCEPT:** Ansible-এ "পরীক্ষা" মানে শুধু "Playbook চলে কি না" দেখা নয়। একটি Playbook সফলভাবে চললেও ভুল কাজ করতে পারে — যেমন ভুল Port-এ Nginx চালু করা, বা ভুল ব্যবহারকারীর মালিকানায় ফাইল তৈরি করা। তাই ভালো পরীক্ষা তিনটি প্রশ্নের উত্তর দেয়: (১) কোডটি কি বৈধ? (২) এটি কি প্রত্যাশিত পরিবর্তনই করবে? (৩) পরিবর্তনের পর সিস্টেমের অবস্থা কি প্রত্যাশিত? --syntax-check প্রথম প্রশ্নের, --check/--diff দ্বিতীয় প্রশ্নের, আর assert ও molecule তৃতীয় প্রশ্নের উত্তর দেয়।

Idempotence (আইডেমপোটেন্স) ধারণাটিও এখানে মনে রাখা জরুরি: একটি ভালো Ansible Task দ্বিতীয়বার চালালে আর কোনো পরিবর্তন করে না — ফলাফল `changed=false` দেখায়। পরীক্ষার সময় এই বৈশিষ্ট্যই আপনার সবচেয়ে বড় বন্ধু: কোনো Playbook দ্বিতীয়বার চালিয়ে যদি এখনো `changed=true` দেখায়, তাহলে বুঝতে হবে Task-টি সত্যিকার অর্থে Idempotent নয় — প্রতিবার কিছু না কিছু বদলাচ্ছে, যা প্রোডাকশনে অস্থিরতার লক্ষণ।

২২.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

ansible-lint ও yamllint — Static Analysis

`yamllint` একটি সাধারণ YAML Linter: এটি YAML ফাইলের গঠন (Syntax), ইন্ডেন্টেশন, লাইনের দৈর্ঘ্য, ট্রেলিং স্পেস ও ডকুমেন্ট শুরুর `---` চিহ্ন পরীক্ষা করে। এটি Ansible-নির্দিষ্ট কিছু জানে না — শুধু YAML বৈধ কি না দেখে। `ansible-lint` তার ওপরে Ansible-নির্দিষ্ট নিয়ম যোগ করে: যেমন Task-এ নাম (`name`) না থাকা, `command` মডিউলের বদলে বিশেষায়িত মডিউল (যেমন `apt` বা `copy`) ব্যবহার না করা, Fully Qualified Collection Name (FQCN) — যেমন `ansible.builtin.apt` — ব্যবহার না করা, এবং Deprecated (পুরনো) সিনট্যাক্স ব্যবহার। দুটিকে একসঙ্গে চালালে YAML-এর ভুল ও Ansible-এর সেরা অনুশীলনের (Best Practice) লঙ্ঘন — দুটোই ধরা পড়ে। ইনস্টল করা হয় `pip install ansible-lint yamllint` দিয়ে; চালানো হয় `ansible-lint playbooks/site.yml` ও `yamllint playbooks/` দিয়ে।

--syntax-check, --check ও --diff

`ansible-playbook --syntax-check site.yml` শুধু Playbook-এর YAML ও Jinja2 টেমপ্লেটের গঠন পরীক্ষা করে — কোনো সার্ভারে সংযোগ করে না, কিছু বদলায় না। এটি সবচেয়ে দ্রুত পরীক্ষা, তাই প্রতিটি সম্পাদনার পর চালানো উচিত। `--check` (বা `-C`) হলো Dry Run মোড: Ansible সার্ভারে সংযোগ করে, প্রতিটি Task-এর জন্য "এই Task চললে কী বদলাত" তা অনুমান করে, কিন্তু আসলে কিছু বদলায় না। মনে রাখবেন: `--check` সব মডিউলের জন্য নিখুঁত নয় — যেসব মডিউল `check_mode` সমর্থন করে না (যেমন কিছু `command` বা `shell` Task), সেগুলো Dry Run-এ এড়িয়ে যাওয়া হয় বা ভুল অনুমান দিতে পারে। `--diff` (`-D`) সাধারণত `--check`-এর সঙ্গে ব্যবহার করা হয়: এটি ফাইল বদলানোর আগে ও পরে কী পার্থক্য হবে তা Unified Diff আকারে দেখায় — যেমন কোনো Template ফাইলে কোন লাইন যোগ বা বাদ যাবে। প্রোডাকশনে কিছু বদলানোর আগে `ansible-playbook site.yml --check --diff` চালানো একটি পেশাদার অভ্যাস।

molecule ও ansible-test

`molecule` হলো Ansible Role পরীক্ষার সবচেয়ে জনপ্রিয় ফ্রেমওয়ার্ক। এটি একটি Role-কে একটি বিচ্ছিন্ন পরিবেশে (সাধারণত Docker Container বা Podman) বারবার চালায়: প্রথমে Container তৈরি করে (Create), তারপর Role প্রয়োগ করে (Converge), তারপর স্বয়ংক্রিয় পরীক্ষা চালায় — যেমন "Nginx কি চলছে?", "Port 80 কি খোলা?" (Verify), শেষে পরিবেশ মুছে ফেলে (Destroy)। এতে Role-টি আসল সার্ভার ছোঁয়ার আগেই প্রমাণিত হয়। `ansible-test` হলো Ansible-এর নিজস্ব পরীক্ষার টুল, যা মূলত Ansible Collection ও মডিউল ডেভেলপারদের জন্য — এটি Unit Test, Sanity Test (যেমন Python কোডের মান, ডকুমেন্টেশনের সম্পূর্ণতা) ও Integration Test চালায়। সাধারণ Playbook লেখকদের জন্য `molecule` বেশি প্রাসঙ্গিক; `ansible-test` জানা দরকার Collection বানাতে গেলে।

assert মডিউল

`ansible.builtin.assert` মডিউল Playbook-এর ভেতরে একটি প্রত্যাশা ঘোষণা করে: "এই শর্ত সত্য হতেই হবে, না হলে Playbook এখানেই থামবে।" যেমন, Nginx ইনস্টলের পর `assert` দিয়ে পরীক্ষা করা যায় যে সার্ভিসটি সত্যিই চলছে, বা কোনো কনফিগারেশন ফাইলে প্রত্যাশিত লাইনটি আছে কি না। `assert`-এর `that` প্যারামিটারে শর্তের তালিকা ও `fail_msg`-এ ব্যর্থ হলে দেখানোর বার্তা লেখা হয়। এটি প্রোডাকশন Playbook-এর "নিরাপত্তা-জাল" (Safety Net): পরের Task-গুলো ভুল অবস্থার ওপর ভিত্তি করে চলতে পারে না।

Verbose মোড: -v থেকে -vvvv

Ansible-এর Verbose ফ্ল্যাগ প্রতিটি স্তরে আরও বিস্তারিত তথ্য দেখায়: `-v` — প্রতিটি Task-এর ফলাফল বিস্তারিতভাবে, `-vv` — Task-এর ফলাফলের সঙ্গে সংযোগের তথ্য, `-vvv` — SSH সংযোগের বিস্তারিত লগ (কোন কমান্ড পাঠানো হচ্ছে), `-vvvv` — সবচেয়ে গভীর স্তর: SSH সংযোগের প্রতিটি ধাপ, পাঠানো ডেটা ও প্রাপ্ত উত্তর। সাধারণ Debugging-এর জন্য `-vvv` যথেষ্ট; `-vvvv` শুধু SSH সংযোগের গভীর সমস্যায় ব্যবহার করা হয়, কারণ এর আউটপুট বিশাল ও পড়া কঠিন।

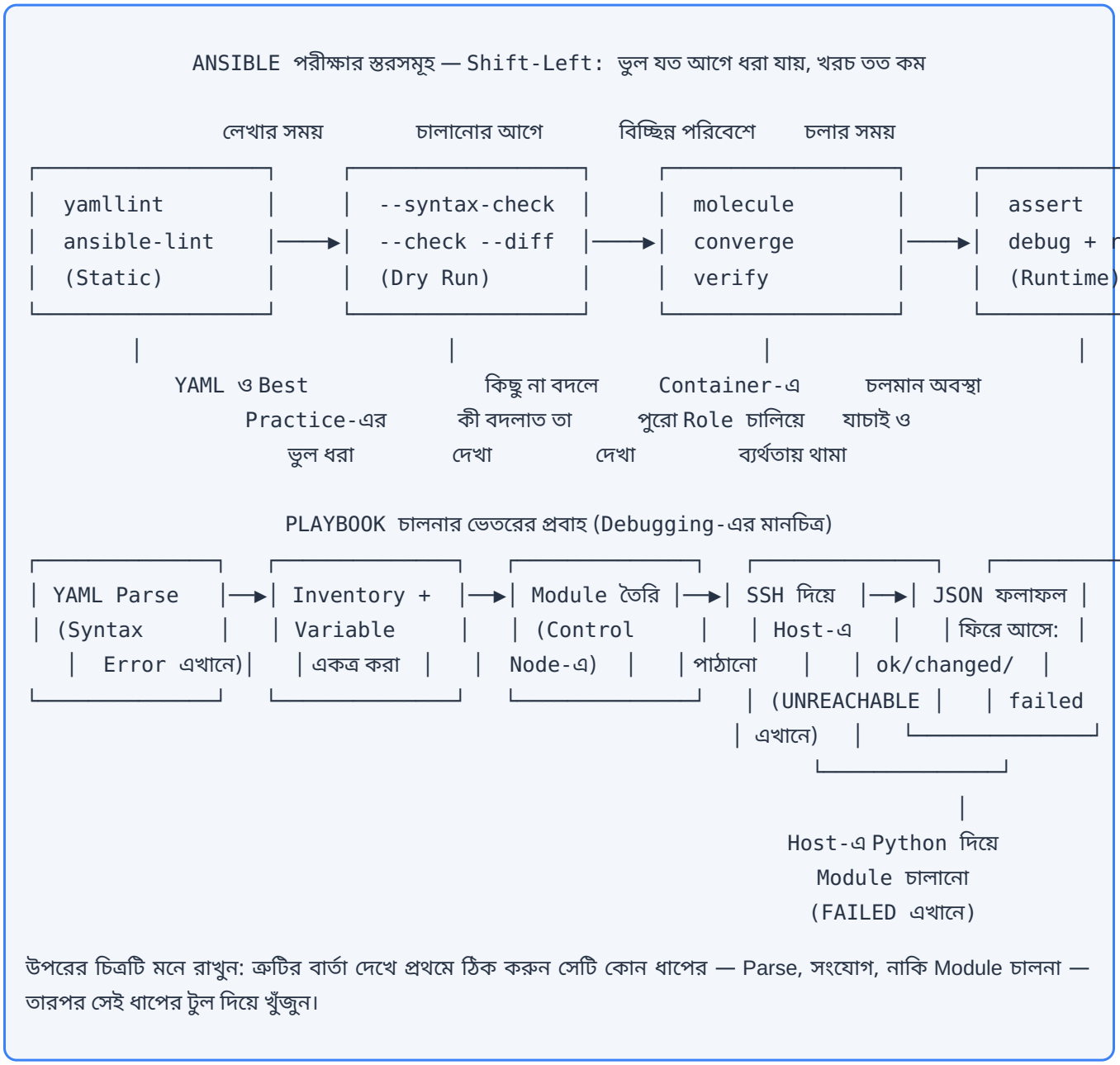
২২.৬ অভ্যন্তরীণ কার্যপ্রণালী

Ansible যখন একটি Playbook চালায়, তখন ভেতরে যা ঘটে তা বুঝলে Debugging অনেক সহজ হয়। প্রথমে Control Node Playbook-এর YAML পার্স (Parse) করে — এই ধাপেই `--syntax-check` কাজ করে; YAML-এ ভুল থাকলে এখানেই ত্রুটি আসে, কোনো সার্ভারে সংযোগের আগেই। তারপর Ansible Inventory পড়ে, Variable একত্র করে (Variable Precedence অনুযায়ী) ও প্রতিটি Host-এর জন্য Task-এর তালিকা তৈরি করে। তারপর প্রতিটি Task-এর জন্য: Control Node একটি ছোট Python প্রোগ্রাম (Module) তৈরি করে, SSH দিয়ে তা Target Host-এ পাঠায়, Host-এ Python দিয়ে তা চালায়, ফলাফল JSON আকারে ফিরিয়ে নেয়, এবং সেই JSON দেখে সিদ্ধান্ত নেয় — Task সফল (`ok`), পরিবর্তন করেছে (`changed`), নাকি ব্যর্থ (`failed`) হয়েছে। `--check` মোডে এই একই প্রবাহ চলে, শুধু মডিউলকে বলা হয় "বদল না করে শুধু বলো কী বদলাতে"। `register` দিয়ে কোনো Task-এর ফিরে আসা JSON একটি Variable-এ জমা রাখা যায়, আর `debug` মডিউল দিয়ে সেই Variable-এর ভেতরটা দেখা যায়। এই প্রবাহের যেকোনো ধাপে সমস্যা হতে পারে: YAML পার্সে ভুল হলে Syntax Error; SSH সংযোগ না হলে `UNREACHABLE`; মডিউল চালাতে গিয়ে ভুল হলে `FAILED`; আর মডিউল সফল হলেও প্রত্যাশিত অবস্থা না মিললে `assert` ব্যর্থ হয়।

> **WARNING:** `--check` মোড কখনোই "নিরাপদ পরীক্ষা"-র সম্পূর্ণ গ্যারান্টি দেয় না। কিছু মডিউল Check মোড সমর্থন করে না — যেমন `command` বা `shell` দিয়ে চালানো স্ক্রিপ্ট আসলে কী করবে তা Ansible অনুমান করতে পারে না। আবার কিছু Task Check মোডে এড়িয়ে যাওয়া হয় (`skipped`), ফলে Dry Run "সব ঠিক আছে" দেখালেও আসল চালনায় ভুল বেরোতে পারে। তাই --check-কে Molecule-এর মতো আসল পরীক্ষার বিকল্প ভাববেন না — এটি একটি দ্রুত প্রাথমিক যাচাই মাত্র।

আরেকটি গুরুত্বপূর্ণ অভ্যন্তরীণ বিষয় হলো **Changed বনাম OK**: Ansible প্রতিটি Task-এর ফলাফলে `changed` নামের একটি Boolean ফ্ল্যাগ পায়। মডিউল নিজেই ঠিক করে এটি `true` হবে না `false` হবে — যেমন `apt` মডিউল প্যাকেজ ইতিমধ্যে ইনস্টল থাকলে `changed=false` জানায়, আর `copy` মডিউল ফাইলের বিষয়বস্তু একই থাকলে `changed=false` জানায়। কিন্তু `command` ও `shell` মডিউল ডিফল্টভাবে প্রতিবার `changed=true` জানায় — কারণ Ansible জানে না কমান্ডটি কিছু বদলেছে কি না। এজন্য `changed_when` দিয়ে নিজে বলে দিতে হয় কখন পরিবর্তন ধরা হবে। এই যান্ত্রিকতাটি না বুঝলে "Playbook প্রতিবার সবকিছু বদলাচ্ছে" দেখে অনেকেই বিভ্রান্ত হন — আসলে বদলাচ্ছে না, শুধু Ansible জানতে পারছে না যে কিছু বদলায়নি।

২২.৭ আর্কিটেকচার চিত্র



উপরের চিত্রটি মনে রাখুন: ত্রুটির বার্তা দেখে প্রথমে ঠিক করুন সেটি কোন ধাপের — Parse, সংযোগ, নাকি Module চালনা — তারপর সেই ধাপের টুল দিয়ে খুঁজুন।

২২.৮ বাস্তব উদাহরণ

TechNova-র Web সার্ভারে Nginx ইনস্টল করার একটি Playbook-এ আমরা ইচ্ছাকৃতভাবে তিনটি ভুল রাখব, তারপর প্রতিটি পরীক্ষার স্তর দিয়ে সেগুলো ধরব। প্রথম ভুল: ইন্ডেন্টেশন ভুল। দ্বিতীয় ভুল: `command` মডিউল দিয়ে প্যাকেজ ইনস্টল

করা (Best Practice লঙ্ঘন)। তৃতীয় ভুল: Nginx চালু আছে কি না যাচাই না করা। নিচের কমান্ডগুলো Control Node-এ `nahid` ব্যবহারকারী হিসেবে, `~/technova-ansible` ডিরেক্টরিতে চালানো হচ্ছে:

```
# ১. YAML-এর গঠনগত ভুল ধরুন — yamllint
yamllint playbooks/web.yml
# চিত্রণমূলক আউটপুট:
# playbooks/web.yml
#   7:5  error  wrong indentation: expected 6 but found 4  (indentation)

# ২. Ansible Best Practice-এর ভুল ধরুন — ansible-lint
ansible-lint playbooks/web.yml
# চিত্রণমূলক আউটপুট:
# playbooks/web.yml:12: [E] fqcn[action-core] Use FQCN for builtin module actions.
# playbooks/web.yml:15: [E] no-changed-when Commands should not change things if nothing needs
doing.
# playbooks/web.yml:18: [W] name[prefix] All tasks should be named.

# ৩. Syntax যাচাই — কোনো সার্ভারে সংযোগ না করে
ansible-playbook playbooks/web.yml --syntax-check
# চিত্রণমূলক আউটপুট: playbook: playbooks/web.yml

# ৪. Dry Run — কী বদলাত তা দেখুন, কিছু না বদলে
ansible-playbook playbooks/web.yml --check --diff
# চিত্রণমূলক আউটপুট:
# --- before: /etc/nginx/nginx.conf
# +++ after: /etc/nginx/nginx.conf
# @@ -1,3 +1,3 @@
# -worker_processes auto;
# +worker_processes 2;

# ৫. Verbose মোডে একটি ব্যর্থ Task-এর বিস্তারিত দেখুন
ansible-playbook playbooks/web.yml -vvv
# -vvv দেখায়: SSH কমান্ড, পাঠানো Module, Host থেকে ফেরা JSON — সংযোগের সমস্যা ধরতে সেরা

# ৬. শুধু একটি Host-এ, শুধু একটি Tag চালিয়ে পরীক্ষা (সীমিত বিস্ফোরণ-ক্ষেত্র)
ansible-playbook playbooks/web.yml --limit web01.technova.example --tags nginx
```

লক্ষ্য করুন, প্রতিটি ধাপ আগের ধাপের চেয়ে একটু "বাস্তবের কাছাকাছি": yamllint শুধু ফাইল পড়ে, --syntax-check শুধু পার্স করে, --check সার্ভারে সংযোগ করে কিন্তু বদলায় না, আর Molecule পুরোপুরি চালিয়ে দেখে। এই ক্রমেই পরীক্ষা করা উচিত — সরাসরি প্রোডাকশনে চালিয়ে দেখা সবচেয়ে শেষ ও সবচেয়ে ব্যয়বহুল উপায়।

২২.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচে একটি সম্পূর্ণ পরীক্ষাযোগ্য Playbook দেওয়া হলো — এতে `assert`, `register`, `debug` ও `changed_when` সবই ব্যবহার করা হয়েছে। সঙ্গে `ansible-lint` ও `yamllint`-এর কনফিগারেশন ফাইলও আছে, যাতে পুরো টিম একই নিয়ম মেনে চলে:

```
# ── ফাইল: playbooks/web.yml ──────────────────────────────────────────
---
- name: TechNova web servers - Nginx install and verify
  hosts: webservers
  become: true
  vars:
    nginx_worker_processes: 2
```

```
    expected_http_port: 80

  tasks:
    - name: Install Nginx package
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true
      tags: [nginx]

    - name: Deploy Nginx main configuration
      ansible.builtin.template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        owner: root
        group: root
        mode: "0644"
      notify: Reload Nginx
      tags: [nginx]

    - name: Ensure Nginx service is running and enabled
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
      tags: [nginx]

    - name: Gather Nginx service facts
      ansible.builtin.service_facts:
      tags: [nginx]

    - name: Assert Nginx is running
      ansible.builtin.assert:
        that:
          - ansible_facts.services['nginx.service'].state == 'running'
        fail_msg: "Nginx is NOT running on {{ inventory_hostname }} - investigate before
continuing."
        success_msg: "Nginx is running on {{ inventory_hostname }}."
      tags: [nginx]

    - name: Check HTTP port responds
      ansible.builtin.uri:
        url: "http://{{ inventory_hostname }}:{{ expected_http_port }}/"
        status_code: 200
        timeout: 10
      register: http_check
      failed_when: http_check.status != 200
      tags: [nginx]

    - name: Show HTTP check result for debugging
      ansible.builtin.debug:
        var: http_check.status
      tags: [nginx]

  handlers:
    - name: Reload Nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
```

```
# ── ফাইল: .ansible-lint (প্রকল্পের মূল ডিরেক্টরিতে) ──────────────────
---
profile: production
skip_list:
  - name[casing]        # Task-এর নামের বড়-ছোট হাতের নিয়ম এই প্রকল্পে শিথিল
exclude_paths:
  - .git/
  - molecule/
use_default_rules: true
warn_list:
  - experimental

# ── ফাইল: .yamllint (প্রকল্পের মূল ডিরেক্টরিতে) ──────────────────────
---
extends: default
rules:
  line-length:
    max: 120
    level: warning
  indentation:
    spaces: 2
  truthy:
    check-keys: false

# ── ফাইল: molecule/default/molecule.yml (Role পরীক্ষার কাঠামো) ─────────
---
dependency:
  name: galaxy
driver:
  name: docker
platforms:
  - name: instance
    image: geerlingguy/docker-ubuntu2404-ansible:latest
    pre_build_image: true
    privileged: true
provisioner:
  name: ansible
  playbooks:
    converge: converge.yml
verifier:
  name: ansible
scenario:
  test_sequence:
    - dependency
    - lint
    - cleanup
    - destroy
    - syntax
    - create
    - prepare
    - converge
    - idempotence
    - verify
    - cleanup
    - destroy
```

এই ফাইলগুলোর মূল কথা: পরীক্ষা কোনো এককালীন কাজ নয় — `.ansible-lint` ও `.yamllint` প্রকল্পের ভেতরে রেখে দিলে প্রত্যেক ডেভেলপারের মেশিনে ও CI-তে একই নিয়ম চলে; `assert` Playbook-এর ভেতরে থাকলে প্রতিটি চালনায় যাচাই হয়; আর Molecule-এর `test_sequence`-এ `idempotence` ধাপটি মনে করিয়ে দেয় যে দ্বিতীয়বার চালালে আর কিছু বদলানো উচিত নয়।

২২.১০ ধাপে ধাপে বাস্তবায়ন

TechNova-র ল্যাবে পরীক্ষার পরিবেশ গড়ে তোলার ধাপগুলো:

ধাপ ১ — টুল ইনস্টল করুন। Control Node-এ `pip install ansible-lint yamllint molecule molecule-plugins[docker]` চালান। `ansible-lint --version` ও `molecule --version` দিয়ে যাচাই করুন।

ধাপ ২ — কনফিগারেশন ফাইল রাখুন। প্রকল্পের মূল ডিরেক্টরিতে `.ansible-lint` ও `.yamllint` ফাইল তৈরি করুন (উপরের উদাহরণ অনুযায়ী), যাতে সবাই একই নিয়ম পায়।

ধাপ ৩ — Lint চালান। প্রতিটি সম্পাদনার পর `yamllint playbooks/` ও `ansible-lint playbooks/web.yml` চালান; ত্রুটি শূন্য না হওয়া পর্যন্ত এগোবেন না।

ধাপ ৪ — Syntax যাচাই করুন। `ansible-playbook playbooks/web.yml --syntax-check` চালান। এটি সেকেন্ডের মধ্যে শেষ হয়, তাই Git Commit-এর আগে চালানোর অভ্যাস করুন।

ধাপ ৫ — Dry Run করুন। প্রথমে `--limit` দিয়ে একটি মাত্র ল্যাব সার্ভারে `--check --diff` চালান; Diff পড়ে নিশ্চিত হোন যে শুধু প্রত্যাশিত পরিবর্তনই দেখাচ্ছে।

ধাপ ৬ — Molecule দিয়ে Role পরীক্ষা করুন। Role ডিরেক্টরিতে `molecule init scenario` চালিয়ে কাঠামো তৈরি করুন, `converge.yml`-এ Role-টি যোগ করুন, `verify.yml`-এ `assert` দিয়ে প্রত্যাশা লিখুন, তারপর `molecule test` চালান।

ধাপ ৭ — আসল চালনায় যাচাই যোগ করুন। Playbook-এ `assert` Task যোগ করুন (যেমন সার্ভিস চলছে কি না) এবং গুরুত্বপূর্ণ Task-এ `register` + `debug` যোগ করুন, যাতে ব্যর্থ হলে কারণ দেখা যায়।

ধাপ ৮ — ব্যর্থতা অনুশীলন করুন। ইচ্ছা করে একটি ভুল করুন (যেমন ভুল Port নম্বর) এবং `-vvv` দিয়ে মূল কারণ খুঁজুন — এই অনুশীলনই আসল ঘটনার সময় আপনাকে শান্ত রাখবে।

২২.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

সব পরীক্ষা পাস হলে আউটপুটটি এরকম দেখায় (চিত্রণমূলক আউটপুট):

```
$ ansible-lint playbooks/web.yml
# (কোনো আউটপুট নেই — মানে কোনো ত্রুটি নেই; নীরবতাই সাফল্য)

$ ansible-playbook playbooks/web.yml --check --diff --limit web01.technova.example

PLAY [TechNova web servers - Nginx install and verify] *******************

TASK [Install Nginx package] *********************************************
ok: [web01.technova.example]

TASK [Deploy Nginx main configuration] ***********************************
changed: [web01.technova.example]
--- before: /etc/nginx/nginx.conf
```

```
+++ after: /etc/nginx/nginx.conf
@@ -1,3 +1,3 @@
-worker_processes auto;
+worker_processes 2;

TASK [Assert Nginx is running] *************************************
ok: [web01.technova.example] => {
    "msg": "Nginx is running on web01.technova.example."
}

PLAY RECAP *********************************************************
web01.technova.example : ok=4  changed=1  unreachable=0  failed=0  skipped=0
```

আউটপুটটি পড়ার নিয়ম: `PLAY RECAP` লাইনটিই সারাংশ — `ok` মানে সফল Task-এর সংখ্যা, `changed` মানে কতটি Task বদলাত (Dry Run-এ "বদলাত"), `unreachable` মানে সংযোগই হয়নি এমন Host-এর সংখ্যা, `failed` মানে ব্যর্থ Task-এর সংখ্যা। `changed=1` মানে শুধু কনফিগারেশন ফাইলটি বদলাত — ঠিক যা প্রত্যাশিত ছিল। `assert` Task-টি `ok` দেখিয়েছে ও তার `success_msg` ছাপিয়েছে — মানে Nginx চলছে। দ্বিতীয়বার একই Playbook চালালে `changed=0` হওয়া উচিত — সেটাই Idempotence-এর প্রমাণ। Molecule-এর ক্ষেত্রে সাফল্যের চিহ্ন হলো শেষ লাইনে `Idempotence completed successfully` ও `Verifier completed successfully` দেখা।

২২.১২ সাধারণ ভুল

ভুল	কেন সমস্যা	সঠিক অভ্যাস
সরাসরি প্রোডাকশনে Playbook চালানো, --check না করে	ভুল সব সার্ভারে একসঙ্গে ছড়িয়ে পড়ে	আগে ল্যাবে, তারপর --check --diff, তারপর সীমিত Host-এ
`command` / `shell` দিয়ে প্যাকেজ ইনস্টল বা ফাইল কপি করা	Idempotent নয়; প্রতিবার changed=true দেখায়; ভুল লুকায়	বিশেষায়িত মডিউল ব্যবহার করুন: apt, copy, template
`changed_when: false` অন্ধভাবে সব Task-এ বসানো	আসল পরিবর্তন লুকিয়ে যায়; Handler কখনো চলে না	শুধু Read-only কমান্ডে (যেমন `systemctl status`) ব্যবহার করুন
Task-এ `name` না লেখা	আউটপুটে বোঝা যায় না কোন Task ব্যর্থ হলো	প্রতিটি Task-এ স্পষ্ট, বর্ণনামূলক নাম দিন
Vault-এনক্রিপ্ট করা Variable-এর মান `debug` দিয়ে ছাপানো	Secret লগে ফাঁস হয়	Secret কখনো debug করবেন না; `no_log: true` ব্যবহার করুন
`ignore_errors: true` দিয়ে ব্যর্থতা ঢেকে রাখা	ভুল চুপচাপ পার হয়ে যায়; পরে বড় সমস্যা হয়	শুধু প্রত্যাশিত, সামলানো যায় এমন ব্যর্থতায় ব্যবহার করুন; সঙ্গে `failed_when` লিখুন
শুধু "Playbook চলেছে" দেখে সন্তুষ্ট থাকা	ভুল কনফিগারেশনও সফলভাবে চলতে পারে	assert ও Molecule Verify দিয়ে অবস্থা যাচাই করুন

২২.১৩ সমস্যা সমাধান (Troubleshooting)

ব্যর্থতার বার্তা দেখে নিয়মতান্ত্রিকভাবে এগোন: প্রথমে বার্তার ধরন শনাক্ত করুন, তারপর ধাপে ধাপে সংকীর্ণ করুন।

UNREACHABLE — সার্ভারে পৌঁছানো যাচ্ছে না। কারণ সাধারণত SSH: ভুল IP বা Hostname, SSH Key-র অনুমতি ভুল (`~/.ssh/id_ed25519`-এর অনুমতি 600 হওয়া উচিত), ফায়ারওয়ালে Port 22 বন্ধ, বা সার্ভার বন্ধ। পরীক্ষা করুন: `ssh nahid@203.0.113.10` হাতে চালিয়ে দেখুন; `ansible webservers -m ping -vvv` দিয়ে SSH-এর বিস্তারিত লগ দেখুন; `ansible-inventory --graph` দিয়ে Inventory ঠিক আছে কি না দেখুন।

FAILED — Task চলতে গিয়ে ব্যর্থ। আউটপুটের `msg` ও `stderr` পড়ুন — Ansible সাধারণত সঠিক কারণটি লিখে দেয় (যেমন প্যাকেজের নাম ভুল, ফাইলের পাথ নেই, অনুমতি নেই)। তারপর সেই Task-টি `--limit` দিয়ে একটি Host-এ `-vvv` সহ চালান। মনে রাখবেন: একটি Task ব্যর্থ হলে Playbook সেই Host-এর জন্য থেমে যায়, অন্য Host-এ চলতে থাকে — এটাই ডিফল্ট আচরণ; `any_errors_fatal: true` দিলে প্রথম ব্যর্থতাতেই সব থেমে যায়।

changed=false বিভ্রান্তি — "Task চলল, কিন্তু কিছু বদলাল না"। এটি সাধারণত সমস্যা নয়, সাফল্যের চিহ্ন: কাঙ্ক্ষিত অবস্থা ইতিমধ্যে আছে বলেই Ansible কিছু বদলায়নি। কিন্তু যদি আপনি নিশ্চিত হন যে বদলানো উচিত ছিল, তাহলে পরীক্ষা করুন: Template-এর Variable-এর মান কি ঠিক আছে? `when` শর্ত কি Task-টিকে এড়িয়ে যাচ্ছে (তাহলে `skipped` দেখাবে, `ok` নয়)? `notify` করা Handler কি আসলেই চলেছে? `debug` দিয়ে Variable-এর মান ছাপিয়ে দেখুন।

Variable-এর মান ভুল। সবচেয়ে সাধারণ ভুল: ভুল নামের Variable (যেমন `nginx_port`-এর বদলে `nginx_ports`), বা Variable Precedence-এর কারণে অন্য ফাইলের মান জিতে যাওয়া। সমাধান: `ansible-inventory --host web01.technova.example` দিয়ে সেই Host-এর চূড়ান্ত Variable-গুলো দেখুন; `debug: var=nginx_worker_processes` দিয়ে চলমান মান ছাপান।

Template রেন্ডারিং ত্রুটি। Jinja2-এ `undefined variable` ত্রুটি মানে Template-এ ব্যবহৃত কোনো Variable সংজ্ঞায়িত নেই। `default()` ফিল্টার ব্যবহার করুন (`{{ nginx_port | default(80) }}`) বা Variable-টি সংজ্ঞায়িত করুন। `ansible-playbook --syntax-check` এই ভুল ধরে না — কারণ Variable-এর মান চালনার সময় ঠিক হয় — তাই Template-এর ভুল ধরতে `--check` বা Molecule দরকার।

> **TIP:** Troubleshooting-এর সময় সবসময় সমস্যার ক্ষেত্রটি ছোট করুন: প্রথমে একটি Host (`--limit`), তারপর একটি Tag (`--tags`), তারপর একটি Task (`--start-at-task`)। "সব সার্ভারে সব Task" দিয়ে শুরু করলে আউটপুটের ভিড়ে মূল কারণ হারিয়ে যায়। আর প্রতিটি চেষ্টার আগে `--check --diff` চালান — এতে ভুল ধরা পড়লে প্রোডাকশন ছোঁয়ার আগেই ধরা পড়বে।

২২.১৪ নিরাপত্তা বিবেচনা

পরীক্ষার সময়ও নিরাপত্তা উপেক্ষা করা যায় না। প্রথম নিয়ম: **পরীক্ষার লগে কখনো Secret ছাপাবেন না।** `debug` মডিউল দিয়ে Variable দেখার সময় নিশ্চিত হোন তাতে কোনো পাসওয়ার্ড, টোকেন বা Private Key নেই; Secret-ধারী Task-এ `no_log: true` দিন, যাতে Ansible তার আউটপুট লুকিয়ে রাখে। দ্বিতীয় নিয়ম: **পরীক্ষার পরিবেশেও আসল Secret ব্যবহার করবেন না** — Molecule বা ল্যাবের জন্য আলাদা, দুর্বল-কিন্তু-আলাদা টেস্ট Secret ব্যবহার করুন, আর আসল Secret শুধু Ansible Vault-এ এনক্রিপ্ট করে রাখুন। তৃতীয় নিয়ম: **Verbose আউটপুট সাবধানে শেয়ার করুন** — `-vvvv` আউটপুটে SSH সংযোগের বিস্তারিত তথ্য থাকে; এটি Chat বা Issue Tracker-এ পেস্ট করার আগে Secret-জাতীয় অংশ মুছে ফেলুন। চতুর্থ নিয়ম: Lint কনফিগারেশনে `risky-file-permissions` ও `risky-shell-pipe` নিয়মগুলো বন্ধ করবেন না — এগুলো বিপজ্জনক অনুমতি ও Shell Pipe-এর মতো ঝুঁকি ধরে।

২২.১৫ প্রোডাকশন ইঞ্জিনিয়ারিং অনুশীলন

প্রোডাকশন টিমে পরীক্ষা একটি ব্যক্তিগত অভ্যাস নয়, একটি প্রক্রিয়া। প্রথমে, **প্রতিটি Git Commit-এর আগে** `ansible-lint` ও `--syntax-check` চালানো বাধ্যতামূলক করুন (Git Hook বা CI-তে)। দ্বিতীয়ত, **প্রতিটি Pull Request-এ** Molecule পরীক্ষা চালান — Role বদলালে তার পরীক্ষা না পাস করলে Merge হবে না। তৃতীয়ত, **প্রোডাকশনে চালানোর আগে** সর্বদা `--check --diff` চালিয়ে Diff-টি পর্যালোচনা (Review) করুন — দুই চোখে দেখা Diff-ই সবচেয়ে নির্ভরযোগ্য নিরাপত্তা-জাল। চতুর্থত, **পরীক্ষার ফলাফল সংরক্ষণ করুন**: CI-র লগ ও `PLAY RECAP` আউটপুট রেখে দিন, যাতে পরে কোনো ঘটনার তদন্তে দেখা যায় কী বদলানো হয়েছিল। পঞ্চমত, **Idempotence-কে চুক্তি হিসেবে দেখুন**: কোনো Role বা Playbook দ্বিতীয়বার চালালে `changed` দেখালে সেটিকে বাগ হিসেবে ধরুন ও ঠিক করুন — প্রোডাকশনে এই বৈশিষ্ট্যই আপনাকে নিরাপদে পুনরায় চালানোর (Re-run) সাহস দেয়।

২২.১৬ অধ্যায়ের সারাংশ

এই অধ্যায়ে আপনি শিখলেন যে Ansible-এ পরীক্ষা একটি স্তরবিন্যাস: yamllint ও ansible-lint দিয়ে Static যাচাই, --syntax-check দিয়ে গঠন যাচাই, --check ও --diff দিয়ে Dry Run, molecule দিয়ে বিচ্ছিন্ন পরিবেশে পূর্ণ পরীক্ষা, assert দিয়ে চলমান অবস্থা যাচাই, এবং -v থেকে -vvvv পর্যন্ত Verbose মোড, debug ও register দিয়ে Debugging। আপনি দেখলেন UNREACHABLE মানে সংযোগের সমস্যা, FAILED মানে Task চালনার সমস্যা, আর changed=false সাধারণত সাফল্যের চিহ্ন — কাঙ্ক্ষিত অবস্থা ইতিমধ্যে আছে। মূল বার্তা: ভুল যত আগে ধরা যায়, তার খরচ তত কম; তাই পরীক্ষা Playbook লেখার অংশ, তার পরের কাজ নয়।

২২.১৭ জ্ঞান যাচাই

১. `yamllint` ও `ansible-lint`-এর মধ্যে পার্থক্য কী? প্রতিটি কোন ধরনের ভুল ধরে?
২. `--check` মোডে Ansible আসলে কী করে, এবং কেন এটি সব মডিউলের জন্য নিখুঁত নয়?
৩. Molecule-এর `converge`, `idempotence` ও `verify` ধাপগুলো প্রত্যেকে কী পরীক্ষা করে?
৪. একটি Playbook দ্বিতীয়বার চালালে `changed=1` দেখাচ্ছে — এর সম্ভাব্য দুটি কারণ লিখুন এবং কীভাবে নির্ণয় করবেন বলুন।
৫. `assert` মডিউল কেন Playbook-এর "নিরাপত্তা-জাল" হিসেবে কাজ করে? একটি উদাহরণ দিন।

২২.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

TechNova-র ল্যাবে একটি নতুন Role `technova_nginx` তৈরি করুন যা Ubuntu 24.04-এ Nginx ইনস্টল করে, একটি Template দিয়ে কনফিগারেশন বসায় ও সার্ভিস চালু করে। তারপর: (ক) Role-টিতে ইচ্ছাকৃতভাবে দুটি ভুল রাখুন — একটি YAML ইন্ডেন্টেশন ভুল ও একটি `command` মডিউলের অপব্যবহার — এবং `yamllint` ও `ansible-lint` দিয়ে দুটোই ধরে ঠিক করুন; (খ) Playbook-এ `assert` যোগ করুন যা পরীক্ষা করবে Nginx সার্ভিস চলছে ও Port 80 সাড়া দিচ্ছে; (গ) Molecule দিয়ে Role-টি পরীক্ষা করুন ও Idempotence ধাপটি পাস করান; (ঘ) একটি ছোট Troubleshooting নোট লিখুন: ইচ্ছা করে সার্ভিস বন্ধ করে Playbook চালালে কী আউটপুট আসে ও `-vvv` দিয়ে কীভাবে মূল কারণ পেলেন।

অধ্যায় ২৩ • PART V — CI/CD, AWX & Automation Controller

Chapter 23 — Ansible CI/CD, AWX, Automation Controller, and Workflow

২৩.১ ভূমিকা

একজন Engineer যখন নিজের ল্যাপটপ থেকে Playbook চালান, তখন সবকিছু তার নিয়ন্ত্রণে থাকে: তিনি জানেন কোন Inventory ব্যবহার করছেন, কোন Vault পাসওয়ার্ড দিচ্ছেন, আর কিছু ভুল হলে তিনিই থামাতে পারবেন। কিন্তু TechNova যখন বড় হলো — দশজন Engineer, তিনটি পরিবেশ (Dev, Staging, Prod), প্রতিদিন কয়েকবার Deploy — তখন "ল্যাপটপ থেকে চালানো" আর চলল না। কে কখন কী চালিয়েছে তার কোনো হিসাব নেই, একজনের ল্যাপটপে থাকা Playbook আরেকজনের ল্যাপটপের চেয়ে আলাদা, আর প্রোডাকশনের পাসওয়ার্ড ছড়িয়ে আছে সবার `~/.ssh` ও নোটে। এই সমস্যার উত্তর দুটি স্তরে আসে: প্রথম স্তর **CI/CD Pipeline** — Git-এ Push করলেই স্বয়ংক্রিয়ভাবে Lint, Syntax Check ও পরীক্ষা চলে, আর অনুমোদন পেলে Playbook চালু হয়; দ্বিতীয় স্তর **AWX ও Ansible Automation Controller** — একটি ওয়েব-ভিত্তিক কেন্দ্রীয় ব্যবস্থা যেখানে Job Template, Inventory, Credential ও Workflow এক জায়গায় থাকে, কে কী চালাতে পারবে তা নিয়ন্ত্রণ করা যায়, আর প্রতিটি চালনার লগ সংরক্ষিত থাকে। এই অধ্যায়ে আপনি GitHub Actions দিয়ে Ansible Pipeline বানানো, CI-তে Secret সামলানো, AWX বনাম Automation Controller-এর পার্থক্য, এবং Job Template ও Workflow Template দিয়ে নিরাপদ স্বয়ংক্রিয়তা গড়ে তোলা শিখবেন।

২৩.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) GitHub Actions-এ Ansible Lint, Syntax Check ও Dry Run চালানোর Workflow লিখতে; (২) CI Pipeline-এ Ansible Playbook চালানোর ধাপগুলো ব্যাখ্যা করতে; (৩) CI-তে Secret (Vault পাসওয়ার্ড, SSH Key) নিরাপদে ব্যবহারের নিয়ম বলতে; (৪) AWX, Ansible Automation Controller ও ansible-core-এর মধ্যে পার্থক্য স্পষ্টভাবে ব্যাখ্যা করতে — কোনটি Open Source, কোনটি বাণিজ্যিক; (৫) Job Template, Inventory, Credential ও Project-এর ভূমিকা বোঝাতে; (৬) Workflow Template দিয়ে একাধিক Job-কে শর্তসাপেক্ষে সাজাতে; (৭) Execution Environment (EE)-এর ধারণা ও প্রয়োজন ব্যাখ্যা করতে; (৮) TechNova-র জন্য একটি ছোট কিন্তু বাস্তবসম্মত CI + AWX নকশা প্রস্তাব করতে।

২৩.৩ পূর্বশর্ত ধারণা

এই অধ্যায়ের জন্য আপনার জানা থাকা দরকার: Git-এর মৌলিক ব্যবহার (Clone, Commit, Push, Pull Request), GitHub-এর প্রাথমিক ধারণা, অধ্যায় ২২-এর পরীক্ষার টুলগুলো (ansible-lint, --check, --syntax-check), Ansible Vault-এর ধারণা, এবং Inventory ও Playbook-এর কাজ। CI/CD-এর ধারণা (Continuous Integration / Continuous Delivery) আগে থেকে না জানলেও চলবে — এখানে প্রথম নীতি থেকে ব্যাখ্যা করা হয়েছে। AWX চালানোর জন্য Kubernetes বা Docker-এর প্রাথমিক ধারণা কাজে লাগবে, তবে AWX-এর ব্যবহার বোঝার জন্য তা আবশ্যক নয়।

২৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

CI/CD-এর মূল নীতিটি হলো: **মানুষ যা বারবার হাতে করে, তা যন্ত্রকে করতে দিন — আর মানুষ শুধু সিদ্ধান্ত নেবে।** একজন Engineer Playbook বদলানোর পর হাতে Lint চালাতে ভুলতে পারেন, Syntax Check এড়িয়ে যেতে পারেন, ভুল Inventory-তে চালিয়ে ফেলতে পারেন। Pipeline এই ভুলগুলোকে অসম্ভব করে দেয়: Git-এ Push করলেই একই পরীক্ষাগুলো একই ক্রমে, একই পরিবেশে চলে — কেউ এড়িয়ে যেতে পারে না। AWX ও Automation Controller এই

নীতিকে আরও এক ধাপ এগিয়ে নেয়: **কে, কী, কোথায়, কখন চালাতে পারবে — তা একটি কেন্দ্রীয় জায়গা থেকে নিয়ন্ত্রণ করা।** Playbook আর কোনো ব্যক্তির ল্যাপটপে থাকে না, Git Repository-তে থাকে; Secret আর কারো নোটে থাকে না, Controller-এর Credential Store-এ এনক্রিপ্ট করে থাকে; আর প্রতিটি চালনার লগ থাকে — কে, কখন, কোন Job চালিয়েছে, কী ফল হয়েছে। এই তিনটি — একই পরীক্ষা, কেন্দ্রীয় নিয়ন্ত্রণ, সম্পূর্ণ হিসাব — মিলেই একটি টিমকে নিরাপদে স্বয়ংক্রিয় হতে দেয়।

> **CONCEPT:** CI (Continuous Integration) মানে প্রতিটি পরিবর্তন মূল কোডের সঙ্গে মিশিয়ে স্বয়ংক্রিয়ভাবে পরীক্ষা করা; CD (Continuous Delivery/Deployment) মানে পরীক্ষায় পাস করা পরিবর্তনকে পরিবেশে পৌঁছে দেওয়া। Ansible-এর ক্ষেত্রে CI মানে: Push করলেই Lint + Syntax Check + Molecule চলে। CD মানে: অনুমোদনের পর Playbook স্বয়ংক্রিয়ভাবে Staging-এ, তারপর Prod-এ চলে। AWX/Automation Controller হলো সেই CD-এর "চালানোর ঘর" — যেখানে Job-গুলো সময়সূচি (Schedule) অনুযায়ী, API দিয়ে, বা Workflow-এর অংশ হিসেবে চলে।

২৩.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

CI Pipeline-এ Ansible

GitHub Actions-এ একটি সাধারণ Ansible Pipeline-এর ধাপগুলো: (১) `actions/checkout` দিয়ে কোড আনা; (২) Python ও Ansible ইনস্টল করা; (৩) `yamllint` ও `ansible-lint` চালানো; (৪) `ansible-playbook --syntax-check` চালানো; (৫) সম্ভব হলে Molecule পরীক্ষা চালানো; (৬) শুধুমাত্র নির্দিষ্ট Branch (যেমন `main`) ও অনুমোদনের পর আসল Playbook চালানো। গুরুত্বপূর্ণ নীতি: **পরীক্ষার ধাপগুলো প্রতিটি Push-এ চলবে, কিন্তু Deploy ধাপটি শুধু অনুমোদিত পরিস্থিতিতে চলবে** — যেমন শুধু `main` Branch-এ, বা GitHub Environments-এর Approval-এর পর। এতে কোনো পরীক্ষামূলক Branch থেকে ভুল করে প্রোডাকশন বদলে যাওয়ার ঝুঁকি থাকে না।

CI-তে Secret সামলানো

CI-তে Secret-এর তিনটি নিয়ম: প্রথম, **Secret কখনো কোডে বা Workflow ফাইলে লিখবেন না** — GitHub-এর Encrypted Secrets (Repository Settings → Secrets and variables → Actions) ব্যবহার করুন। দ্বিতীয়, **Vault পাসওয়ার্ড Secret হিসেবে দিন, ফাইলে নয়** — Workflow-এ Secret-টিকে একটি অস্থায়ী ফাইলে লিখে `--vault-password-file`-এ দিন, কাজ শেষে মুছে ফেলুন; অথবা `ANSIBLE_VAULT_PASSWORD` পরিবেশ-ভেরিয়েবল ব্যবহার করুন। তৃতীয়, **SSH Key-ও Secret হিসেবে** — Private Key-কে GitHub Secret-এ রেখে Runner-এ `~/.ssh/id_ed25519` হিসেবে লিখুন, অনুমতি 600 দিন, কাজ শেষে মুছে ফেলুন। আর সবচেয়ে গুরুত্বপূর্ণ: CI-র লগে Secret যাতে না ছাপা হয়, সেজন্য Secret-ধারী Task-এ `no_log: true` দিন এবং Workflow-এ `set -x` (Debug Echo) বন্ধ রাখুন।

AWX বনাম Automation Controller বনাম ansible-core

এই তিনটি নামের বিভ্রান্তি দূর করা জরুরি। **ansible-core** হলো মূল কমান্ড-লাইন টুল — `ansible-playbook`, `ansible`, `ansible-vault` — যা আপনি টার্মিনালে চালান; এটি সম্পূর্ণ Open Source ও বিনামূল্যে। **AWX** হলো ansible-core-এর ওপরে গড়া একটি Open Source ওয়েব-ভিত্তিক প্ল্যাটফর্ম — এতে আছে ওয়েব UI, REST API, Role-Based Access Control (RBAC), Job Template, Inventory ব্যবস্থাপনা, Scheduling ও Workflow। AWX Red Hat-এর তত্ত্বাবধানে কমিউনিটি দ্বারা তৈরি হয় এবং এর সোর্স কোড সবার জন্য উন্মুক্ত। **Ansible Automation Controller** (আগের নাম Ansible Tower) হলো Red Hat Ansible Automation Platform-এর বাণিজ্যিক (Commercial) পণ্য — এটি AWX-এর ওপর ভিত্তি করে তৈরি, কিন্তু Red Hat-এর সমর্থন (Support), পরীক্ষিত ও স্থিতিশীল রিলিজ, এবং অতিরিক্ত

Enterprise বৈশিষ্ট্য (যেমন উন্নত বিশ্লেষণ, Automation Hub-এর সঙ্গে একীকরণ) সহ আসে, আর এর জন্য লাইসেন্স কিনতে হয়। সহজ ভাষায়: AWX হলো Controller-এর Open Source "আপস্ট্রিম" সংস্করণ; Controller হলো তার বাণিজ্যিক, সমর্থনসহ সংস্করণ। TechNova-র মতো ছোট টিমের জন্য AWX দিয়ে শুরু করা যুক্তিসংগত; বড় Enterprise, যেখানে বিক্রেতার সমর্থন ও দীর্ঘমেয়াদি স্থিতিশীলতা দরকার, সেখানে Automation Controller উপযুক্ত।

Job Template, Inventory, Credential, Project

AWX/Controller-এ কাজের একক হলো **Job Template**: এটি বলে দেয় কোন Project (Git Repository) থেকে কোন Playbook, কোন Inventory-র ওপর, কোন Credential দিয়ে, কোন অতিরিক্ত Variable সহ চালাতে হবে। **Project** হলো Git Repository-র সংযোগ — Playbook সবসময় Git থেকে টানা হয়, কারো ল্যাপটপ থেকে নয়। **Inventory** AWX-এর ভেতরে রাখা যায়, বা ক্লাউড/CMDB থেকে স্বয়ংক্রিয়ভাবে আনা যায় (Inventory Source)। **Credential** হলো SSH Key, Vault পাসওয়ার্ড, ক্লাউড API টোকেন ইত্যাদির নিরাপদ ভান্ডার — এগুলো এনক্রিপ্ট করে রাখা হয় ও শুধু অনুমোদিত Job-ই ব্যবহার করতে পারে। Job চালালে তৈরি হয় একটি **Job** — যার সম্পূর্ণ লগ, শুরু ও শেষের সময়, এবং কে চালিয়েছে তার হিসাব সংরক্ষিত থাকে।

Workflow Template ও Execution Environment

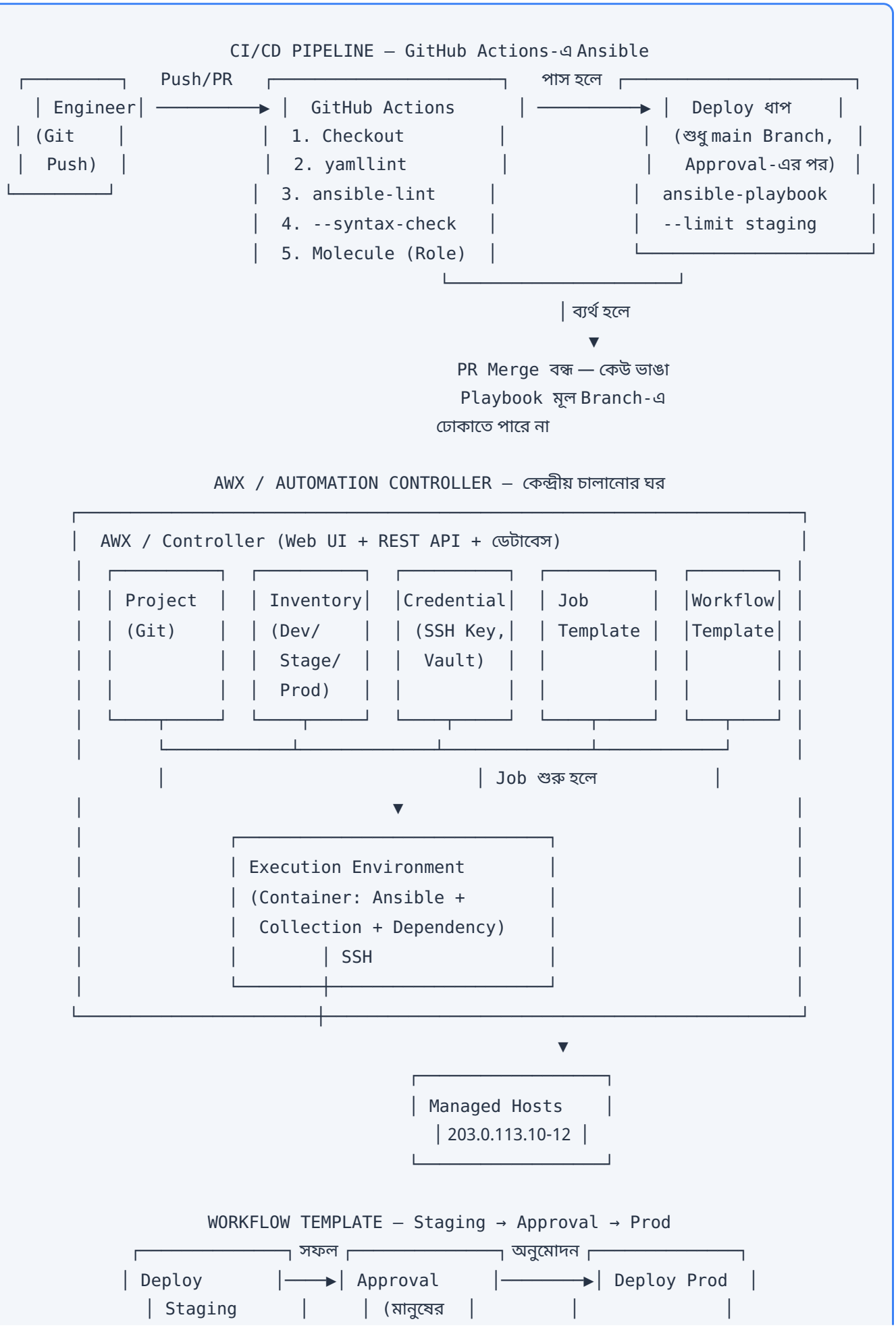
Workflow Template একাধিক Job Template-কে একটি প্রবাহে সাজায়: যেমন প্রথমে Staging-এ Deploy করুন; সফল হলে অনুমোদনের জন্য থামুন (Approval Node); অনুমোদন পেলে Prod-এ Deploy করুন; ব্যর্থ হলে Rollback Job চালান। প্রতিটি Node-এর সাফল্য বা ব্যর্থতার ওপর নির্ভর করে পরের Node ঠিক হয় — এটাই Workflow-কে সাধারণ "Job-এর তালিকা" থেকে আলাদা করে। **Execution Environment (EE)** হলো একটি Container Image যার ভেতরে Ansible, Python, প্রয়োজনীয় Collection ও নির্ভরতা (Dependency) প্যাক করা থাকে। আগে প্রতিটি Job সরাসরি Controller সার্ভারে চলত — ফলে নির্ভরতার সংঘাত হতো; এখন প্রতিটি Job তার নিজের EE Container-এ চলে — ফলে একটি Job-এর নির্ভরতা আরেকটি Job-কে ভাঙতে পারে না, আর একই Job যেকোনো Controller-এ একইভাবে চলে। EE বানানো হয় `ansible-builder` দিয়ে, একটি `execution-environment.yml` ফাইল থেকে।

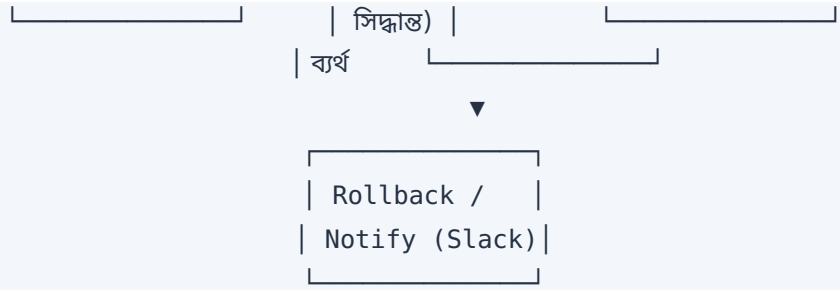
২৩.৬ অভ্যন্তরীণ কার্যপ্রণালী

AWX/Controller-এ একটি Job চালু হলে ভেতরে যা ঘটে: প্রথমে ব্যবহারকারী UI বা API দিয়ে Job Template থেকে একটি Job শুরু করেন — Controller পরীক্ষা করে এই ব্যবহারকারীর অনুমতি আছে কি না (RBAC)। তারপর Controller Project-এর Git Repository থেকে সর্বশেষ কোড টানে (বা ক্যাশ করা কপি ব্যবহার করে), একটি Execution Environment Container শুরু করে, তার ভেতরে প্রয়োজনীয় Credential-গুলো (SSH Key, Vault পাসওয়ার্ড) অস্থায়ীভাবে মাউন্ট করে, এবং `ansible-playbook` চালায়। Playbook-এর প্রতিটি Task-এর আউটপুট সরাসরি (Streaming) Controller-এর ডেটাবেসে লেখা হয় — তাই ব্যবহারকারী ব্রাউজারে লাইভ দেখতে পারেন কী চলছে। Job শেষ হলে Exit Code, পরিসংখ্যান (ok/changed/failed) ও সম্পূর্ণ লগ সংরক্ষিত থাকে, আর Workflow-এর ক্ষেত্রে ফলাফলের ওপর নির্ভর করে পরের Node শুরু হয়। এই নকশার সবচেয়ে বড় সুবিধা: **Control Node আর কোনো ব্যক্তির ল্যাপটপ নয় — Controller নিজেই Control Node**, আর তার সঙ্গে আছে হিসাব (Audit), অনুমতি ও সময়সূচি।

> **WARNING:** AWX-এর ডিফল্ট ইনস্টলেশনে ডেটাবেস ও Secret-গুলো একই সার্ভারে থাকে — প্রোডাকশনে AWX চালালে তার নিজের ব্যাকআপ ও নিরাপত্তা (TLS, শক্তিশালী Admin পাসওয়ার্ড, নেটওয়ার্ক সীমাবদ্ধতা) নিশ্চিত করুন। AWX নিজেই যদি অনিরাপদ হয়, তাহলে তার হাতে থাকা সব Credential — অর্থাৎ আপনার সব সার্ভারের চাবি — ঝুঁকিতে পড়ে। AWX-কে "আরেকটি সার্ভার" নয়, "সব সার্ভারের চাবির আলমারি" হিসেবে দেখুন।

২৩.৭ আর্কিটেকচার চিত্র


CI/CD PIPELINE — GitHub Actions-এ Ansible
Engineer (Git Push)
Push/PR
GitHub Actions
1. Checkout
2. yamllint
3. ansible-lint
4. --syntax-check
5. Molecule (Role)
পাস হলে
Deploy ধাপ (শুধু main Branch, Approval-এর পর)
ansible-playbook --limit staging
ব্যর্থ হলে
PR Merge বন্ধ — কেউ ভাঙা Playbook মূল Branch-এ ঢোকাতে পারে না
AWX / AUTOMATION CONTROLLER — কেন্দ্রীয় চালানোর ঘর
AWX / Controller (Web UI + REST API + ডেটাবেস)
Project (Git)
Inventory (Dev/ Stage/ Prod)
Credential (SSH Key, Vault)
Job Template
Workflow Template
Job শুরু হলে
Execution Environment (Container: Ansible + Collection + Dependency)
SSH
Managed Hosts
203.0.113.10-12
WORKFLOW TEMPLATE — Staging → Approval → Prod
Deploy Staging
সফল
Approval (মানুষের
অনুমোদন
Deploy Prod



চিত্রের মূল কথা: CI পরীক্ষা করে, Controller চালায়, আর Workflow সিদ্ধান্ত নেয় — তিনটি ভিন্ন দায়িত্ব, তিনটি ভিন্ন স্তর।

২৩.৮ বাস্তব উদাহরণ

TechNova-র GitHub Repository-তে Push করলেই যাতে Lint ও Syntax Check চলে, আর `main` Branch-এ Merge হলে Staging-এ Deploy হয় — সেই Pipeline-এর কমান্ড-স্তরের উদাহরণ:

```
# GitHub Actions Runner-এ হাতে পরীক্ষা (স্থানীয়ভাবে একই ধাপগুলো চালানো যায়)
# ১. Lint ধাপ
ansible-lint playbooks/ roles/
# ২. Syntax ধাপ
ansible-playbook playbooks/site.yml --syntax-check
# ৩. Dry Run ধাপ (Staging Inventory-র বিরুদ্ধে, কিছু না বদলে)
ansible-playbook playbooks/site.yml --check --diff -i inventories/staging/hosts.ini
# ৪. Vault পাসওয়ার্ড GitHub Secret থেকে অস্থায়ী ফাইলে লেখা (Workflow-এর ভেতরে)
echo "${{ secrets.VAULT_PASSWORD }}" > /tmp/vault-pass.txt
chmod 600 /tmp/vault-pass.txt
ansible-playbook playbooks/site.yml --vault-password-file /tmp/vault-pass.txt \
  -i inventories/staging/hosts.ini --limit staging
rm -f /tmp/vault-pass.txt
# ৫. AWX-এ Job চালু করা — API দিয়ে (টোকেন GitHub Secret-এ)
curl -s -k -H "Authorization: Bearer ${{ secrets.AWX_TOKEN }}" \
  -H "Content-Type: application/json" \
  -X POST https://awx.technova.example/api/v2/job_templates/7/launch/ \
  -d '{"extra_vars": {"app_version": "1.4.2"}}'
# চিত্রণমূলক আউটপুট: {"job": 128, "status": "pending", "url": "/api/v2/jobs/128/"}
```

এই উদাহরণের শিক্ষা: Pipeline-এর প্রতিটি ধাপ আগের অধ্যায়ের টুলগুলোই ব্যবহার করে — CI নতুন কোনো জাদু নয়, একই পরীক্ষাগুলোকে স্বয়ংক্রিয়ভাবে, প্রতিবার, একইভাবে চালানো। আর AWX-এর API দিয়ে Job চালু করা মানে Deploy-কে অন্য সিস্টেম (যেমন একটি Chatbot বা একটি Release টুল) থেকেও ট্রিগার করা যায়।

২৩.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচে TechNova-র সম্পূর্ণ GitHub Actions Workflow ফাইল ও একটি Execution Environment সংজ্ঞা দেওয়া হলো:

```
# ── ফাইল: .github/workflows/ansible.yml ──────────────────────────────
---
name: Ansible CI/CD

on:
  push:
    branches: [main, develop]
```

```
  pull_request:
    branches: [main]

jobs:
  lint-and-test:
    name: Lint and syntax check
    runs-on: ubuntu-24.04
    steps:
      - name: Check out code
        uses: actions/checkout@v4

      - name: Set up Python 3.12
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install Ansible and linters
        run: |
          python -m pip install --upgrade pip
          pip install ansible-core==2.17.* ansible-lint yamllint

      - name: Run yamllint
        run: yamllint playbooks/ roles/ inventories/

      - name: Run ansible-lint
        run: ansible-lint playbooks/ roles/

      - name: Syntax check all playbooks
        run: |
          for playbook in playbooks/*.yml; do
            ansible-playbook "$playbook" --syntax-check
          done

  deploy-staging:
    name: Deploy to staging
    needs: lint-and-test
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-24.04
    environment:
      name: staging
      url: https://staging.technova.example
    steps:
      - name: Check out code
        uses: actions/checkout@v4

      - name: Set up Python 3.12
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install Ansible
        run: pip install ansible-core==2.17.*

      - name: Write SSH key from secret
        run: |
          mkdir -p ~/.ssh
          echo "${{ secrets.STAGING_SSH_KEY }}" > ~/.ssh/id_ed25519
          chmod 600 ~/.ssh/id_ed25519
          ssh-keyscan -H 203.0.113.11 >> ~/.ssh/known_hosts
```

```
      - name: Write Vault password from secret
        run: |
          echo "${{ secrets.VAULT_PASSWORD }}" > /tmp/vault-pass.txt
          chmod 600 /tmp/vault-pass.txt

      - name: Deploy to staging
        run: |
          ansible-playbook playbooks/site.yml \
            -i inventories/staging/hosts.ini \
            --vault-password-file /tmp/vault-pass.txt

      - name: Clean up secrets
        if: always()
        run: |
          rm -f /tmp/vault-pass.txt ~/.ssh/id_ed25519

# ── ফাইল: execution-environment.yml (ansible-builder-এর জন্য) ──────────
---
version: 3
images:
  base_image:
    name: quay.io/centos/centos:stream9
dependencies:
  galaxy:
    collections:
      - name: community.general
      - name: ansible.posix
  python:
    - boto3
    - requests
  system:
    - openssh-clients
additional_build_steps:
  append_base:
    - RUN microdnf install -y git && microdnf clean all

# ── ফাইল: inventories/staging/hosts.ini ──────────────────────────────
[webservers]
staging-web.technova.example ansible_host=203.0.113.11

[staging:children]
webservers
```

Workflow ফাইলটি পড়ার সময় লক্ষ্য করুন: `lint-and-test` Job প্রতিটি Push ও PR-এ চলে, কিন্তু `deploy-staging` Job শুধু `main` Branch-এ Push হলে চলে (`if: github.ref == 'refs/heads/main'`), আর `environment: staging` মানে GitHub-এ এই Environment-এ Approval সেট করা থাকলে মানুষের অনুমোদন ছাড়া Deploy শুরু হবে না। `if: always()` দিয়ে Cleanup ধাপটি নিশ্চিত করা হয়েছে — Deploy সফল হোক বা ব্যর্থ, Secret ফাইল মুছে যাবেই।

২৩.১০ ধাপে ধাপে বাস্তবায়ন

ধাপ ১ — Repository গোছান। Playbook, Role, Inventory (প্রতি পরিবেশে আলাদা ডিরেক্টরি) ও Workflow ফাইল একই Git Repository-তে রাখুন; Secret কখনো Commit করবেন না — `.gitignore`-এ `*.vault-pass`, `*.pem` ও `group_vars/*/vault.yml` (এনক্রিপ্ট না করা অবস্থায়) যোগ করুন।

ধাপ ২ — GitHub Secrets সেট করুন। Repository Settings → Secrets-এ `STAGING_SSH_KEY` (Private Key) ও `VAULT_PASSWORD` যোগ করুন; Production-এর Secret আলাদা Environment Secret হিসেবে রাখুন।

ধাপ ৩ — Workflow ফাইল যোগ করুন। `.github/workflows/ansible.yml` তৈরি করে একটি পরীক্ষামূলক Branch-এ Push করুন; Actions ট্যাবে Lint ধাপগুলো পাস হচ্ছে কি না দেখুন।

ধাপ ৪ — Staging Deploy পরীক্ষা করুন। `main`-এ Merge করে Staging Deploy চলছে কি না দেখুন; লগে `PLAY RECAP` পরীক্ষা করুন।

ধাপ ৫ — AWX ইনস্টল করুন (ল্যাবে)। AWX Operator দিয়ে Kubernetes-এ বা Docker-ভিত্তিক ল্যাব ইনস্টলে AWX চালু করুন; প্রথমে Admin পাসওয়ার্ড বদলান ও TLS সেট করুন।

ধাপ ৬ — AWX-এ মূল উপাদানগুলো তৈরি করুন। Project (Git Repository-র URL), Inventory (Dev/Staging/Prod আলাদা), Credential (SSH Key + Vault পাসওয়ার্ড), তারপর Job Template (Playbook + Inventory + Credential একত্র করে) তৈরি করুন।

ধাপ ৭ — Job Template হাতে চালিয়ে যাচাই করুন। প্রথমে Staging Inventory-তে Job চালান; লগ দেখে নিশ্চিত হোন। তারপর Schedule সেট করুন (যেমন প্রতি রাতে Drift Check)।

ধাপ ৮ — Workflow Template বানান। Staging Deploy → Approval Node → Prod Deploy — এই তিনটি Node দিয়ে একটি Workflow তৈরি করুন; Approval Node-এ দায়িত্বপ্রাপ্ত ব্যক্তিকে নির্ধারণ করুন।

২৩.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

GitHub Actions-এ সফল Pipeline-এর আউটপুট (চিত্রণমূলক আউটপুট):

```
Ansible CI/CD / Lint and syntax check
✓ Check out code
✓ Set up Python 3.12
✓ Install Ansible and linters
✓ Run yamllint
✓ Run ansible-lint
✓ Syntax check all playbooks
→ All checks passed in 1m 12s

Ansible CI/CD / Deploy to staging
✓ Deploy to staging
  PLAY RECAP: staging-web.technova.example : ok=12 changed=2 unreachable=0 failed=0
→ Deployment to staging completed
```

AWX-এ সফল Job-এর চিহ্ন: Job-এর Status "Successful" (সবুজ), প্রতিটি Task-এর পাশে ok/changed চিহ্ন, আর Job Details-এ চালনাকারীর নাম, শুরুর সময় ও সময়কাল। Workflow-এর ক্ষেত্রে প্রতিটি Node সবুজ হলে Workflow সফল; Approval Node-এ অপেক্ষমাণ (Pending) অবস্থা মানে মানুষের সিদ্ধান্তের অপেক্ষায় আছে — এটি ব্যর্থতা নয়, নকশার অংশ। কোনো Job ব্যর্থ হলে AWX-এর লগে ঠিক কোন Task-এ, কোন Host-এ, কী বার্তাসহ ব্যর্থ হয়েছে তা দেখা যায় — অধ্যায় ২২-এর Troubleshooting পদ্ধতি এখানেও প্রযোজ্য।

২৩.১২ সাধারণ ভুল

ভুল	কেন সমস্যা	সঠিক অভ্যাস

Secret Workflow ফাইলে বা কোডে লেখা	Git ইতিহাসে চিরকাল থেকে যায়; Repository ফাঁস হলেই সব ফাঁস	GitHub Encrypted Secrets ব্যবহার করুন; Secret কখনো Commit নয়
PR থেকেও Deploy ধাপ চালানো	পরীক্ষামূলক কোড প্রোডাকশনে চলে যেতে পারে	Deploy ধাপে Branch ও Environment শর্ত দিন
AWX-কে ansible-core-এর বিকল্প ভাবা	AWX ansible-core-এর ওপরে চলে; Core ছাড়া AWX-এর কোনো অর্থ নেই	তিনটি স্তর আলাদা রাখুন: Core (ইঞ্জিন), AWX (Open Source UI), Controller (বাণিজ্যিক)
সব Job-এ একই Credential ব্যবহার করা	একটি Credential ফাঁস হলে সব পরিবেশ ঝুঁকিতে	প্রতি পরিবেশে আলাদা Credential; সর্বনিম্ন অনুমতি (Least Privilege)
Execution Environment-এ সব Collection ঢোকানো	Image বিশাল হয়, Job শুরু ধীর হয়, নির্ভরতার সংঘাত বাড়ে	শুধু প্রয়োজনীয় Collection ও Python প্যাকেজ রাখুন
Workflow-এ Approval Node না রাখা	প্রোডাকশন Deploy সম্পূর্ণ স্বয়ংক্রিয় — ভুল কোডও থামবে না	Prod-এর আগে সবসময় মানুষের অনুমোদন রাখুন
CI-র লগে Secret ছাপা (set -x, debug)	লগে Secret থেকে যায়; লগ দেখতে পায় এমন যে কেউ Secret পায়	Secret-ধারী Task-এ no_log: true; লগে Secret মাস্ক করা আছে কি না যাচাই করুন

২৩.১৩ সমস্যা সমাধান (Troubleshooting)

CI-তে Lint পাস করে, স্থানীয় মেশিনে করে না (বা উল্টো)। কারণ সাধারণত সংস্করণের পার্থক্য: CI-তে ansible-lint-এর সংস্করণ আলাদা। সমাধান: `requirements.txt` বা Workflow ফাইলে সংস্করণ পিন করুন (`ansible-lint==24.6.0`), আর স্থানীয়ভাবেও একই সংস্করণ ব্যবহার করুন।

CI-তে Playbook ব্যর্থ, কিন্তু স্থানীয়ভাবে চলে। প্রথমে পরীক্ষা করুন: CI Runner থেকে Target Host-এ SSH পৌঁছায় কি না (Runner-এর IP ফায়ারওয়ালে অনুমোদিত কি না); Secret-গুলো ঠিকমতো ফাইলে লেখা হয়েছে কি না (অনুমতি 600 কি না); Inventory-র Host ঠিক আছে কি না। CI-র লগে `-vvv` যোগ করে SSH-এর বিস্তারিত দেখুন — তবে লগে Secret না থাকে সেদিকে খেয়াল রাখুন।

AWX Job "Pending"-এ আটকে আছে। সাধারণত Execution Node বা Container চালু হতে পারছে না, বা Inventory Source আপডেট আটকে আছে। AWX-এর Job Details-এ দেখুন কোন ধাপে আটকে আছে; AWX Operator-এর লগ ও Kubernetes Pod-এর অবস্থা পরীক্ষা করুন।

AWX-এ Playbook পাওয়া যাচ্ছে না। Project Sync ব্যর্থ হয়েছে — Git Repository-র URL, Credential বা Branch ভুল হতে পারে। Project-এর Sync লগ দেখুন; Repository-তে Playbook-এর পাথ Job Template-এর সঙ্গে মিলছে কি না যাচাই করুন।

Workflow Approval Node-এ কেউ অনুমোদন দিতে পারছে না। RBAC পরীক্ষা করুন: অনুমোদনকারী ব্যবহারকারীর Workflow-এ Approval-এর অনুমতি আছে কি না, এবং তিনি সঠিক Organization ও Team-এ আছেন কি না।

> **TIP:** CI/CD সমস্যার ক্ষেত্রে "কোথায় ভিন্নতা" প্রশ্নটি করুন: স্থানীয় মেশিন বনাম CI Runner, Staging বনাম Prod, গতকালের সফল চালনা বনাম আজকের ব্যর্থ চালনা — পার্থক্যটি খুঁজলেই কারণ মেলে। Git-এর ইতিহাস দেখুন: শেষ সফল চালনার পর কোন ফাইলটি বদলেছে? বেশিরভাগ CI সমস্যার উত্তর সেখানেই থাকে।

২৩.১৪ নিরাপত্তা বিবেচনা

CI/CD ও AWX-এর নিরাপত্তা মানে Secret-এর জীবনচক্র নিয়ন্ত্রণ করা। প্রথমত, **Secret-এর পরিধি সীমিত রাখুন**: Staging-এর SSH Key দিয়ে যাতে Prod-এ ঢোকা না যায়, Prod-এর Vault পাসওয়ার্ড যাতে Staging Job-এ না থাকে। দ্বিতীয়ত, **Secret ঘোরান (Rotate)**: নিয়মিত বিরতিতে SSH Key ও Vault পাসওয়ার্ড বদলান, আর পুরনো Key বাতিল করুন। তৃতীয়ত, **Pull Request থেকে Secret ব্যবহার বন্ধ করুন**: Fork করা Repository থেকে আসা PR-এ Secret পাওয়া যায় না — এটি GitHub-এর নিরাপত্তা ব্যবস্থা, একে এড়ানোর চেষ্টা করবেন না; Fork থেকে আসা কোড বিশ্বাসযোগ্য নয় বলে ধরে নেওয়া হয়। চতুর্থত, **AWX-এ RBAC ঠিকমতো সেট করুন**: কে Job চালাতে পারবে, কে Template বদলাতে পারবে, কে Credential দেখতে পারবে — এই তিনটি আলাদা অনুমতি। পঞ্চমত, **AWX-এর API Token-এর মেয়াদ ও পরিধি সীমিত রাখুন** — Personal Access Token তৈরি করে চিরকালের জন্য রেখে দেবেন না।

২৩.১৫ প্রোডাকশন ইঞ্জিনিয়ারিং অনুশীলন

প্রোডাকশনে CI/CD ও AWX ব্যবহারের পেশাদার নিয়মগুলো: **Git-ই একমাত্র সত্যের উৎস (Single Source of Truth)** — Playbook, Inventory (Secret ছাড়া), Workflow — সবকিছু Git-এ; AWX-এ হাতে কিছু বদলানো যাবে না, বদলাতে হলে Git-এ বদলে Project Sync করতে হবে। **পরিবেশের পদোন্নতি (Promotion) একদিকে** — Dev → Staging → Prod, কখনো উল্টোদিকে নয়; Prod-এর কনফিগারেশন Staging-এ পরীক্ষা না করে যাবে না। **প্রতিটি Deploy-এর আগে --check --diff** — এমনকি AWX থেকেও, Workflow-এর প্রথম Node হিসেবে একটি Dry Run Job রাখুন। **Job-এর লগ সংরক্ষণ করুন** — AWX-এর Job ইতিহাসই আপনার Audit Trail; নিয়মিত ব্যাকআপ নিন। **Execution Environment-এর সংস্করণ পিন করুন** — EE Image-এর Tag হিসেবে `latest` ব্যবহার করবেন না; নির্দিষ্ট সংস্করণ ব্যবহার করুন, যাতে আজকের Job আর ছয় মাস পরের Job একইভাবে চলে।

২৩.১৬ অধ্যায়ের সারাংশ

এই অধ্যায়ে আপনি শিখলেন CI/CD Pipeline-এ Ansible চালানো — GitHub Actions-এ Lint, Syntax Check, Molecule ও শর্তসাপেক্ষ Deploy — এবং CI-তে Secret নিরাপদে সামলানোর নিয়ম। আপনি শিখলেন ansible-core (কমান্ড-লাইন ইঞ্জিন), AWX (Open Source ওয়েব প্ল্যাটফর্ম) ও Ansible Automation Controller (AWX-এর ওপর ভিত্তি করে তৈরি বাণিজ্যিক পণ্য, Red Hat-এর সমর্থনসহ)-এর পার্থক্য। আপনি দেখলেন Job Template, Project, Inventory ও Credential কীভাবে একসঙ্গে কাজ করে, Workflow Template কীভাবে Staging → Approval → Prod প্রবাহ সাজায়, আর Execution Environment কীভাবে প্রতিটি Job-কে একটি পরিষ্কার, পুনরুৎপাদনযোগ্য Container-এ চালায়। মূল বার্তা: স্বয়ংক্রিয়তা মানে নিয়ন্ত্রণ ছেড়ে দেওয়া নয় — পরীক্ষা, অনুমতি ও হিসাবসহ স্বয়ংক্রিয়তা।

২৩.১৭ জ্ঞান যাচাই

১. AWX, Ansible Automation Controller ও ansible-core-এর মধ্যে পার্থক্য কী? কোনটি Open Source আর কোনটি বাণিজ্যিক?
২. GitHub Actions Workflow-এ Deploy ধাপটি কেন শুধু `main` Branch-এ ও Approval-এর পর চলা উচিত?
৩. CI-তে Vault পাসওয়ার্ড ও SSH Private Key কীভাবে নিরাপদে ব্যবহার করবেন? দুটি নিয়ম লিখুন।
৪. Job Template কী? এটি Project, Inventory ও Credential-কে কীভাবে একত্র করে?
৫. Execution Environment (EE) কী সমস্যার সমাধান করে? EE ছাড়া Job চালালে কী অসুবিধা হতো?

২৩.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

নিজের GitHub অ্যাকাউন্টে একটি Repository তৈরি করুন যাতে একটি ছোট Ansible Playbook (Nginx ইনস্টল) ও এই অধ্যায়ের Workflow ফাইল থাকে। (ক) Workflow-টি এমনভাবে সাজান যাতে প্রতিটি Push-এ yamllint, ansible-lint ও --syntax-check চলে; (খ) ইচ্ছাকৃতভাবে একটি Lint ভুল ঢুকিয়ে দেখুন Pipeline ব্যর্থ হয় কি না, তারপর ঠিক করুন; (গ) একটি Staging Environment-এর Approval সেট করে Deploy ধাপটি পরীক্ষা করুন (আসল সার্ভার না থাকলে --check মোডে Deploy ধাপটি চালান); (ঘ) একটি নকশা-নথি লিখুন: TechNova-র জন্য AWX-এ কোন Project, Inventory, Credential, Job Template ও Workflow Template লাগবে — নামসহ তালিকা করুন ও Approval Node কোথায় রাখবেন তা যুক্তিসহ লিখুন।

অধ্যায় ২৪ • PART V — Enterprise Governance & Production Operations

Chapter 24 — Enterprise Configuration Management, Governance, and Production Operations

২৪.১ ভূমিকা

একটি সার্ভারে Nginx ইনস্টল করা শেখা আর একটি Enterprise-এর শত শত সার্ভার বছরের পর বছর ধরে সঠিক অবস্থায় রাখা — এই দুটি সম্পূর্ণ ভিন্ন কাজ। প্রথমটি একটি ঘটনা (Event), দ্বিতীয়টি একটি শৃঙ্খলা (Discipline)। TechNova যখন ছোট ছিল, তখন একজন Engineer মনে রাখতে পারতেন কোন সার্ভারে কী আছে। এখন TechNova-র Dev, Staging ও Production — তিনটি পরিবেশে কয়েক ডজন সার্ভার, প্রতিদিন পরিবর্তন, আর প্রতিটি পরিবর্তনের জন্য দায়িত্বশীল কেউ থাকতে হয়। এই অধ্যায় Enterprise-স্তরের Configuration Management নিয়ে: কীভাবে একাধিক পরিবেশের Inventory সাজাবেন, কীভাবে পরিবর্তনের আগে অনুমোদন (Change Approval Gate) নেবেন, কীভাবে Drift — অর্থাৎ সার্ভারের আসল অবস্থা ও কাঙ্ক্ষিত অবস্থার মধ্যে পার্থক্য — শনাক্ত করবেন, কীভাবে Playbook-এর Version Control, Code Review ও Rollback করবেন, কীভাবে Audit Log ও Runbook রাখবেন, এবং কীভাবে Governance ও Compliance-এর মৌলিক চাহিদা মেটাবেন। এই অধ্যায়ের শেষে আপনি শুধু Ansible চালাতে পারবেন না — আপনি একটি সংস্থার Configuration-এর দায়িত্ব নিতে পারবেন।

২৪.২ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Dev, Staging ও Prod-এর জন্য বহু-পরিবেশ (Multi-Environment) Inventory নকশা করতে; (২) Change Approval Gate-এর ধারণা ব্যাখ্যা করতে ও কোথায় বসাতে হয় বলতে; (৩) নির্ধারিত সময়ে `--check` চালিয়ে Drift শনাক্ত করার পদ্ধতি বোঝাতে; (৪) Playbook-এর Version Control, Branching ও Code Review প্রক্রিয়া বর্ণনা করতে; (৫) Rollback কৌশল (পূর্ববর্তী সংস্করণে ফেরা, Snapshot, Blue-Green ধারণা) ব্যাখ্যা করতে; (৬) Audit Logging-এর জন্য কী কী সংরক্ষণ করতে হয় বলতে; (৭) Runbook লেখার কাঠামো বোঝাতে; (৮) Governance ও Compliance-এর মৌলিক নীতিগুলো (Least Privilege, Separation of Duties, প্রমাণ সংরক্ষণ) ব্যাখ্যা করতে।

২৪.৩ পূর্বশর্ত ধারণা

এই অধ্যায়ের জন্য আপনার জানা থাকা দরকার: Inventory ও Group Variable-এর ব্যবহার, Ansible Vault, অধ্যায় ২২-এর পরীক্ষা ও অধ্যায় ২৩-এর CI/CD ও AWX-এর ধারণা, Git-এর Branching ও Pull Request প্রক্রিয়া, এবং Linux সার্ভিস ও ফাইল অনুমতির মৌলিক ধারণা। Enterprise Governance-এর পূর্ব অভিজ্ঞতা প্রয়োজন নেই — প্রতিটি ধারণা এখানে TechNova-র উদাহরণ দিয়ে প্রথম থেকে ব্যাখ্যা করা হয়েছে।

২৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

Enterprise Configuration Management-এর মূলে একটি সহজ সত্য আছে: **সার্ভার সময়ের সঙ্গে সঙ্গে তার কাঙ্ক্ষিত অবস্থা থেকে সরে যায়।** কেউ হাতে একটি ফাইল বদলে দেয়, একটি প্যাকেজ আপডেট হয়ে যায়, একটি সার্ভিস বন্ধ হয়ে যায় — একে বলে **Configuration Drift**। Drift-কে না ধরলে একদিন দেখা যাবে Staging ও Prod আর একরকম নেই, আর Prod-এ যে কনফিগারেশন চলছে তা কেউ জানে না। এর বিপরীতে দাঁড়িয়ে আছে **Desired State** (কাঙ্ক্ষিত অবস্থা) ধারণা: Ansible Playbook-এ লেখা থাকে সার্ভার কেমন হওয়া উচিত; Ansible-এর কাজ সেই অবস্থা বারবার প্রয়োগ করা, যাতে Drift নিজে নিজে ঠিক হয়ে যায়। Governance মানে এই প্রক্রিয়ার ওপর নিয়ন্ত্রণ: কে বদলাতে পারবে, কে অনুমোদন

দেবে, কী বদলানো হয়েছে তার প্রমাণ কোথায় থাকবে, আর ভুল হলে কীভাবে ফেরা যাবে। এই চারটি প্রশ্ন — কে, কী, প্রমাণ, ফেরার পথ — Enterprise-এর সব নিয়মের মূল।

> **CONCEPT:** Configuration Management-এর তিনটি স্তম্ভ: (১) **ঘোষণামূলক অবস্থা (Declarative State)** — Playbook-এ লেখা থাকে "কী হওয়া উচিত", ধাপে ধাপে "কী করতে হবে" নয়; (২) **নিয়মিত সমন্বয় (Continuous Reconciliation)** — নির্ধারিত সময়ে Playbook চালিয়ে Drift ঠিক করা; (৩) **হিসাবযোগ্যতা (Accountability)** — প্রতিটি পরিবর্তনের জন্য কে, কখন, কেন — তার প্রমাণ। এই তিনটির যেকোনো একটি না থাকলে ব্যবস্থাটি Enterprise-স্তরের নয়, শুধু "সুবিধাজনক স্ক্রিপ্ট"।

Separation of Duties (দায়িত্বের বিভাজন) ধারণাটিও মনে রাখুন: যে ব্যক্তি পরিবর্তন লেখে, সে একা তা অনুমোদন করতে পারবে না; যে Prod-এ Deploy করে, সে Prod-এর Secret দেখতে পারবে না। এই বিভাজন কোনো আমলাতন্ত্র নয় — এটি সেই নিরাপত্তা-জাল, যা একজন ভালো Engineer-এর ভুল বা একজন অসৎ ব্যক্তির অপব্যবহার — দুটোকেই আটকায়।

২৪.৫ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

বহু-পরিবেশ Inventory

Enterprise-এ প্রতি পরিবেশের জন্য আলাদা Inventory ডিরেক্টরি রাখা হয়: `inventories/dev/`, `inventories/staging/`, `inventories/prod/` — প্রতিটিতে নিজের `hosts.ini` (বা YAML) ও `group_vars/` থাকে। সাধারণ সেটিং (যেমন NTP সার্ভার, Timezone) থাকে `group_vars/all/`-এ; পরিবেশ-নির্দিষ্ট সেটিং (যেমন Prod-এ বেশি Worker Process, Dev-এ Debug মোড) থাকে সংশ্লিষ্ট পরিবেশের `group_vars/`-এ। এই নকশার সুবিধা: একই Playbook তিন পরিবেশেই চলে, শুধু Inventory বদলায় — ফলে "Staging-এ চলে, Prod-এ চলে না" সমস্যা কমে। আর সবচেয়ে গুরুত্বপূর্ণ নিরাপত্তা নিয়ম: **Prod Inventory-তে শুধু অনুমোদিত ব্যক্তিরা Playbook চালাতে পারবেন** — এটি AWX-এর RBAC বা CI-র Environment Protection দিয়ে নিয়ন্ত্রণ করা হয়।

Change Approval Gate

Approval Gate মানে পরিবর্তনের পথে একটি থামার জায়গা, যেখানে মানুষ সিদ্ধান্ত নেয়। TechNova-তে তিনটি Gate আছে: (১) **Code Review Gate** — Pull Request-এ অন্তত একজন সহকর্মী Playbook-এর পরিবর্তন পর্যালোচনা করবেন; (২) **Staging Gate** — Prod-এ যাওয়ার আগে পরিবর্তনটি Staging-এ চালিয়ে প্রমাণ করতে হবে যে তা কাজ করে; (৩) **Prod Approval Gate** — Prod Deploy-এর আগে দায়িত্বপ্রাপ্ত ব্যক্তি (যেমন Team Lead) অনুমোদন দেবেন — AWX Workflow-এর Approval Node বা GitHub Environment-এর Required Reviewer দিয়ে এটি কার্যকর করা হয়। জরুরি অবস্থার (Incident) জন্য একটি **Emergency Change** পথও থাকে — তবে সেটিও লিপিবদ্ধ হয় ও পরে পর্যালোচনা করা হয়; "জরুরি" মানে "হিসাব ছাড়া" নয়।

Drift Detection

Drift ধরার সবচেয়ে সহজ উপায়: নির্ধারিত সময়ে (যেমন প্রতি রাতে) প্রতিটি পরিবেশের বিরুদ্ধে Playbook-টি `--check` মোডে চালানো। `changed`-এর সংখ্যা শূন্যের বেশি হলে বুঝতে হবে সার্ভারের আসল অবস্থা কাঙ্ক্ষিত অবস্থা থেকে সরে গেছে — কেউ হাতে কিছু বদলেছে, বা কোনো প্যাকেজ নিজে থেকে আপডেট হয়েছে। এই রিপোর্টটি একটি চ্যানেলে (যেমন ইমেইল বা Chat) পাঠানো হয়, আর Drift পাওয়া গেলে দুটি উপায়: হয় Playbook আবার চালিয়ে Drift ঠিক করা (Reconciliation), নয়তো Drift-টি যদি ইচ্ছাকৃত হয় তাহলে Playbook-এ সেই পরিবর্তনটি ঢুকিয়ে দেওয়া — যাতে কাঙ্ক্ষিত অবস্থা ও আসল অবস্থা আবার এক হয়। মনে রাখবেন: Drift-কে উপেক্ষা করে Playbook চালিয়ে যাওয়া মানে প্রতিটি চালনায় অপ্রত্যাশিত পরিবর্তন — আর সেটিই একদিন বড় দুর্ঘটনার কারণ হয়।

Version Control, Code Review ও Rollback

Playbook-এর Version Control মানে: প্রতিটি পরিবর্তন Git Commit, প্রতিটি Commit-এর বার্তায় কেন বদলানো হয়েছে তার কারণ, আর প্রতিটি Release-এ একটি Tag (যেমন `v1.4.2`) — যাতে যেকোনো সময় জানা যায় Prod-এ কোন সংস্করণ চলছে। Code Review মানে: Pull Request-এ পরিবর্তনের Diff দেখে সহকর্মী যাচাই করবেন — শুধু Syntax নয়, যুক্তি: এই পরিবর্তনটি কি সব পরিবেশে নিরাপদ? কোনো Secret কি Diff-এ দেখা যাচ্ছে? Rollback মানে: ভুল হলে আগের অবস্থায় ফেরা। Ansible-এ Rollback-এর তিনটি উপায়: (১) **আগের Tag-এর Playbook চালানো** — যেহেতু Playbook Declarative, আগের সংস্করণ চালালেই আগের অবস্থা ফিরে আসে; (২) **ব্যাকআপ থেকে ফেরা** — গুরুত্বপূর্ণ ফাইল বদলানোর আগে তার ব্যাকআপ নেওয়া (যেমন `copy` মডিউলের `backup: true`); (৩) **Snapshot/Blue-Green** — নতুন কনফিগারেশন প্রথমে নতুন সার্ভারে চালিয়ে, ঠিক থাকলে ট্রাফিক সরানো। কোনো Rollback কৌশলই পরীক্ষা ছাড়া বিশ্বাস করবেন না — Staging-এ Rollback অনুশীলন করুন।

Audit Logging, Runbook, Governance ও Compliance

Audit Log মানে প্রতিটি পরিবর্তনের প্রমাণ: কে, কখন, কোন Playbook-এর কোন সংস্করণ, কোন Inventory-তে, কী ফলাফল — AWX-এর Job ইতিহাস, CI-র লগ ও Git-এর ইতিহাস মিলে এই প্রমাণ তৈরি করে; এগুলো সংরক্ষণ করতে হয় একটি নির্দিষ্ট মেয়াদ পর্যন্ত (যেমন এক বছর)। **Runbook** মানে একটি ঘটনার জন্য ধাপে ধাপে লেখা নির্দেশনা: "Nginx বন্ধ হয়ে গেলে কী করবেন", "ডিস্ক ভরে গেলে কী করবেন" — Runbook-এ থাকে লক্ষণ, নির্ণয়ের কমান্ড, ঠিক করার ধাপ, কখন Escalate করতে হবে, আর কার সঙ্গে যোগাযোগ করতে হবে। Runbook Ansible Playbook-এর বিকল্প নয় — Playbook স্বয়ংক্রিয় করে, Runbook মানুষকে পথ দেখায় যখন স্বয়ংক্রিয়তা যথেষ্ট নয়। **Governance ও Compliance**-এর মৌলিক নীতি: Least Privilege (যার যতটুকু দরকার, ততটুকু অনুমতি), Separation of Duties (লেখক ও অনুমোদনকারী আলাদা), প্রমাণ সংরক্ষণ (Audit Trail), আর নিয়মিত পর্যালোচনা (কে কী অনুমতি পেয়েছে তা নিয়মিত দেখা)। কোনো নিয়ন্ত্রক সংস্থা (Regulator) বা গ্রাহক যখন জানতে চায় "আপনাদের সার্ভার কে বদলাতে পারে, আর কী বদলানো হয়েছে তার প্রমাণ কোথায়?" — এই ব্যবস্থাগুলোই তার উত্তর।

২৪.৬ অভ্যন্তরীণ কার্যপ্রণালী

একটি Enterprise পরিবর্তন যেভাবে প্রবাহিত হয়: একজন Engineer একটি Branch-এ Playbook বদলান → Pull Request খোলেন → CI স্বয়ংক্রিয়ভাবে Lint, Syntax Check ও Molecule চালায় → সহকর্মী Code Review করেন → Merge হলে Staging-এ স্বয়ংক্রিয় Deploy হয় → Staging-এ পরীক্ষা পাস হলে Prod Deploy-এর জন্য Approval চাওয়া হয় → অনুমোদন পেলে AWX Workflow Prod-এ Deploy চালায় → প্রতিটি ধাপের লগ Audit Trail-এ জমা হয় → রাতে নির্ধারিত Drift Check চলে ও রিপোর্ট পাঠায়। এই প্রবাহের প্রতিটি ধাপের একটি উদ্দেশ্য আছে: CI ধরে Syntax ও মানের ভুল, Review ধরে যুক্তির ভুল, Staging ধরে পরিবেশগত ভুল, Approval ধরে সিদ্ধান্তের ভুল, আর Audit Trail ধরে — ভুল হলেও যাতে জানা যায় কী হয়েছিল। কোনো ধাপ এড়িয়ে গেলে সেই ধরনের ভুল আর ধরা পড়ে না। এই প্রবাহটিকে বলে **Change Pipeline** — আর এটি যত স্বয়ংক্রিয়, তত নির্ভরযোগ্য, কারণ মানুষের মনোযোগ ক্লান্ত হয়, যন্ত্রের হয় না।

> **WARNING:** "জরুরি পরিবর্তন" (Emergency Change) হলো Governance-এর সবচেয়ে দুর্বল জায়গা। চাপের মুখে মানুষ Gate এড়িয়ে সরাসরি Prod-এ বদলায় — আর সেই বদল Playbook-এ ফিরিয়ে লেখা হয় না। ফলে পরেরবার Playbook চললে হাতে করা বদলটি মুছে যায়, আবার সমস্যা ফিরে আসে। নিয়মটি কঠোর রাখুন: জরুরি পরিবর্তনও ২৪ ঘণ্টার মধ্যে Playbook-এ লিপিবদ্ধ করতে হবে, আর তার Pull Request-এ "জরুরি পরিবর্তন — ঘটনা নম্বর" লিখতে হবে।

২৪.৭ আর্কিটেকচার চিত্র

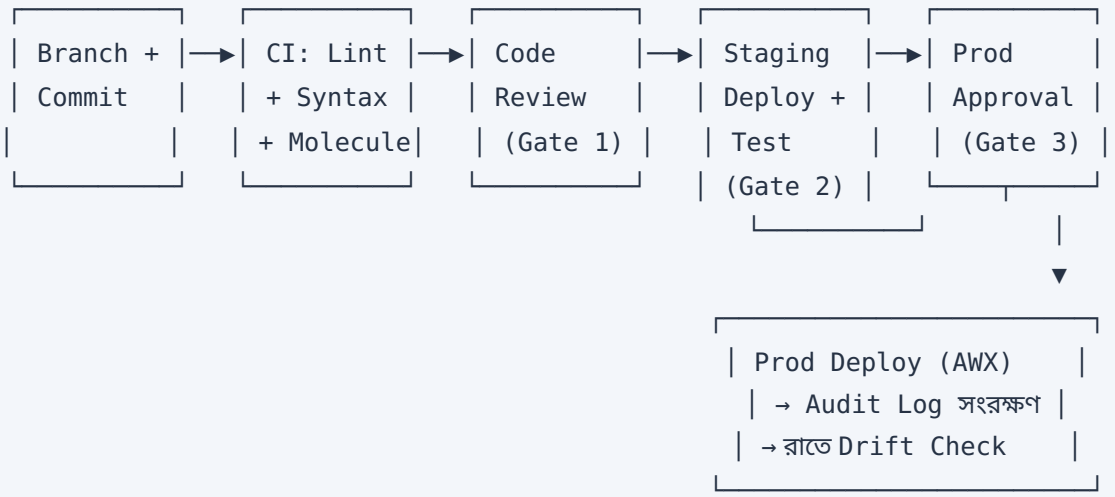
বহু-পরিবেশ INVENTORY কাঠামো

```
inventories/
├── dev/
│   ├── hosts.ini              # Dev সার্ভার: 203.0.113.21
│   └── group_vars/all.yml     # Dev-এর সেটিং (debug on, কম worker)
├── staging/
│   ├── hosts.ini              # Staging সার্ভার: 203.0.113.11
│   └── group_vars/all.yml     # Staging-এর সেটিং (Prod-এর মতো, কম ট্রাফিক)
└── prod/
    ├── hosts.ini              # Prod সার্ভার: 203.0.113.10, 203.0.113.12
    ├── group_vars/all.yml     # Prod-এর সেটিং (বেশি worker, কঠোর সীমা)
    └── group_vars/all/vault.yml   # Vault-এনক্রিপ্ট করা Prod Secret
```

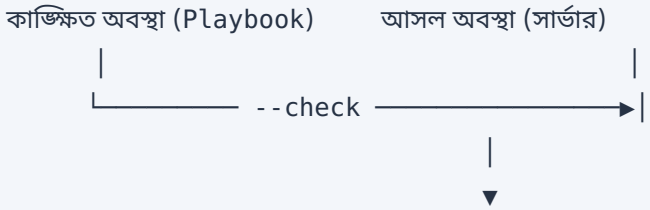
একই Playbook, তিন পরিবেশ — শুধু Inventory বদলায়:

```
ansible-playbook site.yml -i inventories/dev/hosts.ini
ansible-playbook site.yml -i inventories/staging/hosts.ini
ansible-playbook site.yml -i inventories/prod/hosts.ini    # Approval-এর পর
```

CHANGE PIPELINE — পরিবর্তনের যাত্রা



DRIFT DETECTION — নির্ধারিত --check চালনা



changed=0 → কোনো Drift নেই ✓
changed>0 → Drift আছে! → Reconcile করুন
(Playbook আবার চালান, বা
Playbook-এ বদলটি লিপিবদ্ধ করুন)

এই চিত্রগুলো একসঙ্গে পড়ুন: Inventory কাঠামো বলে "কোথায় কী আছে", Change Pipeline বলে "পরিবর্তন কীভাবে যাবে", আর Drift Detection বলে "পরিবর্তনের পরও সব ঠিক আছে কি না"।

২৪.৮ বাস্তব উদাহরণ

TechNova-র Prod-এ Nginx-এর Worker Process ২ থেকে ৪ করার একটি পরিবর্তন — পুরো প্রক্রিয়াটি কমান্ড-স্তরে:

```
# ১. নতুন Branch-এ পরিবর্তন করুন
git checkout -b change/nginx-workers-prod
# group_vars পরিবর্তন: inventories/prod/group_vars/all.yml-এ
#   nginx_worker_processes: 2  →  4

# ২. স্থানীয়ভাবে পরীক্ষা করুন (Staging Inventory-র বিরুদ্ধে Dry Run)
ansible-playbook playbooks/site.yml -i inventories/staging/hosts.ini \
  --check --diff --limit staging-web.technova.example

# ৩. Commit ও Pull Request
git add inventories/prod/group_vars/all.yml
git commit -m "Prod Nginx workers 2 থেকে 4 করা — ট্রাফিক বৃদ্ধির কারণে (টিকিট #1847)"
git push origin change/nginx-workers-prod
# → GitHub-এ PR খুলুন; CI স্বয়ংক্রিয়ভাবে Lint + Syntax Check চালাবে

# ৪. Code Review-এর পর Merge হলে Staging-এ Deploy (স্বয়ংক্রিয় বা AWX Job)
ansible-playbook playbooks/site.yml -i inventories/staging/hosts.ini

# ৫. Staging ঠিক থাকলে Prod Deploy — Approval-এর পর, AWX থেকে বা হাতে
ansible-playbook playbooks/site.yml -i inventories/prod/hosts.ini \
  --vault-password-file ~/.ansible/vault-pass.txt

# ৬. রাতের Drift Check (Cron বা AWX Schedule থেকে)
ansible-playbook playbooks/site.yml -i inventories/prod/hosts.ini \
  --check --diff > /var/log/ansible/drift-check-$(date +%F).log 2>&1
# চিত্রণমূলক আউটপুট: changed=0 — কোনো Drift নেই

# ৭. Rollback — ভুল হলে আগের Tag-এর Playbook চালান
git checkout v1.4.1
ansible-playbook playbooks/site.yml -i inventories/prod/hosts.ini
git checkout main
```

লক্ষ্য করুন, এই প্রক্রিয়ায় কোনো ধাপেই কেউ সরাসরি সার্ভারে গিয়ে ফাইল বদলায়নি — প্রতিটি পরিবর্তন Git-এর মধ্য দিয়ে, পরীক্ষা ও অনুমোদনের পর গেছে। এটাই Enterprise Configuration Management-এর আসল অর্থ।

২৪.৯ সম্পূর্ণ YAML/কনফিগারেশন ফাইল

নিচে বহু-পরিবেশ Inventory, Drift Check Playbook ও একটি Runbook-এর কাঠামো দেওয়া হলো:

```
# ── ফাইল: inventories/prod/hosts.ini ──────────────────────────────
[webservers]
prod-web01.technova.example ansible_host=203.0.113.10
prod-web02.technova.example ansible_host=203.0.113.13

[dbservers]
prod-db01.technova.example ansible_host=203.0.113.12
```

```
[prod:children]
webservers
dbservers

# ── ফাইল: inventories/prod/group_vars/all.yml ──────────────────────────
---
nginx_worker_processes: 4
nginx_worker_connections: 2048
app_environment: production
enable_debug: false
log_level: warn
ntp_servers:
  - 203.0.113.1
  - 203.0.113.2

# ── ফাইল: inventories/dev/group_vars/all.yml ──────────────────────────
---
nginx_worker_processes: 1
nginx_worker_connections: 512
app_environment: development
enable_debug: true
log_level: debug

# ── ফাইল: playbooks/drift-check.yml ──────────────────────────────────
# উদ্দেশ্য: কাঙ্ক্ষিত অবস্থা থেকে সরে যাওয়া (Drift) শনাক্ত করা।
# চালানোর নিয়ম: --check মোডে, নির্ধারিত সময়ে (AWX Schedule বা Cron)।
---
- name: Drift detection - compare desired state with actual state
  hosts: all
  become: true
  gather_facts: false

  tasks:
    - name: Check that required packages are present
      ansible.builtin.apt:
        name:
          - nginx
          - curl
          - ufw
        state: present
      check_mode: true
      register: package_drift

    - name: Check that Nginx configuration matches template
      ansible.builtin.template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        owner: root
        group: root
        mode: "0644"
      check_mode: true
      diff: true
      register: config_drift

    - name: Report drift status
      ansible.builtin.debug:
        msg: >-
          Drift on {{ inventory_hostname }}:
```

```
          packages changed={{ package_drift.changed }},
          config changed={{ config_drift.changed }}

    - name: Fail the run if any drift is found
      ansible.builtin.assert:
        that:
          - not package_drift.changed
          - not config_drift.changed
        fail_msg: "DRIFT detected on {{ inventory_hostname }} - reconcile manually."
        success_msg: "No drift on {{ inventory_hostname }}."

# ── ফাইল: runbooks/nginx-down.md (Runbook-এর কাঠামো) ──────────────────
# শিরোনাম: Nginx বন্ধ হয়ে গেলে করণীয়
# লক্ষণ: ওয়েবসাইট খুলছে না; Monitoring-এ HTTP 502/503; Port 80 সাড়া দিচ্ছে না
# ১. নির্ণয়: systemctl status nginx ; tail -n 50 /var/log/nginx/error.log
# ২. দ্রুত ঠিক করা: ansible-playbook playbooks/site.yml -i inventories/prod/hosts.ini --tags nginx
# ৩. যদি না চলে: nginx -t (কনফিগারেশন পরীক্ষা); journalctl -u nginx -n 100
# ৪. Rollback: git checkout <আগের Tag> ; Playbook আবার চালান
# ৫. Escalate: ৩০ মিনিটে না ঠিক হলে Team Lead-কে জানান; গ্রাহক-যোগাযোগ টিমকে অবহিত করুন
# ৬. ঘটনার পর: কারণ লিপিবদ্ধ করুন; Playbook-এ প্রতিরোধমূলক পরিবর্তন আনুন
```

Drift Check Playbook-টি লক্ষ্য করুন: এটি `check_mode: true` দিয়ে লেখা — অর্থাৎ এটি চালালে কিছু বদলায় না, শুধু বদলাত কি না তা জানায়; আর শেষে `assert` দিয়ে Drift পাওয়া গেলে চালনাটিকে ব্যর্থ করে দেয়, যাতে AWX বা CI-তে সেটি লাল (Failed) দেখায় ও সবাই জানতে পারে।

২৪.১০ ধাপে ধাপে বাস্তবায়ন

ধাপ ১ — Inventory ভাগ করুন। একটিমাত্র Inventory ফাইলের বদলে `inventories/dev`, `staging` ও `prod` ডিরেক্টরি তৈরি করুন; প্রতিটিতে নিজের Host ও Group Variable রাখুন।

ধাপ ২ — Variable-এর স্তর ঠিক করুন। সাধারণ সেটিং `group_vars/all/`-এ, পরিবেশ-নির্দিষ্ট সেটিং সংশ্লিষ্ট পরিবেশের `group_vars/`-এ, আর Secret শুধু Vault-এনক্রিপ্ট করা ফাইলে রাখুন।

ধাপ ৩ — Branching ও Review নিয়ম লিখুন। `main` Branch-এ সরাসরি Push বন্ধ করুন (Branch Protection); প্রতিটি PR-এ অন্তত একজন Reviewer ও CI পাস বাধ্যতামূলক করুন।

ধাপ ৪ — Approval Gate বসান। GitHub Environment-এ Prod-এর জন্য Required Reviewer সেট করুন, বা AWX Workflow-এ Approval Node যোগ করুন।

ধাপ ৫ — Drift Check নির্ধারণ করুন। AWX-এ একটি নির্ধারিত Job (Schedule) তৈরি করুন যা প্রতি রাতে `--check` মোডে Drift Check Playbook চালায় ও ফলাফল রিপোর্ট করে।

ধাপ ৬ — Rollback অনুশীলন করুন। Staging-এ একটি পরিবর্তন করে আগের Tag-এ ফিরে যাওয়ার অনুশীলন করুন; নিশ্চিত হোন Rollback সত্যিই কাজ করে।

ধাপ ৭ — Runbook লিখুন। সবচেয়ে সাধারণ তিনটি ঘটনার (সার্ভিস বন্ধ, ডিস্ক ভরা, Deploy ব্যর্থ) Runbook লিখুন ও টিমের সঙ্গে পর্যালোচনা করুন।

ধাপ ৮ — Audit Trail যাচাই করুন। নিশ্চিত হোন AWX Job ইতিহাস, CI লগ ও Git ইতিহাস সংরক্ষিত হচ্ছে ও প্রয়োজনে খুঁজে পাওয়া যায়।

২৪.১১ প্রত্যাশিত আউটপুট ও ব্যাখ্যা

Drift Check-এর সফল আউটপুট (চিত্রণমূলক আউটপুট):

```
$ ansible-playbook playbooks/drift-check.yml -i inventories/prod/hosts.ini --check

TASK [Report drift status] ***********************************************
ok: [prod-web01.technova.example] => {
    "msg": "Drift on prod-web01.technova.example: packages changed=false, config
changed=false"
}

TASK [Fail the run if any drift is found] ********************************
ok: [prod-web01.technova.example] => {
    "msg": "No drift on prod-web01.technova.example."
}

PLAY RECAP ***************************************************************
prod-web01.technova.example : ok=4  changed=0  unreachable=0  failed=0
```

এই আউটপুটের অর্থ: Prod সার্ভারের আসল অবস্থা কাঙ্ক্ষিত অবস্থার সঙ্গে মিলে গেছে — কোনো Drift নেই। আর Drift থাকলে আউটপুটটি অন্যরকম হতো: `config changed=true` দেখাত, Diff-এ দেখাত কোন লাইন বদলেছে, আর শেষের `assert` ব্যর্থ হয়ে "DRIFT detected" বার্তা দিত — AWX-এ Job-টি লাল দেখাত। Prod Deploy-এর প্রত্যাশিত আউটপুটও একইভাবে পড়তে হয়: `PLAY RECAP`-এ `failed=0` ও `unreachable=0` থাকা চাই; `changed`-এর সংখ্যা পরিবর্তনের আকার অনুযায়ী যুক্তিসংগত হওয়া চাই — অপ্রত্যাশিতভাবে বড় `changed` সংখ্যা মানে কিছু ভুল হচ্ছে।

২৪.১২ সাধারণ ভুল

ভুল	কেন সমস্যা	সঠিক অভ্যাস
একটিমাত্র Inventory সব পরিবেশের জন্য	ভুল করে Dev-এর Playbook Prod-এ চলে যেতে পারে	প্রতি পরিবেশে আলাদা Inventory ডিরেক্টরি
সরাসরি সার্ভারে গিয়ে হাতে ফাইল বদলানো	Drift তৈরি হয়; পরের Playbook চালনায় বদল মুছে যায়	সব পরিবর্তন Playbook-এর মধ্য দিয়ে, Git-এ লিপিবদ্ধ করে
Prod Deploy-এ Approval Gate না রাখা	ভুল কোড বিনা বাধায় প্রোডাকশনে যায়	Staging পরীক্ষা + মানুষের অনুমোদন — দুটোই বাধ্যতামূলক
Rollback পরীক্ষা না করা	জরুরি মুহূর্তে দেখা যায় Rollback কাজ করে না	Staging-এ নিয়মিত Rollback অনুশীলন করুন
Tag ছাড়া Deploy করা	Prod-এ কোন সংস্করণ চলছে তা জানা যায় না	প্রতিটি Release-এ Git Tag দিন; AWX Job-এ সংস্করণ লিপিবদ্ধ করুন
Audit Log না রাখা বা মুছে ফেলা	ঘটনার তদন্তে প্রমাণ নেই; Compliance ব্যর্থ হয়	Job লগ ও Git ইতিহাস নির্দিষ্ট মেয়াদ পর্যন্ত সংরক্ষণ করুন
Runbook না থাকা	ঘটনার সময় প্রত্যেকে নিজের মতো করে ঠিক করার চেষ্টা করে	সাধারণ ঘটনাগুলোর Runbook লিখুন ও নিয়মিত হালনাগাদ করুন

২৪.১৩ সমস্যা সমাধান (Troubleshooting)

Staging-এ চলে, Prod-এ চলে না। প্রথমে Inventory-র পার্থক্য দেখুন: Prod-এর Group Variable-তে কোনো মান কি আলাদা? Prod Host-এর SSH ব্যবহারকারীর অনুমতি কি Staging-এর মতো? ফায়ারওয়াল বা নেটওয়ার্ক নিয়ম কি আলাদা? `ansible-inventory --host prod-web01.technova.example` ও `ansible-inventory --host staging-web.technova.example` — দুটির আউটপুট পাশাপাশি মিলিয়ে দেখুন।

Drift Check প্রতিদিন Drift দেখাচ্ছে, কিন্তু কেউ কিছু বদলায়নি। সম্ভাব্য কারণ: কোনো প্যাকেজ স্বয়ংক্রিয়ভাবে আপডেট হচ্ছে (Unattended Upgrade), কোনো সার্ভিস নিজে থেকে কনফিগারেশন ফাইল লিখছে, বা Playbook-টি Idempotent নয় — প্রতিবার কিছু বদলাচ্ছে। Drift-এর Diff পড়ে দেখুন কোন ফাইল বা প্যাকেজ বদলাচ্ছে; কারণটি খুঁজে Playbook-এ বা সার্ভারের সেটিংয়ে ঠিক করুন।

Rollback-এর পর সার্ভিস চলছে না। আগের Tag-এর Playbook চালালেও কিছু জিনিস ফেরে না — যেমন ডেটাবেসের Schema পরিবর্তন, বা মুছে ফেলা ডেটা। Rollback পরিকল্পনায় আগেই ঠিক করুন কোন পরিবর্তনগুলো ফেরানো যায় আর কোনগুলো যায় না; যেগুলো যায় না সেগুলোর জন্য আলাদা পরিকল্পনা (যেমন ব্যাকআপ থেকে পুনরুদ্ধার) রাখুন।

Approval Gate-এ আটকে আছে, কিন্তু কেউ অনুমোদন দিচ্ছে না। প্রক্রিয়াগত সমস্যা: অনুমোদনকারী কি ছুটিতে? Notification কি পৌঁছাচ্ছে? AWX-এ Approval Node-এর দায়িত্বপ্রাপ্ত ব্যক্তি বা GitHub Environment-এর Reviewer তালিকা হালনাগাদ আছে কি না দেখুন; জরুরি ক্ষেত্রে বিকল্প অনুমোদনকারীর (Backup Approver) ব্যবস্থা রাখুন।

Audit-এ দেখা গেল কে বদলেছে তা জানা যাচ্ছে না। কারণ: কেউ AWX-এর বাইরে, নিজের ল্যাপটপ থেকে Playbook চালিয়েছে, বা Shared অ্যাকাউন্ট ব্যবহার করেছে। সমাধান: Prod-এ শুধু AWX/Controller থেকেই Deploy-এর অনুমতি দিন; ব্যক্তিগত অ্যাকাউন্ট ব্যবহার বাধ্যতামূলক করুন; Shared অ্যাকাউন্ট বন্ধ করুন।

> **TIP:** Enterprise-সমস্যার বেশিরভাগই প্রযুক্তিগত নয়, প্রক্রিয়াগত: "কে বদলেছে", "কেন বদলেছে", "কোথায় লিপিবদ্ধ আছে" — এই তিনটি প্রশ্নের উত্তর না থাকলে প্রযুক্তি যত ভালোই হোক, ব্যবস্থাটি ভঙ্গুর। সমস্যা সমাধানের সময় প্রথমে এই তিনটি প্রশ্ন করুন, তারপর কমান্ডের দিকে যান।

২৪.১৪ নিরাপত্তা বিবেচনা

Enterprise নিরাপত্তার মূল নীতি **Least Privilege**: প্রতিটি ব্যক্তি ও প্রতিটি সিস্টেম শুধু ততটুকু অনুমতি পাবে যতটুকু তার কাজের জন্য দরকার — তার বেশি নয়। বাস্তবে এর অর্থ: Dev Engineer-রা Prod Inventory-তে Playbook চালাতে পারবেন না; Prod-এর SSH Key শুধু AWX-এর Credential Store-এ থাকবে, কারো ল্যাপটপে নয়; Vault পাসওয়ার্ড শুধু অনুমোদিত Job-ই ব্যবহার করতে পারবে। দ্বিতীয় নীতি **Secret-এর পৃথকীকরণ**: Dev, Staging ও Prod-এর Secret আলাদা — একটি পরিবেশের Secret ফাঁস হলে অন্য পরিবেশ নিরাপদ থাকে। তৃতীয় নীতি **নিয়মিত পর্যালোচনা (Access Review)**: প্রতি তিন মাসে দেখুন কে কী অনুমতি পেয়েছে, যারা আর টিমে নেই তাদের অনুমতি বাতিল হয়েছে কি না। চতুর্থ নীতি **পরিবর্তনের প্রমাণ**: প্রতিটি Prod পরিবর্তনের জন্য PR নম্বর, Approval-এর রেকর্ড ও Job লগ সংরক্ষণ করুন — এটি শুধু নিরাপত্তা নয়, Compliance-এরও দাবি।

২৪.১৫ প্রোডাকশন ইঞ্জিনিয়ারিং অনুশীলন

প্রোডাকশন-স্তরের Configuration Management-এর পেশাদার নিয়ম: **Git-ই একমাত্র সত্যের উৎস** — সার্ভারের অবস্থা জানতে চাইলে Git-এর `main` Branch দেখুন, সার্ভারে গিয়ে নয়। **পরিবর্তন ছোট রাখুন** — একটি PR-এ একটি পরিবর্তন; বড় পরিবর্তন ভাগ করে ছোট ছোট PR করুন, যাতে Review ও Rollback সহজ হয়। **পরিবর্তনের সময় নির্ধারণ করুন**

(Change Window) — Prod Deploy শুধু নির্দিষ্ট সময়ে (যেমন কম ট্রাফিকের সময়) করা হয়; জরুরি পরিবর্তন ছাড়া ব্যস্ত সময়ে Deploy নয়। **সবকিছু পরিমাপ করুন** — কতবার Deploy হলো, কতবার ব্যর্থ হলো, Drift কতবার পাওয়া গেল — এই সংখ্যাগুলোই বলে ব্যবস্থাটি সুস্থ আছে কি না। **নিয়মিত মহড়া (Drill) করুন** — Rollback, Drift-এর প্রতিক্রিয়া ও জরুরি পরিবর্তনের মহড়া নিয়মিত করুন, যাতে আসল ঘটনার সময় কেউ প্রথমবারের মতো না করে।

২৪.১৬ অধ্যায়ের সারাংশ

এই অধ্যায়ে আপনি শিখলেন Enterprise Configuration Management-এর পূর্ণ ছবি: বহু-পরিবেশ Inventory (Dev/Staging/Prod আলাদা, একই Playbook), Change Approval Gate (Code Review, Staging পরীক্ষা, Prod অনুমোদন), Drift Detection (নির্ধারিত --check চালনা ও Reconciliation), Version Control ও Code Review (Branch Protection, Tag, PR), Rollback কৌশল (আগের Tag, ব্যাকআপ, Snapshot), Audit Logging (কে, কখন, কী — তার প্রমাণ), Runbook (ঘটনার জন্য ধাপে ধাপে নির্দেশনা), এবং Governance ও Compliance-এর মূল নীতি (Least Privilege, Separation of Duties, প্রমাণ সংরক্ষণ)। মূল বার্তা: Enterprise-এ সাফল্য মানে শুধু সঠিক কনফিগারেশন নয় — সঠিক প্রক্রিয়ায়, সঠিক অনুমোদনে, প্রমাণসহ সঠিক কনফিগারেশন।

২৪.১৭ জ্ঞান যাচাই

১. Configuration Drift কী? নির্ধারিত --check চালনা কীভাবে Drift শনাক্ত করে?
২. একই Playbook তিনটি পরিবেশে চালানোর জন্য Inventory কীভাবে সাজাবেন? সাধারণ ও পরিবেশ-নির্দিষ্ট Variable কোথায় রাখবেন?
৩. Change Approval Gate-এর তিনটি স্তর কী কী? প্রতিটি কোন ধরনের ভুল ধরে?
৪. Ansible-এ Rollback-এর তিনটি উপায় লিখুন এবং কোন পরিস্থিতিতে কোনটি উপযুক্ত তা বলুন।
৫. Least Privilege ও Separation of Duties — এই দুটি নীতি Ansible পরিচালনায় কীভাবে প্রয়োগ করবেন? উদাহরণসহ লিখুন।

২৪.১৮ ব্যবহারিক অ্যাসাইনমেন্ট

TechNova-র জন্য একটি সম্পূর্ণ Enterprise কাঠামো নকশা করুন: (ক) `inventories/dev`, `staging` ও `prod` ডিরেক্টরি তৈরি করুন — প্রতিটিতে Host ও Group Variable-সহ, Dev-এ Debug চালু ও Prod-এ বন্ধ রেখে; (খ) একটি Drift Check Playbook লিখুন যা --check মোডে চলে ও Drift পেলে ব্যর্থ হয়; (গ) একটি Change Pipeline নথি লিখুন — Branch থেকে Prod Deploy পর্যন্ত প্রতিটি ধাপ, প্রতিটি Gate-এর দায়িত্বপ্রাপ্ত ব্যক্তি ও Approval-এর নিয়মসহ; (ঘ) Nginx বন্ধ হয়ে যাওয়ার ঘটনার জন্য একটি পূর্ণ Runbook লিখুন — লক্ষণ, নির্ণয়, ঠিক করার ধাপ, Rollback পরিকল্পনা ও Escalation নিয়মসহ; (ঙ) একটি Rollback মহড়া করুন: Staging-এ একটি পরিবর্তন করে আগের Tag-এ ফিরে যান ও ফলাফল লিপিবদ্ধ করুন।

PART V • বিশেষ বিভাগ — Terraform & Ansible Integration

Terraform ও Ansible Integration — পরিপূরক দায়িত্ব

ভূমিকা

নতুন DevOps Engineer-দের সবচেয়ে সাধারণ বিভ্রান্তিগুলোর একটি হলো: "Terraform আর Ansible — দুটোই তো Infrastructure নিয়ে কাজ করে; কোনটি কখন ব্যবহার করব?" উত্তরটি হলো: দুটি ভিন্ন কাজের জন্য দুটি ভিন্ন টুল — তারা প্রতিদ্বন্দ্বী নয়, পরিপূরক। একটি সহজ উপমা: বাড়ি বানানোর সময় Terraform হলো সেই ঠিকাদার যে জমি কেনে, ভিত্তি গড়ে, দেওয়াল তোলে, বিদ্যুৎ ও পানির সংযোগ আনে — অর্থাৎ **জিনিসটি তৈরি করে**। Ansible হলো সেই অভ্যন্তরীণ সজ্জাকার (Interior Designer) ও রক্ষণাবেক্ষণকারী যে ঘরের ভেতরে আসবাব বসায়, রং করে, প্রতিটি ঘর ব্যবহারের উপযোগী করে, আর পরে নিয়মিত দেখে সব ঠিক আছে কি না — অর্থাৎ **তৈরি জিনিসটিকে ব্যবহারের উপযোগী করে ও সেই অবস্থায় রাখে**। Terraform ছাড়া Ansible-এর কনফিগার করার মতো সার্ভারই থাকবে না; Ansible ছাড়া Terraform-এর তৈরি সার্ভারটি খালি একটি OS-সহ মেশিন হয়েই থাকবে — তাতে Nginx থাকবে না, অ্যাপ থাকবে না, কনফিগারেশন থাকবে না। এই বিভাগে আপনি শিখবেন দুটি টুলের দায়িত্বের সীমানা কোথায়, কীভাবে তারা একসঙ্গে কাজ করে, আর কীভাবে Secret-গুলো দুটি সিস্টেমের মাঝখানে ফাঁস হওয়া থেকে আটকাবেন।

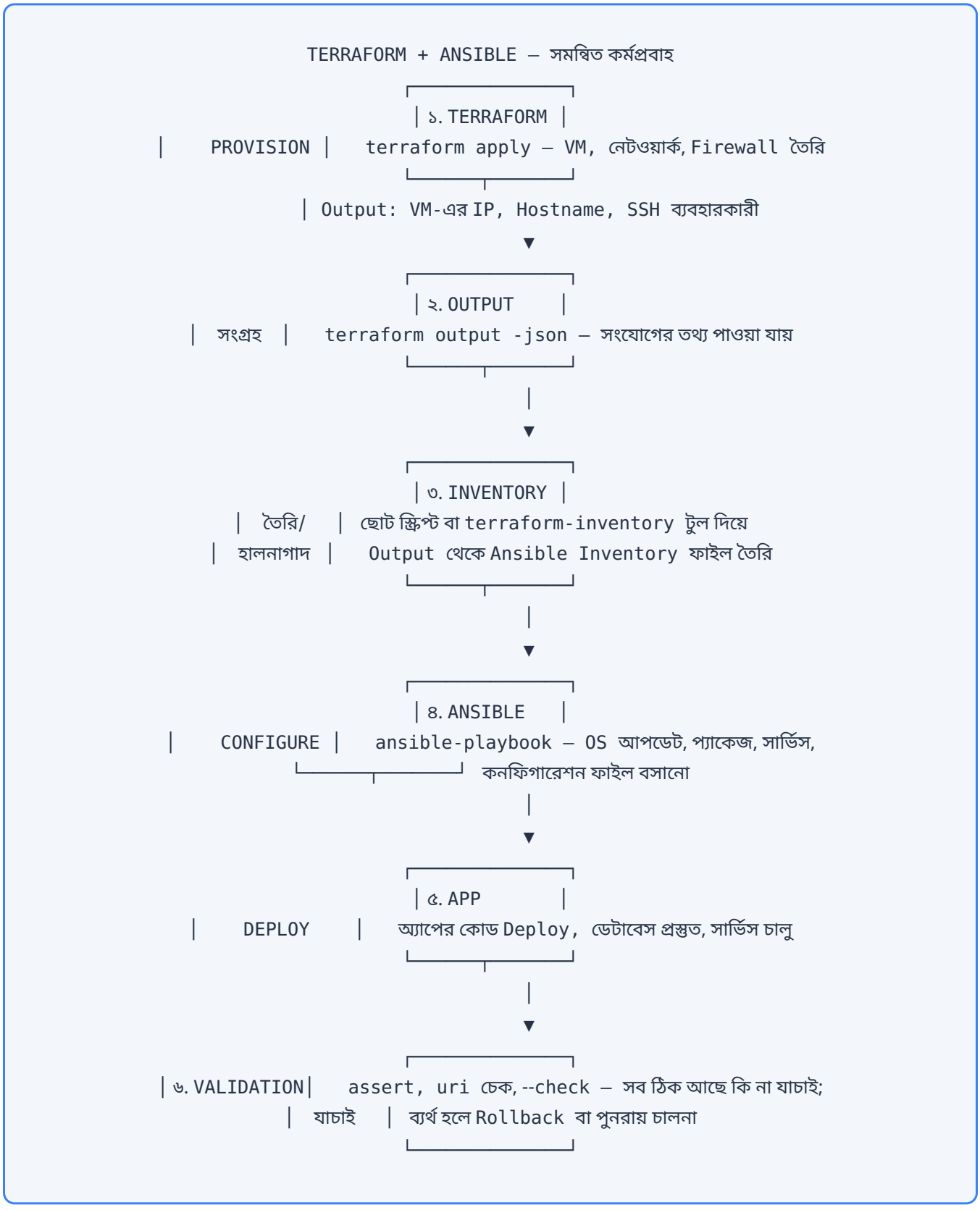
দায়িত্বের তুলনা — কে কী করে

বিষয়	Terraform	Ansible
মূল দায়িত্ব	Infrastructure তৈরি ও পরিচালনা (Provisioning)	সার্ভার কনফিগার করা ও অ্যাপ Deploy করা (Configuration Management)
যা তৈরি করে	VM, নেটওয়ার্ক, Subnet, Firewall নিয়ম, Load Balancer, ডেটাবেস সার্ভিস, DNS রেকর্ড	OS-এর ভেতরের অবস্থা: প্যাকেজ, ফাইল, সার্ভিস, ব্যবহারকারী, অ্যাপ
কাজের স্তর	ক্লাউড/অবকাঠামো API-র স্তর (বাইরে থেকে)	OS-এর ভেতরের স্তর (SSH দিয়ে ভেতরে ঢুকে)
অবস্থা সংরক্ষণ	State ফাইলে (terraform.tfstate) — কী তৈরি হয়েছে তার হিসাব	State ফাইল রাখে না; Playbook-ই কাঙ্ক্ষিত অবস্থার বিবরণ
কীভাবে চলে	`terraform apply` — পরিকল্পনা (Plan) দেখিয়ে, অনুমোদনের পর তৈরি করে	`ansible-playbook` — Task-গুলো ধাপে ধাপে চালিয়ে অবস্থা প্রয়োগ করে
পরিবর্তনের ধরন	Immutable-ধাঁচের: VM বদলাতে হলে প্রায়ই নতুন VM তৈরি করে পুরনোটি বদলে দেয়	Mutable-ধাঁচের: একই সার্ভারের ভেতরে বারবার অবস্থা ঠিক করে
ভালো যেখানে	নতুন পরিবেশ তৈরি, পরিবেশ মুছে ফেলা, ক্লাউড রিসোর্সের হিসাব রাখা	সার্ভার কনফিগার, অ্যাপ Deploy, নিয়মিত Drift ঠিক করা, প্যাচ দেওয়া
খারাপ যেখানে	OS-এর ভেতরের সূক্ষ্ম কনফিগারেশন, সার্ভিস পরিচালনা	VM বা নেটওয়ার্ক তৈরি করা, ক্লাউড রিসোর্সের হিসাব রাখা
TechNova উদাহরণ	203.0.113.10 ও 203.0.113.11 VM দুটি তৈরি করা, তাদের Firewall নিয়ম বসানো	VM দুটিতে Nginx ইনস্টল, কনফিগারেশন বসানো, অ্যাপ Deploy করা

> **CONCEPT:** দায়িত্বের সীমানা মনে রাখার সবচেয়ে সহজ নিয়ম: **Terraform "কী আছে" ঠিক করে, Ansible "কেমন আছে" ঠিক করে।** Terraform বলে: "দুটি VM আছে, তাদের IP এই, Firewall-এ Port 80 খোলা।" Ansible বলে: "VM দুটিতে Nginx ইনস্টল আছে, কনফিগারেশন এইরকম, সার্ভিস চলছে।" এই সীমানা ভাঙলেই সমস্যা: Terraform দিয়ে OS-এর ভেতরের ফাইল বদলাতে গেলে তা ভঙ্গুর হয়; Ansible দিয়ে VM তৈরি করতে গেলে তার হিসাব রাখা যায় না।

একটি সাধারণ ভুল ধারণা দূর করা দরকার: Terraform-এর `provisioner` (যেমন `remote-exec`) দিয়ে VM তৈরির পরপরই ভেতরে কমান্ড চালানো যায় — কিন্তু এটি Terraform-এর সুপারিশকৃত পদ্ধতি নয়। Provisioner-কে Terraform নিজেই "শেষ উপায়" (Last Resort) বলে। সঠিক পদ্ধতি হলো: Terraform শুধু Infrastructure তৈরি করবে, আর কনফিগারেশনের দায়িত্ব Ansible-কে ছেড়ে দেবে — দুটি টুলের মাঝখানে সংযোগ হবে Terraform-এর Output ও Ansible-এর Inventory দিয়ে।

সমন্বিত কর্মপ্রবাহ — ৬ ধাপ



ধাপ ১ — Terraform Provision করে। `terraform apply` দিয়ে VM, নেটওয়ার্ক ও Firewall তৈরি হয়। এই ধাপে কোনো অ্যাপ বা কনফিগারেশন বসানো হয় না — শুধু খালি, চালু VM।

ধাপ ২ — Terraform Output সংযোগের তথ্য দেয়। Terraform কনফিগারেশনে `output` ব্লক লিখে VM-এর Public IP, Hostname ও SSH ব্যবহারকারীর নাম প্রকাশ করা হয়। এই তথ্যগুলো Secret নয় — IP ও Hostname প্রকাশ করা যায় — তাই Output-এ রাখা নিরাপদ।

ধাপ ৩ — Ansible Inventory তৈরি/হালনাগাদ হয়। Terraform-এর Output থেকে একটি ছোট স্ক্রিপ্ট (Python বা Shell) Ansible Inventory ফাইল তৈরি করে, অথবা `terraform-inventory`-এর মতো টুল Terraform State থেকে সরাসরি Dynamic Inventory দেয়। এই ধাপটিই দুটি টুলের সংযোগস্থল — এখানে ভুল হলে Ansible ভুল সার্ভারে বা কোনো সার্ভারেই সংযোগ করতে পারবে না।

ধাপ ৪ — Ansible সার্ভার কনফিগার করে। `ansible-playbook` দিয়ে OS আপডেট, প্রয়োজনীয় প্যাকেজ ইনস্টল, Firewall-এর ভেতরের নিয়ম, ব্যবহারকারী ও কনফিগারেশন ফাইল বসানো হয়।

ধাপ ৫ — অ্যাপ Deploy হয়। একই Playbook-এর পরের অংশে (বা আলাদা Playbook-এ) অ্যাপের কোড Deploy করা হয়, ডেটাবেস প্রস্তুত করা হয় ও সার্ভিস চালু করা হয়।

ধাপ ৬ — Validation প্রস্তুতি নিশ্চিত করে। অধ্যায় ২২-এর `assert` ও `uri` চেক দিয়ে যাচাই করা হয়: সার্ভিস চলছে কি না, HTTP সাড়া দিচ্ছে কি না, প্রত্যাশিত Port খোলা কি না। সব ঠিক থাকলে পরিবেশ "প্রস্তুত" (Ready) ঘোষণা করা হয়; না থাকলে Rollback বা পুনরায় চালনা।

> **TIP:** এই ছয় ধাপকে একটি স্ক্রিপ্ট বা CI Pipeline-এ বেঁধে দিন — `terraform apply && python3 scripts/build-inventory.py && ansible-playbook site.yml` — যাতে কোনো ধাপ হাতে এড়িয়ে যাওয়া না যায়। তবে Prod-এ `terraform apply`-এর আগে `terraform plan` দেখে অনুমোদন নেওয়া বাধ্যতামূলক রাখুন: Plan-ই বলে দেবে কী তৈরি বা ধ্বংস হতে যাচ্ছে।

সম্পূর্ণ উদাহরণ — Terraform Output + Inventory + Playbook

TechNova-র Staging পরিবেশের একটি ছোট কিন্তু সম্পূর্ণ উদাহরণ: Terraform দুটি VM তৈরি করে, তার Output থেকে Inventory তৈরি হয়, আর Ansible সেই VM-গুলোতে Nginx বসায়।

```
# ── ফাইল: terraform/main.tf ──────────────────────────────────────
# দ্রষ্টব্য: Provider ও Resource-এর নাম উদাহরণের জন্য সরলীকৃত;
# আসল ক্লাউড Provider-এর ক্ষেত্রে তার নিজের Resource নাম ব্যবহার করুন।
terraform {
  required_version = ">= 1.5.0"
}

variable "environment" {
  type    = string
  default = "staging"
}

resource "example_compute_instance" "web" {
  count    = 2
  name     = "technova-${var.environment}-web-${count.index + 1}"
  image    = "ubuntu-24.04"
  size     = "small"
  ssh_user = "nahid"
}

resource "example_firewall_rule" "http" {
```

```
  name        = "technova-${var.environment}-http"
  target_port = 80
  source      = "0.0.0.0/0"
}

# ── ফাইল: terraform/outputs.tf ──────────────────────────────────
# Output-এ শুধু Secret-নয় এমন তথ্য রাখুন — IP ও Hostname নিরাপদ।
output "web_public_ips" {
  description = "Public IPs of the web servers"
  value       = example_compute_instance.web[*].public_ip
}

output "web_hostnames" {
  description = "Hostnames of the web servers"
  value       = example_compute_instance.web[*].name
}

output "ssh_user" {
  description = "SSH user for Ansible"
  value       = "nahid"
}

# ── ফাইল: scripts/build-inventory.py ────────────────────────────
# Terraform Output থেকে Ansible Inventory তৈরি করে।
# ব্যবহার: terraform output -json | python3 scripts/build-inventory.py
import json
import sys

data = json.load(sys.stdin)
ips = data["web_public_ips"]["value"]
names = data["web_hostnames"]["value"]
user = data["ssh_user"]["value"]

print("[webservers]")
for name, ip in zip(names, ips):
    print(f"{name}.technova.example ansible_host={ip} ansible_user={user}")
print()
print("[staging:children]")
print("webservers")

# ── তৈরি হওয়া Inventory (চিত্রণমূলক আউটপুট) ──────────────────────
# [webservers]
# technova-staging-web-1.technova.example ansible_host=203.0.113.10 ansible_user=nahid
# technova-staging-web-2.technova.example ansible_host=203.0.113.11 ansible_user=nahid
#
# [staging:children]
# webservers

# ── ফাইল: playbooks/site.yml (Ansible অংশ) ──────────────────────
---
- name: Configure TechNova staging web servers
  hosts: webservers
  become: true
  tasks:
    - name: Install Nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true
```

```
    - name: Ensure Nginx is running and enabled
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true

    - name: Assert Nginx is running
      ansible.builtin.assert:
        that:
          - ansible_facts.services['nginx.service'].state == 'running'
        fail_msg: "Nginx is NOT running on {{ inventory_hostname }}."

# ── সম্পূর্ণ প্রবাহ চালানো ──────────────────────────────────────
# cd terraform && terraform init && terraform apply
# terraform output -json | python3 ../scripts/build-inventory.py > ../inventories/staging/
hosts.ini
# cd .. && ansible-playbook playbooks/site.yml -i inventories/staging/hosts.ini
```

এই উদাহরণে লক্ষ্য করুন: Terraform State ফাইলটি কখনো Inventory-তে ঢোকানো হয়নি — শুধু Output-এর IP ও Hostname নেওয়া হয়েছে। Inventory ফাইলটি স্বয়ংক্রিয়ভাবে তৈরি হয় বলে VM-এর IP বদলালে হাতে Inventory বদলাতে হয় না — Terraform আবার চালালেই Inventory হালনাগাদ হয়। বড় প্রকল্পে এই স্ক্রিপ্টের বদলে Dynamic Inventory ব্যবহার করা হয়, যেখানে Ansible সরাসরি Terraform State বা ক্লাউড API থেকে Host-এর তালিকা টানে — নীতিটি একই: সত্যের উৎস একটিই, Terraform।

Secret ফাঁস রোধ — দুটি সিস্টেমের মাঝখানে

Terraform ও Ansible একসঙ্গে ব্যবহারের সবচেয়ে বিপজ্জনক জায়গাটি হলো Secret-এর চলাচল। দুটি সিস্টেমের মাঝখানে Secret ফাঁস হওয়ার পাঁচটি সাধারণ পথ ও তার প্রতিকার:

১. Terraform State ফাইলে Secret। Terraform State (`terraform.tfstate`)-এ Resource-এর সব Attribute থাকে — কোনো Resource-এ পাসওয়ার্ড বা Key দিলে তা State-এ Plain Text-এ জমা হয়। প্রতিকার: State ফাইল কখনো Git-এ Commit করবেন না; Remote Backend (যেমন S3 + DynamoDB, বা Terraform Cloud) ব্যবহার করুন যেখানে State এনক্রিপ্ট করে রাখা হয় ও অ্যাক্সেস নিয়ন্ত্রিত হয়; আর `sensitive = true` দিয়ে Output-কে Sensitive চিহ্নিত করুন, যাতে তা লগে না ছাপা হয়।

২. Inventory ফাইলে Secret। Inventory-তে SSH পাসওয়ার্ড বা API Token লেখা হলে তা Git-এ চলে যায় ও সবার কাছে পৌঁছায়। প্রতিকার: Inventory-তে শুধু Host, IP ও ব্যবহারকারীর নাম রাখুন; পাসওয়ার্ড বা Key কখনো Inventory-তে নয় — SSH Key ব্যবহার করুন, আর Key-র পাথ শুধু Ansible Vault বা AWX Credential-এ রাখুন।

৩. Terraform Output-এ Secret। Output-এ পাসওয়ার্ড বা Token রাখলে `terraform output` চালালেই তা স্ক্রিনে দেখা যায় ও লগে জমা হয়। প্রতিকার: Output-এ শুধু Secret-নয় এমন তথ্য (IP, Hostname, Port) রাখুন; Secret দরকার হলে Output ব্লকে `sensitive = true` দিন, আর প্রয়োজনে Secret-টি সরাসরি Secret Manager (যেমন HashiCorp Vault, AWS Secrets Manager) থেকে নিন।

৪. দুটি সিস্টেমে একই Credential ব্যবহার। Terraform-এর ক্লাউড Credential আর Ansible-এর SSH Credential এক হলে একটি ফাঁস হলে দুটি সিস্টেমই ঝুঁকিতে পড়ে। প্রতিকার: প্রতিটি সিস্টেমের জন্য আলাদা Credential — Terraform-এর জন্য ক্লাউড API Key (শুধু Infrastructure তৈরির অনুমতিসহ), Ansible-এর জন্য আলাদা SSH Key (শুধু সার্ভার কনফিগার করার অনুমতিসহ); কোনো মানুষের ব্যক্তিগত Key কোনো সিস্টেমে ব্যবহার করবেন না।

৫. Ansible Secret Terraform-এর মধ্যে, বা উল্টো। যেমন Terraform-এর Variable ফাইলে ডেটাবেস পাসওয়ার্ড লিখে তা Ansible-কে দেওয়া। প্রতিকার: Ansible-এর Secret শুধু Ansible Vault-এ (এনক্রিপ্ট করা) রাখুন; Terraform-এর Secret শুধু Terraform-এর Variable-এ (Environment Variable বা `.tfvars` ফাইল — Git-এ নয়) রাখুন; দুটি সিস্টেমের মধ্যে শুধু Secret-নয় এমন তথ্য (IP, Hostname) আদান-প্রদান করুন। একটিই ব্যতিক্রম: দুটি সিস্টেমকেই যদি একই Secret দরকার হয় (যেমন ডেটাবেস পাসওয়ার্ড), তাহলে সেই Secret-টি একটি কেন্দ্রীয় Secret Manager-এ রাখুন ও দুটি সিস্টেমই সেখান থেকে নেবে — Secret কখনো এক সিস্টেম থেকে আরেক সিস্টেমে "পাস" করা হবে না।

> **WARNING:** সবচেয়ে সাধারণ ও সবচেয়ে বিপজ্জনক ভুল: `terraform.tfstate` বা `*.tfvars` ফাইল ভুল করে Git-এ Commit করা। একবার Commit হয়ে গেলে Git-এর ইতিহাস থেকে মুছে ফেলা কঠিন — ফাইলটি মুছে Commit করলেও পুরনো Commit-এ তা থেকে যায়। প্রতিরোধ: প্রকল্পের শুরুতেই `.gitignore`-এ `*.tfstate*`, `*.tfvars` (নমুনা ফাইল ছাড়া) ও `.terraform/` যোগ করুন, আর Secret-ধারী ফাইল Commit করার আগে `git-secrets` বা `gitleaks`-এর মতো টুল দিয়ে পরীক্ষা করুন।

সারাংশ

Terraform ও Ansible পরিপূরক: Terraform Infrastructure তৈরি করে ও তার হিসাব রাখে; Ansible সেই Infrastructure-এর ভেতরে OS, সার্ভিস ও অ্যাপ কনফিগার করে ও সেই অবস্থা বজায় রাখে। সমন্বিত কর্মপ্রবাহ ছয় ধাপে চলে: Terraform Provision করে, Output সংযোগের তথ্য দেয়, সেই Output থেকে Ansible Inventory তৈরি হয়, Ansible সার্ভার কনফিগার করে, অ্যাপ Deploy হয়, আর Validation প্রস্তুতি নিশ্চিত করে। দুটি সিস্টেমের মাঝখানে শুধু Secret-নয় এমন তথ্য (IP, Hostname) আদান-প্রদান হয়; Secret প্রতিটি সিস্টেমের নিজের নিরাপদ ভান্ডারে থাকে — Terraform State কখনো Git-এ নয়, Inventory-তে কখনো Secret নয়, আর Ansible-এর Secret শুধু Vault-এ। এই সীমানা মেনে চললে দুটি টুল একসঙ্গে একটি সম্পূর্ণ, নিরাপদ ও পুনরুৎপাদনযোগ্য পরিবেশ গড়ে তোলে — Terraform ছাড়া Ansible-এর কিছু কনফিগার করার থাকে না, Ansible ছাড়া Terraform-এর তৈরি সার্ভার ব্যবহারের উপযোগী হয় না।

জ্ঞান যাচাই

১. Terraform ও Ansible-এর দায়িত্বের সীমানা কোথায়? "Terraform কী আছে ঠিক করে, Ansible কেমন আছে ঠিক করে" — এই বাক্যটি একটি উদাহরণ দিয়ে ব্যাখ্যা করুন।
২. Terraform-এর Output থেকে Ansible Inventory তৈরির ধাপটি কেন গুরুত্বপূর্ণ? VM-এর IP বদলালে কী সুবিধা হয়?
৩. Terraform State ফাইলে Secret থাকার ঝুঁকি কী? State ফাইল নিরাপদে রাখার দুটি উপায় লিখুন।
৪. Inventory ফাইলে Secret লেখা কেন বিপজ্জনক? SSH সংযোগের জন্য নিরাপদ বিকল্প কী?
৫. দুটি সিস্টেমেরই যদি একই ডেটাবেস পাসওয়ার্ড দরকার হয়, তাহলে Secret-টি কোথায় রাখবেন ও কেন এক সিস্টেম থেকে আরেক সিস্টেমে "পাস" করবেন না?